

GLOBAL
EDITION



Absolute JAVA

SIXTH EDITION

Walter Savitch

ALWAYS LEARNING

PEARSON

ABSOLUTE

JAVA[™]

6th Edition
Global Edition

This page intentionally left blank

ABSOLUTE JAVA™

6th Edition
Global Edition

Walter Savitch

University of California, San Diego

Contributor

Kenrick Mock

University of Alaska Anchorage

PEARSON

Boston Columbus Indianapolis New York San Francisco Hoboken
Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montréal Toronto
Delhi Mexico City São Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo

Vice President and Editorial Director, ECS: Marcia J. Horton
Acquisitions Editor: Matt Goldstein
Assistant Acquisitions Editor, Global Edition: Aditee Agarwal
Editorial Assistant: Kelsey Loanes
Product Marketing Manager: Bram Van Kempen
Marketing Assistant: Jon Bryant
Senior Managing Editor: Scott Disanno
Production Project Manager: Rose Kernan
Project Editor, Global Edition: Radhika Raheja
Program Manager: Carole Snyder
Global HE Director of Vendor Sourcing and Procurement: Diane Hynes

Director of Operations: Nick Sklitsis
Operations Specialist: Maura Zaldivar-Garcia
Cover Designer: Lumina Datamatics
Manager, Rights and Permissions: Rachel Youdelman
Associate Project Manager, Rights and Permissions: Timothy Nicholls
Senior Manufacturing Controller, Production, Global Edition: Trudy Kimber
Media Production Manager, Global Edition: Vikram Kumar
Full-Service Project Management: Niraj Bhatt, iEnergizer Aptara®, Ltd.
Composition: iEnergizer Aptara®, Ltd.
Cover Image: © LeicherOliver/Shutterstock

Pearson Education Limited
Edinburgh Gate
Harlow
Essex CM20 2JE
England

and Associated Companies throughout the world

Visit us on the World Wide Web at:
www.pearsonglobaleditions.com

© Pearson Education Limited 2016

The right of Walter Savitch and Kenrick Mock to be identified as the author of this work has been asserted by him in accordance with the Copyright, Designs and Patents Act 1988.

Authorized adaptation from the United States edition, entitled Absolute JAVA, 6th Edition, 9781292109220 9780134041674 by Walter Savitch and Kenrick Mock published by Pearson Education © 2016.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without either the prior written permission of the publisher or a license permitting restricted copying in the United Kingdom issued by the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS.

All trademarks used herein are the property of their respective owners. The use of any trademark in this text does not vest in the author or publisher any trademark ownership rights in such trademarks, nor does the use of such trademarks imply any affiliation with or endorsement of this book by such owners.

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library

10 9 8 7 6 5 4 3 2 1

ISBN 10: 129210922X

ISBN 13: 9781292109220

Typeset in Adobe Garamond 10.5/12 by iEnergizer Aptara®, Ltd.

Printed and bound by Courier Westford in Malaysia

Preface

This book is designed to serve as a textbook and reference for programming in the Java language. Although it does include programming techniques, it is organized around the features of the Java language rather than any particular curriculum of programming techniques. The main audience I had in mind when writing this book was undergraduate students who have not had extensive programming experience with the Java language. As such, it would be a suitable Java text or reference for either a first programming course or a later computer science course that uses Java. This book is designed to accommodate a wide range of users. The introductory chapters are written at a level that is accessible to beginners, while the boxed sections of those chapters serve to quickly introduce more experienced programmers to basic Java syntax. Later chapters are still designed to be accessible, but are written at a level suitable for students who have progressed to these more advanced topics.

CHANGES IN THIS EDITION

This sixth edition presents the same programming philosophy as the fifth edition. For instructors, you can teach the same course, presenting the same topics in the same order with no changes in the material covered or the chapters assigned. The changes to this edition consist almost exclusively of supplementary material added to the chapters of the previous edition, namely:

- An introduction to functional programming with Java 8's lambda expressions.
- Additional content and examples on looping, networking, and exception handling.
- Introduction to building GUIs using JavaFX.
- Fifteen new programming projects.
- Five new video notes for a total of 51 video notes. These videos cover specific topics and offer solutions to selected programming projects. The videos walk students through the process of problem solving and coding to reinforce key programming concepts. An icon appears in the margin of the book when a video is available about the corresponding topic in the text.

NO NONSTANDARD SOFTWARE

Only classes in the standard Java libraries are used. No nonstandard software is used anywhere in the book.

JAVA COVERAGE

All programs have been tested with Java 8. Oracle is not proposing any changes to future versions of Java that would affect the approach in this book.

OBJECT-ORIENTED PROGRAMMING

This book gives extensive coverage of encapsulation, inheritance, and polymorphism as realized in the Java language. The chapters on Swing GUIs provide coverage of and extensive practice with event driven programming.

FLEXIBILITY IN TOPIC ORDERING

This book allows instructors wide latitude in reordering the material. This is important if a book is to serve as a reference. It is also in keeping with my philosophy of writing books that accommodate themselves to an instructor's style rather than tying the instructor to an author's personal preference of topic ordering. With this in mind, each chapter has a prerequisite section at the beginning; this section explains what material must be covered before doing each section of the chapter. Starred sections, which are explained next, further add to flexibility.

STARRED SECTIONS

Each chapter has a number of starred (★) sections, which can be considered optional. These sections contain material that beginners might find difficult and that can be omitted or delayed without hurting the continuity of the text. It is hoped that eventually the reader would return and cover this material. For more advanced students, the starred sections should not be viewed as optional.

ACCESSIBLE TO STUDENTS

It is not enough for a book to present the right topics in the right order. It is not even enough for it to be clear and correct when read by an instructor or other expert. The material needs to be presented in a way that is accessible to the person who does not yet know the content. Like my other textbooks that have proven to be very popular, this book was written to be friendly and accessible to the student.

SUMMARY BOXES

Each major point is summarized in a short boxed section. These boxed sections are spread throughout each chapter. They serve as summaries of the material, as a quick reference source, and as a way to quickly learn the Java syntax for features the reader knows about in general but for which he or she needs to know the Java particulars.

SELF-TEST EXERCISES

Each chapter contains numerous Self-Test Exercises at strategic points in the chapter. Complete answers for all the Self-Test Exercises are given at the end of each chapter.



VIDEO NOTES

VideoNotes are step-by-step videos that guide readers through the solution to an end-of-chapter problem or further illuminate a concept presented in the text. Icons in the text indicate where a VideoNote enhances a topic. Fully navigable problems allow for self-paced instruction. VideoNotes are located at www.pearsonglobaleditions.com/savitch.

OTHER FEATURES

Pitfall sections, programming tip sections, and examples of complete programs with sample I/O are given throughout each chapter. Each chapter ends with a summary section and a collection of programming projects suitable to assign to students.

HOW TO ACCESS INSTRUCTOR AND STUDENT RESOURCE MATERIALS

Online Practice and Assessment with MyProgrammingLab™. MyProgrammingLab helps students fully grasp the logic, semantics, and syntax of programming. Through practice exercises and immediate, personalized feedback, MyProgrammingLab improves the programming competence of beginning students who often struggle with the basic concepts and paradigms of popular high-level programming languages.

A self-study and homework tool, a MyProgrammingLab course consists of hundreds of small practice problems organized around the structure of this textbook. For students, the system automatically detects errors in the logic and syntax of their code submissions and offers targeted hints that enable students to figure out what went wrong—and why. For instructors, a comprehensive gradebook tracks correct and incorrect answers and stores the code inputted by students for review.

For a full demonstration, to see feedback from instructors and students, or to get started using MyProgrammingLab in your course, visit www.myprogramminglab.com.

SUPPORT MATERIAL

The following support materials are available to all users of this Global Editions book at www.pearsonglobaleditions.com/savitch:

- Source code from the book

The following resources are available to qualified instructors only at www.pearsonglobaleditions.com/savitch. Please contact your local sales representative for access information:

- Instructor's Manual with Solutions
- PowerPoint® slides

ACKNOWLEDGMENTS

Numerous individuals have contributed invaluable help and support in making this book happen: My former editor, Susan Hartman at Addison-Wesley, first conceived of the idea for this book and worked with me on the first editions; My current editor, Matt

Goldstein, provided support and inspiration for getting subsequent editions reviewed, revised, and out the door; Kelsey Loanes, Rose Kernan, Demetrius Hall, and the other fine people at Pearson also provided valuable assistance and encouragement.

The following reviewers provided corrections and suggestions for this book. Their contributions were a great help. I thank them all. In alphabetical order they are:

Jim Adams	Chandler-Gilbert Community College
Gerald W. Adkins	Georgia College & State University
Dr. Bay Arinze	Drexel University
Tamara Babaian	Bentley University
James Baldo	George Mason University
Prof. Richard G. Baldwin	Austin Community College
Kevin Bierre	Rochester Institute of Technology
Jon Bjornstad	Gavilan College
Janet Brown-Sederberg	Massasoit Community College
Tom Brown	Texas A&M University, Commerce
Charlotte Busch	Texas A&M University, Corpus Christi
Stephen Chandler	NW Shoals Community College
Hong Cheng	Southern Arkansas University
KY Daisy Fan	Cornell University
Adrienne Decker	University of Buffalo
Brian Downs	Century College
Jeffrey Edgington	University of Denver
Keith Frikken	Miami University
Ahmad Ghafarian	North Georgia College & State University
Arthur Geis	College of DuPage
Massoud Ghyam	University of Southern California
Susan G. Glenn	Gordon College
Nigel Gwee	Louisiana State University
Judy Hankins	Middle Tennessee State University
May Hou	Norfolk State University
Sterling Hough	NHTI
Chris Howard	DeVry University
Eliot Jacobson	University of California, Santa Barbara
Balaji Janamanchi	Texas Tech University
Suresh Kalathur	Boston University
Edwin Kay	Lehigh University
Dr. Clifford R. Kettembrough	IT Consultant and Professor

Frank Levey	Manatee Community College
Xia Lin	Drexel University
Mark M. Meysenburg	Doane College
Sridhar P. Nerur	The University of Texas at Arlington
Hoang M. Nguyen	Deanza College
Rick Ord	University of California, San Diego
Prof. Bryson R. Payne	North Georgia College & State University
David Primeaux	Virginia Commonwealth University
Neil Rhodes	University of California, San Diego
W. Brent Seales	University of Kentucky
Lili Shashaani	Duquesne University
Riyaz Sikora	The University of Texas at Arlington
Christopher Simpkins	Georgia Tech
Jeff Six	University of Delaware
Donald J Smith	Community College of Allegheny County
Tom Smith	Skidmore College
William Smith	Tulsa Community College
Xueqing (Clare) Tang	Governors State University
Ronald F. Taylor	Wright State University
Thomas VanDrunen	Wheaton College
Shon Vick	University of Maryland, Baltimore County
Natalie S. Wear	University of South Florida
Dale Welch	University of West Florida
David A. Wheeler	
Wook-Sung Yoo	Gannon University

Special thanks goes to Kenrick Mock (University of Alaska Anchorage) who executed the updating of this edition. He once again had the difficult job of satisfying me, the editor, and himself. I thank him for a truly excellent job.

Walter Savitch

Pearson wishes to thank and acknowledge the following people for their work on the Global Edition:

Contributors

Vikas Deep Dhiman	Amity University
Madhurima Hooda	Amity University

Reviewers

Manasa Rengarar	NMAM Institute of Technology
S.H. Chung	Wawasan Open University
Issam A. El-Moughrabi	Gulf University of Science and Technology

LOCATION OF VIDEONOTES IN THE TEXT

www.pearsonglobaleditions.com/savitch



VideoNote

Chapter 1	Compiling a Java Program, page 42 Solution to Programming Project 1.7, page 88
Chapter 2	Using <code>printf</code> , page 94 Pitfalls Involving <code>nextLine()</code> , page 115 Solution to Programming Project 2.11, page 129 Solution to Programming Project 2.12, page 130
Chapter 3	Nested Loop Example, page 177 Debugging Walkthrough, page 184 Generating Random Numbers, page 191 Solution to Programming Project 3.9, page 202 Solution to Programming Project 3.13, page 203
Chapter 4	Information Hiding Example, page 239 Example Using the <code>StringTokenizer</code> Class on a CSV File, page 279 Solution to Programming Project 4.9, page 287
Chapter 5	Deep Copy vs. Shallow Copy Example, page 353 Solution to Programming Project 5.9, page 376
Chapter 6	Arrays of Objects, page 390 Solution to Programming Project 6.8, page 454 Solution to Programming Project 6.15, page 456
Chapter 7	Inheritance Walkthrough, page 464 Solution to Programming Project 7.3, page 509 Solution to Programming Project 7.5, page 511
Chapter 8	Late Binding Example, page 518 Solution to Programming Project 8.1, page 550 Solution to Programming Project 8.9, page 553
Chapter 9	Solution to Programming Project 9.1, page 609 Solution to Programming Project 9.7, page 611
Chapter 10	Reading a Text File, page 625 Solution to Programming Project 10.1, page 679 Solution to Programming Project 10.9, page 681
Chapter 11	Recursion and the Stack, page 696 Solution to Programming Project 11.3, page 720
Chapter 12	Solution to Programming Project 12.9, page 746
Chapter 13	Solution to Programming Project 13.1, page 790 Solution to Programming Project 13.11, page 793
Chapter 14	Solution to Programming Project 14.7, page 836
Chapter 15	Walkthrough of the Hash Table Class, page 906 Solution to Programming Project 15.1, page 931

Chapter 16	Using <code>HashMap</code> with a Custom Class, page 948 Solution to Programming Project 16.3, page 975 Solution to Programming Project 16.5, page 976
Chapter 17	GUI Layout Using an IDE, page 1009 Solution to Programming Project 17.1, page 1053
Chapter 18	Walkthrough of a Simple Drawing Program, page 1082 Solution to Programming Project 18.7, page 1117
Chapter 19	Walkthrough of a Program with Race Conditions, page 1134 Networking with Streams, page 1138 Functional Programming Example, page 1172 Solution to Programming Project 19.3, page 1196 Solution to Programming Project 19.11, page 1197
Chapter 20	No video notes (Chapter on website)

This page intentionally left blank

Brief Contents

Chapter 1	GETTING STARTED	33
Chapter 2	CONSOLE INPUT AND OUTPUT	89
Chapter 3	FLOW OF CONTROL	131
Chapter 4	DEFINING CLASSES I	205
Chapter 5	DEFINING CLASSES II	291
Chapter 6	ARRAYS	377
Chapter 7	INHERITANCE	459
Chapter 8	POLYMORPHISM AND ABSTRACT CLASSES	515
Chapter 9	EXCEPTION HANDLING	555
Chapter 10	FILE I/O	613
Chapter 11	RECURSION	683
Chapter 12	UML AND PATTERNS	725
Chapter 13	INTERFACES AND INNER CLASSES	747
Chapter 14	GENERICS AND THE <code>ArrayList</code> CLASS	795
Chapter 15	LINKED DATA STRUCTURES	839
Chapter 16	COLLECTIONS, MAPS AND ITERATORS	935
Chapter 17	SWING I	981
Chapter 18	SWING II	1057
Chapter 19	JAVA NEVER ENDS	1119
Chapter 20	APPLETS AND HTML (online at www.pearsonglobaleditions.com/savitch)	
Appendix 1	KEYWORDS	1199
Appendix 2	PRECEDENCE AND ASSOCIATIVITY RULES	1201
Appendix 3	ASCII CHARACTER SET	1203
Appendix 4	FORMAT SPECIFICATIONS FOR <code>printf</code>	1205
Appendix 5	SUMMARY OF CLASSES AND INTERFACES	1207
	INDEX	1275

This page intentionally left blank

Contents

Chapter 1 Getting Started 33

1.1 INTRODUCTION TO JAVA 34

- Origins of the Java Language ★ 34
- Objects and Methods 35
- Applets ★ 36
- A Sample Java Application Program 37
- Byte-Code and the Java Virtual Machine 40
- Class Loader ★ 42
- Compiling a Java Program or Class 42
- Running a Java Program 43
- TIP: Error Messages 44

1.2 EXPRESSIONS AND ASSIGNMENT STATEMENTS 45

- Identifiers 45
- Variables 47
- Assignment Statements 48
- TIP: Initialize Variables 50
- More Assignment Statements ★ 51
- Assignment Compatibility 52
- Constants 53
- Arithmetic Operators and Expressions 55
- Parentheses and Precedence Rules ★ 56
- Integer and Floating-Point Division 58
- PITFALL: Round-Off Errors in Floating-Point Numbers 59
- PITFALL: Division with Whole Numbers 60
- Type Casting 61
- Increment and Decrement Operators 62

1.3 THE CLASS `String` 65

- String Constants and Variables 65
- Concatenation of Strings 66
- Classes 67
- String Methods 69
- Escape Sequences 74
- String Processing 75
- The Unicode Character Set ★ 75

1.4 PROGRAM STYLE 78

Naming Constants 78

Java Spelling Conventions 80

Comments 81

Indenting 82

Chapter Summary 83

Answers to Self-Test Exercises 84

Programming Projects 86

Chapter 2 Console Input and Output 89**2.1 SCREEN OUTPUT 90**`System.out.println` 90

TIP: Different Approaches to Formatting Output 93

Formatting Output with `printf` 93TIP: Formatting Monetary Amounts with `printf` 97

TIP: Legacy Code 98

Money Formats Using `NumberFormat` ★ 99

Importing Packages and Classes 102

The `DecimalFormat` Class ★ 104**2.2 CONSOLE INPUT USING THE `Scanner` CLASS 108**The `Scanner` Class 108PITFALL: Dealing with the Line Terminator, `'\n'` 115

The Empty String 116

TIP: Prompt for Input 116

TIP: Echo Input 116

EXAMPLE: Self-Service Checkout 118

Other Input Delimiters 119

2.3 INTRODUCTION TO FILE INPUT 121The `Scanner` Class for Text File Input 121

Chapter Summary 124

Answers to Self-Test Exercises 124

Programming Projects 127

Chapter 3 Flow of Control 131**3.1 BRANCHING MECHANISM 132**`if-else` Statements 132Omitting the `else` 133

Compound Statements 134

TIP: Placing of Braces 135

Nested Statements 136

Multiway `if-else` Statement 136
EXAMPLE: State Income Tax 137
The `switch` Statement 139
PITFALL: Forgetting a `break` in a `switch` Statement 143
The Conditional Operator ★ 144

3.2 BOOLEAN EXPRESSIONS 145

Simple Boolean Expressions 145
PITFALL: Using `=` in Place of `==` 146
PITFALL: Using `==` with Strings 147
Lexicographic and Alphabetic Order 148
Building Boolean Expressions 151
PITFALL: Strings of Inequalities 152
Evaluating Boolean Expressions 152
TIP: Naming Boolean Variables 155
Short-Circuit and Complete Evaluation 156
Precedence and Associativity Rules 157

3.3 LOOPS 164

`while` Statement and `do-while` Statement 164
Algorithms and Pseudocode 166
EXAMPLE: Averaging a List of Scores 169
The `for` Statement 170
The Comma in `for` Statements 173
TIP: Repeat *N* Times Loops 175
PITFALL: Extra Semicolon in a `for` Statement 175
PITFALL: Infinite Loops 176
Nested Loops 177
The `break` and `continue` Statements ★ 180
The `exit` Statement 181

3.4 DEBUGGING 182

Loop Bugs 182
Tracing Variables 182
General Debugging Techniques 183
EXAMPLE: Debugging an Input Validation Loop 184
Preventive Coding 188
Assertion Checks ★ 189

3.5 RANDOM NUMBER GENERATION ★ 191

The `Random` Object 191
The `Math.random()` Method 193
Chapter Summary 194
Answers to Self-Test Exercises 194
Programming Projects 200

Chapter 4 Defining Classes I 205**4.1 CLASS DEFINITIONS 206**

Instance Variables and Methods 209
More about Methods 212
TIP: Any Method Can Be Used as a `void` Method 216
Local Variables 218
Blocks 219
TIP: Declaring Variables in a `for` Statement 220
Parameters of a Primitive Type 220
PITFALL: Use of the Terms “Parameter” and “Argument” 227
Simple Cases with Class Parameters 229
The `this` Parameter 229
Methods That Return a Boolean Value 231
The Methods `equals` and `toString` 234
Recursive Methods 237
TIP: Testing Methods 237

4.2 INFORMATION HIDING AND ENCAPSULATION 239

`public` and `private` Modifiers 240
EXAMPLE: Yet Another Date Class 241
Accessor and Mutator Methods 242
TIP: A Class Has Access to Private Members of All Objects of the Class 247
TIP: Mutator Methods Can Return a Boolean Value ★ 248
Preconditions and Postconditions 249

4.3 OVERLOADING 250

Rules for Overloading 250
PITFALL: Overloading and Automatic Type Conversion 254
PITFALL: You Cannot Overload Based on the Type Returned 256

4.4 CONSTRUCTORS 258

Constructor Definitions 258
TIP: You Can Invoke Another Method in a Constructor 266
TIP: A Constructor Has a `this` Parameter 266
TIP: Include a No-Argument Constructor 267
EXAMPLE: The Final Date Class 268
Default Variable Initializations 269
An Alternative Way to Initialize Instance Variables 269
EXAMPLE: A Pet Record Class 270
The `StringTokenizer` Class ★ 274

Chapter Summary 279
Answers to Self-Test Exercises 280
Programming Projects 285

Chapter 5 **Defining Classes II** 291

5.1 **STATIC METHODS AND STATIC VARIABLES** 293

Static Methods 293

PITFALL: Invoking a Nonstatic Method Within a Static Method 295

TIP: You Can Put a `main` in Any Class 296

Static Variables 300

The `Math` Class 305

Wrapper Classes 309

Automatic Boxing and Unboxing 310

Static Methods in Wrapper Classes 312

PITFALL: A Wrapper Class Does Not Have a No-Argument Constructor 315

5.2 **REFERENCES AND CLASS PARAMETERS** 316

Variables and Memory 317

References 318

Class Parameters 323

PITFALL: Use of `=` and `==` with Variables of a Class Type 327

The Constant `null` 329

PITFALL: Null Pointer Exception 330

The `new` Operator and Anonymous Objects 330

EXAMPLE: Another Approach to Keyboard Input ★ 331

TIP: Use Static Imports ★ 333

5.3 **USING AND MISUSING REFERENCES** 335

EXAMPLE: A Person Class 336

PITFALL: `null` Can Be an Argument to a Method 341

Copy Constructors 345

PITFALL: Privacy Leaks 347

Mutable and Immutable Classes 351

TIP: Deep Copy versus Shallow Copy 353

TIP: Assume Your Coworkers Are Malicious 354

5.4 **PACKAGES AND `javadoc`** 354

Packages and `import` Statements 355

The Package `java.lang` 356

Package Names and Directories 356

PITFALL: Subdirectories Are Not Automatically Imported 359

The Default Package 359

PITFALL: Not Including the Current Directory in Your Class Path 360

Specifying a Class Path When You Compile ★ 360

Name Clashes ★ 361

Introduction to `javadoc` ★ 362

Commenting Classes for `javadoc` ★ 362

Running `javadoc` ★ 364

Chapter Summary	366
Answers to Self-Test Exercises	367
Programming Projects	371

Chapter 6 Arrays 377

6.1 INTRODUCTION TO ARRAYS 378

Creating and Accessing Arrays	379
The <code>length</code> Instance Variable	382
TIP: Use <code>for</code> Loops with Arrays	384
PITFALL: Array Indices Always Start with Zero	384
PITFALL: Array Index Out of Bounds	384
Initializing Arrays	385
PITFALL: An Array of Characters Is Not a String	387

6.2 ARRAYS AND REFERENCES 388

Arrays Are Objects	388
PITFALL: Arrays with a Class Base Type	390
Array Parameters	390
PITFALL: Use of <code>=</code> and <code>==</code> with Arrays	392
Arguments for the Method <code>main</code> ★	397
Methods that Return an Array	399

6.3 PROGRAMMING WITH ARRAYS 400

Partially Filled Arrays	401
EXAMPLE: A Class for Partially Filled Arrays	404
TIP: Accessor Methods Need Not Simply Return Instance Variables	408
The “for-each” Loop ★	408
Methods with a Variable Number of Parameters ★	412
EXAMPLE: A String Processing Example ★	415
Privacy Leaks with Array Instance Variables	416
EXAMPLE: Sorting an Array	420
Enumerated Types ★	424
TIP: Enumerated Types in <code>switch</code> Statements ★	429

6.4 MULTIDIMENSIONAL ARRAYS 431

Multidimensional Array Basics	431
Using the <code>length</code> Instance Variable	434
Ragged Arrays ★	435
Multidimensional Array Parameters and Returned Values	435
EXAMPLE: A Grade Book Class	436
Chapter Summary	442
Answers to Self-Test Exercises	443
Programming Projects	450

Chapter 7 Inheritance 459

7.1 INHERITANCE BASICS 460

- Derived Classes 461
- Overriding a Method Definition 471
- Changing the Return Type of an Overridden Method 471
- Changing the Access Permission of an Overridden Method 472
- PITFALL: Overriding versus Overloading 473
- The `super` Constructor 474
- The `this` Constructor 476
- TIP: An Object of a Derived Class Has More than One Type 477
- PITFALL: The Terms *Subclass* and *Superclass* 480
- EXAMPLE: An Enhanced `StringTokenizer` Class ★ 481

7.2 ENCAPSULATION AND INHERITANCE 484

- PITFALL: Use of Private Instance Variables from the Base Class 485
- PITFALL: Private Methods Are Effectively Not Inherited 486
- Protected and Package Access 487
- PITFALL: Forgetting about the Default Package 490
- PITFALL: A Restriction on Protected Access ★ 490

7.3 PROGRAMMING WITH INHERITANCE 493

- TIP: Static Variables Are Inherited 493
- TIP: “is a” versus “has a” 493
- Access to a Redefined Base Method 493
- PITFALL: You Cannot Use Multiple `supers` 495
- The Class Object 496
- The Right Way to Define `equals` 497
- TIP: `getClass` versus `instanceof` ★ 499
- Chapter Summary 504
- Answers to Self-Test Exercises 505
- Programming Projects 508

Chapter 8 Polymorphism and Abstract Classes 515

8.1 POLYMORPHISM 516

- Late Binding 517
- The `final` Modifier 519
- EXAMPLE: Sales Records 520
- Late Binding with `toString` 527
- PITFALL: No Late Binding for Static Methods 528
- Downcasting and Upcasting 529
- PITFALL: Downcasting 533

TIP: Checking to See Whether Downcasting Is Legitimate ★ 533
A First Look at the `clone` Method 536
PITFALL: Sometimes the `clone` Method Return Type Is `Object` 537
PITFALL: Limitations of Copy Constructors ★ 538

8.2 ABSTRACT CLASSES 541

Abstract Classes 542
PITFALL: You Cannot Create Instances of an Abstract Class 546
TIP: An Abstract Class Is a Type 547

Chapter Summary 548
Answers to Self-Test Exercises 548
Programming Projects 550

Chapter 9 Exception Handling 555

9.1 EXCEPTION HANDLING BASICS 557

`try-catch` Mechanism 557
Exception Handling with the `Scanner` Class 559
TIP: Exception Controlled Loops 560
Throwing Exceptions 562
EXAMPLE: A Toy Example of Exception Handling 564
Exception Classes 569
Exception Classes from Standard Packages 570
Defining Exception Classes 572
TIP: Preserve `getMessage` 576
TIP: An Exception Class Can Carry a Message of Any Type 578
Multiple `catch` Blocks 583
PITFALL: Catch the More Specific Exception First 585

9.2 THROWING EXCEPTIONS IN METHODS 588

Throwing an Exception in a Method 588
Declaring Exceptions in a `throws` Clause 590
Exceptions to the Catch or Declare Rule 593
`throws` Clause in Derived Classes 594
When to Use Exceptions 595
Example: Retrieving a High Score 596
Event-Driven Programming ★ 599

9.3 MORE PROGRAMMING TECHNIQUES FOR EXCEPTION HANDLING 601

PITFALL: Nested `try-catch` Blocks 601
The `finally` Block ★ 601
Rethrowing an Exception ★ 603
The `AssertionError` Class ★ 603

`ArrayIndexOutOfBoundsException` 604

Chapter Summary 604

Answers to Self-Test Exercises 605

Programming Projects 609

Chapter 10 File I/O 613

10.1 INTRODUCTION TO FILE I/O 614

Streams 614

Text Files and Binary Files 615

10.2 TEXT FILES 616

Writing to a Text File 616

PITFALL: A try Block Is a Block 622

PITFALL: Overwriting an Output File 622

Appending to a Text File 623

TIP: `toString` Helps with Text File Output 624

Reading from a Text File 625

Reading a Text File Using `Scanner` 625

Testing for the End of a Text File with `Scanner` 628

Reading a Text File Using `BufferedReader` 635

TIP: Reading Numbers with `BufferedReader` 639

Testing for the End of a Text File with `BufferedReader` 639

Path Names 641

Nested Constructor Invocations 642

`System.in`, `System.out`, and `System.err` 643

10.3 THE `File` CLASS 645

Programming with the `File` Class 645

10.4 BINARY FILES ★ 649

Writing Simple Data to a Binary File 650

UTF and `writeUTF` 654

Reading Simple Data from a Binary File 655

Checking for the End of a Binary File 660

PITFALL: Checking for the End of a File in the Wrong Way 661

Binary I/O of Objects 662

The `Serializable` Interface 663

PITFALL: Mixing Class Types in the Same File 666

Array Objects in Binary Files 666

10.5 RANDOM ACCESS TO BINARY FILES ★ 668

Reading and Writing to the Same File 668

PITFALL: `RandomAccessFile` Need Not Start Empty 674

Chapter Summary	674
Answers to Self-Test Exercises	675
Programming Projects	679

Chapter 11 Recursion 683

11.1 RECURSIVE void METHODS 685

EXAMPLE: Vertical Numbers	685
Tracing a Recursive Call	688
A Closer Look at Recursion	691
PITFALL: Infinite Recursion	693
Stacks for Recursion ★	694
PITFALL: Stack Overflow ★	696
Recursion versus Iteration	696

11.2 RECURSIVE METHODS THAT RETURN A VALUE 697

General Form for a Recursive Method That Returns a Value	698
EXAMPLE: Another Powers Method	698

11.3 THINKING RECURSIVELY 703

Recursive Design Techniques	703
Binary Search ★	704
Efficiency of Binary Search ★	710
EXAMPLE: Finding a File	712
Chapter Summary	715
Answers to Self-Test Exercises	715
Programming Projects	720

Chapter 12 UML and Patterns 725

12.1 UML 726

History of UML	727
UML Class Diagrams	727
Class Interactions	728
Inheritance Diagrams	728
More UML	730

12.2 PATTERNS ★ 731

Adaptor Pattern ★	731
The Model-View-Controller Pattern ★	732
EXAMPLE: A Sorting Pattern	733
Restrictions on the Sorting Pattern	739
Efficiency of the Sorting Pattern ★	739

TIP: Pragmatics and Patterns	740
Pattern Formalism	740
Chapter Summary	741
Answers to Self-Test Exercises	741
Programming Projects	743

Chapter 13 Interfaces and Inner Classes 747

13.1 INTERFACES 749

Interfaces	749
Abstract Classes Implementing Interfaces	751
Derived Interfaces	751
PITFALL: Interface Semantics Are Not Enforced	753
The Comparable Interface	755
EXAMPLE: Using the Comparable Interface	756
Defined Constants in Interfaces	761
PITFALL: Inconsistent Interfaces	762
The Serializable Interface ★	765
The Cloneable Interface	765

13.2 SIMPLE USES OF INNER CLASSES 770

Helping Classes	770
TIP: Inner and Outer Classes Have Access to Each Other's Private Members	771
EXAMPLE: A Bank Account Class	771
The .class File for an Inner Class	775
PITFALL: Other Uses of Inner Classes	776

13.3 MORE ABOUT INNER CLASSES 776

Static Inner Classes	776
Public Inner Classes	777
TIP: Referring to a Method of the Outer Class	779
Nesting Inner Classes	781
Inner Classes and Inheritance	781
Anonymous Classes	782
TIP: Why Use Inner Classes?	784
Chapter Summary	785
Answers to Self-Test Exercises	785
Programming Projects	790

Chapter 14 Generics and the ArrayList Class 795

14.1 THE ArrayList CLASS 797

Using the ArrayList Class	798
TIP: Summary of Adding to an ArrayList	802

Methods in the Class `ArrayList` 803
 The “for-each” Loop 806
 EXAMPLE: Golf Scores 809
 TIP: Use `trimToSize` to Save Memory 812
 PITFALL: The `clone` Method Makes a Shallow Copy ★ 812
 The `Vector` Class 813
 Parameterized Classes and Generics 814
 PITFALL: Nonparameterized `ArrayList` and `Vector` Classes 814

14.2 GENERICS 814

Generic Basics 815
 TIP: Compile with the `-Xlint` Option 817
 EXAMPLE: A Generic Class for Ordered Pairs 817
 PITFALL: A Generic Constructor Name Has No Type Parameter 820
 PITFALL: You Cannot Plug in a Primitive Type for a Type Parameter 821
 PITFALL: A Type Parameter Cannot Be Used Everywhere a Type Name
 Can Be Used 821
 PITFALL: An Instantiation of a Generic Class Cannot be an
 Array Base Type 822
 TIP: A Class Definition Can Have More Than One Type Parameter 823
 PITFALL: A Generic Class Cannot Be an Exception Class 824
 Bounds for Type Parameters 825
 TIP: Generic Interfaces 828
 Generic Methods ★ 828
 Inheritance with Generic Classes ★ 830
 Chapter Summary 832
 Answers to Self-Test Exercises 832
 Programming Projects 835

Chapter 15 Linked Data Structures 839

15.1 JAVA LINKED LISTS 842

EXAMPLE: A Simple Linked List Class 842
 Working with Linked Lists 846
 PITFALL: Privacy Leaks 851
 Node Inner Classes 852
 EXAMPLE: A Generic Linked List 855
 PITFALL: Using `Node` Instead of `Node<T>` 860
 The `equals` Method for Linked Lists 860

15.2 COPY CONSTRUCTORS AND THE `clone` METHOD ★ 862

Simple Copy Constructors and `clone` Methods ★ 862
 Exceptions ★ 863

PITFALL: The `clone` Method Is Protected in `object` ★ 865
TIP: Use a Type Parameter Bound for a Better `clone` ★ 866
EXAMPLE: A Linked List with a Deep Copy `clone` Method ★ 870
TIP: Cloning Is an “All or Nothing” Affair 873

15.3 ITERATORS 873

Defining an Iterator Class 874
Adding and Deleting Nodes 879

15.4 VARIATIONS ON A LINKED LIST 884

Doubly Linked List 884
The Stack Data Structure 893
The Queue Data Structure 895
Running Times and Big-*O* Notation 898
Efficiency of Linked Lists 903

15.5 HASH TABLES WITH CHAINING 904

A Hash Function for Strings 905
Efficiency of Hash Tables 908

15.6 SETS 909

Fundamental Set Operations 910
Efficiency of Sets Using Linked Lists 915

15.7 TREES 916

Tree Properties 916
EXAMPLE: A Binary Search Tree Class ★ 919
Efficiency of Binary Search Trees ★ 924

Chapter Summary 925
Answers to Self-Test Exercises 926
Programming Projects 931

Chapter 16 Collections, Maps and Iterators 935

16.1 COLLECTIONS 936

Wildcards 938
The Collection Framework 938
PITFALL: Optional Operations 944
TIP: Dealing with All Those Exceptions 945
Concrete Collection Classes 946
Differences between `ArrayList<T>` and `Vector<T>` 956
Nonparameterized Version of the Collection Framework ★ 956
PITFALL: Omitting the `<T>` 957

16.2 MAPS 957

Concrete Map Classes 960

16.3 ITERATORS 964

The Iterator Concept 964

The `Iterator<T>` Interface 964

TIP: For-Each Loops as Iterators 967

List Iterators 968

PITFALL: `next` Can Return a Reference 970

TIP: Defining Your Own Iterator Classes 972

Chapter Summary 973

Answers to Self-Test Exercises 973

Programming Projects 974

Chapter 17 Swing I 981**17.1 EVENT-DRIVEN PROGRAMMING 983**

Events and Listeners 983

17.2 BUTTONS, EVENTS, AND OTHER SWING BASICS 984

EXAMPLE: A Simple Window 985

PITFALL: Forgetting to Program the Close-Window Button 990

Buttons 991

Action Listeners and Action Events 992

PITFALL: Changing the Heading for `actionPerformed` 994

TIP: Ending a Swing Program 994

EXAMPLE: A Better Version of Our First Swing GUI 995

Labels 998

Color 999

EXAMPLE: A GUI with a Label and Color 1000

17.3 CONTAINERS AND LAYOUT MANAGERS 1002

Border Layout Managers 1003

Flow Layout Managers 1006

Grid Layout Managers 1007

Panels 1011

EXAMPLE: A Tricolor Built with Panels 1012

The `Container` Class 1016

TIP: Code a GUI's Look and Actions Separately 1019

The Model-View-Controller Pattern ★ 1020

17.4 MENUS AND BUTTONS 1021

EXAMPLE: A GUI with a Menu 1021
Menus, Menu Items, and Menu Bars 1021
Nested Menus ★ 1026
The `AbstractButton` Class 1026
The `setActionCommand` Method 1029
Listeners as Inner Classes ★ 1030

17.5 TEXT FIELDS AND TEXT AREAS 1033

Text Areas and Text Fields 1034
TIP: Labeling a Text Field 1040
TIP: Inputting and Outputting Numbers 1040
A Swing Calculator 1041

Chapter Summary 1046
Answers to Self-Test Exercises 1047
Programming Projects 1053

Chapter 18 Swing II 1057**18.1 WINDOW LISTENERS 1058**

EXAMPLE: A Window Listener Inner Class 1060
The `dispose` Method 1063
PITFALL: Forgetting to Invoke `setDefaultCloseOperation` 1064
The `WindowAdapter` Class 1064

18.2 ICONS AND SCROLL BARS 1066

Icons 1066
Scroll Bars 1072
EXAMPLE: Components with Changing Visibility 1077

18.3 THE `Graphics` CLASS 1081

Coordinate System for Graphics Objects 1081
The Method `paint` and the Class `Graphics` 1082
Drawing Ovals 1087
Drawing Arcs 1087
Rounded Rectangles ★ 1091
`paintComponent` for Panels 1092
Action Drawings and `repaint` 1092
Some More Details on Updating a GUI ★ 1098

18.4 COLORS 1098

Specifying a Drawing Color 1099

Defining Colors 1100

PITFALL: Using doubles to Define a Color 1101

The JColorChooser Dialog Window 1102

18.5 FONTS AND THE drawString METHOD 1105

The drawString Method 1105

Fonts 1108

Chapter Summary 1111

Answers to Self-Test Exercises 1111

Programming Projects 1115

Chapter 19 Java Never Ends 1119**19.1 MULTITHREADING 1120**

EXAMPLE: A Nonresponsive GUI 1121

Thread.sleep 1121

The getGraphics Method 1125

Fixing a Nonresponsive Program Using Threads 1126

EXAMPLE: A Multithreaded Program 1126

The Class Thread 1127

The Runnable Interface ★ 1130

Race Conditions and Thread Synchronization ★ 1133

19.2 NETWORKING WITH STREAM SOCKETS 1138

Sockets 1138

Sockets and Threading 1142

The URL Class 1143

19.3 JAVABEANS 1143

The Component Model 1144

The JavaBeans Model 1144

19.4 JAVA AND DATABASE CONNECTIONS 1145

Relational Databases 1145

Java DB and JDBC 1146

SQL 1147

19.5 WEB PROGRAMMING WITH JAVA SERVER PAGES 1158

Applets, Servlets, and Java Server Pages 1158

Oracle GlassFish Enterprise Server 1160

HTML Forms—the Common Gateway Interface 1161
JSP Declarations, Expressions, Scriptlets, and Directives 1163

19.6 INTRODUCTION TO FUNCTIONAL PROGRAMMING IN JAVA 8 1172

19.7 INTRODUCTION TO JAVAFX 1180

Chapter Summary 1193
Answers to Self-Test Exercises 1194
Programming Projects 1196

Chapter 20 Applets and HTML www.pearsonglobaleditions.com/savitch)

Appendix 1 Keywords 1199

Appendix 2 Precedence and Associativity Rules 1201

Appendix 3 ASCII Character Set 1203

Appendix 4 Format Specifications for `printf` 1205

Appendix 5 Summary of Classes and Interfaces 1207

Index 1275

This page intentionally left blank



Getting Started **1**

1.1 INTRODUCTION TO JAVA 34

- Origins of the Java Language ★ 34
- Objects and Methods 35
- Applets ★ 36
- A Sample Java Application Program 37
- Byte-Code and the Java Virtual Machine 40
- Class Loader ★ 42
- Compiling a Java Program or Class 42
- Running a Java Program 43

1.2 EXPRESSIONS AND ASSIGNMENT STATEMENTS 45

- Identifiers 45
- Variables 47
- Assignment Statements 48
- More Assignment Statements ★ 51
- Assignment Compatibility 52
- Constants 53
- Arithmetic Operators and Expressions 55
- Parentheses and Precedence Rules ★ 56
- Integer and Floating-Point Division 58
- Type Casting 61
- Increment and Decrement Operators 62

1.3 THE CLASS `String` 65

- String Constants and Variables 65
- Concatenation of Strings 66
- Classes 67
- String Methods 69
- Escape Sequences 74
- String Processing 75
- The Unicode Character Set ★ 75

1.4 PROGRAM STYLE 78

- Naming Constants 78
- Java Spelling Conventions 80
- Comments 81
- Indenting 82

*She starts—she moves—she seems to feel
The thrill of life along her keel.*

HENRY WADSWORTH LONGFELLOW, *The Building of the Ship*, 1850.

Introduction

This chapter introduces you to the Java language and gives you enough details to allow you to write simple programs involving expressions, assignments, and console output. The details about assignments and expressions are similar to that of most other high-level languages. Every language has its own way of handling strings and console output, so even the experienced programmer should look at that material. Even if you are already an experienced programmer in some language other than Java, you should read at least the subsection entitled “A Sample Java Application Program” in Section 1.1 and preferably all of Section 1.2. You should also read all of Section 1.3 (on strings) and at least skim Section 1.4 to find out about Java defined constants and comments.

Prerequisites

This book is self-contained and requires no preparation other than some simple high school algebra.

1.1 Introduction to Java

Eliminating the middle man is not necessarily a good idea.

Found in my old economics class notes.

In this section, we give you an overview of the Java programming language.

Origins of the Java Language ★

Java is well-known as a programming language for Internet applications. However, this book, and many other books and programmers, considers Java a general-purpose programming language that is suitable for most any application, whether it involves the Internet or not. The first version of Java was neither of these things, but it evolved into both.

In 1991, James Gosling led a team at Sun Microsystems that developed the first version of Java (which was not yet called *Java*). This first version of the language was

designed for programming home appliances, such as washing machines and television sets. Although that may not be a very glamorous application area, it is no easy task to design such a language. Home appliances are controlled by a wide variety of computer processors (chips). The language that Gosling was designing needed to work on all these different processors. Moreover, a home appliance is typically an inexpensive item, so the manufacturer would be unwilling to invest large amounts of money into developing complicated compilers. (A *compiler* translates a program into a language the processor can understand.) To simplify the tasks of writing compilers (translation programs) for each class of appliances, the team used a two-step translation process. The programs are first translated into an **intermediate language** that is the same for all appliances (or all computers), and then a small, easy-to-write—and hence, inexpensive—program translates this intermediate language into the machine language for a particular appliance or computer. This intermediate language is called **Java byte-code**, or simply, **byte-code**. Since there is only one intermediate language, the hardest step of the two-step translation from program to intermediate language to machine language is the same for all appliances (or all computers); hence, most of the cost of translating to multiple machine languages was saved. The language for programming appliances never caught on with appliance manufacturers, but the Java language into which it evolved has become a widely used programming language. Today, Java is owned by Oracle Corporation, which purchased Sun Microsystems in 2010.

Why call it *byte-code*? The word **code** is commonly used to mean a program or part of a program. A byte is a small unit of storage (eight bits to be precise). Computer-readable information is typically organized into bytes. So the term *byte-code* suggests a program that is readable by a computer as opposed to a person.

In 1994, Patrick Naughton and Jonathan Payne at Sun Microsystems developed a Web browser that could run (Java) programs over the Internet, which has evolved into the browser known as HotJava. This was the start of Java's connection to the Internet. In the fall of 1995, Netscape Incorporated made its Web browser capable of running Java programs. Other companies followed suit and have developed software that accommodates Java programs.

Objects and Methods

Java is an **object-oriented programming (OOP)** language. What is OOP? The world around us is made up of objects, such as people, automobiles, buildings, streets, adding machines, papers, and so forth. Each of these objects has the ability to perform certain actions, and each of these actions has some effect on some of the other objects in the world. OOP is a programming methodology that views a program as similarly consisting of objects that interact with each other by means of actions.

Object-oriented programming has its own specialized terminology. The objects are called, appropriately enough, **objects**. The actions that an object can take are called **methods**. Objects of the same kind are said to have the same *type* or, more often, are said to be in the same **class**. For example, in an airport simulation program, all the

intermediate
language

byte-code

code

OOP

object
method
class

Why Is the Language Named “Java”?

The current custom is to name programming languages according to the whims of their designers. Java is no exception. There are conflicting explanations of the origin of the name “Java.” Despite these differing stories, one thing is clear: The word “Java” does not refer to any property or serious history of the Java language. One believable story about where the name came from is that it was thought of when, after a fruitless meeting trying to come up with a new name for the language, the development team went out for coffee. Hence, the inspiration for the name “Java.”

simulated airplanes might belong to the same class, probably called the `Airplane` class. All objects within a class have the same methods. Thus, in a simulation program, all airplanes have the same methods (or possible actions), such as taking off, flying to a specific location, landing, and so forth. However, all simulated airplanes are not identical. They can have different characteristics, which are indicated in the program by associating different data (that is, some different information) with each particular airplane object. For example, the data associated with an airplane object might be two numbers for its speed and altitude.

If you have used some other programming language, it might help to explain Java terminology in terms of the vocabulary used in other languages. Things that are called *procedures*, *methods*, *functions*, or *subprograms* in other languages are all called *methods* in Java. In Java, all methods (and for that matter, any programming constructs whatsoever) are part of a *class*. As we will see, a Java **application program** is a class with a method named `main`; when you run the Java program, the run-time system automatically invokes the method named `main` (that is, it automatically initiates the `main` action). An application program is a “regular” Java program, and, as we are about to see, there is another kind of Java program known as an *applet*. Other Java terminology is pretty much the same as the terminology in most other programming languages and, in any case, will be explained when each concept is introduced.

Applets ★

Java **applets** and **applications** are two kinds of common Java programs. An application, or application program, is just a regular program. Although the name *applet* may sound like it has something to do with apples, it really means a *little Java application*, not a little apple. Applets and applications are almost identical. The difference is that applications are meant to be run on your computer like any other program, whereas an **applet** is meant to be run from a Web browser, and so can be sent to another location on the Internet and run there. Applets always use a windowing interface, but not all programs with a windowing interface are applets, as you will see in Chapters 16–18.

Although applets were designed to be run from a Web browser, they can also be run with a program known as an **applet viewer**. The applet viewer is really meant

application
program

applet
application

applet viewer

as a debugging aid and not as the final environment to allow users to run applets. Nonetheless, applets are now often run as stand-alone programs using an applet viewer.¹ We find this to be a somewhat unfortunate accident of history. Java has multiple libraries of software for designing windowing interfaces that run without a connection to a browser. We prefer to use these libraries, rather than applets, to write windowing programs that will not be run from a Web browser. In this book, we show you how to do windowing interfaces as applets and as programs with no connection to a Web browser. In fact, the two approaches have a large overlap of both techniques and the Java libraries that they use. Once you know how to design and write either applets or applications, it is easy to learn to write the other of these two kinds of programs.

An applet always has a windowing interface. An application program may have a windowing interface or use simple console I/O. So as not to detract from the code being studied, most of our example programs, particularly early in this book, use simple console I/O (that is, simple text I/O).

A Sample Java Application Program

Display 1.1 contains a simple Java program and the screen displays produced when it is run. A Java program is really a class definition (whatever that is) with a method named `main`. When the program is run, the method named `main` is invoked; that is, the action specified by `main` is carried out. The body of the method `main` is enclosed in braces, `{ }`, so that when the program is run, the statements in the braces are executed. (If you are not even vaguely familiar with the words *class* and *method*, they will be explained. Read on.)

The following line says that this program is a class called `FirstProgram`:

```
public class FirstProgram
{
```

The next two lines, shown below, begin the definition of the `main` method:

```
public static void main(String[] args)
{
```

The details of exactly what a Java class is and what words such as `public`, `static`, `void`, and so forth mean will be explained in the next few chapters. Until then, think of these opening lines, repeated below, as being a rather wordy way of saying “Begin the program named `FirstProgram`.”

```
public class FirstProgram
{
    public static void main(String[] args)
    {
```

¹An applet viewer does indeed use a browser to run an applet, but the look and feel is that of a stand-alone program with no interaction with a browser.

Display 1.1 A Sample Java Program

```

1 public class FirstProgram
2 {
3     public static void main(String[] args)
4     {
5         System.out.println("Hello reader.");
6         System.out.println("Welcome to Java.");
7
8         System.out.println("Let's demonstrate a simple calculation.");
9         int answer;
10        answer = 2 + 2;
11        System.out.println("2 plus 2 is " + answer);
12    }

```

Annotations in the image:

- Name of class (program)* points to `FirstProgram`.
- The main method* points to `main(String[] args)`.

Sample Dialogue

```

Hello reader.
Welcome to Java.
Let's demonstrate a simple calculation.
2 plus 2 is 4

```

The next two lines, shown in what follows, are the first actions the program performs:

```

println        System.out.println("Hello reader.");
               System.out.println("Welcome to Java.");

```

Each of these lines begins with `System.out.println`. Each one causes the quoted string given within the parentheses to be output to the screen. For example, consider

```
System.out.println("Hello reader.");
```

This causes the line

```
Hello reader.
```

to be written to the screen. The output produced by the next line that begins with `System.out.println` will go on the following line. Thus, these two lines cause the following output:

```

Hello reader.
Welcome to Java.

```

```

System.out.
println

```

These lines that begin with `System.out.println` are a way of saying “output what is shown in parentheses,” and the details of why the instruction is written this way need not concern us yet. However, we can tell you a little about what is going on here.

As stated earlier, Java programs work by having things called *objects* perform actions. The actions performed by an object are called *methods*. `System.out` is an object used for sending output to the screen; `println` is the method (that is, the action) that this object performs. The action is to send what is in parentheses to the screen. When an object performs an action using a method, it is called **invoking** (or **calling**) the method. In a Java program, you write such a method invocation by writing the object followed by a **dot** (period), followed by the method name, and some parentheses that may or may not have something inside them. The thing (or things) inside the parentheses is called an **argument(s)**, which provides information needed by the method to carry out its action. In each of these two lines and the similar line that follows them, the method is `println`. The method `println` writes something to the screen, and the argument (a string in quotes) tells it what it should write.

Invoking a method is also sometimes called **sending a message** to the object. With this view, a message is sent to the object (by invoking a method) and in response, the object performs some action (namely, the action taken by the method invoked). We seldom use the terminology *sending a message*, but it is standard terminology used by some programmers and authors.

Variable declarations in Java are similar to what they are in other programming languages. The following line from Display 1.1 declares the variable `answer`:

```
int answer;
```

The type `int` is one of the Java types for integers (whole numbers). So, this line says that `answer` is a variable that can hold a single integer (whole number).

The following line is the only real computing done by this first program:

```
answer = 2 + 2;
```

In Java, the equal sign is used as the **assignment operator**, which is an instruction to set the value of the variable on the left-hand side of the equal sign. In the preceding program line, the equal sign does not mean that `answer` is *equal to* `2 + 2`. Instead, the equal sign is an instruction to the computer to *make* `answer` *equal to* `2 + 2`.

The last program action is

```
System.out.println("2 plus 2 is " + answer);
```

This is an output statement of the same kind as we discussed earlier, but there is something new in it. Note that the string `"2 plus 2 is "` is followed by a plus sign and the variable `answer`. In this case, the plus sign is an operator to concatenate (connect) two strings. However, the variable `answer` is not a string. If one of the two operands to `+` is a string, Java will convert the other operand, such as the value of `answer`, to a string. In this program, `answer` has the value 4, so `answer` is converted to the string `"4"` and then concatenated to the string `"2 plus 2 is "`, so the output statement under discussion is equivalent to

```
System.out.println("2 plus 2 is 4");
```

The remainder of this first program consists of two closing braces. The first closing brace ends the definition of the method `main`. The last closing brace ends the definition of the class named `FirstProgram`.

Self-Test Exercises

1. If the following statement were used in a Java program, it would cause something to be written to the screen. What would it cause to be written to the screen?

```
System.out.println("Java is not a drink.");
```

2. Give a statement or statements that can be used in a Java program to write the following to the screen:

```
I like Java.  
You like tea.
```

3. Write a complete Java program that uses `System.out.println` to output the following to the screen when run:

```
Hello World!
```

Note that you do not need to fully understand all the details of the program in order to write the program. You can simply follow the model of the program in Display 1.1.

Byte-Code and the Java Virtual Machine

Most modern programming languages are designed to be (relatively) easy for people to write and to understand. These languages are called **high-level languages**. The language that the computer can directly understand is called **machine language**. Machine language or any language similar to machine language is called a **low-level language**. A program written in a high-level language, such as Java, must be translated into a program in machine language before the program can be run. The program that does the translating (or at least most of the translating) is called a **compiler**, and the translation process is called **compiling**.

high-level,
low-level, and
machine
languages

compiler

Compiler

A **compiler** is a program that translates a high-level-language program, such as a Java program, into an equivalent low-level-language program.

One disadvantage of most programming languages is that the compiler translates the high-level-language program directly into the machine language for your computer. Since different computers have different machine languages, this means you need a different compiler for each type of computer. Java, however, uses a slightly different and much more versatile approach to compiling.

byte-code

Java Virtual
Machine
(JVM)

interpreter

Just-In-Time
(JIT)

While some versions of Java do translate your program into machine language for your particular computer, the original Java compiler and most compilers today do not. Instead, the Java compiler translates your Java program into a language called **byte-code**. Byte-code is not the machine language for any particular computer; it is the machine language for a fictitious computer called the **Java Virtual Machine (JVM)**. The Java Virtual Machine is very similar to all typical computers. Thus, it is easy to translate a program written in byte-code into a program in the machine language for any particular computer. The term *JVM* is also used to refer to the software that implements the fictitious computer. There are two ways the JVM can do this translation: through an **interpreter** and through a **Just-In-Time (JIT)** compiler.

An interpreter combines the translation of the byte-code and the execution of the corresponding machine language instructions. The interpreter works by translating an instruction of byte-code into instructions expressed in your computer's machine language and then executing those instructions on your computer. It does this one byte-code instruction at a time. Thus, an interpreter translates and executes the instructions in the byte-code one after the other, rather than translating the entire byte-code program at once.

Modern implementations of the JVM use a JIT compiler, which uses a combination of interpretation and compilation. The JIT compiler reads the byte-code in chunks and compiles entire chunks to native machine language instructions as needed. The compiled machine language instructions are remembered—that is, cached—for future use, so the chunk needs to be compiled only once. This model generally runs programs faster than the interpreted model, which always has to translate the next byte-code instruction to machine code instructions.

To run a Java program, first use the compiler to translate the Java program into byte-code. Then, use the JVM for your computer to translate byte-code instructions to machine language and to run the machine language instructions.

It sounds as though Java byte-code just adds an extra step in the process. Why not write compilers that translate directly from Java to the machine language for your particular computer? This is what is done for most other programming languages. However, Java byte-code makes your Java program very portable. After you compile your Java program into byte-code, you can use that byte-code on any computer. When you run your program on another type of computer, you do not need to recompile it. This means that you can send your byte-code over the Internet to another computer and have it easily run on that computer. This is one of the reasons Java is good for Internet applications. This model is also more secure. If a Java program behaves badly, it only does so within the context of the JVM instead of behaving badly directly on your native machine. Of course, every kind of computer must have its own program to implement the Java Virtual Machine.

Byte-Code

The Java compiler translates your Java program into a language called **byte-code**, which is the machine language for a fictitious computer. It is easy to translate this byte-code into the machine language of any particular computer. Each type of computer will have its own software to implement the Java Virtual Machine that translates and executes byte-code instructions.

**run
command**

When compiling and running a Java program, you are usually not even aware of the fact that your program is translated into byte-code and not directly translated into machine language code. You normally give two commands: one to compile your program (into byte-code) and one to run your program. The **run command** executes the Java Virtual Machine on the byte-code.

**source code
object code
code**

When you use a compiler, the terminology can get a bit confusing, because both the input to the compiler program and the output from the compiler program are also programs. Everything in sight is some kind of program. To make sure it is clear which program we mean, we call the input program (which in our case will be a Java program) the **source program**, or **source code**, and call the translated low-level-language program that the compiler produces the **object program**, or **object code**. The word **code** just means a program or a part of a program.

Class Loader ★

A Java program is divided into smaller parts called *classes*, and normally each class definition is in a separate file and is compiled separately. In order to run your program, the byte-code for these various classes needs to be connected together. The connecting is done by a program known as the **class loader**. It is typically done automatically, so you normally need not be concerned with it. (In other programming languages, the program corresponding to the Java class loader is called a *linker*.)



VideoNote
**Compiling a
Java Program**

.java files

Compiling a Java Program or Class

As we noted in the previous subsection, a Java program is divided into classes. Before you can run a Java program, you must compile these classes.

Before you can compile a Java program, each class definition used in the program (and written by you, the programmer) should be in a separate file. Moreover, the name of the file should be the same as the name of the class, except that the file name has `.java` added to the end. The program in Display 1.1 is a class called `FirstProgram`, so it should be in a file named `FirstProgram.java`. This program has only one class, but a more typical Java program would consist of several classes.

If you are using an IDE (Integrated Development Environment), there will be a simple command to compile your Java program from the editor. You will have to check your local documentation to see exactly what this command is, but it is bound to be very simple.

If you want or need to compile your Java program or class with a one-line command given to the operating system, it is easy to do. We will describe the commands for the Java system distributed by Oracle (usually called “the SDK” or “the JDK”) in the following paragraphs.

Suppose you want to compile a class named `FirstProgram`. It will be in a file named `FirstProgram.java`. To compile it, simply give the following command:

```
javac          javac FirstProgram.java
```

You should be in the same directory (folder) as the file `FirstProgram.java` when you give this `javac` command. To compile any Java class, whether it is a full program or not, the command is `javac` followed by the name of the file containing the class.

**.class
files**

When you compile a Java class, the resulting byte-code for that class is placed in a file of the same name, except that the ending is changed from `.java` to `.class`. So, when you compile a class named `FirstProgram` in the file `FirstProgram.java`, the resulting byte-code is stored in a file named `FirstProgram.class`.

Running a Java Program

A Java program can consist of a number of different classes, each in a different file. When you run a Java application program, only run the class that you think of as the program; that is, the class that contains a `main` method. Look for the following line, which starts the `main` method:

```
public static void main(String[] args)
```

The critical words to look for are `public static void main`. The remaining portion of the line might be spelled slightly different in some cases.

If you are using an IDE, you will have a menu command that can be used to run a Java program. You will have to check your local documentation to see exactly what this command is.

If you want or need to run your Java program with a one-line command given to the operating system, then (in most cases) you can run a Java program by giving the command `java` followed by the name of the class containing the `main` method. For example, for the program in Display 1.1, you would give the following one-line command:

```
java FirstProgram
```

Note that when you run a program, you use the class name, such as `FirstProgram`, without any `.java` or `.class` ending.

When you run a Java program, you are actually running the Java byte-code interpreter on the compiled version of your program. When you run your program, the system will automatically load in any classes you need and run the byte-code interpreter on those classes as well.

We have been assuming that the Java compiler and related software are already set up for you. We are also assuming that all the files are in one directory. (Directories are also called *folders*.) If you need to set up the Java compiler and system software, consult the manuals that came with the software. If you wish to spread your class definitions across multiple directories, that is not difficult, but we will not concern ourselves with that detail until later.

Syntax and Semantics

The description of a programming language, or any other kind of language, can be thought of as having two parts, called the *syntax* and *semantics* of the language.

The **syntax** tells what arrangement of words and punctuation is legal in the language. The syntax is often called the language's *grammar rules*. For Java, the syntax describes what arrangements of words and punctuation are allowed in a class or program definition.

The **semantics** of a language describes the meaning of things written while following the syntax rules of the language. For a Java program, the syntax describes how you write a program and the semantics describes what happens when you run the program.

When writing a program in Java, you are always using both the syntax and the semantics of the Java language.



TIP: Error Messages

bug

debugging

syntax error

A mistake in a program is called a **bug**. For this reason, the process of eliminating mistakes in your program is called **debugging**. There are three commonly recognized types of bugs or errors, which are known as *syntax errors*, *run-time errors*, and *logic errors*. Let's consider them in order.

A **syntax error** is a grammatical mistake in your program; that is, a mistake in the allowed arrangement of words and punctuations. If you violate one of these rules—for example, by omitting a required punctuation—it is a syntax error. The compiler will catch syntax errors and output an error message telling you that it has found the error, where it thinks the error is, and what it thinks the error is. If the compiler says you have a syntax error, you undoubtedly do. However, the compiler could be incorrect about where and what the error is.

run-time
error

An error that is not detected until your program is run is called a **run-time error**. If the computer detects a run-time error when your program is run, then it will output an error message. The error message may not be easy to understand, but at least it lets you know that something is wrong.

logic error

A mistake in the underlying algorithm for your program is called a **logic error**. If your program has only logic errors, it will compile and run without any error message. You have written a valid Java program, but you have not written a program that does what you want. The program runs and gives output, but the output is incorrect. For example, if you were to mistakenly use the multiplication sign in place of the addition sign, it would be a logic error. Logic errors are the hardest kind of error to locate, because the computer does not give you any error messages. ■

Self-Test Exercises

4. What is a compiler?
5. What is a source program?
6. What is an object program?
7. What do you call a program that runs Java byte-code instructions?
8. Suppose you define a class named `NiceClass` in a file. What name should the file have?
9. Suppose you compile the class `NiceClass`. What will be the name of the file with the resulting byte-code?

1.2 Expressions and Assignment Statements

Once a person has understood the way variables are used in programming, he has understood the quintessence of programming.

E. W. DIJKSTRA, *University of Texas*, 1969.

Variables, expressions, and assignments in Java are similar to their counterparts in most other general purpose languages. In this section, we describe the details.

Identifiers

identifier

The name of a variable (or other item you might define in a program) is called an **identifier**. A Java identifier must not start with a digit, and all the characters must be letters, digits, or the underscore (`_`) symbol. (The symbol `$` is also allowed, but it is reserved for special purposes only, so you should not typically use `$` in your Java identifiers.) For example, the following are all valid identifiers:

```
x x1 x_1 _abc ABC123z7 sum RATE count data2 bigBonus
```

All of the preceding names are legal and would be accepted by the compiler, but the first five are poor choices for identifiers, since they are not descriptive of the identifier's use. None of the following are legal identifiers, and all would be rejected by the compiler:

```
12 3X %change data-1 myfirst.java PROG.CLASS
```

The first two are not allowed because they start with a digit. The remaining four are not identifiers because they contain symbols other than letters, digits, and the underscore symbol.

case-sensitive

Java is a **case-sensitive** language; that is, it distinguishes between upper- and lowercase letters in the spelling of identifiers. Hence, the following are three distinct identifiers and could be used to name three distinct variables:

```
rate RATE Rate
```

However, it is usually not a good idea to use two such variants in the same program, because that might be confusing. Although it is not required by Java, variables are usually spelled with their first letter in lowercase. The convention that has become universal in Java programming is to spell variable names with a mix of upper- and lowercase letters (and digits), to always start a variable name with a lowercase letter, and to indicate “word” boundaries with an uppercase letter, as illustrated by the following variable names:

```
topSpeed bankRate1 bankRate2 timeOfArrival
```

A Java identifier can theoretically be of any length, and the compiler will accept even unreasonably long identifiers.

Names (Identifiers)

The name of something in a Java program, such as a variable, class, method, or object name, must not start with a digit and may only contain letters, digits (0 through 9), and the underscore character (`_`). Upper- and lowercase letters are considered to be different characters. (The symbol `$` is also allowed, but it is reserved for special purposes only, so you should not typically use `$` in a Java name.)

Names in a program are called **identifiers**.

Although it is not required by the Java language, the common practice, and the one followed in this book, is to start the names of classes with uppercase letters and to start the names of variables, objects, and methods with lowercase letters. These names are usually spelled using only letters and digits.

keyword

There is a special class of identifiers, called **keywords** or **reserved words**, that have a predefined meaning in Java and that you cannot use as names for variables or anything else. In the code displays of this book, keywords are shown in a different color, as illustrated by the keyword `public`. A complete list of keywords is given in Appendix 1.

Some predefined words, such as `System` and `println`, are not keywords. These predefined words are not part of the core Java language and you are allowed to redefine them. Although these words are not keywords, they are defined in libraries required by the Java language standard. Needless to say, using a predefined identifier for anything other than its standard meaning can be confusing and dangerous, and thus should be avoided. The safest and easiest practice is to treat all predefined identifiers as if they are keywords.

Variables

declare

Every variable in a Java program must be declared before it is used. When you **declare** a variable, you are telling the compiler—and, ultimately, the computer—what kind of data you will be storing in the variable. For example, the following are two declarations that might occur in a Java program:

```
int numberOfBeans;  
double oneWeight, totalWeight;
```

The first declares the variable `numberOfBeans` so that it can hold a value of type `int`; that is, a whole number. The name `int` is an abbreviation for “integer.” The type `int` is the default type for whole numbers. The second definition declares `oneWeight` and `totalWeight` to be variables of type `double`, which is the default type for numbers with a decimal point (known as **floating-point numbers**). As illustrated here, when there is more than one variable in a declaration, the variables are separated by commas. Also, note that each declaration ends with a semicolon.

Every variable must be declared before it is used. A variable may be declared anywhere, so long as it is declared before it is used. Of course, variables should always be declared in a location that makes the program easier to read. Typically, variables are declared either just before they are used or at the start of a block (indicated by an opening brace `{`). Any legal identifier, other than a keyword, may be used for a variable name.

floating-point
number

Variable Declarations

In Java, a variable must be declared before it is used. Variables are declared as described here.

SYNTAX

Type Variable_1, Variable_2, ...;

EXAMPLES

```
int count, numberOfDragons, numberOfTrolls;  
char answer;  
double speed, distance;
```

Syntactic Variables

Remember that when you see something such as *Type*, *Variable_1*, or *Variable_2*, these words do not literally appear in your Java code. They are **syntactic variables**, which means they are replaced by something of the category that they describe. For example, *Type* can be replaced by `int`, `double`, `char`, or any other type name. *Variable_1* and *Variable_2* can each be replaced by any variable name.

primitive
types

Java has basic types for characters, different kinds of integers, and different kinds of floating-point numbers (numbers with a decimal point), as well as a type for the values `true` and `false`. These basic types are known as **primitive types**. Display 1.2 shows all of Java’s primitive types. The preferred type for integers is `int`. The type `char` is the type for single characters and can store common Unicode characters. The preferred type for floating-point numbers is `double`. The type `boolean` has the two values `true` and `false`. (Unlike some other programming languages, the Java values `true` and `false` are not integers and will not be automatically converted to integers.) Objects of the predefined class `String` represent strings of characters. `String` is not a primitive type, but is often considered a basic type along with the primitive types. The class `String` is discussed later in this chapter.

Assignment Statements

assignment
statement

assignment
operator

The most direct way to change the value of a variable is to use an **assignment statement**. In Java, the equal sign is used as the **assignment operator**. An assignment statement always consists of a variable on the left-hand side of the assignment operator (the equal sign) and an expression on the right-hand side. An assignment statement ends with a semicolon. The expression on the right-hand side of the equal sign may be a variable, a number, or a more complicated expression made up of variables, numbers,

Display 1.2 Primitive Types

TYPE NAME	KIND OF VALUE	MEMORY USED	SIZE RANGE
<code>boolean</code>	<code>true</code> or <code>false</code>	1 byte	Not applicable
<code>char</code>	Single character (Unicode)	2 bytes	Common Unicode characters
<code>byte</code>	Integer	1 byte	−128 to 127
<code>short</code>	Integer	2 bytes	−32768 to 32767
<code>int</code>	Integer	4 bytes	−2147483648 to 2147483647
<code>long</code>	Integer	8 bytes	−9223372036854775808 to 9223372036854775807
<code>float</code>	Floating-point number	4 bytes	$\pm 3.40282347 \times 10^{+38}$ to $\pm 1.40239846 \times 10^{-45}$
<code>double</code>	Floating-point number	8 bytes	$\pm 1.76769313486231570 \times 10^{+308}$ to $\pm 4.94065645841246544 \times 10^{-324}$

operators, and method invocations. An assignment statement instructs the computer to evaluate (that is, to compute the value of) the expression on the right-hand side of the equal sign and to set the value of the variable on the left-hand side equal to the value of that expression. The following are examples of Java assignment statements:

```
totalWeight = oneWeight * numberOfBeans;  
temperature = 98.6;  
count = count + 2;
```

The first assignment statement sets the value of `totalWeight` equal to the number in the variable `oneWeight` multiplied by the number in `numberOfBeans`. (Multiplication is expressed using the asterisk `*` in Java.) The second assignment statement sets the value of `temperature` to 98.6. The third assignment statement increases the value of the variable `count` by 2.

Note that a variable may occur on both sides of the assignment operator (both sides of the equal sign). The assignment statement

```
count = count + 2;
```

sets the new value of `count` equal to the old value of `count` plus 2.

When used with variables of a class type, the assignment operator requires a bit more explanation, which we will give in Chapter 4.

Assignment Statements with Primitive Types

An assignment statement with a variable of a primitive type on the left-hand side of the equal sign causes the following actions: First, the expression on the right-hand side of the equal sign is evaluated, and then the variable on the left-hand side of the equal sign is set equal to this value.

SYNTAX

Variable = Expression;

EXAMPLE

```
distance = rate * time;  
count = count + 2;
```

An assigned statement may be used as an expression that evaluates to a value. When used this way, the variable on the left-hand side of the equal sign is changed as we have described, and the new value of the variable is also the value of the assignment expression. For example,

```
number = 3;
```


both changes the value of `number` to 3 and evaluates to the value 3. This allows you to chain assignment statements. The following changes the values of both the variables, `number1` and `number2`, to 3:

```
number2 = (number1 = 3);
```

The assignment operator automatically is executed right to left if there are no parentheses, so this is normally written in the following equivalent way:

```
number2 = number1 = 3;
```



uninitialized variable

TIP: Initialize Variables

A variable that has been declared but that has not yet been given a value by some means, such as an assignment statement, is said to be **uninitialized**. In some instances, an uninitialized variable may be given some default value, but this is not true in all cases. Moreover, it makes your program clearer to explicitly give the variable a value, even if you are simply reassigning it the default value. (The exact details on default values have been known to change and should not be counted on.)²

One easy way to ensure that you do not have an uninitialized variable is to initialize it within the declaration. Simply combine the declaration and an assignment statement, as in the following examples:

```
int count = 0;
double speed = 65.5;
char grade = 'A';
int initialCount = 50, finalCount;
```

Note that you can initialize some variables and not initialize others in a declaration.

Sometimes the compiler may say that you have failed to initialize a variable. In most cases, this will indeed have occurred. Occasionally, the compiler is mistaken. However, the compiler will not compile your program until you convince it that the variable in question is initialized. To make the compiler happy, initialize the variable when it is declared, even if the variable will be given a different value before the variable is used for anything. In such cases, you cannot argue with the compiler. ■

²The official rules are that the variables we are now using, which we will later call *local variables*, are not automatically initialized. Later in this book, we will introduce variables called *static variables* and *instance variables*, which are automatically initialized. However, we urge you to never rely on automatic initialization.

Initializing a Variable in a Declaration

You can combine the declaration of a variable with an assignment statement that gives the variable a value.

SYNTAX

Type Variable_1 =, Variable_2 = Expression__2, ...;

Some of the variables may have no equal sign and no expression, as in the first example.

EXAMPLE

```
int numberReceived = 0, lastNumber, numberOfStations = 5;
double speed = 98.9, distance = speed * 10;
char initial = 'J';
```

More Assignment Statements ★

There is a shorthand notation that combines the assignment operator (=) and an arithmetic operator so that a given variable can have its value changed by adding, subtracting, multiplying, or dividing by a specified value. The general form is

Variable Op = Expression

which is equivalent to

Variable = Variable Op (Expression)

The *Expression* can be another variable, a constant, or a more complicated arithmetic expression. The *Op* can be any of +, −, *, /, or %, as well as some operators we have not yet discussed—the operator % has also not yet been discussed but is explained later in this chapter. (A full list of values for *Op* can be seen at the bottom of the precedence table in Appendix 2.) Below are examples:

EXAMPLE:	EQUIVALENT TO:
count += 2;	count = count + 2;
total -= discount;	total = total - discount;
bonus *= 2;	bonus = bonus * 2;
time /= rushFactor;	time = time / rushFactor;
change %= 100;	change = change % 100;
amount *= count1 + count2;	amount = amount * (count1 + count2);

Self-Test Exercises

10. Which of the following may be used as variable names in Java?

```
rate1, 1stPlayer, myprogram.java, long,
TimeLimit, numberOfWindows
```

11. Can a Java program have two different variables named `number` and `Number`?
12. Give the declaration for two variables called `feet` and `inches`. Both variables are of type `int` and both are to be initialized to zero in the declaration.
13. Give the declaration for two variables called `count` and `distance`. `count` is of type `int` and is initialized to zero. `distance` is of type `double` and is initialized to `1.5`.
14. Write a Java assignment statement that will set the value of the variable `distance` to the value of the variable `time` multiplied by `80`. All variables are of type `int`.
15. Write a Java assignment statement that will set the value of the variable `interest` to the value of the variable `balance` multiplied by the value of the variable `rate`. The variables are of type `double`.
16. What is the output produced by the following lines of program code?

```
char a, b;
a = 'b';
System.out.println(a);
b = 'c';
System.out.println(b);
a = b;
System.out.println(a);
```

Assignment Compatibility

As a general rule, you cannot store a value of one type in a variable of another type. For example, the compiler will object to the following:

```
int intValue;
intValue = 2.99;
```

The problem is a type mismatch. The constant `2.99` is of type `double`, and the variable `intValue` is of type `int`.

There are some special cases where it is permitted to assign a value of one type to a variable of another type. It is acceptable to assign a value of an integer type, such as `int`, to a variable of a floating-point type, such as the type `double`. For example, the following is both legal and acceptable style:

```
double doubleVariable;
doubleVariable = 2;
```

The preceding will set the value of the variable named `doubleVariable` equal to `2.0`.

assigning
int values
to double
variables

Similarly, assignments of integer type variables to floating-point type variables are also allowed. For example, the following is permitted:

```
int intVariable;
intVariable = 42;
double doubleVariable;
doubleVariable = intVariable;
```

More generally, you can assign a value of any type in the following list to a variable of any type that appears further down in the list:

```
byte -> short -> int -> long -> float -> double
```

For example, you can assign a value of type `int` to a variable of type `long`, `float`, or `double` (or of course to a variable of type `int`), but you cannot assign a value of type `int` to a variable of type `byte` or `short`. Note that this is not an arbitrary ordering of the types. As you move down the list from left to right, the range of allowed values for the types becomes larger.

You can assign a value of type `char` to a variable of type `int` or to any of the numeric types that follow `int` in our list of types (but not to those that precede `int`). However, in most cases it is not wise to assign a character to an `int` variable, because the result could be confusing.³

If you want to assign a value of type `double` to a variable of type `int`, then you must change the type of the value by using a *type cast*, as explained in the subsection later in this chapter entitled “Type Casting.”

In many languages other than Java, you can assign integers to variables of type `boolean` and assign `boolean` values to integer variables. You cannot do that in Java. In Java, the `boolean` values `true` and `false` are not integers nor will they be automatically converted to integers. (In fact, it is not even legal to do an explicit type cast from the type `boolean` to the type `int` or vice versa. Explicit type casts are discussed later in this chapter in the subsection “Type Casting.”)

integers and booleans

Constants

constants literals

Constants or **literals** are names for one specific value. For example, `2` and `3.1459` are two constants. We prefer the name *constants* because it contrasts nicely with the word *variables*. Constants do not change value; variables can change their values.

³Readers who have used certain other languages, such as C or C++, may be surprised to learn that you cannot assign a value of type `char` to a variable of type `byte`. This is because Java uses the Unicode character set rather than the ASCII character set, and so Java reserves two bytes of memory for each value of type `char`, but naturally only reserves one byte of memory for values of type `byte`. This is one of the few cases where you might notice that Java uses the Unicode character set. Indeed, if you convert from an `int` to a `char` or vice versa, you can expect to get the usual correspondence of ASCII numbers and characters. It is also true that you cannot assign a value of type `char` to a variable of type `short`, even though they both use two bytes of memory.

Assignment Compatibilities

You can assign a value of any type on the following list to a variable of any type that appears further down on the list:

```
byte -> short -> int -> long -> float -> double
```

In particular, note that you can assign a value of any integer type to a variable of any floating-point type. You can also assign a value of type `char` to a variable of type `int` or of any type that follows `int` in the above list.

e notation

Integer constants are written in the way you are used to writing numbers. Constants of type `int` (or any other integer type) must not contain a decimal point. Constants of floating-point types (`float` and `double`) may be written in either of two forms. The simple form for floating-point constants is like the everyday way of writing decimal fractions. When written in this form, a floating-point constant must contain a decimal point. No number constant (neither integer nor floating point) in Java may contain a comma.

A more complicated notation for floating-point constants, such as constants of type `double`, is called **scientific notation** or **floating-point notation** and is particularly handy for writing very large numbers and very small fractions. For instance,

3.67×10^5 , which is the same as `367000.0`,

is best expressed in Java by the constant `3.67e5`. The number

5.89×10^{-4} , which is the same as `0.000589`,

is best expressed in Java by the constant `5.89e-4`. The `e` stands for *exponent* and means “multiply by 10 to the power that follows.” The `e` may be either upper- or lowercase.

Think of the number after the `e` as telling you the direction and number of digits to move the decimal point. For example, to change `3.49e4` to a numeral without an `e`, move the decimal point 4 places to the right to obtain `34900.0`, which is another way of writing the same number. If the number after the `e` is negative, move the decimal point the indicated number of spaces to the left, inserting extra zeros if need be. So, `3.49e-2` is the same as `0.0349`.

The number before the `e` may contain a decimal point, although that is not required. However, the exponent after the `e` definitely must *not* contain a decimal point.

Constants of type `char` are expressed by placing the character in single quotes, as illustrated in what follows:

```
char symbol = 'Z';
```

Note that the left and right single quote symbols are the same symbol.

What Is Doubled?

How did the floating-point type `double` get its name? Is there another type for floating-point numbers called “single” that is half as big? Something like that is true. There is a type that uses half as much storage—namely, the type `float`. Many programming languages traditionally used two types for floating-point numbers. One type used less storage and was very imprecise (that is, it did not allow very many significant digits). The second type used *double* the amount of storage and so could be much more precise; it also allowed numbers that were larger (although programmers tend to care more about precision than about size). The kind of numbers that used twice as much storage were called *double precision* numbers; those that used less storage were called *single precision*. Following this tradition, the type that (more or less) corresponds to this double precision type in Java was named `double` in Java. The type that corresponds to single precision in Java was called `float`.

(Actually, the type name `double` was inherited from C++, but this explanation applies to why the type was named `double` in C++, and so ultimately it is the explanation of why the type is called `double` in Java.)

Constants for strings of characters are given in double quotes, as illustrated by the following line taken from Display 1.1:

```
System.out.println("Welcome to Java.");
```

quotes

Be sure to notice that string constants are placed inside of double quotes, while constants of type `char` are placed inside of single quotes. The two kinds of quotes mean different things. In particular, `'A'` and `"A"` mean different things. `'A'` is a value of type `char` and can be stored in a variable of type `char`. `"A"` is a string of characters. The fact that the string happens to contain only one character does *not* make the string `"A"` a value of type `char`. Also notice that, for both strings and characters, the left and right quotes are the same. We will have more to say about strings later in this chapter.

The type `boolean` has two constants, `true` and `false`. These two constants may be assigned to a variable of type `boolean` or used anywhere else an expression of type `boolean` is allowed. They must be spelled with all lowercase letters.

Arithmetic Operators and Expressions

As in most other languages, Java allows you to form expressions using variables, constants, and the arithmetic operators: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), and `%` (modulo, remainder). These expressions can be used anywhere it is legal to use a value of the type produced by the expression.

All of the arithmetic operators can be used with numbers of type `int`, numbers of type `double`, and even with one number of each type. However, the type of the value produced and the exact value of the result depend on the types of the numbers being

mixing types

combined. If both operands (that is, both numbers) are of type `int`, then the result of combining them with an arithmetic operator is of type `int`. If one, or both, of the operands is of type `double`, then the result is of type `double`. For example, if the variables `baseAmount` and `increase` are both of type `int`, then the number produced by the following expression is of type `int`:

```
baseAmount + increase
```

However, if one, or both, of the two variables is of type `double`, then the result is of type `double`. This is also true if you replace the operator `+` with any of the operators `-`, `*`, `/`, or `%`.

More generally, you can combine any of the arithmetic types in expressions. If all the types are integer types, the result will be the integer type. If at least one of the subexpressions is of a floating-point type, the result will be a floating-point type.

Knowing whether the value produced is of an integer type or a floating-point type is typically all that you need to know. However, if you need to know the exact type of the value produced by an arithmetic expression, it can be determined as follows: The type of the value produced is one of the types used in the expression. Of all the types used in the expression, it is, with rare exceptions, the last type (reading left to right) on the following list:

```
byte -> short -> int -> long -> float -> double
```

Here are the rare exceptions: Of all the types used in the expression, if the last type (reading left to right) is `byte` or `short`, then the type of the value produced is `int`. In other words, an expression never evaluates to either of the types `byte` or `short`. These exceptions have to do with an implementation detail that need not concern us, especially since we almost never use the types `byte` and `short` in this book.

Note that this sequence of types is the same sequence of types we saw when discussing assignment compatibility. As you go from left to right, the types increase in the range of values they allow.⁴

Parentheses and Precedence Rules ★

If you want to specify exactly what subexpressions are combined with each operator, you can fully parenthesize an expression. For example,

```
((base + (rate * hours)) / (2 + rate))
```

If you omit some parentheses in an arithmetic expression, Java will, in effect, put in parentheses for you. When adding parentheses, Java follows rules called **precedence rules**

⁴Although we discourage the practice, you can use values and variables of type `char` in arithmetic expressions using operators such as `+`. If you do so, the `char` values and variables will contribute to the expression as if they were of type `int`.

that determine how the operators, such as + and *, are enclosed in parentheses. These precedence rules are similar to rules used in algebra. For example,

```
base + rate * hours
```

is evaluated by Java as if it were parenthesized as follows:

```
base + (rate * hours)
```

So, the multiplication will be done before the addition.

Except in some standard cases, such as a string of additions or a simple multiplication embedded inside an addition, it is usually best to include the parentheses, even if the intended groupings are the ones dictated by the precedence rules. The parentheses make the expression easier to read and less prone to programmer error.

A partial list of precedence rules is given in Display 1.3. A complete set of Java precedence rules is given in Appendix 2. Operators that are listed higher on the list are said to have **higher precedence**. When the computer is deciding which of two adjacent operations to group with parentheses, it groups the operation of higher precedence and its apparent arguments before the operation of lower precedence. Some operators have equal precedence, in which case the order of operations is determined by **associativity rules**. A brief summary of associativity rules is that binary operators of equal precedence are grouped in left-to-right order.⁵ Unary operators of equal precedence are grouped in right-to-left order. So, for example,

```
base + rate + hours
```

is interpreted by Java to be the same as

```
(base + rate) + hours
```

And, for example,

```
++rate
```

is interpreted by Java to be the same as

```
+(-(rate))
```

For now you can think of the explicit parentheses put in by the programmer and the implicit parentheses determined by precedence and associativity rules as determining the order in which operations are performed. For example, in

```
base + (rate * hours)
```

the multiplication is performed first and the addition is performed second.

⁵There is one exception to this rule. A string of assignment operators, such as `n1 = n2 = n3 ;`, is performed right to left, as we noted earlier in this chapter.

Display 1.3 Precedence Rules

Highest Precedence

First: the unary operators: +, -, ++, --, and !

Second: the binary arithmetic operators: *, /, and %

Third: the binary arithmetic operators: + and -

Lowest Precedence

The actual situation is a bit more complicated than what we have described for evaluating expressions, but we will not encounter any of these complications in this chapter. A complete discussion of evaluating expressions using precedence and associativity rules will be given in Chapter 3.

Integer and Floating-Point Division

integer division

When used with one or both operands of type `double`, the division operator, `/`, behaves as you might expect. However, when used with two operands of type `int`, the division operator yields the integer part resulting from division. In other words, integer division discards the part after the decimal point. So, $10/3$ is 3 (not $3.3333\dots$), $5/2$ is 2 (not 2.5), and $11/3$ is 3 (not $3.6666\dots$). Notice that the number *is not rounded*; the part after the decimal point is discarded no matter how large it is.

the % operator

The operator `%` can be used with operands of type `int` to recover the information lost when you use `/` to do division with numbers of type `int`. When used with values of type `int`, the two operators `/` and `%` yield the two numbers produced when you perform the long division algorithm you learned in grade school. For example, 14 divided by 3 is 4 with a remainder of 2. The `/` operation yields the number of times one number “goes into” another (often called the *quotient*). The `%` operation gives the remainder. For example, the statements

```
System.out.println("14 divided by 3 is " + (14 / 3));
System.out.println("with a remainder of " + (14 % 3));
```

yield the following output:

```
14 divided by 3 is 4
with a remainder of 2
```

The `%` operator can be used to count by 2s, 3s, or any other number. For example, if you want to do something to every other integer, you need to know if the integer is even or odd. Then, you can do it to every even integer (or alternatively every odd integer). An integer `n` is even if `n % 2` is equal to 0 and the integer is odd if `n % 2` is equal to 1. Similarly, to do something to every third integer, your program might step through all integers `n` but only do the action when `n % 3` is equal to 0.



PITFALL: Round-Off Errors in Floating-Point Numbers

For all practical purposes, floating-point numbers are only approximate quantities. For example, in formal mathematics, the floating-point number $1.0/3.0$ is equal to

`0.3333333...`

where the three dots indicate that the 3s go on forever. The computer stores numbers in a format somewhat like this decimal representation, but it has room for only a limited number of digits. If it can store only 10 digits after the decimal, then $1.0/3.0$ is stored as

`0.3333333333`

with only 10 threes. Thus, $1.0/3.0$ is stored as a number that is slightly smaller than one-third. In other words, the value stored as $1.0/3.0$ is only approximately equal to one-third.

In reality, the computer stores numbers in binary notation, rather than in base 10 notation, but the principles and the consequences are the same. Some floating-point numbers lose accuracy when they are stored in the computer.

Floating-point numbers (such as numbers of type `double`) and integers (such as numbers of type `int`) are stored differently. Floating-point numbers are, in effect, stored as approximate quantities. Integers are stored as exact quantities. This difference sometimes can be subtle. For example, the numbers 42 and 42.0 are different in Java. The whole number 42 is of type `int` and is an exact quantity. The number 42.0 is of type `double` because it contains a fractional part (even though the fraction is 0), and so 42.0 is stored with only limited accuracy.

As a result of this limited accuracy, arithmetic done on floating-point numbers only gives approximate results. Moreover, one can easily get results on floating-point numbers that are very far from the true result you would obtain if the numbers could have unlimited accuracy (unlimited number of digits after the decimal point). For example, if a banking program used numbers of type `double` to represent amounts of money and did not do sophisticated manipulations to preserve accuracy, it would quickly bring the bank to ruin since the computed amounts of money would frequently be very incorrect. Dealing with these inaccuracies in floating-point numbers is part of the field of Numerical Analysis, a topic we will not discuss in this book. But, there is an easy way to obtain accuracy when dealing with amounts of money: Use integers instead of floating-point numbers (perhaps one integer for the dollar amount and another integer for the cents amount). ■

Although the `%` operator is primarily used with integers, it can also be used with two floating-point numbers, such as two values of type `double`. However, we will not discuss or use `%` with floating-point numbers.



PITFALL: Division with Whole Numbers

When you use the division operator `/` on two integers, the result is an integer. This can be a problem if you expect a fraction. Moreover, the problem can easily go unnoticed, resulting in a program that looks fine but is producing incorrect output without you even being aware of the problem. For example, suppose you are a landscape architect who charges \$5,000 per mile to landscape a highway, and suppose you know the length in feet of the highway you are working on. The price you charge can easily be calculated by the following Java statement:

```
totalPrice = 5000 * (feet / 5280.0);
```

This works because there are 5,280 feet in a mile. If the stretch of highway you are landscaping is 15,000 feet long, this formula will tell you that the total price is

```
5000 * (15000 / 5280.0)
```

Your Java program obtains the final value as follows: `15000/5280.0` is computed as `2.84`. Then, the program multiplies `5000` by `2.84` to produce the value `14200.00`. With the aid of your Java program, you know that you should charge \$14,200 for the project.

Now suppose the variable `feet` is of type `int`, and you forget to put in the decimal point and the zero, so that the assignment statement in your program reads as follows:

```
totalPrice = 5000 * (feet / 5280);
```

It still looks fine, but will cause serious problems. If you use this second form of the assignment statement, you are dividing two values of type `int`, so the result of the division `feet/5280` is `15000/5280`, which is the `int` value `2` (instead of the value `2.84`, which you think you are getting). So the value assigned to `totalPrice` is `5000*2`, or `10000.00`. If you forget the decimal point, you will charge \$10,000. However, as we have already seen, the correct value is \$14,200. A missing decimal point has cost you \$4,200. Note that this will be true whether the type of `totalPrice` is `int` or `double`; the damage is done before the value is assigned to `totalPrice`. ■

Self-Test Exercises

17. Convert each of the following mathematical formulas to a Java expression:

$$3x \quad 3x + y \quad \frac{x + y}{7} \quad \frac{3x + y}{z + 2}$$

18. What is the output of the following program lines?

```
double number = (1/3) * 3;
System.out.println("(1/3) * 3 is equal to " + number);
```

Self-Test Exercises (continued)

19. What is the output produced by the following lines of program code?

```
int quotient, remainder;
quotient = 7 / 3;
remainder = 7 % 3;
System.out.println("quotient = " + quotient);
System.out.println("remainder = " + remainder);
```

20. What is the output produced by the following code?

```
int result = 11;
result /= 2;
System.out.println("result is " + result);
```

21. Given the following fragment that purports to convert from degrees Celsius to degrees Fahrenheit, answer the following questions:

```
double celsius = 20;
double fahrenheit;
fahrenheit = (9 / 5) * celsius + 32.0;
```

- What value is assigned to `fahrenheit`?
- Explain what is actually happening, and what the programmer likely wanted.
- Rewrite the code as the programmer intended.

Type Casting

A type cast takes a value of one type and produces a value of another type that is Java's best guess of an equivalent value. We will motivate type casts with a simple division example.

Consider the expression $9/2$. In Java, this expression evaluates to 4, because when both operands are of an integer type, Java performs integer division. In some situations, you might want the answer to be the `double` value 4.5. You can get a result of 4.5 by using the “equivalent” floating-point value 2.0 in place of the integer value 2, as in the expression $9/2.0$, which evaluates to 4.5. But, what if the 9 and the 2 are the values of variables of type `int` named `n` and `m`. Then, `n/m` yields 4. If you want floating-point division in this case, you must do a type cast from `int` to `double` (or another floating-point type), such as in the following:

```
double ans = n / (double)m;
```

The expression

```
(double)m
```

is a type cast. The expression takes an `int` (in this example, the value of `m`) and evaluates to an “equivalent” value of type `double`. So, if the value of `m` is 2, the expression `(double)m` evaluates to the `double` value 2.0.

Note that `(double)m` does not change the value of the variable `m`. If `m` has the value 2 before this expression is evaluated, then `m` still has the value 2 after the expression is evaluated.

You may use other type names in place of `double` to obtain a type cast to another type. We said this produces an “equivalent” value of the target type. The word “equivalent” is in quotes because there is no clear notion of equivalent that applies between any two types. In the case of a type cast from an integer type to a floating-point type, the effect is to add a decimal point and a zero. A type cast in the other direction, from a floating-point type to an integer type, simply deletes the decimal point and all digits after the decimal point. Note that when type casting from a floating-point type to an integer type, the number is truncated, not rounded: `(int)2.9` is 2; it is not 3.

As we noted earlier, you can always assign a value of an integer type to a variable of a floating-point type, as in the following:

```
double d = 5;
```

In such cases Java performs an automatic type cast, converting the 5 to 5.0 and placing 5.0 in the variable `d`. You cannot store the 5 as the value of `d` without a type cast, but sometimes Java does the type cast for you. Such an automatic type cast is sometimes called a **type coercion**.

type coercion

By contrast, you cannot place a `double` value in an `int` variable without an explicit type cast. The following is illegal:

```
int i = 5.5; //Illegal
```

Instead, you must add an explicit type cast, like so:

```
int i = (int)5.5;
```

Increment and Decrement Operators

The **increment operator** `++` adds one to the value of a variable. The **decrement operator** `--` subtracts one from the value of a variable. They are usually used with variables of type `int`, but they can be used with any numeric type. If `n` is a variable of a numeric type, then `n++` increases the value of `n` by one and `n--` decreases the value of `n` by one. So, `n++` and `n--` (when followed by a semicolon) are executable statements. For example, the statements

```
int n = 1, m = 7;
n++;
System.out.println("The value of n is changed to " + n);
m--;
System.out.println("The value of m is changed to " + m);
```

yield the following output:

```
The value of n is changed to 2
The value of m is changed to 6
```

An expression such as `n++` also evaluates to a number as well as changing the value of the variable `n`, so `n++` can be used in an arithmetic expression such as the following:

```
2 * (n++)
```

The expression `n++` changes the value of `n` by adding one to it, but it evaluates to the value `n` had *before* it was increased. For example, consider the following code:

```
int n = 2;
int valueProduced = 2 * (n++);
System.out.println(valueProduced);
System.out.println(n);
```

This code produces the following output:

```
4
3
```

Notice the expression `2 * (n++)`. When Java evaluates this expression, it uses the value that number has *before* it is incremented, not the value that it has after it is incremented. Thus, the value produced by the expression `n++` is 2, even though the increment operator changes the value of `n` to 3. This may seem strange, but sometimes it is just what you want. And, as you are about to see, if you want an expression that behaves differently, you can have it.

The expression `++n` also increments the value of the variable `n` by one, but it evaluates to the value `n` has after it is increased. For example, consider the following code:

```
int n = 2;
int valueProduced = 2 * (++n);
System.out.println(valueProduced);
System.out.println(n);
```

This code is the same as the previous piece of code except that the `++` is before the variable, so this code will produce the following output:

```
6
3
```

v++ versus ++v

Notice that the two increment operators `n++` and `++n` have the exact same effect on a variable `n`: They both increase the value of `n` by one. But the two expressions evaluate to different values. Remember, if the `++` is *before* the variable, then the incrementing is done *before* the value is returned; if the `++` is *after* the variable, then the incrementing is done *after* the value is returned.

**decrement
operator**

Everything we said about the increment operator applies to the decrement operator as well, except that the value of the variable is decreased by one rather than increased by one. For example, consider the following code:

```
int n = 8;
int valueProduced = n--;
System.out.println(valueProduced);
System.out.println(n);
```

This produces the following output:

```
8
7
```

On the other hand, the code

```
int n = 8;
int valueProduced = --n;
System.out.println(valueProduced);
System.out.println(n);
```

produces the following output:

```
7
7
```

Both `n--` and `--n` change the value of `n` by subtracting one, but they evaluate to different values. `n--` evaluates to the value `n` had before it was decremented; on the other hand, `--n` evaluates to the value `n` has after it is decremented.

You cannot apply the increment and decrement operators to anything other than a single variable. Expressions such as `(x + y)++`, `--(x + y)`, `5++`, and so forth are all illegal in Java.

The use of the increment and decrement operators can be confusing when used inside of more complicated expressions, and so, we prefer to not use increment or decrement operators inside of expressions, but to only use them as simple statements, such as the following:

```
n++;
```

Self-Test Exercises

22. What is the output produced by the following lines of program code?

```
int n = (int)3.9;
System.out.println("n == " + n);
```

Self-Test Exercises (continued)

23. What is the output produced by the following lines of program code?

```
int n = 3;
n++;
System.out.println("n == " + n);
n--;
System.out.println("n == " + n);
```

1.3 The Class String

Words, words, mere words, no matter from the heart.

WILLIAM SHAKESPEARE, *Troilus and Cressida*, 1602.

String

There is no primitive type for strings in Java. However, there is a class called `String` that can be used to store and process strings of characters. This section introduces the class `String`.

String Constants and Variables

You have already seen constants of type `String`. The quoted string

```
"Hello reader."
```

which appears in the following statement from Display 1.1, is a string constant:

```
System.out.println("Hello reader.");
```

A quoted string is a value of type `String`, although it is normally called an *object* of type `String` rather than a value of type `String`. An object of type `String` is a sequence of characters treated as a single item. A variable of type `String` can name one of these string objects.

For example, the following declares `blessing` to be the name for a `String` variable:

```
String blessing;
```

The following assignment statement sets the value of `blessing` so that `blessing` serves as another name for the `String` object `"Live long and prosper."`:

```
blessing = "Live long and prosper.";
```

The declaration and assignment can be combined into a single statement, as follows:

```
String blessing = "Live long and prosper.";
```


You can write the object named by the `String` variable `blessing` to the screen as follows:

```
System.out.println(blessing);
```

which produces the screen output

```
Live long and prosper.
```

The `String` Class

The class `String` is a predefined class that is automatically made available to you when you are programming in Java. Objects of type `String` are strings of characters that are written within double quotes. For example, the following declares the variable `motto` to be of type `String` and makes `motto` a name for the `String` object "We aim to please.".

```
String motto = "We aim to please.";
```

Concatenation of Strings

+ operator
concatenation

When you use the **+** **operator** on two strings, the result is the string obtained by connecting the two strings to get a longer string. This is called **concatenation**. So, when it is used with strings, the **+** is sometimes called the **concatenation operator**. For example, consider the following:

```
String noun = "Strings";
String sentence;
sentence = noun + "are cool.";
System.out.println(sentence);
```

This will set the variable `sentence` to "Stringsare cool." and will output the following to the screen:

```
Stringsare cool.
```

Note that no spaces are added when you concatenate two strings. If you want `sentence` set to "Strings are cool.", then you should change the assignment statement to add the extra space. For example, the following will add the desired space:

```
sentence = noun + " are cool.";
```

We added a space before the word "are".

You can concatenate any number of `Strings` using the **+** operator. Moreover, you can use the **+** operator to concatenate a `String` to almost any other type of item. The result is always a `String`. In most situations, Java will convert an item of any type to

Using the + Sign with Strings

If you connect two strings with the + operator, the result is the concatenation (pasting) of the two strings.

EXAMPLE

```
String name = "Chiana";  
String farewell = "Good bye " + name;  
System.out.println(farewell);
```

This sets `farewell` to the string "Good bye Chiana". So, it outputs the following to the screen:

```
Good bye Chiana
```

Note that we added a space at the end of "Good bye ".

a string when you connect it to a string with the + operator. For numbers, it does the obvious thing. For example,

```
String solution = "The answer is " + 42;
```

will set the String variable `solution` to "The answer is 42". Java converts the integer constant 42 to the string "42" and then concatenates the two strings "The answer is " and "42" to obtain the longer string "The answer is 42".

Notice that a number or other value is converted to a string object only when it is connected to a string with a plus sign. If it is connected to another number with a plus sign, it is not converted to a string. For example,

```
System.out.println("100" + 42);
```

outputs

```
10042
```

but

```
System.out.println(100 + 42);
```

outputs

```
142
```

Classes

Classes are central to Java, and you will soon be defining and using your own classes. The class `String` gives us an opportunity to introduce some of the notation and

class**object****method**

terminology used for classes. A class is the name for a type whose values are objects. Objects are entities that store data and can take actions. For example, objects of the class `String` store data consisting of strings of characters, such as `"Hello"`. The actions that an object can take are called methods. Most of the methods for the class `String` return some value—that is, produce some value. For example, the method `length()` returns the number of characters in a `String` object. So, `"Hello".length()` returns the integer 5, which can be stored in an `int` variable as follows:

```
int n = "Hello".length();
```

As indicated by the example `"Hello".length()`, a method is called into action by writing a name for the object followed by a dot followed by the method name with parentheses. When you call a method into action, you are (or your code is) said to invoke the method or call the method, and the object before the dot is known as the **calling object**.

Although you can call a method with a constant object, as in `"Hello".length()`, it is more common to use a variable as the calling object, as illustrated by the following:

```
String greeting = "Hello";
int n = greeting.length();
```

Information needed for the method invocation is given in the parentheses. In some cases, such as the method `length`, no information is needed (other than the data in the calling object) and the parentheses are empty. In other cases, which we see soon, there is some information that must be provided inside the parentheses. The information in parentheses is known as an **argument** (or arguments).

Invoking a method is also sometimes called **sending a message** to the object. With this view, a message is sent to the object (by invoking a method) and in response the object performs some action. For example, in response to the message

```
greeting.length()
```

the object `greeting` answers with the value 5.

All objects within a class have the same methods, but each object can have different data. For example, the two `String` objects `"Hello"` and `"Good-Bye"` have different data—that is, different strings of characters. However, they have the same methods. Thus, because we know that the `String` object `"Hello"` has the method `length()`, we know that the `String` object `"Good-Bye"` must also have the method `length()`.

You now have seen two kinds of types in Java: primitive types and class types. The main difference you have seen between these two kinds of types is that classes have methods and primitive types do not. We will later see more differences between classes and primitive types. A smaller difference between primitive types and class types is that all the primitive types are spelled using only lowercase letters, but, by convention, class types are spelled with their first letter in uppercase, as in `String`.

**method call
or method
invocation**

calling object**argument**

**sending a
message**

Classes, Objects, and Methods

A Java program works by having things called **objects** perform actions. The actions are known as **methods** and typically involve data contained in the object. All objects of the same kind are said to be of the same class. So, a **class** is a category of objects. When the object performs the action of a given method, it is called **invoking** the method (or **calling** the method). Information provided to the method in parentheses is called the **argument** (or arguments).

For example, in Display 1.1, `System.out` is an object, `println` is a method, and the following is an invocation of the method by this object using the argument `"Hello reader."`:

```
System.out.println("Hello reader.");
```

String Methods

The class `String` has a number of useful methods that can be used for string-processing applications. A sample of these `String` methods is presented in Display 1.4. Some of the notation and terminology used in Display 1.4 is described in the box entitled “Returned Value.” A more complete list of `String` methods is given in Appendix 5.

Returned Value

An expression such as `numberOfGirls + numberOfBoys` produces a value. If `numberOfGirls` has the value 2 and `numberOfBoys` has the value 10, then the number produced is 12. The number 12 is the result of evaluating the expression.

Some method invocations are simple kinds of expression, and any such method invocation evaluates to some value. If a method invocation produces a value, we say that the method *returns* the value. For example, suppose your program executes

```
String greeting = "Hello!";
```

After that, if you evaluate `greeting.length()`, the value returned will be 6. So the following code outputs the integer 6:

```
String greeting = "Hello!";
System.out.println(greeting.length());
```

A method can return different values depending on what happens in your program. However, each method can return values of only one type. For example, the method `length` of the class `String` always returns an `int` value. In Display 1.4, the type given before the method name is the type of the values returned by that method. Since `length` always returns an `int` value, the entry for `length` begins

```
int length()
```

As with any method, a `String` method is called (invoked) by writing a `String` object, a dot, the name of the method, and finally a pair of parentheses that enclose any arguments to the method. Let's look at some examples.

`length`

As we've already noted, the method `length` can be used to find out the number of characters in a string. You can use a call to the method `length` anywhere that you can use a value of type `int`. For example, all of the following are legal Java statements:

```
String greeting = "Hello";
int count = greeting.length();
System.out.println("Length is " + greeting.length());
```

Display 1.4 Some Methods in the Class `String` (part 1 of 4)

`int length()`

Returns the length of the calling object (which is a string) as a value of type `int`.

EXAMPLE

After program executes `String greeting = "Hello!";`
`greeting.length()` returns 6.

`boolean equals(Other_String)`

Returns `true` if the calling object string and the *Other_String* are equal. Otherwise, returns `false`.

EXAMPLE

After program executes `String greeting = "Hello";`
`greeting.equals("Hello")` returns `true`
`greeting.equals("Good-Bye")` returns `false`
`greeting.equals("hello")` returns `false`

Note that case matters. "Hello" and "hello" are not equal because one starts with an uppercase letter and the other starts with a lowercase letter.

`boolean equalsIgnoreCase(Other_String)`

Returns `true` if the calling object string and the *Other_String* are equal, considering upper- and lowercase versions of a letter to be the same. Otherwise, returns `false`.

EXAMPLE

After program executes `String name = "mary!";`
`greeting.equalsIgnoreCase("Mary!")` returns `true`

Display 1.4 Some Methods in the Class `String` (part 2 of 4)**String toLowerCase()**

Returns a string with the same characters as the calling object string, but with all letter characters converted to lowercase.

EXAMPLE

After program executes `String greeting = "Hi Mary!";`
`greeting.toLowerCase()` returns `"hi mary!"`.

String toUpperCase()

Returns a string with the same characters as the calling object string, but with all letter characters converted to uppercase.

EXAMPLE

After program executes `String greeting = "Hi Mary!";`
`greeting.toUpperCase()` returns `"HI MARY!"`.

String trim()

Returns a string with the same characters as the calling object string, but with leading and trailing white space removed. White space characters are the characters that print as white space on paper, such as the blank (space) character, the tab character, and the new-line character `'\n'`.

EXAMPLE

After program executes `String pause = " Hmm ";`
`pause.trim()` returns `"Hmm"`.

char charAt(Position)

Returns the character in the calling object string at the *Position*. Positions are counted 0, 1, 2, etc.

EXAMPLE

After program executes `String greeting = "Hello!";`
`greeting.charAt(0)` returns `'H'`, and
`greeting.charAt(1)` returns `'e'`.

String substring(Start)

Returns the substring of the calling object string starting from *Start* through to the end of the calling object. Positions are counted 0, 1, 2, etc. Be sure to notice that the character at position *Start* is included in the value returned.

EXAMPLE

After program executes `String sample = "AbcdefG";`
`sample.substring(2)` returns `"cdefG"`.

(continued)

Display 1.4 Some Methods in the Class `String` (part 3 of 4)**String substring(*Start*, *End*)**

Returns the substring of the calling object string starting from position *Start* through, but not including, position *End* of the calling object. Positions are counted 0, 1, 2, etc. Be sure to notice that the character at position *Start* is included in the value returned, but the character at position *End* is not included.

EXAMPLE

After program executes `String sample = "AbcdefG";`
`sample.substring(2, 5)` returns "cde".

int indexOf(*A_String*)

Returns the index (position) of the first occurrence of the string *A_String* in the calling object string. Positions are counted 0, 1, 2, etc. Returns -1 if *A_String* is not found.

EXAMPLE

After program executes `String greeting = "Hi Mary!";`
`greeting.indexOf("Mary")` returns 3, and
`greeting.indexOf("Sally")` returns -1.

int indexOf(*A_String*, *Start*)

Returns the index (position) of the first occurrence of the string *A_String* in the calling object string that occurs at or after position *Start*. Positions are counted 0, 1, 2, etc. Returns -1 if *A_String* is not found.

EXAMPLE

After program executes `String name = "Mary, Mary quite contrary";`
`name.indexOf("Mary", 1)` returns 6.
The same value is returned if 1 is replaced by any number up to and including 6.
`name.indexOf("Mary", 0)` returns 0.
`name.indexOf("Mary", 8)` returns -1.

int lastIndexOf(*A_String*)

Returns the index (position) of the last occurrence of the string *A_String* in the calling object string. Positions are counted 0, 1, 2, etc. Returns -1, if *A_String* is not found.

EXAMPLE

After program executes `String name = "Mary, Mary, Mary quite so";`
`greeting.indexOf("Mary")` returns 0, and
`name.lastIndexOf("Mary")` returns 12.

Display 1.4 Some Methods in the Class String (part 4 of 4)

int compareTo(*A_String*)

Compares the calling object string and the string argument to see which comes first in the lexicographic ordering. Lexicographic order is the same as alphabetical order but with the characters ordered as in Appendix 3. Note that in Appendix 3, all the uppercase letters are in regular alphabetical order and all the lowercase letters are in alphabetical order, but all the uppercase letters precede all the lowercase letters. So, lexicographic ordering is the same as alphabetical ordering provided both strings are either all uppercase letters or both strings are all lowercase letters. If the calling string is first, it returns a negative value. If the two strings are equal, it returns zero. If the argument is first, it returns a positive number.

EXAMPLE

After program executes `String entry = "adventure";`
`entry.compareTo("zoo")` returns a negative number,
`entry.compareTo("adventure")` returns 0, and
`entry.compareTo("above")` returns a positive number.

int compareToIgnoreCase(*A_String*)

Compares the calling object string and the string argument to see which comes first in the lexicographic ordering, treating upper- and lowercase letters as being the same. (To be precise, all uppercase letters are treated as if they were their lowercase versions in doing the comparison.) Thus, if both strings consist entirely of letters, the comparison is for ordinary alphabetical order. If the calling string is first, it returns a negative value. If the two strings are equal ignoring case, it returns zero. If the argument is first, it returns a positive number.

EXAMPLE

After program executes `String entry = "adventure";`
`entry.compareToIgnoreCase("Zoo")` returns a negative number,
`entry.compareToIgnoreCase("Adventure")` returns 0, and
`"Zoo".compareToIgnoreCase(entry)` returns a positive number.

position

Some methods for the class String depend on counting **positions** in the string. Positions are counted starting with 0, not with 1. So, in the string "Surf time", 'S' is in position 0, 'u' is in position 1, and so forth. A position is usually referred to as an **index**. So, it would be preferable to say: 'S' is at index 0, 'u' is at index 1, and so on.

index

The method `indexOf` can be used to find the index of a substring of the calling objects. For example, consider

```
String phrase = "Java is fun.";
```

After this declaration, the invocation `phrase.indexOf("is")` will return 5 because the 'i' of "is" is at index 5. (Remember, the first index is 0, not 1.) This is illustrated in Display 1.5.

Display 1.5 String Indexes

The 12 characters in the string "Java is fun." have indexes 0 through 11.

0	1	2	3	4	5	6	7	8	9	10	11
J	a	v	a		i	s		f	u	n	.

Notice that the blanks and the period count as characters in the string.

Escape Sequences

backslash \
escape
sequence

A backslash, \, preceding a character tells the compiler that the character following the \ does not have its usual meaning. Such a sequence is called an **escape sequence** or an **escape character**. The sequence is typed in as two characters with no space between the symbols. Several escape sequences are defined in Java.

If you want to put a backslash, \, or a quote symbol, ", into a string constant, you must escape the ability of the " to terminate a string constant by using \", or the ability of the \ to escape by using \\. The \\ tells the compiler you mean a real backslash, \, not an escape sequence, and \" means a quote character, not the end of a string constant. A list of escape sequences is given in Display 1.6.

It is important to note that each escape sequence is a single character, even though it is spelled with two symbols. So, the string "Say \"Hi\"!" contains 9 characters ('S', 'a', 'y', the blank character, '\"', 'H', 'i', '\"', and '!'), not 11 characters.

Including a backslash in a quoted string is a little tricky. For example, the string "abc\def" is likely to produce the error message "Invalid escape character." To include a backslash in a string, you need to use two backslashes. The string "abc\\def", if output to the screen, would produce

abc\def

Display 1.6 Escape Sequences

```
\ " Double quote.  
\ ' Single quote.  
\ \ Backslash.  
\ n New line. Go to the beginning of the next line.  
\ r Carriage return. Go to the beginning of the current line.  
\ t Tab. White space up to the next tab stop.
```

The escape sequence `\n` indicates the start of a new line. For example, the statement

```
System.out.println("To be or\nNot to be.");
```

will write the following to the screen:

```
To be or
Not to be.
```

You do not need to use the escape sequence `\'` to include a single quote inside a quoted string. For example, `"Time's up!"` is a valid quoted string. However, you do need `\'` if you want to indicate the constant for the single-quote character, as in

```
char singleQuote = '\'';
```

String Processing

immutable object

In Java, an object of type `String` is an **immutable object**, meaning that the characters in the `String` object cannot be changed. This will eventually prove to be important to us, but at this stage of our exploration of Java, it is a misleading statement. To see that an object of type `String` cannot be changed, note that none of the methods in Display 1.4 changes the value of the `String` calling object. There are more `String` methods than those shown in Display 1.4, but none of them lets you write statements that say things such as “Change the fifth character in the calling object string to 'x'.” This was done intentionally to make the implementation of the `String` class more efficient and for other reasons that we will discuss later in this book. There is another string class, called `StringBuffer`, that has methods for altering its string object. We will not discuss the class `StringBuffer` in this text, but a table explaining many of the methods of the class `StringBuffer` is included in Appendix 5.

Although there is no method that allows you to change the value of a `String` object, such as `"Hello"`, you can still write programs that change the value of a `String` variable, which is probably all you want anyway. To perform the change, simply use an assignment statement, as in the following example:

```
String name = "Soprano";
name = "Anthony " + name;
```

The assignment statement in the second line changes the value of the `name` variable so that the string it names changes from `"Soprano"` to `"Anthony Soprano"`. Display 1.7 contains a demonstration of some simple string processing.

The Unicode Character Set ★

ASCII

Until recently, most programming languages used the ASCII character set, which is given in Appendix 3. The **ASCII** character set is simply a list of all the characters normally used on an English-language keyboard plus a few special characters. In this list, each character has been assigned a number so that characters can be stored by storing the corresponding number. Java (and now many other programming languages) uses the Unicode character set. The **Unicode** character set includes the

Unicode

Display 1.7 Using the String Class

```

1  public class StringProcessingDemo
2  {
3      public static void main(String[] args)
4      {
5          String sentence = "I hate text processing!";
6          int position = sentence.indexOf("hate");
7          String ending =
8              sentence.substring(position + "hate".length());
9
10         System.out.println("01234567890123456789012");
11         System.out.println(sentence);
12         System.out.println("The word \"hate\" starts at index "
13                             + position);
14         sentence = sentence.substring(0, position) + "adore"
15                             + ending;
16         System.out.println("The changed string is:");
17         System.out.println(sentence);
18     }
19 }

```

You could just use 4 here, but if you had a String variable instead of "hate", you would have to use length as shown.

Sample Dialogue

```

01234567890123456789012
I hate text processing!
The word "hate" starts at index 2
The changed string is:
I adore text processing!

```

ASCII character set plus many of the characters used in languages with a different alphabet from English. This is not likely to be a big issue if you are using an English-language keyboard. Normally, you can just program as if Java were using the ASCII character set. The ASCII character set is a subset of the Unicode character set, and the subset you are likely to use. Thus, Appendix 3, which lists the ASCII character set, in fact lists the subset of the Unicode character set that we will use in this book. The advantage of the Unicode character set is that it makes it possible to easily handle languages other than English. For example, it is legal to spell a Java identifier using the letters of the Greek alphabet (although you may want a Greek-language keyboard and monitor to do this). The disadvantage of the Unicode character set is that it sometimes requires more computer memory to store each character than it would if Java used only the ASCII character set.

Self-Test Exercises

24. What is the output produced by the following?

```
String verbPhrase = "is money";  
System.out.println("Time" + verbPhrase);
```

25. What is the output produced by the following?

```
String test = "abcdefg";  
System.out.println(test.length());  
System.out.println(test.charAt(1));
```

26. What is the output produced by the following?

```
String test = "abcdefg";  
System.out.println(test.substring(3));
```

27. What is the output produced by the following?

```
System.out.println("abc\\ndef");
```

28. What is the output produced by the following?

```
System.out.println("abc\\\\ndef");
```

29. What is the output produced by the following?

```
String test = "Hello Tony";  
test = test.toUpperCase();  
System.out.println(test);
```

30. What is the output of the following two lines of Java code?

```
System.out.println("2 + 2 = " + (2 + 2));  
System.out.println("2 + 2 = " + 2 + 2);
```

31. Suppose `sam` is an object of a class named `Person` and suppose `increaseAge` is a method for the class `Person` that takes one argument that is an integer. How do you write an invocation of the method `increaseAge` using `sam` as the calling object and using the argument 10? The method `increaseAge` will change the data in `sam` so that it simulates `sam` aging by 10 years.

32. The following code is supposed to output the string in lowercase letters but it has an error. What is wrong?

```
String test = "WHY ARE YOU SHOUTING?";  
test.toLowerCase();  
System.out.println(test);
```

1.4 Program Style

*In matters of grave importance,
style, not sincerity, is the vital thing.*

OSCAR WILDE, *The Importance of Being Earnest*, 1895.

Java programming style is similar to that used in other languages. The goal is to make your code easy to read and easy to modify. This section gives some basic points on good programming style in general and some information on the conventions normally followed by Java programmers.

Naming Constants

There are two problems with numbers in a computer program. The first is that they carry no mnemonic value. For example, when the number 10 is encountered in a program, it gives no hint of its significance. If the program is a banking program, it might be the number of branch offices or the number of teller windows at the main office. To understand the program, you need to know the significance of each constant. The second problem is that when a program needs to have some numbers changed, the changing tends to introduce errors. Suppose that 10 occurs 12 times in a banking program. Four of the times it represents the number of branch offices, and eight of the times it represents the number of teller windows at the main office. When the bank opens a new branch and the program needs to be updated, there is a good chance that some of the 10s that should be changed to 11 will not be, or some that should not be changed will be. The way to avoid these problems is to name each number and use the name instead of the number within your program. For example, a banking program might have two constants with the names `BRANCH_COUNT` and `WINDOW_COUNT`. Both of these numbers might have a value of 10, but when the bank opens a new branch, all you need to do to update the program is to change the definition of `BRANCH_COUNT`.

One way to name a number is to initialize a variable to that number value, as in the following example:

```
int BRANCH_COUNT = 10;  
int WINDOW_COUNT = 10;
```

There is, however, one problem with this method of naming number constants: You might inadvertently change the value of one of these variables. Java provides a way of marking an initialized variable so that it cannot be changed. The syntax is

```
public static final Type Variable = Constant;
```

For example, the names `BRANCH_COUNT` and `WINDOW_COUNT` can be given values that cannot be changed by your code as follows:

```
public static final int BRANCH_COUNT = 10;
public static final int WINDOW_COUNT = 10;
```

Constants defined this way must be placed outside of the `main` method and, when we start having more methods, outside of any other methods. This is illustrated in Display 1.8. When we start writing programs and classes with multiple methods, you will see that the defined constants can be used in all the methods of a class. However, if a constant is only going to be used inside a single method, then it can be defined inside the method without the keyword `public`.

Display 1.8 Comments and a Named Constant

```

1  /**
2   Program to show interest on a sample account balance.
3   Author: Jane Q. Programmer.
4   E-mail Address: janeq@somemachine.etc.etc.
5   Last Changed: December 17, 2014.
6   */
7  public class ShowInterest
8  {
9      public static final double INTEREST_RATE = 2.5;
10
11     public static void main(String[] args)
12     {
13         double balance = 100;
14         double interest; //as a percent
15
16         interest = balance * (INTEREST_RATE / 100.0);
17         System.out.println("On a balance of $" + balance);
18         System.out.println("you will earn interest of $"
19                             + interest);
20         System.out.println("All in just one short year.");
21     }
22     ← Although it would not be as clear,
23     it is legal to place the definition of
24     INTEREST_RATE here instead.
```

Sample Dialogue

```
On a balance of $100.0
you will earn interest of $2.5
All in just one short year.
```

We will fully explain the modifiers `public static final` later in this book, but we can now explain most of what they mean. The part

```
int BRANCH_COUNT = 10;
```

simply declares `BRANCH_COUNT` as a variable and initializes it to 10. The words that precede this modify the variable `BRANCH_COUNT` in various ways. The word `public` says there are no restrictions on where you can use the name `BRANCH_COUNT`. The word `static` will have to wait until Chapter 5 for an explanation, but be sure to include it. The word `final` means that the value 10 is the *final* value assignment to `BRANCH_COUNT`, or, to phrase it another way, that the program is not allowed to change the value of `BRANCH_COUNT`.

Naming Constants

The syntax for defining a name for a constant outside of a method, such as a name for a number, is as follows:

SYNTAX

```
public static final Type Variable = Constant;
```

EXAMPLE

```
public static final int MAX_SPEED = 65;  
public static final double MIN_SIZE = 0.5;  
public static final String GREETING = "Hello friend!";  
public static final char GOAL = 'A';
```

Although it is not required, it is the normal practice of programmers to spell named constants using all uppercase letters with the underscore symbol used to separate “words.”

Java Spelling Conventions

In Java, as in all programming languages, identifiers for variables, methods, and other items should always be meaningful names that are suggestive of the identifiers’ meanings. Although it is not required by the Java language, the common practice of Java programmers is to start the names of classes with uppercase letters and to start the names of variables, objects, and methods with lowercase letters. Defined constants are normally spelled with all uppercase letters and underscore symbols for “punctuation,” as we did in the previous subsection, “Naming Constants.”

For example, `String`, `FirstProgram`, and `JOptionPane` are classes, although we have not yet discussed the last one. The identifiers `println`, `balance`, and `readLine` should each be either a variable, an object, or a method.

Since blanks are not allowed in Java identifiers, “word” boundaries are indicated by an uppercase letter, as in `numberOfPods`. Since defined constants are spelled with all uppercase letters, the underscore symbol is used for “word” boundaries, as in `MAX_SPEED`.

The identifier `System.out` seems to violate this convention, since it names an object but yet begins with an uppercase letter. It does not violate the convention, but the explanation hinges on a topic we have not yet covered. `System` is the name of a

class. Within the class named `System`, there is a definition of an object named `out`. So, the identifier `System.out` is used to indicate the object `out` (starting with a lowercase letter for an object) that is defined in the class `System` (starting with an uppercase letter for a class). This sort of dot notation will be explained later in the book.

There is one Java convention that people new to Java often find strange. Java programmers normally do not use abbreviations in identifiers, but rather spell things out in full. A Java programmer would not use `numStars`. He or she would use `numberOfStars`. A Java programmer would not use `FirstProg`. He or she would use `FirstProgram`. This can produce long identifiers and sometimes exceedingly long identifiers. For example, the names of two standard Java classes are `BufferedReader` and `ArrayIndexOutOfBoundsException`. The first will be used in the next chapter and the second will be used later in this book. These long names cause you to do more typing, and program lines quickly become too long. However, there is a very good reason for using these long names: There is seldom any confusion on how the identifiers are spelled. With abbreviations, you often cannot recall how the identifier was abbreviated. Was it `BufReader` or `BuffReader` or `BufferedReader` or `BR` or something else? Because all the words are spelled out, you know it must be `BufferedReader`. Once they get used to using these long names, most programmers learn to prefer them.

Comments

`//comments`

There are two ways to insert **comments** in a Java program. In Java, the symbols `//` are used to indicate the start of a comment. All of the text between the `//` and the end of the line is a comment. The compiler simply ignores anything that follows `//` on a line. If you want a comment that covers more than one line, place a `//` on each line of the comment. The symbols `//` are two slashes (without a space between them). Comments indicated with `//` are often called **line comments** or **inline comments**.

line
comments

`/*comments*/`

There is another way to insert comments in a Java program. Anything between the symbol pair `/*` and the symbol pair `*/` is considered a comment and is ignored by the compiler. Unlike the `//` comments, which require an additional `//` on each line, the `/*` to `*/` comments can span several lines like so:

```
/*This is a multi-line comment.  
Note that there is no comment symbol  
of any kind on the second line.*/
```

block
comments

Comments of the `/* */` type are often called **block comments**. These block comments may be inserted anywhere in a program that a space or line break is allowed. However, they should not be inserted anywhere except where they do not distract from the layout of the program. Usually comments are only placed at the ends of lines or on separate lines by themselves.

Java comes with a program called `javadoc` that will automatically extract documentation from the classes you define. The workings of the `javadoc` program dictate when you normally use each kind of comment.

The `javadoc` program will extract a `/* */` comment in certain situations, but it will not extract a `//` comment. We will say more about `javadoc` and comments after

we discuss defining classes. In the meantime, you may notice the following conventions in our code:

We use line comments (that is, the `//` kind) for comments meant only for the code writer or for a programmer who modifies the code and not for any other programmer who merely uses the code.

For comments that would become part of the documentation for users of our code, we use block comments (that is, the `/* */` kind). The `javadoc` program allows you to indicate whether or not a block comment is eligible to be extracted for documentation. If the opening `/*` has an extra asterisk, as in `/**`, then the comment is eligible to be extracted. If there is only one asterisk, `javadoc` will not extract the comment. For this reason, our block comments invariably open with `/**`.

when to comment

It is difficult to say just how many comments a program should contain. The only correct answer is “just enough,” which of course conveys little to the novice programmer. It will take some experience to get a feel for when it is best to include a comment. Whenever something is important and not obvious, it merits a comment. However, providing too many comments is as bad as providing too few. A program that has a comment on each line is so buried in comments that the structure of the program is hidden in a sea of obvious observations. Comments such as the following contribute nothing to understanding and should not appear in a program:

```
interest = balance * rate; //Computes the interest.
```

self- documenting

A well-written program is called **self-documenting**, which means that the structure of the program is clear from the choice of identifier names and the indenting pattern. A completely self-documenting program would need none of these `//` comments that are only for the programmer who reads or modifies the code. That may be an ideal that is not always realizable, but if your code is full of `//` comments and you follow our convention on when to use them, then either you simply have too many comments or your code is poorly designed.

A very simple example of the two kinds of comments is given in Display 1.8.

Indenting

We will say more about indenting as we introduce more Java. However, the general rule is easy to understand and follow. When one structure is nested inside another structure, the inside structure is indented one more level. For example, in our programs, the `main` method is indented one level, and the statements inside the `main` method are indented two levels. We prefer to use four spaces for each level of indenting because more than four spaces eats up too much line length. It is possible to get by with indenting only two or three spaces for each level so long as you are consistent. One space for a level of indenting is not enough to be clearly visible.

Self-Test Exercises

33. What are the two kinds of comments in Java?
34. What is the output produced by the following Java code?

```
/**
 * Code for Exercise.
 */
System.out.println("Hello");
//System.out.print("Mr. or Ms. ");
System.out.println("Student");
```

35. What is the normal spelling convention for named constants?
36. Write a line of Java code that will give the name `ANSWER` to the `int` value 42. In other words, make `ANSWER` a named constant for 42.

Chapter Summary

- Compiling a Java class or program produces *byte-code*, which is the machine language for a fictitious computer. When you run the byte-code, a program called an *interpreter* translates and executes the byte-code instructions on your computer one instruction at a time.
- A variable can be used to hold values, such as numbers. The type of the variable must match the type of the value stored in the variable. All variables must be declared before they are used.
- The equal sign, `=`, is used as the *assignment operator* in Java. An *assignment statement* is an instruction to change the value of a variable.
- Each variable should be initialized before the program uses its value.
- Parentheses in arithmetic expressions indicate which arguments are given to an operator. When parentheses are omitted, Java adds implicit parentheses using *precedence rules* and *associativity rules*.
- You can have variables and constants of type `String`. `String` is a class type, not a *primitive type*.
- You can use the plus sign to concatenate two strings.
- There are methods in the class `String` that can be used for string processing.
- Variables (and all other items in a program) should be given names that indicate how they are used.
- You should define names for number constants in a program and use these names rather than writing out the numbers within your program.
- Programs should be self-documenting to the extent possible. However, you should also insert comments to explain any unclear points.

Answers to Self-Test Exercises

1. Java is not a drink.
2.

```
System.out.println("I like Java.");  
System.out.println("You like tea.");
```
3.

```
public class HelloWorld  
{  
    public static void main(String[] args)  
    {  
        System.out.println("Hello World!");  
    }  
}
```
4. A compiler translates a program written in a programming language such as Java into a program in a low-level language. When you compile a Java program, the compiler translates your Java program into a program expressed in Java byte-code.
5. The program that is input to a compiler is called the *source program*.
6. The translated program that is produced by a compiler is called the *object program* or *object code*.
7. A program that runs Java byte-code instructions is called an *interpreter*. It is also often called the *Java Virtual Machine (JVM)*.
8. NiceClass.java
9. NiceClass.class
10. 1stPlayer may not be used because it starts with a digit; myprogram.java may not be used because it contains an illegal symbol, the dot; long may not be used because it is a keyword. All the others may be used as variable names. However, TimeLimit, while legal, violates the style rule that all variable names begin with a lowercase letter.
11. Yes, a Java program can have two different variables named number and Number. However, it would not be good style to do so.
12.

```
int feet = 0, inches = 0;
```
13.

```
int count = 0;  
double distance = 1.5;
```
14.

```
distance = time * 80;
```
15.

```
interest = balance * rate;
```
16. b
c
c

17. $3 * x$

$3 * x + y$

$(x + y) / 7$ Note that $x + y / 7$ is not correct.

$(3 * x + y) / (z + 2)$

18. $(1/3) * 3$ is equal to 0.0

Since 1 and 3 are of type `int`, the `/` operator performs integer division, which discards the remainder, so the value of $1/3$ is 0, not 0.3333 This makes the value of the entire expression $0 * 3$, which of course is 0.

19. `quotient = 2`

`remainder = 1`

20. `result` is 5

21. a. 52.0

b. $9/5$ has `int` value 1; because the numerator and denominator are both of type `int`, integer division is done; the fractional part is discarded. The programmer probably wanted floating-point division, which does not discard the part after the decimal point.

c. `fahrenheit = (9.0/5) * celsius + 32.0;`

or

`fahrenheit = 1.8 * celsius + 32.0;`

22. `n = = 3`

23. `n = = 4`

`n = = 3`

24. `Time` is `money`

25. 7

b

26. `defg`

27. `abc`

`def`

28. `abc\ndef`

29. `HELLO TONY`

30. The output is

`2 + 2 = 4`

`2 + 2 = 22`

In the expression `"2 + 2 = " + (2 + 2)`, the integers 2 and 2 in `(2 + 2)` are added to obtain the integer 4. When 4 is connected to the string `"2 + 2"` with a plus sign, the integer 4 is converted to the string `"4"` and the result is the string `"2 + 2 = 4"`. However `"2 + 2 = " + 2 + 2` is interpreted by Java to mean

`("2 + 2 = " + 2) + 2`

The first integer 2 is changed to the string "2" because it is being combined with the string "2 + 2". The result is the string "2 + 2 = 2". The last integer 2 is combined with the string "2 + 2 = 2". So, the last 2 is converted to the string "2". So the final result is

```
"2 + 2 = 2" + "2"
```

which is "2 + 2 = 22".

31. `sam.increaseAge(10);`
32. The method `toLowerCase` doesn't change the string `test`. To change it, we must set `test` equal to the string returned by `toLowerCase`:

```
test = test.toLowerCase();
```
33. The two kinds of comments are `//` comments and `/* */` comments. Everything following a `//` on the same line is a comment. Everything between a `/*` and a matching `*/` is a comment.
34. `Hello`
`Student`
35. The normal spelling convention is to spell named constants using all uppercase letters with the underscore symbol used to separate words.
36. `public static final int ANSWER = 42;`

Programming Projects

1. Body Mass Index (BMI) helps in specifying the weight category a person belongs to, depending on their body weight. BMI is estimated using the following formula:

$$\text{BMI} = \frac{\text{Weight in kilograms}}{(\text{Height in meters})^2}$$

Write a program that calculates and outputs the BMI. Assume various input values wherever required.

2. The video game machines at your local arcade output coupons according to how well you play the game. You can redeem 10 coupons for a candy bar or 3 coupons for a gumball. You prefer candy bars to gumballs. Write a program that defines a variable initially assigned to the number of coupons you win. Next,

the program should output how many candy bars and gumballs you can get if you spend all of your coupons on candy bars first, and any remaining coupons on gumballs.

3. Write a program that starts with the string variable `first` set to your first name and the string variable `last` set to your last name. Both names should be all lower-case. Your program should then create a new string that contains your full name in pig latin with the first letter capitalized for the first and last name. Use only the pig latin rule of moving the first letter to the end of the word and adding “ay.” Output the pig latin name to the screen. Use the `substring` and `toUpperCase` methods to construct the new name.

For example, given

```
first = "walt";  
last = "savitch";
```

the program should create a new string with the text “Altway Avitchsay” and print it.

4. A government research lab has concluded that an artificial sweetener commonly used in diet soda pop will cause death in laboratory mice. A friend of yours is desperate to lose weight but cannot give up soda pop. Your friend wants to know how much diet soda pop it is possible to drink without dying as a result. Write a program to supply the answer. The program has no input but does have defined constants for the following items: the amount of artificial sweetener needed to kill a mouse, the weight of the mouse, the starting weight of the dieter, and the desired weight of the dieter. To ensure the safety of your friend, be sure the program uses the weight at which the dieter will stop dieting, rather than the dieter’s current weight, to calculate how much soda pop the dieter can safely drink. You may use any reasonable values for these defined constants. Assume that diet soda contains 1/10th of 1% artificial sweetener. Use another named constant for this fraction. You may want to express the percent as the `double` value `0.001`. (If your program turns out not to use a defined constant, you may remove that defined constant from your program.)
5. Write a program that starts with a line of text and then outputs that line of text with the first occurrence of “hate” changed to “love”. For example, a possible sample output might be

```
The line of text to be changed is:  
I hate you.  
I have rephrased that line to read:  
I love you.
```

You can assume that the word “hate” occurs in the input. If the word “hate” occurs more than once in the line, your program will replace only the first occurrence of “hate”. Since we will not discuss input until Chapter 2, use a defined constant for the string to be changed. To make your program work for another string, you should only need to change the definition of this defined constant.

6. Write a program for calculating the simple interest on a loan when the initial principal amount (`princi_amnt`) is \$1000, the initial interest rate (`int_rate`) is 5.0%, and the number of years (`no_of_yrs`) is 5. Use suitable data types to declare these variables. Simple interest is calculated using the following equation:

$$\text{Simple interest} = \frac{(\text{Principal amount} \times \text{Interest rate} \times \text{Number of years})}{100}$$



VideoNote
Solution to
Programming
Project 1.7

7. Write a program that outputs the number of hours, minutes, and seconds that corresponds to 50,391 total seconds. The output should be 13 hours, 59 minutes, and 51 seconds. Test your program with a different number of total seconds to ensure that it works for other cases.
8. The following program will compile and run, but it uses poor programming style. Modify the program so that it uses the spelling conventions, constant naming conventions, and formatting style recommended in this book.

```
public class vehicleAvgSpeed {
    public static void main(String[] args)
    {
        double TIME;
        System.out.println("This program calculates vehicle average speed
        given a time and distance traveled.");
        TIME = 20.5;
        AVERAGE_SPEED = distance / TIME;
        System.out.println("Car average speed is " + AVERAGE_SPEED
        + " miles per hour.");
    }
    public static final double distance = 180;
}
```

9. A simple rule to estimate your ideal body weight is to allow 110 pounds for the first 5 feet of height and 5 pounds for each additional inch. Write a program with a variable for the height of a person in feet and another variable for the additional inches. Assume the person is at least 5 feet tall. For example, a person that is 6 feet and 3 inches tall would be represented with a variable that stores the number 6 and another variable that stores the number 3. Based on these values, calculate and output the ideal body weight.
10. Scientists estimate that roughly 10 grams of caffeine consumed at one time is a lethal overdose. Write a program with a variable that holds the number of milligrams of caffeine in a drink and outputs how many drinks it takes to kill a person. A 12-ounce can of cola has approximately 34 mg of caffeine, while a 16-ounce cup of coffee has approximately 160 mg of caffeine.



Console Input and Output **2**

2.1 SCREEN OUTPUT 90

`System.out.println` 90

Formatting Output with `printf` 93

Money Formats Using `NumberFormat` ★ 99

Importing Packages and Classes 102

The `DecimalFormat` Class ★ 104

2.2 CONSOLE INPUT USING THE `Scanner` CLASS 108

The `Scanner` Class 108

The Empty String 116

Example: Self-Service Checkout 118

Other Input Delimiters 119

2.3 INTRODUCTION TO FILE INPUT 121

The `Scanner` Class for Text File Input 121

2 Console Input and Output

Don't imagine you know what a computer terminal is. A computer terminal is not some clunky old television with a typewriter in front of it. It is an interface where the mind and the body can connect with the universe and move bits of it about.

DOUGLAS ADAMS, *Mostly Harmless*
(the fifth volume in *The Hitchhiker's Trilogy*), Random House, 2009.

Introduction

console I/O

This chapter covers simple output to the screen and input from the keyboard, often called **console I/O**. We have already used console output, but this chapter covers it in more detail. In particular, this chapter shows you how to format numeric output so that you control such detail as the number of digits shown after the decimal point. This chapter also covers the `Scanner` class, which was introduced in version 5.0 of Java and can be used for console input.

Prerequisites

This chapter uses material from Chapter 1.

2.1 Screen Output

Let me tell the world.

WILLIAM SHAKESPEARE, *Henry IV*, 1598.

In this section, we review `System.out.println` and present some material on formatting numeric output. As part of that material, we give a brief introduction to *packages* and *import statements*. Packages are Java libraries of classes. Import statements make classes from a package available to your program.

System.out.println

System.out println

We have already been using `System.out.println` for screen output. In Display 1.7, we used statements such as the following to send output to the display screen:

```
System.out.println("The changed string is:");  
System.out.println(sentence);
```

`System.out` is an object that is part of the Java language, and `println` is a method invoked by that object. It may seem strange to spell an object name with a dot in it, but that need not concern us for now.

When you use `System.out.println` for output, the data to be output is given as an argument in parentheses, and the statement ends with a semicolon. Things you can output are strings of text in double quotes, such as "The changed string is: "; String variables such as `sentence`; variables of other types such as variables of type `int`; numbers such as 5 or 7.3; and almost any other object or value. If you want to output more than one thing, simply place an addition sign between the things you want to output. For example,

```
System.out.println("Answer is = " + 42
                  + " Accuracy is = " + precision);
```

If the value of `precision` is 0.01, the output will be

```
Answer is = 42 Accuracy is = 0.01
```

Notice the space at the start of " Accuracy is = ". No space is added automatically.

The `+` operator used here is the concatenation operator that we discussed earlier. So the above output statement converts the number 42 to the string "42" and then forms the following string using concatenation:

```
"Answer is = 42 Accuracy is = 0.01"
```

`System.out.println` then outputs this longer string.

Every invocation of `println` ends a line of output. For example, consider the following statements:

```
System.out.println("A wet bird");
System.out.println("never flies at night.");
```

These two statements cause the following output to appear on the screen:

```
A wet bird
never flies at night.
```

**print
versus
println**

If you want the output from two or more output statements to place all their output on a single line, then use `print` instead of `println`. For example, consider the following statements:

```
System.out.print("A ");
System.out.print("wet ");
System.out.println("bird");
System.out.println("never flies at night.");
```

println Output

You can output one line to the screen using `System.out.println`. The items that are output can be quoted strings, variables, numbers, or almost any object you can define in Java. To output more than one item, place a plus sign between the items.

SYNTAX

```
System.out.println(Item_1 + Item_2 + ... + Last_Item);
```

EXAMPLE

```
System.out.println("Welcome to Java.");  
System.out.println("Elapsed time = " + time + " seconds");
```

They produce the same output as our previous example:

```
A wet bird  
never flies at night.
```

Notice that a new line is not started until you use `println`, rather than `print`. Also notice that the new line starts *after* outputting the items specified in the `println`. This is the only difference between `print` and `println`.

println versus print

The only difference between `System.out.println` and `System.out.print` is that with `println`, the *next* output goes on a *new line*, whereas with `print`, the next output is placed on the *same line*.

EXAMPLE

```
System.out.print("Tom ");  
System.out.print("Dick ");  
System.out.println("and ");  
System.out.print("Harry ");
```

This produces the following output:

```
Tom Dick and  
Harry
```

(The output would look the same whether the last line reads `print` or `println`.)

Another way to describe the difference between `print` and `println` is to note that

```
System.out.println(Something);
```

is equivalent to

```
System.out.print(Something + "\n");
```

Self-Test Exercises

1. What output is produced by the following lines?

```
String s = "Hello" + "Joe";  
System.out.println(s);
```

2. Write Java statements that will cause the following to be written to the screen:

```
One two buckle your shoe.  
Three four shut the door.
```

3. What is the difference between `System.out.println` and `System.out.print`?

4. What is the output produced by the following lines?

```
System.out.println(2 + " " + 2);  
System.out.println(2 + 2);
```



TIP: Different Approaches to Formatting Output

If you have a variable of type `double` that stores some amount of money, you would like your programs to output the amount in a nice format. However, if you just use `System.out.println`, you are likely to get output that looks like the following:

```
Your cost, including tax, is $19.98327634144
```

You would like the output to look like this:

```
Your cost, including tax, is $19.98
```

To obtain this nicer form of output, you need some formatting tools.

In this chapter, we will present three approaches to formatting numeric (and other) output. We will discuss the method `printf` and the two formatting classes `NumberFormat` and `DecimalFormat`. The `printf` method is often the simplest way to format output. However, `printf` uses an older methodology and so some authorities prefer to use `NumberFormat`, `DecimalFormat`, or similar formatting classes because these classes use a programming methodology that is perhaps more in the spirit of modern (object-oriented) programming. We will let you (or your instructor if you are in a class) decide which methodology to use. After this chapter, this book seldom uses any of these formatting details. ■

Formatting Output with `printf`

`System.out.`
`printf`

Starting with version 5.0, Java includes a method named `printf` that can be used to give output in a specific format. This method is used the same way as the method



VideoNote

Using printf

print but allows you to add formatting instructions that specify such things as the number of digits to include after a decimal point. For example, consider the following:

```
double price = 19.8;
System.out.print("$");
System.out.printf("%6.2f", price);
System.out.println(" each");
```

This code outputs the following line:

```
$ 19.80 each
```

The line

```
System.out.printf("%6.2f", price);
```

outputs the string " 19.80" (one blank followed by 19.80), which is the value of the variable `price` written in the format `%6.2f`. In these simple examples, the first argument to `printf` is a string known as the **format specifier**, and the second argument is the number or other value to be output in that format. Let's explain this first sample format specifier.

The format specifier `%6.2f` says to output a floating-point number in a **field** (number of spaces) of width 6 (room for six characters) and to show exactly two digits after the decimal point. So, 19.8 is expressed as "19.80" in a field of width 6. Because "19.80" has only five characters, a blank character is added to obtain the six-character string " 19.80". Any extra blank space is added to the front (left-hand end) of the value output. That explains the 6.2 in the format specifier `%6.2f`. The `f` means the output is a floating-point number, that is, a number with a decimal point. We will have more to say about the character `%` shortly, but among other things, it indicates that a format specification (in this case, `6.2f`) follows.

Before we go on, let's note a few details about the method `printf`. Note that the first argument is a string that gives a format specifier. Also, note that `printf`, like `print`, does not advance the output to the next line. The method `printf` is like `print`, not like `println`, in this regard.

The first argument to `printf` can include text as well as a format specifier. For example, consider the following variant on our example:

```
double price = 19.8;
System.out.printf("$%6.2f each", price);
System.out.println();
```

This code also outputs the following line:

```
$ 19.80 each
```

The text before and after the format specifier `%6.2f` is output along with the formatted number. The character `%` signals the end of text to output and the start of the

format
specifier

field width

conversion
character

format specifier. The end of a format specifier is indicated by a **conversion character** (f in our example).

Other possible format specifiers are described in Display 2.1. (A more complete list of specifiers is given in Appendix 4.) The conversion character specifies the type of value that is output in the specified format. Note that the first number specifies the total number of spaces used to output the value. If that number is larger than the number of spaces needed for the output, extra blanks are added to the beginning of the value output. If that number is smaller than the number of spaces needed for the output, enough extra space is added to allow the value to be output; no matter what field width is specified, printf uses enough space to fit in the entire value output. Both of the numbers in a format specifier such as %6.2f are optional. You may omit either or both numbers, in which case Java chooses an appropriate default value or values (for example, %6f and %.2f). Note that the dot goes with the second number. You can use a format specifier that is just a % followed by a conversion character, such as %f or %g, in which case Java decides on the format details for you. For example, the format specifier %f is equivalent to %.6f, meaning six spaces after the decimal point and no extra space around the output.

e and g

The e and g format specifiers are partially explained in Display 2.1. We still need to explain the meaning of the number after the decimal point in e and g format specifiers, such as %8.3e and %8.3g. The first number, 8 in the examples, is the total field width for the value output. The second number (the number after the decimal point) specifies the number of digits after the decimal point of the output. So the numbers, such as 8.3, have the same meaning in the f, e, and g formats.

Display 2.1 Format Specifiers for `System.out.printf`

CONVERSION CHARACTER	TYPE OF OUTPUT	EXAMPLES
d	Decimal (ordinary) integer	%5d %d
f	Fixed-point (everyday notation) floating point	%6.2f %f
e	E-notation floating point	%8.3e %e
g	General floating point (Java decides whether to use E-notation or not)	%8.3g %g
s	String	%12s %s
c	Character	%2c %c

s and c

The `s` and `c` formats, for strings and characters, may include one number that specifies the field width for outputting the value, such as `%15s` and `%2c`. If no number is given, the value is output with no leading or trailing blank space.

right justified

When the value output does not fill the field width specified, blanks are added in front of the value output. The output is then said to be **right justified**. If you add a hyphen after the `%`, any extra blank space is placed after the value output, and the output is said to be **left justified**. For example, the lines

left justified

```
double value = 12.123;
System.out.printf("Start%8.2fEnd", value);
System.out.println();
System.out.printf("Start%-8.2fEnd", value);
System.out.println();
```

produce the following output. The first line has three spaces before the `12.12`, and the second has three spaces after the `12.12`.

```
Start   12.12End
Start12.12   End
```

more arguments**format string**

So far we have used `printf` to output only one value. However, `printf` can output any number of values. The first argument always is a string known as the **format string**, which can be followed with any number of additional arguments, each of which is a value to output. The format string should include one format specifier, such as `%6.2f` or `%s`, for each value output, and they should be in the same order as the values to be output. For example,

```
double price = 19.8;
String name = "magic apple";
System.out.printf("$%6.2f for each %s.", price, name);
System.out.println();
System.out.println("Wow");
```

This code outputs the following:

```
$ 19.80 for each magic apple.
Wow
```

new lines

Note that the format string may contain text as well as format specifiers, and this text is output along with the values of the other arguments to `printf`.

You can include line breaks in a format string. For example, the following two lines

```
System.out.printf("$%6.2f for each %s.", price, name);
System.out.println();
```

can be replaced by the single line below, which uses the escape sequence `\n`:

```
System.out.printf("$%6.2f for each %s.\n", price, name);
```

%n

Although it is legal to use the escape sequence `\n` to indicate a line break in a format string, it is preferable to use `%n`. Exactly what happens when a `\n` is output can be system dependent, whereas `%n` should always mean a simple new line on any system. So our last line of code would be a little more robust if rewritten using `%n` as follows:

```
System.out.printf("$%.2f for each %s.%n", price, name);
```

Many of the details we have discussed about `printf` are illustrated in the program given in Display 2.2.



TIP: Formatting Monetary Amounts with `printf`

A good format specifier for outputting an amount of money stored as a value of type `double` (or other floating-point value) is `%.2f`. It says to include exactly two digits after the decimal point and to use the smallest field width that the value will fit into. For example,

```
double price = 19.99;
System.out.printf("The price is $%.2f each.", price);
```

produces the following output:

```
The price is $19.99 each. ■
```

Display 2.2 The `printf` Method (part 1 of 2)

```
1 public class PrintfDemo
2 {
3     public static void main(String[] args)
4     {
5         String aString = "abc";

6         System.out.println("String output:");
7         System.out.println("START1234567890");
8         System.out.printf("START%sEND %n", aString);
9         System.out.printf("START4sEND %n", aString);
10        System.out.printf("START2sEND %n", aString);
11        System.out.println();

12        char oneCharacter = 'Z';

13        System.out.println("Character output:");
14        System.out.println("START1234567890");
15        System.out.printf("START%cEND %n", oneCharacter);
16        System.out.printf("START4cEND %n", oneCharacter);
17        System.out.println();
```

(continued)

Display 2.2 The `printf` Method (part 2 of 2)

```

18         double d = 12345.123456789;

19         System.out.println("Floating-point output:");
20         System.out.println("START1234567890");
21         System.out.printf("START%fEND %n", d);
22         System.out.printf("START%.4fEND %n", d);
23         System.out.printf("START%.2fEND %n", d);
24         System.out.printf("START%12.4fEND %n", d);
25         System.out.printf("START%eEND %n", d);
26         System.out.printf("START%12.5eEND %n", d);
27     }
28 }

```

Sample Dialogue

String output:

START1234567890

STARTabcEND

START abcEND

STARTabcEND

The value is always output. If the specified field width is too small, extra space is taken.

Character output:

START1234567890

STARTZEND

START ZEND

Floating-point output:

START1234567890

START12345.123457END

START12345.1235END

START12345.12END

START 12345.1235END

START1.234512e+04END

START 1.23451e+04END

Note that the output is rounded, not truncated, when digits are discarded.

**TIP: Legacy Code****legacy code**

Some code is so expensive to replace, it is used even if it is “old fashioned” or otherwise less than ideal. This sort of code is called **legacy code**. One approach to legacy code is to translate it into a more modern language. The Java method `printf` is essentially the same as a function¹ in the C language that is also named `printf`. This was done intentionally so that it would be easier to translate C code into Java code. ■

¹ Methods are called *functions* in the C and C++ languages.

System.out.printf

`System.out.printf` is used for formatted screen output. `System.out.printf` can have any number of arguments. The first argument is always a format string for the remaining arguments. All the arguments except the first are values to be output to the screen, and these values are output in the formats specified by the format string. The format string can contain text as well as format specifiers, and this text is output along with the values.

Self-Test Exercises

5. What output is produced by the following code?

```
String aString = "Jelly beans";

System.out.println("START1234567890");
System.out.printf("START%sEND %n", aString);
System.out.printf("START%4sEND %n", aString);
System.out.printf("START%13sEND %n", aString);
System.out.println();
```

6. What output is produced by the following code? For each output line, describe whether the line begins or ends with a blank or blanks.

```
String aString = "Jelly beans";
double d = 123.1234567890;

System.out.println("START1234567890");
System.out.printf("START%sEND %n %9.4f %n", aString, d);
```

7. Write a Java statement to output the value in variable `d` of type `double` to the screen. The output should be in e-notation with three digits after the decimal point. The output should be in a field of width 15.

Money Formats Using `NumberFormat` ★

`NumberFormat`

Using the class `NumberFormat`, you can tell Java to use the appropriate format when outputting amounts of money. The technique is illustrated in Display 2.3. Let's look at the code in the `main` method that does the formatting. First consider the following:

```
NumberFormat moneyFormatter =
    NumberFormat.getCurrencyInstance();
```

The method invocation `NumberFormat.getCurrencyInstance()` produces an object of the class `NumberFormat` and names the object `moneyFormatter`. You can use any

Display 2.3 Currency Format (part 1 of 2)

```

1  import java.text.NumberFormat;
2  import java.util.Locale;
3
4  public class CurrencyFormatDemo
5  {
6      public static void main(String[] args)
7      {
8          System.out.println("Without formatting:");
9
10         System.out.println(19.8);
11         System.out.println(19.81111);
12         System.out.println(19.89999);
13         System.out.println(19);
14         System.out.println();
15
16         System.out.println("Default location:");
17         NumberFormat moneyFormatter =
18             NumberFormat.getCurrencyInstance();
19
20         System.out.println(moneyFormatter.format(19.8));
21         System.out.println(moneyFormatter.format(19.81111));
22         System.out.println(moneyFormatter.format(19.89999));
23         System.out.println(moneyFormatter.format(19));
24         System.out.println();
25
26         System.out.println("US as location:");
27         NumberFormat moneyFormatter2 =
28             NumberFormat.getCurrencyInstance(Locale.US);
29
30         System.out.println(moneyFormatter2.format(19.8));
31         System.out.println(moneyFormatter2.format(19.81111));
32         System.out.println(moneyFormatter2.format(19.89999));
33         System.out.println(moneyFormatter2.format(19));
34     }
35 }

```

If you use only the default location, you do not need to import Locale.

Notice that this number is rounded to 19.90.

valid identifier (other than a keyword) in place of `moneyFormatter`. This object `moneyFormatter` has a method named `format` that takes a floating-point number as an argument and returns a `String` value representing that number in the local currency (the default currency). For example, the following invocation

```
moneyFormatter.format(19.8)
```

Display 2.3 Currency Format (part 2 of 2)

Sample Dialogue

Without formatting:

19.8
19.81111
19.89999
19

Default location:

\$19.80
\$19.81
\$19.90
\$19.00

This assumes that the system is set to use U.S. as the default location. If you are not in the U.S., you will probably get the format for your local currency.

US as location:

\$19.80
\$19.81
\$19.90
\$19.00

This should give you the format for U.S. currency no matter what country has been set as the default location.

returns the `String` value "\$19.80", assuming the default currency is the U.S. dollar. In Display 2.3, this method invocation occurs inside a `System.out.println` statement, but it is legal anyplace a `String` value is legal. For example, the following would be legal:

```
String moneyString = moneyFormatter.format(19.8);
```

In order to make the class `NumberFormat` available to your code, you must include the following near the start of the file with your program:

```
import java.text.NumberFormat;
```

This is illustrated in Display 2.3.

The method invocation `NumberFormat.getCurrencyInstance()` produces an object that formats numbers according to the default location. In Display 2.3, we are assuming the default location is the United States, and so the numbers are output as U.S. dollars. On other systems, the default should be set to the local currency. If you wish, you can specify the location, and hence the local currency, by giving an argument to `NumberFormat.getCurrencyInstance`. For example, in Display 2.3, we used the constant `Locale.US` to specify that the location is the United States. The relevant line from Display 2.3 is repeated in what follows:

```
NumberFormat moneyFormatter2 =  
    NumberFormat.getCurrencyInstance(Locale.US);
```

Some constants for other countries (and hence other currencies) are given in Display 2.4. However, unless your screen is capable of displaying the currency symbol for the country whose constant you use, the output might not be as desired.

These location constants are objects of the class `Locale`. In order to make the class `Locale` and these constants available to your code, you must include the following near the start of the file with your program:

```
import java.util.Locale;
```

If you do not use any of these location constants and use only the default location, you do not need this `import` statement.

The notation `Locale.US` may seem a bit strange, but it follows a convention that is frequently used in Java code. The constant is named `US`, but we want specifically that constant named `US` that is defined in the class `Locale`. So we use `Locale.US`. The notation `Locale.US` means the constant `US` as defined in the class `Locale`.

Importing Packages and Classes

package

Libraries in Java are called **packages**. A package is simply a collection of classes that has been given a name and stored in such a way as to make it easily accessible to your Java programs. Java has a large number of standard packages that automatically come with Java. Two such packages are named `java.text` and `java.util`. In Display 2.3, we used the class `NumberFormat`, which is a member of the package `java.text`. In order to use `NumberFormat`, you must import the class, which we did as follows:

```
java.text      import java.text.NumberFormat;
```

Display 2.4 Locale Constants for Currencies of Different Countries

<code>Locale.CANADA</code>	Canada (for currency, the format is the same as U.S.)
<code>Locale.CHINA</code>	China
<code>Locale.FRANCE</code>	France
<code>Locale.GERMANY</code>	Germany
<code>Locale.ITALY</code>	Italy
<code>Locale.JAPAN</code>	Japan
<code>Locale.KOREA</code>	Korea
<code>Locale.TAIWAN</code>	Taiwan
<code>Locale.UK</code>	United Kingdom (English pound)
<code>Locale.US</code>	United States

Outputting Amounts of Money

Using the class `NumberFormat`, you can output an amount of money correctly formatted. The procedure to do so is described here.

Place the following near the start of the file containing your program:

```
import java.text.NumberFormat;
```

In your program code, create an object of the class `NumberFormat` as follows:

```
NumberFormat formatterObject =  
    NumberFormat.getCurrencyInstance();
```

When outputting numbers for amounts of money, change the number to a value of type `String` using the method `formatterObject.format`, as illustrated in the following:

```
double moneyAmount = 9.99;  
System.out.println(formatterObject.format(moneyAmount));
```

The string produced by invocations such as `formatterObject.format(moneyAmount)` adds the dollar sign and ensures that there are exactly two digits after the decimal point. (This is assuming the U.S. dollar is the default currency.)

The numbers formatted in this way may be of type `double`, `int`, or `long`. You may use any (nonkeyword) identifier in place of `formatterObject`. A complete example is given in Display 2.3.

The above always outputs the money amount in the default currency, which is typically the local currency. You can specify the country whose currency you want. See the text for details.

import statement

This kind of statement is called an **import statement**. In this example, the import statement tells Java to look in the package `java.text` to find the definition of the class `NumberFormat`.

If you want to import all the classes in the `java.text` package, use the following:

```
import java.text.*;
```

Then you can use any class in the `java.text` package.

You don't lose any efficiency in importing the entire package instead of importing only the classes you use. However, many programmers find that it is an aid to documentation if they import only the classes they use, which is what we will do in this book.

java.util

In Display 2.3, we also used the class `Locale`, which is in the `java.util` package. So we also included the following import statement:

```
import java.util.Locale;
```

`java.lang`

One package requires no import statement. The package `java.lang` contains classes that are fundamental to Java programming. These classes are so basic that the package is always imported automatically. Any class in `java.lang` does not need an import statement to make it available to your code. So, when we say that a class is in the package `java.lang`, you can simply use that class in your program without needing any import statement. For example, the class `String` is in the `java.lang` package, so you can use it without any import statement.

More material on packages is covered in Chapter 5.

Self-Test Exercises

8. What output is produced by the following code? (Assume a proper import statement has been given.)

```
NumberFormat exerciseFormatter =
    NumberFormat.getCurrencyInstance(Locale.US);
double d1 = 1.2345, d2 = 15.67890;
System.out.println(exerciseFormatter.format(d1));
System.out.println(exerciseFormatter.format(d2));
```

9. Suppose the class `Robot` is a part of the standard Java libraries and is in the package named `java.awt`. What import statement do you need to make the class `Robot` available to your program or other class?

The `DecimalFormat` Class ★

`System.out.println` will let you output numbers but has no facilities to format the numbers. If you want to output a number in a specific format, such as having a specified number of digits after the decimal point, then you must convert the number to a string that shows the number in the desired format and then use `System.out.println` to output the string. Earlier in this chapter, we saw one way to accomplish this for amounts of money. The class `DecimalFormat` provides a versatile facility to format numbers in a variety of ways.

The class `DecimalFormat` is in the Java package named `java.text`. So you must add the following (or something similar) to the beginning of the file with your program or other class that uses the class `DecimalFormat`:

```
import java.text.DecimalFormat;
```

An object of the class `DecimalFormat` has a number of different methods that can be used to produce numeral strings in various formats. In this subsection, we discuss one of these methods, which is named `format`. The general approach to using the `format` method is discussed in the following pages.

`patterns`

Create an object of the class `DecimalFormat`, using a *String Pattern* as follows:

```
DecimalFormat Variable_Name = new DecimalFormat(Pattern);
```

For example,

```
DecimalFormat formattingObject = new DecimalFormat("000.000");
```

The method `format` of the class `DecimalFormat` can then be used to convert a floating-point number, such as one of type `double`, to a corresponding numeral `String` following the *Pattern* used to create the `DecimalFormat` object. Specifically, an invocation of `format` takes the form

```
Decimal_Format_Object.format(Double_Expression)
```

which returns a `String` value for a string representation of the value of *Double_Expression*. *Double_Expression* can be any expression, such as a variable or sum of variables, that evaluates to a value of type `double`.

For example, consider the following code:

```
DecimalFormat formattingObject = new DecimalFormat("000.0000");  
String numeral = formattingObject.format(12.3456789);  
System.out.println(numeral);
```

This produces the following output:

```
012.3457
```

Of course, you can use an invocation of `format`, such as `formattingObject.format(12.3456789)`, directly in `System.out.println`. So, the following code produces the same output:

```
System.out.println(formattingObject.format(12.3456789));
```

The format of the string produced is determined by the *Pattern* string that was used as the argument to the constructor that created the object of the class `DecimalFormat`. For example, the pattern `"000.0000"` means that there will be three digits before the decimal point and four digits after the decimal point. Note that the result is rounded when the number of digits is less than the number of digits available in the number being formatted. If the format pattern is not consistent with the value of the number, such as a pattern that asks for two digits before the decimal point for a number such as `123.456`, then the format rules are violated so that no digits are lost.

A pattern can specify the exact number of digits before and after the decimal, or it can specify minimum numbers of digits. The character `'0'` is used to represent a required digit, and the character `'#'` is used to indicate an optional digit. For example, the pattern `"#0.0##"` indicates one or two digits before the decimal point and one, two, or three digits after the decimal point. The optional digit `'#'` is shown if it is a nonzero digit and is not shown if it is a zero digit. The `'#'` optional digits should go where zero placeholders would appear in a numeral string; in other words, any `'#'` optional digits precede the zero digits `'0'` before the decimal point in the pattern, and

any '#' optional digits follow the zero digits '0' after the decimal point in the pattern. Use "#0.0##"; do not use "0#.0##" or "#0.##0".

For example, consider the following code:

```
DecimalFormat formattingObject = new DecimalFormat("#0.0##");
System.out.println(formattingObject.format(12.3456789));
System.out.println(formattingObject.format(1.23456789));
```

This produces the following output:

```
12.346
1.235
```

percentages

The character '%' placed at the end of a pattern indicates that the number is to be expressed as a percentage. The '%' causes the number to be multiplied by 100 and appends a percent sign, '%'. Examples of this and other formatting patterns are given in Display 2.5.

E-notation

E-notation is specified by including an 'E' in the pattern string. For example, the pattern "00.###E0" approximates specifying two digits before the decimal point, three or fewer digits after the decimal point, and at least one digit after the 'E', as in 12.346E1. As you can see by the examples of E-notation in Display 2.5, the exact details of which E-notation string is produced can be a bit more involved than our explanation so far. Here are a couple more details:

The number of digits indicated after the 'E' is the minimum number of digits used for the exponent. As many more digits as are needed will be used.

mantissa

The **mantissa** is the decimal number before the 'E'. The minimum number of significant digits in the mantissa (that is, the sum of the number of digits before and after the decimal point) is the *minimum* of the number of digits indicated before the decimal point plus the *maximum* of the number of digits indicated after the decimal point. For example, 12345 formatted with "#0.##E0" is "12.3E3".

To get a feel for how E-notation patterns work, it would pay to play with a few cases. In any event, do not count on a very precisely specified number of significant digits.

DecimalFormat Class

Objects of the class `DecimalFormat` are used to produce strings of a specified format from numbers. As such, these objects can be used to format numeric output. The object is associated with a pattern when it is created using `new`. The object can then be used with the method `format` to create strings that satisfy the format. See Display 2.5 for examples of the `DecimalFormat` class in use.

Display 2.5 The DecimalFormat Class (part 1 of 2)

```
1  import java.text.DecimalFormat;

2  public class DecimalFormatDemo
3  {
4      public static void main(String[] args)
5      {
6          DecimalFormat pattern00dot000 = new DecimalFormat("00.000");
7          DecimalFormat pattern0dot00 = new DecimalFormat("0.00");

8          double d = 12.3456789;
9          System.out.println("Pattern 00.000");
10         System.out.println(pattern00dot000.format(d));
11         System.out.println("Pattern 0.00");
12         System.out.println(pattern0dot00.format(d));

13         double money = 19.8;
14         System.out.println("Pattern 0.00");
15         System.out.println("$" + pattern0dot00.format(money));
16
17         DecimalFormat percent = new DecimalFormat("0.00%");

18         System.out.println("Pattern 0.00%");
19         System.out.println(percent.format(0.308));

20         DecimalFormat eNotation1 =
21             new DecimalFormat("#0.###E0"); //1 or 2 digits before point
22         DecimalFormat eNotation2 =
23             new DecimalFormat("00.###E0"); //2 digits before point

24         System.out.println("Pattern #0.###E0");
25         System.out.println(eNotation1.format(123.456));
26         System.out.println("Pattern 00.###E0");
27         System.out.println(eNotation2.format(123.456));

28         double smallNumber = 0.0000123456;
29         System.out.println("Pattern #0.###E0");
30         System.out.println(eNotation1.format(smallNumber));
31         System.out.println("Pattern 00.###E0");
32         System.out.println(eNotation2.format(smallNumber));
33     }
34 }
```

(continued)

Display 2.5 The `DecimalFormat` Class (part 2 of 2)

Sample Dialogue

```

Pattern 00.000
12.346
Pattern 0.00
12.35
Pattern 0.00
$19.80
Pattern 0.00%
30.80%
Pattern #0.###E0
1.2346E2
Pattern 00.###E0
12.346E1
Pattern #0.###E0
12.346E-6
Pattern 00.###E0
12.346E-6

```

The number is always given, even if this requires violating the format pattern.

2.2 Console Input Using the Scanner Class

Let the world tell me.

FRANK SHAKESPEARE, *Franky I*

Starting with version 5.0, Java includes a class for doing simple keyboard input. In this section, we show you how to do keyboard input using this class, which is named `Scanner`.

The `Scanner` Class

Display 2.6 contains a simple program that uses the `Scanner` class to read two `int` values typed in on the keyboard. The numbers entered on the keyboard are shown in **bold**. Let's go over the `Scanner`-related details line by line. The following line, which should be placed near the start of the file, tells Java where to find the definition of the `Scanner` class:

```
import java.util.Scanner;
```

This line says the `Scanner` class is in the `java.util` package; `util` is short for *utility*, but in Java code, you always use the abbreviated spelling `util`. A package is simply a library of classes. This `import` statement makes the `Scanner` class available to your program.

Display 2.6 Keyboard Input Demonstration

```

1  import java.util.Scanner;
2  public class ScannerDemo
3  {
4      public static void main(String[] args)
5      {
6          Scanner keyboard = new Scanner(System.in);

7          System.out.println("Enter the number of pods followed by");
8          System.out.println("the number of peas in a pod:");
9          int numberOfPods = keyboard.nextInt();
10         int peasPerPod = keyboard.nextInt();

11         int totalNumberOfPeas = numberOfPods * peasPerPod;

12         System.out.print(numberOfPods + " pods and ");
13         System.out.println(peasPerPod + " peas per pod.");
14         System.out.println("The total number of peas = "
15                             + totalNumberOfPeas);
16     }
17 }

```

Makes the Scanner class available to your program

Creates an object of the class Scanner and names the object keyboard

Each reads one int from the keyboard.

Sample Dialogue 1

```

Enter the number of pods followed by
the number of peas in a pod:
22 10
22 pods and 10 peas per pod.
The total number of peas = 220

```

The numbers that are input must be separated by whitespace, such as one or more blanks.

Sample Dialogue 2

```

Enter the number of pods followed by
the number of peas in a pod:
22
10
22 pods and 10 peas per pod.
The total number of peas = 220

```

A line break is also considered whitespace and can be used to separate the numbers typed in at the keyboard.

The following line creates an object of the class `Scanner` and names the object `keyboard`:

```
Scanner keyboard = new Scanner(System.in);
```

After this line appears, you can use methods of the `Scanner` class with the object `keyboard` to read data that the user types on the keyboard. For example, the method `nextInt` reads one `int` value typed on the keyboard. So, the following line from Display 2.6 reads one value of type `int` and makes that the value of the variable `numberOfPods`:

`nextInt`

```
int numberOfPods = keyboard.nextInt();
```

In Display 2.6, two such statements each read one `int` value that the user types in at the keyboard:

```
int numberOfPods = keyboard.nextInt();
int peasPerPod = keyboard.nextInt();
```

`whitespace`

The numbers typed in must be separated by **whitespace**, such as one or more spaces, a line break, or other whitespace. Whitespace is any string of characters, such as blank spaces, tabs, and line breaks, that prints as whitespace when written on (white) paper.

We often use the identifier `keyboard` for our `Scanner` object because the object is being used for keyboard input. However, you may use other names. If you instead want your object of the class `Scanner` to be named `scannerObject`, you would use the following:

```
Scanner scannerObject = new Scanner(System.in);
```

To read a single `int` value from the keyboard and save it in the `int` variable `n1`, you would then use the following:

```
n1 = scannerObject.nextInt();
```

This is illustrated in the program in Display 2.7.

The program in Display 2.7 also illustrates some of the other `Scanner` methods for reading values from the keyboard. The method `nextDouble` works in exactly the same way as `nextInt`, except that it reads a value of type `double`. The following example is from Display 2.7:

`nextDouble`

```
double d1, d2;
d1 = scannerObject.nextDouble();
d2 = scannerObject.nextDouble();
System.out.println("You entered " + d1 + " and " + d2);
```

Display 2.7 Another Keyboard Input Demonstration (part 1 of 2)

```

1 import java.util.Scanner;

2 public class ScannerDemo2
3 {
4     public static void main(String[] args)
5     {
6         int n1, n2;
7         Scanner scannerObject = new Scanner(System.in);

8         System.out.println("Enter two whole numbers");
9         System.out.println("separated by one or more spaces:");

10        n1 = scannerObject.nextInt();
11        n2 = scannerObject.nextInt();
12        System.out.println("You entered " + n1 + " and " + n2);

13        System.out.println("Next enter two numbers.");
14        System.out.println("Decimal points are allowed.");

15        double d1, d2;
16        d1 = scannerObject.nextDouble();
17        d2 = scannerObject.nextDouble();
18        System.out.println("You entered " + d1 + " and " + d2);

19        System.out.println("Next enter two words:");

20        String word1 = scannerObject.next();
21        String word2 = scannerObject.next();
22        System.out.println("You entered \" +
23                               word1 + "\" and \"" + word2 + "\"");

24        String junk = scannerObject.nextLine(); //To get rid of '\n'

25        System.out.println("Next enter a line of text:");
26        String line = scannerObject.nextLine();
27        System.out.println("You entered: \" + line + "\"");
28    }
29 }

```

Creates an object of the class Scanner and names the object scannerObject

Reads one int from the keyboard

Reads one double from the keyboard

Reads one word from the keyboard

(continued)

Display 2.7 Another Keyboard Input Demonstration (part 2 of 2)

Sample Dialogue

```

Enter two whole numbers
separated by one or more spaces:
    42    43
You entered 42 and 43
Next enter two numbers.
A decimal point is OK.
    9.99  57
You entered 9.99 and 57.0
Next enter two words:
jelly beans
You entered "jelly" and "beans"
Next enter a line of text:
Java flavored jelly beans are my favorite.
You entered "Java flavored jelly beans are my favorite."

```

The method `next` reads in a word, as illustrated by the following lines from Display 2.7:

```

String word1 = scannerObject.next();
String word2 = scannerObject.next();

```

If the input line is

```
jelly beans
```

then this will assign `w1` the string "jelly" and `w2` the string "beans".

word

For the method `next`, a **word** is any string of nonwhitespace characters delimited by whitespace characters such as blanks or the beginning or ending of a line.

If you want to read in an entire line, you would use the method `nextLine`. For example,

```
String line = scannerObject.nextLine();
```

reads in one line of input and places the string that is read into the variable `line`.

The end of an input line is indicated by the escape sequence `'\n'`. This `'\n'` character is what you input when you press the Enter (Return) key on the keyboard. On the screen, it is indicated by the ending of one line and the beginning of the next line. When `nextLine` reads a line of text, it reads this `'\n'` character, so the next reading of input begins on the next line. However, the `'\n'` does not become part of the string value returned. So, in the previous code, the string named by the variable `line` does not end with the `'\n'` character.

These and other methods for reading in values from the keyboard are given in Display 2.8.

Display 2.8 Methods of the `Scanner` Class (part 1 of 2)

The `Scanner` class can be used to obtain input from files as well as from the keyboard. However, here we are assuming it is being used only for input from the keyboard.

To set things up for keyboard input, you need the following at the beginning of the file with the keyboard input code:

```
import java.util.Scanner;
```

You also need the following before the first keyboard input statement:

```
Scanner Scanner_Object_Name = new Scanner(System.in);
```

The `Scanner_Object_Name` can then be used with the following methods to read and return various types of data typed on the keyboard.

Values to be read should be separated by whitespace characters, such as blanks and/or new lines. When reading values, these whitespace characters are skipped. (It is possible to change the separators from whitespace to something else, but whitespace is the default and is what we will use.)

```
Scanner_Object_Name.nextInt()
```

Returns the next value of type `int` that is typed on the keyboard.

```
Scanner_Object_Name.nextLong()
```

Returns the next value of type `long` that is typed on the keyboard.

```
Scanner_Object_Name.nextByte()
```

Returns the next value of type `byte` that is typed on the keyboard.

```
Scanner_Object_Name.nextShort()
```

Returns the next value of type `short` that is typed on the keyboard.

```
Scanner_Object_Name.nextDouble()
```

Returns the next value of type `double` that is typed on the keyboard.

```
Scanner_Object_Name.nextFloat()
```

Returns the next value of type `float` that is typed on the keyboard.

```
Scanner_Object_Name.next()
```

Returns the `String` value consisting of the next keyboard characters up to, but not including, the first delimiter character. The default delimiters are whitespace characters.

(continued)

Display 2.8 Methods of the Scanner Class (part 2 of 2)

```
Scanner_Object_Name.nextBoolean()
```

Returns the next value of type `boolean` that is typed on the keyboard. The values of `true` and `false` are entered as the strings `"true"` and `"false"`. Any combination of upper- and/or lowercase letters is allowed in spelling `"true"` and `"false"`.

```
Scanner_Object_Name.nextLine()
```

Reads the rest of the current keyboard input line and returns the characters read as a value of type `String`. Note that the line terminator `'\n'` is read and discarded; it is not included in the string returned.

```
Scanner_Object_Name.useDelimiter(New_Delimiter);
```

Changes the delimiter for keyboard input with *Scanner_Object_Name*. The *New_Delimiter* is a value of type `String`. After this statement is executed, *New_Delimiter* is the only delimiter that separates words or numbers. See the subsection "Other Input Delimiters," later in this chapter, for details.

Keyboard Input Using the Scanner Class

You can use an object of the class `Scanner` to read input from the keyboard. To make the `Scanner` class available for use in your code, you should include the following at the start of the file that contains your program (or other code that does keyboard input):

```
import java.util.Scanner;
```

Before you do any keyboard input, you must create an object of the class `Scanner` as follows:

```
Scanner Object_Name = new Scanner(System.in);
```

where *Object_Name* is any (nonkeyword) Java identifier. For example,

```
Scanner keyboard = new Scanner(System.in);
```

The methods `nextInt`, `nextDouble`, and `next` read a value of type `int`, a value of type `double`, and a word, respectively. The method `nextLine` reads the remainder of the current input line including the terminating `'\n'`. However, the `'\n'` is not included in the string value returned. Other input methods are given in Display 2.8.

SYNTAX

```
Int_Variable = Object_Name.nextInt()
```

```
Double_Variable = Object_Name.nextDouble();
```

```
String_Variable = Object_Name.next();
```

```
String_Variable = Object_Name.nextLine();
```

EXAMPLE

```
int number;
number = keyboard.nextInt();
double cost;
cost = keyboard.nextDouble();
String word;
word = keyboard.next();
String line;
line = keyboard.nextLine();
```

**PITFALL: Dealing with the Line Terminator, '\n'**

The method `nextLine` of the class `Scanner` reads the *remainder* of a line of text *starting wherever the last keyboard reading left off*. For example, suppose you create an object of the class `Scanner` as follows:

```
Scanner keyboard = new Scanner(System.in);
```

Suppose you continue with the following code:

```
int n = keyboard.nextInt();
String s1 = keyboard.nextLine();
String s2 = keyboard.nextLine();
```

Now, assume that the input typed on the keyboard is the following:

```
2 heads are
better than
1 head.
```

This sets the value of the variable `n` to 2, that of the variable `s1` to "heads are", and that of the variable `s2` to "better than".

So far there are no problems, but suppose the input were instead as follows:

```
2
heads are better than
1 head.
```

You might expect the value of `n` to be set to 2, the value of the variable `s1` to "heads are better than", and that of the variable `s2` to "1 head". But that is not what happens.

What actually happens is that the value of the variable `n` is set to 2, that of the variable `s1` is set to the empty string, and that of the variable `s2` to "heads are better than". The method `nextInt` reads the 2 but does not read the end-of-line character '\n'. So the first `nextLine` invocation reads the rest of the line that contains the 2.

(continued)



VideoNote
Pitfalls
Involving
`nextLine()`



PITFALL: (continued)

There is nothing more on that line (except for `'\n'`), so `nextLine` returns the empty string. The second invocation of `nextLine` begins on the next line and reads "heads are better than".

When combining different methods for reading from the keyboard, you sometimes have to include an extra invocation of `nextLine` to get rid of the end of a line (to get rid of a `'\n'`). This is illustrated in Display 2.7. ■

The Empty String

empty string

A string can have any number of characters. For example, "Hello" has five characters. There is a string with zero characters, which is called the **empty string**. The empty string is written with a pair of double quotes, with nothing in between the quotes, like so: `" "`. The empty string is encountered more often than you might think. If your code is executing the `nextLine` method to read a line of text, and the user types nothing on the line other than pressing the Enter (Return) key, then the `nextLine` method reads the empty string.



TIP: Prompt for Input

Always prompt the user when your program needs the user to input some data, as in the following example:

```
System.out.println("Enter the number of pods followed by");
System.out.println("the number of peas in a pod:"); ■
```



TIP: Echo Input

echo input

You should normally **echo input**. That is, you should write to the screen all input that your program receives from the keyboard. This way, the user can check that the input has been entered correctly. For example, the following two statements from the program in Display 2.9 echo the values that were read for the number of pods and the number of peas per pod:

```
System.out.print(numberOfPods + " pods and ");
System.out.println(peasPerPod + " peas per pod.");
```

It might seem that there is no need to echo input, because the user's input is automatically displayed on the screen as the user enters it. Why bother to write it to the screen a second time? The input might be incorrect even though it looks correct. For example, the user might type a comma instead of a decimal point or the letter O in place of a zero. Echoing the input can expose such problems. ■

Display 2.9 Self-Service Checkout Line

```
1 import java.util.Scanner;

2 public class SelfService
3 {
4     public static void main(String[] args)
5     {
6         Scanner keyboard = new Scanner(System.in);

7         System.out.println("Enter number of items purchased");
8         System.out.println("followed by the cost of one item.");
9         System.out.println("Do not use a dollar sign.");

10        int count = keyboard.nextInt();
11        double price = keyboard.nextDouble();
12        double total = count * price;

13        System.out.printf("%d items at $%.2f each.%n", count, price);
14        System.out.printf("Total amount due $%.2f.%n", total);

15        System.out.println("Please take your merchandise.");
16        System.out.printf("Place $%.2f in an envelope %n", total);
17        System.out.println("and slide it under the office door.");
18        System.out.println("Thank you for using the self-service line.");
19    }
20 }
```

The dot after `%.2f` is a period in the text, not part of the format specifier.

Sample Dialogue

```
Enter number of items purchased
followed by the cost of one item.
Do not use a dollar sign.
10    19.99
10 items at $19.99 each.
Total amount due $199.90.
Please take your merchandise.
Place $199.90 in an envelope
and slide it under the office door.
Thank you for using the self-service line.
```

Self-Test Exercises

10. Write an import statement that makes the `Scanner` class available to your program or other class.
11. Write a line of code that creates a `Scanner` object named `frank` to be used for obtaining keyboard input.
12. Write a line of code that uses the object `frank` from the previous exercise to read in a word from the keyboard and store the word in the `String` variable named `w`.
13. Write a complete Java program that reads in a line of keyboard input containing two values of type `int` (separated by one or more spaces) and outputs the two numbers as well as the sum of the two numbers.
14. Write a complete Java program that reads in a line of text containing exactly three words (separated by any kind or amount of whitespace) and outputs the line with spacing corrected; that is, the output has no space before the first word and exactly one space between each pair of adjacent words.
15. Something could go wrong with the following code. Identify and fix the problem.

```
Scanner keyboard = new Scanner(System.in);
System.out.println("Enter your age.");
int age = keyboard.nextInt();
System.out.println("Enter your name.");
String name = keyboard.nextLine();
System.out.println(name + ", you are " + age + " years old.");
```

EXAMPLE: Self-Service Checkout

Display 2.9 contains a first draft of a program to use in the self-service line of a hardware store. It still needs some more details and even some more hardware for accepting payment. However, it does illustrate the use of the `Scanner` class for keyboard input and the `printf` method for formatted output.

Note that in `printf`, we used the format specifier `%.2f` for amounts of money. This specifies a floating-point number with exactly two digits after the decimal point but gives no field width. Because no field width is given, the number output is placed in the fewest number of spaces that still allows the full value to be shown.

Other Input Delimiters

When using the `Scanner` class for keyboard input, you can change the delimiters that separate keyboard input to almost any combination of characters, but the details are a bit involved. In this book, we will describe only one simple kind of delimiter change. We will tell you how to change the delimiters from whitespace to one specific delimiter string.

For example, suppose you create a `Scanner` object as follows:

```
Scanner keyboard2 = new Scanner(System.in);
```

You can change the delimiter for the object `keyboard2` to `###` as follows:

```
keyboard2.useDelimiter("###");
```

After this invocation of the `useDelimiter` method, `###` will be the only input delimiter for the input object `keyboard2`. Note that whitespace will no longer be a delimiter for keyboard input done with `keyboard2`. For example, suppose the user enters the following keyboard input:

```
one two##three##
```

The following code would read the two strings `"one two"` and `"three"` and make them the values of the variables `word1` and `word2`:

```
String word1, word2;  
word1 = keyboard2.next();  
word2 = keyboard2.next();
```

This is illustrated in Display 2.10. Note that you can have two different objects of the class `Scanner` with different delimiters in the same program.

Note that no whitespace characters, not even line breaks, serve as an input delimiter for `keyboard2` once this change is made to `keyboard2`.

Self-Test Exercises

16. Suppose your code creates an object of the class `Scanner` named `keyboard` (as described in this chapter). Write code to change the delimiter for `keyboard` to a comma followed by a blank.
17. Continue with the object `keyboard` from Self-Test Exercise 16. Consider the following input:

```
one,two three, four, five
```

What values will the following code assign to the variables `word1` and `word2`?

```
String word1 = keyboard.next();  
String word2 = keyboard.next();
```

Display 2.10 Changing the Input Delimiter

```
1  import java.util.Scanner;

2  public class DelimiterDemo
3  {
4      public static void main(String[] args)
5      {
6          Scanner keyboard1 = new Scanner(System.in);
7          Scanner keyboard2 = new Scanner(System.in);
8          keyboard2.useDelimiter("##");
9          //Delimiter for keyboard1 is whitespace.
10         //Delimiter for keyboard2 is ##.

11         String word1, word2;
12         System.out.println("Enter a line of text:");
13         word1 = keyboard1.next();
14         word2 = keyboard1.next();
15         System.out.println("For keyboard1 the two words read are:");
16         System.out.println(word1);
17         System.out.println(word2);
18         String junk = keyboard1.nextLine(); //To get rid of rest of line.
19
20         System.out.println("Reenter the same line of text:");
21         word1 = keyboard2.next();
22         word2 = keyboard2.next();
23         System.out.println("For keyboard2 the two words read are:");
24         System.out.println(word1);
25         System.out.println(word2);
26     }
27 }
```

Sample Dialogue

```
Enter a line of text:
one two##three##
For keyboard1 the two words read are:
one
two##three##
Reenter the same line of text:
one two##three##
For keyboard2 the two words read are:
one two
three
```

2.3 Introduction to File Input

You shall see them on a beautiful quarto page, where a neat rivulet of text shall meander through a meadow of margin.

RICHARD BRINSLEY SHERIDAN, *The School for Scandal*, 1777.

The `Scanner` class can also be used to read data from a text file. To do this, we must create a `Scanner` object and link it to the file on the disk. Once this is done, the program can read from the `Scanner` object in the same exact way that we read from the console, except the input will come from the file instead of typed from the keyboard. Details about reading and writing from files are not discussed until Chapter 10 and require an understanding of programming concepts we have not yet covered. However, we can provide just enough here so that your programs can read from text files. This will allow you to work on problems with real-world data that would otherwise be too much work to type into your program every time it is run.

The `Scanner` Class for Text File Input

To read from a text file, we need to import the classes `FileInputStream` and `FileNotFoundException` in addition to the `Scanner` class:

```
import java.io.FileInputStream;
import java.io.FileNotFoundException;
```

The `FileInputStream` class handles the connection between a Java program and a file on the disk. The `FileNotFoundException` class is used if a program attempts to open a file that doesn't exist.

To open the file, we create an object of type `Scanner` and then connect it with a `FileInputStream` object associated with the file. We have to handle the scenario where we try to open a file that doesn't exist. One way to do this is with a `try/catch` block. This is discussed more thoroughly in Chapter 9, but the basic format to open a file looks like this:

```
Scanner fileIn = null; // Initializes fileIn to empty
try
{
    // Attempt to open the file
    fileIn = new Scanner(new FileInputStream("PathToFile"));
}
catch (FileNotFoundException e)
{
    // If the file could not be found, this code is executed
    // and then the program exits
}
```



```
        System.out.println("File not found.");  
        System.exit(0);  
    }  
    ... Code continues here
```

This code will create a `Scanner` variable named `fileIn` and initialize it to an empty (`null`) object. Next, Java will run the code inside the `try` block. If the file is not found, then control jumps directly to the `catch` block. In our example, we print out an error message and make the program exit immediately with the statement `System.exit(0)`.

If the file is found, then the `catch` block is skipped entirely, and the program continues to execute whatever code comes after the `catch`. At this point, we can use `fileIn` exactly the same way we used a `Scanner` object connected to the console, except input will be provided from the file, not the keyboard.

For example, we can use `fileIn.nextInt()` to read an integer from the file, `fileIn.nextDouble()` to read a double from the file, `fileIn.next()` to read a string token from the file, or `fileIn.nextLine()` to read an entire line from the text file. Java begins reading from the beginning of the file and proceeds toward the end as data is read.

Unlike reading from the console, we might want to know if we have reached the end of the file. We can use `fileIn.hasNextLine()` to determine if there is data to read. When we are done with the file, we can close it with `fileIn.close()`, which will release any resources that have been allocated by Java in association with the file.

A complete example is shown in Displays 2.11 and 2.12. Display 2.11 shows the contents of a text file named `player.txt`. This file can be created by any program that saves in the plain text format. As an example, let's say that the file contains information about the last player to play a game. The first line of the file contains the high score of the player, 100510, and the second line contains the name of the player, Gordon Freeman. The program in Display 2.12 reads in this information and displays it. It reads in the high score using `nextInt()` and then reads in the name using `nextLine()`. Note that we have to use an additional `nextLine()` after the `nextInt()` to deal with the newline character for the exact same reason discussed earlier in this chapter in the Pitfall in Section 2.2 titled "Dealing with the Line Terminator, '\n'."

Display 2.11 Sample Text File, `player.txt`, that Stores a Player's High Score and Name

```
100510  
Gordon Freeman
```

Display 2.12 Program to Read the Text File in Display 2.11

```

1  import java.util.Scanner;
2  import java.io.FileInputStream;
3  import java.io.FileNotFoundException;
4
5  public class TextFileDemo
6  {
7      public static void main(String[] args)
8      {
9          Scanner fileIn = null; // Initializes fileIn to empty
10         try
11         {
12             // Attempt to open the file
13             fileIn = new Scanner(
14                 new FileInputStream("player.txt"));
15         }
16         catch (FileNotFoundException e)
17         {
18             // This block executed if the file is not found
19             // and then the program exits
20             System.out.println("File not found.");
21             System.exit(0);
22         }
23
24         // If the program gets here then
25         // the file was opened successfully
26         int highscore;
27         String name;
28
29         System.out.println("Text left to read? " +
30             fileIn.hasNextLine());
31         highscore = fileIn.nextInt();
32         fileIn.nextLine(); // Read newline left from nextInt()
33         name = fileIn.nextLine();
34
35         System.out.println("Name: " + name);
36         System.out.println("High score: " + highscore);
37         System.out.println("Text left to read? " +
38             fileIn.hasNextLine());
39         fileIn.close();
40     }
41 }

```

try and catch is explained in more detail in Chapter 9.

The file `player.txt` should be in the same directory as the Java program. You can also supply a full pathname to the file.

This line is explained earlier in this chapter in the Pitfall section "Dealing with the Line Terminator, '\n'."

Sample Dialogue

```

Text left to read? true
Name: Gordon Freeman
High score: 100510
Text left to read? False

```

Self-Test Exercises

18. What would the program in Display 2.12 output if there is no file named `player.txt` in the same directory as the Java program?
19. What is missing from the following code, which attempts to open a file and read an integer?

```
import java.util.Scanner;
import java.io.FileInputStream;
import java.io.FileNotFoundException;

public class ReadInteger
{
    public static void main(String[] args)
    {
        Scanner fileIn = new Scanner(
            new FileInputStream("datafile.txt"));
        int num = fileIn.nextInt();
        System.out.println(num);
    }
}
```

Chapter Summary

- You can use `System.out.println` for simple console output.
- You can use `System.out.printf` for formatted console output.
- You can use `NumberFormat.getCurrencyInstance()` to produce an object that can convert numbers to strings that show the number as a correctly formatted currency amount, for example, by adding a dollar sign and having exactly two digits after the decimal point.
- You can use the class `DecimalFormat` to output numbers using almost any format you desire.
- You can use an object of the class `Scanner` for reading keyboard input.
- You can use an object of the class `Scanner` for reading input from a text file.

Answers to Self-Test Exercises

1. `HelloJoe`
2. `System.out.println("One two buckle your shoe.");`
`System.out.println("Three four shut the door.");`
3. `System.out.println` ends a line of input, so the next output goes on the next line. With `System.out.print`, the next output goes on the same line.

4. 2 2

4

Note that

`2 + " " + 2`

contains a string, namely " ". So, Java knows it is supposed to produce a string. On the other hand, `2 + 2` contains only integers. So Java thinks `+` denotes addition in this second case and produces the value 4 to be output.

5. START1234567890

STARTJelly beansEND

STARTJelly beansEND

START Jelly beansEND

6. The last two of the following lines end with a blank. The last line begins with two blanks, one that follows `%n` in the format string and one because the field width is 9 but the number to be output fills only eight spaces.

START1234567890

STARTJelly beansEND

123.1235

7. `System.out.printf("%15.3e", d);`

8. \$1.23

\$15.68

9. Either

`import java.awt.Robot;`

or

`import java.awt.*;`

10. `import java.util.Scanner;`

11. `Scanner frank = new Scanner(System.in);`

12. `w = frank.next();`

13. `import java.util.Scanner;`

`public class Exercise`

`{`

`public static void main(String[] args)`

`{`

`Scanner keyboard = new Scanner(System.in);`

`System.out.println("Enter two numbers.");`

`int n1 = keyboard.nextInt();`

`int n2 = keyboard.nextInt();`

`int sum = n1 + n2;`

`System.out.println(n1 + " plus " + n2 + " is " + sum);`

`}`

`}`

14. `import java.util.Scanner;`

```
public class Exercise2
{
    public static void main(String[] args)
    {
        Scanner keyboard = new Scanner(System.in);

        System.out.println("Enter a line with three words:");
        String w1 = keyboard.next();
        String w2 = keyboard.next();
        String w3 = keyboard.next();

        System.out.println(w1 + " " + w2 + " " + w3);
    }
}
```

15. The newline character is left in the input buffer after the `nextInt()` call and should be removed prior to calling `nextLine()`. This can be fixed by adding another `nextLine()` call:

```
Scanner keyboard = new Scanner(System.in);
System.out.println("Enter your age.");
int age = keyboard.nextInt();
keyboard.nextLine();
System.out.println("Enter your name.");
String name = keyboard.nextLine();
System.out.println(name + ", you are " + age + " years old.");
```

16. `keyboard.useDelimiter(", ");`

17. `w1` is assigned "one,two three"; `w2` is assigned "four".

18. The program will output "File not found." and exit.

19. The statement that attempts to open the file must be inside a try/catch block, as follows:

```
try
{
    Scanner fileIn = new Scanner(
        new FileInputStream("datafile.txt"));
    int num = fileIn.nextInt();
    System.out.println(num);
}
catch (FileNotFoundException e)
{
    System.out.println("File not found.");
}
```

Alternately, the line `throws FileNotFoundException` could be added to the end of the definition of the main method, but this approach is not recommended because the exception will simply be handed off to the JVM and it will halt the program.

Programming Projects

1. The Babylonian algorithm to compute the square root of a positive number n is as follows:
 1. Make a guess at the answer (you can pick $n/2$ as your initial guess).
 2. Compute $r = n / \text{guess}$
 3. Set $\text{guess} = (\text{guess} + r) / 2$
 4. Go back to step 2 for as many iterations as necessary. The more you repeat steps 2 and 3, the closer guess will become to the square root of n .

Write a program that inputs a double for n , iterates through the Babylonian algorithm five times, and outputs the answer as a double to two decimal places. Your answer will be most accurate for small values of n .

2. (This is a version with input of an exercise from Chapter 1.) Write a program that inputs two string variables, `first` and `last`, which the user should enter with his or her name. First, convert both strings to all lowercase. Your program should then create a new string that contains the full name in pig latin with the first letter capitalized for the first and last name. Use only the pig latin rule of moving the first letter to the end of the word and adding “ay.” Output the pig latin name to the screen. Use the `substring` and `toUpperCase` methods to construct the new name.

For example, if the user inputs “Walt” for the first name and “Savitch” for the last name, then the program should create a new string with the text “Altway Avitchsay” and print it.

3. Write a program that reads in two numbers typed on the keyboard and divides the first number by the second number. The program should output the dividend, divisor, quotient, and remainder on the screen.
4. John travels a distance of 55 miles at an average speed of 15 miles per hour. Write a program to calculate the total number of hours John takes to cover this distance. The program should print the total time taken in hours and minutes. Use the following formula for calculations.

$$\text{Time} = \frac{\text{Distance}}{\text{Speed}}$$

5. Grade point average (GPA) in a 4-point scale is calculated by using the following formula:

$$\text{GPA} = \left(\frac{\text{Percentage}}{100} \right) \times 4$$

Write a program that takes as input the percentage from a user. The program should then output the user’s GPA on the screen. The format of the output should be as follows, assuming the user’s percentage is 85:

$$(85/100) * 4 = 3$$

6. (This is a better version of an exercise from Chapter 1.) A government research lab has concluded that an artificial sweetener commonly used in diet soda pop causes death in laboratory mice. A friend of yours is desperate to lose weight but cannot give up soda pop. Your friend wants to know how much diet soda pop it is possible to drink without dying as a result. Write a program to supply the answer. The input to the program is the amount of artificial sweetener needed to kill a mouse, the weight of the mouse, and the desired weight of the dieter. Assume that diet

soda contains 1/10th of 1% artificial sweetener. Use a named constant for this fraction. You may want to express the percent as the `double` value `0.001`.

7. Write a program that determines the change to be dispensed from a vending machine. An item in the machine can cost between 25 cents and 1 dollar, in 5-cent increments (25, 30, 35, . . . , 90, 95, or 100), and the machine accepts only a single dollar bill to pay for the item. For example, a possible sample dialogue might be the following:

```
Enter price of item
(from 25 cents to a dollar, in 5-cent increments):
45
```

```
You bought an item for 45 cents and gave me a dollar,
so your change is
2 quarters,
0 dimes, and
1 nickels.
```

8. Write a program that reads in a string containing three words separated by commas and then outputs that string with each word in a different line.
9. (This is a better version of an exercise from Chapter 1.) Write a program that reads in a line of text and then outputs that line of text with the first occurrence of "hate" changed to "love". For example, a possible sample dialogue might be the following:

```
Enter a line of text.
I hate you.
I have rephrased that line to read:
I love you.
```

You can assume that the word "hate" occurs in the input. If the word "hate" occurs more than once in the line, your program should replace only the first occurrence of "hate".

10. Write a program that inputs the name, quantity, and price of three items. The name may contain spaces. Output a bill with a tax rate of 6.25%. All prices should be output to two decimal places. The bill should be formatted in columns with 30 characters for the name, 10 characters for the quantity, 10 characters for the price, and 10 characters for the total. Sample input and output are shown as follows:

```
Input name of item 1:
lollipops
Input quantity of item 1:
10
Input price of item 1:
0.50
Input name of item 2:
diet soda
```

Input quantity of item 2:

3

Input price of item 2:

1.25

Input name of item 3:

chocolate bar

Input quantity of item 3:

20

Input price of item 3:

0.75

Your bill:

Item	Quantity	Price	Total
lollipops	10	0.50	5.00
diet soda	3	1.25	3.75
chocolate bar	20	0.75	15.00
Subtotal			23.75
6.25% sales tax			1.48
Total			25.23



VideoNote
Solution to
Programming
Project 2.11

- Write a program that calculates the total grade for three classroom exercises as a percentage. Use the `DecimalFormat` class to output the value as a percent. The scores should be summarized in a table. Input the assignment information in this order: name of assignment (may include spaces), points earned (integer), and total points possible (integer). The percentage is the sum of the total points earned divided by the total points possible. Sample input and output are shown as follows:

Name of exercise 1:

Group Project

Score received for exercise 1:

10

Total points possible for exercise 1:

10

Name of exercise 2:

Homework

Score received for exercise 2:

7

Total points possible for exercise 2:

12

Name of exercise 3:

Presentation

Score received for exercise 3:

5

Total points possible for exercise 3:

8

Exercise	Score	Total Possible
Group Project	10	10
Homework	7	12
Presentation	5	8
Total	22	30

Your total is 22 out of 30, or 73.33%.



VideoNote
Solution to
Programming
Project 2.12

12. (This is a variant of an exercise from Chapter 1.) Create a text file that contains the text "I hate programming!" Write a program that reads in this line of text from the file and then the text with the first occurrence of "hate" changed to "love". In this case, the program would output "I love programming!" Your program should work with any line of text that contains the word "hate", not just the example given in this problem. If the word "hate" occurs more than once in the line, your program should replace only the first occurrence of "hate".
13. (This is an extension of an exercise from Chapter 1.) A simple rule to estimate your ideal body weight is to allow 110 pounds for the first 5 feet of height and 5 pounds for each additional inch. Create the following text in a text file. It contains the names and heights in feet and inches of Tom Atto (6'3"), Eaton Wright (5'5"), and Cary Oki (5'11"):

```
Tom Atto
6
3
Eaton Wright
5
5
Cary Oki
5
11
```

Write a program that reads the data in the file and outputs the full name and ideal body weight for each person. In the next chapter, you will learn about loops, which allow for a more efficient way to solve this problem.

14. From Programming Project 10 in Chapter 1, scientists estimate that roughly 10 grams of caffeine consumed at one time is a lethal overdose. Write two programs to compute how many drinks it takes to kill a person.

The first program should input the name of the drink and the number of milligrams of caffeine in the drink. It should then output the drink name and the number of drinks it takes to kill a person. For example, if the user enters "coffee" and "160", then the program should output "It will take approximately 62.5 drinks of coffee to kill a person from caffeine."

The second program should perform the same calculation as the first program, but read the name of the drink and the number of milligrams of caffeine in the drink from a file named `drink.txt` and output the answer to the console.



Flow of Control **3**

3.1 BRANCHING MECHANISM 132

- `if-else` Statements 132
- Omitting the `else` 133
- Compound Statements 134
- Nested Statements 136
- Multiway `if-else` Statement 136
- Example: State Income Tax 137
- The `switch` Statement 139
- The Conditional Operator ★ 144

3.2 BOOLEAN EXPRESSIONS 145

- Simple Boolean Expressions 145
- Lexicographic and Alphabetic Order 148
- Building Boolean Expressions 151
- Evaluating Boolean Expressions 152
- Short-Circuit and Complete Evaluation 156
- Precedence and Associativity Rules 157

3.3 LOOPS 164

- `while` Statement and `do-while` Statement 164
- Algorithms and Pseudocode 166
- Example: Averaging a List of Scores 169
- The `for` Statement 170
- The Comma in `for` Statements 173
- Nested Loops 177
- The `break` and `continue` Statements ★ 180
- The `exit` Statement 181

3.4 DEBUGGING 182

- Loop Bugs 182
- Tracing Variables 182
- General Debugging Techniques 183
- Example: Debugging an Input Validation Loop 184
- Preventive Coding 188
- Assertion Checks ★ 189

3.5 RANDOM NUMBER GENERATION ★ 191

- The `Random` Object 191
- The `Math.random()` Method 193

*If you think we're wax-works," he said, "you ought to pay, you know.
Wax-works weren't made to be looked at for nothing. Nohow!"
"Contrariwise," added the one marked "DEE," "if you think we're alive,
you ought to speak."*

LEWIS CARROLL, *Through the Looking-Glass*, 1871.

Introduction

As in most programming languages, Java handles flow of control with branching and looping statements. Java branching and looping statements are the same as in the C and C++ languages and are very similar to those in other programming languages. (However, the Boolean expressions that control Java branches and loops are a bit different in Java from what they are in C and C++.)

Most branching and looping statements are controlled by Boolean expressions. A Boolean expression is any expression that is either true or false. In Java, the primitive type `boolean` has only the two values, `true` and `false`, and Boolean expressions evaluate to one of these two values. Before we discuss Boolean expressions and the type `boolean`, we will introduce the Java branching statements using only Boolean expressions whose meaning is intuitively obvious. This will serve to motivate our discussion of Boolean expressions.

Prerequisites

This chapter uses material from Chapters 1 and 2.

3.1 Branching Mechanism

When you come to a fork in the road, take it.

I Really Didn't Say Everything I Said! Yogi Berra, NY: Workman, 1998.

`if-else` Statements

An `if-else` statement chooses between two alternative statements based on the value of a Boolean expression. For example, suppose you want to design a program to compute a week's salary for an hourly employee. Assume the firm pays an overtime rate

of one-and-one-half times the regular rate for all hours after the first 40 hours worked. When the employee works 40 or more hours, the pay is then equal to

```
rate*40 + 1.5*rate*(hours - 40)
```

However, if the employee works less than 40 hours, the correct pay formula is simply

```
rate*hours
```

The following `if-else` statement computes the correct pay for an employee whether the employee works less than 40 hours or works 40 or more hours:

```
if (hours > 40)
    grossPay = rate*40 + 1.5*rate*(hours - 40);
else
    grossPay = rate*hours;
```

The syntax for an `if-else` statement is given in the box entitled “`if-else` Statement.” If the Boolean expression in parentheses (after the `if`) evaluates to `true`, then the statement before the `else` is executed. If the Boolean expression evaluates to `false`, then the statement after the `else` is executed.

parentheses

Remember that when you use a Boolean expression in an `if-else` statement, the Boolean expression must be enclosed in **parentheses**.

Notice that an `if-else` statement has smaller statements embedded in it. Most of the statement forms in Java allow you to make larger statements out of smaller ones by combining the smaller statements in certain ways.

Omitting the `else`

Sometimes you want one of the two alternatives in an `if-else` statement to do nothing at all. In Java, this can be accomplished by omitting the `else` part. These sorts of statements are referred to as **`if` statements** to distinguish them from `if-else` statements. For example, the first of the following two statements is an `if` statement:

if statement

```
if (sales > minimum)
    salary = salary + bonus;
System.out.println("salary = $" + salary);
```

If the value of `sales` is greater than the value of `minimum`, the assignment statement is executed, and then the following `System.out.println` statement is executed. On the other hand, if the value of `sales` is less than or equal to `minimum`, then the embedded assignment statement is not executed, so the `if` statement causes no change (that is, no bonus is added to the base salary), and the program proceeds directly to the `System.out.println` statement.

**if-else
with multiple
statements**

**compound
statement**

Compound Statements

You will often want the branches of an `if-else` statement to execute more than one statement each. To accomplish this, enclose the statements for each branch between a pair of braces, `{` and `}`. A list of statements enclosed in a pair of braces is called a **compound statement**. A compound statement is treated as a single statement by Java and may be used anywhere that a single statement may be used. Thus, the “Multiple Statement Alternatives” version described in the box entitled “`if-else` Statement” is really just a special case of the “simple” case with one statement in each branch.

if-else Statement

The `if-else` statement chooses between two alternative actions based on the value of a `Boolean_Expression`; that is, an expression that is either true or false, such as `balance < 0`.

SYNTAX

```
if (Boolean_Expression)
    Yes_Statement
else
    No_Statement
```

*Be sure to note that the
`Boolean_Expression` must be
enclosed in parentheses.*

If `Boolean_Expression` is true, then `Yes_Statement` is executed. If `Boolean_Expression` is false, then `No_Statement` is executed.

EXAMPLE

```
if (time < limit)
    System.out.println("You made it.");
else
    System.out.println("You missed the deadline.");
```

Omitting the else Part

You may omit the `else` part to obtain what is often called an **if statement**.

SYNTAX

```
if (Boolean_Expression)
    Action_Statement
```

If `Boolean_Expression` is true, then `Action_Statement` is executed; otherwise, nothing happens and the program goes on to the next statement.

EXAMPLE

```
if (weight > ideal)
    calorieAllotment = calorieAllotment - 500;
```

Multiple Statement Alternatives

In an if-else statement, you can have one or both alternatives contain several statements. To accomplish this, group the statements using braces, as in the following example:

```
if (myScore > yourScore)
{
    System.out.println("I win!");
    wager = wager + 100;
}
else
{
    System.out.println("I wish these were golf scores.");
    wager = 0;
}
```



TIP: Placing of Braces

There are two commonly used ways of indenting and placing braces in if-else statements. They are illustrated as follows:

```
if (myScore > yourScore)
{
    System.out.println("I win!");
    wager = wager + 100;
}
else
{
    System.out.println("I wish these were golf scores.");
    wager = 0;
}
```

and

```
if (myScore > yourScore) {
    System.out.println("I win!");
    wager = wager + 100;
} else {
    System.out.println("I wish these were golf scores.");
    wager = 0;
}
```

The only difference is the placement of braces. The first form is called the *Allman style*, named after programmer Eric Allman. We find the Allman style easier to read, and so we prefer it in this book. The second form is called the *Kernighan & Ritchie* or *K&R style*, named after Dennis Ritchie (the designer of C) and Brian Kernighan (author of the first C tutorial). The K&R style saves lines, so some programmers prefer it or some minor variant of it.

Be sure to note the indenting pattern in these examples. ■

Nested Statements

As you have seen, `if-else` statements and `if` statements contain smaller statements within them. Thus far, we have used compound statements and simple statements, such as assignment statements, as these smaller substatements, but there are other possibilities. In fact, any statement at all can be used as a subpart of an `if-else` statement or other statement that has one or more statements within it.

indenting

When nesting statements, you normally indent each level of nested substatements, although there are some special situations (such as a multiway `if-else` statement) where this rule is not followed.

Self-Test Exercises

1. Write an `if-else` statement that outputs the word "High" if the value of the variable `score` is greater than 100 and outputs "Low" if the value of `score` is at most 100. The variable `score` is of type `int`.
2. Suppose `savings` and `expenses` are variables of type `double` that have been given values. Write an `if-else` statement that outputs the word "Solvent", decreases the value of `savings` by the value of `expenses`, and sets the value of `expenses` to zero, provided that `savings` is larger than `expenses`. If, however, `savings` is less than or equal to `expenses`, the `if-else` statement should simply output the word "Bankrupt" without changing the value of any variables.
3. Suppose `number` is a variable of type `int`. Write an `if-else` statement that outputs the word "Positive" if the value of the variable `number` is greater than 0 and outputs the words "Not positive" if the value of `number` is less than or equal to 0.
4. Suppose `salary` and `deductions` are variables of type `double` that have been given values. Write an `if-else` statement that outputs the word "Crazy" if `salary` is less than `deductions`; otherwise, it should output "OK" and set the variable `net` equal to `salary` minus `deductions`.

Multiway if-else Statement

multiway if-else statement

The **multiway `if-else` statement** is not really a different kind of Java statement. It is simply an ordinary `if-else` statement nested inside `if-else` statements, but it is thought of as a different kind of statement and is indented differently from other nested statements so as to reflect this thinking.

The syntax for a multiway `if-else` statement and a simple example are given in the box entitled "Multiway `if-else` Statement." Note that the Boolean expressions are aligned with one another, and their corresponding actions are also aligned with one another. This makes it easy to see the correspondence between Boolean expressions and actions. The Boolean expressions are evaluated in order until a `true` Boolean expression is found. At that point, the evaluation of Boolean expressions stops, and the

action corresponding to the first `true` Boolean expression is executed. The final `else` is optional. If there is a final `else` and all the Boolean expressions are `false`, the final action is executed. If there is no final `else` and all the Boolean expressions are `false`, then no action is taken. An example of a multiway `if-else` statement is given in the following Programming Example.

Multiway `if-else` Statement

SYNTAX

```
if (Boolean_Expression_1)
    Statement_1
else if (Boolean_Expression_2)
    Statement_2
    .
    .
    .
else if (Boolean_Expression_n)
    Statement_n
else
    Statement_For_All_Other_Possibilities
```

EXAMPLE

```
if (numberOfPeople < 50)
    System.out.println("Less than 50 people");
else if (numberOfPeople < 100)
    System.out.println("At least 50 and less than 100 people");
else if (numberOfPeople < 200)
    System.out.println("At least 100 and less than 200 people");
else
    System.out.println("At least 200 people");
```

The Boolean expressions are checked in order until the first `true` Boolean expression is encountered, and then the corresponding statement is executed. If none of the Boolean expressions is `true`, then the *Statement_For_All_Other_Possibilities* is executed.

EXAMPLE: State Income Tax

Display 3.1 contains a program that uses a multiway `if-else` statement to compute state income tax. This state computes tax according to the following rate schedule:

1. No tax is paid on the first \$15,000 of net income.
2. A tax of 5% is assessed on each dollar of net income from \$15,001 to \$30,000.
3. A tax of 10% is assessed on each dollar of net income over \$30,000.

(continued)

EXAMPLE: (continued)

The program uses a multiway `if-else` statement with one action for each of the above three cases. The condition for the second case is actually more complicated than it needs to be. The computer will not get to the second condition unless it has already tried the first condition and found it to be `false`. Thus, you know that whenever the computer tries the second condition, it knows that `netIncome` is greater than 15000. Hence, you can replace the line

```
else if ((netIncome > 15000) && (netIncome <= 30000))
```

with the following, and the program will perform exactly the same:

```
else if (netIncome <= 30000)
```

Self-Test Exercises

5. What output will be produced by the following code?

```
int extra = 2;
if (extra < 0)
    System.out.println("small");
else if (extra == 0)
    System.out.println("medium");
else
    System.out.println("large");
```

6. What would be the output in Self-Test Exercise 5 if the assignment were changed to the following?

```
int extra = -37;
```

7. What would be the output in Self-Test Exercise 5 if the assignment were changed to the following?

```
int extra = 0;
```

8. Write a multiway `if-else` statement that classifies the value of an `int` variable `n` into one of the following categories and writes out an appropriate message:

```
n < 0 or 0 ≤ n < 100 or n ≥ 100
```

Hint: Remember that the Boolean expressions are checked in order.

Display 3.1 Tax Program

```
1  import java.util.Scanner;

2  public class IncomeTax
3  {
4      public static void main(String[] args)
5      {
6          Scanner keyboard = new Scanner(System.in);
7          double netIncome, tax, fivePercentTax, tenPercentTax;

8          System.out.println("Enter net income.\n"
9                             + "Do not include a dollar sign or any commas.");
10         netIncome = keyboard.nextDouble( );

11         if (netIncome <= 15000)
12             tax = 0;
13         else if ((netIncome > 15000) && (netIncome <= 30000))
14             //tax = 5% of amount over $15,000
15             tax = (0.05*(netIncome - 15000));
16         else //netIncome > $30,000
17             {
18                 //fivePercentTax = 5% of income from $15,000 to $30,000.
19                 fivePercentTax = 0.05*15000;
20                 //tenPercentTax = 10% of income over $30,000.
21                 tenPercentTax = 0.10*(netIncome - 30000);
22                 tax = (fivePercentTax + tenPercentTax);
23             }

24         System.out.printf("Tax due = $%.2f", tax);
25     }
26 }
27
```

Sample Dialogue

```
Enter net income.
Do not include a dollar sign or any commas.
40000
Tax due = $1750.00
```

The switch Statement

switch statement

The **switch statement** is the only other kind of Java statement that implements multiway branches. The syntax for a switch statement and a simple example are shown in the box entitled “The switch Statement.”

**controlling
expression****case labels**

When a `switch` statement is executed, one of a number of different branches is executed. The choice of which branch to execute is determined by a **controlling expression** given in parentheses after the keyword `switch`. Following this are a number of occurrences of the reserved word `case` followed by a constant and a colon. These constants are called **case labels**. The controlling expression for a `switch` statement must be one of the types `char`, `int`, `short`, `byte`, or `String`.¹ The `String` data type is allowed only in Java 7 or higher. The case labels must all be of the same type as the controlling expression. No case label can occur more than once, because that would be an ambiguous instruction. There may also be a section labeled `default`;, which is usually last.

When the `switch` statement is executed, the controlling expression is evaluated, and the computer looks at the case labels. If it finds a case label that equals the value of the controlling expression, it executes the code for that case label.

**break
statement**

The `switch` statement ends when either a `break` statement is executed or the end of the `switch` statement is reached. A **break statement** consists of the keyword `break` followed by a semicolon. When the computer executes the statements after a case label, it continues until it reaches a `break` statement. When the computer encounters a `break` statement, the `switch` statement ends. If you omit the `break` statements, then after executing the code for one case, the computer will go on to execute the code for the next case.

Note that you can have two case labels for the same section of code, as in the following portion of a `switch` statement:

```
case 'A':
case 'a':
    System.out.println("Excellent. You need not take the final.");
    break;
```

Because the first case has no `break` statement (in fact, no statements at all), the effect is the same as having two labels for one case, but Java syntax requires one keyword `case` for each label, such as `'A'` and `'a'`.

default

If no case label has a constant that matches the value of the controlling expression, then the statements following the **default** label are executed. You need not have a default section. If there is no default section and no match is found for the value of the controlling expression, then nothing happens when the `switch` statement is executed. However, it is safest to always have a default section. If you think your case labels list all possible outcomes, you can put an error message in the default section.

The default case need not be the last case in a `switch` statement, but making it the last case, as we have always done, makes the code clearer.

A sample `switch` statement is shown in Display 3.2. Notice that the case labels do not need to be listed in order and do not need to span a complete interval.

¹As we will learn in Chapter 6, the type may also be an enumerated type.

The switch Statement

SYNTAX

```

switch (Controlling_Expression)
{
    case Case_Label_1:
        Statement_Sequence_1
        break;
    case Case_Label_2:
        Statement_Sequence_2
        break;
        .
        .
        .
    case Case_Label_n:
        Statement_Sequence_n
        break;
    default:
        Default_Statement_Sequence
        break;
}

```

Each Case_Label is a constant of the same type as the Controlling_Expression. The Controlling_Expression must be of type char, int, short, byte, or String.

A break may be omitted. If there is no break, execution just continues to the next case.

The default case is optional.

EXAMPLE

```

int vehicleClass;
double toll;
.
.
.
switch (vehicleClass)
{
    case 1:
        System.out.println("Passenger car.");
        toll = 0.50;
        break;
    case 2:
        System.out.println("Bus.");
        toll = 1.50;
        break;
    case 3:
        System.out.println("Truck.");
        toll = 2.00;
        break;
    default:
        System.out.println("Unknown vehicle class!");
        break;
}

```

Display 3.2 A switch Statement (part 1 of 2)

```

1  import java.util.Scanner;
2
3  public class SwitchDemo
4  {
5      public static void main(String[] args)
6      {
7          Scanner keyboard = new Scanner(System.in);
8
9          System.out.println("Enter number of ice cream flavors:");
10         int numberOfFlavors = keyboard.nextInt( );
11
12         switch (numberOfFlavors)
13         {
14             case 32:
15                 System.out.println("Nice selection.");
16                 break;
17             case 1:
18                 System.out.println("I bet it's vanilla.");
19                 break;
20             case 2:
21                 System.out.println(numberOfFlavors + "flavors");
22                 System.out.println("is acceptable.");
23                 break;
24             case 3:
25                 System.out.println(numberOfFlavors + "flavors");
26                 System.out.println("is acceptable.");
27                 break;
28             case 4:
29                 System.out.println(numberOfFlavors + "flavors");
30                 System.out.println("is acceptable.");
31                 break;
32             default:
33                 System.out.println("I didn't plan for");
34                 System.out.println(numberOfFlavors + " flavors.");
35                 break;
36         }
37     }
38 }

```

Annotations in the image:

- Controlling expression* points to `numberOfFlavors` in the `switch` statement.
- Case labels* points to the `case` keywords.
- break statement* points to a `break;` statement.

Sample Dialogue 1

```

Enter number of ice cream flavors:
1
I bet it's vanilla.

```

Sample Dialogue 2

```

Enter number of ice cream flavors:
32
Nice selection.

```

Display 3.2 A `switch` Statement (part 2 of 2)

Sample Dialogue 3

```
Enter number of ice cream flavors:  
3  
3 flavors  
is acceptable.
```

Sample Dialogue 4

```
Enter number of ice cream flavors:  
9  
I didn't plan for  
9 flavors.
```

**PITFALL: Forgetting a `break` in a `switch` Statement**

If you forget a `break` in a `switch` statement, the compiler does not issue an error message. You will have written a syntactically correct `switch` statement, but it will not do what you intended it to do. Notice the annotation in the example in the box entitled “The `switch` Statement.”

The last case in a `switch` statement does not need a `break`, but it is a good idea to include it nonetheless. That way, if a new case is added after the last case, you will not forget to add a `break` (because it is already there). This advice about `break` statements also applies to the default case when it is last. It is best to place the default case last, but that is not required by the Java language, so there is always a possibility of somebody adding a case after the default case. ■

Self-Test Exercises

9. What is the output produced by the following code?

```
char letter = 'B';  
switch (letter)  
{  
    case 'A':  
    case 'a':  
        System.out.println("Some kind of A.");  
    case 'B':  
    case 'b':  
        System.out.println("Some kind of B.");  
        break;  
    default:  
        System.out.println("Something else.");  
        break;  
}
```

(continued)

Self-Test Exercises (continued)

10. What is the output produced by the following code?

```
int key = 1;
switch (key + 1)
{
    case 1:
        System.out.println("Apples");
        break;
    case 2:
        System.out.println("Oranges");
        break;
    case 3:
        System.out.println("Peaches");
    case 4:
        System.out.println("Plums");
        break;
    default:
        System.out.println("Fruitless");
}
```

11. What would be the output in Self-Test Exercise 10 if the first line were changed to the following?

```
int key = 3;
```

12. What would be the output in Self-Test Exercise 10 if the first line were changed to the following?

```
int key = 5;
```

The Conditional Operator ★

**conditional
operator**

**ternary
operator**

arithmetic if

You can embed a branch inside of an expression by using a ternary operator known as the **conditional operator** (also called the **ternary operator** or **arithmetic if**). Its use is reminiscent of an older programming style, and we do not advise using it. It is included here for the sake of completeness (and in case you disagree with our programming style).

The conditional operator is a notational variant on certain forms of the `if-else` statement. The following example illustrates the conditional operator. Consider the following `if-else` statement:

```
if (n1 > n2)
    max = n1;
else
    max = n2;
```

This can be expressed using the *conditional operator* as follows:

```
max = (n1 > n2) ? n1 : n2;
```

The expression on the right-hand side of the assignment statement is the conditional operator expression:

```
(n1 > n2) ? n1 : n2
```

The `?` and `:` together form a ternary operator known as the conditional operator. A conditional operator expression starts with a Boolean expression followed by a `?` and then followed by two expressions separated with a colon. If the Boolean expression is `true`, then the value of the first of the two expressions is returned as the value of the entire expression; otherwise, the value of the second of the two expressions is returned as the value of the entire expression.

3.2 Boolean Expressions

“Contrariwise,” continued Tweedledee, “if it was so, it might be; and if it were so, it would be; but as it isn’t, it ain’t. That’s logic.”

LEWIS CARROLL, *Through the Looking-Glass*, 1871.

Boolean expression

Now that we have motivated Boolean expressions by using them in `if-else` statements, we will discuss them and the type `boolean` in more detail. A **Boolean expression** is simply an expression that is either `true` or `false`. The name *Boolean* is derived from George Boole, a 19th-century English logician and mathematician whose work was related to these kinds of expressions.

Simple Boolean Expressions

We have already been using simple Boolean expressions in `if-else` statements. The simplest Boolean expressions are comparisons of two expressions, such as

```
time < limit
```

and

```
balance <= 0
```

A Boolean expression does not need to be enclosed in parentheses to qualify as a Boolean expression, although it does need to be enclosed in parentheses when it is used in an `if-else` statement.

Display 3.3 shows the various Java comparison operators you can use to compare two expressions.



PITFALL: Using = in Place of ==

Because the equal sign, =, is used for assignment in Java, something else is needed to indicate equality. In Java, equality is indicated with two equal signs with no space between them, as in

```
if (yourScore == myScore)
    System.out.println("A tie.");
```

Fortunately, if you do use = in place of ==, Java will probably give you a compiler error message. (The only case that does not give an error message is when the expression in parentheses happens to form a correct assignment to a boolean variable.) ■

Display 3.3 Java Comparison Operators

MATH NOTATION	NAME	JAVA NOTATION	JAVA EXAMPLES
=	Equal to	==	x + 7 == 2*y answer == 'y'
≠	Not equal to	!=	score != 0 answer != 'y'
>	Greater than	>	time > limit
≥	Greater than or equal to	>=	age >= 21
<	Less than	<	pressure < max
≤	Less than or equal to	<=	time <=limit

The Methods equals and equalsIgnoreCase

When testing strings for equality, do not use ==. Instead, use either equals or equalsIgnoreCase.

SYNTAX

```
String.equals (Other_String)
String.equalsIgnoreCase (Other_String)
```

EXAMPLE

```
String s1;
.
.
.
```

```
if ( s1.equals("Hello") )
    System.out.println("The string is Hello.");
else
    System.out.println("The string is not Hello.");
```



PITFALL: Using == with Strings

Although `==` correctly tests two values of a primitive type, such as two numbers, to see whether they are equal, it has a different meaning when applied to objects, such as objects of the class `String`.² Recall that an object is something whose type is a class, such as a string. All strings are in the class `String` (that is, are of type `String`), so `==` applied to two strings does not test to see whether the strings are equal. Instead, it tests whether two strings refer to the same object. We will discuss references in Chapter 15. To test two strings (or any two objects) to see if they have equal values, you should use the method `equals` rather than `==`. For example, suppose `s1` and `s2` are `String` variables that have been given values, and consider the statement

```
if (s1.equals(s2))
    System.out.println("They are equal strings.");
else
    System.out.println("They are not equal strings.");
```

If `s1` and `s2` name strings that contain the same characters in the same order, then the output will be

```
They are equal strings.
```

The notation may seem a bit awkward at first, because it is not symmetric between the two things being tested for equality. The two expressions

```
s1.equals(s2)
s2.equals(s1)
```

are equivalent.

The method `equalsIgnoreCase` behaves similarly to `equals`, except that with `equalsIgnoreCase`, the upper- and lowercase versions of the same letter are considered the same. For example, "Hello" and "hello" are not equal because their first characters, 'H' and 'h', are different characters. But they would be considered equal by the method `equalsIgnoreCase`. For example, the following will output `Equal ignoring case.`:

```
if ("Hello".equalsIgnoreCase("hello"))
    System.out.println("Equal ignoring case.");
```

(continued)

²When applied to two strings (or any two objects), `==` tests to see if they are stored in the same memory location, but we will not discuss that until Chapter 4. For now, we need only note that `==` does something other than test for the equality of two strings.

**PITFALL: (continued)**

Notice that it is perfectly legal to use a quoted string with a `String` method, as in the preceding use of `equalsIgnoreCase`. A quoted string is an object of type `String` and has all the methods that any other object of type `String` has.

For the kinds of applications we are looking at in this chapter, you could also use `==` to test for equality of objects of type `String`, and it would deliver the correct answer. However, there are situations in which `==` does not correctly test strings for equality, so you should get in the habit of using `equals` rather than `==` to test strings. ■

Lexicographic and Alphabetic Order

lexicographic ordering

The method `compareTo` tests two strings to determine their lexicographic order. **Lexicographic ordering** is similar to alphabetic ordering and is sometimes, but not always, the same as alphabetic ordering. The easiest way to think about lexicographic ordering is to think of it as being the same as alphabetic ordering *but with the alphabet ordered differently*. Specifically, in lexicographic ordering, the letters and other characters are ordered as in the ASCII ordering, which is shown in Appendix 3.

compareTo

If `s1` and `s2` are two variables of type `String` that have been given `String` values, then

```
s1.compareTo(s2)
```

returns a negative number if `s1` comes before `s2` in lexicographic ordering, returns zero if the two strings are equal, and returns a positive number if `s2` comes before `s1`. Thus,

```
s1.compareTo(s2) < 0
```

returns true if `s1` comes before `s2` in lexicographic order and returns false otherwise. For example, the following will produce correct output:

```
if (s1.compareTo(s2) < 0)
    System.out.println(
        s1 + " precedes " + s2 + " in lexicographic ordering");
else if (s1.compareTo(s2) < 0)
    System.out.println(
        s1 + " follows " + s2 + " in lexicographic ordering");
else //s1.compareTo(s2) == 0
    System.out.println(s1 + " equals " + s2);
```

If you look at the ordering of characters in Appendix 3, you will see that *all* uppercase letters come before *all* lowercase letters. For example, 'z' comes before 'a' in lexicographic order. So when comparing two strings consisting of a mix of lower- and uppercase letters, lexicographic and alphabetic ordering are not the same. However, as

shown in Appendix 3, all the lowercase letters are in alphabetic order. So for any two strings of all lowercase letters, lexicographic order is the same as ordinary alphabetic order. Similarly, in the ordering of Appendix 3, all the uppercase letters are in alphabetic order. So for any two strings of all uppercase letters, lexicographic order is the same as ordinary alphabetic order. Thus, if you treat all uppercase letters as if they were lowercase, then lexicographic ordering becomes the same as alphabetic ordering. This is exactly what the method `compareToIgnoreCase` does. Thus, the following produces correct output:

`compareToIgnoreCase`

```
if (s1.compareToIgnoreCase(s2) < 0)
    System.out.println(
        s1 + " precedes " + s2 + " in ALPHABETIC ordering");
else if (s1.compareToIgnoreCase(s2) > 0)
    System.out.println(
        s1 + " follows " + s2 + " in ALPHABETIC ordering");
else //s1.compareToIgnoreCase(s2) == 0
    System.out.println(s1 + " equals " + s2 + " IGNORING CASE");
```

The above code will compile and produce results no matter what characters are in the strings `s1` and `s2`. However, alphabetic order and the output make sense only if the two strings consist entirely of letters.

The program in Display 3.4 illustrates some of the string comparisons we have just discussed.

Self-Test Exercises

13. Suppose `n1` and `n2` are two `int` variables that have been given values. Write a Boolean expression that returns `true` if the value of `n1` is greater than or equal to the value of `n2`; otherwise, it should return `false`.
14. Suppose `n1` and `n2` are two `int` variables that have been given values. Write an `if-else` statement that outputs "`n1`" if `n1` is greater than or equal to `n2`, and that outputs "`n2`" otherwise.
15. Suppose `variable1` and `variable2` are two variables that have been given values. How do you test whether they are equal when the variables are of type `int`? How do you test whether they are equal when the variables are of type `String`?
16. Assume that `nextWord` is a `String` variable that has been given a `String` value consisting entirely of letters. Write some Java code that outputs the message "First half of the alphabet", provided `nextWord` precedes "N" in alphabetic ordering. If `nextWord` does not precede "N" in alphabetic ordering, the code should output "Second half of the alphabet". (Note that "N" uses double quotes to produce a `String` value, as opposed to using single quotes to produce a `char` value.)

Display 3.4 Comparing Strings

```
1 public class StringComparisonDemo
2 {
3     public static void main(String[] args)
4     {
5         String s1 = "Java isn't just for breakfast.";
6         String s2 = "JAVA isn't just for breakfast.";
7
8         if (s1.equals(s2))
9             System.out.println("The two lines are equal.");
10        else
11            System.out.println("The two lines are not equal.");
12        if (s2.equals(s1))
13            System.out.println("The two lines are equal.");
14        else
15            System.out.println("The two lines are not equal.");
16        if (s1.equalsIgnoreCase(s2))
17            System.out.println(
18                "But the lines are equal, ignoring case.");
19        else
20            System.out.println(
21                "Lines are not equal, even ignoring case.");
22
23        String s3 = "A cup of java is a joy forever.";
24        if (s3.compareToIgnoreCase(s1) < 0)
25        {
26            System.out.println "\"" + s3 + "\"";
27            System.out.println("precedes");
28            System.out.println "\"" + s1 + "\"";
29            System.out.println("in alphabetic ordering");
30        }
31        else
32            System.out.println("s3 does not precede s1.");
33    }
34 }
```

Sample Dialogue

```
The two lines are not equal.
The two lines are not equal.
But the lines are equal, ignoring case.
"A cup of java is a joy forever."
precedes
"Java isn't just for breakfast."
in alphabetic ordering
```

Building Boolean Expressions

**&& means
“and”**

You can combine two Boolean expressions using the “and” operator, which is spelled `&&` in Java. For example, the following Boolean expression is `true` provided `number` is greater than 2 and `number` is less than 7:

```
(number > 2) && (number < 7)
```

When two Boolean expressions are connected using `&&`, the entire expression is `true`, provided both of the smaller Boolean expressions are `true`; otherwise, the entire expression is `false`.

The “and” Operator `&&`

You can form a more elaborate Boolean expression by combining two simpler Boolean expressions using the “and” operator `&&`.

SYNTAX (FOR A BOOLEAN EXPRESSION USING `&&`)

```
(Boolean_Exp_1) && (Boolean_Exp_2)
```

EXAMPLE (WITHIN AN `if-else` STATEMENT)

```
if ( (score > 0) && (score < 10) )
    System.out.println("score is between 0 and 10.");
else
    System.out.println("score is not between 0 and 10.");
```

If the value of `score` is greater than 0 and the value of `score` is also less than 10, then the first `System.out.println` statement is executed; otherwise, the second `System.out.println` statement is executed.

**|| means
“or”**

You can also combine two Boolean expressions using the “or” operator, which is spelled `||` in Java. For example, the following is `true` provided `count` is less than 3 or `count` is greater than 12:

```
(count < 3) || (count > 12)
```

When two Boolean expressions are connected using `||`, the entire expression is `true`, provided that one or both of the smaller Boolean expressions are `true`; otherwise, the entire expression is `false`.

You can negate any Boolean expression using the `!` operator. If you want to negate a Boolean expression, place the expression in parentheses and place the `!` operator in front of it. For example, `!(savings < debt)` means “savings is not less than debt.” The `!` operator can usually be avoided. For example,

```
!(savings < debt)
```

is equivalent to `savings >= debt`. In some cases, you can safely omit the parentheses, but the parentheses never do any harm. The exact details on omitting parentheses are given later in this chapter in the subsection entitled “Precedence and Associativity Rules.”

The “or” Operator `||`

You can form a more elaborate Boolean expression by combining two simpler Boolean expressions using the “or” operator `||`.

SYNTAX (FOR A BOOLEAN EXPRESSION USING `||`)

```
(Boolean_Exp_1) || (Boolean_Exp_2)
```

EXAMPLE (WITHIN AN `if-else` STATEMENT)

```
if ((salary > expenses) || (savings > expenses))
    System.out.println("Solvent");
else
    System.out.println("Bankrupt");
```

If salary is greater than expenses or savings is greater than expenses (or both), then the first `System.out.println` statement is executed; otherwise, the second `System.out.println` statement is executed.



PITFALL: Strings of Inequalities

Do not use a string of inequalities such as `min < result < max`. If you do, your program will produce a compiler error message. Instead, you must use two inequalities connected with an `&&`, as follows:

```
(min < result) && (result < max) ■
```

Self-Test Exercises

17. Write an `if-else` statement that outputs the word “Passed” provided the value of the variable `exam` is greater than or equal to 60 and also the value of the variable `programsDone` is greater than or equal to 10. Otherwise, the `if-else` statement should output the word “Failed”. The variables `exam` and `programsDone` are both of type `int`.
18. Write an `if-else` statement that outputs the word “Emergency” provided the value of the variable `pressure` is greater than 100 or the value of the variable `temperature` is greater than or equal to 212. Otherwise, the `if-else` statement should output the word “OK”. The variables `pressure` and `temperature` are both of type `int`.

Evaluating Boolean Expressions

Boolean expressions are used to control branch and loop statements. However, a Boolean expression has an independent identity apart from any branch statement or loop statement you might use it in. A Boolean expression returns either `true` or `false`.

A variable of type `boolean` can store the values `true` and `false`. Thus, you can set a variable of type `boolean` equal to a Boolean expression. For example,

```
boolean madeIt = (time < limit) && (limit < max);
```

A Boolean expression can be evaluated in the same way that an arithmetic expression is evaluated. The only difference is that an arithmetic expression uses operations such as `+`, `*`, and `/` and produces a number as the final result, whereas a Boolean expression uses relational operations such as `==` and `<` and Boolean operations such as `&&`, `||`, and `!`, and produces one of the two values `true` and `false` as the final result.

First, let's review evaluating an arithmetic expression. The same technique will work in the same way to evaluate Boolean expressions. Consider the following arithmetic expression:

```
(number + 1) * (number + 3)
```

Assume that the variable `number` has the value 2. To evaluate this arithmetic expression, you evaluate the two sums to obtain the numbers 3 and 5, and then you combine these two numbers 3 and 5 using the `*` operator to obtain 15 as the final value. Notice that in performing this evaluation, you do not multiply the expressions `(number + 1)` and `(number + 3)`. Instead, you multiply the values of these expressions. You use 3; you do not use `(number + 1)`. You use 5; you do not use `(number + 3)`.

The computer evaluates Boolean expressions the same way. Subexpressions are evaluated to obtain values, each of which is either `true` or `false`. In particular, `==`, `!=`, `<`, `<=`, and so forth operate on pairs of any primitive type to produce a Boolean value of `true` or `false`. These individual values of `true` or `false` are then combined according to the rules in the **truth tables** shown in Display 3.5. For example, consider the Boolean expression

```
!( ( count < 3 ) || (count > 7) )
```

which might be the controlling expression for an `if-else` statement. Suppose the value of `count` is 8. In this case, `(count < 3)` evaluates to `false` and `(count > 7)` evaluates to `true`, so the preceding Boolean expression is equivalent to

```
!(false || true)
```

Consulting the tables for `||` (which is labeled “OR”), the computer sees that the expression inside the parentheses evaluates to `true`. Thus, the computer sees that the entire expression is equivalent to

```
!(true)
```

Consulting the tables again, the computer sees that `!(true)` evaluates to `false`, and so it concludes that `false` is the value of the original Boolean expression.

truth tables

The boolean Values Are true and false

`true` and `false` are predefined constants of type `boolean`. (They must be written in lowercase.) In Java, a Boolean expression evaluates to the `boolean` value `true` when it is satisfied, and it evaluates to the `boolean` value `false` when it is not satisfied.

Display 3.5 Truth Tables

AND		
Exp_1	Exp_2	Exp_1 && Exp_2
true	true	true
true	false	false
false	true	false
false	false	false
OR		
Exp_1	Exp_2	Exp_1 Exp_2
true	true	true
true	false	true
false	true	true
false	false	false

NOT	
Exp	! (Exp)
true	false
false	true

boolean
variables in
assignments

A boolean variable—that is, one of type `boolean`—can be given the value of a Boolean expression by using an assignment statement, in the same way that you use an assignment statement to set the value of an `int` variable or any other type of variable. For example, the following sets the value of the `boolean` variable `isPositive` to `false`:

```
int number = -5;
boolean isPositive;
isPositive = (number > 0);
```

If you prefer, you can combine the last two lines as follows:

```
boolean isPositive = (number > 0);
```

The parentheses are not needed, but they do make it a bit easier to read. Once a `boolean` variable has a value, you can use the `boolean` variable just as you would use any other Boolean expression. For example, the following code

```
boolean isPositive = (number > 0);
if (isPositive)
    System.out.println("The number is positive.");
else
    System.out.println("The number is negative or zero.");
```

is equivalent to

```
if (number > 0)
    System.out.println("The number is positive.");
else
    System.out.println("The number is negative or zero.");
```

Of course, this is just a toy example. It is unlikely that anybody would use the first of the preceding two examples, but you might use something like it if the value of `number`, and therefore the value of the Boolean expression, might change. For example, the following code could (by some stretch of the imagination) be part of a program to evaluate lottery tickets:

```
boolean isPositive = (number > 0);
while (number > 0);
{
    System.out.println("Wow!");
    number = number - 1000;
}
if (isPositive)
    System.out.println("Your number is positive.");
else
    System.out.println("Sorry, number is not positive.");
System.out.println("Only positive numbers can win.");
```

true and false Are Not Numbers

Many programming languages traditionally use 1 and 0 for true and false. The latest versions of most languages have changed things so that now most languages have a type such as `boolean` with values for true and false. However, even in these newer language versions, values of type `boolean` are automatically converted to integers and vice versa when context requires it. In particular, C++ automatically makes such conversions.

In Java, the values `true` and `false` are not numbers, nor can they be type cast to any numeric type. Similarly, values of type `int` cannot be type cast to `boolean` values.



TIP: Naming Boolean Variables

Name a `boolean` variable with a statement that will be true when the value of the `boolean` variable is true, such as `isPositive`, `pressureOK`, and so forth. That way you can easily understand the meaning of the `boolean` variable when it is used in an `if-else` statement or other control statement. Avoid names that do not unambiguously describe the meaning of the variable's value. Do not use names such as `numberSign`, `pressureStatus`, and so forth. ■

Short-Circuit and Complete Evaluation

Java takes an occasional shortcut when evaluating a Boolean expression. Notice that in many cases, you need to evaluate only the first of two or more subexpressions in a Boolean expression. For example, consider the following:

```
(savings >= 0) && (dependents > 1)
```

If `savings` is negative, then `(savings >= 0)` is `false`, and, as you can see in the tables in Display 3.5, when one subexpression in an `&&` expression is `false`, then the whole expression is `false`, no matter whether the other expression is `true` or `false`. Thus, if we know that the first expression is `false`, there is no need to evaluate the second expression. A similar thing happens with `||` expressions. If the first of two expressions joined with the `||` operator is `true`, then you know the entire expression is `true`, whether the second expression is `true` or `false`. In some situations, the Java language can and does use these facts to save itself the trouble of evaluating the second subexpression in a logical expression connected with an `&&` or an `||`. Java first evaluates the leftmost of the two expressions joined by an `&&` or an `||`. If that gives it enough information to determine the final value of the expression (independent of the value of the second expression), then Java does not bother to evaluate the second expression. This method of evaluation is called **short-circuit evaluation** or **lazy evaluation**.

Now let's look at an example using `&&` that illustrates the advantage of short-circuit evaluation, and let's give the Boolean expression some context by placing it in an `if` statement:

```
if ( (kids != 0) && ((pieces/kids) >= 2) )  
    System.out.println("Each child may have two pieces!");
```

If the value of `kids` is not zero, this statement involves no subtleties. However, suppose the value of `kids` is zero and consider how short-circuit evaluation handles this case. The expression `(kids != 0)` evaluates to `false`, so there would be no need to evaluate the second expression. Using short-circuit evaluation, Java says that the entire expression is `false`, without bothering to evaluate the second expression. This prevents a run-time error, since evaluating the second expression would involve dividing by zero.

Java also allows you to ask for **complete evaluation**. In complete evaluation, when two expressions are joined by an “and” or an “or,” both subexpressions are always evaluated, and then the truth tables are used to obtain the value of the final expression. To obtain complete evaluation in Java, you use `&` rather than `&&` for “and” and use `|` in place of `||` for “or.”

In most situations, short-circuit evaluation and complete evaluation give the same result, but, as you have just seen, there are times when short-circuit evaluation can avoid a run-time error. There are also some situations in which complete evaluation is preferred, but we will not use those techniques in this book. We will always use `&&` and `||` to obtain short-circuit evaluation.

short-circuit
evaluation

lazy
evaluation

complete
evaluation

Precedence and Associativity Rules

precedence
rules

associativity
rules

higher
precedence

Boolean expressions (and arithmetic expressions) need not be fully parenthesized. If you omit parentheses, Java follows **precedence rules** and **associativity rules** in place of the missing parentheses. One easy way to think of the process is to think of the computer adding parentheses according to these precedence and associativity rules. Some of the Java precedence and associativity rules are given in Display 3.6. (A complete set of precedence and associativity rules is given in Appendix 2.) The computer uses precedence rules to decide where to insert parentheses, but the precedence rules do not differentiate between two operators at the same precedence level, in which case the computer uses the associativity rules to “break the tie.”

If one operator occurs higher on the list than another in the precedence table (Display 3.6), the higher one is said to have **higher precedence**. If one operator has higher precedence than another, the operator of higher precedence is grouped with its operands (its arguments) before the operator of lower precedence. For example, if the computer is faced with the expression

```
balance * rate + bonus
```

it notices that `*` has a higher precedence than `+`, so it first groups `*` and its operands, as follows:

```
(balance * rate) + bonus
```

Next, it groups `+` with its operands to obtain the fully parenthesized expression

```
((balance * rate) + bonus)
```

Sometimes two operators have the same precedence, in which case the parentheses are added using the associativity rules. To illustrate this, let’s consider another example:

```
bonus + balance * rate / correctionFactor - penalty
```

The operators `*` and `/` have higher precedence than either `+` or `-`, so `*` and `/` are grouped first. But `*` and `/` have equal precedence, so the computer consults the associativity rule for `*` and `/`, which says they associate from left to right. This means that the `*`, which is the leftmost of `*` and `/`, is grouped first. So the computer interprets the expression as

```
bonus + (balance * rate) / correctionFactor - penalty
```

which in turn is interpreted as

```
bonus + ((balance * rate) / correctionFactor) - penalty
```

because `/` has higher precedence than either `+` or `-`.

This expression is still not fully parenthesized, however. The computer still must choose to group `+` first or `-` first. According to Display 3.6, `+` and `-` have equal precedence. So the computer must use the associativity rules, which say that `+` and `-` are associated left to right. So, it interprets the expression as

```
(bonus + ((balance * rate) / correctionFactor)) - penalty
```

Display 3.6 Precedence and Associativity Rules

Highest
Precedence

Lowest
Precedence

PRECEDENCE	ASSOCIATIVITY
From highest at top to lowest at bottom. Operators in the same group have equal precedence.	
Dot operator, array indexing, and method invocation ., [], ()	Left to right
++ (postfix, as in x++), -- (postfix)	Right to left
The unary operators: +, -, ++ (prefix, as in ++x), -- (prefix), and !	Right to left
Type casts (Type)	Right to left
The binary operators *, /, %	Left to right
The binary operators +, -	Left to right
The binary operators <, >, <=, >=	Left to right
The binary operators ==, !=	Left to right
The binary operator &	Left to right
The binary operator	Left to right
The binary operator &&	Left to right
The binary operator	Left to right
The ternary operator (conditional operator) ?:	Right to left
The assignment operators =, *=, /=, %=, +=, -=, &=, =	Right to left

which in turn is interpreted as the following fully parenthesized expression:

```
((bonus + ((balance * rate) / correctionFactor)) - penalty)
```

As you can see from studying the table in Display 3.6, most binary operators associate from left to right. But the assignment operators associate from right to left. So the expression

```
number1 = number2 = number3
```

means

```
number1 = (number2 = number3)
```

which in turn is interpreted as the following fully parenthesized expression:

```
(number1 = (number2 = number3))
```

However, this fully parenthesized expression may not look like it means anything until we explain a bit more about the assignment operator.

Although we do not advocate using the assignment operator `=` as part of a complex expression, it is an operator that returns a value, just as `+` and `*` do. When an assignment operator `=` is used in an expression, it changes the value of the variable on the left-hand side of the assignment operator and also returns a value—namely, the new value of the variable on the left-hand side of the expression. So `(number2 = number3)` sets `number2` equal to the value of `number3` and returns the value of `number3`. Thus,

```
number1 = number2 = number3
```

which is equivalent to

```
(number1 = (number2 = number3))
```

sets both `number2` and `number1` equal to the value of `number3`. It is best to not use assignment statements inside of expressions, although simple chains of assignment operators such as the following are clear and acceptable:

```
number1 = number2 = number3;
```

Although we discourage using expressions that combine the assignment operator and other operators in complicated ways, let's try to parenthesize one just for practice. Consider the following:

```
number1 = number2 = number3 + 7 * factor
```

The operator of highest precedence is `*`, and the operator of next-highest precedence is `+`, so this expression is equivalent to

```
number1 = number2 = (number3 + (7 * factor))
```

which leaves only the assignment operators to group. They associate right to left, so the fully parenthesized equivalent version of our expression is

```
(number1 = (number2 = (number3 + (7 * factor))))
```

(Note that there is no case where two operators have equal precedence, but one associates from left to right while the other associates from right to left. That must be true, or else there would be cases with conflicting instructions for inserting parentheses.)

binding

The association of operands with operators is called **binding**. For example, when parentheses determine which two expressions (two operands) are being added by a particular `+` sign, that is called binding the two operands to the `+` sign. A fully parenthesized expression accomplishes binding for all the operators in an expression.

These examples should make it clear that it can be risky to depend too heavily on the precedence and associativity rules. It is best to include most parentheses and to omit parentheses only in situations where the intended meaning is very obvious, such as a simple combination of `*` and `+`, or a simple chain of `&&`'s or a simple chain of `||`'s. The following examples have some omitted parentheses, but their meaning should be clear:

```
rate * time + lead
(time < limit) && (yourScore > theirScore) && (yourScore > 0)
(expenses < income) || (expenses < savings) || (creditRating > 0)
```

Notice that the precedence rules include both arithmetic operators such as `+` and `*` as well as Boolean operators such as `&&` and `||`. This is because many expressions combine arithmetic and Boolean operations, as in the following simple example:

```
(number + 1) > 2 || (number + 5) < -3
```

If you check the precedence rules given in Display 3.6, you will see that this expression is equivalent to

```
((number + 1) > 2) || ((number + 5) < (-3))
```

because `>` and `<` have higher precedence than `||`. In fact, you could omit all the parentheses in the above expression and it would have the same meaning (but would be less clear).

It might seem that once an expression is fully parenthesized, the meaning of the expression is then determined. It would seem that to evaluate the expression, you (or the computer) simply evaluate the inner expressions before the outer ones. So, in

```
((number + 1) > 2) || ((number + 5) < (-3))
```

first the expressions `(number + 1)`, `(number + 5)`, and `(-3)` are evaluated (in any order), then the `>` and `<` are evaluated, and then the `||` is applied. That happens to work in this simple case. In this case, it does not matter which of `(number + 1)`, `(number + 5)`, and `(-3)` is evaluated first, but in certain other expressions it will be necessary to specify which subexpression is evaluated first. The rules for evaluating a fully parenthesized expression are (and indeed must be) more complicated than just evaluating inner expressions before outer expressions.

For an expression with no side effects, the rule of performing inner parenthesized expressions before outer ones is all you need. That rule will get you through most simple expressions, but for expressions with side effects, you need to learn the rest of the story, which is what we will do next.

side effects The complications come from the fact that some expressions have *side effects*. When we say an expression has **side effects**, we mean that in addition to returning a value, the expression also changes something, such as the value of a variable. Expressions with the assignment operator have side effects; `pay = bonus`, for example, changes the value of `pay`. Increment and decrement operators have side effects; `++n` changes the value of `n`. In expressions that include operators with side effects, you need more rules.

For example, consider

```
((result = (++n)) + (other = (2*(++n))))
```

The parentheses seem to say that you or the computer should first evaluate the two increment operators, `++n` and `++n`, but the parentheses do not say which of the two `++n`'s to do first. If `n` has the value 2 and we evaluate the leftmost `++n` first, then

the variable `result` is set to 3 and the variable `other` is set to 8 (and the entire expression evaluates to 11). But if we evaluate the rightmost `++n` first, then `other` is set to 6 and `result` is set to 4 (and the entire expression evaluates to 10). We need a rule to determine the order of evaluation when we have a tie such as this. However, rather than simply adding a rule to break such ties, Java instead takes a completely different approach.

To evaluate an expression, Java uses the following three rules:

1. Java first does binding; that is, it first fully parenthesizes the expression using precedence and associativity rules, just as we have outlined.
2. Then it simply evaluates expressions left to right.
3. If an operator is waiting for its two (or one or three) operands to be evaluated, then that operator is evaluated as soon as its operands have been evaluated.

We will first do an example with no side effects and then an example of an expression with side effects. First, the simple example; consider the expression

$$6 + 7 * n - 12$$

and assume the value of `n` is 2. Using the precedence and associativity rules, we add parentheses one pair at a time as follows:

$$6 + (7 * n) - 12$$

then

$$(6 + (7 * n)) - 12$$

and finally the fully parenthesized version

$$((6 + (7 * n)) - 12)$$

Next, we evaluate subexpressions left to right. (6 evaluates to 6 and 7 evaluates to 7, but that is so obvious we will not make a big deal of it.) The variable `n` evaluates to 2. (Remember, we assumed the value of `n` was 2.) So, we can rewrite the expression as

$$((6 + (7 * 2)) - 12)$$

The `*` is the only operator that has both of its operands evaluated, so it evaluates to 14 to produce

$$((6 + 14) - 12)$$

Now `+` has both of its operands evaluated, so `(6 + 14)` evaluates to 20 to yield

$$(20 - 12)$$

which in turn evaluates to 8. So 8 is the value for the entire expression.

This may seem like more work than it should be, but remember, the computer is following an algorithm and proceeds step by step; it does not get inspired to make simplifying assumptions.

Next, let's consider an expression with side effects. In fact, let's consider the one we fully parenthesized earlier. Consider the following fully parenthesized expression and assume the value of `n` is 2:

```
((result = (++n)) + (other = (2*(++n))))
```

Subexpressions are evaluated left to right. So, `result` is evaluated first. When used with the assignment operator `=`, a variable simply evaluates to itself. So, `result` is evaluated and waiting. Next, `++n` is evaluated, and it returns the value 3. The expression is now known to be equivalent to

```
((result = 3) + (other = (2*(++n))))
```

Now the assignment operator `=` has its two operands evaluated, so `(result = 3)` is evaluated. Evaluating `(result = 3)` sets the value of `result` equal to 3 and returns the value 3. Thus, the expression is now known to be equivalent to

```
(3 + (other = (2*(++n))))
```

(and the side effect of setting `result` equal to 3 has happened). Proceeding left to right, the next thing to evaluate is the variable `other`, which simply evaluates to itself, so you need not rewrite anything.

Proceeding left to right, the next subexpression that can be evaluated is `n`, which evaluates to 3. (Remember, `n` has already been incremented once, so `n` now has the value 3.) Then `++` has its only argument evaluated, so it is ready to be evaluated. The evaluation of `(++n)` has the side effect of setting `n` equal to 4 and evaluates to 4. So, the entire expression is equivalent to

```
(3 + (other = (2*4)))
```

The only subexpression that has its operands evaluated is `(2*4)`, so it is evaluated to 8 to produce

```
(3 + (other = 8))
```

Now the assignment operator `=` has both of its operands evaluated, so it evaluates to 8 and has the side effect of setting `other` equal to 8. Thus, we know the value of the expression is

```
(3 + 8)
```

which evaluates to 11. So, the entire expression evaluates to 11 (and has the side effects of setting `result` equal to 3, setting `n` equal to 4, and setting `other` equal to 8). These rules also allow for method invocations in expressions. For example, in

```
(++n > 0) && (s.length() > n)
```

the variable `n` is incremented before `n` is compared to `s.length()`. When we start defining and using more methods, you will see less-contrived examples of expressions that include method invocations.

All of these rules for evaluating expressions are summarized in the box entitled “Rules for Evaluating Expressions.”

Rules for Evaluating Expressions

Expressions are evaluated as follows:

1. Binding: Determine the equivalent fully parenthesized expression using the precedence and associativity rules.
2. Proceeding left to right, evaluate whatever subexpressions you can evaluate. (These subexpressions will be operands or method arguments. For example, in simple cases they may be numeric constants or variables.)
3. Evaluate each outer operation (and method invocation) as soon as all of its operands (all its arguments) have been evaluated.

Self-Test Exercises

19. Determine the value, `true` or `false`, of each of the following Boolean expressions, assuming that the value of the variable `count` is 0 and the value of the variable `limit` is 10. (Give your answer as one of the values `true` or `false`.)

a. `(count == 0) && (limit < 20)`

b. `count == 0 && limit < 20`

c. `(limit > 20) || (count < 5)`

d. `!(count == 12)`

e. `(count == 1) && (x < y)`

f. `(count < 10) || (x < y)`

g. `!((count < 10) || (x < y)) && (count >= 0)`

h. `((limit/count) > 7) || (limit < 20)`

i. `(limit < 20) || ((limit/count) > 7)`

j. `((limit/count) > 7) && (limit < 0)`

k. `(limit < 0) && ((limit/count) > 7)`

20. Does the following sequence produce a division by zero?

```
int j = -1;
if ((j > 0) && (1/(j+1) > 10))
    System.out.println(i);
```

21. Convert the following expression to an equivalent fully parenthesized expression:

```
bonus + day * rate / correctionFactor * newGuy - penalty
```

3.3 Loops

Few tasks are more like the torture of Sisyphus than housework, with its endless repetition: the clean becomes soiled, the soiled is made clean, over and over, day after day.

SIMONE DE BEAUVOIR

body of
the loop

iteration

while and
do-while
compared

Looping mechanisms in Java are similar to those in other high-level languages. The three Java loop statements are the `while` statement, the `do-while` statement, and the `for` statement. The same terminology is used with Java as with other languages. The code that is repeated in a loop is called the **body of the loop**. Each repetition of the loop body is called an **iteration** of the loop.

while Statement and do-while Statement

The syntax for the `while` statement and its variant, the `do-while` statement, is given later in this chapter in the box entitled “Syntax for `while` and `do-while` Statements.” In both cases, the multistatement body is a special case of the loop with a single-statement body. The multistatement body is a single compound statement. Examples of `while` and `do-while` statements are given in Display 3.7.

Display 3.7 Demonstration of `while` Loops and `do-while` Loops (part 1 of 2)

```

1 public class WhileDemo
2 {
3     public static void main(String[] args)
4     {
5         int countDown;

6         System.out.println("First while loop:");
7         countDown = 3;
8         while (countDown > 0)
9         {
10            System.out.println("Hello");
11            countDown = countDown - 1;
12        }

13        System.out.println("Second while loop:");
14        countDown = 0;
15        while (countDown > 0)
16        {
17            System.out.println("Hello");
18            countDown = countDown - 1;
19        }

20        System.out.println("First do-while loop:");
21        countDown = 3;
22        do
23        {

```

Display 3.7 Demonstration of while Loops and do-while Loops (part 2 of 2)

```

24         System.out.println("Hello");
25         countDown = countDown - 1;
26     } while (countDown > 0);

27     System.out.println("Second do-while loop:");
28     countDown = 0;
29     do
30     {
31         System.out.println("Hello");
32         countDown = countDown - 1;
33     } while (countDown > 0);
34 }
35 }

```

Sample Dialogue

```

First while loop:
Hello
Hello
Hello
Second while loop:
First do-while loop:
Hello
Hello
Hello
Second do-while loop:
Hello

```

A while loop can iterate its body zero times.

A do-while loop always iterates its body at least one time.

The important difference between the while and do-while loops is when the controlling Boolean expression is checked. With a while statement, the Boolean expression is checked *before* the loop body is executed. If the Boolean expression evaluates to false, then the body is not executed at all. With a do-while statement, the body of the loop is executed first, and the Boolean expression is checked *after* the loop body is executed. Thus, the do-while statement always executes the loop body at least once. After this start-up, the while loop and the do-while loop behave the same way. After each iteration of the loop body, the Boolean expression is again checked, and if it is true, the loop is iterated again. If it has changed from true to false, then the loop statement ends.

executing
the body
zero times

The first thing that happens when a while loop is executed is that the controlling Boolean expression is evaluated. If the Boolean expression evaluates to false at that point, the body of the loop is never executed. It might seem pointless to execute the body of a loop zero times, but that is sometimes the desired action. For example, a while loop is often used to sum a list of numbers, but the list could be empty. To be more specific, a checkbook-balancing program might use a while loop to sum the values of all the checks you have written in a month, but you might take a month's vacation and write no checks at all. In that case, there are zero numbers to sum, so the loop is iterated zero times.

Anything that you can write with a `while` loop can be written with a `do-while` loop and vice versa. Given the following structure for a `while` loop:

```
while (Boolean condition)
{
    Statements;
}
```

the equivalent `do-while` loop is

```
if (Boolean condition)
{
    do
    {
        Statements;
    } while (Boolean condition);
}
```

The `if` statement is needed in case the Boolean condition is initially false and the loop is never entered. In the other direction, given the following structure for a `do-while` loop:

```
do
{
    Statements;
} while (Boolean condition);
```

the equivalent `while` loop is

```
Statements;
while (Boolean condition)
{
    Statements;
}
```

The statements in the body must be replicated before the loop, since the `do-while` loop will always execute the body at least once.

Algorithms and Pseudocode

algorithm

Dealing with the syntax rules of a programming language is not the hard part of solving a problem with a computer program. The hard part is coming up with the underlying method of solution. This method of solution is called an algorithm. An **algorithm** is a set of precise instructions that leads to a solution. Some approximately equivalent words to *algorithm* are *recipe*, *method*, *directions*, *procedure*, and *routine*.

pseudocode

An algorithm is normally written in a mixture of a programming language (in our case, Java) and English (or other human language). This mixture of programming language and human language is known as **pseudocode**. Using pseudocode frees you from worrying about fine details of Java syntax so that you can concentrate on the method of solution. Underlying the program in Display 3.8 is an algorithm that can be expressed as the following pseudocode:

Syntax for while and do-while Statements

A while STATEMENT WITH A SINGLE-STATEMENT BODY

```
while (Boolean_Expression)
    Statement
```

A while STATEMENT WITH A MULTISTatement BODY

```
while (Boolean_Expression)
{
    Statement_1
    Statement_2
    .
    .
    .
    Statement_Last
}
```

A do-while STATEMENT WITH A SINGLE-STATEMENT BODY

```
do
    Statement
while (Boolean Expression);
```

A do-while STATEMENT WITH A MULTISTatement BODY

```
do
{
    Statement_1
    Statement_2
    .
    .
    .
    Statement_Last
} while (Boolean_Expression);
```

*Do not forget the
final semicolon.*

```
Give the user instructions.
count = 0;
sum = 0;
Read a number and store it in a variable named next.
while (next >= 0)
{
    sum = sum + next;
    count++;
    Read a number and store it in next.
}
The average is sum/count provided count is not zero.
Output the results.
```

Display 3.8 Averaging a List of Scores

```
1  import java.util.Scanner;
2  public class Averager
3  {
4      public static void main(String[] args)
5      {
6          Scanner keyboard = new Scanner(System.in);
7          System.out.println("Enter a list of nonnegative scores.");
8          System.out.println("Mark the end with a negative number.");
9          System.out.println("I will compute their average.");

10         double next, sum = 0;
11         int count = 0;

12         next = keyboard.nextDouble( );
13         while (next >= 0)
14         {
15             sum = sum + next;
16             count++;
17             next = keyboard.nextDouble( );
18         }

19         if (count == 0)
20             System.out.println("No scores entered.");
21         else
22         {
23             double average = sum/count;
24             System.out.println(count + " scores read.");
25             System.out.println("The average is " + average);
26         }
27     }
28 }
```

Sample Dialogue

```
Enter a list of nonnegative scores.
Mark the end with a negative number.
I will compute their average.
87.5 0 89 99.9 -1
4 scores read.
The average is 69.1.
```

Note that when using pseudocode, we do not necessarily declare variables or worry about the fine syntax details of Java. The only rule is that the pseudocode must be precise and clear enough for a good programmer to convert the pseudocode to syntactically correct Java code.

As you will see, significant programs are written not as a single algorithm, but as a set of interacting algorithms; however, each of these algorithms is normally designed in pseudocode unless the algorithm is exceedingly simple.

EXAMPLE: Averaging a List of Scores

Display 3.8 shows a program that reads in a list of scores and computes their average. It illustrates a number of techniques that are commonly used with loops.

sentinel
value

The scores are all nonnegative. This allows the program to use a negative number as an end marker. Note that the negative number is not one of the numbers being averaged in. This sort of end marker is known as a **sentinel value**. A sentinel value need not be a negative number, but it must be some value that cannot occur as a “real” input value. For example, if the input list were a list of even integers, then you could use an odd integer as a sentinel value.

To get the loop to end properly, we want the Boolean expression

```
next >= 0
```

checked before adding in the number read. This way we avoid adding in the sentinel value. So, we want the loop body to end with

```
next = keyboard.nextDouble( );
```

To make things work out, this in turn requires that we also place this line before the loop. A loop often needs some preliminary statements to set things up before the loop is executed.

Self-Test Exercises

22. What is the output produced by the following?

```
int n = 10;
while (n > 0)
{
    System.out.println(n);
    n = n - 3;
}
```

23. What output would be produced in Self-Test Exercise 22 if the > sign were replaced with < ?

(continued)

Self-Test Exercises (continued)

24. What is the output produced by the following?

```
int n = 10;
do
{
    System.out.println(n);
    n = n - 3;
} while (n > 0);
```

25. What output would be produced in Self-Test Exercise 24 if the > sign were replaced with < ?

26. What is the output produced by the following?

```
int n = -42;
do
{
    System.out.println(n);
    n = n - 3;
} while (n > 0);
```

27. What is the most important difference between a while statement and a do-while statement?

The for Statement

for statement

The third and final loop statement in Java is the **for statement**. The for statement is most commonly used to step through some integer variable in equal increments. The for statement is, however, a completely general looping mechanism that can do anything that a while loop can do.

For example, the following for statement sums the integers 1 through 10:

```
sum = 0;
for (n = 1; n <= 10; n++)
    sum = sum + n;
```

A for statement begins with the keyword **for** followed by three expressions in parentheses that tell the computer what to do with the controlling variable(s). The beginning of a for statement looks like the following:

```
for (Initialization; Boolean_Expression; Update)
```

The first expression tells how the variable, variables, or other things are initialized, the second expression gives a Boolean expression that is used to check for when the loop should end, and the last expression tells how the loop control variable or variables are updated after each iteration of the loop body. The loop body is a single statement (typically a compound statement) that follows the heading we just described.

The three expressions at the start of a `for` statement are separated by two, and only two, semicolons. Do not succumb to the temptation to place a semicolon after the third expression. (The technical explanation is that these three things are expressions, not statements, and so do not require a semicolon at the end.)

A `for` statement often uses a single `int` variable to control loop iteration and loop ending. However, the three expressions at the start of a `for` statement may be any Java expressions and therefore may involve more (or even fewer) than one variable, and the variables can be of any type.

The semantics of the `for` statement are given in Display 3.9. The syntax for a `for` statement is given in Display 3.10. Display 3.10 also explains how the `for` statement can be viewed as a notational variant of the `while` loop.

The `for` Statement

SYNTAX

```
for (Initialization; Boolean_Expression; Update)  
    Body
```

The *Body* may be any Java statement—either a simple statement or, more likely, a compound statement consisting of a list of statements enclosed in braces, `{}`. Notice that the three things in parentheses are separated by two, not three, semicolons.

You are allowed to use any Java expression for the *Initializing* and the *Update* expressions. Therefore, you may use more, or fewer, than one variable in the expressions; moreover, the variables may be of any type.

EXAMPLE

```
int next, sum = 0;  
for (next = 0; next <= 10; next++)  
{  
    sum = sum + next;  
    System.out.println("sum up to " + next + " is " + sum);  
}
```

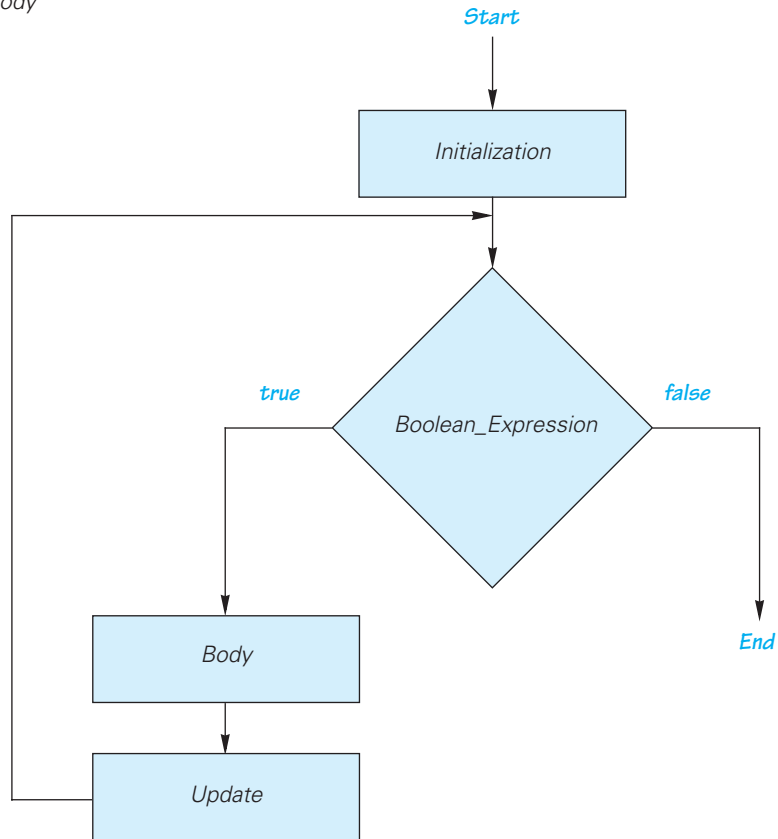
A variable can be declared in the heading of a `for` statement at the same time that it is initialized. For example,

```
for (int n = 1; n < 10; n++)  
    System.out.println(n);
```

There are some subtleties to worry about when you declare a variable in the heading of a `for` statement. These subtleties are discussed in Chapter 4 in the Programming Tip subsection entitled “Declaring Variables in a `for` Statement.” It might be wise to avoid such subtle declarations within a `for` statement until you reach Chapter 4, but we mention it here for reference value.

Display 3.9 Semantics of the `for` Statement

```
for (Initialization; Boolean_Expression; Update)  
    Body
```

Display 3.10 `for` Statement Syntax and Alternate Semantics (part 1 of 2)**for Statement Syntax**

```
for (Initialization; Boolean_Expression; Update)  
    Body
```

EXAMPLE

```
for (number = 100; number >= 0; number--)  
    System.out.println(number + " bottles of beer on the shelf.");
```

Display 3.10 `for` Statement Syntax and Alternate Semantics (part 2 of 2)

Equivalent `while` Loop Syntax

```
Initialization;  
while (Boolean_Expression)  
{  
    Body;  
    Update;  
}
```

EQUIVALENT EXAMPLE

```
number = 100;  
while (number >= 0)  
{  
    System.out.println(number + " bottles of beer on the shelf.");  
    number--;  
}
```

Sample Dialogue

```
100 bottles of beer on the shelf.  
99 bottles of beer on the shelf.  
.  
.  
.  
0 bottles of beer on the shelf.
```

The Comma in `for` Statements

A `for` loop can contain multiple initialization actions. Simply separate the actions with commas, as in the following:

```
for (term = 1, sum = 0; term <= 10; term++)  
    sum = sum + term;
```

This `for` loop has two initializing actions. The variable `term` is initialized to 1, and the variable `sum` is also initialized to 0. Note that you use a comma, not a semicolon, to separate the initialization actions.

You can also use commas to place multiple update actions in a `for` loop. This can lead to a situation in which the `for` loop has an empty body but still does something useful. For example, the previous `for` loop can be rewritten to the following equivalent version:

```
for (term = 1, sum = 0; term <= 10; sum = sum + term, term++)  
    //Empty body;
```

This, in effect, makes the loop body part of the update action. We find that it makes for a more readable style if you use the update action only for variables that control the loop, as in the previous version of this `for` loop. We do not advocate using `for` loops with no body, but if you do so, annotate it with a comment such as we did in the preceding `for` loop. As indicated in the upcoming Pitfall, “Extra Semicolon in a `for` Statement,” a `for` loop with no body can also occur as the result of a programmer error.

The comma used in a `for` statement, as we just illustrated, is quite limited in how it can be used. You can use it with assignment statements and with incremented and decremented variables (such as `term++` or `term--`), but not with just any arbitrary statements. In particular, both declaring variables and using the comma in `for` statements can be troublesome. For example, the following is illegal:

```
for (int term = 1, double sum = 0; term <= 10; term++)
    sum = sum + term;
```

Even the following is illegal:

```
double sum;
for (int term = 1, sum = 0; term <= 10; term++)
    sum = sum + term;
```

Java will interpret

```
int term = 1, sum = 0;
```

as declaring both `term` and `sum` to be `int` variables and complain that `sum` is already declared.

If you do not declare `sum` anywhere else (and it is acceptable to make `sum` an `int` variable instead of a `double` variable), then the following, although we discourage it, is legal:

```
for (int term = 1, sum = 0; term <= 10; term++)
    sum = sum + term;
```

The first part in parentheses (up to the semicolon) declares both `term` and `sum` to be `int` variables and initializes both of them.

It is best to simply avoid these possibly confusing examples. When using the comma in a `for` statement, it is safest to simply declare all variables outside the `for` statement. If you declare all variables outside the `for` loop, the rules are no longer complicated.

A `for` loop can have only one Boolean expression to test for ending the `for` loop. However, you can perform multiple tests by connecting the tests using `&&` or `||` operators to form one larger Boolean expression.

(C, C++, and some other programming languages have a general-purpose comma operator. Readers who have programmed in one of these languages need to be warned that, in Java, there is no comma operator. In Java, the comma is a separator, not an operator, and its use is very restricted compared with the comma operator in C and C++.)



TIP: Repeat *N* Times Loops

The simplest way to produce a loop that repeats the loop body a predetermined number of times is with a `for` statement. For example, the following is a loop that repeats its loop body three times:

```
for (int count = 1; count <= 3; count++)
    System.out.println("Hip, Hip, Hurray");
```

The body of a `for` statement need not make any reference to a loop control variable, such as the variable `count`. ■



PITFALL: Extra Semicolon in a `for` Statement

You normally do not place a semicolon after the closing parenthesis at the beginning of a `for` loop. To see what can happen, consider the following `for` loop:

```
for (int count = 1; count <= 10; count++); ← Problem
    System.out.println("Hello");           semicolon
```

If you did not notice the extra semicolon, you might expect this `for` loop to write `Hello` to the screen 10 times. If you did notice the semicolon, you might expect the compiler to issue an error message. Neither of those things happens. If you embed this `for` loop in a complete program, the compiler will not complain. If you run the program, only one `Hello` will be output instead of 10 `Hello`s. What is happening? To answer that question, we need a little background.

One way to create a statement in Java is to put a semicolon after something. If you put a semicolon after `number++`, you change the expression

```
number++
```

into the statement

```
number++;
```

If you place a semicolon after nothing, you still create a statement. Thus, the semicolon by itself is a statement, which is called the **empty statement** or the **null statement**. The empty statement performs no action, but still is a statement. Therefore, the following is a complete and legitimate `for` loop, whose body is the empty statement:

```
for (int count = 1; count <= 10; count++);
```

(continued)

**empty
statement**

**null
statement**

**PITFALL: (continued)**

This `for` loop is indeed iterated 10 times, but since the body is the empty statement, nothing happens when the body is iterated. This loop does nothing, and it does nothing 10 times! After completing this `for` loop, the computer goes on to execute the following, which writes `Hello` to the screen one time:

```
System.out.println("Hello");
```

This same sort of problem can arise with a `while` loop. Be careful to not place a semicolon after the closing parenthesis that encloses the Boolean expression at the start of a `while` loop. A `do-while` loop has just the opposite problem. You must remember to always end a `do-while` loop with a semicolon. ■

**PITFALL: Infinite Loops**

A `while` loop, `do-while` loop, or `for` loop does not terminate as long as the controlling Boolean expression evaluates to `true`. This Boolean expression normally contains a variable that will be changed by the loop body, and usually the value of this variable eventually is changed in a way that makes the Boolean expression false and therefore terminates the loop. However, if you make a mistake and write your program so that the Boolean expression is always true, then the loop will run forever. A loop that runs forever is called an **infinite loop**.

Unfortunately, examples of infinite loops are not hard to come by. First, let's describe a loop that *does* terminate. The following Java code writes out the positive even numbers less than 12. That is, it outputs the numbers 2, 4, 6, 8, and 10, one per line, and then the loop ends.

```
number = 2;
while (number != 12)
{
    System.out.println(number);
    number = number + 2;
}
```

The value of `number` is increased by 2 on each loop iteration until it reaches 12. At that point, the Boolean expression after the word `while` is no longer true, so the loop ends.

Now suppose you want to write out the odd numbers less than 12, rather than the even numbers. You might mistakenly think that all you need to do is change the initializing statement to

```
number = 1;
```

But this mistake will create an infinite loop. Because the value of `number` goes from 11 to 13, the value of `number` is never equal to 12, so the loop never terminates.

infinite loop



PITFALL: (continued)

This sort of problem is common when loops are terminated by checking a numeric quantity using `==` or `!=`. When dealing with numbers, it is always safer to test for passing a value. For example, the following will work fine as the first line of our `while` loop:

```
while (number < 12)
```

With this change, `number` can be initialized to any number, and the loop will still terminate.

There is one subtlety about infinite loops that you need to keep in mind. A loop might terminate for some input values but be an infinite loop for other values. Just because you tested your loop for some program input values and found that the loop ended does not mean that it will not be an infinite loop for some other input values.

A program that is in an infinite loop might run forever unless some external force stops it, so it is a good idea to learn how to force a program to terminate. The method for forcing a program to stop varies from operating system to operating system. The keystrokes Control-C will terminate a program on many operating systems. (To type Control-C, hold down the Control key while pressing the C key.)

In simple programs, an infinite loop is almost always an error. However, some programs are intentionally written to run forever, such as the main outer loop in an airline reservation program that just keeps asking for more reservations until you shut down the computer (or otherwise terminate the program in an atypical way). ■

Nested Loops

nested loops



VideoNote
Nested Loop
Example

It is perfectly legal to nest one loop statement inside another loop statement. For example, the following nests one `for` loop inside another `for` loop:

```
int rowNum, columnNum;
for (rowNum = 1; rowNum <= 3; rowNum++)
{
    for (columnNum = 1; columnNum <= 2; columnNum++)
        System.out.print(" row " + rowNum + " column " + columnNum);
    System.out.println( );
}
```

This produces the following output:

```
row 1 column 1 row 1 column 2
row 2 column 1 row 2 column 2
row 3 column 1 row 3 column 2
```

For each iteration of the outer loop, the inner loop is iterated from beginning to end and then one `println` statement is executed to end the line.

(It is best to avoid nested loops by placing the inner loop inside a method definition and placing a method invocation inside the outer loop. Method definitions are covered in Chapters 4 and 5.)

Self-Test Exercises

28. What is the output of the following?

```
for (int count = 1; count < 5; count++)  
    System.out.print((2 * count) + " ");
```

29. What is the output of the following?

```
for (int n = 10; n > 0; n = n - 2)  
    System.out.println("Hello " + n);
```

30. What is the output of the following?

```
for (double sample = 2; sample > 0; sample = sample - 0.5)  
    System.out.print(sample + " ");
```

31. Rewrite the following for statement as a while loop (and possibly some additional statements):

```
int n;  
for (n = 10; n > 0; n = n - 2)  
    System.out.println("Hello " + n);
```

32. What is the output of the following loop? Identify the connection between the value of `n` and the value of the variable `log`.

```
int n = 1024;  
int log = 0;  
for (int i = 1; i < n; i = i * 2)  
    log++;  
System.out.println(n + " " + log);
```

33. What is the output of the following loop? Comment on the code. (This is not the same as the previous exercise.)

```
int n = 1024;  
int log = 0;  
for (int i = 1; i < n; i = i * 2);  
    log++;  
System.out.println(n + " " + log);
```

34. Predict the output of the following nested loops:

```
int n, m;  
for (n = 1; n <= 10; n++)  
    for (m = 10; m >= 1; m--)  
        System.out.println(n + " times " + m  
                               + " = " + n*m);
```

Self-Test Exercises (continued)

35. For each of the following situations, tell which type of loop (`while`, `do-while`, or `for`) would work best:
- Summing a series, such as $1/2 + 1/3 + 1/4 + 1/5 + \dots + 1/10$.
 - Reading in the list of exam scores for one student.
 - Reading in the number of days of sick leave taken by employees in a department.

36. What is the output of the following?

```
int number = 10;
while (number > 0)
{
    System.out.println(number);
    number = number + 3;
}
```

37. What is the output of the following?

```
int n, limit = 10;
for (n = 1; n < limit; n++)
{
    System.out.println("n == " + n);
    System.out.println("limit == " + limit);
    limit = n + 2;
}
```

38. What is the output produced by the following?

```
int number = 10;
while (number > 0)
{
    number = number - 2;
    if (number == 4)
        break;
    System.out.println(number);
}
System.out.println("The end.");
```

39. What is the output produced by the following?

```
int number = 10;
while (number > 0)
{
    number = number - 2;
    if (number == 4)
        continue;
    System.out.println(number);
}
System.out.println("The end.");
```

The `break` and `continue` Statements ★

In previous subsections, we described the basic flow of control for the `while`, `do-while`, and `for` loops. This is how the loops should normally be used, and they are usually are. However, you can alter the flow of control in two additional ways: You can either insert a `break` statement or insert a `continue` statement. The `break` statement ends the loop. The `continue` statement ends the current iteration of the loop body. The `break` and `continue` statements can be used with any of the Java loop statements.

`break`
statement

We described the `break` statement earlier in this chapter when we discussed the `switch` statement. The **`break` statement** consists of the keyword `break` followed by a semicolon. When executed, the `break` statement ends the nearest enclosing `switch` or loop statement.

`continue`
statement

The **`continue` statement** consists of the keyword `continue` followed by a semicolon. When executed, the `continue` statement ends the current loop body iteration of the nearest enclosing loop statement.

One point that you should note when using the `continue` statement in a `for` loop is that the `continue` statement transfers control to the update expression. Thus, any loop control variable will be updated immediately after the `continue` statement is executed.

Note that a `break` statement completely ends the loop. In contrast, a `continue` statement merely ends one loop iteration, and the next iteration (if any) continues the loop.

You never absolutely need a `break` or `continue` statement. Any code that uses a `break` or `continue` statement can be rewritten to do the same thing without a `break` or `continue` statement. The `continue` statement can be particularly tricky and can make your code hard to read. It may be best to avoid the `continue` statement completely or at least use it only on very rare occasions. The use of the `break` and `continue` statements in loops is controversial, with many experts saying they should never be used. You will need to make your own decision on whether you will use either or both of these statements.

You can nest one loop statement inside another loop statement. When doing so, remember that any `break` or `continue` statement applies to the innermost loop statement containing the `break` or `continue` statement. If there is a `switch` statement inside a loop, any `break` statement applies to the innermost loop or `switch` statement.

There is a type of `break` statement that, when used in nested loops, can end any containing loop, not just the innermost loop. If you label an enclosing loop statement with an *Identifier*, then the following version of the `break` statement will exit the labeled loop, even if it is not the innermost enclosing loop:

```
break Identifier;
```

label

To **label** a loop statement, simply precede it with an *Identifier* and a colon. The following is an outline of some sample code that uses a labeled `break` statement:

```
outerLoop:  
do
```

```

{
    ...
    while (next >= 0)
    {
        next = keyboard.nextInt( );
        if (next < -100)
            break outerLoop;
        ...
    }
    ...
    answer = ...
} while (answer.equalsIgnoreCase("yes"));

```

The identifier `outerLoop` labels the outer loop, which is a `do` loop. If the number read into the variable `next` is negative but not less than `-100`, then the inner `while` loop ends normally. If, however, the number read is less than `-100`, then the labeled `break` statement is executed, and that ends the enclosing `do` loop.

You can actually label any statement, not just loop statements and `switch` statements. A labeled `break` will always end the enclosing statement with the matching label, no matter what kind of statement is labeled.

The labeled `break` can be handy when you have a `switch` statement in the body of a loop and you want a `break` statement that ends the loop rather than just ending the `switch` statement.

The `exit` Statement

The `break` statement ends a loop (or `switch` statement) but does not end the program. The following statement immediately ends the program:

```
System.exit(0);
```

`System` is a predefined Java class that is automatically provided by Java, and `exit` is a method in the class `System`. The method `exit` ends the program as soon as it is invoked. In the programs that we will write, the integer argument `0` can be any integer, but by tradition we use `0`, because `0` is used to indicate a normal ending of the program.

The following is a bit of code that uses the `exit` statement:

```

System.out.println("Enter a negative number:");
int negNumber = keyboard.nextInt( );
if (negNumber >= 0)
{
    System.out.println(negNumber + " is not a negative number.");
    System.out.println("Program aborting.");
    System.exit(0);
}

```

There are more examples of the use of `System.exit` in Chapter 4.

3.4 Debugging

A man who has committed a mistake and doesn't correct it is committing another mistake.

CONFUCIUS

Loop Bugs

There is a pattern to the kinds of mistakes you are most likely to make when programming with loops. Moreover, there are some standard techniques you can use to locate and fix bugs in your loops.

The two most common kinds of loop errors are unintended infinite loops and *off-by-one* errors. We have already discussed infinite loops, but we still need to consider off-by-one errors.

off-by-one
error

If your loop has an **off-by-one error**, that means the loop repeats the loop body one too many or one too few times. These sorts of errors can result from carelessness in designing a controlling Boolean expression. For example, if you use less-than when you should use less-than-or-equal, this can easily make your loop iterate the body the wrong number of times.

Use of `==` to test for equality in the controlling Boolean expression of a loop can often lead to an off-by-one error or an infinite loop. This sort of equality testing can work satisfactorily for integers and characters, but is not reliable for floating-point numbers. This is because the floating-point numbers are approximate quantities, and `==` tests for exact equality. The result of such a test is unpredictable. When comparing floating-point numbers, always use something involving less-than or greater-than, such as `<=`; do not use `==` or `!=`. Using `==` or `!=` to test floating-point numbers can produce an off-by-one error or an unintended infinite loop or even some other type of error. Even when using integer variables, it is best to avoid using `==` and `!=` and to instead use something involving less-than or greater-than.

Off-by-one errors can easily go unnoticed. If a loop is iterated one too many times or one too few times, the results might still look reasonable but be off by enough to cause trouble later on. Always make a specific check for off-by-one errors by comparing your loop results to results you know to be true by some other means, such as a pencil-and-paper calculation.

Tracing Variables

tracing
variables

One good way to discover errors in a loop or any kind of code is to trace some key variables. **Tracing variables** means watching the variables change value while the program is running. Most programs do not output each variable's value every time the variable changes, but being able to see all of these variable changes can help you to debug your program.

Many IDEs (Integrated Development Environments) have a built-in utility that lets you easily trace variables without making any changes to your program. These debugging systems vary from one IDE to another. If you have such a debugging facility, it is worth learning how to use it.

If you do not want to use such a debugging facility, you can trace variables by inserting some temporary output statements in your program. For example, the following code compiles but still contains an error:

```
int n = 10;
int sum = 10;
while (n > 1)
{
    sum = sum + n;
    n--;
}
System.out.println("The sum of the integers 1 to 10 is " + sum);
```

To find out what is wrong, you can trace the variables `n` and `sum` by inserting output statements as follows:

```
int n = 10;
int sum = 10;
while (n > 1)
{
    //trace
    System.out.println("At the beginning of the loop: n = " + n);
    //trace
    System.out.println("At the beginning of the loop: sum = " + sum);
    sum = sum + n;
    n--;
    //trace
    System.out.println("At the end of the loop: n = " + n);
    //trace
    System.out.println("At the end of the loop: sum = " + sum);
}
System.out.println("The sum of the integers 1 to 10 is " + sum);
```

The first four lines of the execution are as follows:

```
At the beginning of the loop: n = 10
At the beginning of the loop: sum = 10
At the end of the loop: n = 9
At the end of the loop: sum = 20
```

We can immediately see that something is wrong. The variable `sum` has been set to 20. Since it was initialized to 10, it is set to $10 + 10$, which is incorrect if we want to sum the numbers from 1 to 10. There are several ways to correct the problem. One solution is given as the answer to Self-Test Exercise 40.

General Debugging Techniques

Tracing errors can sometimes be a difficult and time-consuming task. It is not uncommon to spend more time debugging a piece of code than it took to write the code in the first place. If you are having difficulties finding the source of your errors, then there are some general debugging techniques to consider.

Examine the system as a whole and do not assume that the bug occurs in one particular place. If the program is giving incorrect output values, then you should

examine the source code, different test cases using a range of input and output values, and the logic behind the algorithm itself. For example, consider the tax program in Display 3.1. If the wrong tax is displayed, you might spend a lot of time trying to find an error in the code that calculates the tax. However, the error might simply be that the input values were different from those you were expecting, leading to an apparently incorrect program output. For example, in German the decimal point and comma are reversed from the English usage. Thus, an income of 25,000.50 becomes 25.000,50. A German programmer might make this type of error if the code was written assuming input in the English format. Although this scenario might seem like a stretch, consider the \$125 million Mars Climate Orbiter launched by NASA in 1998. In 1999, it was lost approaching the planet because one team used metric units while another used English units to control the spacecraft's thrusters.

Determining the precise cause and location of a bug is one of the first steps in fixing the error. Examine the input and output behavior for different test cases to try to localize the error. A related technique is to trace variables to show what code the program is executing and what values are contained in key variables. You might also focus on code that has recently changed or code that has had errors before. Finally, you can also try removing code. If you comment out blocks of code and the error still remains, then the culprit is in the uncommented code. The process can be repeated until the location of the error can be pinpointed. The `/*` and `*/` notation is particularly useful to comment out large blocks of code. After the error has been fixed, it is easy to remove the comments and reactivate the code.

The first mistakes you should look for are common errors that are easy to make. These are described throughout this textbook in the Pitfall sections. Examples of common errors include off-by-one errors, comparing floating-point types with `==`, adding extra semicolons that terminate a loop early, or using `==` to compare strings for equality.

Some novice programmers may become frustrated if they cannot find the bug and may resort to guessing. This technique involves changing the code without really understanding the effect of the change but hoping that it will fix the error. Avoid such slipshod hackery at all costs! Sometimes this method will work for the first few simple programs that you write. However, it will almost certainly fail for larger programs and will most likely introduce new errors to the program. Make sure that you understand the logical impact a change to the code will make before committing the modification.

Finally, if allowed by your instructor, you could show the program to someone else. A fresh set of eyes can sometimes quickly pinpoint an error that you have been missing. Taking a break and returning to the problem a few hours later or the next day can also sometimes help in discovering an error.

EXAMPLE: Debugging an Input Validation Loop



Let's illustrate both good and bad debugging techniques with an example. Suppose our program is presenting a menu where the user can select 'A' or 'B'. The purpose of the following code is to validate user input from the keyboard and to make the user type a choice again if something other than 'A' or 'B' is entered. To be more user-friendly, the program should allow users to make their selections in either upper- or lowercase.

EXAMPLE: (continued)

```
String s = "";
char c = ' ';
Scanner keyboard = new Scanner(System.in);

do
{
    System.out.println("Enter 'A' for option A " +
        "or 'B' for option B.");
    s = keyboard.next( );
    s.toLowerCase( );
    c = s.substring(0,1);
}
while ((c != 'a') || (c != 'b'));
```

This program generates a syntax error when compiled:

```
c = s.substring(0,1); : incompatible types
found   : java.lang.String
required: char
```

The intent was to extract the first character from the string entered by the user and check to see whether it is 'a' or 'b'. The `substring(0,1)` call returns a `String` containing the first character of `s`, but `c` is of type `char` and the types need to match on both sides of the assignment. If we employ the “guessing” debugging technique, then we might make the types match by changing the data type of `c` to `String`. Such a change will “fix” this error, but it will cause new errors because the rest of the code treats `c` like a `char`. As a result, we have added even more errors! Before making any change, consider the larger context and what the effect of the change will be. In this case, the simplest and best fix is to use

```
c = s.charAt(0)
```

to retrieve the first character from `s` instead of retrieving a substring.

At this point, we have corrected the syntax error and our program will compile, but it will still not run correctly. A sample execution is shown as follows:

```
Enter 'A' for option A or 'B' for option B.
C
Enter 'A' for option A or 'B' for option B.
B
Enter 'A' for option A or 'B' for option B.
A
Enter 'A' for option A or 'B' for option B.
(Control-C)
```

The program is stuck in an infinite loop even when we type in a valid choice. The only way to stop it is to break out of the program (in the sample output this is done by hitting Control-C, but you may have to use a different method depending on your computing environment).

(continued)

EXAMPLE: (continued)

At this point, we could employ tracing to try to locate the source of the error. Here is the code with output statements inserted:

```
do
{
    System.out.println("Enter 'A' for option A " +
        "or 'B' for option B.");
    s = keyboard.next( );
    System.out.println("String s = " + s);
    s.toLowerCase( );
    System.out.println("Lowercase s = " + s);
    c = s.charAt(0);
    System.out.println("c = " + c);
}
while ((c != 'a') || (c != 'b'));
```

Sample output is as follows:

```
Enter 'A' for option A or 'B' for option B.
A
String s = A
Lowercase s = A
c = A
Enter 'A' for option A or 'B' for option B.
```

The `println` statements make it clear what is wrong—the string `s` does not change to lowercase. A review of the `toLowerCase( )` documentation reveals that this method does not change the calling string, but instead returns a new string converted to lowercase. The calling string remains unchanged. To fix the error, we can assign the lowercase string back to the original string with

```
s = s.toLowerCase( );
```

However, we are not done yet. Even after fixing the lowercase error, the program is still stuck in an infinite loop, even when we enter 'A' or 'B'. A novice programmer might “patch” the program like so to exit the loop:

```
do
{
    System.out.println("Enter 'A' for option A " +
        "or 'B' for option B.");
    s = keyboard.next( );
    s = s.toLowerCase( );
    c = s.charAt(0);
    if ( c == 'a')
        break;
    if (c == 'b')
        break;
}
```

EXAMPLE: (continued)

```
while ((c != 'a') || (c != 'b'));
```

This forces the loop to exit if 'a' or 'b' is entered, and it will make the program work. Unfortunately, the result is a coding atrocity that should be avoided at all costs. This “quick fix” does not address the root cause of the error—only the symptoms. Moreover, such patches usually will not work for new cases. This particular fix also results in inconsistent code because the expression `((c != 'a') || (c != 'b'))` becomes meaningless when we already handle the 'a' and 'b' with the `if` and `break` statements.

To really find the bug, we can turn again to tracing, this time focusing on the Boolean values that control the `do-while` loop:

```
do
{
    System.out.println("Enter 'A' for option A " +
        "or 'B' for option B.");
    s = keyboard.next();
    s = s.toLowerCase();
    c = s.charAt(0);
    System.out.println("c != 'a' is " + (c != 'a'));
    System.out.println("c != 'b' is " + (c != 'b'));
    System.out.println("(c != 'a') || (c != 'b') is "
        + ((c != 'a') || (c != 'b')));
}
while ((c != 'a') || (c != 'b'));
```

The sample output is now as follows:

```
Enter 'A' for option A or 'B' for option B.
A
c != 'a' is false
c != 'b' is true
(c != 'a') || (c != 'b') is true
```

Since `c` equals 'a', the statement `(c != 'a')` evaluates to `false` and the statement `(c != 'b')` evaluates to `true`. When combined, `(false || true)` is `true`, which makes the loop repeat. In spoken English, it sounds like “c not equal to 'a'” or “c not equal to 'b'” is a correct condition to repeat the loop. After all, if the character typed in is not 'a' or the character typed in is not 'b', then the user should be prompted to try again. Logically however, if `(c != 'a')` is `false` (i.e., the character is 'a'), then `(c != 'b')` must be `true`. A character cannot make both expressions `false`, so the final Boolean condition will always be `true`. The solution is to replace the “or” with an “and” so that the loop repeats only if `(c != 'a') && (c != 'b')`. This makes the loop repeat as long as the input character is not 'a' and it is not 'b'.

(continued)

EXAMPLE: (continued)

An even better solution is to declare a `boolean` variable to control the `do-while` loop. Inside the body of the loop, we can set this variable to `false` when the loop should exit. This technique has the benefit of making the code logic easier to follow, especially if we pick a meaningful name for the variable. In the following example, it is easy to see that the loop repeats if `invalidKey` is `true`:

```
boolean invalidKey;
do
{
    System.out.println("Enter 'A' for option A " +
        "or 'B' for option B.");
    s = keyboard.next( );
    s = s.toLowerCase( );
    c = s.charAt(0);
    if (c == 'a')
        invalidKey = false;
    else if (c == 'b')
        invalidKey = false;
    else
        invalidKey = true;
}
while (invalidKey);
```

Preventive Coding

The best way to make debugging easier is to make no mistakes in the first place. Although this is unrealistic for programs of any complexity, there are some techniques we can use to eliminate or reduce the number of bugs in a program.

Incremental development is the technique of writing a small amount of code and testing it before moving on and writing more code. The test may require some new code, or a “test harness,” that will not be part of your final program but exercises your code in some way to see if it is working. This technique makes debugging easier because if the test fails, then the error is likely in the small section of the new code that was just written.

When an error is made, be sure to learn from your mistake so you do not make it again in the future. Did the mistake occur because of sloppy programming? Was there some aspect of the program’s design that you did not understand or left off before writing the code? Was there something you could have done to find the error more quickly or prevent it from happening at all? Are there other errors in your program similar to the one you just fixed? A critical review of your coding and debugging techniques should become a learning experience so you do not repeat your mistakes.

Finally, show your code to other programmers. Another developer might be able to immediately spot an error in your code and eliminate a lengthy debugging process. Many software development organizations have a formal process called **code review** that involves the inspection of code by other programmers. Such reviews have the

incremental
development

code review

pair programming

additional benefit that programmers end up sharing coding techniques and learning best practices in the process of reviewing each other's code. A related technique is called **pair programming**, in which two programmers work together at the same computer. The programmers take turns, one typing while the other watches and looks for errors and thinks about the task at hand.

Assertion Checks ★

assertion

An **assertion** is a sentence that says (asserts) something about the state of your program. An assertion must be a sentence that is either true or false and should be true if there are no mistakes in your program. You can place assertions in your code by making them comments. For example, all the comments in the following code are assertions:

```
int n = 0;
int sum = 0;
//n == 0 and sum == 0
while (n < 100)
{
    n++;
    sum = sum + n;
    //sum == 1 + 2 + 3 + ... + n
}
//sum == 1 + 2 + 3 + ... + 100
```

Note that each of these assertions can be either true or false, depending on the values of `n` and `sum`, and they all should be true if the program is performing correctly.

Java has a special statement to check whether an assertion is true. An assertion check statement has the following form:

```
assert Boolean_Expression;
```

If you run your program in the proper way, the assertion check behaves as follows: If the *Boolean_Expression* evaluates to `true`, nothing happens, but if the *Boolean_Expression* evaluates to `false`, the program ends and outputs an error message saying that an assertion failed.

For example, the previously displayed code can be written as follows, with the first comment replaced by an assertion check:

```
int n = 0;
int sum = 0;
assert (n == 0) && (sum == 0);
while (n < 100)
{
    n++;
    sum = sum + n;
    //sum == 1 + 2 + 3 + ... + n
}
//sum == 1 + 2 + 3 + ... + 100
```

Note that we translated only one of the three comments into an assertion check. Not all assertion comments lend themselves to becoming assertion checks. For example, there is no simple way to convert the other two comments into Boolean expressions. Doing so would not be impossible, but you would need to use code that would itself be more complicated than what you would be checking.

assertion check

Assertion Checking

An **assertion check** is a Java statement consisting of the keyword `assert` followed by a Boolean expression and a semicolon. If assertion checking is turned on and the Boolean expression in the assertion check evaluates to `false` when the assertion check is executed, the program will end and output a suitable error message. If assertion checking is not turned on, the assertion check is treated as a comment.

SYNTAX

```
assert Boolean_Expression;
```

EXAMPLE

```
assert (n == 0) && (sum == 0);
```

You can turn assertion checking on and off. When debugging code, you can turn assertion checking on so that a failed assertion will produce an error message. Once your code is debugged, you can turn assertion checking off, and your code will run more efficiently.

A program or other class containing assertions is compiled in the usual way. After all classes used in a program are compiled, you can run the program with assertion checking either turned on or turned off.

If you compile your classes from the command line, you would compile a class with assertion in the usual way:

```
javac YourProgram.java
```

You can then run your program with assertion checking turned on or off. The normal way of running a program has assertion checking turned off. To run your program with assertion checking turned on, use the following command:

```
java -enableassertions YourProgram
```

If you are using an IDE, check the documentation for your IDE to see how to handle assertion checking. If you do not find an entry for “assertion checking,” which is likely, check to see how you set run options.

Self-Test Exercises

40. Fix the bug in the code in the earlier subsection “Tracing Variables.”
41. Add some suitable output statements to the following code so that all variables are traced:

```
int n, sum = 0;
for (n = 1; n < 10; n++)
    sum = sum + n;
System.out.println("1 + 2 + ... + 9 + 10 == " + sum);
```

42. What is the bug in the following code? What do you call this kind of loop bug?

```
int n, sum = 0;
for (n = 1; n < 10; n++)
    sum = sum + n;
System.out.println("1 + 2 + ... + 9 + 10 == " + sum);
```

43. Write an assertion check that checks to see that the value of the variable `time` is less than or equal to the value of the variable `limit`. Both variables are of type `int`.

3.5 Random Number Generation ★

The generation of random numbers is too important to be left to chance.

Coveyou, Robert. “Random Number Generation Is Too Important to Be Left to Chance,” *Studies in Applied Mathematics*, III (1970), pp. 70–111, 1970.

Games and simulation programs often require the computer to generate random numbers. For example, a card game might need a way to randomly shuffle the cards in the deck or to roll a pair of dice. In this section, we briefly discuss two ways to generate random numbers in Java. Although we generally use the term **random**, Java really generates **pseudorandom** numbers. That is, Java can generate a sequence of numbers that looks random but this sequence of numbers is initialized by a “seed” value. If the same seed value is used to initialize the random number generator, then the exact same sequence of numbers will be generated.

The Random Object

Java includes an object called `Random` that can be used to generate many different types of random numbers. In this section, we discuss only how to generate random integers and doubles from a uniform distribution (this is when every number that could possibly be generated has an equally likely chance to appear) but the `Random` class supports other distributions.

random
numbers
pseudorandom

To use the `Random` class, we first have to import it just like we imported the `Scanner` class:

```
import java.util.Random;
```

Next, we have to create an object of type `Random` that can generate the random numbers for us. This follows the same pattern as creating a `Scanner` object to read from the keyboard.

```
Random randomGenerator = new Random();
```

Similarly, just as you created only one `Scanner` object to read in all of your keyboard inputs, in general you should create only one `Random` object to generate all of your random numbers. In particular, older versions of Java used the computer's clock to seed the random number generator. This meant that two `Random` objects created within the same millisecond would generate the same sequence of numbers. Newer versions of Java do not have this limitation, but normally only one instance of a `Random` object is needed.

To generate a random integer in the range of all possible integers, use

```
int r = randomGenerator.nextInt();
```

To generate a random integer in the range from 0 to $n-1$, use

```
int r = randomGenerator.nextInt(n);
```

If you want a random number in a different range, then you can scale the number by adding an offset. For example, to generate a random number that is 4, 5, or 6, use

```
int r = randomGenerator.nextInt(3) + 4;
```

This generates a number that is 0, 1, or 2 and then adds 4 to get a number that is 4, 5, or 6.

To generate a random `double`, use

```
double r = randomGenerator.nextDouble();
```

This returns a number that is greater than or equal to 0.0 but less than 1.0. Display 3.11 demonstrates flipping a virtual coin five times by generating five random numbers that are either 0 or 1, where 0 corresponds to tails and 1 corresponds to heads.

Display 3.11 Comparing Strings

```
1 import java.util.Random;
2 public class CoinFlipDemo
3 {
4     public static void main(String[] args)
5     {
6         Random randomGenerator = new Random();
7         int counter = 1;
8
9         while (counter <= 5)
10        {
11            System.out.print("Flip number " + counter + ": ");
12            int coinFlip = randomGenerator.nextInt(2);
13            if (coinFlip == 1)
14                System.out.println("Heads");
15            else
16                System.out.println("Tails");
17            counter++;
18        }
19    }
20 }
```

Sample Dialogue (output will vary)

```
Flip number 1: Heads
Flip number 2: Tails
Flip number 3: Heads
Flip number 4: Heads
Flip number 5: Tails
```

The `Math.random()` Method

Java also includes a method to generate random doubles without requiring the user to create an instance of the `Random` class. The method `Math.random()` returns a random double that is greater than or equal to 0.0 but less than 1.0. In fact, when this method is called for the first time, Java internally creates an instance of the `Random` class and invokes the `nextDouble()` method. This can be convenient if you do not want to create your own `Random` object.

Often the range between 0.0 and 1.0 is not what is desired, so it becomes necessary to scale the range by multiplying and translating the value by addition. Commonly, an `int` is desired, which requires a typecast. For example, if you need an `int` in the range from 1 to 6, the following code could be used:

```
int num = (int) (Math.random() * 6) + 1;
```


Self-Test Exercises

44. What numbers could be generated by `randomGenerator.nextInt(5) + 10`; where `randomGenerator` is an object of type `Random`?
45. What numbers could be generated by `randomGenerator.nextDouble() * 3 + 1`; where `randomGenerator` is an object of type `Random`?
46. Use the method `Math.random()` to generate a random double that is greater than or equal to 10.0 but less than 20.0.

Chapter Summary

- The Java branching statements are the `if-else` statement and the `switch statement`.
- A `switch` statement is a multiway branching statement. You can also form multiway branching statements by nesting `if-else` statements to form a multiway `if-else` statement.
- *Boolean expressions* are evaluated similar to the way arithmetic expressions are evaluated. The value of a Boolean expression can be saved in a variable of type `boolean`.
- The Java loop statements are the `while`, `do-while`, and `for` statements.
- A `do-while` statement always iterates its loop body at least one time. Both a `while` statement and a `for statement` might iterate its loop body zero times.
- A `for` loop can be used to obtain the equivalent of the instruction “repeat the loop body *n* times.”
- Tracing variables is a good method for debugging loops.
- An *assertion* check can be added to your Java code so that if the assertion is false, your program halts with an error message.
- The object `Random` can be used to generate *pseudorandom* integers or doubles.

Answers to Self-Test Exercises

1.

```
if (score > 100)
    System.out.println("High")
else
    System.out.println("Low");
```

2.

```
if (savings > expenses)
{
    System.out.println("Solvent");
    savings = savings - expenses;
    expenses = 0;}
else
{
    System.out.println("Bankrupt");
}
```
3.

```
if (number > 0)
    System.out.println("Positive");
else
    System.out.println("Not positive");
```
4.

```
if (salary < deductions)
{
    System.out.println("Crazy");
}
else
{
    System.out.println("OK");
    net = salary - deductions;
}
```
5. large
6. small
7. medium
8.

```
if (n < 0)
    System.out.println(n + " is less than zero.");
else if (n < 100)
    System.out.println(
        n + " is between 0 and 99 (inclusive).");
else
    System.out.println(n + " is 100 or larger.");
```
9. Some kind of B.
10. Oranges
11. Plums
12. Fruitless
13. $n_1 \geq n_2$
14.

```
if (n1 >= n2)
    System.out.println("n1");
else
    System.out.println("n2");
```

15. When the variables are of type `int`, you test for equality using `==`, as follows:

```
variable1 == variable2
```

When the variables are of type `String`, you test for equality using the method `equals`, as follows:

```
variable1.equals(variable2)
```

In some cases, you might want to use `equalsIgnoreCase` instead of `equals`.

16.

```
if (nextWord.compareToIgnoreCase("N") < 0)
    System.out.println("First half of the alphabet");
else
    System.out.println("Second half of the alphabet");
```
17.

```
if ( (exam >= 60) && (programsDone >= 10) )
    System.out.println("Passed");
else
    System.out.println("Failed");
```
18.

```
if ( (pressure > 100) || (temperature >= 212) )
    System.out.println("Emergency");
else
    System.out.println("OK");
```
19. a. `true`.
- b. `true`. Note that expressions a and b mean exactly the same thing. Because the operators `==` and `<` have higher precedence than `&&`, you do not need to include the parentheses. The parentheses do, however, make it easier to read. Most people find the expression in option a easier to read than the expression in option b, even though they mean the same thing.
- c. `true`.
- d. `true`.
- e. `false`. Because the value of the first subexpression, `(count == 1)`, is `false`, you know that the entire expression is `false` without bothering to evaluate the second subexpression. Thus, it does not matter what the values of `x` and `y` are. This is called *short-circuit evaluation*, which is what Java does.
- f. `true`. Since the value of the first subexpression, `(count < 10)`, is `true`, you know that the entire expression is `true` without bothering to evaluate the second subexpression. Thus, it does not matter what the values of `x` and `y` are. This is called *short-circuit evaluation*, which is what Java does.
- g. `false`. Notice that the expression in g includes the expression in option f as a subexpression. This subexpression is evaluated using short-circuit evaluation as we described for option f. The entire expression in g is equivalent to
- ```
!((true || (x < y)) && true)
```
- which in turn is equivalent to `!( true && true )`, and that is equivalent to `!(true)`, which is equivalent to the final value of `false`.

- h. This expression produces an error when it is evaluated because the first subexpression, `((limit/count) > 7)`, involves a division by zero.
  - i. `true`. Since the value of the first subexpression, `(limit < 20)`, is `true`, you know that the entire expression is `true` without bothering to evaluate the second subexpression. Thus, the second subexpression, `((limit/count) > 7)`, is never evaluated, so the fact that it involves a division by zero is never noticed by the computer. This is *short-circuit evaluation*, which is what Java does.
  - j. This expression produces an error when it is evaluated because the first subexpression, `((limit/count) > 7)`, involves a division by zero.
  - k. `false`. Since the value of the first subexpression, `(limit < 0)`, is `false`, you know that the entire expression is `false` without bothering to evaluate the second subexpression. Thus, the second subexpression, `((limit/count) > 7)`, is never evaluated, so the fact that it involves a division by zero is never noticed by the computer. This is *short-circuit evaluation*, which is what Java does.
20. No. Since `(j > 0)` is `false` and Java uses short-circuit evaluation for `&&`, the expression `(1/(j+1) > 10)` is never evaluated.
21. `((bonus + (((day * rate) / correctionFactor) * newGuy)) - penalty)`
22. 10  
7  
4  
1
23. There will be no output. Because `n > 0` is `false`, the loop body is executed zero times.
24. 10  
7  
4  
1
25. 10  
A `do-while` loop always executes its body at least one time.
26. -42  
A `do-while` loop always executes its body at least one time.
27. With a `do-while` statement, the loop body is always executed at least once. With a `while` statement, there can be conditions under which the loop body is not executed at all.
28. 2 4 6 8
29. Hello 10  
Hello 8  
Hello 6  
Hello 4  
Hello 2

30. 2.0 1.5 1.0 0.5

31. `n = 10;`

`while (n > 0)`

`{`

`System.out.println("Hello " + n);`

`n = n - 2;`

`}`

32. The output is 1024 10. The second number is the log to the base 2 of the first number. (If the first number is not a power of 2, then only an approximation to the log base 2 is produced.)

33. The output is 1024 1. The semicolon after the first line of the `for` loop is probably a pitfall error.

34. The output is too long to reproduce here. The pattern is as follows:

1 times 10 = 10

1 times 9 = 9

.

.

.

1 times 1 = 1

2 times 10 = 20

2 times 9 = 18

.

.

.

2 times 1 = 2

3 times 10 = 30

.

.

.

35. a. A `for` loop

b. and c. Both require a `while` loop because the input list might be empty. (A `for` loop also might work, but a `do-while` loop definitely would not work.)

36. This is an infinite loop. The first few lines of output are

10

12

16

19

21

37. This is an infinite loop. The first few lines of output are

`n == 1`

`limit == 10;`

`n == 2`

`limit == 3`

`n == 3`

```

limit == 4
n == 4
limit == 5

```

38. 8

```

6
The end.

```

39. 8

```

6
2
0
The end.

```

40. If you look at the trace, you will see that after one iteration, the value of `sum` is 20. But the value should be 10 + 9, or 19. This should lead you to think that the variable `n` is not decremented at the correct time. Indeed, the bug is that the two statements

```

sum = sum + n;
n-- ;

```

should be reversed to

```

n-- ;
sum = sum + n;

```

41. `int n, sum = 0;`

```

for (n = 1; n < 10; n++)

```

```

{

```

```

 System.out.println("n == " + n + " sum == " + sum);

```

```

 //Above line is a trace.

```

```

 sum = sum + n;

```

```

}

```

```

System.out.println("After loop");//trace

```

```

System.out.println("n == " + n + " sum == " + sum);//trace

```

```

System.out.println("1 + 2 + ... + 9 + 10 == " + sum);

```

If you study the output of this trace, you will see that 10 is never added in. This is a bug in the loop.

42. This is the code you traced in the previous exercise. If you study the output of this trace, you will see that 10 is never added in. This is an off-by-one error.

43. `assert (time <= limit);`

44. 10, 11, 12, 13 or 14

45. A double that is greater than or equal to 1 but less than 4.

46. `double d = Math.random() * 10 + 10;`

## Programming Projects

1. (This is a version of Programming Project 2.1 from Chapter 2.) The Babylonian algorithm to compute the square root of a positive number `n` is as follows:
  1. Make a guess at the answer (you can pick `n/2` as your initial guess).
  2. Compute `r = n / guess`.
  3. Set `guess = (guess + r) / 2`.
  4. Go back to step 2 until the last two `guess` values are within 1% of each other.Write a program that inputs a `double` for `n`, iterates through the Babylonian algorithm until the `guess` is within 1% of the previous `guess` and outputs the answer as a `double` to two decimal places. Your answer should be accurate even for large values of `n`.
2. A designer is trying to create a new pattern of five stripes using three colors. These colors and their codes are mentioned in the table below.

| Color | Character Code |
|-------|----------------|
| Red   | R              |
| Green | G              |
| Blue  | B              |

The designer has to keep in mind that no two adjacent stripes are of the same color. For example, `RRGBR` is an invalid pattern, but `RGBRB` is valid.

Write a program that accepts a sequence of five colors as input from the designer to form the pattern. At each selection the designer makes, the program should check if the input is from among the available set of colors. Also, the program should check that adjacent colors are not the same. At the end, the program should display the final stripe pattern created.

3. Weight is defined as the gravitational force acting on the mass of a body. The weight of a body with a mass of 1kg on Earth can be calculated with the formula given below. The weight is expressed in Newtons (N).

$$\text{Weight on earth} = (1 \text{ kg}) \times (9.81 \text{ m/s}^2) = 9.81 \text{ N}$$

The weight of the same body in N can be calculated with another formula:

$$\text{Weight on moon} = ((1 \text{ kg}) \times (9.81 \text{ m/s}^2)) / 6 = 1.64 \text{ N}$$

Write a program that takes as input the mass of a car in kilograms and then prompts the user to choose to calculate the car's weight in Newtons either on Earth or on the Moon. The program should allow the user to enter new values and output the car's weight on Earth or on the Moon till the user decides to exit. Use the `double` data type to store the mass.

4. It is difficult to make a budget that spans several years, because prices are not stable. If your company needs 200 pencils per year, you cannot simply use this year's price as the cost of pencils two years from now. Because of inflation, the cost is likely to be higher than it is today. Write a program to gauge the expected cost of an item in a specified number of years. The program asks for the cost of the item, the number of years from now that the item will be purchased, and the rate of inflation. The program then outputs the estimated cost of the item after the specified period. Have the user enter the inflation rate as a percentage, such as 5.6 (percent). Your program should then convert the percent to a fraction, such as 0.056 and should use a loop to estimate the price adjusted for inflation.
5. You have just purchased a stereo system that cost \$1,000 on the following credit plan: no down payment, an interest rate of 18% per year (and hence 1.5% per month), and monthly payments of \$50. The monthly payment of \$50 is used to pay the interest, and whatever is left is used to pay part of the remaining debt. Hence, the first month you pay 1.5% of \$1,000 in interest. That is \$15 in interest. So, the remaining \$35 is deducted from your debt, which leaves you with a debt of \$965.00. The next month, you pay interest of 1.5% of \$965.00, which is \$14.48. Hence, you can deduct \$35.52 (which is \$50 – \$14.48) from the amount you owe. Write a program that tells you how many months it will take you to pay off the loan, as well as the total amount of interest paid over the life of the loan. Use a loop to calculate the amount of interest and the size of the debt after each month. (Your final program need not output the monthly amount of interest paid and remaining debt, but you may want to write a preliminary version of the program that does output these values.) Use a variable to count the number of loop iterations and hence, the number of months until the debt is zero. You may want to use other variables as well. The last payment may be less than \$50 if the debt is small, but do not forget the interest. If you owe \$50, your monthly payment of \$50 will not pay off your debt, although it will come close. One month's interest on \$50 is only 75 cents.
6. The Fibonacci numbers  $F_n$  are defined as follows:  $F_0$  is 1,  $F_1$  is 1, and

$$F_{i+2} = F_i + F_{i+1}$$

$i = 0, 1, 2, \dots$  In other words, each number is the sum of the previous two numbers. The first few Fibonacci numbers are 1, 1, 2, 3, 5, and 8. One place where these numbers occur is as certain population growth rates. If a population has no deaths, then the series shows the size of the population after each time period. It takes an organism two time periods to mature to reproducing age, and then the organism reproduces once every time period. The formula applies most straightforwardly to asexual reproduction at a rate of one offspring per time period. In any event, the green crud population grows at this rate and has a time period of five days. Hence, if a green crud population starts out as 10 pounds of crud, then in 5 days, there is still 10 pounds of crud; in 10 days, there is 20 pounds of crud; in 15 days, 30 pounds; in 20 days, 50 pounds; and so forth. Write a program that takes both the initial size of a green crud population (in pounds) and a number of days as input and outputs the number of pounds of green crud after that many days. Assume that the population size is the same for four days and then increases every fifth day. Your program should allow the user to repeat this calculation as often as desired.



7. An Armstrong number is an  $n$ -digit number that equals the sum of the  $n^{\text{th}}$  power of its digits. For example 153 is a three-digit number where the sum of the cubes of the individual digits ( $1^3 + 5^3 + 3^3$ ) equals the number itself (153).

Write a program that takes as input the start and end numbers of an Armstrong number range to be printed. Your program should prompt for new start and end numbers until the user decides that she or he is through. Use variables of the type integer to store the start and end numbers of the range.

8. In a certain code language, numerals are each represented by a symbol or a letter. They are as follows

| N numeral   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|-------------|---|---|---|---|---|---|---|---|---|---|
| Symbol code | * | B | E | A | @ | F | K | % | R | M |

Numerals are to be coded as per the codes in the table and the following conditions:

- If the first and the last digits are odd, both are to be coded as 'X'.
- If the first and the last digits are even, both are to be coded as '\$'.
- If the last digit is '0', it is to be coded as '#'.

For example, 487692 is coded as \$R%KM\$

Write a program that takes as input a sequence of numbers and displays the corresponding code to represent it.

9. Write a program that calculates the total grade for  $N$  classroom exercises as a percentage. Use the `DecimalFormat` class to output the value as a percent. The user should input the value for  $N$  followed by each of the  $N$  scores and totals. Calculate the overall percentage (sum of the total points earned divided by the total points possible) and output it using the `DecimalFormat` class. Sample input and output are shown below.

How many exercises to input?

3

Score received for exercise 1:

10

Total points possible for exercise 1:

10

Score received for exercise 2:

7

Total points possible for exercise 2:

12

Score received for exercise 3:

5



VideoNote  
Solution to  
Programming  
Project 3.9

Total points possible for exercise 3:

8

Your total is 22 out of 30, or 73.33%.

10. The game of Pig is a simple two-player dice game in which the first player to reach 100 or more points wins. Players take turns. On each turn, a player rolls a six-sided die:

- If the player rolls a 1, then the player gets no new points and it becomes the other player's turn.
- If the player rolls 2 through 6, then he or she can either
  - ROLL AGAIN or
  - HOLD. At this point, the sum of all rolls is added to the player's score, and it becomes the other player's turn.

Write a program that plays the game of Pig, where one player is a human and the other is the computer. When it is the human's turn, the program should show the score of both players and the previous roll. Allow the human to input "r" to roll again or "h" to hold.

The computer program should play according to the following rule:

- Keep rolling when it is the computer's turn until it has accumulated 20 or more points, then hold. If the computer wins or rolls a 1, then the turn ends immediately.

Allow the human to roll first.

11. You have three identical prizes to give away and a pool of 30 finalists. The finalists are assigned numbers from 1 to 30. Write a program to randomly select the numbers of three finalists to receive a prize. Make sure not to pick the same number twice. For example, picking finalists 3, 15, 29 would be valid but picking 3, 3, 31 would be invalid because finalist number 3 is listed twice and 31 is not a valid finalist number.
12. Redo or do for the first time Programming Project 2.13 from Chapter 2, but this time use a loop to read the names from the file. Your program should also handle an arbitrary number of entries in the file instead of handling only three entries. To do this, your program must check to see if there is still data left to read (i.e., it has reached the end of the file). The appropriate methods to read from a file are described in Section 2.3.
13. The file `words.txt` on the book's website contains 87,314 words from the English language. Write a program that reads through this file and finds the longest word that is a palindrome.
14. The file `words.txt` on the book's website contains 87,314 words from the English language. Write a program that reads through this file and finds the word that has the most consecutive vowels. For example, the word "bedouin" has three consecutive vowels.



VideoNote  
Solution to  
Programming  
Project 3.13

15. This problem is based on a “Nifty Assignment” by Steve Wolfman (<http://nifty.stanford.edu/2006/wolfman-pretid>). Consider lists of numbers from real-life data sources—for example, a list containing the number of students enrolled in different course sections, the number of comments posted for different Facebook status updates, the number of books in different library holdings, the number of votes per precinct, etc. It might seem like the leading digit of each number in the list could be 1–9 with an equally likely probability. However, Benford’s Law states that the leading digit is 1 about 30% of the time and drops with larger digits. The leading digit is 9 only about 5% of the time.

Write a program that tests Benford’s Law. Collect a list of at least 100 numbers from some real-life data source and enter them into a text file. Your program should loop through the list of numbers and count how many times 1 is the first digit, 2 is the first digit, etc. For each digit, output the percentage it appears as the first digit.

16. Suppose a text file contains the following verse from the Bible:

*Let us fix our eyes on Jesus, the author and perfecter of our faith, who for the joy set before him endured the cross, scorning its shame, and sat down at the right hand of the throne of God.*

Write a program that takes one of the five vowels (a, e, i, o, and u) as input from the user and checks each word of the text file to determine whether or not it starts with that vowel. Then, it should also display those words on the screen, one word per line.



# Defining Classes I 4

## 4.1 CLASS DEFINITIONS 206

- Instance Variables and Methods 209
- More about Methods 212
- Local Variables 218
- Blocks 219
- Parameters of a Primitive Type 220
- Simple Cases with Class Parameters 229
- The `this` Parameter 229
- Methods That Return a Boolean Value 231
- The Methods `equals` and `toString` 234
- Recursive Methods 237

## 4.2 INFORMATION HIDING AND ENCAPSULATION 239

- `public` and `private` Modifiers 240
- Example: Yet Another Date Class 241
- Accessor and Mutator Methods 242
- Preconditions and Postconditions 249

## 4.3 OVERLOADING 250

- Rules for Overloading 250

## 4.4 CONSTRUCTORS 258

- Constructor Definitions 258
- Example: The Final Date Class 268
- Default Variable Initializations 269
- An Alternative Way to Initialize Instance Variables 269
- Example: A Pet Record Class 270
- The `StringTokenizer` Class ★ 274

# 4 Defining Classes I

*The loftier the building,  
the deeper must the  
foundation be laid.*

THOMAS KEMPIS, 1400s.

## Introduction

Classes are the single most important language feature that facilitates object-oriented programming (OOP), the dominant programming methodology in use today. You have already been using predefined classes. `String` and `Scanner` are two of the classes we have used. An object is a value of a class type and is referred to as an *instance of the class*. An object differs from a value of a primitive type in that it has methods (actions) as well as data. For example, `"Hello"` is an object of the class `String`. It has the characters in the string as its data and also has a number of methods, such as `length`. You already know how to use classes, objects, and methods. This chapter tells you how to define classes and their methods.

## Prerequisites

This chapter uses material from Chapters 1, 2, and 3. This chapter requires a basic understanding of the Java programming language, including the ability to write simple programs using expressions, assignments, and console I/O. You should be able to output numbers, as well as have an understanding of how the `Scanner` class can be used in console I/O. You should also know how to manage the flow of control using branching and looping statements, as well as understand how to use Boolean expressions.

## 4.1 Class Definitions

*"The Time has come," the Walrus said,  
"to talk of many things:  
of shoes and ships and sealing wax  
of cabbages and kings."*

LEWIS CARROLL, *Through the Looking-Glass*, 1871.

A Java program consists of objects from various classes interacting with one another. Before we go into the details of how you define classes, let's review some of the terminology used with classes. Among other things, a class is a type, and you can declare variables of a class type. A value of a class type is called an **object**. An object has both data and actions. The actions are called **methods**. Each object can have different data,

object  
method

**instance**

but all objects of a class have the same types of data and all objects in a class have the same methods. An object is usually referred to as an object of the class or as an **instance of the class** rather than as a value of the class, but it is a value of the class type. To make this abstract discussion come alive, we need a sample definition.

**A Class Is a Type**

If A is a class, then the phrases “bla is of type A,” “bla is an instance of the class A,” and “bla is an object of the class A” all mean the same thing.

**member****field****instance  
variable**

Display 4.1 contains a definition for a class named `DateFirstTry` and a program that demonstrates using the class. Objects of this class represent dates such as December 31, 2012 and July 4, 1776. This class is unrealistically simple, but it will serve to introduce you to the syntax for a class definition. Each object of this class has three pieces of data: a string for the month name, an integer for the day of the month, and another integer for the year. The objects have only one method, which is named `writeOutput`. Both the data items and the methods are sometimes called **members** of the object, because they belong to the object. The data items are also sometimes called **fields**. We will call the data items **instance variables** and use the term *method* instead of *member*.

The following three lines from the start of the class definition define three instance variables (three data members):

```
public String month;
public int day;
public int year; //a four digit number.
```

The word `public` simply means that there are no restrictions on how these instance variables are used. Each of these lines declares one instance variable name. You can think of an object of the class as a complex item with instance variables inside of it. So, an instance variable can be thought of as a smaller variable inside each object of the class. In this case, the instance variables are called `month`, `day`, and `year`.

An object of a class is typically named by a variable of the class type. For example, the program `DateFirstTryDemo` in Display 4.1 declares the two variables `date1` and `date2` to be of type `DateFirstTry`, as follows:

```
DateFirstTry date1, date2;
```

This gives us variables of the class `DateFirstTry`, but so far there are no objects of the class. Objects are class values that are named by the variables. To obtain an object, you must use the `new` operator to create a “new” object. For example, the following creates an object of the class `DateFirstTry` and names it with the variable `date1`:

**new**

```
date1 = new DateFirstTry();
```

## Display 4.1 A Simple Class

---

```

1 public class DateFirstTry
2 {
3 public String month;
4 public int day;
5 public int year; //a four digit number.

6 public void writeOutput()
7 {
8 System.out.println(month + " " + day + ", " + year);
9 }
10 }
```

*This class definition goes in a file named DateFirstTry.java.*

*Later in this chapter, we will see that these three public modifiers should be replaced with private.*

---

```

1 public class DateFirstTryDemo
2 {
3 public static void main(String[] args)
4 {
5 DateFirstTry date1, date2;
6 date1 = new DateFirstTry();
7 date2 = new DateFirstTry();
8 date1.month = "December";
9 date1.day = 31;
10 date1.year = 2012;
11 System.out.println("date1:");
12 date1.writeOutput();

13 date2.month = "July";
14 date2.day = 4;
15 date2.year = 1776;
16 System.out.println("date2:");
17 date2.writeOutput();
18 }
19 }
```

*This class definition (program) goes in a file named DateFirstTryDemo.java.*

---

## Sample Dialogue

```

date1:
December 31, 2012
date2:
July 4, 1776
```

---

We will discuss this kind of statement in more detail later in this chapter when we discuss something called a *constructor*. For now simply note that

*Class\_Variable* = `new Class_Name( )`;

creates a new object of the specified class and associates it with the class variable.<sup>1</sup> Because the class variable now names an object of the class, we will often refer to the class variable as an object of the class. (This is really the same usage as when we refer to an `int` variable `n` as “the integer `n`,” even though the integer is, strictly speaking, not `n` but the value of `n`.)

Unlike what we did in Display 4.1, the declaration of a class variable and the creation of the object are more typically combined into one statement, as follows:

```
DateFirstTry date1 = new DateFirstTry();
```

### The new Operator

The `new` operator is used to create an object of a class and associate the object with a variable that names it.

#### SYNTAX

```
Class_Variable = new Class_Name();
```

#### EXAMPLE

```
DateFirstTry date;
date = new DateFirstTry();
```

which is usually written in the following equivalent form:

```
DateFirstTry date = new DateFirstTry();
```

## Instance Variables and Methods

We will illustrate the details about instance variables using the class and program in Display 4.1. Each object of the class `DateFirstTry` has three instance variables, which can be named by giving the object name followed by a dot and the name of the instance variable. For example, the object `date1` in the program `DateFirstTryDemo` has the following three instance variables:

```
date1.month
date1.day
date1.year
```

Similarly, if you replace `date1` with `date2`, you obtain the three instance variables for the object `date2`. Note that `date1` and `date2` together have a total of six instance variables. The instance variables `date1.month` and `date2.month`, for example, are two different (instance) variables.

---

<sup>1</sup> For many, the word “new” suggests a memory allocation. As we will see, the `new` operator does indeed produce a memory allocation.



The instance variables in Display 4.1 can be used just like any other variables. For example, `date1.month` can be used just like any other variable of type `String`. The instance variables `date1.day` and `date1.year` can be used just like any other variables of type `int`. Thus, although the following is not in the spirit of the class definition, it is legal and would compile:

```
date1.month = "Hello friend.";
```

More likely assignments to instance variables are given in the program `DateFirstTryDemo`.

The class `DateFirstTry` has only one method, which is named `writeOutput`. We reproduce the definition of the method here:

```
public void writeOutput () ← Heading
{
 System.out.println(month + " " + day + ", " + year); ← Body
}
```

heading  
method body

All method definitions belong to some class, and all method definitions are given inside the definition of the class to which they belong. A method definition is divided into two parts, a **heading** and a **method body**, as illustrated by the annotation on the method definition. The word `void` means this is a method for performing an action as opposed to producing a value. We will say more about method definitions later in this chapter (including some indication of why the word `void` was chosen to indicate an action). You have already been using methods from predefined classes. The way you invoke a method from a class definition you write is the same as the way you do it for a predefined class. For example, the following from the program `DateFirstTryDemo` is an invocation of the method `writeOutput` with `date1` as the calling object:

```
date1.writeOutput();
```

This invocation is equivalent to execution of the method body. So, this invocation is equivalent to

```
System.out.println(month + " " + day + ", " + year);
```

However, we need to say more about exactly how this is equivalent. If you simply replace the method invocation with this `System.out.println` statement, you will get a compiler error message. Note that within the definition for the method `writeOutput`, the names of the instance variables are used without any calling object. This is because the method will be invoked with different calling objects at different times. When an instance variable is used in a method definition, it is understood to be the instance variable of the calling object. So in the program `DateFirstTryDemo`,

```
date1.writeOutput();
```

## Class Definition

The following shows the form of a class definition that is most commonly used; however, it is legal to intermix the method definitions and the instance variable declarations.

### SYNTAX

```
public class Class_Name
{
 Instance_Variable_Declaration_1
 Instance_Variable_Declaration_2
 ...
 Instance_Variable_Declaration_Last

 Method_Definition_1
 Method_Definition_2
 ...
 Method_Definition_Last
}
```

### EXAMPLES

See Displays 4.1 and 4.2.

is equivalent to

```
System.out.println(date1.month + " " + date1.day
 + ", " + date1.year);
```

Similarly,

```
date2.writeOutput();
```

is equivalent to

```
System.out.println(date2.month + " " + date2.day
 + ", " + date2.year);
```

## File Names and Locations

Remember that a file must be named the same as the class it contains with an added `.java` at the end. For example, a class named `MyClass` must be in a file named `MyClass.java`.

We will eventually see other ways to arrange files, but at this point, your program and all the classes it uses should be in the same directory (same folder).

### Self-Test Exercises

1. Write a method called `makeItNewYears` that could be added to the class `DateFirstTry` in Display 4.1. The method `makeItNewYears` has no parameters and sets the `month` instance variable to "January" and the `day` instance variable to 1. It does not change the `year` instance variable.
2. Write a method called `yellIfNewYear` that could be added to the class `DateFirstTry` in Display 4.1. The method `yellIfNewYear` has no parameters and outputs the string "Hurrah!" provided the `month` instance variable has the value "January" and the `day` instance variable has the value 1. Otherwise, it outputs the string "Not New Year's Day."

### More about Methods

As we noted for predefined methods, methods of the classes you define are of two kinds: methods that return (compute) some value and methods that perform an action other than returning a value. For example, the method `println` of the object `System.out` is an example of a method that performs an action other than returning a value; in this case, the action is to write something to the screen. The method `nextInt` of the class `Scanner`, introduced in Chapter 2, is a method that returns a value; in this case, the value returned is a number typed in by the user. A method that performs some action other than returning a value is called a `void` method. This same distinction between `void` methods and methods that return a value applies to methods in the classes you define. The two kinds of methods require slight differences in how they are defined.

Both kinds of methods have a method heading and a method body, which are similar but not identical for the two kinds of methods. The method heading for a `void` method is of the form

```
public void Method_Name(Parameter_List)
```

The method heading for a method that returns a value is

```
public Type_Returned Method_Name (Parameter_List)
```

Later in the chapter, we will see that `public` may sometimes be replaced by a more restricted modifier and that it is possible to add additional modifiers, but these templates will do right now. For now, our examples will have an empty *Parameter\_List*.

If a method returns a value, then it can return different values in different situations, but all values returned must be of the same type, which is specified as the type returned. For example, if a method has the heading

```
public double myMethod()
```

then the method always returns a value of type `double`, and the heading

```
public String yourMethod()
```

indicates a method that always returns a value of type `String`.

The following is a `void` method heading:

```
public void ourMethod()
```

Notice that when the method returns no value at all, we use the keyword `void` in place of a type. If you think of `void` as meaning “no returned type,” the word `void` begins to make sense.

### invocation

An **invocation** of a method that returns a value can be used as an expression anywhere that a value of the *Type\_Returned* can be used. For example, suppose `anObject` is an object of a class with methods having our sample heading; in this case, the following are legal:

```
double d = anObject.myMethod();
String aStringVariable = anObject.yourMethod();
```

A `void` method does not return a value, but simply performs an action, so an invocation of a `void` method is a statement. A `void` method is invoked as in the following example:

```
anObject.ourMethod();
```

Note the ending semicolon.

So far, we have avoided the topic of parameter lists by only giving examples with empty parameter lists, but note that parentheses are required even for an empty parameter list. Parameter lists are discussed later in this chapter.

### body

The **body** of a `void` method definition is simply a list of declarations and statements enclosed in a pair of braces, `{ }`. For example, the following is a complete `void` method definition:

```
public void ourMethod()
{
 System.out.println("Hello");
 System.out.println("from our method.");
}
```

The body of a method that returns a value is the same as the body of a `void` method but with one additional requirement. The body of a method that returns a value must contain at least one `return` statement. A **return statement** is of the form

### return statement

```
return Expression;
```

where *Expression* can be any expression that evaluates to something of the *Type\_Returned* that is listed in the method heading. For example, the following is a complete definition of a method that returns a value:

```
public String yourMethod()
{
 Scanner keyboard = new Scanner(System.in);
 System.out.println("Enter a line of text");
 String result = keyboard.nextLine();
 return result + " was entered.";
}
```

Notice that a method that returns a value can do other things besides returning a value, but style rules dictate that whatever else it does should be related to the value returned.

A return statement always ends a method invocation. Once the return statement is executed, the method ends, and any remaining statements in the method definition are not executed.

#### return in a void method

If you want to end a void method before it runs out of statements, you can use a return statement without any expression, as follows:

```
return;
```

A void method need not have any return statements, but you can place a return statement in a void method if there are situations that require the method to end before all the code is executed.

### Method Definitions

There are two kinds of methods: methods that return a value and methods, known as void methods, that perform some action other than returning a value.

#### Definition of a Method That Returns a Value

##### SYNTAX

```
public Type_Returned Method_Name(Parameter_List)
{
 <List of statements, at least one of which
 must contain a return statement.>
}
```

If there are no *Parameters*, then the parentheses are empty.

**EXAMPLE**

```
public int getDay()
{
 return day;
}
```

**void Method Definition****SYNTAX**

```
public void Method_Name (Parameter_List)
{
 <List of statements>
}
```

If there are no *Parameters*, then the parentheses are empty.

**EXAMPLE**

```
public void writeOutput ()
{
 System.out.println(month + " " + day + ", " + year);
}
```

All method definitions are inside of some class definition. See Display 4.2 to see these example method definitions in the context of a class.

When an instance variable name is used in a method definition, it refers to an instance variable of the calling object.

**return Statements**

The definition of a method that returns a value must have one or more `return` statements. A `return` statement specifies the value returned by the method and ends the method invocation.

**SYNTAX**

```
return Expression;
```

**EXAMPLE**

```
public int getYear()
{
 return year;
}
```

A `void` method definition need not have a `return` statement. However, a `return` statement can be used in a `void` method to cause the method to immediately end. The form for a `return` statement in a `void` method is

```
return;
```

Although it may seem that we have lost sight of the fact, all these method definitions must be inside of some class definition. Java does not have any stand-alone methods that are not in any class. Display 4.2 rewrites the class given in Display 4.1, but this time we have added a more diverse set of methods. Display 4.3 contains a sample program that illustrates how the methods of the class in Display 4.2 are used.



### TIP: Any Method Can Be Used as a `void` Method

A method that returns a value can also perform some action besides returning a value. If you want that action, but do not need the returned value, you can invoke the method as if it were a `void` method and the returned value will simply be discarded. For example, the following contains two invocations of the method `nextLine()`, which returns a value of type `String`. Both are legal.

```
Scanner keyboard = new Scanner(System.in);
. . .
String inputString = keyboard.nextLine();
. . .
System.out.println("Press Enter to continue with program.");
keyboard.nextLine(); //Reads a line and discards it. ■
```

Display 4.2 A Class with More Methods (part 1 of 2)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>1  import java.util.Scanner; 2  public class DateSecondTry 3  { 4      private String month; 5      private int day; 6      private int year; //a four digit number.  7      public void writeOutput() 8      { 9          System.out.println(month + " " + day + ", " + year); 10     }  11     public void readInput() 12     { 13         Scanner keyboard = new Scanner(System.in); 14         System.out.println("Enter month, day, and year."); 15         System.out.println("Do not use a comma."); 16         month = keyboard.next(); 17         day = keyboard.nextInt(); 18         year = keyboard.nextInt(); 19     }</pre> | <p><i>The significance of the modifier <b>private</b> is discussed in the subsection “<b>public</b> and <b>private</b> Modifiers” in Section 4.2 a bit later in this chapter.</i></p> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Display 4.2 A Class with More Methods (part 2 of 2)

---

```
20 public int getDay()
21 {
22 return day;
23 }
24 public int getYear()
25 {
26 return year;
27 }

28 public int getMonth()
29 {
30 if (month.equalsIgnoreCase("January"))
31 return 1;
32 else if (month.equalsIgnoreCase("February"))
33 return 2;
34 else if (month.equalsIgnoreCase("March"))
35 return 3;
36 else if (month.equalsIgnoreCase("April"))
37 return 4;
38 else if (month.equalsIgnoreCase("May"))
39 return 5;
40 else if (month.equalsIgnoreCase("June"))
41 return 6;
42 else if (month.equalsIgnoreCase("July"))
43 return 7;
44 else if (month.equalsIgnoreCase("August"))
45 return 8;
46 else if (month.equalsIgnoreCase("September"))
47 return 9;
48 else if (month.equalsIgnoreCase("October"))
49 return 10;
50 else if (month.equalsIgnoreCase("November"))
51 return 11;
52 else if (month.equalsIgnoreCase("December"))
53 return 12;
54 else
55 {
56 System.out.println("Fatal Error");
57 System.exit(0);
58 return 0; //Needed to keep the compiler happy
59 }
60 }
61 }
```

---



## Self-Test Exercises

- Write a method called `getNextYear` that could be added to the class `DateSecondTry` in Display 4.2. The method `getNextYear` returns an `int` value equal to the value of the `year` instance variable plus one.

Display 4.3 Using the Class in Display 4.2

```

1 public class DemoOfDateSecondTry
2 {
3 public static void main(String[] args)
4 {
5 DateSecondTry date = new DateSecondTry();
6 date.readInput();

7 int dayNumber = date.getDay();
8 System.out.println("That is the " + dayNumber
9 + "th day of the month.");
10 }
11 }
```

*An invocation of a void method is a statement.*

*An invocation of a method that returns a value is an expression that can be used anywhere that a value of the type returned by the method can be used.*

### Sample Dialogue

```

Enter month, day, and year.
Do not use a comma.
July 4 1776
That is the 4th day of the month.
```

## Local Variables

### local variable

Look at the definition of the method `readInput()` given in Display 4.2. That method definition includes the declaration of a variable called `keyboard`. A variable declared within a method is called a **local variable**. It is called *local* because its meaning is local to—that is, confined to—the method definition. If you have two methods and each of them declares a variable of the same name—for example, if both were named `keyboard`—they would be two different variables that just happen to have the same name. Any change that is made to the variable named `keyboard` within one method would have no effect upon the variable named `keyboard` in the other method.

As we noted in Chapter 1, the `main` part of a program is itself a method. All variables declared in `main` are variables local to the method `main`. If a variable declared in `main`

happens to have the same name as a variable declared in some other method, they are two different variables that just happen to have the same name. Thus, all the variables we have seen so far are either local variables or instance variables. There is only one more kind of variable in Java, which is known as a *static variable*. Static variables will be discussed in Chapter 5.

### Local Variable

A variable declared within a method definition is called a **local variable**. If two methods each have a local variable of the same name, they are two different variables that just happen to have the same name.

### Global Variables

Thus far, we have discussed two kinds of variables: instance variables, whose meaning is confined to an object of a class, and local variables, whose meaning is confined to a method definition. Some other programming languages have another kind of variable called a **global variable**, whose meaning is confined only to the program. Java does not have these global variables.

## Blocks

**block**  
**compound**  
**statement**

The terms **block** and **compound statement** mean the same thing—namely, a set of Java statements enclosed in braces, `{ }`. However, programmers tend to use the two terms in different contexts. When you declare a variable within a compound statement, the compound statement is usually called a *block*.

### Blocks

A **block** is another name for a compound statement—that is, a list of statements enclosed in braces. However, programmers tend to use the two terms in different contexts. When you declare a variable within a compound statement, the compound statement is usually called a *block*. The variables declared in a block are local to the block, so these variables disappear when the execution of the block is completed. However, even though the variables are local to the block, their names cannot be used for anything else within the same method definition.

If you declare a variable within a block, that variable is local to the block. This means that when the block ends, all variables declared within the block disappear. In many programming languages, you can even use that variable's name to name some other variable outside the block. However, *in Java, you cannot have two variables with the same name inside a single method definition*. Local variables within blocks

can sometimes create problems in Java. It is sometimes easier to declare the variables outside the block. If you declare a variable outside a block, you can use it both inside and outside the block, and it will have the same meaning in both locations.



### TIP: Declaring Variables in a `for` Statement

You can declare a variable (or variables) within the initialization portion of a `for` statement, as in the following:

```
int sum = 0;
for (int n = 1; n < 10; n++)
 sum = sum + n;
```

If you declare `n` in this way, the variable `n` will be *local to the `for` loop*. This means that `n` cannot be used outside the `for` loop. For example, the following use of `n` in the `System.out.println` statement is illegal:

```
for (int n = 1; n < 10; n++)
 sum = sum + n;
System.out.println(n); //Illegal
```

Declaring variables inside a `for` loop can sometimes be more of a nuisance than a helpful feature. We tend to avoid declaring variables inside a `for` loop except for very simple cases that have no potential for confusion. ■

## Self-Test Exercises

4. Write a method called `happyGreeting` that could be added to the class `DateSecondTry` in Display 4.2. The method `happyGreeting` writes the string "Happy Days!" to the screen a number of times equal to the value of the instance variable `day`. For example, if the value of `day` is 3, then it should write the following to the screen:

```
Happy Days!
Happy Days!
Happy Days!
```

Use a local variable.

## Parameters of a Primitive Type

### parameter

All the method definitions we have seen thus far had no parameters, which was indicated by an empty set of parentheses in the method heading. A **parameter** is like a blank that is filled in with a particular value when the method is invoked. (What we are calling *parameters* are also called *formal parameters*.) The value that is plugged in for

**argument**

the parameter is called an **argument**.<sup>2</sup> We have already used arguments with predefined methods. For example, the string "Hello" is the argument to the method `println` in the following method invocation:

```
System.out.println("Hello");
```

Display 4.4 contains the definition of a method named `setDate` that has the three parameters `newMonth`, `newDay`, and `newYear`. It also contains the definition of a method named `monthString` that has one parameter of type `int`.

Arguments are given in parentheses at the end of the method invocation. For example, in the following call from Display 4.4, the integers 6 and 17 and the variable `year` are the arguments plugged in for `newMonth`, `newDay`, and `newYear`, respectively:

```
date.setDate(6, 17, year);
```

When you have a method invocation such as the preceding, the argument (such as 6) is plugged in for the corresponding formal parameter (such as `newMonth`) *everywhere that the parameter occurs in the method definition*. After all the arguments have been plugged in for their corresponding parameters, the code in the body of the method definition is executed.

The following invocation of the method `monthString` occurs within the definition of the method `setDate` in Display 4.4:

```
month = monthString(newMonth);
```

The argument is `newMonth`, which is plugged in for the parameter `monthNumber` in the definition of the method `monthString`.

Note that each of the formal parameters must be preceded by a type name, even if there is more than one parameter of the same type. Corresponding arguments must match the type of their corresponding formal parameter, although in some simple cases, an automatic type cast might be performed by Java. For example, if you plug in an argument of type `int` for a parameter of type `double`, Java automatically type casts the `int` value to a value of type `double`. The following list shows the type casts that Java automatically performs for you. An argument in a method invocation that is of any of these types is automatically type cast to any of the types that appear to its right, if that is needed to match a formal parameter.<sup>3</sup>

```
byte -> short -> int -> long -> float -> double
```

---

<sup>2</sup> Some programmers use the term *actual parameters* for what we are calling *arguments*.

<sup>3</sup> An argument of type `char` is also converted to a matching number type, if the formal parameter is of type `int` or any type to the right of `int` in our list of types.

## Display 4.4 Methods with Parameters (part 1 of 2)

```

1 import java.util.Scanner;

2 public class DateThirdTry
3 {
4 private String month;
5 private int day;
6 private int year; //a four digit number.

7 public void setDate(int newMonth, int newDay, int newYear)
8 {
9 month = monthString(newMonth);
10 day = newDay;
11 year = newYear;
12 }

13 public String monthString(int monthNumber)
14 {
15 switch (monthNumber)
16 {
17 case 1:
18 return "January";
19 case 2:
20 return "February";
21 case 3:
22 return "March";
23 case 4:
24 return "April";
25 case 5:
26 return "May";
27 case 6:
28 return "June";
29 case 7:
30 return "July";
31 case 8:
32 return "August";
33 case 9:
34 return "September";
35 case 10:
36 return "October";
37 case 11:
38 return "November";
39 case 12:
40 return "December";

```

*The significance of the modifier **private** is discussed later in the subsection “public and private Modifiers” in Section 4.2.*

*The method **setDate** has an **int** parameter for the month, even though the month instance variable is of type **String**. The method **setDate** converts the month **int** value to a string with a call to the method **monthString**.*

*This is the file **DateThirdTry.java**.*

## Display 4.4 Methods with Parameters (part 2 of 2)

---

```

41 default:
42 System.out.println("Fatal Error");
43 System.exit(0);
44 return "Error"; //to keep the compiler happy
45 }
46 }

```

*<The rest of the method definitions are identical to the ones given in Display 4.2.>*

*This is the file DateThirdTry.java.*

---

```

1 public class DateThirdTryDemo
2 {
3 public static void main(String[] args)
4 {
5 DateThirdTry date = new DateThirdTry();
6 int year = 1882;
7 date.setDate(6, 17, year);
8 date.writeOutput();
9 }
10 }

```

*This is the file DateThirdTryDemo.java.*

*The variable `year` is NOT plugged in for the parameter `newYear` in the definition of the method `setDate`. Only the value of `year`, namely 1882, is plugged in for the parameter `newYear`.*




---

## Sample Dialogue

June 17, 1882

---

Note that this is exactly the same as the automatic type casting we discussed in Chapter 1 for storing values of one type in a variable of another type. The more general rule is that you can use a value of any of the listed types anywhere that Java expects a value of a type further down on the list.

Note that the correspondence of the parameters and arguments is determined by their order in the lists in parentheses. In a method invocation, there must be exactly the same number of arguments in parentheses as there are formal parameters in the method definition heading. The first argument in the method invocation is plugged in for the first parameter in the method definition heading, the second argument in the method invocation is plugged in for the second parameter in the heading of the method definition, and so forth. This is diagrammed in Display 4.5.

Display 4.5 Correspondence between Formal Parameters and Arguments



### Parameters of a Primitive Type

Parameters are given in parentheses after the method name in the heading of a method definition. A parameter of a primitive type, such as `int`, `double`, or `char`, is a local variable. When the method is invoked, the parameter is initialized to the value of the corresponding argument in the method invocation. This mechanism is known as the **call-by-value** parameter mechanism. The argument in a method invocation can be a literal constant, such as `2` or `'A'`; a variable; or any expression that yields a value of the appropriate type. This is the only kind of parameter that Java has for parameters of a primitive type. (Parameters of a class type are discussed in Chapter 5.)

### main Is a void Method

The main part of a program is a void method, as indicated by its heading:

```
public static void main(String[] args)
```

The word `static` will be explained in Chapter 5. The identifier `args` is a parameter of type `String[]`, which is the type for an array of strings. Arrays are discussed in Chapter 6, and you need not be concerned about them until then. In the examples in this book, we never use the parameter `args`. Because `args` is a parameter, you may replace it with any other nonkeyword identifier and your program will have the same meaning. Aside from possibly changing the name of the parameter `args`, the heading of the `main` method must be exactly as shown above. Although we will not be using the parameter `args`, we will tell you how to use it in Chapter 6.

A program in Java is just a class that has a `main` method. When you give a command to run a Java program, the run-time system invokes the method `main`.

#### call-by-value

It is important to note that only the value of the argument is used in this substitution process. If an argument in a method invocation is a variable (such as `year` in Display 4.4), it is the value of the variable that is plugged in for its corresponding parameter; it is not the variable name that is plugged in. For example, in Display 4.4, the value of the variable `year` (that is, 1882) is plugged in for the parameter `newYear`. The variable `year` is not plugged into the body of the method `setDate`. Because only the value of the argument is used, this method of plugging in arguments for formal parameters is known as the **call-by-value** mechanism. In Java, this is the only method of substitution that is used with parameters of a primitive type, such as `int`, `double`, and `char`. As you will eventually see, this is, strictly speaking, also the only method of substitution that is used with parameters of a class type. However, there are other differences that make parameters of a class type appear to use a different substitution mechanism. For now, we are concerned only with parameters and arguments of primitive types, such as `int`, `double`, and `char`. (Although the type `String` is a class type, you will not go wrong if you consider it to behave like a primitive type when an argument of type `String` is plugged in for its corresponding parameter. However, for most class types, you need to think a bit differently about how arguments are plugged in for parameters. We discuss parameters of a class type in Chapter 5.)

#### parameters as local variables

In most cases, you can think of a parameter as a kind of blank, or placeholder, that is filled in by the value of its corresponding argument in the method invocation. However, parameters are more than just blanks; a parameter is actually a local variable. When the method is invoked, the value of an argument is computed, and the corresponding parameter, which is a local variable, is initialized to this value. Occasionally, it is useful to use a parameter as a local variable. An example of a parameter used as a local variable is given in Display 4.6. In that display, notice the parameter `minutesWorked` in the method `computeFee`. The value of `minutesWorked` is changed within the body of the method definition. This is allowed because a parameter is a local variable.



Display 4.6 A Formal Parameter Used as a Local Variable (part 1 of 2)

---

```

1 import java.util.Scanner; This is the file Bill.java.

2 public class Bill
3 {
4 public static final double RATE = 150.00; //Dollars per quarter hour

5 private int hours;
6 private int minutes;
7 private double fee;

8 public void inputTimeWorked()
9 {
10 System.out.println("Enter number of full hours worked");
11 System.out.println("followed by number of minutes:");
12 Scanner keyboard = new Scanner(System.in); computeFee uses the
13 hours = keyboard.nextInt(); parameter minutesWorked
14 minutes = keyboard.nextInt(); as a local variable.
15 }

16 private double computeFee(int hoursWorked, int minutesWorked)
17 {
18 minutesWorked = hoursWorked*60 + minutesWorked;
19 int quarterHours = minutesWorked/15;
20 //Any remaining fraction of a quarter hour is not
21 //charged for.
22 return quarterHours*RATE;
23 }

24 public void updateFee()
25 {
26 fee = computeFee(hours, minutes);
27 }

28 public void outputBill()
29 {
30 System.out.println("Time worked: ");
31 System.out.println(hours + " hours and " + minutes +
32 " minutes");
33 System.out.println("Rate: $" + RATE + " per quarter hour.");
34 System.out.println("Amount due: $" + fee);
35 }

```

*Although minutes is plugged in for minutesWorked and minutesWorked is changed, the value of minutes is not changed.*

---

Display 4.6 A Formal Parameter Used as a Local Variable (part 2 of 2)

```

1 public class BillingDialog
2 {
3 public static void main(String[] args)
4 {
5 System.out.println("Welcome to the law offices of");
6 System.out.println("Dewey, Cheatham, and Howe.");
7 Bill yourBill = new Bill();
8 yourBill.inputTimeWorked();
9 yourBill.updateFee();
10 yourBill.outputBill();
11 System.out.println("We have placed a lien on your house.");
12 System.out.println("It has been our pleasure to serve you.");
13 }
14 }

```

This is the file  
BillingDialog.java.

## Sample Dialogue

```

Welcome to the law offices of
Dewey, Cheatham, and Howe.
Enter number of full hours worked
followed by number of minutes:
3 48
Time worked:
2 hours and 48 minutes
Rate: $150.0 per quarter hour.
Amount due: $2250.0
We have placed a lien on your house.
It has been our pleasure to serve you.

```



## PITFALL: Use of the Terms “Parameter” and “Argument”

The use of the terms *parameter* and *argument* that we follow in this book is consistent with common usage, but people also often use the terms *parameter* and *argument* interchangeably. When you see these terms, you must determine their exact meaning from context. Many people use the term *parameter* for both what we call *parameters* and what we call *arguments*. Other people use the term *argument* for both what we call *parameters* and what we call *arguments*. Do not expect consistency in how people use these two terms.

The term **formal parameter** is often used for what we describe as a *parameter*. We will sometimes use this term for emphasis. The term **actual parameter** is often used for what we call an *argument*. We do not use this term in this book, but you will encounter it in other books. ■

**formal  
parameters**

**actual  
parameter**

### Self-Test Exercises

- Write a method called `fractionDone` that could be added to the class `DateThirdTry` in Display 4.4. The method `fractionDone` has a parameter `targetDay` of type `int` (for a day of the month) and returns a value of type `double`. The value returned is the value of the `day` instance variable divided by the `int` parameter `targetDay`. (So it returns the fraction of the time passed so far this month where the goal is reaching the `targetDay`.) Use floating-point division, not integer division. To get floating-point division, copy the value of the `day` instance variable into a local variable of type `double` and use this local variable in place of the `day` instance variable in the division. (You may assume the parameter `targetDay` is a valid day of the month that is greater than the value of the `day` instance variable.)
- Write a method called `advanceYear` that could be added to the class `DateThirdTry` in Display 4.4. The method `advanceYear` has one parameter of type `int`. The method `advanceYear` increases the value of the `year` instance variable by the amount of this one parameter.
- Suppose we redefine the method `setDate` in Display 4.4 to the following:

```
public void setDate(int newMonth, int newDay, int newYear)
{
 month = monthString(newMonth);
 day = newDay;
 year = newYear;
 System.out.println("Date changed to "
 + newMonth + " " + newDay + ", " + newYear);
}
```

Indicate all instances of `newMonth` that have their value changed to 6 in the following invocation (also from Display 4.4):

```
date.setDate(6, 17, year);
```

- Is the following a legal method definition that could be added to the class `DateThirdTry` in Display 4.4?

```
public void multiWriteOutput(int count)
{
 while (count > 0)
 {
 writeOutput();
 count--;
 }
}
```

- Consider the definition of the method `monthString` in Display 4.4. Why are there no `break` statements in the `switch` statement?

## Simple Cases with Class Parameters

Methods can have parameters of a class type. Parameters of a class type are more subtle and more powerful than parameters of a primitive type. We will discuss parameters of class types in detail in Chapter 5. In the meantime, we will occasionally use a class type parameter in very simple situations. For these cases, you do not need to know any details about class type parameters except that, in some sense or other, the class argument is plugged in for the class parameter.

### The `this` Parameter

As we noted earlier, if `today` is of type `DateSecondTry` (see Display 4.2), then

```
today.writeOutput();
```

is equivalent to

```
System.out.println(today.month + " " + today.day
 + ", " + today.year);
```

This is because, although the definition of `writeOutput` reads

```
public void writeOutput()
{
 System.out.println(month + " " + day + ", " + year);
}
```

it really means

```
public void writeOutput()
{
 System.out.println(<the calling object>.month + " "
 + <the calling object>.day + ", " + <the calling object>.year);
}
```

The instance variables are understood to have `<the calling object>.` in front of them. Sometimes it is handy, and on rare occasions even necessary, to have an explicit name for the calling object. Inside a Java method definition, you can use the keyword `this` as a name for the calling object. So, the following is a valid Java method definition that is equivalent to the one we are discussing:

```
public void writeOutput()
{
 System.out.println(this.month + " " + this.day
 + ", " + this.year);
}
```

The definition of `writeOutput` in Display 4.2 could be replaced by this completely equivalent version. Moreover, this version is in some sense the true version. The version

**this  
parameter**

without the `this` and a dot in front of each instance variable is just an abbreviation for this version. However, the abbreviation of omitting the `this` is used frequently.

The keyword `this` is known as the **this parameter**. The `this` parameter is a kind of hidden parameter. It does not appear on the parameter list of a method, but is still a parameter. When a method is invoked, the calling object is automatically plugged in for `this`.

**The this Parameter**

Within a method definition, you can use the keyword `this` as a name for the calling object. If an instance variable or another method in the class is used without any calling object, then `this` is understood to be the calling object.

There is one common situation that requires the use of the `this` parameter. You often want to have the parameters in a method such as `setDate` be the same as the instance variables. A first, although incorrect, try at doing this is the following rewriting of the method `setDate` from Display 4.4:

```
public void setDate(int month, int day, int year) //Not correct
{
 month = monthString(month);
 day = day;
 year = year;
}
```

This rewritten version does not do what we want. When you declare a local variable in a method definition, then within the method definition, that name always refers to the local variable. A parameter is a local variable, so this rule applies to parameters. Consider the following assignment statement in our rewritten method definition:

```
day = day;
```

**mask a  
variable**

Both the identifiers `day` refer to the parameter `day`. The identifier `day` does not refer to the instance variable `day`. All occurrences of the identifier `day` refer to the parameter `day`. This is often described by saying the parameter `day` **masks** or hides the instance variable `day`. Similar remarks apply to the parameters `month` and `year`.

This rewritten method definition of the method `setDate` will produce a compiler error message because the following attempts to assign a `String` value to the `int` variable (the parameter) `month`:

```
month = monthString(month);
```

However, in many situations, this sort of rewriting will produce a method definition that will compile but that will not do what it is supposed to do.

To correctly rewrite the method `setDate`, we need some way to say “the instance variable `month`” as opposed to the parameter `month`. The way to say “the instance variable `month`” is `this.month`. Similar remarks apply to the other two parameters. So, the correct rewriting of the method `setDate` is as follows:

```
public void setDate(int month, int day, int year)
{
 this.month = monthString(month);
 this.day = day;
 this.year = year;
}
```

This version is completely equivalent to the version in Display 4.4.

### Self-Test Exercises

10. The method `writeOutput` in Display 4.2 uses the instance variables `month`, `day`, and `year`, but gives no object name for these instance variables. Every instance variable must belong to some object. To what object or objects do these instance variables in the definition of `writeOutput` belong?
11. Rewrite the definitions of the methods `getDay` and `getYear` in Display 4.2 using the `this` parameter.
12. Rewrite the method `getMonth` in Display 4.2 using the `this` parameter.

### Methods That Return a Boolean Value

There is nothing special about methods that return a value of type `boolean`. The type `boolean` is a primitive type, just like the types `int` and `double`. A method that returns a value of type `boolean` must have a `return` statement of the form

```
return Boolean_Expression;
```

So, an invocation of a method that returns a value of type `boolean` returns either `true` or `false`. It thus makes sense to use an invocation of such a method to control an `if-else` statement, to control a `while` loop, or to control anyplace else that a Boolean expression is allowed. Although there is nothing new here, people who have not used `boolean` valued methods before sometimes find them to be uncomfortable. So we will go through one small example.

The following is a method definition that could be added to the class `DateThirdTry` in Display 4.4:

```
public boolean isBetween(int lowYear, int highYear)
{
 return ((year > lowYear) && (year < highYear));
}
```

Consider the following lines of code:

```
DateThirdTry date = new DateThirdTry();
date.setDate(1, 2, 3001);
if (date.isBetween(2000, 4000))
 System.out.println(
 "The date is between the years 2000 and 4000");
else
 System.out.println(
 "The date is not between the years 2000 and 4000");
```

The expression `date.isBetween(2000, 4000)` is an invocation of a method that returns a boolean value—that is, returns one of the two values `true` and `false`. So, it makes perfectly good sense to use it as the controlling Boolean expression in an `if-else` statement. The expression `year` in the definition of `isBetween` really means `this.year`, and `this` stands for the calling object. In `date.isBetween(2000, 4000)` the calling object is `date`. So, this returns the value

```
(date.year > lowYear) && (date.year < highYear)
```

But, 2000 and 4000 are plugged in for the parameters `lowYear` and `highYear`, respectively. So, this expression is equivalent to

```
(date.year > 2000) && (date.year < 4000)
```

Thus, the `if-else` statement is equivalent to<sup>4</sup>

```
if ((date.year > 2000) && (date.year < 4000))
 System.out.println(
 "The date is between the years 2000 and 4000.");
else
 System.out.println(
 "The date is not between the years 2000 and 4000.");
```

Thus, the output produced is

```
The date is between the years 2000 and 4000.
```

Another example of a boolean valued method, which we will, in fact, add to our `date` class, follows:

```
public boolean precedes(DateFourthTry otherDate)
{
 return ((year < otherDate.year) ||
```

---

<sup>4</sup> Later in this chapter, we will see that because `year` is marked `private`, it is not legal to write `date.year` in a program, but the meaning of such an expression is clear even if you cannot include it in a program.

```

 (year == otherDate.year && getMonth() < otherDate.getMonth())
 ||
 (year == otherDate.year && month.equals(otherDate.month)
 && day < otherDate.day));
 }

```

The version of our date class with this method is given in Display 4.7. The other new methods in that class will be discussed shortly in the subsection entitled “The Methods `equals` and `toString`.” Right now, let’s discuss this new method named `precedes`.

An invocation of the method `precedes` has the following form, where `date1` and `date2` are two objects of our date class:

```
date1.precedes(date2)
```

This is a Boolean expression that returns `true` if `date1` comes before `date2`. Because it is a Boolean expression, it can be used anywhere a Boolean expression is allowed, such as to control an `if-else` or `while` statement. For example,

```

if (date1.precedes(date2))
 System.out.println("date1 comes before date2.");

else
 System.out.println("date2 comes before or is equal to date1.");

```

Display 4.7 A Class with Methods `equals` and `toString` (part 1 of 2)

---

```

1 import java.util.Scanner;

2 public class DateFourthTry
3 {
4 private String month;
5 private int day;
6 private int year; //a four digit number.

7 public String toString()
8 {
9 return (month + " " + day + ", " + year);
10 }

11 public void writeOutput()
12 {
13 System.out.println(month + " " + day + ", " + year);
14 }

15 public boolean equals(DateFourthTry otherDate)
16 {
17 return ((month.equals(otherDate.month))
18 && (day == otherDate.day) && (year == otherDate.year));
19 }

```

*This is the method `equals` in the class `DateFourthTry`.*

*This is the method `equals` in the class `String`.*

(continued)



Display 4.7 A Class with Methods `equals` and `toString` (part 2 of 2)

---

```

20 public boolean precedes(DateFourthTry otherDate)
21 {
22 return ((year < otherDate.year) ||
23 (year == otherDate.year && getMonth() <
24 otherDate.getMonth()) ||
25 (year == otherDate.year && month.equals(otherDate.month)
26 && day < otherDate.day));
 }

```

*<The rest of the method definitions are identical to the ones in `DateThirdTry` in Display 4.4.>*

```

27 }

```

---

The return statement in the definition of the method `precedes` may look intimidating, but it is really straightforward. It says that `date1.precedes(date2)` returns true, provided one of the following three conditions is satisfied:

```

date1.year < date2.year
date1.year equals date2.year and date1.month comes before
date2.month date1 and date2 have the same year and month and
date1.day < date2.day.

```

If you give it a bit of thought, you will realize that `date1` precedes `date2` in time precisely when one of these three conditions is satisfied.

## The Methods `equals` and `toString`

Java expects certain methods to be in all, or almost all, classes. This is because some of the standard Java libraries have software that assumes such methods are defined. Two of these methods are `equals` and `toString`. Therefore, you should include such methods and be certain to spell their names exactly as we have done. Use `equals`, not `same` or `areEqual`. Do not even use `equal` without the `s`. Similar remarks apply to the `toString` method. After we have developed more material, we will explain this in more detail. In particular, we will then explain how to give a better method definition for `equals`. For now, just get in the habit of including these methods.

### `equals`

The method `equals` is a boolean valued method to compare two objects of the class to see if they satisfy the intuitive notion of “being equal.” So, the heading should be

```
public boolean equals(Class_Name Parameter_Name)
```

Display 4.7 contains definitions of the methods `equals` and `toString` that we might add to our date class, which is now named `DateFourthTry`. The heading of that `equals` method is

```
public boolean equals(DateFourthTry otherDate)
```

When you use the method `equals` to compare two objects of the class `DateFourthTry`, one object is the calling object and the other object is the argument, like so:

```
date1.equals(date2)
```

or equivalently,

```
date2.equals(date1)
```

Because the method `equals` returns a value of type `boolean`, you can use an invocation of `equals` as the Boolean expression in an `if-else` statement, as shown in Display 4.8. Similarly, you can also use it anywhere else that a Boolean expression is allowed.

There is no absolute notion of “equality” that you must follow in your definition of `equals`. You can define the method `equals` any way you wish, but to be useful, it should reflect some notion of “equality” that is useful for the software you are

Display 4.8 Using the Methods `equals` and `toString`

```

1 public class EqualsAndToStringDemo
2 {
3 public static void main(String[] args)
4 {
5 DateFourthTry date1 = new DateFourthTry(),
6 date2 = new DateFourthTry();
7 date1.setDate(6, 17, 1882);
8 date2.setDate(6, 17, 1882);

9 if (date1.equals(date2))
10 System.out.println(date1 + " equals " + date2);
11 else
12 System.out.println(date1 + " does not equal " + date2);

13 date1.setDate(7, 28, 1750);

14 if (date1.precedes(date2))
15 System.out.println(date1 + " comes before " + date2);
16 else
17 System.out.println(date2 + " comes before or is equal to "
18 + date1);
19 }
20 }
```

*These are equivalent to date1.toString().*

*These are equivalent to date2.toString().*

Sample Dialogue

```

June 17, 1882 equals June 17, 1882
July 28, 1750 comes before June 17, 1882

```

designing. A common way to define `equals` for simple classes of the kind we are looking at now is to say `equals` returns `true` if each instance variable of one object equals the corresponding instance variable of the other object. This is how we defined `equals` in Display 4.7.

If the definition of `equals` in Display 4.7 seems less than clear, it may help to rewrite it as follows using the `this` parameter:

```
public boolean equals(DateFourthTry otherDate)
{
 return (((this.month).equals(otherDate.month))
 && (this.day == otherDate.day) && (this.year ==
 otherDate.year));
}
```

So, if `date1` and `date2` are objects of the class `DateFourthTry`, then `date1.equals(date2)` returns `true` provided the three instance variables in `date1` have values that are equal to the three instance variables in `date2`.

Also, note that the method in the definition of `equals` that is used to compare months is not the `equals` for the class `DateFourthTry` but the `equals` for the class `String`. You know this because the calling object, which is `this.month`, is of type `String`.

Remember that we use the `equals` method of the class `String` because `==` does not work correctly for comparing `String` values. (This was discussed in the Pitfall section of Chapter 3 entitled “Using `==` with Strings.”)

In Chapter 7, you will see that there are reasons to make the definition of the `equals` method a bit more involved. But the spirit of what an `equals` method should be is very much like what we are now doing, and it is the best we can do with what we know so far.

#### `toString`

The method `toString` should be defined so that it returns a `String` value that represents the data in the object. One nice thing about the method `toString` is that it makes it easy to output an object to the screen. If `date` is of type `DateFourthTry`, then you can output the date to the screen as follows:

```
System.out.println(date.toString());
```

#### `println` used with objects

In fact, `System.out.println` was written so that it automatically invokes `toString()` if you do not include it. So, the object `date` can also be output by the following simpler and equivalent statement:

```
System.out.println(date);
```

This means that the method `writeOutput` in Display 4.7 is superfluous and could safely be omitted from the class definition.

If you look at Display 4.8, you will see that `toString` is also called automatically when the object is connected to some other string with a `+`, as in

```
System.out.println(date1 + " equals " + date2);
```

+ used  
with objects

In this case, it is really the plus operator that causes the automatic invocation of `toString()`. So, the following is also legal:

```
String s = date1 + " equals " + date2;
```

The preceding is equivalent to

```
String s = date1.toString() + " equals " + date2.toString();
```

## Recursive Methods

recursive  
method

Java does allow **recursive method** definitions. Recursive methods are covered in Chapter 11. If you do not know what recursive methods are, do not be concerned until you reach that chapter. If you want to read about these methods early, you can read Sections 11.1 and 11.2 of Chapter 11 after you complete Chapter 5.

### The Methods `equals` and `toString`

Usually, your class definitions should contain an `equals` method and a `toString` method. An `equals` method compares the calling object to another object and should return `true` when the two objects are intuitively equal. When comparing objects of a class type, you normally use the method `equals`, not `==`.

The `toString` method should return a string representation of the data in the calling object. If a class has a `toString` method, you can use an object of the class as an argument to the methods `System.out.println` and `System.out.print`.

See Display 4.7 for an example of a class with `equals` and `toString` methods.



### TIP: Testing Methods

driver  
program

Each method should be tested in a program in which it is the only untested program. If you test methods this way, then when you find an error, you will know which method contains the error. A program that does nothing but test a method is called a **driver program**.

bottom-up  
testing

If one method contains an invocation of another method in the same class, this can complicate the testing task. One way to test a method is to first test all the methods invoked by that method and then test the method itself. This is called **bottom-up testing**.

(continued)

**TIP: (continued)****stub**

It is sometimes impossible or inconvenient to test a method without using some other method that has not yet been written or has not yet been tested. In this case, you can use a simplified version of the missing or untested method. These simplified methods are called **stubs**. These stubs will not necessarily perform the correct calculation, but they will deliver values that suffice for testing, and they are simple enough that you can have confidence in their performance. For example, the following is a possible stub:

```
/**
 * Computes the probability of rain based on temperature, barometric
 * pressure, and relative humidity. Returns the probability as a
 * fraction between 0 and 1.
 */
public double rainChance(double temperature,
 double pressure, double humidity)
{
 return 0.5; //Not correct but good enough for a stub.
}
```

**The Fundamental Rule for Testing Methods**

Every method should be tested in a program in which every other method in the testing program has already been fully tested and debugged.

**Self-Test Exercises**

13. In the definition of `precedes` in Display 4.7, we used

```
month.equals(otherDate.month)
```

to test whether two months are equal; but we used

```
getMonth() < otherDate.getMonth()
```

to test whether one month comes before another. Why did we use `month` in one case and `getMonth` in another case?

14. What is the fundamental rule for testing methods?

## 4.2 Information Hiding and Encapsulation

*We all know—the Times knows—but we pretend we don't.*

VIRGINIA WOOLF, *Monday or Tuesday*, London: Hogarth Press, 1921.

### information hiding

**Information hiding** means that you separate the description of how to use a class from the implementation details, such as how the class methods are defined. You do this so that a programmer who uses the class does not need to know the implementation details of the class definition. The programmer who uses the class can consider the implementation details as hidden, since he or she does not need to look at them. Information hiding is a way of avoiding information overloading. It keeps the information needed by a programmer using the class within reasonable bounds. Another term for information hiding is **abstraction**. The use of the term *abstraction* for information hiding makes sense if you think about it a bit. When you abstract something, you are discarding some of the details.

### abstraction

### encapsulation

**Encapsulation** means grouping software into a unit in such a way that it is easy to use because there is a well-defined simple interface. So, encapsulation and information hiding are two sides of the same coin.

Java has a way of officially hiding details of a class definition. To hide details, mark them as `private`, a concept we discuss next.



VideoNote  
Information  
Hiding  
Example

### Encapsulation

**Encapsulation** means that the data and the actions are combined into a single item (in our case, a class object) and that the details of the implementation are hidden. The terms *information hiding* and *encapsulation* deal with the same general principle: If a class is well designed, a programmer who uses a class need not know all the details of the implementation of the class but need only know a much simpler description of how to use the class.

### API

The term **API** stands for *application programming interface*. The API for a class is a description of how to use the class. If your class is well designed, using the encapsulation techniques we discuss in this book, then a programmer who uses your class need only read the API and need not look at the details of your code for the class definition.

### ADT

The term **ADT** is short for *abstract data type*. An ADT is a data type that is written using good information hiding techniques.

### public and private Modifiers

public  
private

Compare the instance variables in Displays 4.1 and 4.2. In Display 4.1, each instance variable is prefaced with the modifier `public`. In Display 4.2, each instance variable is prefaced with the modifier `private`. The modifier `public` means that there are no restrictions on where the instance variable can be used. The modifier `private` means that the instance variable cannot be accessed by name outside of the class definition.

For example, the following would produce a compiler error message if used in a program:

```
DateSecondTry date = new DateSecondTry();
date.month = "January";
date.day = 1;
date.year = 2006;
```

In fact, any one of the three assignments would be enough to trigger a compiler error. This is because, as shown in Display 4.2, each of the instance variables `month`, `day`, and `year` is labeled `private`.

If, on the other hand, we had used the class `DateFirstTry` from Display 4.1 instead of the class `DateSecondTry` in the preceding code, the code would be legal and would compile and run with no error messages. This is because, in the definition of `DateFirstTry` (Display 4.1), each of the instance variables `month`, `day`, and `year` is labeled `public`.

It is considered good programming practice to make all instance variables `private`. As we will explain a little later in this chapter, this is intended to simplify the task of any programmer using the class. But before we say anything about how, on balance, this simplifies the job of a programmer who uses the class, let's see how it complicates the job of a programmer who uses the class.

Once you label an instance variable as `private`, there is then no way to change its value (nor to reference the instance variable in any other way) except by using one of the methods belonging to the class. Note that even when an instance variable is `private`, you can still access it through methods of the class. For the class `DateSecondTry`, you can change the values of the instance variables with the method `readInput`, and you can obtain the values of the instance variables with the methods whose names start with `get`. So, the qualifier `private` does not make it impossible to access the instance variables. It just makes it illegal to use their names, which can be a minor nuisance.

The modifiers `public` and `private` before a method definition have a similar meaning. If the method is labeled `public`, there are no restrictions on its usage. If the method is labeled `private`, the method can only be used in the definition of another method of the same class.

Any instance variable can be labeled either `public` or `private`. Any method can be `public` or `private`. However, normal good programming practices require that *all* instance variables be `private` and that typically, most methods be `public`. Normally, a method is `private` only if it is being used solely as a helping method in the definition of other methods.

### EXAMPLE: Yet Another Date Class

Display 4.9 contains another, much improved, definition of a class for a date. Note that all instance variables are private and that two methods are private. We made the methods `dateOK` and `monthString` private because they are just helping methods used in the definitions of other methods. A user of the class `DateFifthTry` would not (in fact, cannot) use either of the methods `dateOK` or `monthString`. This is all hidden information that need not concern a programmer using the class. The method `monthString` was public in previous versions of our date classes because we had not yet discussed the `private` modifier. It is now marked `private` because it is just a helping method.

Note that the class `DateFifthTry` uses the method `dateOK` to make sure that any changes to instance variables make sense. Because the methods of the class `DateFifthTry` use the method `dateOK` to check for impossible dates, you cannot use any methods, such as `readInput` or `setDate`, to set the instance variables so that they represent an impossible date such as January 63, 2005. If you try to do so, your program would end with an error message. (To make our definition of the method `dateOK` simple, we did not check for certain impossible dates, such as February 31, but it would be easy to exclude these dates as well.)

The methods `dateOK` and `equals` each return a value of type `boolean`. That means they return a value that is either `true` or `false` and so can be used as the Boolean expression in an `if-else` statement, `while` statement, or other loop statement. This is illustrated by the following, which is taken from the definition of the method `setDate` in Display 4.9:

```
if (dateOK(month, day, year))
{
 this.month = monthString(month);
 this.day = day;
 this.year = year;
}
else
{
 System.out.println("Fatal Error");
 System.exit(0);
}
```

Note that, although all the instance variables are private, a programmer using the class can still change or access the value of an instance variable using the methods that start with `set` or `get`. This is discussed more fully in the next subsection, “Accessor and Mutator Methods.”

(continued)



**EXAMPLE:** (continued)

Note that there is a difference between what we might call the *inside view* and the *outside view* of the class `DateFifthTry`. A date such as July 4, 1776, is represented inside the class object as the string value "July" and the two `int` values 4 and 1776. But if a programmer using the same class object asks for the date using `getMonth`, `getDay`, and `getYear`, he or she will get the three `int` values 7, 4, and 1776. From inside the class, a month is a string value, but from outside the class, a month is an integer. The description of the data in a class object need not be a simple direct description of the instance variables. (To further emphasize the fact that the month has an inside view as a string but an outside view as a number, we have written the method `readInput` for the class `DateFifthTry` so that the user enters the month as an integer rather than a string.)

Note that the method definitions in a class need not be given in any particular order. In particular, it is perfectly acceptable to give the definition the method `dateOK` after the definitions of methods that use `dateOK`. Indeed, any ordering of the method definitions is acceptable. Use whatever order seems to make the class easiest to read. (Those who come to Java from certain other programming languages should note that there is no kind of forward reference needed when a method is used before it is defined.)

**Self-Test Exercises**

15. Following the style guidelines given in this book, when should an instance variable be marked `private`?
16. Following the style guidelines given in this book, when should a method be marked `private`?

**Accessor and Mutator Methods**

You should always make all instance variables in a class `private`. However, you may sometimes need to do something with the data in a class object. The special-purpose methods, such as `toString`, `equals`, and any input methods, will allow you to do many things with the data in an object. But sooner or later you will want to do something with the data for which there are no special-purpose methods. How can you do anything new with the data in an object? The answer is that you can do anything that you might reasonably want (and that the class design specifications consider to be legitimate), provided you equip your classes with suitable *accessor* and *mutator* methods. These are methods that allow you to access and change the data in an object, usually in a very general way. **Accessor methods** allow you to obtain the data. In Display 4.9, the methods `getMonth`, `getDay`, and `getYear` are accessor methods. The

accessor methods need not literally return the values of each instance variable, but they must return something equivalent to those values. For example, the method `getMonth` returns the number of the month, even though the month is stored in a `String` instance variable. Although it is not required by the Java language, it is a generally accepted good programming practice to spell the names of accessor methods starting with `get`.

### mutator methods

**Mutator methods** allow you to change the data in a class object. In Display 4.9, the methods whose names begin with the word `set` are mutator methods. It is a generally accepted good programming practice to use names that begin with the word `set` for mutator methods. Your class definitions will typically provide a complete set of public accessor methods and at least some public mutator methods. There are, however, important classes, such as the class `String`, that have no public mutator methods.

At first glance, it may seem as if accessor and mutator methods defeat the purpose of making instance variables private, but if you look carefully at the mutator methods in Display 4.9, you will see that the mutator and accessor methods are not equivalent to making the instance variables public. Notice the mutator methods, that is, the ones that begin with `set`. They all test for an illegal date and end the program with an error message if there is an attempt to set the instance variables to any illegal values. If the variables were public, you could set the data to values that do not make sense for a date, such as January 42, 1930. With mutator methods, you can control and filter changes to the data. (As it is, you can still set the data to values that do not represent a real date, such as February 31, but as we already noted, it would be easy to exclude these dates as well. We did not exclude these dates to keep the example simple. See Self-Test Exercise 19 for a more complete date check method.)

Display 4.9 Yet Another Date Class (part 1 of 4)

```

1 import java.util.Scanner;

2 public class DateFifthTry
3 {
4 private String month;
5 private int day;
6 private int year; //a four digit number.

7 public void writeOutput()
8 {
9 System.out.println(month + " " + day + ", " + year);
10 }

11 public void readInput()
12 {

```

*Note that this version of `readInput` has the user enter the month as an integer rather than as a string. In this class, a month is an integer to the user, but is a string inside the class.*

(continued)

Display 4.9 Yet Another Date Class (part 2 of 4)

```

13 boolean tryAgain = true;
14 Scanner keyboard = new Scanner(System.in);
15 while (tryAgain)
16 {
17 System.out.println("Enter month, day, and year");
18 System.out.println("as three integers:");
19 System.out.println("do not use commas or other punctuations.");
20 int monthInput = keyboard.nextInt();
21 int dayInput = keyboard.nextInt();
22 int yearInput = keyboard.nextInt();
23 if (dateOK(monthInput, dayInput, yearInput))
24 {
25 setDate(monthInput, dayInput, yearInput);
26 tryAgain = false;
27 }
28 else
29 System.out.println("Illegal date. Reenter input.");
30 }
31 }

32 public void setDate(int month, int day, int year)
33 {
34 if (dateOK(month, day, year))
35 {
36 this.month = monthString(month);
37 this.day = day;
38 this.year = year;
39 }
40 else
41 {
42 System.out.println("Fatal Error");
43 System.exit(0);
44 }
45 }

46 public void setMonth(int monthNumber)
47 {
48 if ((monthNumber <= 0) || (monthNumber > 12))
49 {
50 System.out.println("Fatal Error");
51 System.exit(0);
52 }
53 else
54 month = monthString(monthNumber);
55 }

```

*Note that this version of readInput checks to see that the input is reasonable.*

## Display 4.9 Yet Another Date Class (part 3 of 4)

```

56 public void setDay(int day)
57 {
58 if ((day <= 0) || (day > 31))
59 {
60 System.out.println("Fatal Error");
61 System.exit(0);
62 }
63 else
64 this.day = day;
65 }

66 public void setYear(int year)
67 {
68 if ((year < 1000) || (year > 9999))
69 {
70 System.out.println("Fatal Error");
71 System.exit(0);
72 }
73 else
74 this.year = year;
75 }

76 public boolean equals(DateFifthTry otherDate)
77 {
78 return ((month.equalsIgnoreCase(otherDate.month))
79 && (day == otherDate.day) && (year ==
80 otherDate.year));
81 }

82 public boolean precedes(DateFifthTry otherDate)
83 {
84 return ((year < otherDate.year) ||
85 (year == otherDate.year && getMonth() <
86 otherDate.getMonth()) ||
87 (year == otherDate.year && month.equals(otherDate.month)
88 && day < otherDate.day));
89 }

```

*Within the definition of DateFifthTry, you can directly access private instance variables of any object of type DateFifthTry.*

*Within the definition of DateFifthTry, you can directly access private instance variables of any object of type DateFifthTry.*

<The definitions of the following methods are the same as in Displays 4.2 and 4.7:  
getMonth, getDay, getYear, and toString.>

```

88 private boolean dateOK(int monthInt, int dayInt, int yearInt)
89 {

```

(continued)

Display 4.9 Yet Another Date Class (part 4 of 4)

---

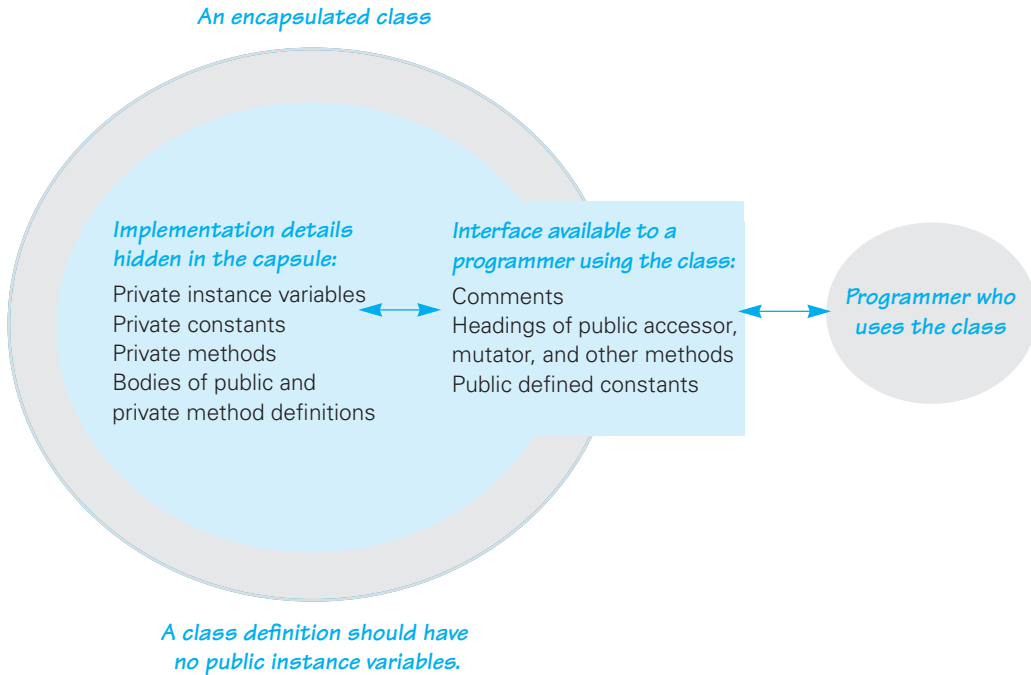
```
90 return ((monthInt >= 1) && (monthInt <= 12) &&
91 (dayInt >= 1) && (dayInt <= 31) &&
92 (yearInt >= 1000) && (yearInt <= 9999));
93 }

94 private String monthString(int monthNumber)
95 {
96 switch (monthNumber)
97 {
98 case 1:
99 return "January";
100 case 2:
101 return "February";
102 case 3:
103 return "March";
104 case 4:
105 return "April";
106 case 5:
107 return "May";
108 case 6:
109 return "June";
110 case 7:
111 return "July";
112 case 8:
113 return "August";
114 case 9:
115 return "September";
116 case 10:
117 return "October";
118 case 11:
119 return "November";
120 case 12:
121 return "December";
122 default:
123 System.out.println("Fatal Error");
124 System.exit(0);
125 return "Error"; //to keep the compiler happy
126 }
127 }
128 }
```

---

The way that a well-designed class definition uses private instance variables and public accessor and mutator methods to implement the principle of encapsulation is diagrammed in Display 4.10.

Display 4.10 Encapsulation



### TIP: A Class Has Access to Private Members of All Objects of the Class

Consider the definition of the method `equals` for the class `DateFifthTry`, given in Display 4.9 and repeated as follows:

```
public boolean equals(DateFifthTry otherDate)
{
 return ((month.equalsIgnoreCase(otherDate.month))
 && (day == otherDate.day) &&
 (year == otherDate.year));
}
```

You might object that `otherDate.month`, `otherDate.day`, and `otherDate.year` are illegal because `month`, `day`, and `year` are private instance variables of some object other than the calling object. Normally, that objection would be correct. However, the object `otherDate` is of the same type as the class being defined, so this is legal. In the definition of a class, you can access private members of any object of the class, not just private members of the calling object.

(continued)

**TIP: (continued)**

Similar remarks apply to the method `precedes` in the same class. In one place in the definition of `precedes`, we used `otherDate.getMonth()` rather than `otherDate.month` only because we wanted the month as an integer instead of a string. We did, in fact, use `otherDate.month` elsewhere in the definition of `precedes`. ■

**TIP: Mutator Methods Can Return a Boolean Value ★**

In the definition of the class `DateFifthTry` (Display 4.9), the mutator methods tested to see if the new values for instance variables were sensible values. If they were not, the mutator method ended the program and issued an error message. An alternative approach is to have the mutator do the test but to never have it end the program. Instead, it returns a boolean value. It makes the changes to the instance variables and returns `true` if the changes are sensible. If the attempted changes are not sensible, the mutator method returns `false`. That way a programmer can program in an alternative action to be taken if the attempted changes to instance variables do not make sense.

For example, an alternative definition of the method `setMonth` of the class `DateFifthTry` (Display 4.9) is the following:

```
public boolean setMonth(int monthNumber)
{
 if ((monthNumber <= 0) || (monthNumber > 12))
 return false;
 else
 {
 month = monthString(monthNumber);
 return true;
 }
}
```

A sample use of this boolean valued version of `setMonth` could be the following:

```
DateFifthTry date = new DateFifthTry();
...
System.out.println("Enter month as a number:");
int number = keyboard.nextInt();
while (!date.setMonth(number))
{
 System.out.println("Not a legal month number. Try again.");
 System.out.println("Enter month as a number:");
 number = keyboard.nextInt();
}
```

 ■

## Preconditions and Postconditions

### precondition

### postcondition

One good way to write a method comment is to break it down into two kinds of information, called the *precondition* and the *postcondition*. The **precondition** states what is assumed to be true when the method is called. The method should not be used and cannot be expected to perform correctly unless the precondition holds. The **postcondition** describes the effect of the method call; that is, the postcondition tells what will be true after the method is executed in a situation in which the precondition holds. For a method that returns a value, the postcondition describes the value returned by the method.

The following is an example of a method heading from Display 4.9 with a precondition and postcondition added:

```
/**
Precondition: All instance variables of the calling object have
values.
Postcondition: The data in the calling object has been written to
the screen.
*/
public void writeOutput()
```

You do not need to know the definition of the method `writeOutput` to use this method. All that you need to know to use this method is given by the precondition and postcondition. (The importance of this is more dramatic when the definition of the method is longer than that of `writeOutput`.)

When the only postcondition is a description of the value returned, programmers usually omit the word `Postcondition`, as in the following example:

```
/**
Precondition: All instance variables of the calling object have
values.
Returns a string describing the data in the calling object.
*/
public String toString()
```

Some programmers choose not to use the words *precondition* and *postcondition* in their method comments. However, whether you use the words or not, you should always think in terms of precondition and postcondition when designing a method and when deciding what to include in the method comment.

## Self-Test Exercises

17. List all the accessor methods in the class `DateFifthTry` in Display 4.9.
18. List all the mutator methods in the class `DateFifthTry` in Display 4.9.

(continued)



### Self-Test Exercises (continued)

19. Write a better version of the method `dateOK` with three `int` parameters (Display 4.9). This version should check for the correct number of days in each month and should not just allow 31 days in any month. It will help to define another helping method named `leapYear`, which takes an `int` argument for a year and returns `true` if the year is a leap year. February has 29 days in leap years and only 28 days in other years. Use the following rule for determining if the year is a leap year: A year is a leap year if it is divisible by 4 but is not divisible by 100, or if it is divisible by 400.

## 4.3 Overloading

*A good name is better than precious ointment.*

Ecclesiastes 7:1

Two (or more) different classes can have methods with the same name. For example, many classes have a method named `toString`. It is easy to see why this is acceptable. The type of the calling object allows Java to decide which definition of the method `toString` to use. It uses the definition of `toString` given in the definition of the class for the calling object. You may be more surprised to learn that two or more methods *in the same class* can have the same method name. This is called **overloading** and is the topic of this section.

overloading

### Rules for Overloading

In Display 4.11, we have added two methods named `setDate` to our date class so that there is a total of three methods named `setDate`. This is an example of overloading the method name `setDate`. On the following three lines, we display the headings of these three methods:

```
public void setDate(int month, int day, int year)
public void setDate(String month, int day, int year)
public void setDate(int year)
```

Notice that each method has a different parameter list. The first two differ in the type of their first parameter. The last one differs from the other two by having a different number of parameters.

The name of a method and the list of parameter types in the heading of the method definition is called the **method signature**. The signatures for these three method definitions are

```
setDate(int, int, int)
setDate(String, int, int)
setDate(int)
```

Method  
signature

Display 4.11 Overloading Method Names (part 1 of 2)

```
1 import java.util.Scanner;

2 public class DateSixthTry
3 {
4 private String month;
5 private int day;
6 private int year; //a four digit number.

7 public void setDate(int monthInt, int day, int year)
8 {
9 if (dateOK(monthInt, day, year))
10 {
11 this.month = monthString(monthInt);
12 this.day = day;
13 this.year = year;
14 }
15 else
16 {
17 System.out.println("Fatal Error");
18 System.exit(0);
19 }
20 }

21 public void setDate(String monthString, int day, int year)
22 {
23 if (dateOK(monthString, day, year))
24 {
25 this.month = monthString;
26 this.day = day;
27 this.year = year;
28 }
29 else
30 {
31 System.out.println("Fatal Error");
32 System.exit(0);
33 }
34 }

35 public void setDate(int year)
36 {
37 setDate(1, 1, year);
38 }
```

There are three different methods named `setDate`.

Two different methods named `setDate`

(continued)

Display 4.11 Overloading Method Names (part 2 of 2)

---

```

39 private boolean dateOK(int monthInt, int dayInt, int yearInt)
40 {
41 return ((monthInt >= 1) && (monthInt <= 12) &&
42 (dayInt >= 1) && (dayInt <= 31) &&
43 (yearInt >= 1000) && (yearInt <= 9999));
44 }

45 private boolean dateOK(String monthString, int dayInt, int yearInt)
46 {
47 return (monthOK(monthString) &&
48 (dayInt >= 1) && (dayInt <= 31) &&
49 (yearInt >= 1000) && (yearInt <= 9999));
50 }

51 private boolean monthOK(String month)
52 {
53 return (month.equals("January") || month.equals("February") ||
54 month.equals("March") || month.equals("April") ||
55 month.equals("May") || month.equals("June") ||
56 month.equals("July") || month.equals("August") ||
57 month.equals("September") || month.equals("October") ||
58 month.equals("November") || month.equals("December"));
59 }

60 public void readInput()
61 {
62 boolean tryAgain = true;
63 Scanner keyboard = new Scanner(System.in);
64 while (tryAgain)
65 {
66 System.out.println("Enter month, day, and year.");
67 System.out.println("Do not use a comma.");
68 String monthInput = keyboard.next();
69 int dayInput = keyboard.nextInt();
70 int yearInput = keyboard.nextInt();
71 if (dateOK(monthInput, dayInput, yearInput))
72 {
73 setDate(monthInput, dayInput, yearInput);
74 tryAgain = false;
75 }
76 else
77 System.out.println("Illegal date. Reenter input.");
78 }
79 }

 <The rest of the methods are the same as in Display 4.9, except that
 the parameter to equals and precedes is, of course, of type DateSixthTry.>
80 }

```

---

Two different  
methods named  
dateOK

When you overload a method name, each of the method definitions in the class must have a different signature.

### Signature

The **signature** of a method consists of the method name and the list of types for parameters that are listed in the heading of the method name.

#### EXAMPLE

If a method has the heading

```
public int computeSomething(int n1, double x1,
 double x2, String name);
```

then the signature is

```
computeSomething(int, double, double, String)
```

Note that the return type is not part of the method signature.

In Display 4.11, we also overloaded the method name `dateOK` so that there are two different methods named `dateOK`. The two signatures for the two methods named `dateOK` are

```
dateOK(int, int, int)
dateOK(String, int, int)
```

Display 4.12 gives a simple example of a program using the overloaded method name `setDate`. Note that for each invocation of a method named `setDate`, only one of the definitions of `setDate` has a signature that matches the types of the arguments.

### Overloading

Within one class, you can have two (or more) definitions of a single method name. This is called **overloading** the method name. When you overload a method name, any two definitions of the method name must have different signatures; that is, any two definitions of the method name either must have different numbers of parameters or some parameter position must be of differing types in the two definitions.



## PITFALL: Overloading and Automatic Type Conversion

Automatic type conversion of arguments (such as converting an `int` to a `double` when the parameter is of type `double`) and overloading can sometimes interact in unfortunate ways. So, you need to know how these two things interact.

For example, consider the following method that might be added to the class `DateSixthTry` in Display 4.11:

```
public void increase(double factor)
{
 year = (int)(year + factor*year);
}
```

If you add this method to the class `DateSixthTry`, then the following presents no problems, where `date` is an object of type `DateSixthTry` that has been set to some date:

```
date.increase(2);
```

The `int` value of 2 is type cast to the `double` value 2.0, and the value of `date.year` is changed as follows:

```
date.year = (int)(date.year + 2.0*date.year);
```

(Because `year` is private in the class `DateSixthTry`, you cannot write this in a program that uses the class `DateSixthTry`, but the meaning of this expression is clear.)

So far, so good. But now suppose we also add the following method definition to the class `DateSixthTry`:

```
public void increase(int term)
{
 year = year + term;
}
```

This is a valid overloading because the two methods named `increase` take parameters of different types. With both of these methods named `increase` added to the class, the following now behaves differently:

```
date.increase(2);
```

If Java can find an exact match of types, it uses the method definition with an exact match before it tries to do any automatic type casts. So now the displayed invocation of `date.increase` is equivalent to

```
date.year = date.year + 2;
```

However, if you meant to use an argument of 2.0 for `date.increase` and instead used 2, counting on an automatic type cast, then this is not what you want. It is best to avoid overloading where there is a potential for interacting dangerously with automatic type casting, as in the examples discussed in this Pitfall section.



## PITFALL: (continued)

In some cases of overloading, a single method invocation can be resolved in two different ways, depending on how overloading and type conversion interact. Such ambiguous method invocations are not allowed in Java and will produce an error message. For example, you can overload a method named `doSomething` by giving two definitions that have the following two method headings in a `SampleClass`:

```
public class SampleClass
{
 public void doSomething(double n1, int n2)
 .
 .
 .
 public void doSomething(int n1, double n2)
 .
 .
 .
}
```

Such overloading is legal, but there is a problem. Suppose `aSampleObject` is an object of type `SampleClass`. An invocation such as the following will produce an error message, because Java cannot decide which overloaded definition of `doSomething` to use:

```
aSampleObject.doSomething(5, 10);
```

Java cannot decide whether it should convert the `int` value 5 to a `double` value and use the first definition of `doSomething`, or whether it should convert the `int` value 10 to a `double` value and use the second definition. In this situation, the Java compiler issues an error message indicating that the method invocation is ambiguous.

The following two method invocations are allowed:

```
aSampleObject.doSomething(5.0, 10);
aSampleObject.doSomething(5, 10.0);
```

However, such situations, while legal, are confusing and should be avoided. ■

Display 4.12 Using an Overloaded Method Name (part 1 of 2)

```
1 public class OverloadingDemo
2 {
3 public static void main(String[] args)
4 {
5 DateSixthTry date1 = new DateSixthTry(),
6 date2 = new DateSixthTry(),
7 date3 = new DateSixthTry();
```

(continued)

Display 4.12 Using an Overloaded Method Name (part 2 of 2)

```

8 date1.setDate(1, 2, 2008);
9 date2.setDate("February", 2, 2008);
10 date3.setDate(2008);

11 System.out.println(date1);
12 System.out.println(date2);
13 System.out.println(date3);
14 }
15 }

```

Sample Dialogue

```

January 2, 2008
February 2, 2008
January 1, 2008

```

### Overloading and Automatic Type Conversion

Java always looks for a method signature that exactly matches the method invocation before it tries to use automatic type conversion. If Java can find a definition of a method that exactly matches the types of the arguments, it uses that definition. Only after it fails to find an exact match does Java try automatic type conversions to find a method definition that matches the (type cast) types of the method invocation.



### PITFALL: You Cannot Overload Based on the Type Returned

Note that the signature of a method lists only the method name and the types of the parameters and does not include the type returned. When you overload a method name, any two methods must have different signatures. The type returned has nothing to do with the signature of a method. For example, a class could not have two method definitions with the following headings:

```

public class SampleClass2
{
 public int computeSomething(int n)
 .
 .
 .
 public double computeSomething(int n)
 .
 .
 .
}

```



## PITFALL: (continued)

If you think about it, there is no way that Java could allow this sort of overloading. Suppose `anObject` is an object of the class `SampleClass2`. Then in the following assignment, Java could not decide which of the above two method definitions to use:

```
double answer = anObject.computeSomething(10);
```

Either a value of type `int` or a value of type `double` can legally be assigned to the variable `answer`. So, either method definition could be used. Because of such problems, Java says it is illegal to have both of these method headings in the same class. ■

## You Cannot Overload Operators in Java

Many programming languages, such as C++, allow you to overload an operator, such as `+`, so that the operator can be used with objects of some class you define as well as be used for such things as numbers. You cannot do this in Java. If you want to have an “addition” in your class, you must use a method name, such as `add`, and ordinary method syntax; you cannot define operators, such as the `+` operator, to work with objects of a class you define.

## Self-Test Exercises

20. What is the signature of each of the following method headings?

```
public void doSomething(int p1, char p2, int p3)
public void setMonth(int newMonth)
public void setMonth(String newMonth)
public int amount(int balance, double duration)
public double amount(int balance, double duration)
```

21. Consider the class `DateSixthTry` in Display 4.11. Would it be legal to add two method definitions with the following two method headings to the class `DateSixthTry`?

```
public void setMonth(int newMonth)
public void setMonth(String newMonth)
```

22. Consider the class `DateSixthTry` in Display 4.11. Would it be legal to add two method definitions with the following two method headings to the class `DateSixthTry`?

```
public void setMonth(int newMonth)
private void setMonth(int newMonth)
```

(continued)



### Self-Test Exercises (continued)

23. Consider the class `DateSixthTry` in Display 4.11. Would it be legal to add two method definitions with the following two method headings to the class `DateSixthTry`?

```
public int getMonth()

public String getMonth()
```

## 4.4 Constructors

*Well begun is half done.*

Anonymous.

### constructor

You often want to initialize the instance variables for an object when you create the object. As you will see later in this book, there are other initializing actions you might also want to take, but initializing instance variables is the most common sort of initialization. A **constructor** is a special variety of method that is designed to perform such initialization. In this section, we tell you how to define and use constructors.

### Constructor Definitions

Although you may not have realized it, you have already been using constructors every time you used the `new` operator to create an object, as in the following example:

```
DateSixthTry date1 = new DateSixthTry();
```

The expression `new DateSixthTry()` is an invocation of a constructor. A constructor is a special variety of method that, among other things, must have the same name as the class. So, the first occurrence of `DateSixthTry` in the previous code is a class name, and the second occurrence of `DateSixthTry` is the name of a constructor. If you add no constructor definitions to your class, then Java automatically creates a constructor that takes no arguments. We have been using this automatically provided constructor up until now. The automatically provided constructor creates the object but does little else. It is preferable to define your own constructors so that you can have the constructor initialize instance variables, or do whatever other initialization actions you want.

In Display 4.13, we have rewritten our date class one last time by adding five constructors. Since this is our final date class, we have included all method definitions in the display so you can see the entire class definition. (We have omitted `writeOutput` because it would be superfluous, as noted in the earlier subsection entitled “The Methods `equals` and `toString`.”)

Display 4.13 A Class with Constructors (part 1 of 5)

```

1 import java.util.Scanner;

2 public class Date
3 {
4 private String month;
5 private int day;
6 private int year; //a four digit number.

7 public Date()
8 {
9 month = "January";
10 day = 1;
11 year = 1000;
12 }

13 public Date(int monthInt, int day, int year)
14 {
15 setDate(monthInt, day, year);
16 }

17 public Date(String monthString, int day, int year)
18 {
19 setDate(monthString, day, year);
20 }

21 public Date(int year)
22 {
23 setDate(1, 1, year);
24 }

25 public Date(Date aDate)
26 {
27 if (aDate == null) //Not a real date.
28 {
29 System.out.println("Fatal Error.");
30 System.exit(0);
31 }
32 month = aDate.month;
33 day = aDate.day;
34 year = aDate.year;
35 }

```

*This is our final definition of a class whose objects are dates.*

*No-argument constructor*

*You can invoke another method inside a constructor definition.*

*A constructor usually initializes all instance variables, even if there is not a corresponding parameter.*

*We will have more to say about this constructor in Chapter 5. Although you have had enough material to use this constructor, you need not worry about it until Section 5.3 of Chapter 5.*

(continued)

Display 4.13 A Class with Constructors (part 2 of 5)

```
36 public void setDate(int monthInt, int day, int year)
37 {
38 if (dateOK(monthInt, day, year))
39 {
40 this.month = monthString(monthInt);
41 this.day = day;
42 this.year = year;
43 }
44 else
45 {
46 System.out.println("Fatal Error");
47 System.exit(0);
48 }
49 }

50 public void setDate(String monthString, int day, int year)
51 {
52 if (dateOK(monthString, day, year))
53 {
54 this.month = monthString;
55 this.day = day;
56 this.year = year;
57 }
58 else
59 {
60 System.out.println("Fatal Error");
61 System.exit(0);
62 }
63 }

64 public void setDate(int year)
65 {
66 setDate(1, 1, year);
67 }

68 public void setYear(int year)
69 {
70 if ((year < 1000) || (year > 9999))
71 {
72 System.out.println("Fatal Error");
73 System.exit(0);
74 }
75 else
76 this.year = year;
77 }
```

Display 4.13 A Class with Constructors (part 3 of 5)

```

78 public void setMonth(int monthNumber)
79 {
80 if ((monthNumber <= 0) || (monthNumber > 12))
81 {
82 System.out.println("Fatal Error");
83 System.exit(0);
84 }
85 else
86 month = monthString(monthNumber);
87 }

88 public void setDay(int day)
89 {
90 if ((day <= 0) || (day > 31))
91 {
92 System.out.println("Fatal Error");
93 System.exit(0);
94 }
95 else
96 this.day = day;
97 }

98 public int getMonth()
99 {
100 if (month.equals("January"))
101 return 1;
102 else if (month.equals("February"))
103 return 2;
104 else if (month.equals("March"))
105 return 3;
106 . . .
107 <The omitted cases are obvious, but if need be, you can see all the cases in Display 4.2.>
108 . . .
109 else if (month.equals("November"))
110 return 11;
111 else if (month.equals("December"))
112 return 12;
113 else
114 {
115 System.out.println("Fatal Error");
116 System.exit(0);
117 return 0; //Needed to keep the compiler happy
118 }
119 }

```

(continued)

Display 4.13 A Class with Constructors (part 4 of 5)

---

```

117 public int getDay()
118 {
119 return day;
120 }
121 public int getYear()
122 {
123 return year;
124 }

125 public String toString()
126 {
127 return (month + " " + day + ", " + year);
128 }

129 public boolean equals(Date otherDate)
130 {
131 return ((month.equals(otherDate.month))
132 && (day == otherDate.day)
133 && (year == otherDate.year));
134 }

134 public Boolean precedes(Date otherDate)
135 {
136 return ((year < otherDate.year) ||
137 (year == otherDate.year && getMonth() <
138 otherDate.getMonth()) ||
138 (year == otherDate.year && month.equals(otherDate.month)
139 && day < otherDate.day));
140 }

141 public void readInput()
142 {
143 boolean tryAgain = true;
144 Scanner keyboard = new Scanner(System.in);
145 while (tryAgain)
146 {
147 System.out.println("Enter month, day, and year.");
148 System.out.println("Do not use a comma.");
149 String monthInput = keyboard.next();
150 int dayInput = keyboard.nextInt();
151 int yearInput = keyboard.nextInt();
152 if (dateOK(monthInput, dayInput, yearInput))
153 {
154 setDate(monthInput, dayInput, yearInput);
155 tryAgain = false;
156 }

```

*We have omitted the method `writeOutput` because it would be superfluous, as noted in the subsection entitled "The Methods `equals` and `toString`."*

*The method `equals` of the class `String`*



Display 4.13 A Class with Constructors (part 5 of 5)

---

```

157 else
158 System.out.println("Illegal date. Reenter input.");
159 }
160 }

161 private boolean dateOK(int monthInt, int dayInt, int yearInt)
162 {
163 return ((monthInt >= 1) && (monthInt <= 12) &&
164 (dayInt >= 1) && (dayInt <= 31) &&
165 (yearInt >= 1000) && (yearInt <= 9999));
166 }

167 private boolean dateOK(String monthString, int dayInt, int
168 yearInt)
169 {
170 return (monthOK(monthString) &&
171 (dayInt >= 1) && (dayInt <= 31) &&
172 (yearInt >= 1000) && (yearInt <= 9999));
173 }

174 private boolean monthOK(String month)
175 {
176 return (month.equals("January") || month.equals("February") ||
177 month.equals("March") || month.equals("April") ||
178 month.equals("May") || month.equals("June") ||
179 month.equals("July") || month.equals("August") ||
180 month.equals("September") || month.equals("October") ||
181 month.equals("November") || month.equals("December"));
182 }

183 private String monthString(int monthNumber)
184 {
185 switch (monthNumber)
186 {
187 case 1:
188 return "January";
189 . . .
190 default:
191 System.out.println("Fatal Error");
192 System.exit(0);
193 return "Error"; //to keep the compiler happy
194 }
195 }

```

<The omitted cases are obvious, but if need be, you can see all the cases in Display 4.9.>

---

In Display 4.13, we have used overloading to create five constructors for the class `Date`. It is normal to have more than one constructor. Because every constructor must have the same name as the class, all the constructors in a class must have the same name. So, when you define multiple constructors, you must use overloading.

Note that when you define a constructor, you do not give any return type for the constructor; you do not even use `void` in place of a return type. Also notice that constructors are normally public.

All the constructor definitions in Display 4.13 initialize all the instance variables, even if there is no parameter corresponding to that instance variable. This is normal.

In a constructor definition, you can do pretty much anything that you can do in any ordinary method definition, but normally you perform only initialization tasks such as initialization of instance variables.

When you create a new object with the operator `new`, you must always include the name of a constructor after the operator `new`. This is the way you invoke a constructor. As with any method invocation, you list any arguments in parentheses after the constructor name (which is the same as the class name). For example, suppose you want to use `new` to create a new object of the class `Date` defined in Display 4.13. You might do so as follows:

```
Date birthday = new Date("December", 16, 1770);
```

This is a call to the constructor for the class `Date` that takes three arguments: one of type `String` and two of type `int`. This creates a new object to represent the date December 16, 1770, and sets the variable `birthday` so that it names this new object. Another example is the following:

```
Date newYearsDay = new Date(3000);
```

This creates a new object to represent the date January 1, 3000, and sets the variable `newYearsDay` so that it names this new object.

### constructor arguments

## Constructor

A **constructor** is a variety of method that is called when an object of the class is created using `new`. Constructors are used to initialize objects. A constructor must have the same name as the class to which it belongs. Arguments for a constructor are given in parentheses after the class name, as in the following examples.

### EXAMPLES

```
Date birthday = new Date("December", 16, 1770),
 theDate = new Date(2008);
```

A constructor is defined very much like any ordinary method except that it does not have a type returned and does not even include a `void` in the constructor heading. See Display 4.13 for examples of constructor definitions.

### resetting object values

A constructor is called when you create a new object, such as with the operator `new`. An attempt to call a constructor in any other way, such as the following, is illegal:

```
birthday.Date("January", 27, 1756); //Illegal!
```

Because you cannot call a constructor for an object after it is created, you need some other way to change the values of the instance variables of an object. That is the purpose of the `setDate` methods and other methods that begin with `set` in Display 4.13. If `birthday` already names an object that was created with `new`, you can change the values of the instance variables as follows:

```
birthday.setDate("January", 27, 1756);
```

Although it is not required, such methods that reset instance variables normally are given names that start with `set`.

Although you cannot use a constructor to reset the instance variables of an already created object, you can do something that looks very similar. The following is legal:

```
Date birthday = new Date("December", 16, 1770);
 .
 .
 .
birthday = new Date("January", 27, 1756);
```

However, the second invocation of the constructor does not simply change the values of instance variables for the object. Instead, it discards the old object and allocates storage for a new object before setting the instance variables. So, for efficiency (and occasionally for other reasons we have not yet discussed), it is preferable to use a method such as `setDate` to change the data in the instance variables of an already created object.

Display 4.14 contains a demonstration program for the constructors defined in Display 4.13.

Display 4.14 Use of Constructors (part 1 of 2)

```
1 public class ConstructorsDemo
2 {
3 public static void main(String[] args)
4 {
5 Date date1 = new Date("December", 16, 1770),
6 date2 = new Date(1, 27, 1756),
7 date3 = new Date(1882),
8 date4 = new Date();
9 System.out.println("Whose birthday is " + date1 + "?");
10 System.out.println("Whose birthday is " + date2 + "?");
11 System.out.println("Whose birthday is " + date3 + "?");
12 System.out.println("The default date is " + date4 + ".");
13 }
14 }
```

(continued)



Display 4.14 Use of Constructors (part 2 of 2)

## Sample Dialogue

```
Whose birthday is December 16, 1770?
Whose birthday is January 27, 1756?
Whose birthday is January 1, 1882?
The default date is January 1, 1000.
```

**Is a Constructor Really a Method?**

There are differing opinions on whether or not a constructor should be called a *method*. Most authorities call a constructor a method but emphasize that it is a special kind of method with many properties not shared with other kinds of methods. Some authorities say a constructor is a method-like entity but not, strictly speaking, a method. All authorities agree about what a constructor is; the only disagreement is over whether or not it should be referred to as a *method*. Thus, this is not a major issue. However, whenever you hear a phrase such as “all methods,” you should make sure you know whether it does or does not include constructors. To avoid confusion, we try to use the phrase “constructors and methods” when we want to include constructors.

**TIP: You Can Invoke Another Method in a Constructor**

It is perfectly legal to invoke another method within the definition of a constructor. For example, several of the constructors in Display 4.13 invoke a mutator method to set the values of the instance variables. This is legal because the first action taken by a constructor is to automatically create an object with instance variables. You do not write any code to create this object. Java creates it automatically when the constructor is invoked. Any method invocation in the body of the constructor definition has this object as its calling object.

You can even include an invocation of one constructor within the definition of another constructor. However, we will not discuss the syntax for doing that in this chapter. It will be covered in Chapter 7. ■

**TIP: A Constructor Has a `this` Parameter**

Just like the ordinary methods we discussed before we introduced constructors, every constructor has a `this` parameter. The `this` parameter can be used explicitly, but it is more often understood to be present although not written down. Whenever an instance variable of the class is used in a constructor (without an object name and a dot before it), it is understood to have an implicit `this` and dot before it. Similarly, whenever



### TIP: (continued)

a method is used in a constructor and the method has no explicit calling object, the method is understood to have `this` and a dot before it; that is, it is understood to have `this` as its calling object.

As noted in the previous Programming Tip, the first action taken by a constructor is to automatically create an object with instance variables. This object is automatically plugged in for the `this` parameter. So, within the definition of a constructor, the `this` parameter refers to the object created by the constructor. ■



### TIP: Include a No-Argument Constructor

no-argument  
constructor

A constructor that takes no arguments is called a **no-argument constructor** or **no-arg constructor**. If you define a class and include absolutely no constructors of any kind, then a no-argument constructor is automatically created. This no-argument constructor does not do much, but it does give you an object of the class type. So, if the definition of the class `MyClass` contains absolutely no constructor definitions, then the following is legal:

```
MyClass myObject = new MyClass();
```

If your class definition includes one or more constructors of any kind, then no constructor is generated automatically. So, for example, suppose you define a class called `YourClass`. If you include one or more constructors that each take one or more arguments, but you do not include a no-argument constructor in your class definition, then there is not a no-argument constructor, and the following is illegal:

```
YourClass yourObject = new YourClass();
```

The problem with the above declaration is that it asks the compiler to invoke the no-argument constructor, but there is not a no-argument constructor present.

To avoid problems, you should normally include a no-argument constructor in any class you define. If you do not want the no-argument constructor to initialize any instance variables, you can simply give it an empty body when you implement it. The following constructor definition is perfectly legal. It does nothing but create an object (and, as we will see later in this chapter, set the instance variables equal to default values):

```
public MyClass()
{/*Do nothing.*/}
```

(continued)



### default constructor

#### TIP: (continued)

A no-argument constructor is also known as a **default constructor**. However, the term *default constructor* is misleading because, as we have explained, a no-argument constructor is not always provided by default. There is now a movement to replace *default constructor* with the term *no-argument constructor*, but you will frequently encounter the former term. ■

#### No-Argument Constructor

A constructor with no parameters is called a **no-argument constructor**. If your class definition contains absolutely no constructor definitions, then Java will automatically create a no-argument constructor. If your class definition contains one or more constructor definitions, then Java does not automatically generate any constructor; in this case, what you define is what you get. Most of the classes you define should include a definition of a no-argument constructor.

#### EXAMPLE: The Final Date Class

The final version of our class for a date is given in Display 4.13. We will be using this class `Date` again in Chapter 5.

#### Self-Test Exercises

24. If a class is named `CoolClass`, what names are allowed as names for constructors in the class `CoolClass`?
25. Suppose you have defined a class such as the following for use in a program:

```
public class YourClass
{
 private int information;
 private char moreInformation;
 public YourClass(int newInfo, char moreNewInfo)
 {
 <Details not shown.>
 }
 public YourClass()
 {
```

### Self-Test Exercises (continued)

```

 <Details not shown.>
 }
 public void doStuff()
 {
 <Details not shown.>
 }
}

```

Which of the following are legal in a program that uses this class?

```

YourClass anObject = new YourClass(42, 'A');
YourClass anotherObject = new YourClass(41.99, 'A');
YourClass yetAnotherObject = new YourClass();
yetAnotherObject.doStuff();
YourClass oneMoreObject;
oneMoreObject.doStuff();
oneMoreObject.YourClass(99, 'B');

```

26. What is a no-argument constructor? Does every class have a no-argument constructor? What is a default constructor?

## Default Variable Initializations

Local variables are not automatically initialized in Java, so you must explicitly initialize a local variable before using it. Instance variables, on the other hand, are automatically initialized. Instance variables of type `boolean` are automatically initialized to `false`. Instance variables of other primitive types are automatically initialized to the zero of their type. Instance variables of a class type are automatically initialized to `null`, which is a kind of placeholder for an object that will be filled in later. We will discuss `null` in Chapter 5. Although instance variables are automatically initialized, we prefer to always explicitly initialize them in a constructor, even if the initializing value is the same as the default initialization. That makes the code clearer.

## An Alternative Way to Initialize Instance Variables

Instance variables are normally initialized in constructors, which is where we prefer to initialize them. However, there is an alternative. You can initialize instance variables when you declare them in a class definition, as illustrated by the following:

```

public class Date
{
 private String month = "January";
 private int day = 1;
 private int year = 1000;
}

```

If you initialize instance variables in this way, you may or may not want to define constructors. But if you do define any constructors, it is usually best to define a no-argument constructor even if the body of the no-argument constructor is empty.

### EXAMPLE: A Pet Record Class

Display 4.15 contains another example of a class definition. In this case, the objects of the class represent pet records consisting of the pet's name, age, and weight. Notice the similarities and differences between the constructors and the mutator methods (the ones whose names begin with `set`). They both set instance variables, but they are used differently. The constructors are used to create and initialize new objects of the class. However, after the object is created using a constructor and `new`, any changes to the object are performed by the mutator methods such as `set` or `setAge`. This is illustrated by the program in Display 4.16.

It would be possible to use constructors in place of the mutators, such as the method `set`. For example, the program in Display 4.16 would produce the same dialogue if you replace the line

```
usersPet.set(name, age, weight);
```

with

```
usersPet = new Pet(name, age, weight);
```

Even so, this use of constructors is a bad idea.

The following mutator method invocation simply changes the values of the instance variables of the object named by `usersPet`:

```
usersPet.set(name, age, weight);
```

However, the following use of a constructor creates a completely new object, which is a much less efficient process than just changing the values of some instance variables:

```
usersPet = new Pet(name, age, weight);
```

Display 4.15 A Class for Pet Records (part 1 of 4)

```
1 /**
2 Class for basic pet records: name, age, and weight.
3 */
4 public class Pet
5 {
6 private String name;
7 private int age; //in years
8 private double weight; //in pounds
9 }
```

Display 4.15 A Class for Pet Records (part 2 of 4)

```

10 public String toString()
11 {
12 return ("Name: " + name + " Age: " + age + " years"
13 + "\nWeight: " + weight + " pounds");
14 }
15
16 public Pet(String initialName,int initialAge,
17 double initialWeight)
18 {
19 name = initialName;
20 if ((initialAge < 0) || (initialWeight < 0))
21 {
22 System.out.println("Error: Negative age or weight.");
23 System.exit(0);
24 }
25 else
26 {
27 age = initialAge;
28 weight = initialWeight;
29 }
30 }
31
32 public void set(String newName,int newAge,double newWeight)
33 {
34 name = newName;
35 if ((newAge < 0) || (newWeight < 0))
36 {
37 System.out.println("Error: Negative age or weight.");
38 System.exit(0);
39 }
40 else
41 {
42 age = newAge;
43 weight = newWeight;
44 }
45 }
46 public Pet(String initialName)
47 {
48 name = initialName;
49 age = 0;
50 weight = 0;
51 }
52
53 public void setName(String newName)
54 {
55 name = newName;
56 }
57

```

*Constructors are only called when you create an object, such as with `new`. To change an already existing object, you use one or more methods such as these set methods.*

*Constructors normally set all instance variables, even if there is not a full set of parameters.*

*Age and weight are unchanged.*

(continued)

Display 4.15 A Class for Pet Records (part 3 of 4)

```
58 public Pet(int initialAge)
59 {
60 name = "No name yet.";
61 weight = 0;
62 if (initialAge < 0)
63 {
64 System.out.println("Error: Negative age.");
65 System.exit(0);
66 }
67 else
68 age = initialAge;
69 }
70
71 public void setAge(int newAge)
72 {
73 if (newAge < 0)
74 {
75 System.out.println("Error: Negative age.");
76 System.exit(0);
77 }
78 else
79 age = newAge;
80 }
81
82
83
84
85
86 public Pet(double initialWeight)
87 {
88 name = "No name yet.";
89 age = 0;
90 if (initialWeight < 0)
91 {
92 System.out.println("Error: Negative weight.");
93 System.exit(0);
94 }
95 else
96 weight = initialWeight;
97 }
98 public void setWeight(double newWeight)
99 {
```

*Name and weight are unchanged.* ←

Display 4.15 A Class for Pet Records (part 4 of 4)

```
100 if (newWeight < 0)
101 {
102 System.out.println("Error: Negative weight.");
103 System.exit(0);
104 }
105 else
106 weight = newWeight; 
107 }
108 public Pet()
109 {
110 name = "No name yet.";
111 age = 0;
112 weight = 0;
113 }
114
115 public String getName()
116 {
117 return name;
118 }
119
120 public int getAge()
121 {
122 return age;
123 }
124
125 public double getWeight()
126 {
127 return weight;
128 }
129 }
```

Display 4.16 Using Constructors and Set Methods (part 1 of 2)

```
1 import java.util.Scanner;
2 public class PetDemo
3 {
4 public static void main(String[] args)
5 {
```

(continued)



Display 4.16 Using Constructors and Set Methods (part 2 of 2)

```

6 Pet usersPet = new Pet("Jane Doe");
7 System.out.println("My records on your pet are incomplete.");
8 System.out.println("Here is what they currently say:");
9 System.out.println(usersPet);

10 Scanner keyboard = new Scanner(System.in);
11 System.out.println("Please enter the pet's name:");
12 String name = keyboard.nextLine();
13 System.out.println("Please enter the pet's age:");
14 int age = keyboard.nextInt();
15 System.out.println("Please enter the pet's weight:");
16 double weight = keyboard.nextDouble();
17 usersPet.set(name, age, weight);
18 System.out.println("My records now say:");
19 System.out.println(usersPet);
20 }
21 }

```

*This is equivalent to*  
**`System.out.println(usersPet.toString());`**

## Sample Dialogue

```

My records on your pet are incomplete.
Here is what they currently say:
Name: Jane Doe Age: 0 years
Weight: 0.0 pounds
Please enter the pet's name:
Fang Junior
Please enter the pet's age:
5
Please enter the pet's weight:
87.5
My records now say:
Name: Fang Junior Age: 5 years
Weight: 87.5 pounds

```

## The StringTokenizer Class ★

The `StringTokenizer` class is used to recover the words in a multiword string. It is often used when reading input. However, when we covered input in Chapter 2, we could not cover the `StringTokenizer` class because use of the `StringTokenizer` class normally involves knowledge of loops and constructors, two topics that we had not yet covered. We now have covered enough material to explain the `StringTokenizer` class.

One approach to reading keyboard input is to read an entire line of input into a variable of type `String`—for example, with the method `nextLine` of the `Scanner` class—and then to use the `StringTokenizer` class to decompose the string in the variable into words.

The class `StringTokenizer` is in the standard Java package (library) `java.util`. To tell Java where to find the class `StringTokenizer`, any class or program that uses the class `StringTokenizer` must contain the following (or something similar) at the start of the file:

```
import java.util.StringTokenizer;
```

Perhaps the most common use of the `StringTokenizer` class is to decompose a line of input. However, the `StringTokenizer` class can be used to decompose any string. The following example illustrates a typical way that the class `StringTokenizer` is used:

```
StringTokenizer wordFactory =
 new StringTokenizer("A single word can be critical.");
while(wordFactory.hasMoreTokens())
{
 System.out.println(wordFactory.nextToken());
}
```

This will produce the following output:

```
A
single
word
can
be
critical.
```

The constructor invocation

```
new StringTokenizer("A single word can be critical.")
```

produces a new object of the class `StringTokenizer`. The assignment statement

```
StringTokenizer wordFactory =
 new StringTokenizer("A single word can be critical.");
```

gives this `StringTokenizer` object the name `wordFactory`. You may use any string in place of `"A single word can be critical."` and any variable name in place of `wordFactory`. The `StringTokenizer` object created in this way can be used to produce the individual words in the string used as the argument to the `StringTokenizer` constructor. These individual words are called **tokens**.

tokens

**nextToken**

The method `nextToken` returns the first token (word) when it is invoked for the first time, returns the second token when it is invoked the second time, and so forth. If your code invokes `nextToken` after it has returned all the tokens in its string, then your program will halt and issue an error message.

**hasMore  
Tokens**

The method `hasMoreTokens` is a method that returns a value of type `boolean`; that is, it returns either `true` or `false`. Thus, an invocation of `hasMoreTokens`, such as the following

```
wordFactory.hasMoreTokens()
```

is a Boolean expression, and so it can be used to control a `while` loop. The method `hasMoreTokens` returns `true` as long as `nextToken` has not yet returned all the tokens in the string, and it returns `false` after the method `nextToken` has returned all the tokens in the string.

**choosing  
delimiters**

When the constructor for `StringTokenizer` is used with a single argument, as in the preceding example, the tokens are substrings of nonwhitespace characters, and the whitespace characters are used as the separators for the tokens. Any string of one or more whitespace characters is considered a separator. Thus, in the preceding example, the last token produced by the method `nextToken` is `"critical."` including the period. This is because the period is not a whitespace character and so is not a separator.

You can specify your own set of separator characters. When you create your own set of separator characters, you give a second argument to the constructor for `StringTokenizer`. The second argument is a string consisting of all the separator characters. Thus, if you want your separators to consist of the blank, the new-line character, the period, and the comma, you could proceed as in the following example:

```
StringTokenizer wordfactory2 =
 new StringTokenizer("Give me the word, my friend.", " \n.,");
while(wordfactory2.hasMoreTokens())
{
 System.out.println(wordfactory2.nextToken());
}
```

This will produce the output

```
Give
me
the
word
my
friend
```

Notice that the period and comma are not part of the tokens produced, because they are now token separators. Also note that the string of token separators is the second argument to the constructor.

Some of the methods for the class `StringTokenizer` are summarized in Display 4.17. A sample use of `StringTokenizer` is given in Display 4.18.

Display 4.17 Some Methods in the Class `StringTokenizer`

---

The class `StringTokenizer` is in the `java.util` package.

```
public StringTokenizer(String theString)
```

Constructor for a tokenizer that will use whitespace characters as separators when finding tokens in `theString`.

```
public StringTokenizer(String theString, String delimiters)
```

Constructor for a tokenizer that will use the characters in the string `delimiters` as separators when finding tokens in `theString`.

```
public boolean hasMoreTokens()
```

Tests whether there are more tokens available from this tokenizer's string. When used in conjunction with `nextToken`, it returns true as long as `nextToken` has not yet returned all the tokens in the string; returns false otherwise.

```
public String nextToken()
```

Returns the next token from this tokenizer's string. (Throws `NoSuchElementException` if there are no more tokens to return.)<sup>5</sup>

```
public String nextToken(String delimiters)
```

First changes the delimiter characters to those in the string `delimiters`. Then returns the next token from this tokenizer's string. After the invocation is completed, the delimiter characters are those in the string `delimiters`. (Throws `NoSuchElementException` if there are no more tokens to return. Throws `NullPointerException` if `delimiters` is null.)<sup>5</sup>

```
public int countTokens()
```

Returns the number of tokens remaining to be returned by `nextToken`.

---

<sup>5</sup>Exceptions are covered in Chapter 9. You can ignore any reference to `NoSuchElementException` until you reach Chapter 9. We include it here for reference value only.

Display 4.18 Use of the StringTokenizer Class

---

```

1 import java.util.Scanner;
2 import java.util.StringTokenizer;

1 public class StringTokenizerDemo
2 {
3 public static void main(String[] args)
4 {
5 Scanner keyboard = new Scanner(System.in);

6 System.out.println("Enter your last name");
7 System.out.println("followed by your first and middle names.");
8 System.out.println("If you have no middle name,");
9 System.out.println("enter \"None\".");
10 String inputLine = keyboard.nextLine();

11 String delimiters = ", "; //Comma and blank space
12 StringTokenizer nameFactory =
13 new StringTokenizer(inputLine, delimiters);

14 String lastName = nameFactory.nextToken();
15 String firstName = nameFactory.nextToken();
16 String middleName = nameFactory.nextToken();
17 if (middleName.equalsIgnoreCase("None"))
18 middleName = ""; //Empty string
19 System.out.println("Hello " + firstName
20 + " " + middleName + " " + lastName);
21 }
22 }

```

---

## Sample Dialogue

```

Enter your last name
followed by your first and middle names.
If you have no middle name,
enter None.
Savitch, Walter None
Hello Walter Savitch

```

*Note that the comma is  
not read because it is a  
delimiter.*

## Self-Test Exercises

27. What would be the last line in the dialogue in Display 4.18 if the user entered the following input line instead of the one shown in Display 4.18? (The comma is omitted.)

Savitch Walter None

28. What would be the last line in the dialogue in Display 4.18 if the user entered the following input line instead of the one shown in Display 4.18?

Tom, Dick, and Harry



VideoNote  
Example Using  
the String  
Tokenizer Class  
on a CSV File

## Chapter Summary

- Objects have both instance variables and methods. A class is a type whose values are objects. All objects in a class have the same methods and the same types of instance variables.
- There are two main kinds of methods: methods that return a value and `void` methods. (Some specialized methods, such as constructors, are neither `void` methods nor methods that return a value.)
- When defining a method, the `this` parameter is a name used for the calling object.
- Normally, your classes should have both an `equals` method and a `toString` method.
- If an instance variable or method is marked `private`, then it cannot be directly referenced anywhere except in the definition of a method of the same class.
- Outside of the class in which it is defined, a `private` instance variable can be accessed via accessor methods and changed via mutator methods.
- A variable declared in a method is said to be a *local variable*. The meaning of a local variable is confined to the method in which it is declared. The local variable goes away when a method invocation ends. The name of a local variable can be reused for something else outside of the method in which it is declared.
- A parameter is like a blank in a method definition that is filled in with an argument when the method is invoked. A parameter is actually a local variable that is initialized to the value of the corresponding argument. This is known as the *call-by-value* parameter-passing mechanism.
- If a variable is used as an argument to a method, then only the value of the variable, not the variable itself, is plugged in to the corresponding parameter.

- *Encapsulation* means that the data and the actions are combined into a single item (in our case, a class object) and that the *details of the implementation are hidden*. Making all instance variables private is part of the encapsulation process.
- A class can have two (or more) different definitions for the same method name, provided the two definitions have different numbers of parameters or some parameters of differing types. This is called *overloading* the method name.
- A constructor is a variety of method that is called when you create an object of the class using `new`. A constructor is intended to be used to perform initialization tasks such as initializing instance variables. A constructor must have the same name as the class to which it belongs.
- A constructor with no parameters is called a *no-argument constructor*. If your class definition includes no constructor definitions at all, then Java automatically provides a no-argument constructor. If your class definition contains any constructor definitions at all, then no additional constructors are provided by Java. Your class definitions should usually include a no-argument constructor.

## Answers to Self-Test Exercises

1. 

```
public void makeItNewYears()
{
 month = "January";
 day = 1;
}
```
2. 

```
public void yellIfNewYear()
{
 if ((month.equalsIgnoreCase("January")) && (day == 1))
 System.out.println("Hurrah!");
 else
 System.out.println("Not New Year's Day.");
}
```
3. 

```
public int getNextYear()
{
 int nextYear = year + 1;
 return nextYear;
}
```
4. 

```
public void happyGreeting()
{
 int count;
 for (count = 1; count <= day; count++)
 System.out.println("Happy Days!");
}
```

5. `public double fractionDone (int targetDay)`

```
{
 double doubleDay = day;
 return doubleDay/targetDay;
}
```

6. `public void advanceYear(int increase)`

```
{
 year = year + increase;
}
```

7. The instances of `newMonth` that have their values changed to 6 are indicated in color as follows:

```
public void setDate(int newMonth, int newDay, int newYear)
{
 month = monthString(newMonth);
 day = newDay;
 year = newYear;
 System.out.println("Date changed to "
 + newMonth + " " + newDay + ", " + newYear);
}
```

The point being emphasized here is that all instances of `newMonth` have their values changed to 6. Technically speaking, the parameter `newMonth` is a local variable. So, there is only one local variable named `newMonth` whose value is changed to 6, but the net effect, in this case, is the same as replacing all occurrences of `newMonth` with 6.

8. Yes, it is legal. The point being emphasized here is that the parameter `count` is a local variable and so can have its value changed, in this case by the decrement operator.

9. Each case has a `return` statement. A `return` statement always ends the method invocation, and hence ends the execution of the `switch` statement. So, a `break` statement would be redundant.

10. They are assumed to be instance variables of the calling object.

11. `public int getDay()`

```
{
 return this.day;
}
public int getYear()
{
 return this.year;
}
```



```

12. public int getMonth()
 {
 if (this.month.equals("January"))
 return 1;
 else if (this.month.equals("February"))
 return 2;
 else if (this.month.equals("March"))
 return 3;
 else if (this.month.equals("April"))
 return 4;
 else if (this.month.equals("May"))
 return 5;
 else if (this.month.equals("June"))
 return 6;
 else if (this.month.equals("July"))
 return 7;
 else if (this.month.equals("August"))
 return 8;
 else if (this.month.equals("September"))
 return 9;
 else if (this.month.equals("October"))
 return 10;
 else if (this.month.equals("November"))
 return 11;
 else if (this.month.equals("December"))
 return 12;
 else
 {
 System.out.println("Fatal Error");
 System.exit(0);
 return 0; //Needed to keep the compiler happy
 }
 }

```

13. The instance variable `month` contains a string, so we used `month` with `equals`. It would have been just as good to use

```
getMonth() == otherDate.getMonth()
```

We used `getMonth()` with the less-than sign because it is of type `int` and so works with the less-than sign. The instance variable `month` is of type `String` and does not work with the less-than sign.

14. Every method should be tested in a program in which every other method in the testing program has already been fully tested and debugged.
15. All instance variables should be marked `private`.

16. Normally, a method is private only if it is being used solely as a helping method in the definition of other methods.

17. `getMonth`, `getDay`, and `getYear`.

18. `setDate`, `setMonth`, `setDay`, and `setYear`.

19. `private boolean dateOK(int monthInt, int dayInt, int yearInt)`

```
{
 if ((yearInt < 1000) || (yearInt > 999))
 return false;

 switch (monthInt)
 {
 case 1:
 return (dayInt >= 1) && (dayInt <= 31);
 case 2:
 if (leapYear(yearInt))
 return (dayInt >= 1) && (dayInt <= 29);
 else
 return (dayInt >= 1) && (dayInt <= 28);
 case 3:
 return (dayInt >= 1) && (dayInt <= 31);
 case 4:
 return (dayInt >= 1) && (dayInt <= 30);
 case 5:
 return (dayInt >= 1) && (dayInt <= 31);
 case 6:
 return (dayInt >= 1) && (dayInt <= 30);
 case 7:
 return (dayInt >= 1) && (dayInt <= 31);
 case 8:
 return (dayInt >= 1) && (dayInt <= 31);
 case 9:
 return (dayInt >= 1) && (dayInt <= 30);
 case 10:
 return (dayInt >= 1) && (dayInt <= 31);
 case 11:
 return (dayInt >= 1) && (dayInt <= 30);
 case 12:
 return (dayInt >= 1) && (dayInt <= 31);
 default:
 return false;
 }
}
```

```

/**
Returns true if yearInt is a leap year.
*/
private boolean leapYear(int yearInt)
{
 return ((yearInt % 4 == 0) && (yearInt % 100 != 0))
 || (yearInt % 400 == 0);
}

```

20. `doSomething(int, char, int)`  
`setMonth(int)`  
`setMonth(String)`  
`amount(int, double)`  
`amount(int, double)`
21. Yes, it is legal because they have different signatures. This is a valid example of overloading.
22. No, it would be illegal because they have the same signature.
23. No, it would be illegal. You cannot overload on the basis of the type of the returned value.
24. If a class is named `CoolClass`, then all constructors must be named `CoolClass`.
25. `YourClass anObject = new YourClass(42, 'A');` //Legal  
`YourClass anotherObject = new YourClass(41.99, 'A');` //Not legal  
`YourClass yetAnotherObject = new YourClass();` //Legal  
`yetAnotherObject.doStuff();` //Legal  
`YourClass oneMoreObject;` //Legal  
`oneMoreObject.doStuff();` //Not legal  
`oneMoreObject>YourClass(99, 'B');` //Not legal
26. A no-argument constructor is a constructor with no parameters. If you define a class and define some constructors but do not define a no-argument constructor, then the class will have no no-argument constructor. *Default constructor* is another name for a no-argument constructor.
27. The last line would be the same. Because the blank space is a delimiter, a blank space is enough to separate the tokens.
28. Hello Dick and Tom  
The other token in the input line is just not used.

## Programming Projects

1. Write a program that outputs a certain coded language. The program should print 26 code words, each comprising one letter and one digit, for example, A0, B1, C2, D3, and so on.

Your program should not use 26 output statements!

Design your program with a class named `PrintCodeword` whose constructor takes an initial value for the starting value of the letter, for example D, and also an integer parameter that is the starting value of the number, for example 5. In this case, the series of codewords will be D5, E6, F7, and so on. There will always be a total of 26 codewords. If the value of letter reaches Z, it will then automatically move to A. Similarly, if the value of digit reaches 9, then it should also move automatically to 0 as shown below.

D5  
E6  
F7  
G8  
.  
.  
.  
.  
X5  
Y6  
Z7  
A8  
B9  
C0

Design your program with a public method called `print_Code` that outputs all the code words. Provide initial values of letters and digits from the main method class during object creation.

2. Define a class called `CalAge`. This class is used to calculate age of a person from her or his date of birth and the current date. Include a mutator method that allows the user to enter her or his date of birth and set the value for current date. Also include a method to return the age in years and months (for example, 25.5 years) as a double value. Include an additional method to check if the date of birth entered by the user is a valid one. For example, 30 February 2008 is an invalid date. Embed your class in a test program.
3. Define a class called `Vehicle` that will be used to check the amount of fuel left in a vehicle after traveling a certain distance. The class should have the instance variable `tankSize` to store the initial size of the tank and `efficiency` to store initial efficiency of the vehicle. Set to zero the variable `fuelInTank` that is used to store the initial amount of fuel in a tank. Include a mutator method that returns `initTankSize`, `initEfficiency` and `fuelInTank`. Include an accessor method `addPetrol` that calculates how much fuel can be filled depending on the fuel existing in the tank and the tank's capacity. Also, include a method `driveTo` that returns what distance can be traveled with the available fuel and provided efficiency. Use your class with a test program. You should decide which variables should be public, if any. Also, define if any other method(s) are needed.

4. Define a class called `Journal` that could be used to store an entry for a research paper that will be published. The class should have instance variables to store the author's name, title of the paper, and the date of submission using the `Date` class from this chapter. Add a constructor to the class that allows the user of the class to set all instance variables. Also add a method, `displayDetails`, that outputs all the instance variables, and another method called `getSubmissionDetails` that returns the title of the paper, with the first letter of each word capitalized. Test your class from the main method.
5. Define a class called `WordCount` whose objects count the number of words in a sentence. An object of this class maintains a variable count that is a nonnegative integer. Include methods to set the counter to 0, to increase the count by 1, and to decrease the count by 1 if any word encountered starts with 'A' or 'a'. Be sure that no method allows the value of the counter to become negative. Include an accessor method that returns the current count value and a method that outputs the count to the screen. There should be no input method or other mutator methods. The only method that can set the counter is the one that sets it to 0. Write a program (or programs) to test all the methods in your class definition.
6. Write a grading program for a class with the following grading policies:
  - a. There are three quizzes, each graded on the basis of 10 points.
  - b. There is one midterm exam, graded on the basis of 100 points.
  - c. There is one final exam, graded on the basis of 100 points.

The final exam counts for 40% of the grade. The midterm counts for 35% of the grade. The three quizzes together count for a total of 25% of the grade. (Do not forget to convert the quiz scores to percentages before they are averaged in.)

Any grade of 90 or more is an A, any grade of 80 or more (but less than 90) is a B, any grade of 70 or more (but less than 80) is a C, any grade of 60 or more (but less than 70) is a D, and any grade below 60 is an F. The program should read in the student's scores and output the student's record, which consists of three quiz scores and two exam scores, as well as the student's overall numeric score for the entire course and final letter grade.

Define and use a class for the student record. The class should have instance variables for the quizzes, midterm, final, overall numeric score for the course, and final letter grade. The overall numeric score is a number in the range 0 to 100, which represents the weighted average of the student's work. The class should have methods to compute the overall numeric grade and the final letter grade. These last methods should be `void` methods that set the appropriate instance variables. Your class should have a reasonable set of accessor and mutator methods, an `equals` method, and a `toString` method, whether or not your program uses them. You may add other methods if you wish.

7. Write a `Temperature` class that has two instance variables: a temperature value (a floating-point number) and a character for the scale, either `C` for Celsius or `F` for Fahrenheit. The class should have four constructor methods: one for each instance

variable (assume zero degrees if no value is specified and Celsius if no scale is specified), one with two parameters for the two instance variables, and a no-argument constructor (set to zero degrees Celsius). Include the following: (1) two accessor methods to return the temperature—one to return the degrees Celsius, the other to return the degrees Fahrenheit—use the following formulas to write the two methods, and round to the nearest tenth of a degree:

$$\text{DegreesC} = 5(\text{degreesF} - 32)/9$$

$$\text{DegreesF} = (9(\text{degreesC})/5) + 32;$$

(2) three mutator methods: one to set the value, one to set the scale (F or C), and one to set both; (3) three comparison methods: an `equals` method to test whether two temperatures are equal, one method to test whether one temperature is greater than another, and one method to test whether one temperature is less than another (note that a Celsius temperature can be equal to a Fahrenheit temperature as indicated by the above formulas); and (4) a suitable `toString` method. Then write a driver program (or programs) that tests all the methods. Be sure to use each of the constructors, to include at least one true and one false case for each of the comparison methods, and to test at least the following temperature equalities: 0.0 degrees C = 32.0 degrees F, -40.0 degrees C = -40.0 degrees F, and 100.0 degrees C = 212.0 degrees F.

8. Redefine the class `Date` in Display 4.13 so that the instance variable for the month is of type `int` instead of type `String`. None of the method headings should change in any way. In particular, no `String` type parameters should change to `int` type parameters. You must redefine the methods to make things work out. Any program that uses the `Date` class from Display 4.13 should be able to use your `Date` class without any changes in the program. In particular, the program in Display 4.14 should work the same whether the `Date` class is defined as in Display 4.13 or is defined as you do it for this project. Write a test program (or programs) that tests each method in your class definition.
9. Define a class whose objects are records on animal species. The class should have instance variables for the species name, population, and growth rate. The growth rate is a percentage that can be positive or negative and can exceed 100%. Include a suitable collection of constructors, mutator methods, and accessor methods. Include a `toString` method and an `equals` method. Include a boolean valued method named `endangered` that returns `true` when the growth rate is negative and returns `false` otherwise. Write a test program (or programs) that tests each method in your class definition.
10. Your vet's office is using the `Pet` class defined in Display 4.15 and would like to include a way to calculate the dosage amount for drugs that are commonly administered for dogs and cats. Make the following modifications to the class:
  - Add an instance variable that indicates if the type of the pet is a dog or a cat.
  - Modify the constructor and the `set` method to include the type of pet (i.e., dog or cat).



VideoNote  
Solution to  
Programming  
Project 4.9

- Add a method named `acepromazine()` that returns as a `double` the dosage in ml for the sedative acepromazine.
- Add a method named `carprofen()` that returns as a `double` the dosage in ml for the pain killer carprofen.

The dosage calculation is

$$\text{Dosage (ml)} = \frac{\text{Weight}}{2.2} \times \frac{\text{mg per kg}}{\text{mg per ml}}$$

*Weight* is in pounds.

- For acepromazine, use mg per ml = 10, and mg per kg = 0.03 for dogs and 0.002 for cats.
- For carprofen, use mg per ml = 12, and mg per kg = 0.5 for dogs and 0.25 for cats.

Modify the main method in Display 4.16 to include tests of the new methods.

11. Create a class named `Pizza` that stores information about a single pizza. It should contain the following:

- Private instance variables to store the size of the pizza (either small, medium, or large), the number of cheese toppings, the number of pepperoni toppings, and the number of ham toppings.
- Constructor(s) that set all of the instance variables.
- Public methods to get and set the instance variables.
- A public method named `calcCost()` that returns a `double` that is the cost of the pizza.

Pizza cost is determined by:

Small: \$10 + \$2 per topping

Medium: \$12 + \$2 per topping

Large: \$14 + \$2 per topping

- A public method named `getDescription()` that returns a `String` containing the pizza size, quantity of each topping, and the pizza cost as calculated by `calcCost()`.

Write test code to create several pizzas and output their descriptions. For example, a large pizza with one cheese, one pepperoni and two ham toppings should cost a total of \$22.

12. This programming project extends Programming Project 4.11. Create a `PizzaOrder` class that allows up to three pizzas to be saved in an order. Each pizza saved should be a `Pizza` object as described in Programming Project 4.11. In addition to appropriate instance variables and constructors, add the following methods:

- `public void setNumPizzas(int numPizzas)`—sets the number of pizzas in the order. `numPizzas` must be between 1 and 3.

- `public void setPizza1(Pizza pizza1)`—sets the first pizza in the order.
- `public void setPizza2(Pizza pizza2)`—sets the second pizza in the order.
- `public void setPizza3(Pizza pizza3)`—sets the third pizza in the order.
- `public double calcTotal()`—returns the total cost of the order.

Write a main method to test the class. The `setPizza2` and `setPizza3` methods will be used only if there are two or three pizzas in the order, respectively. Sample code illustrating the methods is shown below. Note that first three lines are incomplete. You must complete them as part of the Programming Project.

```
Pizza pizza1 = // Code to create a large pizza, 1 cheese, 1 ham
Pizza pizza2 = // Code to create a medium pizza, 2 cheese, 2 pepperoni
PizzaOrder order = // Code to create an order
order.setNumPizzas(2); // 2 pizzas in the order
order.setPizza1(pizza1); // Set first pizza
order.setPizza2(pizza2); // Set second pizza
double total = order.calcTotal(); // Should be 18+20 = 38
```

13. Your Community Supported Agriculture (CSA) farm delivers a box of fresh fruits and vegetables to your house once a week. For this Programming Project, define the class `BoxOfProduce` that contains exactly three bundles of fruits or vegetables. You can represent the fruits or vegetables as three instance variables of type `String`. Add appropriate constructor, accessor, and mutator methods. Also write a `toString()` method that returns as a `String` the complete contents of the box.

Next, write a main method that creates a `BoxOfProduce` with three items randomly selected from this list:

```
Broccoli
Tomato
Kiwi
Kale
Tomatillo
```

This list should be stored in a text file that is read in by your program. For now you can assume that the list contains exactly five types of fruits or vegetables.

Do not worry if your program randomly selects duplicate produce for the three items. Next, the main method should display the contents of the box and allow the user to substitute any one of the five possible fruits or vegetables for any of the fruits or vegetables selected for the box. After the user is done with substitutions, output the final contents of the box to be delivered. If you create additional methods to select the random items and to select valid substitutions, then your main method will be simpler to write.



14. A comma-separated values (CSV) file is a simple text format used to store a list of records. A comma is used as a delimiter to separate the fields for each record. This format is commonly used to transfer data between a spreadsheet or database. In this Programming Project, consider a store that sells five products abbreviated as A, B, C, D, and E. Customers can rate each product from 1–5, where 1 is poor and 5 is excellent. The ratings are stored in a CSV file where each row contains the customer's rating for each product. Here is a sample file with three customer ratings:

```
A,B,C,D,E
3,0,5,1,2
1,1,4,2,1
0,0,5,1,3
```

In this file format, the first line gives the products. The digit 0 indicates that a customer did not rate a product. In this case, the first customer rated A as 3, C as 5, D as 1, and E as 2. Product B was not rated. The third customer rated C as 5, D as 1, and E as 3. The third customer did not rate A or B.

Create a text file in this format with sample ratings. Then, write a program that reads in this text file and extracts each rating using the `StringTokenizer` class. Finally, the program should output the average rating for each product. Customers that did not rate a product should not be considered when computing the average rating for that product. Your program can assume there will always be exactly five products but it should work with an arbitrary number of customer ratings.

15. The goal of this Programming Project is to extend Programming Project 14 to find the customer from the CSV file who made ratings that are most similar to ratings input from the keyboard. First, do Programming Project 14. Then modify your solution so that it asks the user to input ratings for the first four products before the program reads from the file. The program should then predict whether or not the user will like the final product by outputting the rating made by the most similar customer. Use the formula  $|A_u - A_c| + |B_u - B_c| + |C_u - C_c| + |D_u - D_c|$  to compute the similarity, where  $A_u$  is the rating for product A made by the user at the keyboard and  $A_c$  is the rating for product A made by a customer from the file. A lower total indicates greater similarity. For example, if the user inputs 1 for product A, 1 for product B, 3 for product C, and 2 for product D, then with the values from Programming Project 14, the similarity to the customer in the first row is  $|1 - 3| + |1 - 0| + |3 - 5| + |2 - 1| = 6$ , while the similarity to the customer in the second row is  $|1 - 1| + |1 - 1| + |3 - 4| + |2 - 2| = 1$ . The customer in the second row has the greatest similarity, so the program would output that the prediction for product E is 1, which is the second customer's rating for product E.



# Defining Classes II **5**

## 5.1 STATIC METHODS AND STATIC VARIABLES 293

- Static Methods 293
- Static Variables 300
- The `Math` Class 305
- Wrapper Classes 309
- Automatic Boxing and Unboxing 310
- Static Methods in Wrapper Classes 312

## 5.2 REFERENCES AND CLASS PARAMETERS 316

- Variables and Memory 317
- References 318
- Class Parameters 323
- The Constant `null` 329
- The `new` Operator and Anonymous Objects 330
- Example: Another Approach to Keyboard Input ★ 331

## 5.3 USING AND MISUSING REFERENCES 335

- Example: A Person Class 336
- Copy Constructors 345
- Mutable and Immutable Classes 351

## 5.4 PACKAGES AND `javadoc` 354

- Packages and `import` Statements 355
- The Package `java.lang` 356
- Package Names and Directories 356
- The Default Package 359
- Specifying a Class Path When You Compile ★ 360
- Name Clashes ★ 361
- Introduction to `javadoc` ★ 362
- Commenting Classes for `javadoc` ★ 362
- Running `javadoc` ★ 364

# 5 Defining Classes II

*After a certain high level of technical skill is achieved, science and art tend to coalesce in esthetics, plasticity, and form. The greatest scientists are always artists as well.*

ALBERT EINSTEIN, Quoted in Alice Calaprice, *The Quotable Einstein Archive*, Princeton University Press, 1996, 33–257, 1923.

## Introduction

This chapter is a continuation of Chapter 4. It covers the rest of the core material on defining classes. We start by discussing *static methods* and *static variables*, which are methods and variables that belong to the class as a whole and not to particular objects. We then go on to discuss how class type variables name objects of their class and how class type parameters are handled in Java.

This chapter also discusses *packages*, which are Java's way of grouping classes into libraries. We end this chapter with a discussion of `javadoc`, a program that automatically extracts documentation from classes and packages.

## Prerequisites

This chapter uses material from Chapters 1 through 4.

Sections 5.3 and 5.4 are independent of each other and may be covered in any order. Section 5.3 covers some subtle points about references, and Section 5.4 covers packages and `javadoc`. The material on `javadoc` is not used in the rest of this book. The other material in Sections 5.3 and 5.4 is not heavily used in the next few chapters and can be digested as needed if the material seems difficult on first reading.

The material on packages in Section 5.4 assumes that you know about directories (which are called folders in some operating systems), that you know about path names for directories (folders), and that you know about `PATH` (environment) variables. These are not Java topics. They are part of your operating system, and the details depend on your particular operating system. If you can find out how to set the `PATH` variable on your operating system, then you will know enough about these topics to understand the material on packages in Section 5.4.

## 5.1 Static Methods and Static Variables

*All for one, one for all, that is our device.*

ALEXANDRE DUMAS, *The Three Musketeers*, 1844.

### Static Methods

#### static methods

Some methods do not require a calling object. Methods to perform simple numeric calculations are good examples. For example, a method to compute the maximum of two integers has no obvious candidate for a calling object. In Java, you can define a method so that it requires no calling object. Such methods are known as **static methods**. You define a static method in the same way as any other method, but you add the keyword `static` to the method definition heading, as in the following example:

```
public static int maximum(int n1, int n2)
{
 if (n1 > n2)
 return n1;
 else
 return n2;
}
```

Although a static method requires no calling object, it still belongs to some class, and its definition is given inside the class definition. When you invoke a static method, you normally use the class name in place of a calling object. So if the above definition of the method `maximum` were in a class named `SomeClass`, then the following is a sample invocation of `maximum`:

```
int budget = SomeClass.maximum(yourMoney, myMoney);
```

where `yourMoney` and `myMoney` are variables of type `int` that contain some values.

A sample of some static method definitions, as well as a program that uses the methods, are given in Display 5.1.

We have already been using one static method. The method `exit` in the class `System` is a static method. To end a program immediately, we have used the following invocation of the static method `exit`:

```
System.exit(0);
```

Note that with a static method, the class name serves the same purpose as a calling object. (It would be legal to create an object of the class `System` and use it to invoke the method `exit`, but that is confusing style; we usually use the class name when invoking a static method.)

## Display 5.1 Static Methods (part 1 of 2)

```
1 /**
2 Class with static methods for circles and spheres.
3 */
4 public class RoundStuff
5 {
6 public static final double PI = 3.14159;
7
8 /**
9 Return the area of a circle of the given radius.
10 */
11 public static double area(double radius)
12 {
13 return(PI*radius*radius);
14 }
15
16 /**
17 Return the volume of a sphere of the given radius.
18 */
19 public static double volume(double radius)
20 {
21 return((4.0/3.0)*PI*radius*radius*radius);
22 }
23 }
```

*This is the file  
RoundStuff.java.*

```
1 import java.util.Scanner;
2
3 public class RoundStuffDemo
4 {
5 public static void main(String[] args)
6 {
7 Scanner keyboard = new Scanner(System.in);
8 System.out.println("Enter radius:");
9 double radius = keyboard.nextDouble();
10
11 System.out.println("A circle of radius"
12 + radius + "inches");
13 System.out.println("has an area of " +
14 RoundStuff.area(radius) + " square inches.");
15 System.out.println("A sphere of radius"
16 + radius + "inches");
17 System.out.println("has an volume of " +
18 RoundStuff.volume(radius) + "cubic inches.");
19 }
20 }
```

*This is the file  
RoundStuffDemo.java.*

## Display 5.1 Static Methods (part 2 of 2)

## Sample Dialogue

```
Enter radius:
2
A circle of radius 2.0 inches
has an area of 12.56636 square inches.
A sphere of radius 2.0 inches
has a volume of 33.51029333333333 cubic inches.
```

Within the definition of a static method, you cannot do anything that refers to a calling object, such as accessing an instance variable. This makes perfectly good sense, because a static method can be invoked without using any calling object and so can be invoked when there are no instance variables. (Remember instance variables belong to the calling object.) The best way to think about this restriction is in terms of the `this` parameter. In a static method, you cannot use the `this` parameter, either explicitly or implicitly. For example, the name of an instance variable by itself has an implicit `this` and a dot before it. So you cannot use an instance variable in the definition of a static method.

### Static Methods

A **static method** is one that can be used without a calling object. With a static method, you normally use the class name in place of a calling object.

When you define a static method, you place the keyword `static` in the heading of the definition.

Since it does not need a calling object, a static method cannot refer to an instance variable of the class, nor can it invoke a nonstatic method of the class (unless it creates a new object of the class and uses that object as the calling object). Another way to phrase it is that, in the definition of a static method, you cannot use an instance variable or method that has an implicit or explicit `this` for a calling object.



### PITFALL: Invoking a Nonstatic Method Within a Static Method

If `myMethod()` is a nonstatic (that is, ordinary) method in a class, then within the definition of any method of this class, an invocation of the form

```
myMethod();
```

means

```
this.myMethod();
```

(continued)

**PITFALL: (continued)**

and so it is illegal within the definition of a static method. (A static method has no `this`.)

However, it is legal to invoke a static method within the definition of another static method.

There is one way that you can invoke a nonstatic method within a static method: if you create an object of the class and use that object (rather than `this`) as the calling object. For example, suppose `myMethod()` is a nonstatic method in the class `MyClass`. Then, as we already discussed, the following is illegal within the definition of a static method in the class `MyClass`:

```
myMethod();
```

However, the following is perfectly legal in a static method or any method definition:

```
MyClass anObject = new MyClass();
anObject.myMethod();
```

The method `main` is a static method, and you will often see code similar to this in the `main` method of a class. This point is discussed in the Tip “You Can Put a `main` in Any Class.” ■

**TIP: You Can Put a `main` in Any Class**

So far, whenever we have used a class in the `main` part of a program, that `main` method was by itself in a different class definition within another file. However, sometimes it makes sense to have a `main` method within a regular class definition. The class can then be used for two purposes: It can be used to create objects in other classes, or it can be run as a program. For example, you can combine the class definition `RoundStuff` and the program `RoundStuffDemo` (both in Display 5.1) by placing the `main` method inside the definition of the class `RoundStuff`, to obtain the class definition shown in Display 5.2.

Another example of a class with a `main` added is given in Display 5.3. Note that in addition to the static method `main`, the class has another static method named `toCelsius`. The class has both static and nonstatic methods. Note that the static method `toCelsius` can be invoked without the class name or a calling object because it is in another static method (namely `main`) in the same class. However, the nonstatic method `toString` requires an explicit calling object (`temperatureObject`). Java requires that a program's `main` method be static. Thus, within a `main` method, you cannot invoke a nonstatic method of the same class (such as `toString`) unless you create an object of the class and use it as a calling object for the nonstatic method.

You do not want to place just any `main` method in a class definition that is to be used as a regular class to create objects. One handy trick is to place a small diagnostic program in a `main` method that is inside of your class definition. ■

Display 5.2 Class Definition with a main Added

---

```
1 import java.util.Scanner;

2 /**
3 Class with static methods for circles and spheres.
4 */
5 public class RoundStuff2
6 {
7 public static final double PI = 3.14159;

8 /**
9 Return the area of a circle of the given radius.
10 */
11 public static double area(double radius)
12 {
13 return (PI*radius*radius);
14 }

15 /**
16 Return the volume of a sphere of the given radius.
17 */
18 public static double volume(double radius)
19 {
20 return ((4.0/3.0)*PI*radius*radius*radius);
21 }

22 public static void main(String[] args)
23 {
24 Scanner keyboard = new Scanner(System.in);
25 System.out.println("Enter radius:");
26 double radius = keyboard.nextDouble();
27
28 System.out.println("A circle of radius "
29 + radius + "inches");
30 System.out.println("has an area of " +
31 RoundStuff2.area(radius) + " square inches.");
32 System.out.println("A sphere of radius "
33 + radius + "inches");
34 System.out.println("has an volume of " +
35 RoundStuff2.volume(radius) + " cubic inches.");
36 }
37 }
38 }
```

*The dialogue is the same as in Display 5.1.*

---



## Display 5.3 Another Class with a main Added (part 1 of 2)

```
1 import java.util.Scanner;

2 /**
3 Class for a temperature (expressed in degrees Celsius).
4 */
5 public class Temperature
6 {
7 private double degrees; //Celsius

8 public Temperature()
9 {
10 degrees = 0;
11 }

12 public Temperature (double initialDegrees)
13 {
14 degrees = initialDegrees;
15 }

16 public void setDegrees (double newDegrees)
17 {
18 degrees = newDegrees;
19 }

20 public double getDegrees()
21 {
22 return degrees;
23 }

24 public String toString()
25 {
26 return (degrees + "C");
27 }
28
29 public boolean equals(Temperature otherTemperature)
30 {
31 return (degrees == otherTemperature.degrees);
32 }
33 /**
34 Returns number of Celsius degrees equal to
35 degreesF Fahrenheit degrees.
36 */
```

*Note that this class has a main method and both static and nonstatic methods.*

Display 5.3 Another Class with a main Added (part 2 of 2)

```

37 public static double toCelsius(double degreesF)
38 {
39
40 return 5*(degreesF - 32)/9;
41 }
42 public static void main(String[] args)
43 {
44 double degreesF, degreesC;
45
46 Scanner keyboard = new Scanner(System.in);
47 System.out.println("Enter degrees Fahrenheit:");
48 degreesF = keyboard.nextDouble();
49
50 degreesC = toCelsius(degreesF);
51
52 Temperature temperatureObject = new Temperature(degreesC);
53 System.out.println("Equivalent Celsius temperature is"
54 + temperatureObject.toString());
55 }
56 }

```

*Because this is in the definition of the class Temperature, this is equivalent to Temperature.toCelsius(degreesF).*

*Because main is a static method, toString must have a specified calling object such as temperatureObject.*

## Sample Dialogue

```

Enter degrees Fahrenheit:
212
Equivalent Celsius temperature is 100.0 C

```

## Self-Test Exercises

1. Is the following legal? The class RoundStuff is defined in Display 5.1.

```

RoundStuff roundObject = new RoundStuff();
System.out.println("A circle of radius 5.5 has area"
 + roundObject.area(5.5));

```

2. In Display 5.1, we did not define any constructors for the class RoundStuff. Is this poor programming style?
3. Can a class contain both static and nonstatic (that is, regular) methods?
4. Can you invoke a nonstatic method within a static method?
5. Can you invoke a static method within a nonstatic method?
6. Can you reference an instance variable within a static method? Why or why not?

## Static Variables

### static variable

A class can have static variables as well as static methods. A **static variable** is a variable that belongs to the class as a whole and not just to one object. Each object has its own copies of the instance variables. However, with a static variable, there is only one copy of the variable, and all the objects can use this one variable. Thus, a static variable can be used by objects to communicate between the objects. One object can change the static variable, and another object can read that change. To make a variable static, you declare it like an instance variable but add the modifier `static` as follows:

```
private static int turn;
```

Or if you wish to initialize the static variable, which is typical, you might declare it as follows instead:

```
private static int turn = 0;
```

### default initialization

If you do not initialize a static variable, it is automatically initialized to a default value: Static variables of type `boolean` are automatically initialized to `false`. Static variables of other primitive types are automatically initialized to the zero of their type. Static variables of a class type are automatically initialized to `null`, which is a kind of placeholder for an object that we will discuss later in this chapter. However, we prefer to explicitly initialize static variables, either as just shown or in a constructor.

Display 5.4 shows an example of a class with a static variable along with a demonstration program. Notice that the two objects, `lover1` and `lover2`, access the same static variable `turn`.

As we already noted, you cannot directly access an instance variable within the definition of a static method. However, it is perfectly legal to access a static variable within a static method, because a static variable belongs to the class as a whole. This is illustrated by the method `getTurn` in Display 5.4. When we write `turn` in the definition of the static method `getTurn`, it does not mean `this.turn`; it means `TurnTaker.turn`. If the static variable `turn` were marked `public` instead of `private`, it would even be legal to use `TurnTaker.turn` outside of the definition of the class `TurnTaker`.

Defined constants that we have already been using, such as the following, are a special kind of static variable:

```
public static final double PI = 3.14159;
```

The modifier `final` in this example means that the static variable `PI` cannot be changed. Such defined constants are normally `public` and can be used outside the class. This defined constant appears in the class `RoundStuff` in Display 5.1. To use this constant outside of the class `RoundStuff`, you write the constant in the form `RoundStuff.PI`.

Good programming style dictates that static variables should normally be marked `private` unless they are marked `final`, that is, unless they are defined constants. The reason is the same as the reason for making instance variables `private`.

Display 5.4 A Static Variable (part 1 of 2)

```
1 public class TurnTaker
2 {
3 private static int turn = 0;

4 private int myTurn;
5 private String name;

6 public TurnTaker(String theName, int theTurn)
7 {
8 name = theName;
9 if (theTurn >= 0)
10 myTurn = theTurn;
11 else
12 {
13 System.out.println("Fatal Error.");
14 System.exit(0);
15 }
16 }

17 public TurnTaker()
18 {
19 name = "No name yet";
20 myTurn = 0; //Indicating no turn.
21 }

22 public String getName()
23 {
24 return name;
25 }

26 public static int getTurn()
27 {
28 turn++;
29 return turn;
30 }

31 public boolean isMyTurn()
32 {
33 return (turn == myTurn);
34 }
35 }
```

*This is the file  
TurnTaker.java.*

*You cannot access an instance  
variable in a static method, but you  
can access a static variable in a  
static method.*

(continued)

Display 5.4 A Static Variable (part 2 of 2)

---

```

36 public class StaticDemo
37 {
38 public static void main(String[] args)
39 {
40 TurnTaker lover1 = new TurnTaker("Romeo", 1);
41 TurnTaker lover2 = new TurnTaker("Juliet", 3);
42 for (int i = 1; i < 5; i++)
43 {
44 System.out.println("Turn = " + TurnTaker.getTurn());
45 if (lover1.isMyTurn())
46 System.out.println("Love from" + lover1.getName());
47 if (lover2.isMyTurn())
48 System.out.println("Love from" + lover2.getName());
49 }
50 }
51 }

```

*This is the file  
StaticDemo.java.*

## Sample Dialogue

```

Turn = 1
Love from Romeo
Turn = 2
Turn = 3
Love from Juliet
Turn = 4

```

---

Another example of a static variable is given in Display 5.5. The static variable `numberOfInvocations` is used to keep track of how many invocations have been made by all objects of the class `StaticDemo`. The program counts all invocations of the methods defined in Display 5.4, except for the method `main`.

Display 5.5 A Static Variable (part 1 of 2)

---

```

1 public class InvocationCounter
2 {
3 private static int numberOfInvocations = 0;
4
5 public void demoMethod()
6 {
7 numberOfInvocations++;
8 }

```

*object1 and object2 use  
the same static variable  
numberOfInvocations.*

*//In a real example, more code would go here.*

## Display 5.5 A Static Variable (part 2 of 2)

```

9 public void outPutCount ()
10 {
11 numberOfInvocations++;
12 System.out.println("Number of invocations so far = "
13 + numberOfInvocations);
14 }

15 public static int numberSoFar ()
16 {
17 numberOfInvocations++;
18 return numberOfInvocations;
19 }

20 public static void main(String[] args)
21 {
22 int i;
23 InvocationCounter object1 = new InvocationCounter();
24 for (i= 1; i <= 5 ; i++)
25 object1.demoMethod();
26 object1.outPutCount();
27
28 InvocationCounter object2 = new InvocationCounter();
29 for (i= 1; i <= 5 ; i++)
30 {
31 object2.demoMethod();
32 object2.outPutCount();
33 }

34 System.out.println("Totalnumber of invocations = "
35 + numberSoFar());
36 }
37 }

```

*Outputs 6 for five invocations of demoMethod and one invocation of outPutCount.*

## Sample Dialogue

```

Number of invocations so far = 6
Number of invocations so far = 8
Number of invocations so far = 10
Number of invocations so far = 12
Number of invocations so far = 14
Number of invocations so far = 16
Total number of invocations = 17

```

## Static Variables

A **static variable** belongs to the class as a whole. All objects of the class can read and change the static variable. Static variables should normally be private, unless they happen to be defined constants.

### SYNTAX

```
private static Type Variable_Name;
private static Type Variable_Name = Initial_Value;
public static final Type Variable_Name = Constant_Value;
```

### EXAMPLES

```
private static String lastUser;
private static int turn = 0;
public static final double PI = 3.14159;
```

## Self-Test Exercises

7. What is the difference between a static variable and an instance variable?
8. Can you use an *instance variable* (without an object name and dot) in the definition of a *static method* of the same class? Can you use an *instance variable* (without an object name and dot) in the definition of a *nonstatic (ordinary) method* of the same class?
9. Can you use a *static variable* in the definition of a static method of the same class? Can you use a *static variable* in the definition of a *nonstatic (ordinary) method* of the same class?
10. Can you use the `this` parameter in the definition of a static method?
11. When we defined the class `Date` in Display 4.11 in Chapter 4, we had not yet discussed static methods, so we did not mark any of the methods `static`. However, some of the methods could have been marked `static` (and should have been marked `static`, if only we had known what that meant). Which of the methods can be marked `static`? (If you omit the modifier `static` when it is appropriate, then the method cannot be invoked with the class name; it must be invoked with a calling object.)
12. Following the style guidelines given in this book, when should a static variable be marked `private`?
13. What do static methods and static variables have in common? After all, they are both called *static*, so it sounds like they have something in common.

## The Math Class

### Math methods

The class `Math` provides a number of standard mathematical methods. The class `Math` is provided automatically and requires no `import` statement. Some of the methods in the class `Math` are described in Display 5.6. A more complete list of methods is given in Appendix 5. All of these methods are static, which means that you normally use the class name `Math` in place of a calling object.

Display 5.6 Some Methods in the Class `Math` (part 1 of 2)

The `Math` class is in the `java.lang` package, so it requires no `import` statement.

```
public static double pow(double base, double exponent)
```

Returns `base` to the power `exponent`.

#### EXAMPLE

`Math.pow(2.0, 3.0)` returns 8.0.

```
public static double abs(double argument)
public static float abs(float argument)
public static long abs(long argument)
public static int abs(int argument)
```

Returns the absolute value of the `argument`. (The method name `abs` is overloaded to produce four similar methods.)

#### EXAMPLE

`Math.abs(-6)` and `Math.abs(6)` both return 6. `Math.abs(-5.5)` and `Math.abs(5.5)` both return 5.5.

```
public static double min(double n1, double n2)
public static float min(float n1, float n2)
public static long min(long n1, long n2)
public static int min(int n1, int n2)
```

Returns the minimum of the arguments `n1` and `n2`. (The method name `min` is overloaded to produce four similar methods.)

#### EXAMPLE

`Math.min(3, 2)` returns 2.

```
public static double max(double n1, double n2)
public static float max(float n1, float n2)
public static long max(long n1, long n2)
public static int max(int n1, int n2)
```

Returns the maximum of the arguments `n1` and `n2`. (The method name `max` is overloaded to produce four similar methods.)

#### EXAMPLE

`Math.max(3, 2)` returns 3.

(continued)



Display 5.6 Some Methods in the Class `Math` (part 2 of 2)

```
public static long round(double argument)
public static int round(float argument)
```

Rounds its argument.

**EXAMPLE**

`Math.round(3.2)` returns 3; `Math.round(3.6)` returns 4.

```
public static double ceil(double argument)
```

Returns the smallest whole number greater than or equal to the argument.

**EXAMPLE**

`Math.ceil(3.2)` and `Math.ceil(3.9)` both return 4.0.

```
public static double floor(double argument)
```

Returns the largest whole number less than or equal to the argument.

**EXAMPLE**

`Math.floor(3.2)` and `Math.floor(3.9)` both return 3.0.

```
public static double sqrt(double argument)
```

Returns the square root of its argument.

**EXAMPLE**

`Math.sqrt(4)` returns 2.0.

```
public static double random()
```

Returns a random number greater than or equal to 0.0 and less than 1.0.

**EXAMPLE**

`Math.random()` returns 0.5505562535943004 (sample number; returns a pseudorandom number that is less than 1 and greater than or equal to 0).

The class `Math` has three similar methods named `round`, `floor`, and `ceil`. Some of these return a value of type `double`, but they all return a value that is intuitively a whole number that is close to the value of their arguments. The method `round` rounds a number to the nearest whole number, and (if the argument is a `double`) it returns that whole number as a value of type `long`. If you want that whole number as a value of type `int`, you must use a type cast as in the following:

```
double exact = 7.56;
int roundedValue = (int) Math.round(exact);
```

You cannot assign a `long` value to a variable of type `int`, even if it is a value such as 8, which could just as well have been an `int`. A value such as 8 can be of type either `int` or `long` (or even of type `short` or `byte`) depending on how it was created.

### floor and ceil

The methods `floor` and `ceil` are similar to, but not identical to, `round`. Neither one rounds, although they both yield a whole number that is close to their argument. They both return a whole number as a value of type `double` (not of type `int` or `long`). The method `floor` returns the nearest whole number that is less than or equal to its argument. So `Math.floor(5.9)` returns `5.0`, not `6.0`. `Math.floor(5.2)` also returns `5.0`.

The method `ceil` returns the nearest whole number that is greater than or equal to its argument. The word `ceil` is short for “ceiling.” `Math.ceil(5.1)` returns `6.0`, not `5.0`. `Math.ceil(5.9)` also returns `6.0`.

If you want to store the value returned by either `floor` or `ceil` in a variable of type `int`, you must use a type cast as in the following example:

```
double exact = 7.56;
int lowEstimate = (int)Math.floor(exact);
int highEstimate = (int)Math.ceil(exact);
```

`Math.floor(exact)` returns the `double` value `7.0`, and the variable `lowEstimate` receives the `int` value `7`. `Math.ceil(exact)` returns the `double` value `8.0`, and the variable `highEstimate` receives the `int` value `8`.

Because values of type `double` are effectively approximate values, a safer way to compute the floor or ceiling as an `int` value is the following:

```
double exact = 7.56;
int lowEstimate = (int)Math.round(Math.floor(exact));
int highEstimate = (int)Math.round(Math.ceil(exact));
```

This way, if `Math.floor(exact)` returns slightly less than `7.0`, the final result will still be `7` and not `6`, and if `Math.ceil(exact)` returns slightly less than `8.0`, the final result will still be `8` and not `7`.

### Math constants

The class `Math` also has the two predefined constants `E` and `PI`. The constant `PI` (often written  $\pi$  in mathematical formulas) is used in calculations involving circles, spheres, and other geometric figures based on circles. `PI` is approximately `3.14159`. The constant `E` is the base of the natural logarithm system (often written  $e$  in mathematical formulas) and is approximately `2.72`. (We do not use the predefined constant `E` in this text.) The constants `PI` and `E` are defined constants, as described in Chapter 1. For example, the following computes the area of a circle, given its radius:

```
area = Math.PI * radius * radius;
```

Notice that because the constants `PI` and `E` are defined in the class `Math`, they must have the class name `Math` and a dot before them. For example, in Display 5.7, we have redone the program in Display 5.2, but this time we used the constant `Math.PI` instead of including our own definition of `PI`.

Display 5.7 Using `Math.PI`

```
1 import java.util.Scanner;

2 /**
3 Class with static methods for circles and spheres.
4 */
5 public class RoundStuff3
6 {

7 /**
8 Return the area of a circle of the given radius.
9 */
10 public static double area(double radius)
11 {
12 return (Math.PI*radius*radius);
13 }

14 /**
15 Return the volume of a sphere of the given radius.
16 */
17 public static double volume(double radius)
18 {
19 return ((4.0/3.0)*Math.PI*radius*radius*radius);
20 }

21

22 public static void main(String[] args)
23 {
24 Scanner keyboard = new Scanner(System.in);
25 System.out.println("Enter radius:");
26 double radius = keyboard.nextDouble();
27
28 System.out.println("A circle of radius"
29 + radius + "inches");
30 System.out.println("has an area of" +
31 RoundStuff3.area(radius) + "square inches.");
32 System.out.println("A sphere of radius"
33 + radius + "inches");
34 System.out.println("has a volume of " +
35 RoundStuff3.volume(radius) + "cubic inches.");
36 }
37 }

38
39
```

*The dialogue is the same as in  
Display 5.1.*

Finally, the class `Math` also includes a method to generate random numbers. The method `random` returns a pseudorandom number that is greater than or equal to 0.0 and less than 1.0. A pseudorandom number is a number that appears random, but is really generated by a deterministic function. See Chapter 3 for additional discussion about random number generation.

## Self-Test Exercises

14. What values are returned by each of the following?

```
Math.round(3.2), Math.round(3.6),
Math.floor(3.2), Math.floor(3.6),
Math.ceil(3.2), and Math.ceil(3.6).
```

15. Suppose `answer` is a variable of type `double`. Write an assignment statement to assign `Math.round(answer)` to the `int` variable `roundedAnswer`.
16. Suppose `n` is of type `int` and `m` is of type `long`. What is the type of the value returned by `Math.min(n, m)`? Is it `int` or `long`?

## Wrapper Classes

Java treats the primitive types, such as `int` and `double`, differently from the class types, such as the class `String` and the programmer-defined classes. For example, later in this chapter you will see that an argument to a method is treated differently depending on whether the argument is of a primitive or class type. At times, you may find yourself in a situation where you want to use a value of a primitive type but you want or need the value to be an object of a class type. **Wrapper classes** provide a class type corresponding to each of the primitive types so that you can have an object of a class type that behaves somewhat like its corresponding value of a primitive type.

To convert a value of a primitive type to an “equivalent” value of a class type, you create an object of the corresponding wrapper class using the primitive type value as an argument to the wrapper class constructor. The wrapper class for the primitive type `int` is the predefined class `Integer`. If you want to convert an `int` value, such as 42, to an object of type `Integer`, you can do so as follows:

```
Integer integerObject = new Integer(42);
```

The variable `integerObject` now names an object of the class `Integer` that corresponds to the `int` value 42. (The object `integerObject` does, in fact, have the `int` value 42 stored in an instance variable of the object `integerObject`.) This process of going from a value of a primitive type to an object of its wrapper class is sometimes called **boxing**, and as you will see in the next subsection, you can let Java automatically do all the work of boxing for you.

To go in the reverse direction, from an object of type `Integer` to the corresponding `int` value, you can do the following:

```
int i = integerObject.intValue();
```

wrapper class

integer class

boxing

**unboxing**

The method `intValue()` recovers the corresponding `int` value from an object of type `Integer`. This process of going from an object of a wrapper class to the corresponding value of a primitive type is sometimes called **unboxing**, and as you will see in the next subsection, you can let Java automatically do all the work of unboxing for you.

**other  
wrapper  
classes**

The wrapper classes for the primitive types `byte`, `short`, `long`, `float`, `double`, and `char` are `Byte`, `Short`, `Long`, `Float`, `Double`, and `Character`, respectively. The methods for converting from the wrapper class object to the corresponding primitive type are `intValue` for the class `Integer`, as we have already seen, `byteValue` for the class `Byte`, `shortValue` for the class `Short`, `longValue` for the class `Long`, `floatValue` for the class `Float`, `doubleValue` for the class `Double`, and `charValue` for the class `Character`.

**Wrapper Classes**

Every primitive type has a corresponding wrapper class. A **wrapper class** allows you to have a class object that corresponds to a value of a primitive type. Wrapper classes also contain a number of useful predefined constants and static methods.

**Automatic Boxing and Unboxing**

Converting from a value of a primitive type, such as `int`, to a corresponding object of its associated wrapper class, such as `Integer`, is called **boxing**. You can think of the object as a “box” that contains the value of the primitive type. In fact, the wrapper object does contain the value of the primitive type as the value of a private instance variable. The following are examples of boxing:

```
Integer numberOfSamuri = new Integer(47);
Double price = new Double(499.99);
Character grade = new Character('A');
```

**automatic  
boxing**

Starting with version 5.0, Java will automatically do this boxing, so the previous three assignments can be written in the following equivalent, but simpler, forms:

```
Integer numberOfSamuri = 47;
Double price = 499.99;
Character grade = 'A';
```

This is an automatic type cast. What is actually done by Java is what we showed in the forms using the `new`, but it is handy to be able to write the assignments in the simpler form.

**automatic  
unboxing**

The reverse conversion from an object of a wrapper class to a value of its associated primitive type is called **unboxing**. Unboxing is also done automatically in Java (starting in version 5.0). The following are examples of automatic unboxing:

```
Integer numberOfSamuri = new Integer(47);
int n = numberOfSamuri;
Double price = new Double(499.99);
double d = price;
Character grade = new Character('A');
char c = grade;
```

Java automatically applies the appropriate accessor method (`intValue`, `doubleValue`, or `charValue` in these cases) to obtain the value of the primitive type that is assigned to the variable. So the previous examples of automatic unboxing are equivalent to the following code, which is what you had to write in older versions of Java that did not do automatic unboxing:

```
Integer numberOfSamuri = new Integer(47);
int n = numberOfSamuri.intValue();
Double price = new Double(499.99);
double d = price.doubleValue();
Character grade = new Character('A');
char c = grade.charValue();
```

Our previous examples involved either only automatic boxing or only automatic unboxing. That was done to simplify the discussion by allowing you to see each of automatic boxing and automatic unboxing in isolation. However, code can often involve a combination of automatic boxing and unboxing. For example, consider the following code, which uses both automatic boxing and automatic unboxing:

```
Double price = 19.90;
price = price + 5.12;
```

This code is equivalent to the following, which is what you had to write in older versions of Java that did not do automatic boxing and unboxing:

```
Double price = new Double(19.90);
price = new Double(price.doubleValue() + 5.12);
```

Automatic boxing and unboxing applies to parameters as well as to the simple assignment statements we just discussed. You can plug in a value of a primitive type, such as a value of type `int`, for a parameter of the associated wrapper class, such as `Integer`. Similarly, you can plug in a wrapper class argument, such as an argument of type `Integer`, for a parameter of the associated primitive type, such as `int`.

### Self-Test Exercises

17. Which of the following are legal?

```
Integer n = new Integer(42);
int m = 42;
n = m;
m = n;
```

If any are illegal, explain how to write a valid Java statement that does what the illegal statement is trying to do.

18. In the following, is the value of the variable `price` after the assignment statement an object of the class `Double` or a value of the primitive type `double`?

```
Double price = 1.99;
```

19. In the following, is the value of the variable `count` after the assignment statement an object of the class `Integer` or a value of the primitive type `int`?

```
int count = new Integer(12);
```

### Static Methods in Wrapper Classes

The material on wrapper classes that we have seen thus far explains why they are called *wrapper classes*. However, possibly more importantly, the wrapper classes contain a number of useful constants and static methods. So, wrapper classes have two distinct personalities: One is their ability to produce class objects corresponding to values of primitive types, and the other is as a repository of useful constants and methods. It was not necessary to combine these two personalities into one kind of class. Java could have had two sets of classes, one for each personality, but the designers of the Java libraries chose to have only one set of classes for both personalities.

You can use the associated wrapper class to find the value of the **largest and smallest values** of any of the primitive number types. For example, the largest and smallest values of type `int` are

```
Integer.MAX_VALUE and Integer.MIN_VALUE
```

The largest and smallest values of type `double` are

```
Double.MAX_VALUE and Double.MIN_VALUE
```

Wrapper classes have static methods that can be used to convert back and forth between string representations of numbers and the corresponding number of type `int`, `double`, `long`, or `float`. For example, the static method `parseDouble` of the wrapper class `Double` converts a string to a value of type `double`. So, the code

```
Double.parseDouble("199.98")
```

largest and  
smallest  
values

`parseDouble`

returns the double value 199.98. If there is any possibility that the string named by `theString` has extra leading or trailing blanks, you should instead use

```
Double.parseDouble(theString.trim())
```

The method `trim` is a method in the class `String` that trims off leading and trailing whitespace, such as blanks.

If the string is not a correctly formed numeral, then the invocation of `Double.parseDouble` will cause your program to end. The use of `trim` helps somewhat in avoiding this problem.

#### `parseInt`

Similarly, the static methods `Integer.parseInt`, `Long.parseLong`, and `Float.parseFloat` convert from string representations to numbers of the corresponding primitive types `int`, `long`, and `float`, respectively.

Each of the numeric wrapper classes also has a static method called `toString` that converts in the other direction, from a numeric value to a string representation of the numeric value. For example,

```
Double.toString(123.99)
```

returns the string value `"123.99"`.

#### `Character`

`Character`, the wrapper class for the primitive type `char`, contains a number of static methods that are useful for string processing. Some of these methods are shown in Display 5.8. A simple example of using the static method `toUpperCase` of the class `Character` is given in Display 5.9. As is typical, this program combines the string-processing methods of the class `String` with the character-processing methods in the class `Character`.

#### `Boolean`

There is also a wrapper class `Boolean` corresponding to the primitive type `boolean`. It has names for two constants of type `Boolean`: `Boolean.TRUE` and `Boolean.FALSE`, which are the `Boolean` objects corresponding to the values `true` and `false` of the primitive type `boolean`.

Display 5.8 Some Methods in the Class `Character` (part 1 of 2)

The class `Character` is in the `java.lang` package, so it requires no `import` statement.

```
public static char toUpperCase(char argument)
```

Returns the uppercase version of its argument. If the argument is not a letter, it is returned unchanged.

#### **EXAMPLE**

`Character.toUpperCase('a')` and `Character.toUpperCase('A')` both return `'A'`.

```
public static char toLowerCase(char argument)
```

Returns the lowercase version of its argument. If the argument is not a letter, it is returned unchanged.

#### **EXAMPLE**

`Character.toLowerCase('a')` and `Character.toLowerCase('A')` both return `'a'`.

(continued)



Display 5.8 Some Methods in the Class Character (part 2 of 2)

```
public static boolean isUpperCase(char argument)
```

Returns true if its argument is an uppercase letter; otherwise returns false.

**EXAMPLE**

`Character.isUpperCase('a')` returns true. `Character.isUpperCase('a')` and `Character.isUpperCase('%')` both return false.

```
public static boolean isLowerCase(char argument)
```

Returns true if its argument is a lowercase letter; otherwise returns false.

**EXAMPLE**

`Character.isLowerCase('a')` returns true. `Character.isLowerCase('A')` and `Character.isLowerCase('%')` both return false.

```
public static boolean isWhitespace(char argument)
```

Returns true if its argument is a whitespace character; otherwise returns false. Whitespace characters are those that print as white space, such as the space character (blank character), the tab character (`'\t'`), and the line break character (`'\n'`).

**EXAMPLE**

`Character.isWhitespace(' ')` returns true.  
`Character.isWhitespace('A')` returns false.

```
public static boolean isLetter(char argument)
```

Returns true if its argument is a letter; otherwise returns false.

**EXAMPLE**

`Character.isLetter('A')` returns true. `Character.isLetter('%')` and `Character.isLetter('5')` both return false.

```
public static boolean isDigit(char argument)
```

Returns true if its argument is a digit; otherwise returns false.

**EXAMPLE**

`Character.isDigit('5')` returns true. `Character.isDigit('A')` and `Character.isDigit('%')` both return false.

```
public static boolean isLetterOrDigit(char argument)
```

Returns true if its argument is a letter or a digit; otherwise returns false.

**EXAMPLE**

`Character.isLetterOrDigit('A')` and `Character.isLetterOrDigit('5')` both return true. `Character.isLetterOrDigit('&')` returns false.



## PITFALL: A Wrapper Class Does Not Have a No-Argument Constructor

Normally, it is good programming practice to define a no-argument constructor for any class you define. However, on rare occasions, a no-argument constructor simply does not make sense. The wrapper classes discussed in the previous subsection do not have a no-argument constructor, which is reasonable if you think about it. To use the static methods in a wrapper class, you need no calling object and hence need no constructor at all. The other function of a wrapper class is to provide a class object corresponding to a value of a primitive type. For example,

```
new Integer(42)
```

creates an object of the class `Integer` that corresponds to the `int` value 42. There is no no-argument constructor for the class `Integer`, because it makes no sense to have an object of the class `Integer` unless it corresponds to an `int` value, and if it does correspond to an `int` value, that `int` value is naturally an argument to the constructor. ■

### Display 5.9 String Processing with a Method from the Class `Character` (part 1 of 2)

```

1 import java.util.Scanner;

2 /**
3 Illustrate the use of a static method from the class Character.
4 */
5
6 public class StringProcessor
7 {
8 public static void main (String[] args)
9 {
10 System.out.println("Enter a one line sentence:");
11 Scanner keyboard = new Scanner(System.in);
12 String sentence = keyboard.nextLine();
13
14 sentence = sentence.toLowerCase();
15 char firstCharacter = sentence.charAt(0);
16 sentence = Character.toUpperCase(firstCharacter)
17 + sentence.substring(1);
18
19 System.out.println("The revised sentence is:");
20 System.out.println(sentence);
21 }
22 }
```

(continued)

Display 5.9 String Processing with a Method from the Class `Character` (part 2 of 2)

## Sample Dialogue

```
Enter a one line sentence:
is you is OR is you ain't my BABY?
The revised sentence is:
Is you is or is you ain't my baby?
```

## Self-Test Exercises

20. What is the output produced by the following code?

```
Character characterObject1 = new Character('a');
Character characterObject2 = new Character('A');
if (characterObject1.equals(characterObject2))
 System.out.println("Objects are equal.");
else
 System.out.println("Objects are Not equal.");
```

21. Suppose `result` is a variable of type `double` that has a value. Write a Java expression that returns a string that is the normal way of writing the value in `result`.
22. Suppose `stringForm` is a variable of type `String` that names a `String` that is the normal way of writing some `double`, such as `"41.99"`. Write a Java expression that returns the `double` value named by `stringForm`.
23. How would you do Self-Test Exercise 22 if the string might contain leading and/or trailing blanks, such as `" 41.99 "`?
24. Write Java code to output the largest and smallest values of type `long` allowed in Java.
25. How do you create an object of the class `Character` that corresponds to the letter `'Z'`?
26. Does the class `Character` have a no-argument constructor?

## 5.2 References and Class Parameters

*Do not mistake the pointing finger for the moon.*

ZEN SAYING

Variables of a class type and variables of a primitive type behave quite differently in Java. Variables of a primitive type name their values in a straightforward way. For

example, if `n` is an `int` variable, then `n` can contain a value of type `int`, such as 42. If `v` is a variable of a class type, then `v` does not directly contain an object of its class. Instead, `v` names an object by containing the memory address of where the object is located in memory. In this section, we discuss how a variable of a class type names objects, and we also discuss the related topic of how method parameters of a class type behave in Java.

## Variables and Memory

secondary  
and main  
memory

A computer has two forms of memory called *secondary memory* and *main memory*. The **secondary memory** is used to hold files for more or less permanent storage. The **main memory** is used by the computer when it is running a program. Values stored in a program's variables are kept in this main memory. It will help our understanding of class type variables to learn a few details about how program variables are represented in main memory. For now, assume that each variable in a program is of some primitive type, such as `int`, `double`, or `char`. Once you understand how variables of a primitive type are stored in memory, it will be easier to describe how variables of a class type behave.

byte

address

Main memory consists of a long list of numbered locations called **bytes**, each containing eight bits; that is, eight 0/1 digits. The number that identifies a byte is called its **address**. A data item, such as a number or a letter, can be stored in one of these bytes, and the address of the byte is then used to find the data item when it is needed.

memory  
location

variables of  
a primitive  
type

Most data types have values that require more than one byte of storage. When a data type requires more than one byte of storage, several adjacent bytes are used to hold the data item. In this case, the entire chunk of memory that holds the data item is still called a **memory location**. The address of the first of the bytes that make up this memory location is used as the address for this larger memory location. Thus, as a practical matter, you can think of the computer's main memory as a long list of memory locations of *varying sizes*. The size of each of these locations is expressed in bytes, and the address of the first byte is used as the address (name) of that memory location. Display 5.10 shows a picture of a hypothetical computer's main memory. Each primitive type variable in a program is assigned one of these memory locations, and the value of the variable is stored in this memory location.

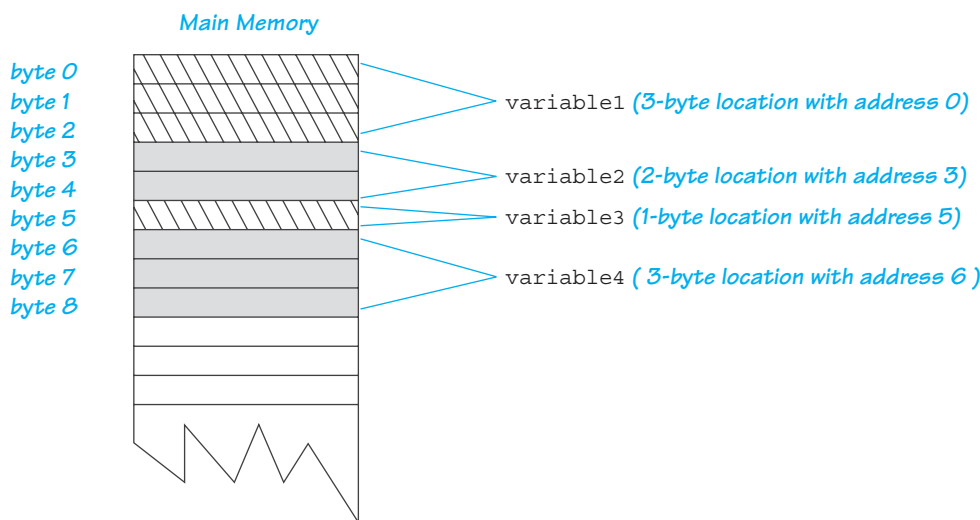
### Bytes and Addresses

Main memory is divided into numbered locations called **bytes**. The number associated with a byte is called its **address**. A group of consecutive bytes is used as the location for the value of a variable. The address of the first byte in the group is used as the address of this larger memory location.

### Why Eight Bits?

A **byte** is a memory location that can hold 8 bits. What is so special about 8? Why not 10 bits? There are two reasons why 8 is special. First, 8 is a power of 2 (8 is  $2^3$ ). Since computers use bits, which have only two possible values, powers of 2 are more convenient than powers of 10. Second, it turns out that 7 bits are required to code a single character of the ASCII character set. So 8 bits (1 byte) is the smallest power of 2 that will hold a single ASCII character.

Display 5.10 Variables in Memory



### References

In order to have a simple example to help explain *references*, we will use the class `ToyClass` defined in Display 5.11.

Variables of a class type name objects of their class differently than how variables of primitive types, such as `int` or `char`, store their values. Every variable, whether of a primitive type or a class type, is implemented as a location in the computer memory. For a variable of a primitive type, the value of the variable is stored in the memory location assigned to the variable. However, a variable of a class type stores only the memory address of where an object is located. The object named by the variable is stored in some other location in memory, and the variable contains only the memory

Display 5.11 A Simple Class

---

```
1 public class ToyClass
2 {
3 private String name;
4 private int number;

5 public ToyClass(String initialName, int initialNumber)
6 {
7 name = initialName;
8 number = initialNumber;
9 }

10 public ToyClass()
11 {
12 name = "No name yet.";
13 number = 0;
14 }

15 public void set(String newName, int newNumber)
16 {
17 name = newName;
18 number = newNumber;
19 }

20 public String toString()
21 {
22 return (name + " " + number);
23 }

24 public static void changer(ToyClass aParameter)
25 {
26 aParameter.name = "Hot Shot";
27 aParameter.number = 42;
28 }

29 public boolean equals(ToyClass otherObject)
30 {
31 return ((name.equals(otherObject.name))
32 && (number == otherObject.number));
33 }
34 }
```

---

**references**

address of where the object is stored. This memory address is called a **reference** (to the object).<sup>1</sup> This is diagrammed in Display 5.12.

Variables of a primitive type and variables of a class type are different for a reason. A value of a primitive type, such as the type `int`, always requires the same amount of memory to store one value. There is a maximum value of type `int`, so values of type `int` have a limit on their size. However, an object of a class type, such as an object of the class `String`, might be of any size. The memory location for a variable of type `String` is of a fixed size, so it cannot store an arbitrarily long string. It can, however, store the address of any string because there is a limit to the size of an address.

Because variables of a class type contain a reference (memory address), two variables may contain the same reference, and in such a situation, both variables name the same object. Any change to the object named by one of these variables will produce a change to the object named by the other variable, because they are the same object. For example, consider the following code. (The class `ToyClass` is defined in Display 5.11, but the meaning of the code should be obvious and you should not need to look up the definition.)

```
ToyClass variable1 = new ToyClass("Joe", 42);
ToyClass variable2;
variable2 = variable1; //Now both variables name the same object.
variable2.set("Josephine", 1);
System.out.println(variable1); //Invokes variable1's toString
 //method
```

The output is

```
Josephine 1
```

The object named by `variable1` has been changed without ever using the name `variable1`. This is diagrammed in Display 5.13.

### Variables of a Class Type Hold References

A variable of a primitive type stores a value of that type. However, a variable of a class type does not store an object of that class. A variable of a class type stores the reference (memory address) of where the object is located in the computer's memory. This causes some operations, such as `=` and `==`, to behave quite differently for variables of a class type than they do for variables of a primitive type.

### Reference Types

A type whose variables contain references are called **reference types**. In Java, class types are reference types, but primitive types are not reference types.

<sup>1</sup> Readers familiar with languages that use pointers will recognize a reference as another name for a pointer. However, Java does not use the term *pointer* but instead uses the term *reference*. Moreover, these references are handled automatically. There are no programmer-accessible pointer (reference) operations for dereferencing or other pointer operations. The details are all handled automatically in Java.

Display 5.12 Class Type Variables Store a Reference

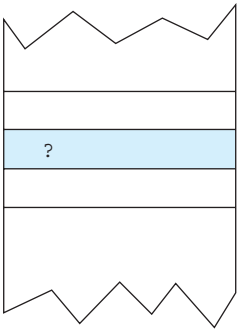
```
public class ToyClass
{
 private String name;
 private int number;
```

*The complete definition of the class  
ToyClass is given in Display 5.11.*

```
ToyClass sampleVariable;
```

*Creates the variable sampleVariable in  
memory but assigns it no value*

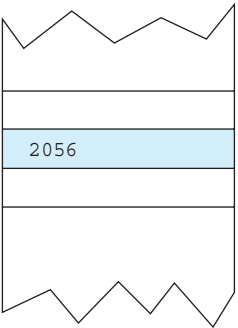
sampleVariable



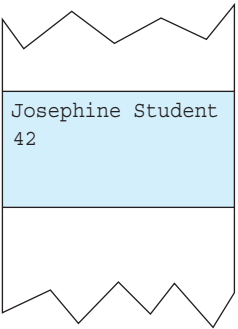
```
sampleVariable =
new ToyClass("Josephine Student", 42);
```

*Creates an object, places the object someplace in memory, and then  
places the address of the object in the variable sampleVariable. We  
do not know what the address of the object is, but let's assume it is  
2056. The exact number does not matter.*

sampleVariable



2056



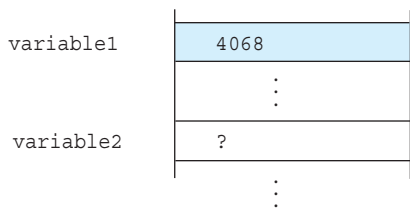
*For emphasis, we made the  
arrow point to the memory  
location referenced.*





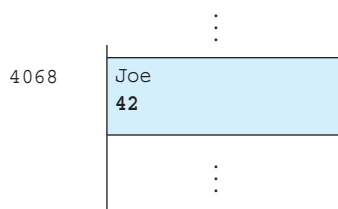
Display 5.13 Assignment Operator with Class Type Variables (part 1 of 2)

```
ToyClass variable1 = new ToyClass("Joe", 42);
ToyClass variable2;
```



*We do not know what memory address (reference) is stored in the variable variable1. Let's say it is 4068. The exact number does not matter.*

*Someplace else in memory:*

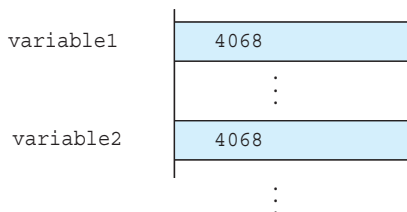


*Note that you can think of*

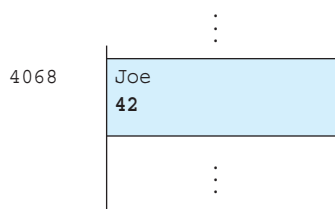
```
new ToyClass("Joe", 42)
```

*as returning a reference.*

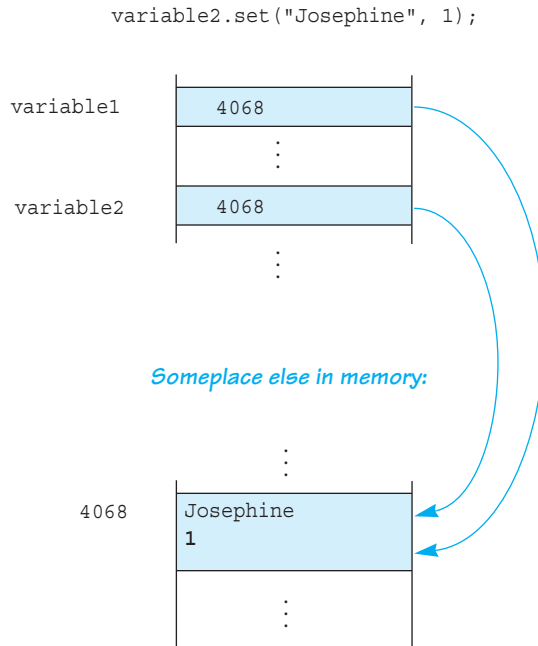
```
variable2 = variable1;
```



*Someplace else in memory:*



Display 5.13 Assignment Operator with Class Type Variables (part 2 of 2)



### assignment with variables of a class type

Note that when you use the assignment operator with variables of a class type, you are assigning a reference (memory address), so the result of the following is to make `variable1` and `variable2` two names for the same object:

```
variable2 = variable1;
```

A variable of a class type stores a memory address, and a memory address is a number. However, a variable of a class type cannot be used like a variable of a number type, such as `int` or `double`. This is intentional. The important property of a memory address is that it identifies a memory location. The fact that the implementors used numbers, rather than letters or strings or something else, to name memory locations is an accidental property. Java prevents you from using this accidental property to stop you from doing things such as obtaining access to restricted memory or otherwise screwing up the computer.

## Class Parameters

Strictly speaking, all parameters in Java are call-by-value parameters. This means that when an argument is plugged in for a parameter (of any type), the argument is evaluated and the value obtained is used to initialize the value of the parameter. (Recall that a parameter is really a local variable.) However, in the case of a parameter of a

class type, the value plugged in is a reference (memory address), which makes class parameters behave quite differently from parameters of a primitive type.

Recall that the following makes `variable1` and `variable2` two names for the same object:

```
ToyClass variable1 = new ToyClass("Joe", 42);
ToyClass variable2;
variable2 = variable1;
```

So, any change made to `variable2` is, in fact, made to `variable1`. The same thing happens with parameters of a class type. The parameter is a local variable that is set equal to the value of its argument. But if its argument is a variable of a class type, this copies a reference into the parameter. So, the parameter becomes another name for the argument, and any change made to the object named by the parameter is made to the object named by the argument, because they are the same object. Thus, a method can change the instance variables of an object given as an argument. A simple program to illustrate this is given in Display 5.14. Display 5.15 contains a diagram of the computer's memory as the program in Display 5.14 is executed.

Many programming languages have a parameter passing mechanism known as *call-by-reference*. If you are familiar with call-by-reference parameters, we should note that the Java parameter passing mechanism is similar to, but is not exactly the same as, call-by-reference.

Display 5.14 Parameters of a Class Type

---

```
1 public class ClassParameterDemo
2 {
3 public static void main(String[] args)
4 {
5 ToyClass anObject = new ToyClass("Mr. Cellophane", 0);
6 System.out.println(anObject);
7 System.out.println(
8 "Now we call changer with anObject as argument.");
9 ToyClass.changer(anObject);
10 System.out.println(anObject);
11 }
12 }
```

*ToyClass is defined in Display 5.11.*

*Notice that the method changer changed the instance variables in the object anObject.*

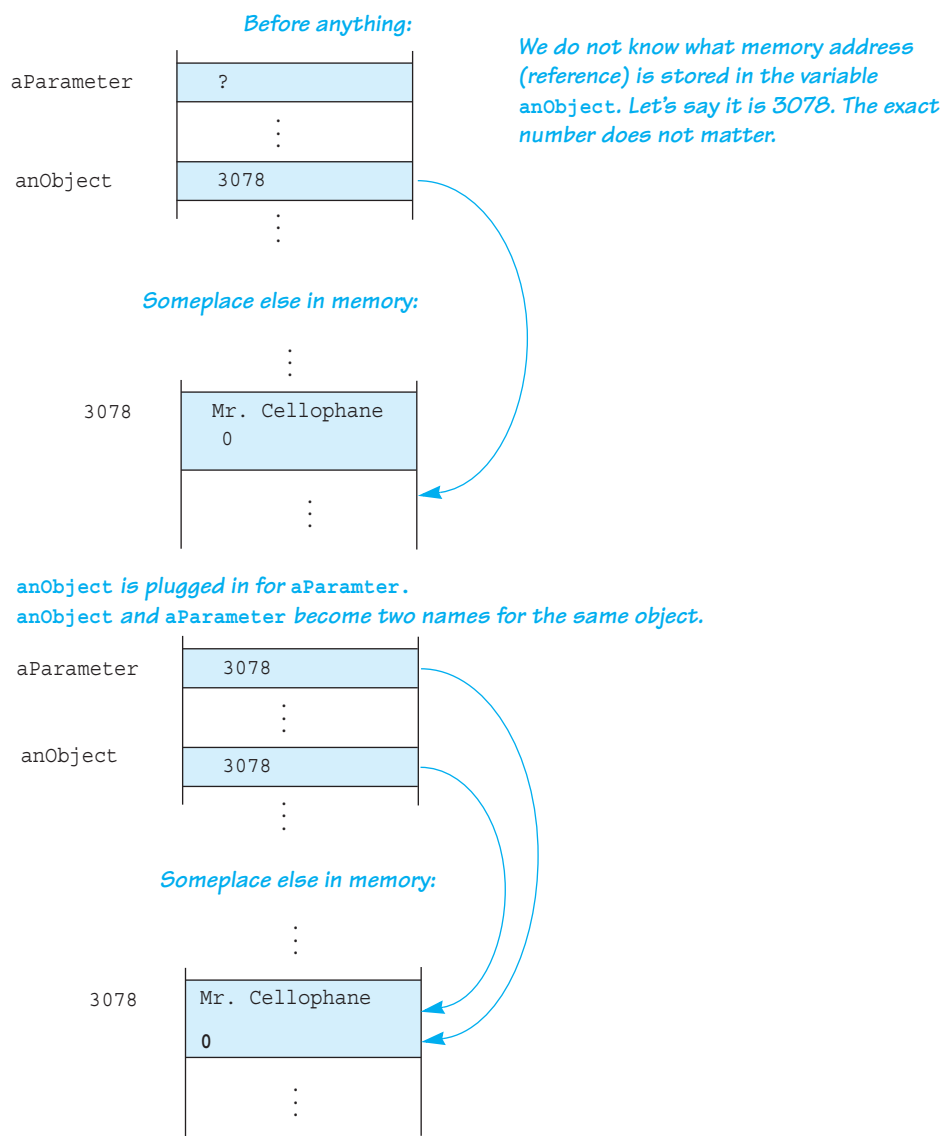
#### Sample Dialogue

```
Mr. Cellophane 0
Now we call changer with anObject as argument.
Hot Shot 42
```

Differences Between Primitive and Class-Type Parameters

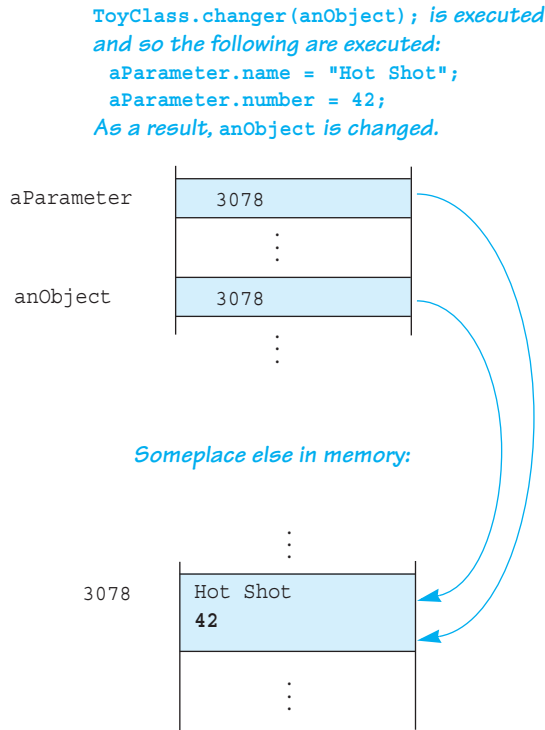
A method cannot change the value of a variable of a primitive type that is an argument to the method. On the other hand, a method can change the values of the instance variables of an argument of a class type. This is illustrated in Display 5.16.

Display 5.15 Memory Picture for Display 5.14 (part 1 of 2)



(continued)

Display 5.15 Memory Picture for Display 5.14 (part 2 of 2)



Display 5.16 Comparing Parameters of a Class Type and a Primitive Type (part 1 of 2)

```

1 public class ParametersDemo
2 {
3 public static void main(String[] args)
4 {
5 ToyClass2 object1 = new ToyClass2(),
6 object2 = new ToyClass2();
7 object1.set("Scorpius", 1);
8 object2.set("John Crichton", 2);
9 System.out.println("Value of object2 before call to method:");
10 System.out.println(object2);
11 object1.makeEqual(object2);
12 System.out.println("Value of object2 after call to method:");
13 System.out.println(object2);
14 }

```

*ToyClass2 is defined in Display 5.17.*

Display 5.16 Comparing Parameters of a Class Type and a Primitive Type (part 2 of 2)

```

15 int aNumber = 42;
16 System.out.println("Value of aNumber before call to method:"
17 + aNumber);
18 object1.tryToMakeEqual(aNumber);
19 System.out.println("Value of aNumber after call to method:"
20 + aNumber);
21 }
22 }

```

## Sample Dialogue

|                                            |                                                         |
|--------------------------------------------|---------------------------------------------------------|
| Value of object2 before call to method:    |                                                         |
| John Crichton 2                            | <i>← An argument of a class type can change.</i>        |
| Value of object2 after call to method:     |                                                         |
| Scorpius 1                                 | <i>←</i>                                                |
| Value of aNumber before call to method: 42 | <i>← An argument of a primitive type cannot change.</i> |
| Value of aNumber after call to method: 42  | <i>←</i>                                                |

**PITFALL: Use of = and == with Variables of a Class Type**

**== with  
variables of  
a class type**

You have already seen that the assignment operator used with variables of a class type produces two variables that name the same object, which is very different from how assignment behaves with variables of a primitive type.

The test for equality using == with variables of a class type also behaves in what may seem like a peculiar way. The operator == does not check that the objects have the same values for their instance variables. It merely checks for equality of memory addresses, so two objects in two different locations in memory would test as being “not equal” when compared using ==, even if their instance variables contain equivalent data. For example, consider the following code. (The class `ToyClass2` is defined in Display 5.17.)

```

ToyClass2 variable1 = new ToyClass2("Chiana", 3),
 variable2 = new ToyClass2("Chiana", 3);
if (variable1 == variable2)
 System.out.println("Equal using ==");
else
 System.out.println("Not equal using ==");

```

This code will produce the output

```
Not equal using ==
```

(continued)



### PITFALL: (continued)

Even though these two variables name objects that are intuitively equal, they are stored in two different locations in the computer's memory. This is why you usually use an `equals` method to compare objects of a class type. The variables `variable1` and `variable2` would be considered "equal" if compared using the `equals` method as defined for the class `ToyClass2` (Display 5.17). ■

Display 5.17 A Toy Class to Use in Display 5.16

```

1 public class ToyClass2
2 {
3 private String name;
4 private int number;

5 public void set(String newName, int newNumber)
6 {
7 name = newName;
8 number = newNumber;
9 }

10 public String toString()
11 {
12 return (name + " " + number);
13 }

14 public void makeEqual(ToyClass2 anObject)
15 {
16 anObject.name = this.name;
17 anObject.number = this.number;
18 }

19 public void tryToMakeEqual(int aNumber)
20 {
21 aNumber = this.number;
22 }

23 public boolean equals(ToyClass2 otherObject)
24 {
25 return ((name.equals(otherObject.name))
26 && (number == otherObject.number));
27 }

```

*Read the text for a discussion of  
the problem with this method.*

*<Other methods can be the same as in Display 5.11, although no  
other methods are needed or used in the current discussion.>*

```

28 }

```

## Self-Test Exercises

27. What is a reference type? Are class types reference types? Are primitive types (such as `int`) reference types?
28. When comparing two objects of a class type to see if they are “equal” or not, should you use `==` or the method `equals`?
29. When comparing two objects of a primitive type (such as `int`) to see if they are “equal” or not, should you use `==` or the method `equals`?
30. Can a method with an argument of a class type change the values of the instance variables in the object named by the argument? For example, if the argument is of type `ToyClass` defined in Display 5.11, can the method change the name of its argument?
31. Suppose a method has a parameter of type `int` and the method is given an `int` variable as an argument. Could the method have been defined so that it changes the value of the variable given as an argument?

## The Constant `null`

### `null`

The constant `null` is a special constant that may be assigned to a variable of any class type. It is used to indicate that the variable has no “real value.” If the compiler insists that you initialize a variable of a class type and there is no suitable object with which to initialize it, you can use the value `null`, as in the following example:

```
YourClass yourObject = null;
```

It is also common to use `null` in constructors to initialize instance variables of a class type when there is no obvious object to use. We will eventually see other uses for the constant `null`.

Note that `null` is not an object. It is like a reference (memory address) that does not refer to any object (does not name any memory location). So if you want to test whether a class variable contains `null`, you use `==` or `!=`; you do not use an `equals` method. For example, the following correctly tests for `null`:

```
if (yourObject == null)
 System.out.println("No real object here.");
```

### `null`

`null` is a special constant that can be used to give a value to any variable of any class type. The constant `null` is not an object but a sort of placeholder for a reference to an object. Because it is like a reference (memory address), use `==` and `!=` rather than the method `equals` when you test to see whether a variable contains `null`.





## PITFALL: Null Pointer Exception

If the compiler asks you to initialize a class variable, you can always initialize the variable to `null`. However, `null` is not an object, so you cannot invoke a method using a variable that is initialized to `null`. If you try, you will get an error message that says “Null Pointer Exception.” For example, the following code would produce a “Null Pointer Exception” if it were included in a program:

```
ToyClass2 aVariable = null;
String representation = aVariable.toString();
```

The problem is that you are trying to invoke the method `toString()` using `null` as a calling object. But `null` is not an object; it is just a placeholder. So `null` has no methods. Because you are using `null` incorrectly, the error message reads “Null Pointer Exception.” You get this error message any time a class variable has not been assigned a (reference to an) object, even if you have not assigned `null` to the variable. Any time you get a “Null Pointer Exception,” look for an uninitialized class variable.

The way to correct the problem is to use `new` to create a class object, as follows:

```
ToyClass2 aVariable = new ToyClass2("Chiana", 3);
String representation = aVariable.toString();
```

## The `new` Operator and Anonymous Objects

Consider an expression such as the following, where `ToyClass` is defined in Display 5.11:

```
ToyClass variable1 = new ToyClass("Joe", 42);
```

As illustrated in Display 5.13, the portion `new ToyClass("Joe", 42)` is an invocation of a constructor. You can think of the constructor as returning a reference to the location in memory of the object created by the constructor. If you take this view, the equal sign in this line of code is just an ordinary assignment operator.

There are times when you create an object using `new` and use the object as an argument to a method, but then never again use the object. In such cases, you need not give the object a variable name. You can instead use the expression with the `new` operator and the constructor directly as the argument. For example, suppose you want to test to see whether the object in `variable1` (in the earlier line of code) is equal to an object with the same number and with the name spelled in all uppercase letters. You can do so as follows:

```
if (variable1.equals(new ToyClass("JOE", 42)))
 System.out.println("Equal");
else
 System.out.println("Not equal");
```

This is equivalent to the following:

```
ToyClass temp = new ToyClass("JOE", 42);
if (variable1.equals(temp))
 System.out.println("Equal");
else
 System.out.println("Not equal");
```

In the second version, the object is created, and its reference is placed in the variable `temp`. Then `temp` is plugged in for the parameter in the `equals` method. But all the parameter passing mechanism does is to take the reference stored in `temp` and plug it into the parameter for `equals`. The first version simplifies the process. It creates the reference to the object and directly plugs it into the parameter in `equals`. It bypasses the variable `temp` but ends up plugging in the same reference as the argument to `equals`.

When not assigned to a variable, an expression such as

```
new ToyClass("JOE", 42)
```

anonymous  
object

is known as an **anonymous object**. It evaluates a reference to an object of the class. It is called *anonymous* because the object is not assigned a variable to serve as its name. We will eventually encounter situations where the use of such anonymous objects is common.

### Anonymous Objects

An expression with a `new` operator and a constructor creates a new object and returns a reference to the object. If this reference is not assigned to a variable, but instead the expression with `new` and the constructor is used as an argument to some method, then the object produced is called an **anonymous object**.

#### EXAMPLE

```
if (variable1.equals(new ToyClass("JOE", 42)))
 System.out.println("Equal");
else
 System.out.println("Not equal");
```

The expression `new ToyClass("JOE", 42)` (or more exactly, the object it creates) is an example of an anonymous object.

### EXAMPLE: Another Approach to Keyboard Input ★

This example uses the class `StringTokenizer`, which was covered in a starred section of Chapter 4. If you have not yet studied the `StringTokenizer` class, you may omit this example until you have done so.

The program in Display 5.18 is an example of the use of both the `StringTokenizer` class and the method `Double.parseDouble` to read multiple values of type `double` entered on a single line and separated with something other than whitespace. The entire

(continued)

**EXAMPLE:** (continued)

line is read as a single long string using the method `nextLine` of the `Scanner` class. The long string is decomposed into tokens using the class `StringTokenizer`. The tokens are the numbers that were input, but they are in the form of string values, not values of type `double`. Finally, the tokens are converted to values of type `double` using the method `Double.parseDouble`.

Note that the `String` variables are initialized to `null`. If you omit the `null`s, the compiler will complain that the variables `string1` and `string2` might not be initialized before they are used. The compiler is incorrect but it does no good to argue with it. Including `null` as initial values for the variables `string1` and `string2` will keep the compiler happy and allow you to run the program.

Display 5.18 Use of the Method `Double.parseDouble` (part 1 of 2)

```
1 import java.util.Scanner;
2 import java.util.StringTokenizer;

3 public class InputExample
4 {
5 public static void main(String[] args)
6 {
7 Scanner keyboard = new Scanner(System.in);

8 System.out.println("Enter two numbers on a line.");
9 System.out.println("Place a comma between the numbers.");
10 System.out.println("Extra blank space is OK.");
11 String inputLine = keyboard.nextLine();

12 String delimiters = ", "; //Comma and blank space
13 StringTokenizer numberFactory =
14 new StringTokenizer(inputLine, delimiters);

15 String string1 = null;
16 String string2 = null;
17 if (numberFactory.countTokens() >= 2)
18 {
19 string1 = numberFactory.nextToken();
20 string2 = numberFactory.nextToken();
21 }
22 else
23 {
24 System.out.println("Fatal Error.");
25 System.exit(0);
26 }
```

Display 5.18 Use of the Method `Double.parseDouble` (part 2 of 2)

```

27 double number1 = Double.parseDouble(string1);
28 double number2 = Double.parseDouble(string2);

29 System.out.print("You input ");
30 System.out.println(number1 + " and " + number2);
31 }
32 }

```

## Sample Dialogue

```

Enter two numbers on a line.
Place a comma between the numbers.
Extra blank space is OK.
 41.98, 42
You input is 41.98 and 42.0

```

## Self-Test Exercises

32. What is wrong with a program that starts as follows? The class `ToyClass` is defined in Display 5.11.

```

ToyClass anObject = null ;
anObject.set("Josephine", 42);

```

33. What is the type of the constant `null`?
34. Suppose `aVariable` is a variable of a class type. Which of the following correctly tests to see whether `aVariable` contains `null`?

```

aVariable.equals(null)
aVariable == null

```

35. Is the following legal in Java? The class `ToyClass` is defined in Display 5.11.

```

System.out.println(new ToyClass("Mr. Cellophane", 0));

```



## TIP: Use Static Imports ★

You have already seen `import` statements, such as the following from Display 5.9:

```
import java.util.Scanner;
```

There is another form of `import` statement which, while not essential, can be a convenience. These are called **static import statements** and are best explained with an example.

(continued)

**TIP: (continued)**

It would be convenient to be able to write static method invocations in the following simple form:

```
toUpperCase(firstCharacter)
```

instead of having to write the following longer version (taken from Display 5.9):

```
Character.toUpperCase(firstCharacter)
```

If you add the following static `import` statement to the start of your program, you can then write the invocation of `toUpperCase` in the desired shorter way:

```
import static java.lang.Character.toUpperCase;
```

The class `Character` is in the Java package `java.lang`. Note that you need to give the package name as well as the class name, just as you did with ordinary `import` statements, such as the above `import` statement for the class `Scanner` in the `java.util` package.

The package `java.lang` is imported automatically. So you can, for example, use the method `Character.toUpperCase` without any `import` statement of any kind. But note that there is nothing special about the package `java.lang` when it comes to static `import` statements. If you want to use the abbreviated form `toUpperCase`, you must give a static `import` statement.

If you use the following form of the static `import` statement, then your code can use the name of any static method in the class `Character` without the preface of `Character` and a dot.

```
import static java.lang.Character.*;
```

For example, consider the program in Display 5.9. If you replace

```
import java.util.Scanner;
```

with either

```
import java.util.Scanner;
import static java.lang.Character.toUpperCase;
```

or

```
import java.util.Scanner;
import static java.lang.Character.*;
```

then you can change the statement

```
sentence = Character.toUpperCase(firstCharacter)
 + sentence.substring(1);
```

to

**TIP: (continued)**

```
sentence = toUpperCase(firstCharacter)
 + sentence.substring(1);
```

One word of warning: This works for static methods only. This does not in any way apply to ordinary nonstatic methods.

The program `StaticImportDemo.java` on the accompanying website redoes the program in Display 5.9 using a static `import` statement in this way.

You can, of course, do this with `Character` replaced by any other class, and then you can use static methods in that class without the need to preface the method name with the class name and a dot.

You can use static `import` statements for constants as well as methods. If you want to use `PI` instead of `Math.PI` in the program in Display 5.7, just include one of the following `import` statements at the beginning of the program file:

```
import static java.lang.Math.PI;
```

or

```
import static java.lang.Math.*;
```

The program `StaticImportDemo2.java` on the accompanying website redoes the program in Display 5.7 using a static `import` statement in this way. ■

extra code  
on website

static import  
of constants

extra code  
on website

## 5.3 Using and Misusing References

*Loose lips sink ships.*

MILITARY SLOGAN

Just as a military campaign requires constant vigilance to ensure that its plans are kept secret, so your programming requires constant vigilance to ensure that private instance variables remain truly private. As we will see, adding the `private` modifier before instance variable declarations is not always all that you need to do. There can be privacy leaks in a poorly designed class just as there can be privacy leaks in a military campaign.

The material in this section is important but more subtle and harder to digest than the material we have seen before now. If you want, you may postpone reading this section until you have had more practice defining and using classes. You do not need the material in this section to understand Section 5.4.

### EXAMPLE: A Person Class

It is common to have instance variables of a class type. The class `Person` defined in Display 5.19 has two instance variables of type `Date`. So, the class `Person` has instance variables of a class type. (The class `Date` was defined in Chapter 4, Display 4.11. We have reproduced the relevant portions of the date class definition in Display 5.20.) In fact, all the instance variables for the class `Person` are of class types. An object of the class `Person` has the basic data about people that is found in such places as on tombstones and in author listings in library catalogues. It describes a person by giving the person's name, date of birth, and date of death. If the person is still alive, then the value `null` is used as the date of death. (So `null` is good.) A simple program illustrating the class `Person` is given in Display 5.21. We will discuss a few details about the class `Person` here, but most of the various methods in the class `Person` will be discussed as we cover the corresponding topic in the following subsections.

Normally, a class definition should include a no-argument constructor. However, there are cases where a no-argument constructor makes little sense. For example, the wrapper classes such as `Integer` and `Double` do not have no-argument constructors, as we explained in the Pitfall subsection “A Wrapper Class Does Not Have a No-Argument Constructor,” which appeared earlier in this chapter. The class `Person` also does not have a no-argument constructor for a reason. A person may have no date of death, but a person always has a date of birth. A no-argument constructor should initialize all instance variables, but there is no suitable value to initialize the instance variable `born` unless it is provided as an argument to the constructor. In particular, it makes no sense to initialize the instance variable `born` to `null`; that would indicate that the person was never born. It makes little sense to have a person who was never born, so it makes little sense to have a no-argument constructor for the class `Person`. Note that because we defined some constructors for the class `Person` but did not define a no-argument constructor, it follows that the class `Person` does not have a no-argument constructor.

Since we are assuming that an object of the class `Person` always has a birth date (which is not `null`), the following should always be true of an object of the class `Person`:

An object of the class `Person` has a date of birth (which is not `null`), and if the object has a date of death, then the date of death is equal to or later than the date of birth.

If you check the definition of the class `Person`, you will see that this statement is always true. It is true of every object created by a constructor, and all the other methods preserve the truth of this statement. In fact, the private method `consistent` was designed to provide a check for this property. A statement, such as the above, that is always true for every object of the class is called a **class invariant**.

Note that the definition of `equals` for the class `Person` includes an invocation of `equals` for the class `String` and an invocation of the method `equals` for the class `Date`. Java determines which `equals` method is being invoked from the type of its calling object. Because the instance variable `name` is of type `String`, the invocation `name.equals(...)` is an invocation of the method `equals` for the class `String`. Because the instance variable `born` is of type `Date`, the invocation `born.equals(...)` is an invocation of the method `equals` for the class `Date`.

Similarly, the definition of the method `toString` for the class `Person` includes invocations of the method `toString` for the class `Date`.

Display 5.19 A Person Class (part 1 of 5)

```

1 /**
2 Class for a person with a name and dates for birth and death.
3 Class invariant: A Person always has a date of birth, and if the Person
4 has a date of death, then the date of death is equal to or later than the
5 date of birth.
6 */
7 public class Person
8 {
9 private String name;
10 private Date born;
11 private Date died; //null indicates still alive.

12 public Person(String initialName, Date birthDate, Date deathDate)
13 {
14 if (consistent(birthDate, deathDate))
15 {
16 name = initialName;
17 born = new Date(birthDate);
18 if (deathDate == null)
19 died = null;
20 else
21 died = new Date(deathDate);
22 }
23 else
24 {
25 System.out.println("Inconsistent dates.Aborting.");
26 System.exit(0);
27 }
28 }

29 public Person(Person original)
30 {
31 if (original == null)
32 {
33 System.out.println("Fatal error.");
34 System.exit(0);
35 }

36 name = original.name;
37 born = new Date(original.born);

38 if (original.died == null)
39 died = null;
40 else
41 died = new Date(original.died);
42 }

```

*The class Date was defined in Display 4.11 and many of the details are repeated in Display 5.20.*

*We will discuss Date and the significance of these constructor invocations later in this chapter in the subsection entitled "Copy Constructors."*

*Copy constructor*

(continued)



Display 5.19 A Person Class (part 2 of 5)

```

43 public void set(String newName, Date birthDate, Date deathDate)
 <Definition of this method is Self-Test Exercise 4.1.>

44 public String toString()
45 {
46 String diedString;
47 if (died == null)
48 diedString = ""; //Empty string
49 else
50 diedString = died.toString();
51
52 return (name + ", " + born + "-" + diedString);
53
54 public boolean equals(Person otherPerson)
55 {
56 if (otherPerson == null)
57 return false;
58 else
59 return (name.equals(otherPerson.name)
60 && born.equals(otherPerson.born)
61 && datesMatch(died, otherPerson.died));
62
63 /**
64 To match, date1 and date2 either must be the same date or must both be null.
65 */
66 private static boolean datesMatch(Date date1, Date date2)
67 {
68 if (date1 == null)
69 return (date2 == null);
70 else if (date2 == null) //&& date1 != null
71 return false ;
72 else //Both dates are not null.
73 return (date1.equals(date2));
74
75 /**
76 Precondition: newDate is a consistent date of birth.
77 Postcondition: Date of birth of the calling object is newDate.
78 */
79 public void setBirthDate(Date newDate)
80 {
81 if (consistent(newDate, died))
82 born = new Date(newDate);
83 else

```

*This is the toString method of the class Date.*

*This is equivalent to born.toString().*

*This is the equals method for the class String.*

*This is the equals method for the class Date.*

Display 5.19 A Person Class (part 3 of 5)

```

83 {
84 System.out.println("Inconsistent dates. Aborting.");
85 System.exit(0);
86 }
87 }

88 /**
89 * Precondition: newDate is a consistent date of death.
90 * Postcondition: Date of death of the calling object is newDate.
91 */
92 public void setDeathDate(Date newDate)
93 {
94
95 if (!consistent(born, newDate))
96 {
97 System.out.println("Inconsistent dates. Aborting.");
98 System.exit(0);
99 }
100
101 if (newDate == null)
102 died = null;
103 else
104 died = new Date(newDate);
105 }

106 public void setName(String newName)
107 {
108 name = newName;
109 }

110 /**
111 * Precondition: The date of birth has been set, and changing the year
112 * part of the date of birth will give a consistent date of birth.
113 * Postcondition: The year of birth is (changed to) newYear.
114 */
115 public void setBirthYear(int newYear)
116 {
117 if (born == null) //Precondition is violated.
118 {
119 System.out.println("Fatal Error. Aborting.");
120 System.exit(0);
121 }
122 born.setYear(newYear);
123 if (!consistent(born, died))
124 {
125 System.out.println("Inconsistent dates. Aborting.");
126 System.exit(0);
127 }
128 }

```

The date of death can be null. However, there is no corresponding code in `setBirthDate` because the method consistent ensures that the date of birth is never null.

(continued)

Display 5.19 A Person Class (part 4 of 5)

---

```
129 /**
130 Precondition: The date of death has been set, and changing the year
131 part of the date of death will give a consistent date of death.
132 Postcondition: The year of death is (changed to) newYear.
133 */
134 public void setDeathYear(int newYear)
135 {
136 if (died == null) //Precondition is violated.
137 {
138 System.out.println("Fatal Error. Aborting.");
139 System.exit(0);
140 }
141 died.setYear(newYear);
142 if (!consistent(born, died))
143 {
144 System.out.println("Inconsistent dates. Aborting.");
145 System.exit(0);
146 }
147 }

148 public String getName()
149 {
150 return name;
151 }

152 public Date getBirthDate()
153 {
154 return new Date(born);
155 }

156 public Date getDeathDate()
157 {
158 if (died == null)
159 return null;
160 else
161 return new Date(died);
162 }

163 /**
164 To be consistent, birthDate must not be null. If there is no date of
165 death (deathDate == null), that is consistent with any birthDate.
166 Otherwise, the birthDate must come before or be equal to the
167 deathDate.
168 */
```

Display 5.19 A Person Class (part 5 of 5)

```
168 private static boolean consistent(Date birthDate, Date deathDate)
169 {
170 if (birthDate == null)
171 return false;
172 else if (deathDate == null)
173 return true;
174 else
175 return (birthDate.precedes(deathDate)
176 || birthDate.equals(deathDate));
177 }
178 }
```

### Class Invariant

A statement that is always true for every object of the class is called a **class invariant**. A class invariant can help to define a class in a consistent and organized way.



### PITFALL: null Can Be an Argument to a Method

If a method has a parameter of a class type, then `null` can be used as the corresponding argument when the method is invoked. Sometimes using `null` as an argument can be the result of an error, but it can sometimes be an intentional argument. For example, the class `Person` (Display 5.19) uses `null` for a date of death to indicate that the person is still alive. So `null` is sometimes a perfectly normal argument for methods such as `consistent`. Method definitions should account for `null` as a possible argument and not assume the method always receives a true object to plug in for a class parameter.

Notice the definition of the method `equals` for the class `Person`. A test for equality has the form

```
object1.equals(object2)
```

The calling object `object1` must be a true object of the class `Person`; a calling object cannot be `null`. However, the argument `object2` can be either a true object or `null`. If the argument is `null`, then `equals` should return `false`, because a true object cannot reasonably be considered to be equal to `null`. In fact, the Java documentation specifies that when the argument to an `equals` method is `null`, the `equals` method should return `false`. Notice that our definition does return `false` when the argument is `null`. ■

Display 5.20 The Class `Date` (Partial Definition) (part 1 of 2)

---

```

1 public class Date
2 {
3 private String month;
4 private int day;
5 private int year; //A four digit number.

6 public Date(String monthString, int day, int year)
7 {
8 setDate (monthString, day, year);
9 }

10 public Date(Date aDate)
11 {
12 if (aDate == null) //Not a real date.
13 {
14 System.out.println("Fatal Error.");
15 System.exit(0);
16 }

17 month = aDate.month;
18 day = aDate.day;
19 year = aDate.year;
20 }

21 public void setDate(String monthString, int day, int year)
22 {
23 if (dateOK(monthString, day, year))
24 {
25 this.month = monthString;
26 this.day = day;
27 this.year = year;
28 }
29 else
30 {
31 System.out.println("Fatal Error");
32 System.exit(0);
33 }
34 }
35 public void setYear(int year)
36 {
37 if ((year < 1000) || (year > 9999))
38 {

```

*This is not a complete definition of the class `Date`.  
The complete definition of the class `Date` is in Display 4.11,  
but this has the details that are important to what we are  
discussing in this chapter.*

*Copy constructor*

*The method `dateOK` checks that the  
date is a legitimate date, such as  
not having more than 31 days.*

Display 5.20 The Class `Date` (Partial Definition) (part 2 of 2)

---

```

39 System.out.println("Fatal Error");
40 System.exit(0);
41 }
42 else
43 this.year = year;
44 }
45 public String toString()
46 ...
47 public boolean equals(Date otherDate)
48
49 /**
50 Returns true if the calling object date is before otherDate (in time).
51 */
52 public boolean precedes(Date otherDate)
53 ...
54 private boolean dateOK(String monthString, int dayInt, int yearInt)
55 ...
56 }
```

*The complete definition of equals is given later in this chapter in the answer to Self-Test Exercise 37, and is a better version than the one given in Chapter 4.*

*These methods have the obvious meanings. If you need to see a full definition, see Display 4.11 in Chapter 4 and Self-Test Exercise 37 later in this chapter.*

---

### Self-Test Exercises

36. What is the difference between the following two pieces of code? The first piece appears in Display 5.21.

```

Person adams =
 new Person("John Adams",
 new Date("February", 15, 1947), null);
//Second piece is below:

Date theDate = new Date("February", 15, 1947);
Person adams = new Person("John Adams", theDate, null);

```

37. When we defined the class `Date` in Chapter 4 (Display 4.11), we had not yet discussed `null`. So, the definition of `equals` given there did not account for the possibility that the argument could be `null`. Rewrite the definition of `equals` for the class `Date` to account for the possibility that the argument might be `null`.

Display 5.21 Demonstrating the Class `Person`

---

```
1 public class PersonDemo
2 {
3 public static void main(String[] args)
4 {
5 Person bach =
6 new Person("Johann Sebastian Bach",
7 new Date("March", 21, 1685), new Date("July", 28, 1750));
8 Person stravinsky =
9 new Person("Igor Stravinsky",
10 new Date("June", 17, 1882), new Date("April", 6, 1971));
11 Person adams =
12 new Person("John Adams",
13 new Date("February", 15, 1947), null);
14
15 System.out.println("A Short List of Composers:");
16 System.out.println(bach);
17 System.out.println(stravinsky);
18 System.out.println(adams);
19
20 Person bachTwin = new Person(bach);
21 System.out.println("Comparing bach and bachTwin:");
22 if (bachTwin == bach)
23 System.out.println("Same reference for both.");
24 else
25 System.out.println("Distinct copies.");
26
27 if (bachTwin.equals(bach))
28 System.out.println("Same data.");
29 else
30 System.out.println("Not same data.");
31 }
32 }
```

#### Sample Dialogue

```
A Short List of Composers:
Johann Sebastian Bach, March 21, 1685-July 28, 1750
Igor Stravinsky, June 17, 1882-April 6, 1971
John Adams, February 15, 1947-
Comparing bach and bachTwin:
Distinct copies.
Same data.
```

---

## Copy Constructors

### copy constructor

A **copy constructor** is a constructor with a single argument of the same type as the class. The copy constructor should create an object that is a separate, independent object but with the instance variables set so that it is an exact copy of the argument object.

For example, Display 5.20 reproduces the copy constructor for the class `Date` defined in Display 4.11. The copy constructor, or any other constructor, creates a new object of the class `Date`. This happens automatically and is not shown in the code for the copy constructor. The code for the copy constructor then goes on to set the instance variables to the values equal to those of its one parameter, `aDate`. But the new date created is a separate object even though it represents the same date. Consider the following code:

```
Date date1 = new Date("January", 1, 2015);
Date date2 = new Date(date1);
```

After this code is executed, both `date1` and `date2` represent the date January 1, 2015, but they are two different objects. So, if we change one of these objects, it will not change the other. For example, consider

```
date2.setDate("July", 4, 1776);
System.out.println(date1);
```

The output produced is

```
January 1, 2015
```

When we changed `date2`, we did not change `date1`. This may not be a difficult or even subtle point, but it is critically important to much of what we discuss in this section of the chapter. (See Self-Test Exercise 39 in this chapter to see the copy constructor contrasted with the assignment operator.)

Now let's consider the copy constructor for the class `Person` (Display 5.19), which is a bit more complicated. It is reproduced in what follows:

```
public Person(Person original)
{
 if (original == null)
 {
 System.out.println("Fatal error.");
 System.exit(0);
 }
 name = original.name;
 born = new Date(original.born);
 if (original.died == null)
 died = null;
 else
 died = new Date(original.died);
}
```



We want the object created to be an independent copy of `original`. That would not happen if we had used the following instead:

```
public Person(Person original) //Unsafe
{
 if (original == null)
 {
 System.out.println("Fatal error.");
 System.exit(0);
 }
 name = original.name;
 born = original.born; //Not good.
 died = original.died; //Not good.
}
```

Although this alternate definition looks innocent enough and may work fine in many situations, it does have serious problems.

Assume we had used the unsafe version of the copy constructor instead of the one in Display 5.19. The “Not good.” code simply copies references from `original.born` and `original.died` to the corresponding arguments of the object being created by the constructor. So, the object created is not an independent copy of the `original` object. For example, consider the code

```
Person original =
 new Person("Natalie Dressed",
 new Date("April", 1, 1984), null);
Person copy = new Person(original);
copy.setBirthYear(1800);
System.out.println(original);
```

The output would be

```
Natalie Dressed, April 1, 1800-
```

When we changed the birth year in the object `copy`, we also changed the birth year in the object `original`. This is because we are using our unsafe version of the copy constructor. Both `original.born` and `copy.born` contain the same reference to the same `Date` object.

This all happens because we used the unsafe version of the copy constructor. Fortunately, here we use a safer version of the copy constructor that sets the `born` instance variables as follows:

```
born = new Date(original.born);
```

which is equivalent to

```
this.born = new Date(original.born);
```

This version, which we did use, makes the instance variable `this.born` an independent `Date` object that represents the same date as `original.born`. So if you change a date in the `Person` object created by the copy constructor, you will not change that date in the original `Person` object.

Note that if a class, such as `Person`, has instance variables of a class type, such as the instance variables `born` and `died`, then to define a correct copy constructor for the class `Person`, you must already have copy constructors for the class `Date` of the instance variables. The easiest way to ensure this for all your classes is to always include a copy constructor in every class you define.

### Copy Constructor

A **copy constructor** is a constructor with one parameter of the same type as the class. A copy constructor should be designed so the object it creates is intuitively an exact copy of its parameter, but a completely independent copy. See Displays 5.19 and 5.20 for examples of copy constructors.

#### clone

The Java documentation says to use a method named `clone` instead of a copy constructor, and, as you will see later in this book, there are situations where the copy constructor will not work as desired and you need the `clone` method. However, we do not yet have enough background to delve into this method. (It is discussed later in this book in Chapters 8 and 13.) Despite the Java documentation, many excellent programmers prefer to sometimes use copy constructors. In this book, we will use both copy constructors and the `clone` method.



### PITFALL: Privacy Leaks

#### leaking accessor methods

Consider the accessor method `getBirthDate` for the class `Person` (Display 5.19), which we reproduce in what follows:

```
public Date getBirthDate()
{
 return new Date(born);
}
```

Do not make the mistake of defining the accessor method as follows:

```
public Date getBirthDate() //Unsafe
{
 return born; //Not good
}
```

(continued)

**PITFALL: (continued)**

Assume we had used the unsafe version of `getBirthDate` instead of the one in Display 5.19. It would then be possible for a program that uses the class `Person` to change the private instance variable `born` to any date whatsoever and bypass the checks in constructor and mutator methods of the class `Person`. For example, consider the following code, which might appear in some program that uses the class `Person`:

```
Person citizen = new Person(
 "Joe Citizen", new Date("January", 1, 1900), new Date("January", 1,
 1990));
Date dateName = citizen.getBirthDate();
dateName.setDate("April", 1, 3000);
```

This code changes the date of birth so it is after the date of death (an impossibility in the universe as we know it). This citizen was not born until after he or she died! This sort of situation is known as a **privacy leak**, because it allows a programmer to circumvent the `private` modifier before an instance variable such as `born`, and to change the private instance variable to anything whatsoever.

The following code would be illegal in our program:

```
citizen.born.setDate("April", 1, 3000); //Illegal
```

This is illegal because `born` is a private instance variable. However, with the unsafe version of `getBirthDate` (and we are now assuming that we did use the unsafe version), the variable `dateName` contains the same reference as `citizen.born` and so the following is legal and equivalent to the illegal statement:

```
dateName.setDate("April", 1, 3000); //Legal and equivalent to
//illegal statement.
```

It is as if you have a friend named Robert who is also known as Bob. Some bully wants to beat up Robert, so you say “You cannot beat up Robert.” The bully says “OK, I will not beat up Robert, but I will beat up Bob.” Bob and Robert are two names for the same person. So, if you protect Robert but do not protect Bob, you have really accomplished nothing.

This is all if we used the unsafe version of `getBirthDate`, which simply returns the reference in the private instance variable `born`. Fortunately, here we use a safer version of `getBirthDate`, which has the following return statement:

```
return new Date(born);
```

privacy leak



## PITFALL: (continued)

### leaking mutator methods

This return statement does not return the reference in the private instance variable `born`. Instead, it uses the copy constructor to return a reference to a new object that is an exact copy of the object named by `born`. If the copy is changed, that has no effect on the date whose reference is in the instance variable `born`. Thus, a privacy leak is avoided.

Note that returning a reference is not the only possible source of privacy leaks. A privacy leak can also arise from an incorrectly defined constructor or mutator method. Notice the definition for the method `setBirthDate` in Display 5.19 and reproduced as follows:

```
public void setBirthDate(Date newDate)
{
 if (consistent(newDate, died))
 born = new Date(newDate);
 else
 {
 System.out.println("Inconsistent dates. Aborting.");
 System.exit(0);
 }
}
```

Note that the instance variable `born` is set to a copy of the parameter `newDate`. Suppose that instead of

```
born = new Date(newDate);
```

we simply use

```
born = newDate;
```

And suppose we use the following code in some program:

```
Person personObject = new Person(
 "Josephine", new Date("January", 1, 2000), null);
Date dateName = new Date("February", 2, 2002);
personObject.setBirthDate(dateName);
```

where `personObject` names an object of the class `Person`. The following will change the year part of the `Date` object named by the `born` instance variable of the object `personObject` and will do so without going through the checks in the mutator methods for `Person`:

```
dateName.setYear(1000);
```

(continued)



### PITFALL: (continued)

Because `dateName` contains the same reference as the private instance variable `born` of the object `personObject`, changing the year part of `dateName` changes the year part of the private instance variable `born` of `personObject`. Not only does this bypass the consistency checks in the mutator method `setBirthDate`, but it also is a likely source of an inadvertent change to the `born` instance variable.

If we define `setBirthDate` as we did in Display 5.19 and as shown in the following, this problem does not happen. (If you do not see this, go through the code step by step and trace what happens.)

```
public void setBirthDate(Date newDate)
{
 if (consistent(newDate, died))
 born = new Date(newDate);
 . . .
```

**clone**

One final word of warning: Using copy constructors in this manner is not the officially sanctioned way to make copies of an object in Java. The authorized way to is to define a method named `clone`. We will discuss `clone` methods in Chapters 8 and 13. In Chapter 8, we show you that, in some situations, there are advantages to using a `clone` method instead of a copy constructor. In Chapter 13, we describe the official way to define the `clone` method. For what we will be doing until then, a copy constructor will be a very adequate way of creating copies of an object.

### Self Test Exercises

38. What is a copy constructor?
39. What output is produced by the following code?

```
Date date1 = new Date("January", 1, 2006);
Date date2;
date2 = date1;
date2.setDate("July", 4, 1776);
System.out.println(date1);
```

What output is produced by the following code? Only the third line is different from the previous case.

```
Date date1 = new Date("January", 1, 2006);
Date date2;
date2 = new Date(date1);
date2.setDate("July", 4, 1776);
System.out.println(date1);
```

## Self Test Exercises

40. What output is produced by the following code?

```
Person original =
 new Person("Natalie Dressed",
 new Date("April", 1, 1984), null);
Person copy = new Person(original);
copy.setBirthDate(new Date("April", 1, 1800));
System.out.println(original)
```

## Mutable and Immutable Classes

Contrast the accessor methods `getName` and `getBirthDate` of the class `Person` (Display 5.19). We reproduce the two methods in what follows:

```
public String getName()
{
 return name;
}

public Date getBirthDate()
{
 return new Date(born);
}
```

Notice that the method `getBirthDate` does not simply return the reference in the instance variable `born`, but instead uses the copy constructor to return a reference to a copy of the birth date object. We already explained why we do this. If we return the reference in the instance variable `born`, then we can place this reference in a variable of type `Date`, and that variable could serve as another name for the private instance variable `born`, which would allow us to violate the privacy of the instance variable `born` by changing it using a mutator method of the class `Date`. This is exactly what we discussed in the previous subsection. So why not do something similar in the method `getName`?

The method `getName` simply returns the reference in the private instance variable `name`. So, if we do the following in a program, then the variable `nameAlias` will be another name for the `String` object of the private instance variable `name`:

```
Person citizen = new Person(
 "Joe Citizen", new Date("January", 1, 1900), new Date("January",
 1, 1990));
String nameAlias = citizen.getName();
```

It looks as though we could use a mutator method from the class `String` to change the name referenced by `nameAlias` and so violate the privacy of the instance variable `name`. Is something wrong? Do we have to rewrite the method `getName` to use the copy constructor for the class `String`? No, everything is fine. We cannot use a mutator method with `nameAlias` because the class `String` has no mutator methods! The class `String` contains no methods that change any of the data in a `String` object.

At first, it may seem as though you can change the data in an object of the class `String`. What about the string processing we have seen, such as the following?

```
String greeting = "Hello";
greeting = greeting + "friend.";
```

Have we not changed the data in the `String` object from "Hello" to "Hello friend."? No, we have not. The expression `greeting + "friend."` does not change the object "Hello"; it creates a new object, so the assignment statement

```
greeting = greeting + "friend.";
```

replaces the reference to "Hello" with a reference to the different `String` object "Hello friend." The object "Hello" is unchanged. To see that this is true, consider the following code:

```
String greeting = "Hello";
String helloVariable = greeting;
greeting = greeting + "friend.";
System.out.println(helloVariable);
```

This produces the output "Hello". If the object "Hello" had been changed, the output would have been "Hello friend."

### immutable

A class that contains no methods (other than constructors) that change any of the data in an object of the class is called an **immutable class**, and objects of the class are called **immutable objects**. The class `String` is an immutable class. It is perfectly safe to return a reference to an immutable object, because the object cannot be changed in any undesirable way; in fact, it cannot be changed in any way whatsoever.

### mutable

A class that contains public mutator methods or other public methods, such as input methods, that can change the data in an object of the class is called a **mutable class**, and objects of the class are called **mutable objects**. The class `Date` is an example of a mutable class; many, perhaps most, of the classes you define will be mutable classes. As we noted in the Pitfall entitled "Privacy Leaks" (but using other words): You should never write a method that returns a mutable object, but should instead use a copy constructor (or other means) to return a reference to a completely independent copy of the mutable object.



## TIP: Deep Copy versus Shallow Copy

In the previous two subsections, we contrasted the following two ways of defining the method `getBirthDate` (Display 5.19):

```
public Date getBirthDate()
{
 return new Date(born);
}
public Date getBirthDate() //Unsafe
{
 return born; //Not good
}
```



VideoNote  
Deep Copy  
vs. Shallow  
Copy Example

As we noted, the first definition is the better one (and the one used in Display 5.19). The first definition returns what is known as a *deep copy* of the object `born`. The second definition returns what is known as a *shallow copy* of the object `born`.

### deep copy

A **deep copy** of an object is a copy that, with one exception, has no references in common with the original. The one exception is that references to immutable objects are allowed to be shared (because immutable objects cannot change in any way and so cannot be changed in any undesirable way). For example, the first definition of `getBirthDate` returns a deep copy of the date stored by the instance variable `born`. So, if you change the object returned by `getBirthDate`, this does not change the `Date` object named by the instance variable `born`. The reason is that we defined the copy constructor for the class `Date` to create a deep copy (Display 5.20). Normally, copy constructors and accessor methods should return a deep copy.

### shallow copy

Any copy that is not a deep copy is called a **shallow copy**. For example, the second definition of `getBirthDate` returns a shallow copy of the date stored by the instance variable `born`.

We will have more to say about deep and shallow copies in later chapters. ■

## Never Return a Reference to a Mutable Private Object

A class that contains mutator methods or other methods, such as input methods, that can change the data in an object of the class is called a **mutable class**, and objects of the class are called **mutable objects**. When defining accessor methods (or almost any methods), your method should not return a reference to a mutable object. Instead, use a copy constructor (or other means) to return a reference to a completely independent copy of the mutable object.



**TIP: Assume Your Coworkers Are Malicious**

Our discussion of privacy leaks in the previous subsections was concerned with the effect of somebody trying to defeat the privacy of an instance variable. You might object that your coworkers are nice people and would not knowingly sabotage your software. That is probably true, and we do not mean to accuse your coworkers of malicious intent. However, the same action that can be performed intentionally by a malicious enemy can also be performed inadvertently by your friends or even by you yourself. The best way to guard against such honest mistakes is to pretend that you are defending against a malicious enemy. ■

**Self-Test Exercises**

41. Complete the definition of the method `set` for the class `Person` (Display 5.19).
42. Classify each of the following classes as either mutable or immutable: `Date` (Display 4.11), `Person` (Display 5.19), and `String`.
43. Normally, it is dangerous to return a reference to a private instance variable of class type, but it is OK if the class type is `String`. What is special about the class `String` that makes this true?

## 5.4 Packages and `javadoc`

*... he furnished me,  
From mine own library with volumes that  
I prize above my dukedom.*

WILLIAM SHAKESPEARE, *The Tempest*, 1611.

In this section, we cover packages, which are Java libraries, and then cover the `javadoc` program, which automatically extracts documentation from packages and classes. Although these are important topics, they are not used in the rest of this book. You can study this section at any time you wish; you need not cover this section before studying any other topic in this book.

This section does not use any of the material in Section 5.3, and so can be covered before Section 5.3.

This section assumes that you know about directories (which are called folders in some operating systems), that you know about path names for directories (folders), and that you know about `PATH` (environment) variables. These are not Java topics. They are part of your operating system, and the details depend on your particular

operating system. If you can find out how to set the PATH variable on your operating system, you will know enough about these topics to understand this section.

## Packages and `import` Statements

### package

A **package** is Java's way of forming a library of classes. You can make a package from a group of classes and then use the package of classes in any other class or program you write without the need to move the classes to the directory (folder) in which you are working. All you need to do is include an **import statement** that names the package. We have already used `import` statements with some predefined standard Java packages. For example, the following, which we have used before, makes available the class `Scanner` of the package `java.util`:

```
import java.util.Scanner;
```

You can make all the classes in the package available by using the following instead:

```
Import java.util.*;
```

There is no overhead cost for importing the entire package as opposed to just a few classes.

The `import` statements should be at the beginning of the file. Only blank lines, comments, and `package` statements may precede the list of `import` statements. We discuss `package` statements next.

### **import Statement**

You can use a class from a package in any program or class definition by placing an **import statement** that names the package and the class from the package at the start of the file containing the program (or class definition). The program (or class definition) need not be in the same directory as the classes in the package.

#### **SYNTAX**

```
import Package_Name.Class;
```

#### **EXAMPLE**

```
import java.util.Scanner;
```

You can import all the classes in a package by using an asterisk in place of the class's name.

#### **SYNTAX**

```
import Package_Name.*;
```

#### **EXAMPLE**

```
import java.util.*;
```

To make a package, group all the classes together into a single directory (folder) and add the following package statement to the beginning of each class file:

```
package Package_Name;
```

This package statement should be at the beginning of the file. Only blank lines and comments may precede the package statement. If there are both `import` statements and package statements, any package statements come before the `import` statements. Aside from the addition of the package statement, class files are just as we have already described them. (It is technically only the `.class` files that must be in the package directory.)

## The Package `java.lang`

The package `java.lang` contains classes that are fundamental to Java programming. These classes are so basic that the package is always imported automatically. Any class in `java.lang` does not need an `import` statement to make it available to your code. For example, the classes `Math` and `String` and the wrapper classes introduced earlier in this chapter are all in the package `java.lang`.

### Package

A **package** is a collection of classes that have been grouped together into a directory and given a package name. The classes in the package are each placed in a separate file, and the file is given the same name as the class, just as we have been doing all along. Each file in the package must have the following at the beginning of the file. Only comments and blank lines may precede this package statement.

#### SYNTAX

```
package Package_Name;
```

#### EXAMPLES

```
package utilities.numericstuff;
package java.util;
```

## Package Names and Directories

A package name is not an arbitrary identifier. It is a form of path name to the directory containing the classes in the package.

In order to find the directory for a package, Java needs two things: the name of the package and the value of your `CLASSPATH` variable.

You should already be familiar with the environment variable of your operating system that is known as the `PATH` variable. The **CLASSPATH** variable is a similar environment variable used to help locate Java packages. The value of your `CLASSPATH`

variable tells Java where to begin its search for a package. It is not a Java variable. It is an environment variable that is part of your operating system. The value of your CLASSPATH variable is a list of directories. The exact syntax for this list varies from one operating system to another, but it should be the same syntax as that used for the (ordinary) PATH variable. When Java is looking for a package, it begins its search in the directories listed in the CLASSPATH variable.

The name of a package specifies the relative path name for the directory that contains the package classes. It is a relative path name because it assumes that you start in one of the directories listed in the value of your CLASSPATH variable. For example, suppose the following is a directory listed in your CLASSPATH variable (your operating system might use / instead of \):

```
\libraries\newlibraries
```

And suppose your package classes are in the directory

```
\libraries\newlibraries\utilities\numericstuff
```

In this case, the package *should* be named

```
utilities.numericstuff
```

and all the classes in the file must start with the package statement

```
package utilities.numericstuff;
```

The dot in the package name means essentially the same thing as the \ or /, whichever symbol your operating system uses for directory paths. The package name tells you (and Java) what subdirectories to go through to find the package classes, starting from a directory on the class path. This is depicted in Display 5.22. (If there happen to be two directories in the CLASSPATH variable that can be used, then of all the ones that can be used, Java always uses the first one listed in the CLASSPATH variable.)

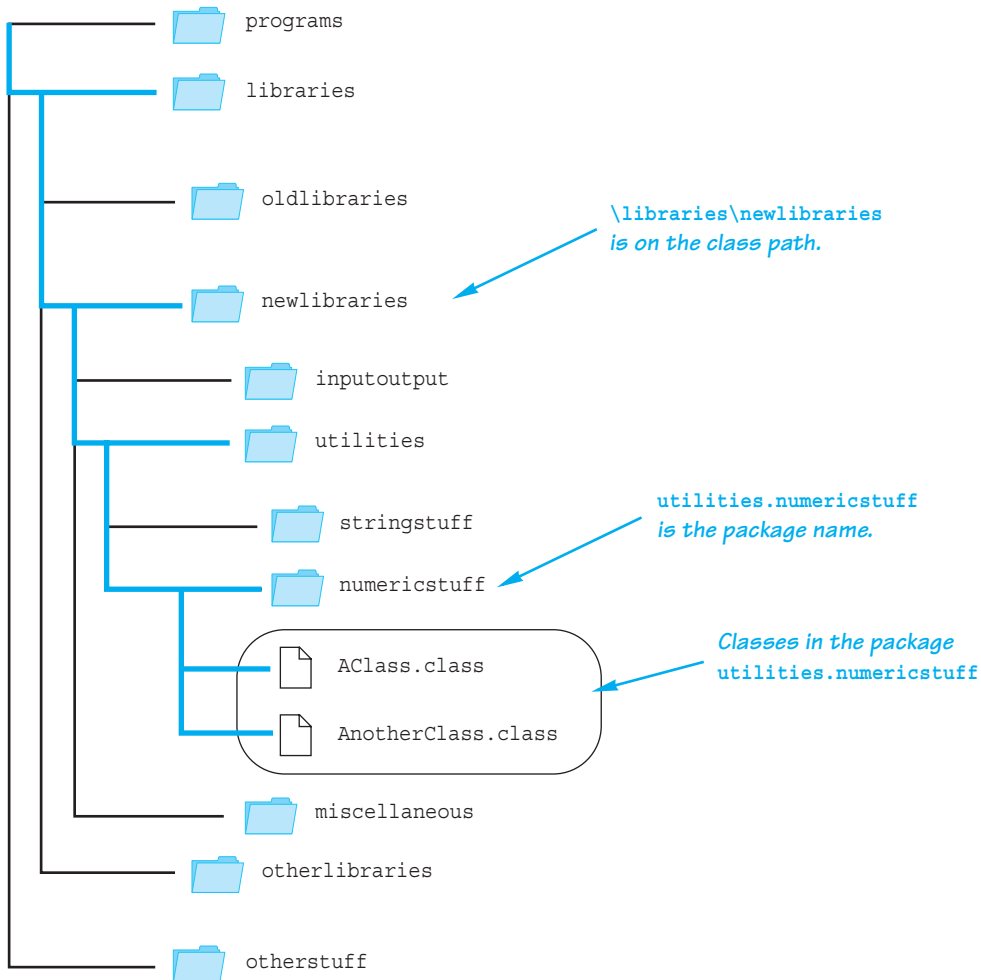
Any class that uses the class in this utilities.numericstuff package must contain either the import statement

```
import utilities.numericstuff.*;
```

or an import statement for each class in the package that is used.

The way you set the value of your CLASSPATH variable depends on your operating system, but we can give you some suggestions that may work. The CLASSPATH variable is usually spelled as one word with all uppercase letters, as in CLASSPATH. You will probably have a plain old PATH variable that tells the operating system where to find the code for commands such as javac and other commands that you can give as single-line commands. If you can find out how to set the PATH variable, you should be able to set the CLASSPATH variable in the same way.

Display 5.22 A Package Name



If you are on a UNIX system, you are likely to be able to set the CLASSPATH with some command similar to the following:

```
set CLASSPATH=/libraries/newlibraries;/otherstuff/specialjava;.
export CLASSPATH
```

If this does not work, you might try omitting the word `set` or replacing `set` with `setenv`. You might also try placing the list of directories in quotes. There are many versions of UNIX, all with their own minor variations. You may need to consult a local expert or check the documentation for your operating system.

If you are using a Windows machine, you can set the CLASSPATH variable by setting or creating an environment variable named CLASSPATH using the Control Panel.

In this book, we are assuming that class path directories are separated by a semi-colon. That is common, but some operating systems use some other separator, such as a colon. Check your documentation if a semicolon does not work as a separator.

### Package Names and the CLASSPATH Variable

A package name must be a path name for the directory that contains the classes in the package, but the package name uses dots in place of \ or / (whichever your operating system uses). When naming the package, use a relative path name that starts from any directory listed in the value of the CLASSPATH (environment) variable.

#### EXAMPLES

```
utilities.numericstuff
java.util
```



### PITFALL: Subdirectories Are Not Automatically Imported

Suppose you have two packages, `utilities.numericstuff` and `utilities.numericstuff.statistical`. In this case, you know that `utilities.numericstuff.statistical` is in a subdirectory (subfolder) of the directory (folder) containing `utilities.numericstuff`. This leads some programmers to assume that the following import statement imports both packages:

```
import utilities.numericstuff.*;
```

This is not true. When you import an entire package, you do not import subdirectory packages.

To import all classes in both of these packages, you need

```
import utilities.numericstuff.*;
import utilities.numericstuff.statistical.*;
```



### The Default Package

All the classes in your current directory (that do not belong to some other package) belong to an unnamed package called the **default package**. As long as the current directory is on your CLASSPATH, all the classes in the default package are automatically available to your code. This is why we always assume that all the classes we defined are in the same directory. That way, we need not clutter our discussion with concern about import statements.

default  
package



current  
directory

### PITFALL: Not Including the Current Directory in Your Class Path

Your CLASSPATH variable allows you to list more than one directory. Most operating systems use the dot to indicate the current directory. The **current directory** is not any one specific directory; it is the directory in which you are currently “located.” If you do not know what your current directory is, then it is probably the directory that contains the class you are writing. For example, the following value for a CLASSPATH variable lists two ordinary directories and the current directory:

```
\libraries\newlibraries;\otherstuff\specialjava;.
```

Whenever you set or change the CLASSPATH variable, be sure to include the current directory as one of the alternatives. With the above displayed CLASSPATH value, if the package is not found by starting in either of the previous two directories, Java will look in the subdirectories of the current directory. If you want Java to check the current directory before the other directories on the CLASSPATH variable, list the current directory (the dot) first, as follows:

```
.;\libraries\newlibraries;\otherstuff\specialjava
```

When looking for a package, Java tries the directories in the class path in order and uses the first one that works.

Omitting the current directory from the CLASSPATH variable can interfere with running Java programs, regardless of whether or not the programs use packages. If the current directory is omitted, then Java may not even be able to find the `.class` file for the program itself, so you may not be able to run any programs at all. Thus, if you do set the CLASSPATH variable, it is critical that you include the current directory in the CLASSPATH. No such problems will occur if you have not set the CLASSPATH variable at all; it arises only if you decide to set the CLASSPATH variable.

If you are having problems setting the CLASSPATH variable, one interim solution is to delete the CLASSPATH variable completely and to keep all the class files for one program in the same directory. This will allow you to still do some work while you seek advice on setting the CLASSPATH variable. ■

## Specifying a Class Path When You Compile ★

You can specify a class path when you compile a class. To do so, add `-classpath` followed by the class path as illustrated in the following:

```
javac -classpath .;C:\lib\numeric;C:\otherstuff YourClass.java
```

In a UNIX environment, replace the semicolons with colons. This will compile `YourClass.java`, overriding any CLASSPATH setting, and use the class path given after `-classpath`. Note that the directories are separated by semicolons. If you want

classes in the current directory to be available to your class, then be sure the class path includes the current directory, which is indicated by a dot.

When you run the class compiled as just shown, you should again use the `-classpath` option as follows:

```
java -classpath .;C:\libraries\numeric;C:\otherstuff YourClass
```

It is important to include the current directory on the class path when you run the program. If your program is in the default package, it will not be found unless you include the current directory. It is best to get in the habit of always including the current directory in all class paths.

Because the class path specified in compiling and running your classes is input to a program (`javac` or `java`) that is part of the Java environment and is not a command to the operating system, you can use either `/` or `\` in the class path, no matter which of these two your operating system uses.

## Name Clashes ★

### name clash

In addition to being a way of organizing libraries, packages also provide a way to deal with *name clashes*. A **name clash** is a situation in which two classes have the same name. If different programmers writing different packages used the same name for a class, the ambiguity can be resolved by using the package name.

Suppose a package named `sallyspack` contains a class called `HighClass`, and another package named `joespack` also contains a class named `HighClass`. You can use both classes named `HighClass` in the same program by using the more complete names `sallyspack.HighClass` and `joespack.HighClass`. For example,

```
sallyspack.HighClass object1 = new sallyspack.HighClass();
joespack.HighClass object2 = new joespack.HighClass();
```

These names that include the package name, such as `sallyspack.HighClass` and `joespack.HighClass`, are called **fully qualified class names**.

If you use fully qualified class names, you do not need to import the class, since this longer class name includes the package name.

### fully qualified class name

## Self-Test Exercises

44. Suppose you want to use the class `CoolClass` in the package `mypackages.library1` in a program you write. What do you need to do to make this class available to your program? What do you need to do to make all the classes in the package available to your program?
45. What do you need to do to make a class a member of the package named `mypackages.library1`?
46. Can a package have any name you want, or are there restrictions on what you can use for a package name? Explain any restrictions.



### Introduction to javadoc ★

The principles of encapsulation using information hiding say that you should separate the *interface* of a class (the instructions on how to use the class) from the *implementation* (the detailed code that tells the computer how the class does its work). In some other programming languages, such as C++, you normally define a class in two files. One file contains something like the interface or API that tells a programmer all that he or she needs to know to use the class. The other file contains the implementation details that are needed for the class code to run. This system is an obvious way to separate interface from implementation, but it is not what Java does.

javadoc

Java does not divide a class definition into two files. Instead, Java has the interface and implementation of a class mixed together into a single file. If this were the end of the story, Java would not do a good job of encapsulation using information hiding. However, Java has a very good way of separating the interface from the implementation of a class. Java comes with a program named `javadoc` that automatically extracts the interface from a class definition. If your class definition is correctly commented, a programmer using your class need only look at this API (documentation) produced by `javadoc`. The documentation produced by `javadoc` is identical in format to the documentation for the standard Java library classes.

The result of running `javadoc` on a class is to produce an HTML file with the API (interface) documentation for the class. *HTML* is the basic language used to produce documents to view with a Web browser, so the documentation produced by `javadoc` is viewed on a Web browser. A brief introduction to HTML is given in Chapter 20. However, you need not know any HTML to run `javadoc` or to read the documentation it produces.

`javadoc` can be used to obtain documentation for a single class. However, it is primarily intended to obtain documentation for an entire package.

We will first discuss how you should comment your classes so that you can get the most value out of `javadoc`. We will then describe how you run the `javadoc` program.

### Commenting Classes for javadoc ★

To get a more useful `javadoc` document, you must write your comments in a particular way. All the classes in this book have been commented for use with `javadoc`. However, to save space, the comments in this book are briefer than what would be ideal for `javadoc`.

The `javadoc` program extracts the heading for your class (or classes) as well as the headings for all public methods, public instance variables, public static variables, and certain comments. No method bodies and no private items are extracted when `javadoc` is run in the normal default mode.

For `javadoc` (in default mode) to extract a comment, the comment must satisfy two conditions:

1. The comment must *immediately precede* a public class definition, a public method definition, or other public item.
2. The comment must be a block comment (that is, the `/*` and `*/` style of comment), and the opening `/*` must contain an extra `*`. So the comment must be marked by `/**` at the beginning and `*/` at the end.

Unless you explicitly set an extra option to `javadoc`, line comments (that is, `//` style comments) are not extracted, and comments preceding any private items also are not extracted.

The comment that precedes a public method definition can include any general information that you would like. There is also special syntax for inserting descriptions of parameters, any value returned, and any exceptions that might be thrown. We have not yet discussed exceptions. That is done in Chapter 9, but we include mention of them here, so this section will serve as a more complete reference on `javadoc`. You need not worry about exceptions or the details about “throws” discussed here until you reach Chapter 9.

#### @ tag

The special information about parameters and so forth are preceded by the `@` symbol and are called **@ tags**. The following is an example of a method comment for use with `javadoc`:

```
/**
 * Tests for equality of two objects of type Person. To be equal,
 * the two objects must have the same name, same birth date, and
 * same death date.
 *
 * @param otherPerson The person being compared to the calling
 * object.
 * @return Returns true if the calling object equals otherPerson.
 */
public boolean equals(Person otherPerson)
```

(The method `equals` is from the class `Person` defined in Display 5.19. If you need more context, look at that display.)

Note that the `@` tags all come after any general comment and that each `@` tag is on a line by itself. The following are some of the `@` tags allowed:

```
@param Parameter_Name Parameter_Description
@return Description_Of_Value_Returned
@throws Exception_Type Explanation
@deprecated
@see Package_Name.Class_Name
@author Author
@version Version_Information
```

The `@` tags should appear in the above order—first `@param`, then `@return`, then `@throws`, and so forth. If there are multiple parameters, they should each have their

own `@param` and appear on a separate line. The parameters and their `@param` description should be listed in their left-to-right order in the parameter list. If there are multiple authors, they should each have their own `@author` and appear on a separate line. The author and version information are not extracted unless suitable option flags have been set, as described in the next subsection.

### deprecated

If `@deprecated` is included in a method comment, then the documentation notes that the method is *deprecated*. A **deprecated** method is one that is being phased out. To allow for backward compatibility, the method still works, but it should not be used in new code.

If an `@` tag is included for an item, `javadoc` extracts the explanation for that item and includes it in the documentation. You should always include a more or less complete set of `@` tags in the comment for each of your methods. In this book, we omit the `@` tags to save space, but we encourage you to always include them. The comments that are not part of an `@` tag appear as a general comment for the method, along with the method heading.

You can also insert HTML commands in your comments so that you gain more control over `javadoc`, but that is not necessary and may not even be desirable. HTML commands can clutter the comments, making them harder to read when you look at the source code.

## Running javadoc ★

To run `javadoc` on a package, all you need to do is give the following command:

```
javadoc -d Documentation_Directory Package_Name
```

It would be normal to run this command from the directory containing the classes in the package, but it can be run from any directory, provided you have correctly set the `CLASSPATH` environment variable. The *Documentation\_Directory* is the name of the directory in which you want `javadoc` to place the HTML documents that it produces. For example, the following might be used to obtain documentation on the package `mylibraries.numericstuff`:

```
javadoc -d documentation/mypackages mylibraries.numericstuff
```

The HTML documents produced will be in the subdirectory `documentation/mypackages` of where this command is run. If you prefer, you may use a complete path name in place of the relative path name `documentation/mypackages`. If you omit the `-d` and *Documentation\_Directory*, `javadoc` will create suitable directories for the documentation.

You can link to standard Java documents so that your HTML documents include live links to standard classes and methods. The syntax is as follows:

```
javadoc -link Link_To_Standard_Docs -d Documentation_Directory
Package_Name
```

*Link\_To\_Standard\_Docs* is either a path to a local version of the Java documentation or the URL of the Oracle Web site with standard Java documentation. As of this writing, that URL is

```
http://download.oracle.com/javase/8/docs/api/
```

You need not run `javadoc` on an entire package. You can run `javadoc` on a single class file. For example, the following should be run from the directory containing `Date.java` and will produce documentation for the class `Date` :

```
javadoc Date.java
```

You can run `javadoc` on all classes in a directory with

```
javadoc *.java
```

You can add the `-d` and/or `-link` options to any of these commands. For example,

```
javadoc -link http://download.oracle.com/javase/8/docs/api/ -d
mydocs *.java
```

These and other options for `javadoc` are summarized in Display 5.23.

When running `javadoc`, you typically get more directories and many more HTML files than you might expect. To get a better understanding of `javadoc`, you should try running it in various settings and observe the files it produces.

Display 5.23 Options for `java.doc`

| OPTIONS                 | DESCRIPTION                                                                                                                                                                                                                       |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>-link</code>      | Provides a link to another set of documentation. Normally, this is used with either a path name to a local version of the Java documentation or the URL of the Oracle Web site with standard Java documentation.                  |
| <code>-d</code>         | Specifies a directory to hold the documentation generated. <i>Documentation_Directory</i> may be a relative or absolute path name.                                                                                                |
| <code>-author</code>    | Includes author information (from <code>@author</code> tags). This information is omitted unless this option is set.                                                                                                              |
| <code>-version</code>   | Includes version information (from <code>@version</code> tags). This information is omitted unless this option is set.                                                                                                            |
| <code>-classpath</code> | Overrides the <code>CLASSPATH</code> environment variable and makes the <code>CLASSPATH</code> for the execution of this invocation of <code>javadoc</code> . It does not permanently change the <code>CLASSPATH</code> variable. |
| <code>-private</code>   | Includes private members as well as public members in the documentation.                                                                                                                                                          |

### Self-Test Exercises

47. When run in default mode, does `javadoc` ever extract the body of a method definition? When run in default mode, does `javadoc` ever extract anything marked `private` in a class definition?
48. When run in default mode, what comments does `javadoc` extract and what comments does it not extract?

### Chapter Summary

- A *static method* is one that does not require a calling object but can use the class name in place of the calling object.
- A *static variable* is similar to an instance variable except that there is only one copy of the static variable that is used by all objects of the class.
- A *wrapper class* allows you to have a class object that corresponds to a value of a primitive type. Wrapper classes also contain a number of useful predefined constants and static methods.
- A variable of a class type stores the reference (memory address) of where the object is located in the computer's memory. This causes some operations, such as `=` and `==`, to behave quite differently for variables of a class type than they do for variables of a primitive type.
- When you use the assignment operator with two variables of a class type, the two variables become two names for the same object.
- A method cannot change the value of a variable of a primitive type that is an argument to the method. On the other hand, a method can change the values of the instance variables of an argument of a class type. This is because with class parameters, it is a reference that is plugged in to the parameter.
- `null` is a special constant that can be used to give a value to any variable of any class type.
- An expression with a `new` operator and a constructor can be used as an argument to a method. Such an argument is called an *anonymous object*.
- A *copy constructor* is a constructor with one parameter of the same type as the class. A copy constructor should be designed so the object it creates is intuitively an exact copy of its parameter, but is a completely independent copy—that is, a *deep copy*.
- A class that contains mutator methods or any methods that can change the data in an object of the class is called a *mutable class*, and objects of the class are called *mutable objects*. When defining accessor methods (or almost any methods), your method should not return a reference to a mutable object. Instead, use a copy constructor (or other means) to return a reference to a deep copy of the mutable object.

- *Packages* are Java's version of class libraries.
- Java comes with a program named `javadoc` that automatically extracts the interface from all the classes in a package or from a single class definition.

## Answers to Self-Test Exercises

1. Yes, it is legal, although it would be preferable style to use the class name `RoundStuff` in place of `roundObject`.
2. No, all methods in the class are static, so there is no need to create objects. If we follow our style rules, no constructors would ever be used, so there is no need for constructors.
3. Yes, a class can contain both static and nonstatic (that is, regular) methods.
4. You cannot invoke a nonstatic method within a static method (unless you create an object to serve as the calling object of the nonstatic method).
5. You can invoke a static method within a nonstatic method.
6. You cannot reference an instance variable within a static method, because a static method can be used without a calling object and hence without any instance variables.
7. Each object of a class has its own copy of each instance variable, but a single copy of each static variable is shared by all objects.
8. No, you cannot use an instance variable (without an object name and dot) in the definition of a static method of the same class. Yes, you can use an instance variable (without an object name and dot) in the definition of a nonstatic method of the same class.
9. Yes, you can use a static variable in the definition of a static method of the same class. Yes, you can use a static variable in the definition of a nonstatic method of the same class. So, you can use a static variable in any method.
10. No, you cannot use either an explicit or an implicit occurrence of the `this` parameter in the definition of a static method.
11. All methods with the following names could and should be marked `static`: `dateOK`, `monthOK`, and `monthString`.
12. All static variables should be marked `private` with the exception of one case: If the static variable is used as a named constant (that is, if it is marked `final`), then it can be marked either `public` or `private` depending on the particular situation.
13. They can both be named by using the class name and a dot, rather than an object name and a dot.

14. 3, 4,  
3.0, 3.0,  
4.0, and 4.0
15. `roundedAnswer = (int) Math.round(answer);`
16. `long`. Because one argument is of type `long`, the `int` argument is automatically type cast to `long`.
17. They are all legal.
18. An object of the class `Double`; the assignment is equivalent to  
`Double price = new Double(1.99);`
19. A value of the primitive type `int`; the assignment is equivalent to  
`int count = (new Integer(12)).intValue();`
20. Objects are Not equal.
21. `Double.toString(result)`
22. `Double.parseDouble(stringForm)`
23. `Double.parseDouble(stringForm.trim())`
24. `System.out.println("Largest long is" + Long.MAX_VALUE +  
"Smallest long is" + Long.MIN_VALUE);`
25. `Character zeeObject = new Character('Z');`
26. No, none of the wrapper classes discussed in this chapter have no-argument constructors.
27. A *reference type* is a type whose variables contain references, that is, memory addresses. Class types are reference types. Primitive types are not reference types.
28. When comparing two objects of a class type, you should use the method `equals`.
29. When comparing two objects of a primitive type, you should you use `==`.
30. Yes, a method with an argument of a class type can change the values of the instance variables in the object named by the argument.
31. No, a method cannot change the value of an `int` variable given as an argument to the method.
32. The variable `anObject` names no object, so the invocation of the `set` method is an error. One way to fix things is as follows:  
`ToyClass anObject = new ToyClass();  
anObject.set("Josephine", 42);`
33. The constant `null` can be assigned to a variable of any class type. It does not really have a type, but you can think of its type as being the type of a memory address. You can also think of `null` as being of every class type.

extra code  
on website

34. `aVariable == null`
35. It is unlikely, but it is legal. This is an example of an anonymous object, as described in the text.
36. The only difference is that the object of type `Date` is given the name `theDate` in the second version. It makes no difference to the object `adams`.
37. The following definition of `equals` is used in the file `Date.java` in the Chapter 5 directory of the source code on the website:

```
public boolean equals(Date otherDate)
{
 if (otherDate == null)
 return false;
 else
 return ((month.equals(otherDate.month)) &&
 (day == otherDate.day) && (year == otherDate.year));
}
```

38. A copy constructor is a constructor with one parameter of the same type as the class. A copy constructor should be designed so that the object it creates is intuitively an exact copy of its parameter, but a completely independent copy, that is, a deep copy.
39. The first piece of code produces the output

July 4, 1776

The second piece of code produces the output

January 1, 2006

40. Natalie Dressed, April 1, 1984-
41. 

```
public void set(String newName, Date birthDate, Date deathDate)
{
 if (consistent(birthDate, deathDate))
 {
 name = newName;
 born = new Date(birthDate);
 if (deathDate == null)
 died = null;
 else
 died = new Date(deathDate);
 }
 else
 {
 System.out.println("Inconsistent dates. Aborting.");
 System.exit(0);
 }
}
```



Note that the following is not a good definition because it could lead to a privacy leak:

```
public void set(String newName, Date birthDate, Date deathDate)
{ //Not good
 name = newName;
 born = birthDate;
 died = deathDate;
}
```

- 42. The class `String` is an immutable class. The classes `Date` and `Person` are mutable classes.
- 43. The class `String` is an immutable class.
- 44. To make the class available to your program, you need to insert the following at the start of the file containing your class:

```
import mypackages.library1.CoolClass;
```

To make all the classes in the package available to your program, insert the following instead:

```
import mypackages.library1.*;
```

- 45. To make a class a member of the package named `mypackages.library1`, you must insert the following at the start of the file with the class definition and place the file in the directory corresponding to the package (as described in the text):

```
package mypackages.library1;
```

(Only the `.class` file is required to be in the directory corresponding to the package, but it may be cleaner and easier to place both the `.java` file and the `.class` file there.)

- 46. A package name must be a path name for the directory that contains the classes in the package, but the package name uses dots in place of `\` or `/` (whichever your operating system uses). When naming the package, you use a relative path name that starts from any directory named in the value of the `CLASSPATH` (environment) variable.
- 47. `javadoc` never extracts the body of a method definition, nor (when run in default mode) does `javadoc` ever extract anything marked `private` in a class definition.
- 48. When run in default mode, `javadoc` extracts only comments that satisfy the following two conditions:
  - 1. The comment must immediately precede a public class definition, a public method definition, or other public item.
  - 2. The comment must use the `/*` and `*/` style, and the opening `/*` must contain an extra `*`. So the comment must be marked by `/**` at the beginning and `*/` at the end. In particular, `javadoc` does not extract any `//` style comments.

## Programming Projects

1. Define a class called `BookStore` to maintain the record of books sold. The store contains three categories of books i.e. “Kids”, “Engineering”, and “Story”. The following details should be maintained for each book.
  - a. Book category
  - b. Author
  - c. Title
  - d. Publisher
  - e. Selling price of the book
  - f. Quantity

Create a constructor that initializes the `Store ID` and the rest of the details of the book mentioned above. Include a method named `trackSalesStatus` that will display the total number of books sold by the store (add a static variable that tracks the total number of books sold). Also, include a method to display the quantity available corresponding to each Book ID. Write a main method to test your class.

2. Define a class called `Fraction`. This class is used to represent a ratio of two integers. Include mutator methods that allow the user to set the numerator and the denominator. Also include a method that displays the fraction on the screen as a ratio (e.g., 5/9). This method does not need to reduce the fraction to lowest terms. Include an additional method, `equals`, that takes as input another `Fraction` and returns `true` if the two fractions are identical and `false` if they are not. This method should treat the fractions reduced to lowest terms; that is, if one fraction is 20/60 and the other is 1/3, then the method should return `true`.

Embed your class in a test program that allows the user to create a fraction. Then the program should loop repeatedly until the user decides to quit. Inside the body of the loop, the program should allow the user to enter a target fraction into an anonymous object and learn whether the fractions are identical.

3. Create a class to represent a container. The class `Container` should have the following properties.
  - a. Maximum capacity of the container in liters.
  - b. Quantity of liquid at any given time in liters.

The following operations can be performed on the containers:

- a. Completely fill a container.
- b. Completely empty a container.
- c. Transfer liquid from one container to another.

Define the class named `Container` that implements the properties and operations defined above. Create a constructor of the `Container` class that allows the user to specify the maximum capacity of the container in liters. Initially, assume that all the containers are empty.

Also, implement the following methods in the `Container` class.

- a. `Quantity` to return the current quantity of liquid at any given time in liters.
- b. `Leftover` to return the quantity of liquid that can be filled in the current container before it is full.
- c. `Full` to fill the current container fully.
- d. `Empty` to make the container completely empty.
- e. `Transfer` to transfer a certain amount of liquid from one container to another.
- f. `displayQuantity` to display the current quantity in liters/ milliliters/ kiloliters.

Note: While transferring liquid from one container to another, check the maximum capacity of the container.

Also, include other supporting variables or methods wherever necessary.

4. You are interested in keeping track of the team members and competition information for your school's annual entries in computer science programming competitions. Each team consists of exactly four team members. Every year your team competes in two competitions. As an initial start for your database, create a class named `Team` that has the following instance variables:

```
// Name for the team
String teamName;
// Names for each team members.
String name1, name2, name3, name4;
// Info on each competition
Competition competition1, competition2;
```

Note that there is a much better way to represent the team members and competitions using arrays; this is covered in a subsequent chapter. The class should also have a method that outputs the names of all team members and the competition information to the console.

The `Competition` class contains variables to track the following:

String: Name of the competition, Name of the winning team, Name of the runner-up

Integer: Year of the competition

Implement the `Team` and `Competition` classes with appropriate constructor, accessor, and mutator methods. In entering data for past competitions, you note that an entry is usually very similar to the previous year's entry. To help with the data entry, create a deep copy constructor for the `Team` class. Test your copy constructor by creating a copy of an existing team object, changing the competition information for the copy, and outputting the data for the original and the copy. The original object should be unchanged if your deep copy constructor is working properly.

5. Part One: Define a class named `Money` whose objects represent amounts of U.S. money. The class should have two instance variables of type `int` for the dollars and cents in the amount of money. Include a constructor with two parameters of type `int` for the dollars and cents, one with one constructor of type `int` for an amount of dollars with zero cents and a no-argument constructor. Include the methods `add` and `minus` for addition and subtraction of amounts of money. These methods should be static methods, should each have two parameters of type `Money`, and return a value of type `Money`. Include a reasonable set of accessor and mutator methods as well as the methods `equals` and `toString`. Write a test program for your class.

Part Two: Add a second version of the methods for addition and subtraction. These methods should have the same names as the static version but should use a calling object and a single argument. For example, this version of the `add` method (for addition) has a calling object and one argument. So `m1.add(m2)` returns the result of adding the `Money` objects `m1` and `m2`. Note that your class should have all these methods; for example, there should be two methods named `add`.

Alternate Part Two: Add a second version of the methods for addition and subtraction. These methods should have the same names as the static version but should use a calling object and a single argument. The methods should be `void` methods. The result should be given as the changed value of the calling object. For example, this version of the `add` method (for addition) has a calling object and one argument. Therefore,

```
m1.add(m2);
```

changes the values of the instance variables of `m1` so they represent the result of adding `m2` to the original version of `m1`. Note that your class should have all these methods; for example, there should be two methods named `add`.

(If you want to do both Part Two and Alternate Part Two, they must be two classes. You cannot include the methods from both Part Two and Alternate Part Two in a single class. Do you know why?)

6. Part One: Define a class for rational numbers. A rational number is a number that can be represented as the quotient of two integers. For example,  $1/2$ ,  $3/4$ ,  $64/2$ , and so forth are all rational numbers. (By  $1/2$  and so forth, we mean the everyday meaning of the fraction, not the integer division this expression would produce in a Java program.) Represent rational numbers as two values of type `int`, one for the numerator and one for the denominator. Your class should have two instance variables of type `int`. Call the class `Rational`. Include a constructor with two arguments that can be used to set the instance variables of an object to any values. Also include a constructor that has only a single parameter of type `int`; call this single parameter `wholeNumber` and define the constructor so that the object will be initialized to the rational number `wholeNumber/1`. Also include a no-argument constructor that initializes an object to 0 (that is, to  $0/1$ ). Note that the numerator,

the denominator, or both may contain a minus sign. Define methods for addition, subtraction, multiplication, and division of objects of your class `Rational`. These methods should be static methods that each have two parameters of type `Rational` and return a value of type `Rational`. For example, `Rational.add(r1, r2)` will return the result of adding the two rational numbers (two objects of the class `Rational`, `r1` and `r2`). Define accessor and mutator methods as well as the methods `equals` and `toString`. You should include a method to normalize the sign of the rational number so that the denominator is positive and the numerator is either positive or negative. For example, after normalization,  $4/-8$  would be represented the same as  $-4/8$ . Also write a test program to test your class.

*Hints:* Two rational numbers  $a/b$  and  $c/d$  are equal if  $a*d$  equals  $c*b$ .

Part Two: Add a second version of the methods for addition, subtraction, multiplication, and division. These methods should have the same names as the static version but should use a calling object and a single argument. For example, this version of the `add` method (for addition) has a calling object and one argument. So `r1.add(r2)` returns the result of adding the rationals `r1` and `r2`. Note that your class should have all these methods; for example, there should be two methods named `add`.

Alternate Part Two: Add a second version of the methods for addition, subtraction, multiplication, and division. These methods should have the same names as the static version but should use a calling object and a single argument. The methods should be `void` methods. The result is given as the changed value of the calling object. For example, this version of the `add` method (for addition) has a calling object and one argument. Therefore,

```
r1.add(r2);
```

changes the values of the instance variables of `r1` so they represent the result of adding `r2` to the original version of `r1`. Note that your class should have all these methods; for example, there should be two methods named `add`.

(If you want to do both Part Two and Alternate Part Two, they must be two classes. You cannot include the methods from both Part Two and Alternate Part Two in a single class. Do you know why?)

7. Create a class to represent the phone billing system. For this purpose, you need to define two classes namely, `NationalCall` and `InternationalCall`.

Define the `NationalCall` with the following attributes. You can include your own attributes also, if required.

- a. Source phone number
- b. Destination phone number
- c. Total duration of current call in seconds
- d. Total duration of all call
- e. Current call price
- f. Total price of all calls

Include a constructor that constructs a `NationalCall` object from the parameters for:

- a. Source phone number
- b. Destination phone number
- c. Total duration of current call in seconds

The `InternationalCall` class is defined with the following additional attributes apart from the attributes of `NationalCall` class.

- a. Source phone number Country code
- b. Destination phone number Country code

The following table shows call rates as per the call durations.

| Call Duration                       | National Call Rates | International Call Rates |
|-------------------------------------|---------------------|--------------------------|
| <= First minute                     | 0.20 \$             | 0.60 \$                  |
| > First Minute and <= second minute | 0.15 \$             | 0.40 \$                  |
| > Second minute                     | 0.10 \$             | 0.20 \$                  |

Also, define a method as `DiscountCall` that will calculate the discount provided to the customer depending upon the duration of the call. The discount rates are different for national and international calls. The table below shows some of the sample values for the discount offer.

| Total duration of all calls | National Call Discounted on Call Price | International Call Discounted Number of Call Units |
|-----------------------------|----------------------------------------|----------------------------------------------------|
| > 10 and <= 20 minutes      | 5%                                     | 8%                                                 |
| > 20 minutes                | 7%                                     | 10%                                                |

Write a suitable constructor that will initialize all data members and construct the objects for the class.

Write a suitable method to display the details of `NationalCall` and `InternationalCall` class. Define the main method class to test and implement the code.

8. Programming Project 4.12 asked you to create a `PizzaOrder` class that stores an order consisting of up to three pizzas. Extend this class with the following methods and constructor:
  - `public int getNumPizzas()`—returns the number of pizzas in the order.
  - `public Pizza getPizza1()`—returns the first pizza in the order or null if `pizza1` is not set.
  - `public Pizza getPizza2()`—returns the second pizza in the order or null if `pizza2` is not set.
  - `public Pizza getPizza3()`—returns the third pizza in the order or null if `pizza3` is not set.

- A copy constructor that takes another `PizzaOrder` object and makes an independent copy of its pizzas. This might be useful if using an old order as a starting point for a new order.

Write a main method to test the new methods. Changing the pizzas in the new order should not change the pizzas in the original order. For example,

```
Pizza pizza1 = // Code to create a large pizza, 1 cheese, 1 ham
Pizza pizza2 = // Code to create a medium pizza, 2 cheese,
 // 2 pepperoni
PizzaOrder order1 = // Code to create an order
order1.setNumPizzas(2); // 2 pizzas in the order
order1.setPizza1(pizza1); // Set first pizza
order1.setPizza2(pizza2); // Set second pizza
double total = order1.calcTotal(); // Should be 18+20 = 38
PizzaOrder order2 = new PizzaOrder(order1); // Use copy
 // constructor
order2.getPizza1().setNumCheeseToppings(3); // Change toppings
double total = order2.calcTotal(); // Should be 22 + 20 = 42
double origTotal = order1.calcTotal(); // Should still be 38
```

Note that the first three lines of code are incomplete. You must complete them as part of the Programming Project.

9. Use `javadoc` to generate HTML documentation for the code in Display 5.19. Use the `@author` and `@version` tag for the description of the entire class. Add a comment for every public method or constructor using the `@param` and `@return` tags when appropriate.
10. First, complete Programming Project 4.13 from Chapter 4.

Modify the main method with a loop so that an arbitrary number of `BoxOfProduce` objects are created and substitutions are allowed for each box. Add a menu so the user can decide when to stop creating boxes.

You would like to throw in a free recipe flyer for salsa verde if the box contains tomatillos. However, there are only five recipe flyers. Add a static variable to the `BoxOfProduce` class that counts the number of recipe flyers remaining and initialize it to 5. Also add an instance variable that indicates whether or not the box contains a recipe flyer and modify the `toString()` method to also output “salsa verde recipe” if the box contains a recipe flyer. Finally, add logic inside the class so that if the box contains at least one order of tomatillos then it automatically gets a recipe flyer until all of the recipe flyers are gone. Note that a box should only get one recipe flyer even if there are multiple orders of tomatillos.

Test your class by creating boxes with tomatillos from your menu until all of the flyers are gone.

11. Do Programming Project 5 Part One and Programming Project 6 Part One. For this Programming Project, put your `Money` class into a package named `Finance`, and put your `Rational` class into a package named `MyMath`. Your main method should be in a separate package of your choice. Your test code for both classes should be in the main method.



VideoNote  
Solution to  
Programming  
Project 5.9



# Arrays 6

## 6.1 INTRODUCTION TO ARRAYS 378

Creating and Accessing Arrays 379  
The `length` Instance Variable 382  
Initializing Arrays 385

## 6.2 ARRAYS AND REFERENCES 388

Arrays Are Objects 388  
Array Parameters 390  
Arguments for the Method `main` ★ 397  
Methods That Return an Array 399

## 6.3 PROGRAMMING WITH ARRAYS 400

Partially Filled Arrays 401  
Example: A Class for Partially Filled Arrays 404  
The “for-each” Loop ★ 408  
Methods with a Variable Number  
of Parameters ★ 412  
Example: A String Processing Example ★ 415  
Privacy Leaks with Array Instance Variables 416  
Example: Sorting an Array 420  
Enumerated Types ★ 424

## 6.4 MULTIDIMENSIONAL ARRAYS 431

Multidimensional Array Basics 431  
Using the `length` Instance Variable 434  
Ragged Arrays ★ 435  
Multidimensional Array Parameters and Returned  
Values 435  
Example: A Grade Book Class 436



# 6 Arrays

*Memory is necessary for all the operations of reason.*

BLAISE PASCAL, *Pensées*, 1688.

## Introduction

An *array* is a data structure used to process a collection of data that is all of the same type, such as a list of numbers of type `double` or a list of strings. In this chapter, we introduce you to the basics of defining and using arrays in Java.

## Prerequisites

Section 6.1 requires understanding of only Chapters 1 through 3 and Section 4.1 of Chapter 4. Indeed, much less than all of Section 4.1 is needed. All you really need from Section 4.1 is to have some idea of what an object is and what an instance variable is.

The remaining sections require Chapters 1 through 5 with the exception that an understanding of Section 5.4 on packages and `javadoc` is not required.

## 6.1 Introduction to Arrays

*It is a capital mistake to theorize before one has data.*

SIR ARTHUR CONAN DOYLE, *Scandal in Bohemia*, 1891.

**array**

Suppose we wish to write a program that reads in five test scores and performs some manipulations on these scores. For instance, the program might compute the highest test score and then output the amount by which each score falls short of the highest. The highest score is not known until all five scores are read in. Hence, all five scores must be retained in storage so that after the highest score is computed, each score can be compared to it. To retain the five scores, we will need something equivalent to five variables of type `int`. We could use five individual variables of type `int`, but keeping track of five variables is hard, and we may later want to change our program to handle 100 scores; certainly, keeping track of 100 variables is impractical. An array is the perfect solution. An **array** behaves like a list of variables with a uniform naming mechanism that can be declared in a single line of simple code. For example, the names for the five individual variables we need might be `score[0]`, `score[1]`, `score[2]`, `score[3]`, and `score[4]`. The part that does not change—in this case, `score`—is the name of the array. The part that can change is the integer in the square brackets `[]`.

## Creating and Accessing Arrays

In Java, an array is a special kind of object, but it is often more useful to think of it as a collection of variables all of the same type. For example, an array that behaves like a collection of five variables of type `double` can be created as follows:

```
double[] score = new double[5];
```

This is like declaring the following to be five variables of type `double`:

```
score[0], score[1], score[2], score[3], score[4]
```

The individual variables that make up the array are referred to in a variety of different ways. We will call them **indexed variables**, though they are also sometimes called **subscripted variables** or **elements** of the array. The number in square brackets is called an **index** or a **subscript**. In Java, *indices are numbered starting with 0, not any number*. The number of indexed variables in an array is called the **length** or **size** of the array. When an array is created, the length of the array is given in square brackets after the array name. The indexed variables are then numbered (also using square brackets) starting with 0 and ending with the integer that is *one less than the length of the array*.

The following example:

```
double[] score = new double[5];
```

is really shorthand for the following two statements:

```
double[] score;
score = new double[5];
```

The first statement declares the variable `score` to be of the array type `double[]`. The second statement creates an array with five indexed variables of type `double` and makes the variable `score` a name for the array. You may use any expression that evaluates to a nonnegative `int` value in place of the 5 in square brackets. In particular, you can fill a variable with a value read from the keyboard and use the variable in place of the 5. In this way, the size of the array can be determined when the program is run.

An array can have indexed variables of any type, as long as they are all of the same type. This type is called the **base type** of the array. In our example, the base type of the array `score` is `double`. To declare an array with base type `int`, simply use the type name `int` instead of `double` when the array is declared and created. The base type of an array can be any type. In particular, it can be a class type.

Each of the five indexed variables of our example array `score` can be used just like any other variable of type `double`. For example, all of the following are allowed in Java:

```
score[3] = 32;
score[0] = score[3] + 10;
System.out.println(score[0]);
```

The five indexed variables of our sample array `score` are more than just five plain old variables of type `double`. That number in square brackets is part of the indexed variable's name. So, your program can compute the name of one of these variables.

indexed  
variable  
subscripted  
variable  
element  
index,  
subscript  
length, size

base type

## Declaring and Creating an Array

Declare an array name and create an array in almost the same way that you create and name objects of classes. There is only a slight difference in the syntax.

### SYNTAX

```
Base_Type[] Array_Name = new Base_Type[Length];
```

The *Length* may be given as any expression that evaluates to a nonnegative integer. In particular, *Length* can be an `int` variable.

### EXAMPLES

```
char[] line = new char[80];
double[] reading = new double[300];
Person[] specimen = new Person[100];
```

Person is a class.

Instead of writing an integer constant in the square brackets, you can use any expression that evaluates to an integer that is at least 0 and at most 4. So, the following is allowed:

```
System.out.println(score[index] + " is at position " + index);
```

where *index* is a variable of type `int` that has been given one of the values 0, 1, 2, 3, or 4.

When we refer to these indexed variables grouped together into one collective item, we will call them an *array*. So, we can refer to the array named *score* (without using any square brackets).

The program in Display 6.1 shows an example of using our sample array *score* as five indexed variables, all of type `double`.

Note that the program can compute the name of an indexed variable by using a variable as the index, as in the following `for` loop:

```
for (index = 0; index < 5; index++)
 System.out.println(score[index] + " differs from max by "
 + (max - score[index]));
```

**square  
brackets []**

Do not confuse the three ways to use the **square brackets** `[]` with an array name. First, the square brackets can be used to create a type name, such as the `double[]` in the following:

```
double[] score;
```

Second, the square brackets can be used with an integer value as part of the special syntax Java uses to create a new array, as in

```
score = new double[5];
```

## Display 6.1 An Array Used in a Program (part 1 of 2)

```

1 import java.util.Scanner;

2 public class ArrayOfScores
3 {
4 /**
5 Reads in 5 scores and shows how much each
6 score differs from the highest score.
7 */
8 public static void main(String[] args)
9 {
10 Scanner keyboard = new Scanner(System.in);
11 double[] score = new double[5];
12 int index;
13 double max;

14 System.out.println("Enter 5 scores:");
15 score[0] = keyboard.nextDouble();
16 max = score[0];
17 for (index = 1; index < 5; index++)
18 {
19 score[index] = keyboard.nextDouble();
20 if (score[index] > max)
21 max = score[index];
22 //max is the largest of the values score[0],..., score[index].
23 }

24 System.out.println("The highest score is " + max);
25 System.out.println("The scores are:");
26 for (index = 0; index < 5; index++)
27 System.out.println(score[index] + " differs from max by "
28 + (max - score[index]));
29 }
30 }

```

## Sample Dialogue

```

Enter 5 scores:
80 99.9 75 100 85.5
The highest score is 100
The scores are:
80.0 differs from max by 20
99.9 differs from max by 0.1
75.0 differs from max by 25
100.0 differs from max by 0.0
85.5 differs from max by 14.5

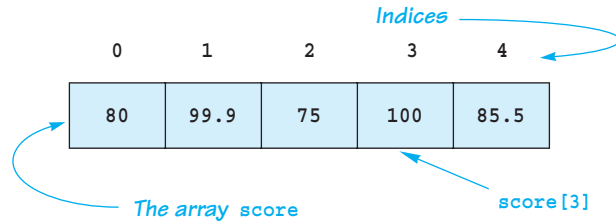
```

*Due to imprecision in floating-point arithmetic, this value probably will only be a close approximation to 0.1.*

(continued)

Display 6.1 An Array Used in a Program (part 2 of 2)

*A common way to visualize an array:*



The third use of square brackets is to name an indexed variable of the array, such as `score[0]` or `score[3]`, as illustrated by the following line:

```
max = score[0];
```

As we mentioned previously, the integer inside the square brackets can be any expression that evaluates to a suitable integer, as illustrated by the following:

```
int next = 1;
score[next + 3] = 100;
System.out.println(
 "Score at position 4 is " + score[next + 3]);
```

Note that, in the preceding code, `score[next + 3]` and `score[4]` are the same indexed variable, because `next + 3` evaluates to 4.

## The `length` Instance Variable

In Java, an array is considered to be an object, and, like other objects, it might have instance variables. As it turns out, an array has only one public instance variable, which is named `length`. This instance variable is automatically set to the size of the array when the array is created. For example, if you create an array as follows:

```
double[] score = new double[5];
```

then `score.length` has a value of 5.

The `length` instance variable can be used to make your program clearer by replacing an unnamed constant (such as 5), whose meaning may not be obvious, with a meaningful name such as `score.length`. In Display 6.2, we rewrote the program in Display 6.1 using the `length` instance variable.

Display 6.2 The `length` Instance Variable

---

```

1 import java.util.Scanner;

2 public class ArrayOfScores2
3 {
4 /**
5 * Reads in 5 scores and shows how much each
6 * Score differs from the highest score.
7 */
8 public static void main(String[] args)
9 {
10 Scanner keyboard = new Scanner(System.in);
11 double[] score = new double[5];
12 int index;
13 double max;

14 System.out.println("Enter " + score.length + " scores:");
15 score[0] = keyboard.nextDouble();
16 max = score[0];
17 for (index = 1; index < score.length; index++)
18 {
19 score[index] = keyboard.nextDouble();
20 if (score[index] > max)
21 max = score[index];
22 //max is the largest of the values score[0], ..., score[index].
23 }

24 System.out.println("The highest score is " + max);
25 System.out.println("The scores are:");
26 for (index = 0; index < score.length; index++)
27 System.out.println(score[index] + " differs from max by "
28 + (max - score[index]));
29 }
30 }

```

---

*The sample dialogue is the same as in Display 6.1.*

The `length` instance variable cannot be changed by your program (other than by creating a new array with another use of `new`).<sup>1</sup> For example, the following is illegal:

```
score.length = 10; //Illegal
```

---

<sup>1</sup>The technical details are as follows: The instance variable `length` is created when the array is created and is declared to be `public final int`.



### TIP: Use for Loops with Arrays

The second for loop in Display 6.2 illustrates a common way to step through an entire array using a for loop:

```
for (index = 0; index < score.length; index++)
 System.out.println(score[index] + " differs from max by "
 + (max - score[index]));
```

The for loop is ideally suited for performing array manipulations. ■



### PITFALL: Array Indices Always Start with Zero

The indices of an array always start with 0 and end with the integer that is one less than the size of the array. ■



### PITFALL: Array Index Out of Bounds

The most common programming error made when using arrays is attempting to use a nonexistent array index. For example, consider the following:

```
int[] a = new int[6];
```

When using the array `a`, every index expression must evaluate to one of the integers 0 through 5. For example, if your program contains the indexed variable `a[i]`, the `i` must evaluate to one of the six integers 0, 1, 2, 3, 4, or 5. If `i` evaluates to anything else, that is an error. When an index expression evaluates to some value other than those allowed by the array declaration, the index is said to be **out of bounds**. If your program attempts to use an array index that is out of bounds, then your program will end with an error message.<sup>2</sup> Note that this is a run-time error message, not a compiler error message.

Array indices get out of bounds most commonly at the first or last iteration of a loop that processes the array. So, it pays to carefully check all array processing loops to be certain that they begin and end with legal array indices. ■

illegal  
array index  
out of bounds

<sup>2</sup>Technically speaking, an `ArrayIndexOutOfBoundsException` exception is thrown. We will discuss exceptions in Chapter 9. Until you learn about handling exceptions, they will simply appear as error conditions to you.

## Initializing Arrays

An array can be initialized when it is declared. When initializing the array, the values for the various indexed variables are enclosed in braces and separated with commas. The expression with the braces is placed on the right-hand side of an assignment operator. For example,

```
int[] age = {2, 12, 1};
```

The array length (size) is automatically set to the number of values in the braces. So, this initializing declaration is equivalent to the following statements:

```
int[] age = new int[3];
age[0] = 2;
age[1] = 12;
age[2] = 1;
```

You can also initialize array elements using a `for` loop. For example,

```
double[] reading = new double[100];
int index;
for (index = 0; index < reading.length; index++)
 reading[index] = 42.0;
```

**automatic  
initialization**

If you do not initialize the elements of an array, they will automatically be initialized to a default value for the base type. The default values are the usual ones. For numeric types, the default value is the zero of the type. For base type `char`, the default value is the nonprintable zeroth character `(char)0`, not the space character. For the type `boolean`, the default value is `false`. For class types, the default value is `null`. For example, if you do not initialize an array of `doubles`, each element of the array will be initialized to `0.0`.

## Self-Test Exercises

1. In the array declaration

```
String[] word = new String[5];
```

what is

- a. the array name?
- b. the base type?
- c. the length of the array?
- d. the range of values an index accessing this array can have?
- e. one of the indexed variables (or elements) of this array?

(continued)



**Self-Test Exercises** (continued)

2. In the array

```
double[] score = new double[10];
```

what is

- a. the value of `score.length`?
- b. the first index of `score`?
- c. the last index of `score`?

3. What is the output of the following code?

```
char[] letter = {'a', 'b', 'c'};
for (int index = 0; index < letter.length; index++)
 System.out.print(letter[index] + " ");
```

4. What is the output of the following code?

```
double[] a = {1.1, 2.2, 3.3};
System.out.println(a[0] + " " + a[1] + " " + a[2]);
a[1] = a[2];
System.out.println(a[0] + " " + a[1] + " " + a[2]);
```

5. What is wrong with the following piece of code?

```
int[] sampleArray = new int[10];
for (int index = 1; index <= sampleArray.length; index++)
 sampleArray[index] = 3*index;
```

6. Suppose we expect the elements of the array `a` to be ordered so that

$$a[0] \leq a[1] \leq a[2] \leq \dots$$

However, to be safe we want our program to test the array and issue a warning in case it turns out that some elements are out of order. The following code is supposed to output such a warning, but it contains a bug. What is the bug?

```
double[] a = new double[10];
 <Some code to fill the array a goes here.>
for (int index = 0; index < a.length; index++)
 if (a[index] > a[index + 1])
 System.out.println("Array elements " + index +
 " and " + (index + 1) + " are out of order.");
```



## PITFALL: An Array of Characters Is Not a String

An array of characters, such as the array `a` created below, is conceptually a list of characters; therefore, it is conceptually like a string:

```
char[] a = {'A', ' ', 'B', 'i', 'g', ' ', 'H', 'i', '!'};
```

However, an array of characters, such as `a`, is not an object of the class `String`. In particular, the following is illegal in Java:

```
String s = a;
```

Similarly, you cannot normally use an array of characters, such as `a`, as an argument for a parameter of type `String`.

It is, however, easy to convert an array of characters to an object of type `String`. The class `String` has a constructor that has a single parameter of type `char[]`. So, you can obtain a `String` value corresponding to an array of characters, such as `a`, as follows:

```
String s = new String(a);
```

The object `s` will have the same sequence of characters as the array `a`. The object `s` is an independent copy; any changes made to `a` will have no effect on `s`. Note that this always uses the entire array `a`.

There is also a `String` constructor that allows you to specify a subrange of an array of characters `a`. For example,

```
String s2 = new String(a, 2, 3);
```

produces a `String` object with 3 characters from the array `a` starting at index 2. So, if `a` is as above, then

```
System.out.println(s2);
```

outputs

```
Big
```

Although an array of characters is not an object of the class `String`, it does have some things in common with `String` objects. For example, you can output an array of characters using `println`, as follows:

```
System.out.println(a);
```

which produces the output

```
A Big Hi!
```

provided `a` is as given previously. ■

## 6.2 Arrays and References

*A little more than kin, and less than kind.*

WILLIAM SHAKESPEARE, *Hamlet*, 1603.

Just like a variable of one of the class types you have seen, a variable of an array type holds a reference. In this section, we explore the consequences of this fact, including a discussion of array parameters. We will see that arrays are objects and that array types can be considered class types but somewhat different kinds of class types than what you are used to. Arrays and the kinds of classes we have seen before this chapter are *a little more than kin, and less than kind*.

### Arrays Are Objects

There are two ways to view an array: as a collection of indexed variables and as a single item whose value is a collection of values of the base type. In Section 6.1, we discussed using arrays as a collection of indexed variables. We will now discuss arrays from the second point of view.

An array can be viewed as a single item whose value is a collection of values of the base type. An array variable (as opposed to an array indexed variable) names the array as a single item. For example, the following declares a variable of an array type:

```
double[] a;
```

This variable `a` can and will contain a single value. The expression

```
new double[10]
```

creates an array object and stores the object in memory. The following assignment statement places a reference to (the memory address of) this array object in the variable `a`:

```
a = new double[10];
```

Typically, we combine all this into a single statement as follows:

```
double[] a = new double[10];
```

Notice that this is almost exactly the same as the way that we view objects of a class type. In Java, an array is considered an object. Whenever Java documentation says that something applies to objects, it means that it applies to arrays as well as objects of the class types we have seen up to now. You will eventually see examples of methods that can take arguments that may be objects of any kind. These methods will accept array objects as arguments as well as objects of an ordinary class type. Arrays are somewhat peculiar in how they relate to classes. Some authorities say array types are not classes,

and some say they are. But, all authorities agree that the arrays themselves are objects. Given that arrays are objects, it seems that one should view array types as classes, and we will do so. However, although an array type `double[]` is a class, the syntax for creating an array object is a bit different. To create an array, use the following syntax:

```
double[] a = new double[10];
```

You can view the expression `new double[10]` as an invocation of a constructor that uses a nonstandard syntax. (The nonstandard syntax was used to be consistent with the syntax used for arrays in older programming languages.)

As we have already seen, every array has an instance variable named `length`, which is a good example of viewing an array as an object. As with any other class type, array variables contain memory addresses, or, as they are usually called in Java, *references*. So, array types are reference types.<sup>3</sup>

Since an array is an object, you might be tempted to think of the indexed variables of an array, such as `a[0]`, `a[1]`, and so forth, as being instance variables of the object. This is actually a pretty good analogy, but it is not literally true. Indexed variables are not instance variables of the array. Indexed variables are a special kind of variable peculiar to arrays. The only instance variable in an array is the `length` instance variable.

An array object is a collection of items of the base type. Viewed as such, an array is an object that can be assigned with the assignment operator and plugged in for a parameter of an array type. Because an array type is a reference type, the behaviors of arrays with respect to assignment `=`, `==`, and parameter passing mechanisms are the same as what we have already described for classes. In the next few subsections, we discuss these details about arrays.

### Arrays Are Objects

In Java, arrays are considered to be objects, and, although there is some disagreement on this point, you can safely view an array type as a class type.

### Array Types Reference Types

A variable of an array type holds the address of where the array object is stored in memory. This memory address is called a **reference** to the array object.

---

<sup>3</sup>In many programming languages, such as C++, arrays are also reference types just as they are in Java. So, this detail about arrays is not peculiar to Java.



## PITFALL: Arrays with a Class Base Type

The base type of an array can be of any type, including a class type. For example, suppose `Date` is a class and consider the following:

```
Date[] holidayList = new Date[20];
```

This creates the 20 indexed variables (`holidayList[0]`, `holidayList[1]`, ..., `holidayList[19]`). It is important to note that this creates 20 *indexed variables* of type `Date`. This does not create 20 *objects* of type `Date`. (The index variables are automatically initialized to `null`, not to an object of the class `Date`.) Like any other variable of type `Date`, the indexed variables require an invocation of a constructor using `new` to create an object. One way to complete the initialization of the array `holidayList` is as follows:

```
Date[] holidayList = new Date[20];
for (int i = 0; i < holidayList.length; i++)
 holidayList[i] = new Date();
```

If you omit the `for` loop (and do not do something else more or less equivalent), then when you run your code, you will undoubtedly get an error message indicating a “null pointer exception.” If you do not use `new` to create an object, an indexed variable like `holidayList[i]` is just a variable that names no object and hence cannot be used as the calling object for any method. Whenever you are using an array with a class base type and you get an error message referring to a “Null Pointer Exception,” it is likely that your indexed variables do not name any objects and you need to add something such as the above `for` loop. ■

## Array Parameters

You can use both array indexed variables and entire arrays as arguments to methods, although they are different types of parameters. We first discuss array indexed variables as arguments to methods.

An indexed variable can be an argument to a method in exactly the same way that any variable of the array base type can be an argument. For example, suppose a program contains the following declarations:

```
double n = 0;
double[] a = new double[10];
int i;
```

If `myMethod` takes one argument of type `double`, then the following is legal:

```
myMethod(n);
```



VideoNote  
Arrays of  
Objects

indexed  
variable  
arguments

Since an indexed variable of the array `a` is also a variable of type `double`, just like `n`, the following is equally legal:

```
myMethod(a[3]);
```

There is one subtlety that does apply to indexed variables used as arguments. For example, consider the following method call:

```
myMethod(a[i]);
```

If the value of `i` is 3, then the argument is `a[3]`. On the other hand, if the value of `i` is 0, then this call is equivalent to the following:

```
myMethod(a[0]);
```

The indexed expression is evaluated to determine exactly which indexed variable is given as the argument.

### Array Indexed Variables as Arguments

An array indexed variable can be used as an argument anyplace that a variable of the array's base type can be used. For example, suppose you have the following:

```
double[] a = new double[10];
```

Indexed variables such as `a[3]` and `a[index]` can then be used as arguments to any method that accepts a `double` as an argument.

### entire array parameters

You can also define a method that has a formal parameter for an entire array so that when the method is called, the argument that is plugged in for this formal parameter is an entire array. Whenever you need to specify an array type, the type name has the form *Base\_Type*[], so this is how you specify a parameter type for an entire array. For example, the method `doubleArrayElements`, given in what follows, will accept any array of `double` as its single argument:

```
public class SampleClass
{
 public static void doubleArrayElements(double[] a)
 {
 int i;
 for (i = 0; i < a.length; i++)
 a[i] = a[i]*2;
 }
 < The rest of the class definition goes here.>
}
```

To illustrate this, suppose you have the following in some method definition:

```
double[] a = new double[10];
double[] b = new double[30];
```

and suppose that the elements of the arrays `a` and `b` have been given values. Both of the following are then legal method invocations:

```
SampleClass.doubleArrayElements(a);
SampleClass.doubleArrayElements(b);
```

Note that no square brackets are used when you give an entire array as an argument to a method.

An array type is a reference type just as a class type is, so, as with a class type argument, a method can change the data in an array argument. To phrase it more precisely, a method can change the values stored in the indexed variables of an array argument. This is illustrated by the preceding method `doubleArrayElements`.

An array type parameter does not specify the length of the array argument that may be plugged in for the parameter. An array knows its length and stores it in the `length` instance variable. The same array parameter can be replaced with array arguments of different lengths. Note that the preceding method `doubleArrayElements` can take an array of any length as an argument.

length of  
array  
arguments



### PITFALL: Use of = and == with Arrays

assignment  
with arrays

Array types are reference types; that is, an array variable contains the memory address of the array it names. The assignment operator copies this memory address. For example, consider the following code:

```
double[] a = new double[10];
double[] b = new double[10];
int i;
for (i = 0; i < a.length; i++)
 a[i] = i;
b = a;
System.out.println("a[2] = " + a[2] + " b[2] = " + b[2]);
a[2] = 42;
System.out.println("a[2] = " + a[2] + " b[2] = " + b[2]);
```

This will produce the following output:

```
a[2] = 2.0 b[2] = 2.0
a[2] = 42.0 b[2] = 42.0
```



## PITFALL: (continued)

The assignment statement `b = a`; copies the memory address from `a` to `b` so that the array variable `b` contains the same memory address as the array variable `a`. After the assignment statement, `a` and `b` are two different names for the same array. Thus, when we change the value of `a[2]`, we are also changing the value of `b[2]`.

Unless you want two array variables to be two names for the same array (and on rare occasions, you do want this), you should not use the assignment operator with arrays. If you want the arrays `a` and `b` in the preceding code to be different arrays with the same values in each index position, then instead of the assignment statement

```
b = a;
```

you need to use something such as the following:

```
int i;
for (i = 0; (i < a.length) && (i < b.length); i++)
 b[i] = a[i];
```

Note that the above code will not make `b` an exact copy of `a`, unless `a` and `b` have the same length.

The equality operator `==` does not test two arrays to see if they contain the same values. It tests two arrays to see if they are stored in the same location in the computer's memory. For example, consider the following code:

```
int[] c = new int[10];
int[] d = new int[10];
int i;
for (i = 0; i < c.length; i++)
 c[i] = i;
for (i = 0; i < d.length; i++)
 d[i] = i;

if (c == d)
 System.out.println("c and d are equal by ==.");
else
 System.out.println("c and d are not equal by ==.");
```

This produces the output

```
c and d are not equal by ==
```

(continued)

**==**  
**with arrays**





### PITFALL: (continued)

even though `c` and `d` contain the same integers in the same indexed variables. A comparison using `==` will say they are not equal because `==` checks only the contents of the array variables `c` and `d`, which are memory addresses, and `c` and `d` contain different memory addresses.

If you want to test two arrays to see if they contain the same elements, then you can define an `equalArrays` method for the arrays, just as you defined an `equals` method for a class. Display 6.3 contains one possible definition of `equalArrays` for arrays in a small demonstration class. ■

### Self-Test Exercises

7. Consider the following class definition:

```
public class SomeClass
{
 public static void doSomething(int n)
 {
 <Some code goes in here.>
 }
}
```

<The rest of the definition is irrelevant to this question.>

Which of the following are acceptable method calls?

```
int[] a = {4, 5, 6};
int number = 2;
SomeClass.doSomething(number);
SomeClass.doSomething(a[2]);
SomeClass.doSomething(a[3]);
SomeClass.doSomething(a[number]);
SomeClass.doSomething(a);
```

8. Write a method definition for a static `void` method called `oneMore`, which has a formal parameter for an array of integers and increases the value of each array element by one. (The definition will go in a class, but you need only give the method definition.)
9. Write a method named `outOfOrder` that takes as a parameter an array of `double` and returns a value of type `int`. This method will test the array for being out of order, meaning that the array violates the condition:

```
a[0] <= a[1] <= a[2] <= ...
```

### Self-Test Exercises (continued)

The method returns `-1` if the elements are not out of order; otherwise, it returns the index of the first element of the array that is out of order. For example, consider the following declaration:

```
double[] a = {1.2, 2.1, 3.3, 2.5, 4.5,
 7.9, 5.4, 8.7, 9.9, 1.0};
```

In the array above, `a[2]` and `a[3]` are the first pair out of order, and `a[3]` is the first element out of order, so the method returns 3. If the array is sorted, the method returns `-1`.

10. The following method definition will compile but does not work as you might hope. What is wrong with it?

```
public static void doubleSize(int[] a)
{
 a = new int[a.length * 2];
}
```

### Array Parameters and Array Arguments

An argument to a method may be an entire array. Array arguments are like objects of a class, in that the method can change the data in an array argument; that is, a method can change the values stored in the indexed variables of an array argument. A method with an array parameter is defined and invoked as illustrated by the following examples. Note that the array parameter specifies the base type of the array but not the length of the array.

#### EXAMPLE (OF ARRAY PARAMETERS)

```
public class AClass
{
 public static void listChars(char[] a)
 {
 int i;
 for (i = 0; i < a.length; i++)
 System.out.println(a[i] + " ");
 }
 public static void zeroAll(int[] anArray)
 {
 int i;
 for (i = 0; i < anArray.length; i++)
 anArray[i] = 0;
 }
 ...
}
```

**EXAMPLE (OF ARRAY ARGUMENTS)**

```
char[] c = new char[10];
int[] a = new int[10];
int[] b = new int[20];
```

<Some code to fill the arrays goes here.>

```
Aclass.listChars(c);
Aclass.zeroAll(a);
Aclass.zeroAll(b);
```

*Note that arrays a and b have different lengths. Also note that no square brackets are used with array arguments.*

## Display 6.3 Testing Arrays for Equality (part 1 of 2)

```
1 public class DifferentEquals
2 {
3 /**
4 * A demonstration to see how == and an equalArrays method are different.
5 */
6 public static void main(String[] args)
7 {
8 int[] c = new int[10];
9 int[] d = new int[10];

10 int i;
11 for (i = 0; i < c.length; i++)
12 c[i] = i;
13 for (i = 0; i < d.length; i++)
14 d[i] = i;
15 if (c == d)
16 System.out.println("c and d are equal by ==.");
17 else
18 System.out.println("c and d are not equal by ==.");

19 System.out.println("== only tests memory addresses.");

20 if (equalArrays(c, d))
21 System.out.println(
22 "c and d are equal by the equalArrays method.");
23 else
24 System.out.println(
25 "c and d are not equal by the equalArrays method.");
```

*The arrays c and d contain the same integers in each index position.*

---

Display 6.3 Testing Arrays for Equality (part 2 of 2)

---

```
26 System.out.println(
27 "An equalArrays method is usually a more useful test.");
28 }

29 public static boolean equalArrays(int[] a, int[] b)
30 {
31 if (a.length != b.length)
32 return false;
33 else
34 {
35 int i = 0;
36 while (i < a.length)
37 {
38 if (a[i] != b[i])
39 return false;
40 i++;
41 }
42 }

43 return true;
44 }

45 }
```

## Sample Dialogue

```
c and d are not equal by ==.
== only tests memory addresses.
c and d are equal by the equalArrays method.
An equalArrays method is usually a more useful test.
```

---

## Arguments for the Method `main` ★

The heading for the `main` method of a program looks as if it has a parameter for an array of base type of `String`:

```
public static void main(String[] args)
```

The identifier `args` is in fact a parameter of type `String[]`. Because `args` is a parameter, it could be replaced by any other nonkeyword identifier. The identifier `args` is traditional, but it is perfectly legal to use some other identifier.

We have never given `main` an array argument, or any other kind of argument, when we ran any of our programs. So, what did Java use as an argument to plug in for

args? If no argument is given when you run your program, then a default empty array of strings is automatically provided as a default argument to `main` when you run your program.

It is possible to run a Java program in a way that provides an argument to plug in for this array of `String` parameters. You do not provide it as an array. You provide any number of string arguments when you run the program, and those string arguments will automatically be made elements of the array argument that is plugged in for `args` (or whatever name you use for the parameter to `main`). This is normally done by running the program from the command line of the operating system, like so:

```
java YourProgram Do Be Do
```

This will set `args[0]` to "Do", `args[1]` to "Be", `args[2]` to "Do", and `args.length` to 3. These three indexed variables can be used in the method `main`, as in the following sample program:

```
public class YourProgram
{
 public static void main(String[] args)
 {
 System.out.println(args[1] + " " + args[0]
 + " " + args[1]);
 }
}
```

If the above program is run from the command line as follows:

```
java YourProgram Do Be Do
```

the output produced by the program will be

```
Be Do Be
```

Be sure to note that the argument to `main` is an array of *strings*. If you want numbers, you must convert the string representations of the numbers to values of a number type or types.

### The Method `main` Has an Array Parameter

The heading for the `main` method of a program is as follows:

```
public static void main(String[] args)
```

The identifier `args` is a parameter for an array of base type `String`. The details are explained in the text.

## Methods That Return an Array

In Java, a method may return an array. You specify the return type for a method that returns an array in the same way that you specify a type for an array parameter. For example, the following is an example of a method that returns an array:

```
public static char[] upperCaseVersion(char[] a)
{
 char[] temp = new char[a.length];
 int i;
 for (i = 0; i < a.length; i++)
 temp[i] = Character.toUpperCase(a[i]);
 return temp;
}
```

### Returning an Array

A method can return an array. The details are basically the same as for a method that returns an object of a class type.

#### SYNTAX (FOR A TYPICAL WAY OF RETURNING AN ARRAY)

```
public static Base_Type [] Method_Name(Parameter_List)
{
 Base_Type[] temp = new Base_Type[Array_Size]
 <Some code to fill temp goes here.>
 return temp;
}
```

The method need not be static and need not be public. You do not necessarily need to use a local array variable such as `temp`.

#### EXAMPLE (ASSUMED TO BE IN A CLASS DEFINITION)

```
public static int [] incrementedArray(int[] a,int increment)
{
 int[] temp = new int[a.length];
 int i;
 for (i = 0; i < a.length; i++)
 temp[i] = a[i] + increment;
 return temp;
}
```

### Array Type Names

Whenever you need an array type name, whether for the type of an array variable declaration, the type of an array parameter, or the type for a method that returns an array, you specify the type name in the same way.

#### SYNTAX

*Base\_Type*[]

#### EXAMPLES

```
double[] a = new double[10];
int[] giveIntArray(char[] arrayParameter)
{ ... }
```

### Self-Test Exercises

11. Give the definition of a method called `halfArray` that has a single parameter for an array of base type `double` and that returns another array of base type `double` that has the same length and in which each element has been divided by 2.0. Make it a static method. To test it, you can add it to any class or, better yet, write a class with a test program in the method `main`.
12. What is wrong with the following method definition? It is an alternate definition of the method by the same name defined in the previous subsection. It will compile.

```
public static char[] upperCaseVersion(char[] a)
{
 char i;
 for (i = 0; i < a.length; i++)
 a[i] = Character.toUpperCase(a[i]);
 return a;
}
```

## 6.3 Programming with Arrays

*Never trust to general impressions, my boy,  
but concentrate yourself upon details.*

SIR ARTHUR CONAN DOYLE, *A Case of Identity*, 1891.

In this section, we discuss partially filled arrays as well as how to use arrays as class instance variables.

## Partially Filled Arrays

Often the exact size needed for an array is not known when a program is written, or the size may vary from one run of the program to another. One common and easy way to handle this situation is to declare the array to be of the largest size the program could possibly need. The program is then free to use as much or as little of the array as needed.

partially  
filled array

**Partially filled arrays** require some care. The program must keep track of how much of the array is used and must not reference any indexed variable that has not been given a meaningful value. The program in Display 6.4 illustrates this point. It reads in a list of golf scores and shows how much each score differs from the average. This program will work for lists as short as 1 score, as long as 10 scores, and of any length in between. The scores are stored in the array `score`, which has 10 indexed variables, but the program uses only as much of the array as it needs. The variable `numberUsed` keeps track of how many elements are stored in the array. The elements (that is, the scores) are stored in positions `score[0]` through `score[numberUsed - 1]`. The details are very similar to what they would be if `numberUsed` were `score.length` and the entire array were used. Note that the variable `numberUsed` usually must be an argument to any method that manipulates the partially filled array. For example, the methods `showDifference` and `computeAverage` use the argument `numberUsed` to ensure that only meaningful array indices are used.

Display 6.4 Partially Filled Array (part 1 of 3)

---

```

1 import java.util.Scanner;
2 public class GolfScores
3 {
4 public static final int MAX_NUMBER_SCORES = 10;

5 /**
6 Shows differences between each of a list of golf scores and their
7 average.
8 */
9 public static void main(String[] args)
10 {
11 double[] score = new double[MAX_NUMBER_SCORES];
12 int numberUsed = 0;

13 System.out.println("This program reads golf scores and shows");
14 System.out.println("how much each differs from the average.");
15 System.out.println("Enter golf scores:");
16 numberUsed = fillArray(score);
17 showDifference(score, numberUsed);
18 }

```

(continued)

*Contrary to normal practice, this allows fractional scores, such as 71.5. However, this makes it a better example for our purposes. (Anyway, when I play golf, losing a ball is only half a stroke penalty. Try it sometime.)*



## Display 6.4 Partially Filled Array (part 2 of 3)

```

18 /**
19 Reads values into the array a. Returns the number of values placed
 in the array a.
20 */
21 public static int fillArray(double[] a)
22 {
23 System.out.println("Enter up to " + a.length
24 + " nonnegative numbers.");
25 System.out.println("Mark the end of the list with a negative
 number.");
26 Scanner keyboard = new Scanner(System.in);
27
28 double next;
29 int index = 0;
30 next = keyboard.nextDouble();
31 while ((next >= 0) && (index < a.length))
32 {
33 a[index] = next;
34 index++;
35 next = keyboard.nextDouble();
36 //index is the number of array indexed variables used so far.
37 }
38 //index is the total number of array indexed variables used.
39 if (next >= 0)
40 System.out.println("Could only read in "
41 + a.length + " input values.");
42
43 return index;
44
45 /**
46 Precondition: numberUsed <= a.length.
47 a[0] through a[numberUsed-1] have values.
48 Returns the average of numbers a[0] through a[numberUsed-1].
49 */
50 public static double computeAverage(double[] a, int numberUsed)
51 {
52 double total = 0;
53 for (int index = 0; index < numberUsed; index++)
54 total = total + a[index];
55 if (numberUsed > 0)
56 {
57 return (total/numberUsed);
58 }
59 else
60 {
61 System.out.println("ERROR: Trying to average 0 numbers.");
62 System.out.println("computeAverage returns 0.");
63 return 0;
64 }
65 }

```

The value of `index` is the number of values stored in the array.

## Display 6.4 Partially Filled Array (part 3 of 3)

```

65 /**
66 Precondition: numberUsed <= a.length.
67 The first numberUsed indexed variables of a have values.
68 Postcondition: Gives screen output showing how much each of the first
69 numberUsed elements of the array a differ from their average.
70 */
71 public static void showDifference(double[] a, int numberUsed)
72 {
73 double average = computeAverage(a, numberUsed);
74 System.out.println("Average of the " + numberUsed
75 + " scores = " + average);
76 System.out.println("The scores are:");
77 for (int index = 0; index < numberUsed; index++)
78 System.out.println(a[index] + " differs from average by "
79 + (a[index] - average));
80 }
81 }

```

## Sample Dialogue

```

This program reads golf scores and shows
how much each differs from the average.
Enter golf scores:
Enter up to 10 nonnegative numbers.
Mark the end of the list with a negative number.
69 74 68 -1
Average of the 3 scores = 70.3333
The scores are:
69.0 differs from average by -1.33333
74.0 differs from average by 3.66667
68.0 differs from average by -2.33333

```

## Self-Test Exercises

- Complete the definition of the following method that could be added to the class `GolfScores` in Display 6.4:

```

/**
Precondition: numberUsed <= argumentArray.length;
the first numberUsed indexed variables of argumentArray
have values.
Returns an array of length numberUsed whose ith element
is argumentArray[i] - adjustment.
*/
public static double[] differenceArray(
 double[] argumentArray, int numberUsed, double adjustment)

```

(continued)

**Self-Test Exercises** (continued)

14. Rewrite the class `GolfScores` from Display 6.4 using the method `differenceArray` from Self-Test Exercise 13.
15. Rewrite the class `GolfScores` from Display 6.4 making the array of scores a static variable. Also, make the `int` variable `numberUsed` a static variable. Start with Display 6.4, not with the answer to Self-Test Exercise 14. *Hint:* All, or at least most, methods will have no parameters.

**EXAMPLE: A Class for Partially Filled Arrays**

If you are going to use some array in a disciplined way, such as using the array as a partially filled array, then it is often best to create a class that has the array as an instance variable and to have the constructors and methods of the class provide the needed operations as methods. For example, in Display 6.5, we wrote a class for a partially filled array of doubles. In Display 6.6, we wrote the program in Display 6.4 using this class.

In Display 6.6, we wrote the code to be exactly analogous to that of Display 6.4 so that you could see how one program mirrors the other. However, this resulted in occasionally recomputing a value several times. For example, the method `computeAverage` contains the following expression three times:

```
a.getNumberOfElements()
```

Because the `PartiallyFilledArray a` is not changed in this method, these each return the same value. Some programmers advocate computing this value once only and saving the value in a variable. These programmers would use something such as the following for the definition of `computeAverage` rather than what we used in Display 6.6. The variable `numberOfElementsInA` is used to save a value so it need not be recomputed.

```
public static double computeAverage(PartiallyFilledArray a)
{
 double total = 0;
 double numberOfElementsInA = a.getNumberOfElements();
 for (int index = 0; index < numberOfElementsInA; index++)
 total = total + a.getElement(index);
 if (numberOfElementsInA > 0)
 {
 return (total/numberOfElementsInA);
 }
 else
 {

```

**EXAMPLE:** (continued)

```

 System.out.println(
 "ERROR: Trying to average 0 numbers.");
 System.out.println("computeAverage returns 0.");
 return 0;
 }
}

```

This is not likely to produce a noticeable difference in the efficiency of the program in Display 6.6, but if the number of elements in the `PartiallyFilledArray` were large so that the for loop would be executed many times, it might make a difference in a situation where efficiency is critical.

## Display 6.5 Partially Filled Array Class (part 1 of 4)

```

1 /**
2 Class for a partially filled array of doubles. The class enforces the
3 following invariant: All elements are at the beginning of the array in
4 locations 0, 1, 2, and so forth up to a highest index with no gaps.
5 */
6 public class PartiallyFilledArray
7 {
8 private int maxNumberElements; //Same as a.length
9 private double[] a;
10 private int numberUsed; //Number of indices currently in use

11 /**
12 Sets the maximum number of allowable elements to 10.
13 */
14 PartiallyFilledArray()
15 {
16 maxNumberElements = 10;
17 a = new double[maxNumberElements];
18 numberUsed = 0;
19 }

20 /**
21 Precondition arraySize > 0.
22 */
23 PartiallyFilledArray(int arraySize)
24 {
25 if (arraySize <= 0)
26 {
27 System.out.println("Error Array size zero or negative.");
28 System.exit(0);
29 }

```

(continued)

Display 6.5 Partially Filled Array Class (part 2 of 4)

```

30 maxNumberElements = arraySize;
31 a = new double[maxNumberElements];
32 numberUsed = 0;
33 }
34 PartiallyFilledArray(PartiallyFilledArray original)
35 {
36 if (original == null)
37 {
38 System.out.println("Fatal Error: aborting program.");
39 System.exit(0);
40 }
41 maxNumberElements =
42 original.maxNumberElements;
43 numberUsed = original.numberUsed;
44 a = new double[maxNumberElements];
45 for (int i = 0; i < numberUsed; i++)
46 a[i] = original.a[i];
47 }

48 /**
49 * Adds newElement to the first unused array position.
50 */
51 public void add(double newElement)
52 {
53 if (numberUsed >= a.length)
54 {
55 System.out.println("Error: Adding to a full array.");
56 System.exit(0);
57 }
58 else
59 {
60 a[numberUsed] = newElement;
61 numberUsed++;
62 }
63 }

64 public double getElement(int index)
65 {
66 if (index < 0 || index >= numberUsed)
67 {
68 System.out.println("Error: Illegal or unused index.");
69 System.exit(0);
70 }
71 return a[index];
72 }

```

*Note that the instance variable a is a copy of original.a. The following would not be correct: a = original.a;.*

*This point is discussed later in this chapter in the subsection entitled "Privacy Leaks with Array Instance Variables."*

## Display 6.5 Partially Filled Array Class (part 3 of 4)

```
73 /**
74 index must be an index in use or the first unused index.
75 */
76 public void resetElement(int index, double newValue)
77 {
78 if (index < 0 || index >= maxNumberElements)
79 {
80 System.out.println("Error:Illegal index.");
81 System.exit(0);
82 }
83 else if (index > numberUsed)
84 {
85 System.out.println(
86 "Error: Changing an index that is too large.");
87 System.exit(0);
88 }
89 else
90 a[index] = newValue;
91 }

92 public void deleteLast()
93 {
94 if (empty())
95 {
96 System.out.println("Error:Deleting from an empty array.");
97 System.exit(0);
98 }
99 else
100 numberUsed--;
101 }
102 /**
103 Deletes the element in position index. Moves down all elements with
104 indices higher than the deleted element.
105 */
106 public void delete(int index)
107 {
108 if (index < 0 || index >= numberUsed)
109 {
110 System.out.println("Error:Illegal or unused index.");
111 System.exit(0);
112 }

113 for (int i = index; i < numberUsed; i++)
114 a[i] = a[i + 1];
115 numberUsed--;
116 }
```

(continued)

Display 6.5 Partially Filled Array Class (part 4 of 4)

```
117 public boolean empty()
118 {
119 return (numberUsed == 0);
120 }

121 public boolean full()
122 {
123 return (numberUsed == maxNumberElements);
124 }

125 public int getMaxCapacity()
126 {
127 return maxNumberElements;
128 }
129
130 public int getNumberOfElements()
131 {
132 return numberUsed;
133 }
134 }
```



### TIP: Accessor Methods Need Not Simply Return Instance Variables

Note that in the class `PartiallyFilledArray` in Display 6.5, there is no accessor method that returns a copy of the entire instance variable `a`. The reason this was not done is that, when the class is used as intended, a user of the class `PartiallyFilledArray` would have no need for the entire array `a`. That is an implementation detail. The other methods that start with `get` allow a programmer using the class to obtain all the data that he or she needs. ■

## The “for-each” Loop ★

As you have already seen, you can use a `for` loop to cycle through all the elements in an array. For example,

```
double[] a = new double[10];
<Some code to fill the array a>
for (int i = 0; i < a.length; i++)
 System.out.println(a[i]);
```

The standard Java libraries contain definitions of a number of so-called *collection classes*. A collection class is a class whose objects store a collection of values. You cannot cycle

Display 6.6 Display 6.4 Redone Using the Class `PartiallyFilledArray` (part 1 of 2)

```

1 import java.util.Scanner; Sample dialogue is the same as in Display 6.4.

2 /**
3 Demonstrates using the class PartiallyFilledArray,
4 */
5 public class GolfScoresVersion2
6 {

7 public static final int MAX_NUMBER_SCORES = 10;

8 /**
9 Shows the differences between each of a list of golf scores and
10 their average.
11 */
12 public static void main(String[] args)
13 {
14 PartiallyFilledArray score =
15 new PartiallyFilledArray(MAX_NUMBER_SCORES);

16 System.out.println("This program reads golf scores and shows");
17 System.out.println("how much each differs from the average.");

18 System.out.println("Enter golf scores:");
19 fillArray(score);
20 showDifference(score);
21 }
22 /**
23 Reads values into the PartiallyFilledArray a.
24 */
25 public static void fillArray(PartiallyFilledArray a)
26 {
27 System.out.println("Enter up to " + a.getMaxCapacity()
28 + " nonnegative numbers, one per line.");
29 System.out.println("Mark the end of the list with a negative
30 number");
31 Scanner keyboard = new Scanner(System.in);

32 double next = keyboard.nextDouble();
33 while ((next >= 0) && (!a.full()))
34 {
35 a.add(next);
36 next = keyboard.nextDouble();
37 }
38 if (next >= 0)
39 System.out.println("Could only read in "
40 + a.getMaxCapacity() + " input values.");
41 }

```

(continued)



Display 6.6 Display 6.4 Redone Using the Class `PartiallyFilledArray` (part 2 of 2)

---

```

41 /**
42 *Returns the average of numbers in the PartiallyFilledArray a.
43 */
44 public static double computeAverage(PartiallyFilledArray a)
45 {
46 double total = 0;
47 for (int index = 0; index < a.getNumberOfElements(); index++)
48 total = total + a.getElement(index);

49 if (a.getNumberOfElements() > 0)
50 {
51 return (total/a.getNumberOfElements());
52 }
53 else
54 {
55 System.out.println("ERROR: Trying to average 0 numbers.");
56 System.out.println("computeAverage returns 0.");
57 return 0;
58 }
59 }

60 /**
61 *Gives screen output showing how much each of the
62 *elements in the PartiallyFilledArray a differs from the average.
63 */
64 public static void showDifference(PartiallyFilledArray a)
65 {
66 double average = computeAverage(a);
67 System.out.println("Average of the " + a.getNumberOfElements()
68 + " scores = " + average);
69 System.out.println("The scores are:");
70 for (int index = 0; index < a.getNumberOfElements(); index++)
71 System.out.println(a.getElement(index)
72 + "differs from average by"
73 + (a.getElement(index) - average));
74 }

```

---

through all the elements of a collection object with this kind of `for` loop, because these collection classes normally do not have indices associated with their elements, as an array does.<sup>4</sup> However, starting with version 5.0, Java has added a new kind of `for` loop that can cycle through all the elements in a collection even though there are no indices

---

<sup>4</sup>You can construct a similar `for` loop using something called an *iterator* in place of the array index, but we will not go into that until later in this book.

## for-each loop

for the elements (as there are with an array). This new kind of `for` loop is called a **for-each loop** or **enhanced for loop**. We will discuss these for-each loops in detail when we cover collections (Chapter 16). However, these new for-each loops can be used with arrays as well as with objects of these collection classes. In this subsection, we tell you how to use for-each loops with arrays in case you want to get started using them. However, we do not use the for-each loop in this book until we discuss collection classes in Chapter 16.

The following code ends with a for-each loop that is equivalent to the regular `for` loop that we gave at the start of this subsection:

```
double[] a = new double[10];
<Some code to fill the array a>
for (double element : a)
 System.out.println(element);
```

You can read the line beginning with `for` as “for each `element` in `a`, do the following.” Note that the variable, `element`, has the same type as the elements in the array. The variable, like `element`, must be declared in the for-each loop as we have done. If you attempt to declare `element` before the for-each loop, you will get a compiler error message.

The general syntax for a for-each loop statement used with an array is

```
for (Array_Base_Type Variable : Array_Name)
 Statement
```

Be sure to notice that you use a colon (not a semicolon) after the *Variable*. You may use any legal variable name for the *Variable*; you do not have to use `element`. Although it is not required, the *Statement* typically contains the *Variable*. When the for-each loop is executed, the *Statement* is executed once for each element of the array. More specifically, for each `element` of the array, the *Variable* is set to the array element and then the *Statement* is executed.

The for-each loop can make your code a lot cleaner and a lot less error prone. If you are not using the indexed variable in a `for` loop for anything other than as a way to cycle through all the array elements, then a for-each loop is preferable. For example,

```
for (double element : a)
 sum += element;
```

is preferable to

```
for (int i = 0; i < a.length; i++)
 sum += a[i];
```

The two loops do the same thing, but the second one mentions an index `i` that is not used for anything other than enumerating the array elements. Also, the syntax for the for-each loop is simpler than that of the regular `for` loop.

On the other hand, you should leave the following `for` loop as is and not attempt to convert it to a `for-each` loop:

```
for (int i = 0; i < a.length; i++)
 a[i] = 2*i;
```

Because this `for` loop uses the index `i` in the body of the `for` loop and uses it in an essential way, it does not make sense to convert this `for` loop to a `for-each` loop.

### For-Each Loop for Arrays

#### SYNTAX

```
for (Array_Base_Type Variable : Array_Name)
 Statement
```

#### EXAMPLES

```
for (double element : a)
 sum += element;
```

The array `a` has base type `double`. This `for-each` loop sets each element of the array `a` to `0.0`.

A good way to read the first line of the example is "For each `element` in `a`, do the following."

### Methods with a Variable Number of Parameters ★

Because of overloading, you can have a method named `max` that returns the largest of two `int` values and have another method named `max` that takes three `int` arguments and returns the largest of the three. If you decide you need a method that returns the largest of four `int` values, you can define a version of the method `max` that takes four arguments. However, with this approach, there is no way to cover all cases of computing the maximum of some number of `int` values. Covering all cases in this way would require an infinite number of definitions for the method name `max`, and no programmer has enough time to write an infinite number of definitions. What we would like is a single method definition for a method named `max` that can take any number of `int` arguments. Starting with version 5.0, Java lets you define methods that take any number of arguments. For example, the following is the definition of a method named `max` that takes any number of `int` arguments and returns the largest of its arguments:

```
public static int max(int... arg)
{
 if (arg.length == 0)
 {
 System.out.println("Fatal Error: "+
 "maximum of zero values.");
 System.exit(0);
 }
}
```

```

 int largest = arg[0];
 for (int i = 1; i < arg.length; i++)
 if (arg[i] > largest)
 largest = arg[i];
 return largest;
}

```

This method `max` works by taking its `int` arguments and placing them in an array named `arg` whose base type is `int`. For example, suppose this definition of the method `max` is in a class named `UtilityClass`, and consider the following method call:

```
int highestScore = UtilityClass.max(3, 2, 5, 1);
```

The array `arg` is automatically declared and initialized as follows:

```
int[] arg = {3, 2, 5, 1};
```

So, `arg[0] == 3`, `arg[1] == 2`, `arg[2] == 5`, and `arg[3] == 1`. After this, the code in the body of the method definition is executed. Display 6.7 shows a sample program that uses this method `max`.

Note that a method (such as `max`) that takes any number of arguments is basically a method that takes an array as an argument, except that the job of placing values in the

Display 6.7 Method with a Variable Number of Parameters (part 1 of 2)

---

```

1 public class UtilityClass
2 {
3 /**
4 * Returns the largest of any number of int values.
5 */
6 public static int max(int... arg)
7 {
8 if (arg.length == 0)
9 {
10 System.out.println("Fatal Error: maximum of zero values.");
11 System.exit(0);
12 }
13
14 int largest = arg[0];
15 for (int i = 1; i < arg.length; i++)
16 if (arg[i] > largest)
17 largest = arg[i];
18 return largest;
19 }
20 }

```

*This is the file `UtilityClass.java`.*

(continued)

Display 6.7 Method with a Variable Number of Parameters (part 2 of 2)

```

1 import java.util.Scanner;
2 public class VariableParameterDemo
3 {
4 public static void main(String[] args)
5 {
6 System.out.println("Enter scores for Tom, Dick, and Harriet:");
7 Scanner keyboard = new Scanner(System.in);
8 int tomsScore = keyboard.nextInt();
9 int dicksScore = keyboard.nextInt();
10 int harrietsScore = keyboard.nextInt();
11 int highestScore = UtilityClass.max(tomsScore, dicksScore,
12 harrietsScore);
13 System.out.println("Highest score = " + highestScore);
14 }

```

*This is the file*  
*VariableParameterDemo.java.*

#### Sample Dialogue

```

Enter scores for Tom, Dick, and Harriet:
55 100 99
Highest score = 100

```

**vararg  
specification  
ellipsis**

array is done automatically for the programmer. The values for the array are given as arguments, and Java automatically creates an array and places the arguments in the array.

A parameter specification that specifies any number of parameters, such as `int... arg`, is called a **vararg specification**. (It would make more sense to call it a *varparameter* specification, but the word *vararg* is too well entrenched, so we will go along with common usage.) The three dots in a vararg specification are called an **ellipsis**. Note that the ellipsis is part of the Java syntax and not an abbreviation used in this book. You type in the three dots.

You can have only one variable parameter specification, such as `int... arg`, in a method definition. However, you may also have any number of ordinary parameters, in which case the vararg specification must be the last item on the parameter list. This is illustrated in Display 6.8, which we discuss in the next subsection.

In Chapter 2, you saw one example of a method that accepts a variable number of arguments, namely the method `System.out.printf`. However, we could not tell you how to define such methods yourself until we covered the basics about arrays.

### Method with a Variable Number of Parameters

A method with a variable number of parameters has a vararg specification as the last item on its parameter list. A vararg specification has the following form:

*Type... Array\_Name*

Some examples of vararg specifications are

```
int... arg
double... a
String... unwanted
```

Displays 6.7 and 6.8 show two of these vararg specifications in complete method definitions.

In any invocation of a method with a vararg specification, you handle arguments corresponding to regular parameters in the usual way. Following the arguments for regular parameters, you can have any number of arguments of the type given in the vararg specification. These arguments are automatically placed in an array, and the array can be used in the method definition. A full description of the details is given in this chapter.

### Self-Test Exercises

16. Redo the definition of the method `max` in Display 6.7 using a for-each loop in place of the regular `for` loop.
17. What would be the dialogue in Display 6.8 if we omit the following line from the program?

```
sentence = Utility2.censor(sentence, " ,"); //Deletes extra commas
```

### EXAMPLE: A String Processing Example ★

This example uses material from the earlier starred subsection “Methods with a Variable Number of Parameters.” If you have not read that subsection, you should skip this example.

Display 6.8 contains a utility class with the string processing method named `censor` and an example of a program that uses that method. The method `censor` takes a single `String` parameter followed by any number of additional parameters of type `String`. The first parameter will be a sentence or other string that may contain substrings you want to delete. The method returns its first parameter with all occurrences of the remaining string parameters removed.

(continued)

**EXAMPLE:** (continued)

Note that the method `sensor` has one regular parameter followed by a specification for any number of additional string parameters. In this case, all parameters are of type `String`. However, that first regular parameter (or parameters) in a method heading can be of any type (or types); they need not match the type of the vararg specification. We just happen to want the type `String` here.

Because the first parameter is of type `String` and the vararg specification in this case says the remaining arguments are of type `String`, you might wonder why we did not omit the first `String` parameter sentence, have only a vararg specification, and then use `unwanted[0]` to serve the same role as `sentence`. If we did so, then the method `sensor` could be called with no arguments at all. A vararg specification allows any number of arguments, including the possibility of zero arguments. However, we want to insist that the method `sensor` have at least one argument, and the parameter sentence ensures that `sensor` will always have at least one argument.

## Privacy Leaks with Array Instance Variables

In Chapter 5, we explained why it is a compromise of privacy for a class to have an accessor (or other method) that returns a reference to a private mutable object. As we noted there, an accessor method should instead return a reference to a *deep copy* of the private object. (See the Pitfall subsection of Chapter 5 entitled “Privacy Leaks.”) At the time, we had in mind returning the contents of a private instance variable of a class type. However, the lesson applies equally well to private instance variables of an array type.

For example, suppose that you decide that you want an accessor method for the array instance variable in the class `PartiallyFilledArray` in Display 6.5. You might be tempted to define the accessor method as follows:

```
public double[] getInsideArray() // Problematic version
{
 return a;
}
```

As indicated in the comment, this definition has a problem, which is this accessor method allows a programmer to change the array object named by the private instance variable `a` in ways that bypass the checks built into the mutator methods of the class `PartiallyFilledArray`. To see why this is true, *suppose we added this definition of the method* `getInsideArray` *to the class* `PartiallyFilledArray`, and then consider the following code:

```
PartiallyFilledArray leakyArray =
 new PartiallyFilledArray(10);
double[] arrayName = leakyArray.getInsideArray();
```

The variable `arrayName` and the private instance variable `a` now contain the same reference, so both `arrayName` and the private instance variable `a` name the same array. Using `arrayName` as a name for the array named by the private instance variable `a`, we

## Display 6.8 String Processing Method with a Variable Number of Parameters (part 1 of 2)

---

```

1 public class Utility2
2 {
3 /**
4 * Returns the first argument with all occurrences of other arguments
5 * deleted.
6 */
7 public static String censor(String sentence, String... unwanted)
8 {
9 for (int i = 0; i < unwanted.length; i++)
10 sentence = deleteOne(sentence, unwanted[i]);
11 return sentence;
12 }
13 /**
14 * Returns sentence with all occurrences of oneUnwanted removed.
15 */
16 private static String deleteOne(String sentence, String oneUnwanted)
17 {
18 String ending;
19 int position = sentence.indexOf(oneUnwanted);
20 while (position >= 0) //While word was found in sentence
21 {
22 ending = sentence.substring(position + oneUnwanted.length());
23 sentence = sentence.substring(0, position) + ending;
24 position = sentence.indexOf(oneUnwanted);
25 }
26 return sentence;
27 }

```

*Both methods use the parameter sentence as a local variable. If this puzzles you, review the material on parameters in Chapters 4 and 5, particularly Display 4.5 in Chapter 4.*

*If you have trouble following this string processing, review the subsection entitled "String Processing" in Chapter 1.*

*This is the file Utility2.java.*

---

```

1 import java.util.Scanner;
2
3 public class StringProcessingDemo
4 {
5
6 public static void main(String[] args)
7 {
8 System.out.println("What did you eat for dinner?");
9 Scanner keyboard = new Scanner(System.in);
10 String sentence = keyboard.nextLine();
11 sentence = Utility2.censor(sentence,
12 "candy", "french fries", "salt", "beer");
13 sentence = Utility2.censor(sentence, " ,"); //Deletes extra commas
14 System.out.println("You would be healthier if you could answer:");
15 System.out.println(sentence);
16 }
17 }

```

*This is the file StringProcessingDemo.java.*

---

(continued)



## Display 6.8 String Processing Method with a Variable Number of Parameters (part 2 of 2)

## Sample Dialogue

```

What did you eat for dinner?
I ate salt cod, broccoli, french fries, salt peanuts, and apples.
You would be healthier if you could answer:
I ate cod, broccoli, peanuts, and apples.

```

can now fill the indexed variables of `a` in any order and need not fill the array starting at the first element. This violates the spirit of the `private` modifier for the array instance variable `a`. For this reason, the accessor method `getInsideArray` should return a deep copy of the array named by the private instance variable `a`. A safe definition of `getInsideArray` is the following:

```

public double[] getInsideArray() // Good version
{
 //Recall that maxNumberElements == a.length.
 double[] temp = new double[maxNumberElements];
 for (int i = 0; i < maxNumberElements; i++)
 temp[i] = a[i];
 return temp;
}

```

If a private instance variable is an array type that has a class as its base type, then you need to be sure to make copies of the class objects in the array when you make a copy of the array. This is illustrated by the toy class in Display 6.9.

Display 6.9 also includes a copy constructor. As illustrated in that display, the copy constructor should make a completely independent copy of the array instance variable (that is, a deep copy) in the same way that the accessor method does. This same point is also illustrated by the copy constructor in Display 6.5.

## Display 6.9 Accessor Method for an Array Instance Variable (part 1 of 2)

```

1 /**
2 * Demonstrates the correct way to define an accessor
3 * method to a private array of class objects.
4 */
5 public class ToyExample
6 {
7 private Date[] a;

```

*The class `Date` is defined in Display 4.11, but you do not need to know the details of the definition to understand the point of this example.*

Display 6.9 Accessor Method for an Array Instance Variable (part 2 of 2)

```

8 public ToyExample(int arraySize)
9 {
10 a = new Date[arraySize];
11 for (int i = 0; i < arraySize; i++)
12 a[i] = new Date();
13 }

14 public ToyExample(ToyExample object) ← Copy constructor for ToyExample
15 {
16 int lengthOfArrays = object.a.length;
17 this.a = new Date[lengthOfArrays];
18 for (int i = 0; i < lengthOfArrays; i++) ← Copy constructor for Date
19 this.a[i] = new Date(object.a[i]);
20 }

21 public Date[] getDateArray() ← Accessor method
22 {
23 Date[] temp = new Date[a.length];
24 for (int i = 0; i < a.length; i++)
25 temp[i] = new Date(a[i]); ← Copy constructor for Date
26 return temp;
27 }

28 }

```

<There presumably are other methods that are not shown,  
 but they are irrelevant to the point at hand.>

## Self-Test Exercises

18. Define a method named `removeAll` that can be added to the class `PartiallyFilledArray`. This method has no parameters. When invoked, the method `removeAll` deletes all the elements in its calling object.
19. Define a method named `increaseCapacity` that can be added to the class `PartiallyFilledArray` in Display 6.5. The method has one `int` parameter named `newCapacity` that increases the capacity of the `PartiallyFilledArray` so that it can hold up to `newCapacity` numbers. If `newCapacity` is less than or equal to `maxNumberOfElements`, then the method does nothing. If `newCapacity` is greater than `maxNumberOfElements`, then `maxNumberOfElements` is set equal to `newCapacity` and a new array of length `newCapacity` is created for the array instance variable `a`. The old values of the array instance variable are copied to the newly created array.

### EXAMPLE: Sorting an Array

In this example, we define a method called `sort` that will sort a partially filled array of numbers so that they are ordered from smallest to largest.

The procedure `sort` has one array parameter, `a`. The array `a` will be partially filled, so there is an additional formal parameter called `numberUsed`, which tells how many array positions are used. Thus, the heading for the method `sort` will be

```
public static void sort(double[] a, int numberUsed)
```

The method `sort` rearranges the elements in array `a` so that after the method call is completed, the elements are sorted as follows:

$$a[0] \leq a[1] \leq a[2] \leq \dots \leq a[\text{numberUsed} - 1]$$

The algorithm we use to do the sorting is called **selection sort**. It is one of the easiest of the sorting algorithms to understand.

One way to design an algorithm is to rely on the definition of the problem. In this case, the problem is to sort an array `a` from smallest to largest. That means rearranging the values so that `a[0]` is the smallest, `a[1]` the next smallest, and so forth. That definition yields an outline for the selection sort algorithm:

```
for (int index = 0; index < numberUsed; index++)
 Place the indexth smallest element in a[index]
```

There are many ways to realize this general approach. The details could be developed by using two arrays and copying the elements from one array to the other in sorted order, but using one array should be both adequate and economical. Therefore, the method `sort` uses only the one array containing the values to be sorted. The method `sort` rearranges the values in the array `a` by interchanging pairs of values. Let us go through a concrete example so that you can see how the algorithm works.

Consider the array shown in Display 6.10. The selection sort algorithm will place the smallest value in `a[0]`. The smallest value is the value in `a[4]`. So the algorithm interchanges the values of `a[0]` and `a[4]`. The algorithm then looks for the next smallest element. The value in `a[0]` is now the smallest element, so the next smallest element is the smallest of the remaining elements `a[1]`, `a[2]`, `a[3]`, ..., `a[9]`. In the example in Display 6.10, the next smallest element is in `a[6]`, so the algorithm interchanges the values of `a[1]` and `a[6]`. This positioning of the second smallest element is illustrated in the fourth and fifth array pictures in Display 6.10. The algorithm then positions the third smallest element, and so forth. As the sorting proceeds, the beginning array elements are set equal to the correct sorted values. The sorted portion of the array grows by adding elements one after the other from the elements in the unsorted end of the array. Notice that the algorithm need not do anything with the value in the last indexed variable, `a[9]`, because once the other elements are positioned correctly, `a[9]` must also have the correct value. After all, the correct value for `a[9]` is the smallest value left to be moved, and the only value left to be moved is the value that is already in `a[9]`.

EXAMPLE: (continued)

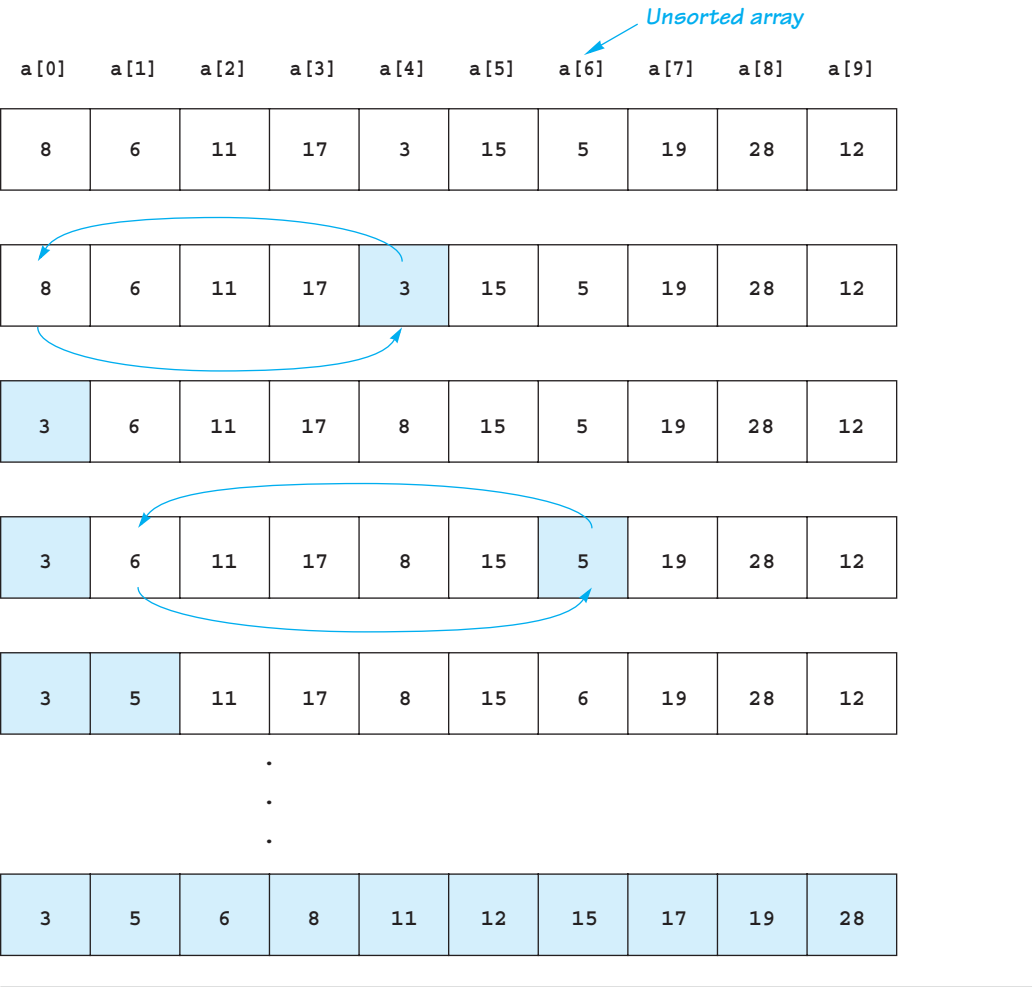
The definition of the method `sort`, included in a class, is given in Display 6.11. `sort` uses the method `indexOfSmallest` to find the index of the smallest element in the unsorted end of the array, and then it does an interchange to move this next smallest element down into the sorted part of the array.

The method `interchange`, shown in Display 6.11, is used to interchange the values of indexed variables. For example, the following call will interchange the values of `a[0]` and `a[4]`:

```
interchange(0, 4, a);
```

A sample use of the `sort` method is given in Display 6.12.

Display 6.10 Selection Sort



### Self-Test Exercises

20. How would you need to change the method `sort` in Display 6.11 so that it can sort an array of values of type `double` into decreasing order, instead of increasing order?
21. If an array of `int` values has a value that occurs twice (such as `b[0] == 42` and `b[7] == 42`) and you sort the array using the method `SelectionSort.sort`, will there be one or two copies of the repeated value after the array is sorted?

Display 6.11 Selection Sort Class (part 1 of 2)

```

1 public class SelectionSort
2 {
3 /**
4 * Precondition: numberUsed <= a.length;
5 * The first numberUsed indexed variables have values.
6 * Action: Sorts a so that a[0] <= a[1] <= ... <= a[numberUsed - 1].
7 */
8 public static void sort(double[] a, int numberUsed)
9 {
10 int index, indexOfNextSmallest;
11 for (index = 0; index < numberUsed - 1; index++)
12 { // Place the correct value in a[index]:
13 indexOfNextSmallest = indexOfSmallest(index, a, numberUsed);
14 interchange(index, indexOfNextSmallest, a);
15 // a[0] <= a[1] <= ... <= a[index] and these are the smallest
16 // of the original array elements. The remaining positions
17 // contain the rest of the original array elements.
18 }
19 }

20 /**
21 * Returns the index of the smallest value among
22 * a[startIndex], a[startIndex+1], ... a[numberUsed - 1]
23 */
24 private static int indexOfSmallest(int startIndex,
25 double[] a, int numberUsed)
26 {
27 double min = a[startIndex];
28 int indexOfMin = startIndex;
29 int index;
30 for (index = startIndex + 1; index < numberUsed; index++)
31 if (a[index] < min)
32 {
33 min = a[index];
34 indexOfMin = index;
35 // min is smallest of a[startIndex] through a[index]
36 }
37 return indexOfMin;
38 }

```

Display 6.11 Selection Sort Class (part 2 of 2)

---

```
39 /**
40 Precondition: i and j are legal indices for the array a.
41 Postcondition: Values of a[i] and a[j] have been interchanged.
42 */
43 private static void interchange(int i, int j, double[] a)
44 {
45 double temp;
46 temp = a[i];
47 a[i] = a[j];
48 a[j] = temp; //original value of a[i]
49 }
50 }
```

---

Display 6.12 Demonstration of the SelectionSort Class

---

```
1 public class SelectionSortDemo
2 {
3 public static void main(String[] args)
4 {
5 double[] b = {7.7, 5.5, 11, 3, 16, 4.4, 20, 14, 13, 42};
6
7 System.out.println("Array contents before sorting:");
8 int i;
9 for (i = 0; i < b.length; i++)
10 System.out.print(b[i] + " ");
11 System.out.println();
12 SelectionSort.sort(b, b.length);
13
14 System.out.println("Sorted array values:");
15 for (i = 0; i < b.length; i++)
16 System.out.print(b[i] + " ");
17 System.out.println();
18 }
```

---

#### Sample Dialogue

```
Array contents before sorting:
7.7 5.5 11.0 3.0 16.0 4.4 20.0 14.0 13.0 42.0
Sorted array values:
3.0 4.4 5.5 7.7 11.0 13.0 14.0 16.0 20.0 42.0
```

---

enumerated  
type

## Enumerated Types ★

Sometimes you need a simple type consisting of a short list of named values. For example, the values might be clothing sizes, the days of the week, or some other brief list. Starting with version 5.0, Java allows you to have such an **enumerated type**. For example, the following is an enumerated type for the days of a five-day work week:

```
enum WorkDay {MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY};
```

A value of an enumerated type is a kind of named constant and so, by convention, is spelled with all uppercase letters. So, we used, for example, `MONDAY`, not `Monday`, in the above definition of the enumerated type `WorkDay`. Using `Monday` would have been legal, but poor style.

As with any other type, you can have variables of an enumerated type; for example,

```
WorkDay meetingDay, availableDay;
```

A variable of an enumerated type can have a value that must be either one of the values listed in the definition of the type or else the special value `null`, which serves as a placeholder indicating that the variable has no “real” value. For example, you can set the value of a variable of an enumerated type with an assignment statement, as follows:

```
meetingDay = WorkDay.THURSDAY;
```

Note that when you write the value of an enumerated type, you need to preface the name of the value, such as `THURSDAY`, with the name of the type. For example, you use `WorkDay.THURSDAY`, not `THURSDAY`.

As with any other type, you can combine the declaration of a variable with the assignment of a value to the variable, as in

```
WorkDay meetingDay = WorkDay.THURSDAY;
```

A program that demonstrates the syntax for using enumerated types is given in Display 6.13. Be sure to notice that we placed the definition of the enumerated type outside of the `main` method in the same place that you would give named constants.

Display 6.13 An Enumerated Type (part 1 of 2)

```
1 public class EnumDemo
2 {
3 enum WorkDay {MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY};
4
5 public static void main(String[] args)
6 {
7 WorkDay startDay = WorkDay.MONDAY;
7 WorkDay endDay = WorkDay.FRIDAY;
```

## Display 6.13 An Enumerated Type (part 2 of 2)

```
8 System.out.println("Work starts on " + startDay);
9 System.out.println("Work ends on " + endDay);
10 }
11 }
```

## Sample Dialogue

```
Work starts on MONDAY
Work ends on FRIDAY
```

### Enumerated Type

An **enumerated type** is a type for which you give all the values of the type in a typically short list. A value of an enumerated type is a kind of named constant and so, by convention, is spelled with all uppercase letters.

#### SYNTAX

```
enum Type_Name {FIRST_VALUE, SECOND_VALUE, ..., LAST_VALUE};
```

Starting with version 5.0, `enum` is a keyword in Java.

#### EXAMPLE

```
enum Flavor {VANILLA, CHOCOLATE, STRAWBERRY};
```

The definition of an enumerated type is normally placed outside of all methods in the same place that you give named constants. The location for an enumerated type definition is illustrated in Display 6.13. (The definition can be placed in other locations, but we will not need to place them anywhere else.)

You can output the value of a variable of an enumerated type using `println`. For example,

```
System.out.println(WorkDay.THURSDAY);
```

will produce the following screen output:

```
THURSDAY
```

Note that the type name `WorkDay` is not output. Other examples of outputting an enumerated type value are given in Display 6.13.



The values of an enumerated type, such as `WorkDay.THURSDAY`, are not `String` values. In fact, you should not care what kind of values they are. How they are implemented is not relevant to being able to use the values of an enumerated type. All you really need to know is that, for example, `WorkDay.THURSDAY` and `WorkDay.FRIDAY` are different values and will test as being different if you compare them with `==`.

Although values of an enumerated type are not `String` values, they are used for tasks that could be done by `String` values; however, enumerated types work better than `String` values for some tasks. You could use a `String` variable in place of a variable of an enumerated type. For example, you could use

```
String meetingDay = "THURSDAY";
```

instead of

```
WorkDay meetingDay = WorkDay.THURSDAY;
```

However, using a `String` variable allows for the possibility of setting the variable equal to a nonsense value, such as `"SUNDAY"` or `"GaGa"` for a work day, and to do so without the computer issuing any warning statement. With an enumerated type, you know the only possible values for a variable of that type are the values given in the enumerated type definition; if you try to give the variable a different value, you will get an error message.

An enumerated type is actually a class, and its values are objects of the class. Some methods that are automatically provided with every enumerated type are given in Display 6.14.

Display 6.14 Some Methods Included with Every Enumerated Type (part 1 of 2)

```
public boolean equals (Any_Value_Of_An_Enumerated_Type)
```

Returns `true` if its argument is the same value as the calling value. While it is perfectly legal to use `equals`, it is easier and more common to use `==`.

#### EXAMPLE

For enumerated types, `(Value1.equals(Value2))` is equivalent to `(Value1 == Value2)`.

```
public String toString()
```

Returns the calling value as a string. This is often invoked automatically. For example, this method is invoked automatically when you output a value of the enumerated type using `System.out.println` or when you concatenate a value of the enumerated type to a string. See Display 6.15 for an example of this automatic invocation.

#### EXAMPLE

`WorkDay.MONDAY.toString( )` returns `"MONDAY"`. The enumerated type `WorkDay` is defined in Display 6.13.

## Display 6.14 Some Methods Included with Every Enumerated Type (part 2 of 2)

```
public int ordinal()
```

Returns the position of the calling value in the list of enumerated type values. The first position is 0.

**EXAMPLE**

`WorkDay.MONDAY.ordinal( )` returns 0, `WorkDay.TUESDAY.ordinal( )` returns 1, and so forth. The enumerated type `WorkDay` is defined in Display 6.13.

```
public int compareTo(Any_Value_Of_The_Enumerated_Type)
```

Returns a negative value if the calling object precedes the argument in the list of values, returns 0 if the calling object equals the argument, and returns a positive value if the argument precedes the calling object.

**EXAMPLE**

`WorkDay.TUESDAY.compareTo(WorkDay.THURSDAY)` returns a negative value. The type `WorkDay` is defined in Display 6.13.

```
public EnumeratedType [] values()
```

Returns an array whose elements are the values of the enumerated type in the order in which they are listed in the definition of the enumerated type.

**EXAMPLE**

See Display 6.15.

```
public static EnumeratedType valueOf(String name)
```

Returns the enumerated type value with the specified name. The string name must be an exact match.

**EXAMPLE**

`WorkDay.valueOf("THURSDAY")` returns `WorkDay.THURSDAY`. The type `WorkDay` is defined in Display 6.13.

When comparing two variables (or constants) of an enumerated type, you can use the `equals` method, but it is more common to instead use the `==` operator. For example,

```
if (meetingDay == availableDay)
 System.out.println("Meeting will be on schedule.");
if (meetingDay == WorkDay.THURSDAY)
 System.out.println("Long weekend!");
```

With enumerated types, the `equals` method and the `==` operator are equivalent, but the `==` operator has nicer syntax.

To get the full potential from an enumerated type, you need some way to cycle through all its values. The static method `values()` provides you with that ability. This method returns an array whose elements are the values of the enumerated type, and is provided automatically for every enumerated type. Display 6.15 gives a simple example of using the method `values()` to cycle through all the values in an enumerated type. (This is one situation where it is much cleaner to use a for-each loop instead of an ordinary for loop. If you have read the starred section on the for-each loop, be sure to do Self-Test Exercise 22, which redoes Display 6.15 using a for-each loop.)

### The `values` Method

Every enumerated type has a static method named `values()`, which returns an array whose elements are the values of the enumerated type in the order in which they are listed in the definition of the enumerated type. The base type for the array returned is the enumerated type. See Display 6.15 for an example.

Display 6.15 The Method `values` (part 1 of 2)

```

1 import java.util.Scanner;
2
3 public class EnumValuesDemo
4 {
5
6 enum WorkDay {MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY};
7
8 public static void main(String[] args)
9 {
10 WorkDay[] day = WorkDay.values();
11
12 Scanner keyboard = new Scanner(System.in);
13 double hours = 0, sum = 0;
14
15 for (int i = 0; i < day.length; i++)
16 {
17 System.out.println("Enter hours worked for " + day[i]);
18 hours = keyboard.nextDouble();
19 sum = sum + hours;
20 }
21
22 System.out.println("Total hours work = " + sum);
23 }
24 }
```

*This is equivalent to `day[i].toString()`.*



Display 6.15 The Method `values` (part 2 of 2)

## Sample Dialogue

```
Enter hours worked for MONDAY
8
Enter hours worked for TUESDAY
8
Enter hours worked for WEDNESDAY
8
Enter hours worked for THURSDAY
8
Enter hours worked for FRIDAY
7.5
Total hours worked = 39.5
```

**TIP: Enumerated Types in `switch` Statements ★**

You can use an enumerated type to control a `switch` statement. In other words, the type of the controlling expression in a `switch` statement can be an enumerated type. This is illustrated in Display 6.16. Note that the case labels must be unqualified names; use `VANILLA`, not `Flavor.VANILLA`.

This program uses the static method `valueOf` to convert an input string to a value of the enumerated type. For example,

```
Flavor.valueOf("STRAWBERRY")
```

returns `Flavor.STRAWBERRY`. Note that the program changes the input to all uppercase letters before giving it as an argument to the method `valueOf`. The method `valueOf` requires an exact match. An invocation of `Flavor.valueOf("Vanilla")` will end your program with an error message;<sup>5</sup> you must use `"VANILLA"` to match the exact spelling (including upper- versus lowercase) of the value in `Flavor`.

At this point, you may wonder what the difference is between `STRAWBERRY` and `Flavor.STRAWBERRY` and how to tell which one to use in a given situation. The value of the enumerated type is `STRAWBERRY`. We write `Flavor.STRAWBERRY` to say we mean `STRAWBERRY` as defined in `Flavor`, as opposed to `STRAWBERRY` as defined in some other type, such as

```
enum Berry {STRAWBERRY, BLUEBERRY, RASPBERRY};
```

(continued)

<sup>5</sup>After you cover exceptions in Chapter 9, you will be able to cope with answers such as `PISTACHIO` that do not correspond to any value of type `Flavor`. An invocation of `Flavor.valueOf("PISTACHIO")` will throw an *IllegalArgumentException*, something explained in Chapter 9. Until then, your program will simply give an error message when `valueOf` cannot cope with its argument.

**TIP: (continued)**

A single program with both type definitions (Flavor and Berry) could use both `Flavor.STRAWBERRY` and `Berry.STRAWBERRY`.

So, when can you use `STRAWBERRY` instead of `Flavor.STRAWBERRY`? The approximate answer is when there is enough context for the compiler to know `STRAWBERRY` means `STRAWBERRY` as defined in the type `Flavor`. For example, in a `switch` statement, if the type of the controlling expression is `Flavor`, then `STRAWBERRY` can only mean `Flavor.STRAWBERRY`. This rule will help in remembering when to use `STRAWBERRY` and when to use `Flavor.STRAWBERRY`. But, sometimes you may simply have to check a reference or try the two possibilities out and see which one (or ones) the compiler accepts. ■

Display 6.16 Enumerated Type in a `switch` Statement (part 1 of 2)

```

1 import java.util.Scanner;
2
3 public class EnumSwitchDemo
4 {
5 enum Flavor {VANILLA, CHOCOLATE, STRAWBERRY};

6 public static void main(String[] args)
7 {
8 Flavor favorite = null;
9 Scanner keyboard = new Scanner(System.in);

10 System.out.println("What is your favorite flavor?");
11 String answer = keyboard.next();
12 answer = answer.toUpperCase();
13 favorite = Flavor.valueOf(answer);

14 switch (favorite)
15 {
16 case VANILLA:
17 System.out.println("Classic");
18 break;
19 case CHOCOLATE:
20 System.out.println("Rich");
21 break;
22 default:
23 System.out.println("I bet you said STRAWBERRY.");
24 break;
25 }
26 }
27 }
```

*The case labels must have just the name of the value without the type name and dot.*

Display 6.16 Enumerated Type in a `switch` Statement (part 2 of 2)

## Sample Dialogue

```
What is your favorite flavor?
Vanilla
Classic
```

## Sample Dialogue

```
What is your favorite flavor?
STRAWBERRY
I bet you said STRAWBERRY.
```

## Sample Dialogue

```
What is your favorite flavor?
PISTACHIO
```

*This input causes the program to end and issue an error message.*

## Self-Test Exercise

22. Rewrite the program in Display 6.15 using a for-each loop.

## 6.4 Multidimensional Arrays

*Two indices are better than one.*

ANONYMOUS

Java allows you to declare arrays with more than one index. In this section, we describe these multidimensional arrays.

### Multidimensional Array Basics

It is sometimes useful to have an array with more than one index, and this is allowed in Java. The following creates an array of characters called `page`. The array `page` has two indices, the first index ranging from 0 to 29 and the second from 0 to 99.

```
char[] [] page = new char[30][100];
```

This is equivalent to the following two steps:

```
char[] [] page;
page = new char[30][100];
```

array  
declarations

**indexed  
variables**

The **indexed variables** for this array each have two indices. For example, `page[0][0]`, `page[15][32]`, and `page[29][99]` are three of the indexed variables for this array. Note that each index must be enclosed in its own set of square brackets. As was true of the one-dimensional arrays we have already seen, each indexed variable for a multidimensional array is a variable of the base type—in this case, the type `char`.

An array may have any number of indices, but perhaps the most common number is two. A two-dimensional array can be visualized as a two-dimensional display with the first index giving the row and the second index giving the column. For example, the array indexed variables of the two-dimensional array `a`, declared and created as

```
char[] [] a = new char[5][12];
```

can be visualized as follows:

```
a[0][0], a[0][1], a[0][2], ..., a[0][11]
a[1][0], a[1][1], a[1][2], ..., a[1][11]
a[2][0], a[2][1], a[2][2], ..., a[2][11]
a[3][0], a[3][1], a[3][2], ..., a[3][11]
a[4][0], a[4][1], a[4][2], ..., a[4][11]
```

You might use the array `a` to store all the characters on a (very small) page of text that has 5 lines (numbered 0 through 4) and 12 characters on each line (numbered 0 through 11).

### Declaring and Creating a Multidimensional Array

You declare a multidimensional array variable and create a multidimensional array object in basically the same way that you create and name a one-dimensional array. You simply use as many square brackets as there are indices.

#### SYNTAX

```
Base_Type [] ... [] Variable_Name = new Base_Type [Length_1]...[Length_n];
```

#### EXAMPLES

```
char[] [] a = new char[5][12];
char[] [] page = new char[30][100];
double[] [] table = new double[100][10];
int[] [] [] figure = new int[10][20][30];
Person[] [] entry = new Person[10][10];
```

`Person` is a class.

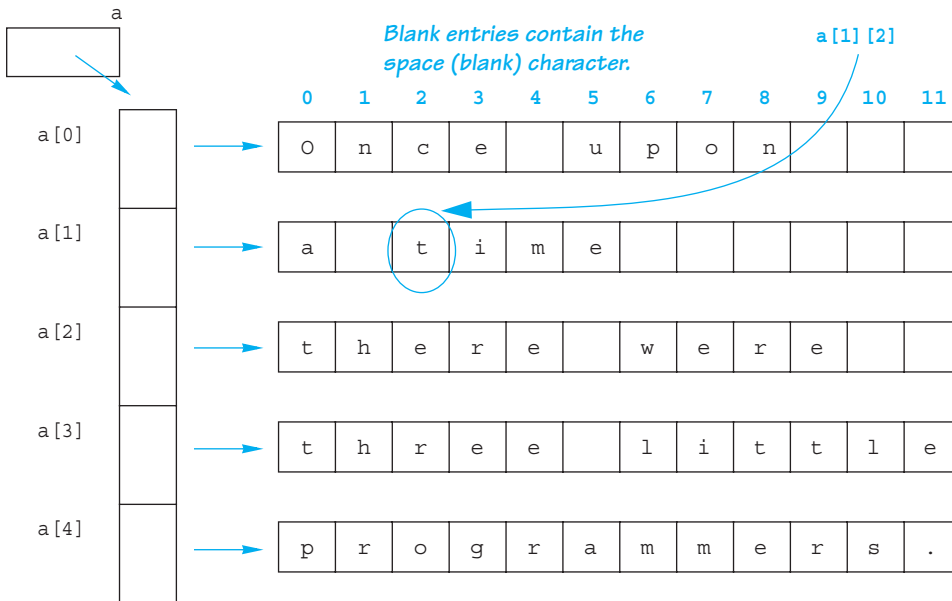
**a multi-  
dimensional  
array is an  
array of  
arrays**

In Java, a two-dimensional array, such as `a`, is actually an array of arrays. The above array `a` is actually a one-dimensional array of size 5, whose base type is a one-dimensional array of characters of size 12. This is diagrammed in Display 6.17. As shown in that display, the array variable `a` contains a reference to a one-dimensional array of length 5 with a base type of `char[]`; that is, the base type of `a` is the type for an entire one-dimensional array of characters. Each indexed variable `a[0]`, `a[1]`, and so forth contains a reference to a one-dimensional array of characters.

Display 6.17 Two-Dimensional Array as an Array of Arrays

```
char[] [] a = new char[5][12];
```

*Code that fills the array is not shown.*



```
int row, column;
for (row = 0; row < 5; row++)
{
 for (column = 0; column < 12; column++)
 System.out.print(a[row][column]);
 System.out.println();
}
```

*We will see that these can and should be replaced with expressions involving the length instance variable.*

*Produces the following output:*

```
Once upon
a time
there were
three little
programmers.
```

A three-dimensional array is an array of arrays of arrays, and so forth for higher dimensions.

Normally, the fact that a two-dimensional array is an array of arrays need not concern you, and you can usually act as if the array *a* is actually an array with two indices (rather than an array of arrays, which is harder to keep track of). There are, however, some



situations where a two-dimensional array looks very much like an array of arrays. For example, you will see that when using the instance variable `length`, you must think of a two-dimensional array as an array of arrays.

## Using the `length` Instance Variable

Suppose you want to fill all the elements in the following two-dimensional array with 'Z':

```
char[] [] page = new char[30][100];
```

You can use a nested `for` loop such as the following:

```
int row, column;
for (row = 0; row < page.length; row++)
 for (column = 0; column < page[row].length; column++)
 page[row][column] = 'Z';
```

Let's analyze this nested `for` loop in a bit more detail. The array `page` is actually a one-dimensional array of length 30, and each of the 30 indexed variables `page[0]` through `page[29]` is a one-dimensional array with base type `char` and with a length of 100. That is why the first `for` loop is terminated using `page.length`. For a two-dimensional array such as `page`, the value of `length` is the number of first indices or, equivalently, the number of rows—in this case, 30. Now let's consider the inside `for` loop.

The 0<sup>th</sup> row in the two-dimensional array `page` is the one-dimensional array `page[0]`, and it has `page[0].length` entries. More generally, `page[row]` is a one-dimensional array of chars, and it has `page[row].length` entries. This is why the inner `for` loop is terminated using `page[row].length`. Of course, in this case, `page[0].length`, `page[1].length`, and so forth through to `page[29].length` are all equal and all equal to 100. (If you read the optional section entitled “Ragged Arrays,” which follows this section, you will see that these need not all be equal.)

## Self-Test Exercise

23. What is the output produced by the following code?

```
int[] [] myArray = new int[4][4];
int index1, index2;
for (index1 = 0; index1 < myArray.length; index1++)
 for (index2 = 0;
 index2 < myArray[index1].length; index2++)
 myArray[index1][index2] = index2;
for (index1 = 0; index1 < myArray.length; index1++)
{
 for (index2 = 0;
 index2 < myArray[index1].length; index2++)
 System.out.print(myArray[index1][index2] + " ");
 System.out.println();
}
```

## ragged arrays

## Ragged Arrays ★

Most programmers typically create a two-dimensional array with the same number of entries for each row. However, it is possible for different rows to have a different numbers of columns. These sorts of arrays are called **ragged arrays**.

To help explain the details, let's start with an ordinary, nonragged two-dimensional array, created as follows:

```
double[] [] a = new double[3][5];
```

This is equivalent to the following:

```
double[] [] a;
a = new double[3] [];
a[0] = new double[5];
a[1] = new double[5];
a[2] = new double[5];
```

The line

```
a = new double[3] [];
```

makes `a` the name of an array with room for three entries, each of which can be an array of `doubles` that can be of any length. The next three lines each create an array of `doubles` of length 5 to be named by `a[0]`, `a[1]`, and `a[2]`. The net result is a two-dimensional array of base type `double` with three rows and five columns.

If you want, you can make each of `a[0]`, `a[1]`, and `a[2]` a different length. The following code makes a ragged array `b` in which each row has a different length:

```
double[] [] b;
b = new double[3] [];
b[0] = new double[5];
b[1] = new double[10];
b[2] = new double[4];
```

There are situations in which you can profitably use ragged arrays, but most applications do not require them. However, if you understand ragged arrays, you will have a better understanding of how all multidimensional arrays work in Java.

## Multidimensional Array Parameters and Returned Values

array  
arguments

Methods may have multidimensional array parameters and may have a multidimensional array type as the type for the value returned. The situation is similar to that of the one-dimensional case, except that you use more square brackets when specifying the type

name. For example, the following method will display a two-dimensional array in the usual way as rows and columns:<sup>6</sup>

```
public static void showMatrix(int[] [] a)
{
 int row, column;
 for (row = 0; row < a.length; row++)
 {
 for (column = 0; column < a[row].length; column++)
 System.out.print(a[row][column] + " ");
 System.out.println();
 }
}
```

### returning an array

If you want to return a multidimensional array, you use the same kind of type specification as you use for a multidimensional array parameter. For example, the following method returns a two-dimensional array with base type double:

```
/**
 * Precondition: Each dimension of a is at least the value of size.
 * The array returned is the same as the size-by-size upper upper-
 * left corner of the array a.
 */
public static double[] [] corner(double[] [] a, int size)
{
 double[] [] temp = new double[size][size];
 int row, column;
 for (row = 0; row < size; row++)
 for (column = 0; column < size; column++)
 temp[row][column] = a[row][column];
 return temp;
}
```

## EXAMPLE: A Grade Book Class

Display 6.18 contains a class for grade records in a class whose only recorded scores are quiz scores. An object of this class has three array instance variables. One is a two-dimensional array named `grade` that records the grade of each student on each quiz. For example, the score that student number 4 received on quiz number 1 is recorded in `grade[3][0]`. Because the student numbers and quiz numbers start with 1 and the array indices start with 0, we subtract one from the student number or quiz number to obtain the corresponding array index.

<sup>6</sup>It is worth noting that this method works fine for ragged arrays.

**EXAMPLE:** (continued)

All the raw data is in the array `grade`, but two other arrays hold computed data. The array `studentAverage` is used to record the average quiz score for each of the students. For example, the program sets `studentAverage[0]` equal to the average of the quiz scores received by student 1, `studentAverage[1]` equal to the average of the quiz scores received by student 2, and so forth. The array `quizAverage` is used to record the average score for each quiz. For example, the program sets `quizAverage[0]` equal to the average of all the student scores for quiz 1, `quizAverage[1]` records the average score for quiz 2, and so forth. Display 6.19 illustrates the relationship between the arrays `grade`, `studentAverage`, and `quizAverage`. In that display, we have shown some sample data for the array `grade`. The data in `grade`, in turn, determines the values that are stored in `studentAverage` and in `quizAverage`. Display 6.19 also shows these computed values for `studentAverage` and `quizAverage`. The two arrays `studentAverage` and `quizAverage` are created and filled by the constructor that creates the `GradeBook` object. (The constructors do this by calling private helping methods.)

The no-argument constructor for the class `GradeBook` obtains the data for the array instance variable `grade` via a dialogue with the user. Although this is not my favorite way to define a no-argument constructor, some programmers like it, and you should see an example of it. Another alternative would be to have a no-argument constructor that essentially does nothing and then have an input method that sets all the instance variables, including creating the array objects.

A very simple demonstration program along with the dialogue it produces is given in Display 6.20.

Display 6.18 A Grade Book Class (part 1 of 4)

```

1 import java.util.Scanner;

2 public class GradeBook
3 {

4 private int numberOfStudents; // Same as studentAverage.length.
5 private int numberOfQuizzes; // Same as quizAverage.length.

6 private int[] [] grade; //numberOfStudents rows and numberOfQuizzes
 //columns.
7 private double[] studentAverage;
8 private double[] quizAverage;

9 public GradeBook(int[] [] a)
10 {
11 if (a.length == 0 || a[0].length == 0)
12 {
13 System.out.println("Empty grade records. Aborting.");
14 System.exit(0);
15 }

```

(continued)

Display 6.18 A Grade Book Class (part 2 of 4)

---

```

16 numberOfStudents = a.length;
17 numberOfQuizzes = a[0].length;
18 fillGrade(a);
19 fillStudentAverage();
20 fillQuizAverage();
21 }

22 public GradeBook(GradeBook book)
23 {
24 numberOfStudents = book.numberOfStudents;
25 numberOfQuizzes = book.numberOfQuizzes;
26 fillGrade(book.grade);
27 fillStudentAverage();
28 fillQuizAverage();
29 }

30 public GradeBook()
31 {
32 Scanner keyboard = new Scanner(System.in);

33 System.out.println("Enter number of students:");
34 numberOfStudents = keyboard.nextInt();

35 System.out.println("Enter number of quizzes:");
36 numberOfQuizzes = keyboard.nextInt();

37 grade = new int[numberOfStudents][numberOfQuizzes];
38 for (int studentNumber = 1;
39 studentNumber <= numberOfStudents; studentNumber++)
40 for (int quizNumber = 1;
41 quizNumber <= numberOfQuizzes; quizNumber++)
42 {
43 System.out.println("Enter score for student number "
44 + studentNumber);
45 System.out.println("on quiz number " + quizNumber);
46 grade[studentNumber - 1][quizNumber - 1] =
47 keyboard.nextInt();
48 }

49 fillStudentAverage();
50 fillQuizAverage();
51 }

52 private void fillGrade(int[][] a)
53 {
54 grade = new int[numberOfStudents][numberOfQuizzes];
55 for (int studentNumber = 1;
56 studentNumber <= numberOfStudents; studentNumber++)
57 {

```

*This class should have more accessor and mutator methods, but we have omitted them to save space. See Self-Test Exercises 24 through 27.*

Display 6.18 A Grade Book Class (part 3 of 4)

```

58 for (int quizNumber = 1;
59 quizNumber <= numberOfQuizzes; quizNumber++)
60 grade[studentNumber-1][quizNumber-1] =
61 a[studentNumber-1][quizNumber-1];
62 }
63 }

64 /**
65 * Fills the array studentAverage using the data from the array grade.
66 */
67 private void fillStudentAverage()
68 {
69 studentAverage = new double[numberOfStudents];

70 for (int studentNumber = 1;
71 studentNumber <= numberOfStudents; studentNumber++)
72 { //Process one studentNumber:
73 double sum = 0;
74 for (int quizNumber = 1;
75 quizNumber <= numberOfQuizzes; quizNumber++)
76 sum = sum + grade[studentNumber - 1][quizNumber - 1];
77 //sum contains the sum of the quiz scores for student number
78 //studentNumber.
79 studentAverage[studentNumber - 1] = sum/numberOfQuizzes;
80 //Average for student studentNumber is
81 //studentAverage[studentNumber - 1]
82 }
83 }
84 /**
85 * Fills the array quizAverage using the data from the array grade.
86 */
87 private void fillQuizAverage()
88 {
89 quizAverage = new double[numberOfQuizzes];

90 for (int quizNumber = 1; quizNumber <= numberOfQuizzes; quizNumber++)
91 { //Process one quiz (for all students):
92 double sum = 0;
93 for (int studentNumber = 1;
94 studentNumber <= numberOfStudents;
95 studentNumber++)
96 sum = sum + grade[studentNumber - 1][quizNumber - 1];
97 //sum contains the sum of all student scores on quiz number
98 //quizNumber.
99 quizAverage[quizNumber - 1] = sum/numberOfStudents;
100 //Average for quiz quizNumber is the value of
101 //quizAverage[quizNumber - 1]
102 }
103 }

```

(continued)

Display 6.18 A Grade Book Class (part 4 of 4)

```

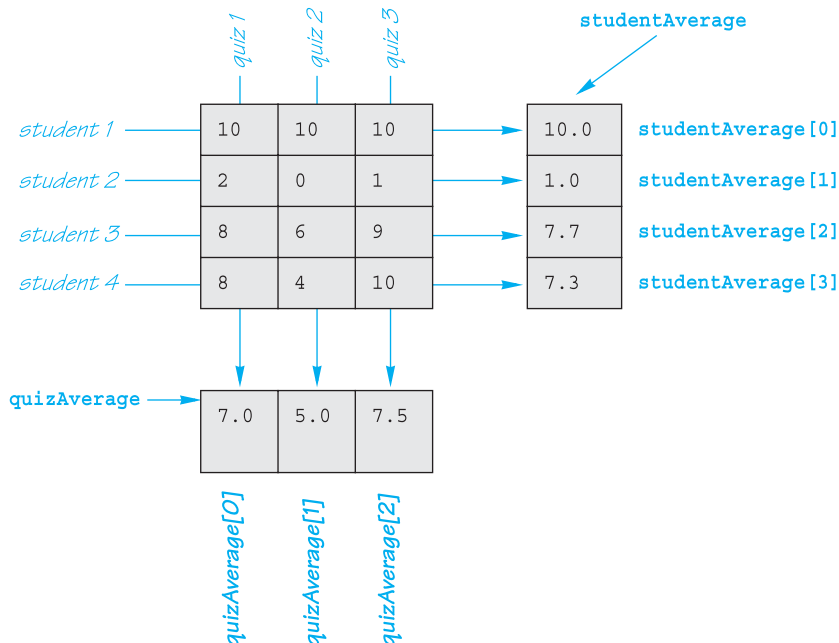
99 public void display()
100 {
101 for (int studentNumber = 1;
102 studentNumber <= numberOfStudents; studentNumber++)

103 {//Display for one studentNumber:
104 System.out.print("Student " + studentNumber + " Quizzes: ");
105 for (int quizNumber = 1;
106 quizNumber <= numberOfQuizzes; quizNumber++)
107 System.out.print(grade[studentNumber - 1][quizNumber - 1] +
108 " ");
109 System.out.println(" Ave = " +
110 studentAverage[studentNumber - 1]);
111 }

112 System.out.println("Quiz averages: ");
113 for (int quizNumber = 1; quizNumber <= numberOfQuizzes;
114 quizNumber++)
115 System.out.print("Quiz " + quizNumber
116 + " Ave = " + quizAverage[quizNumber - 1] +
117 " ");
118 System.out.println();
119 }

```

Display 6.19 The Two-Dimensional Array grade



### Display 6.20 Demonstration of the Class GradeBook

```

1 public class GradeBookDemo
2 {
3 public static void main(String[] args)
4 {
5 GradeBook book = new GradeBook();
6 book.display();
7 }
8 }

```

#### Sample Dialogue

```

Enter number of students:
4
Enter number of quizzes:
3
Enter score for student number 1
on quiz number 1
10
Enter score for student number 1
on quiz number 2
10

 <The rest of the input dialogue is omitted to save space.>

Student 1 Quizzes: 10 10 10 Ave = 10.0
Student 2 Quizzes: 2 0 1 Ave = 1.0
Student 3 Quizzes: 8 6 9 Ave = 7.666666666667
Student 4 Quizzes: 8 4 10 Ave = 7.333333333333
Quiz averages:
Quiz 1 Ave = 7.0 Quiz 2 Ave = 5.0 Quiz 3 Ave = 7.5

```

### Self-Test Exercises

24. Write a method definition for a method with the following heading. The method is to be added to the class `GradeBook` in Display 6.18.

```

/**
 * Returns the grade that student numbered studentNumber received
 * on quiz number quizNumber.
 */
public int getGrade(int studentNumber, int quizNumber)

```

(continued)



**Self-Test Exercises** (continued)

25. Write a method definition for a method with the following heading. The method is to be added to the class `GradeBook` in Display 6.18.

```
/**
 * Changes the grade for student number studentNumber on quiz
 * number quizNumber to newGrade.
 */
public void changeGrade(int studentNumber,
 int quizNumber, int newGrade)
```

26. Write a method definition for a method with the following heading. The method is to be added to the class `GradeBook` in Display 6.18.

```
/**
 * Returns an array with the average quiz score for each student.
 */
public double[] getStudentAverages()
```

27. Write a method definition for a method with the following heading. The method is to be added to the class `GradeBook` in Display 6.18.

```
/**
 * Returns an array with the average score for each quiz.
 */
public double[] getQuizAverages()
```

**Chapter Summary**

- An *array* can be used to store and manipulate a collection of data that is all of the same type.
- The *indexed variables* of an array can be used just like any other variables of the *base type* of the array.
- Arrays are objects that are created with `new` just like the class objects we discussed before this chapter (although there is a slight difference in the syntax used).
- A `for` loop is a good way to step through the elements of an array and perform some program action on each indexed variable.
- The most common programming error made when using arrays is to attempt to access a nonexistent array index. Always check the first and last iterations of a loop that manipulates an array to make sure it does not use an index that is illegally small or illegally large.

- The indexed variables of an array can be used as an argument to be plugged in for a parameter of the array's base type.
- A method can have parameters of an array type. When the method is invoked, an entire array is plugged in for the array parameter.
- A method may return an array as the value returned by the method.
- When using a *partially filled array*, your program needs an additional variable of type `int` to keep track of how much of the array is being used.
- An instance variable of a class can be of an array type.
- If you need an array with more than one index, you can use a multidimensional array, which is actually an array of arrays.

## Answers to Self-Test Exercises

- word
  - String
  - 5
  - 0 through 4 inclusive
  - Any of the following would be correct:  
`word[0], word[1], word[2], word[3], word[4]`
- 10
  - 0
  - 9
- a, b, c,
- 1.1 2.2 3.3  
1.1 3.3 3.3
- The for loop uses indices 1 through `sampleArray.length`, but the correct indices are 0 through `sampleArray.length - 1`. The last index, `sampleArray.length`, is out of bounds. What was probably intended is the following:  

```
int[] sampleArray = new int[10];
for (int index = 0; index < sampleArray.length; index++)
 sampleArray[index] = 3*index;
```
- The last value of `index` is `a.length - 1`, which is the last index of the array. However, when `index` has the value `a.length - 1`, `a[index + 1]` has an index that is out of bounds because `index + 1` is one more than the largest array index. The for loop ending condition should instead be `index < a.length - 1`.

7. `SomeClass.doSomething(number); //Legal.`  
`SomeClass.doSomething(a[2]); //Legal.`  
`SomeClass.doSomething(a[3]); //Illegal. Index out of bounds.`  
`SomeClass.doSomething(a[number]); //Legal.`  
`SomeClass.doSomething(a); //Illegal.`
8. `public static void oneMore(int[] a)`  
`{`  
`for (int i = 0; i < a.length; i++)`  
`a[i] = a[i] + 1;`  
`}`
9. `public static int outOfOrder(double[] a)`  
`{`  
`for (int index = 0; index < a.length - 1; index++)`  
`if (a[index] > a[index + 1])`  
`return (index + 1);`  
`return - 1;`  
`}`
10. This method is legal but pointless. When invoked, it has no effect on its argument. The parameter `a` is a local variable that contains a reference. The reference does indeed get changed so it refers to an array of double the size of the argument, but that reference goes away when the method ends. A method can change the values of the indexed variables of its argument, but it cannot change the reference in the array variable used as an argument.
11. `public static double[] halfArray(double[] a)`  
`{`  
`double[] temp = new double[a.length];`  
`for (int i = 0; i < a.length; i++)`  
`temp[i] = a[i]/2.0;`  
`return temp;`  
`}`
12. The method will compile and run. The array returned has the correct values for its elements. However, the method will change the values of its array argument. If you want to change the values in the array argument, a `void` method would make more sense. If you want to return an array, you should probably return a new array (as in the version in the previous subsection), not return a changed version of the argument array.
13. `/**`  
`Precondition: numberUsed <= argumentArray.length;`  
`the first numberUsed indexed variables of argumentArray`  
`have values.`  
`Returns an array of length numberUsed whose ith element`  
`is argumentArray[i] - adjustment.`  
`*/`

```
public static double[] differenceArray(
 double[] argumentArray, int numberUsed, double adjustment)
{
 double[] temp = new double[numberUsed];
 for (int i = 0; i < numberUsed; i++)
 temp[i] = argumentArray[i] - adjustment;
 return temp;
}
```

14. The only changes are to add the method `differenceArray` and to rewrite the method `showDifference` as follows (the complete class definition is in the file `GolfScoresExercise.java` on the accompanying website):

```
public static void showDifference(double[] a,
 int numberUsed)
{
 double average = computeAverage(a, numberUsed);
 System.out.println("Average of the " + numberUsed
 + " scores = " + average);

 double[] difference =
 differenceArray(a, numberUsed, average);
 System.out.println("The scores are:");
 for (int index = 0; index < numberUsed; index++)
 System.out.println(a[index] +
 " differs from average by "
 + difference[index]);
}
```

15. The main differences are to remove parameters, replace the array name `a` by `score`, and make the method `fillArray` a void method. This code is in the file `GolfScoresStaticExercise.java` on the accompanying website.

```
import java.util.Scanner;

public class GolfScoresStaticExercise
{
 public static final int MAX_NUMBER_SCORES = 10;
 private static double[] score =
 new double[MAX_NUMBER_SCORES];
 private static int numberUsed = 0;

 /**
 * Shows differences between each of a list of golf scores
 * and their average.
 */
 public static void main(String[] args)
 {
```

extra code  
on website

extra code  
on website

```

 System.out.println(
 "This program reads golf scores and shows");
 System.out.println(
 "how much each differs from the average.");

 System.out.println("Enter golf scores:");
 fillArray();
 showDifference();
 }

 /**
 * Reads values into the array score.
 */
 public static void fillArray()
 {
 System.out.println("Enter up to " + score.length
 + " nonnegative numbers:");
 System.out.println(
 "Mark the end of the list with a negative number.");
 Scanner keyboard = new Scanner(System.in);

 double next;
 int index = 0;
 next = keyboard.nextDouble();
 while ((next >= 0) && (index < score.length))
 {
 score[index] = next;
 index++;
 next = keyboard.nextDouble();
 //index is the number of
 //array indexed variables used so far.
 }
 //index is the total number of array indexed variables used.

 if (next >= 0)
 System.out.println("Could only read in "
 + score.length + " input values.");

 numberUsed = index;
 }

 /**
 * Precondition: numberUsed <= score.length.
 * score[0] through score[numberUsed-1] have values.
 * Returns the average of numbers ascore[0] through
 * score[numberUsed-1].
 */

```

```

public static double computeAverage()
{
 double total = 0;
 for (int index = 0; index < numberUsed; index++)
 total = total + score[index];
 if (numberUsed > 0)
 {
 return (total/numberUsed);
 }
 else
 {
 System.out.println(
 "ERROR: Trying to average 0 numbers.");
 System.out.println("computeAverage returns 0.");
 return 0;
 }
}

/**
Precondition: numberUsed <= score.length.
The first numberUsed indexed variables of score have values.
Postcondition: Gives screen output showing how much each of the
first numberUsed elements of the array a differs from the average.
*/
public static void showDifference()
{
 double average = computeAverage();
 System.out.println("Average of the " + numberUsed
 + " scores = " + average);
 System.out.println("The scores are:");
 for (int index = 0; index < numberUsed; index++)
 System.out.println(score[index] +
 " differs from average by "
 + (score[index] - average));
}
}

16. public static int max(int... arg)
 {
 if (arg.length == 0)
 {
 System.out.println(
 "Fatal Error: maximum of zero values.");
 System.exit(0);
 }
 }

```

```

int largest = Integer.MIN_VALUE;
for (int element : arg)
 if (element > largest)
 largest = element;
return largest;
}

```

17. The last line would change to the following:

I ate cod, broccoli, , peanuts, and apples.

18. `public void removeAll( )`

```

{
 numberUsed = 0;
}

```

19. `public void increaseCapacity(int newCapacity)`

```

{
 if (newCapacity > numberUsed)
 {
 maxNumberElements = newCapacity;
 double[] temp = new double[newCapacity];
 for (int i = 0; i < a.length; i++)
 temp[i] = a[i];
 a = temp;
 } //else do nothing.
}

```

20. All you need to do to make your code work for sorting into decreasing order is to replace the `<` with `>` in the following line of the definition of `indexOfSmallest`:

```
if (a[index] < min)
```

However, to make your code easy to read, you should also rename the method `indexOfSmallest` to `indexOfLargest`, rename the variable `min` to `max`, and rename the variable `indexOfMin` to `indexOfMax`. You should rewrite some of the comments to reflect these changes as well.

21. If an array has a value that occurs more than once and you sort the array using the method `SelectionSort.sort`, then there will be as many copies of the repeated value after the array is sorted as there originally were in the array.

22. We give two slightly different versions. Both versions are on the accompanying website.

```

import java.util.Scanner;

public class ForEachEnumDemo
{
 enum WorkDay {MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY};

 public static void main(String[] args)
 {
 WorkDay[] day = WorkDay.values();
 }
}

```

extra code  
on website

```

Scanner keyboard = new Scanner(System.in);
double hours = 0, sum = 0;

for (WorkDay oneDay : day)
{
 System.out.println("Enter hours worked for " +
 oneDay);
 hours = keyboard.nextDouble();
 sum = sum + hours;
}

System.out.println("Total hours work = " + sum);
}
}

import java.util.Scanner;

public class ForEachEnumDemo2
{
 enum WorkDay {MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY};
 public static void main(String[] args)
 {
 Scanner keyboard = new Scanner(System.in);
 double hours = 0, sum = 0;

 for (WorkDay oneDay : WorkDay.values())
 {
 System.out.println("Enter hours worked for " + oneDay);
 hours = keyboard.nextDouble();
 sum = sum + hours;
 }

 System.out.println("Total hours work = " + sum);
 }
}

```

23. 0 1 2 3  
 0 1 2 3  
 0 1 2 3  
 0 1 2 3

24. If the array indices are out of bounds, then Java will halt the program with an error message, so no other checks on the parameters are needed.

```

/**
 * Returns the grade that student number studentNumber received on
 * quiz number quizNumber.
 */
public int getGrade(int studentNumber, int quizNumber)
{
 return grade[studentNumber][quizNumber];
}

```



25. If the array indices are out of bounds, then Java will halt the program with an error message, so no other checks on the parameters are needed.

```
/**
 * Changes the grade for student number studentNumber on quiz number
 * quizNumber to newGrade.
 */
public void changeGrade(int studentNumber,
 int quizNumber, int newGrade)
{
 grade[studentNumber][quizNumber] = newGrade;
}
```

26. /\*\*  
Returns an array with the average quiz score for each student.

```
*/
public double[] getStudentAverages()
{
 int arraySize = studentAverage.length;
 double[] temp = new double[arraySize];
 for (int i = 0; i < arraySize; i++)
 temp[i] = studentAverage[i];
 return temp;
}
```

27. /\*\*  
Returns an array with the average score for each quiz.

```
*/
public double[] getQuizAverages()
{
 int arraySize = quizAverage.length;
 double[] temp = new double[arraySize];
 for (int i = 0; i < arraySize; i++)
 temp[i] = quizAverage[i];
 return temp;
}
```

## Programming Projects

1. You are running a courier agency. The weight of a parcel determines the number of stamps that will be needed to send that parcel. For each kilogram, a stamp of \$2 is needed. Create a class to accept the weight of five parcels in floating-point values. Also, the courier company charges an additional rate depending on where the courier has to be delivered. The charges are \$20 for delivery within the city of posting, and \$40 for delivery anywhere else in the country. Write a computer program to calculate and display the total cost of each parcel depending on the weight and delivery location of the parcel.

2. A common memory matching game played by young children is to start with a deck of cards that contain identical pairs. For example, given six cards in the deck, two might be labeled 1, two labeled 2, and two labeled 3. The cards are shuffled and placed face down on the table. A player then selects two cards that are face down, turns them face up, and if the cards match, they are left face up. If the two cards do not match, they are returned to their original face down position. The game continues until all cards are face up.

Write a program that plays the memory matching game. Use 16 cards that are laid out in a  $4 \times 4$  square and are labeled with pairs of numbers from 1 to 8. Your program should allow the player to specify the cards that he or she would like to select through a coordinate system.

For example, in the following layout:

|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 8 | * | * | * |
| 2 | * | * | * | * |
| 3 | * | 8 | * | * |
| 4 | * | * | * | * |

all of the face down cards are indicated by \*. The pairs of 8 that are face up are at coordinates (1,1) and (2,3). To hide the cards that have been temporarily placed face up, output a large number of newlines to force the old board off the screen.

*Hint:* Use a 2D array for the arrangement of cards and another 2D array that indicates if a card is face up or face down. Or, a more elegant solution is to create a single 2D array where each element is an object that stores both the card's value and face. Write a function that "shuffles" the cards in the array by repeatedly selecting two cards at random and swapping them.

3. Write a program to calculate the average salary of an employee in a company. The program should read the monthly salary and overtime hours of an employee for each of the previous 12 months. The program should then print out a nicely formatted table showing the salary for each of the previous 12 months including salary for overtime hours. It should also show how much above or below average the total salary was for each month. The output should correctly label the months. There are a variety of ways to deal with the month names. One straightforward method is to code the months as integers and then do a conversion to a string for the month name before doing the output. The month input can be handled in any manner you wish so long as it is relatively easy and pleasant for the user.

The salary for the overtime hours is calculated as per the following rates:

- a. If salary is above \$10,000, then the rate per overtime hour is \$25.
- b. If salary is greater than \$5,000 and below \$10,000, then the rate per overtime hour is \$20.
- c. If salary is greater than \$2,000 and below \$5,000, then the rate per overtime hour is \$15.

If salary is below \$2,000, then the rate per overtime hour is \$10.

4. Write a program to document the marks of an additional value added course taught to the students. The marks of the students cannot be negative and cannot be more than 100 i.e.  $0 \leq \text{marks} \leq 100$ . Create a class that should maintain the following information about each student of the class.
  - a. Student name
  - b. Student ID
  - c. Array of marks in five subjects

Also, write a method called `validateMarks` that deletes all the marks less than zero and greater than 100 from the array. When a value of marks is deleted, the remaining marks are moved one position to fill in the gap. This creates empty positions at the end of the array so that less of the array is used. For example, consider the following given array:

```
int arrMarks [] = new int[5];
arrMarks [0] = 10;
arrMarks [1] = -15;
arrMarks [2] = 25;
arrMarks [3] = 102;
arrMarks [4] = 30;
```

After execution of `validateMarks`, the value of `arrMarks [0]` is 10, the value of `arrMarks [1]` is -15, the value of `arrMarks [2]` is 30, the value of `arrMarks [3]` is 102 and the value of `arrMarks [4]` is 30 and the value of `length` is 5. (The value of `arrMarks [1]` and `arrMarks [3]` is no longer of any concern, because the partially filled array no longer uses this indexed variable). Write a suitable test program for your method.

5. Write a program that takes as input a set of 15 numbers from the keyboard into an array of type `int []`. Create another array that will also read 15 other numbers of type `int` into it. Now merge the elements of these two arrays into one. The output is to be a two-column list. The first column is a list of the distinct array elements; the second column is the count of the number of occurrences of each element.

For example, if the elements of the first array are:

-22, 3, -22, 4, 1, 1, -22, 1, -1, 1, 2, 3, 4, 2, -22

And the elements of second array are:

-1, 1, 2, 3, 4, 2, -22, -22, 3, -22, 4, 1, 1, -22, 1

Then output should be

| N   | Count |
|-----|-------|
| -22 | 8     |
| -1  | 2     |
| 1   | 8     |
| 2   | 4     |
| 3   | 4     |
| 4   | 4     |

6. Write a program that reads numbers from the keyboard into an array of type `int[]`. You may assume that there will be 50 or fewer entries in the array. Your program allows any number of numbers to be entered, up to 50. The output is to be a two-column list. The first column is a list of the distinct array elements; the second column is the count of the number of occurrences of each element. The list should be sorted on entries in the first column, largest to smallest.

For the array

```
-12 3 -12 4 1 1 -12 1 -1 1 2 3 4 2 3 -12
```

the output should be

| N   | Count |
|-----|-------|
| 4   | 2     |
| 3   | 3     |
| 2   | 2     |
| 1   | 4     |
| -1  | 1     |
| -12 | 4     |

7. An array can be used to store large integers one digit at a time.

For example, the integer 1234 could be stored in the array `a` by setting `a[0]` to 1, `a[1]` to 2, `a[2]` to 3, and `a[3]` to 4. However, for this exercise you might find it more useful to store the digits backward; that is, place 4 in `a[0]`, 3 in `a[1]`, 2 in `a[2]`, and 1 in `a[3]`. In this exercise, write a program that reads in 2 positive integers that are 20 or fewer digits in length and then outputs the sum of the 2 numbers. Your program will read the digits as values of type `char` so that the number 1234 is read as the four characters '1', '2', '3', and '4'. After they are read into the program, the characters are changed to values of type `int`. The digits should be read into a partially filled array; you might find it useful to reverse the order of the elements in the array after the array is filled with data from the keyboard. (Whether or not you reverse the order of the elements in the array is up to you. It can be done either way, and each way has its advantages and disadvantages.) Your program should perform the addition by implementing the usual paper-and-pencil addition algorithm. The result of the addition should be stored in an array of size 20, and the result should then be written to the screen. If the result of the addition is an integer with more than the maximum number of digits (that is, more than 20 digits), then your program should issue a message saying that it has encountered "integer overflow." You should be able to change the maximum length of the integers by changing only one named constant. Include a loop that allows the user to continue to do more additions until the user says the program should end.



VideoNote  
Solution to  
Programming  
Project 6.8

8. Design a class called `BubbleSort` that is similar to the class `SelectionSort` given in Display 6.11. The class `BubbleSort` will be used in the same way as the class `SelectionSort`, but it will use the bubble sort algorithm.

The bubble sort algorithm checks all adjacent pairs of elements in the array from the beginning to the end and interchanges any two elements that are out of order. This process is repeated until the array is sorted. The algorithm is as follows:

#### **Bubble Sort Algorithm to Sort an Array `a`**

Repeat the following until the array `a` is sorted:

```
for (index = 0; index < a.length - 1; index++)
 if (a[index] > a[index + 1])
 Interchange the values of a[index] and a[index + 1].
```

The bubble sort algorithm is good for sorting an array that is “almost sorted.” It is not competitive with other sorting methods for most other situations.

9. Enhance the definition of the class `PartiallyFilledArray` (Display 6.5) in the following way: When the user attempts to add one additional element and there is no room in the array instance variable `a`, the user is allowed to add the element. The object creates a second array that is twice the size of the array `a`, copies values from the array `a` to the user’s new array, makes this array (or more precisely its reference) the new value of `a`, and then adds the element to this new larger array `a`. Hence, this new class should have no limit (other than the physical size of the computer) to how many numbers it can hold. The instance variable `maxNumberOfElements` remains and the method `getMaxCapacity` is unchanged, but these now refer to the currently allocated memory and not to an absolute upper bound. Write a suitable test program.
10. Write a program to simulate a simple game using arrays. The program should create a grid of stars using 2D arrays for the user to move in. When the program is started, the user’s current position should be generated randomly in row and column format. It should then ask the user for the movement selection, i.e., left, right, up, and down. The program should then reprint the grid with the old location of user replaced with a `$` symbol and the new location with an `N` symbol. The program displays the game positions as follows:

```



```

A sample grid configuration is

```

$ N *

```

11. Write a program to assign passengers seats in an airplane. Assume a small airplane with seat numberings as follows:

```
1 A B C D
2 A B C D
3 A B C D
4 A B C D
5 A B C D
6 A B C D
7 A B C D
```

The program should display the seat pattern, with an 'x' marking the seats already assigned. For example, after seats 1A, 2B, and 4C are taken, the display should look like the following:

```
1 X B C D
2 A X C D
3 A B C D
4 A B X D
5 A B C D
6 A B C D
7 A B C D
```

After displaying the seats available, the program should prompt for the seat desired, the user can type in a seat, and then the display of available seats should be updated. This continues until all seats are filled or until the user signals that the program should end. If the user types in a seat that is already assigned, the program should say that that seat is occupied and ask for another choice.

12. Write a program that plays a simple trivia game. The game should have five questions. Each question has a corresponding answer and point value between 1 and 3 based on the difficulty of the question. Implement the game using three arrays. An array of type `String` should be used for the questions. Another array of type `String` should be used to store the answers. An array of type `int` should be used for the point values. All three arrays should be declared to be of size 5.

The index into the three arrays can be used to tie the question, answer, and point value together. For example, the item at index 0 for each array would correspond to question 1, answer 1, and the point value for question 1. The item at index 1 for each array would correspond to question 2, answer 2, and the point value for question 2, and so forth. Manually hardcode the five questions, answers and point values into your program using trivia of your choice.

Your program should ask the player each question one at a time and allow the player to enter an answer. If the player's answer matches the actual answer, the player wins the number of points for that question. If the player's answer is incorrect, the player wins no points for the question. Your program should show the correct answer if the player is incorrect. After the player has answered all five questions, the game is over, and your program should display the player's total score.

13. Modify Programming Project 12 to use a single array instead of three arrays. This can be accomplished by creating a `Trivia` object that encapsulates the question,

answer, and point value for a particular trivia question. Next, create a single array of five `Trivia` objects instead of three separate arrays for the question, answer, and point values. This change will make your game more scalable if there were ever additional properties to add to a `Trivia` object (you would not need to add another array for each property). Although the program has internally changed to a single array of objects, the execution of the program should be identical to before.

14. Traditional password entry schemes are susceptible to “shoulder surfing” in which an attacker watches an unsuspecting user enter his or her password or PIN number and uses it later to gain access to the account. One way to combat this problem is with a randomized challenge-response system. In these systems, the user enters different information every time based on a secret in response to a randomly generated challenge. Consider the following scheme in which the password consists of a five-digit PIN number (00000 to 99999). Each digit is assigned a random number that is 1, 2, or 3. The user enters the random numbers that correspond to their PIN instead of their actual PIN numbers.

For example, consider an actual PIN number of 12345. To authenticate it, the user would be presented with a screen such as the following:

```
PIN: 0 1 2 3 4 5 6 7 8 9
NUM: 3 2 3 1 1 3 2 2 1 3
```

The user would enter 23113 instead of 12345. This does not divulge the password even if an attacker intercepts the entry because 23113 could correspond to other PIN numbers, such as 69440 or 70439. The next time the user logs in, a different sequence of random numbers would be generated, such as the following:

```
PIN: 0 1 2 3 4 5 6 7 8 9
NUM: 1 1 2 3 1 2 2 3 3 3
```

Your program should simulate the authentication process. Store an actual PIN number in your program. The program should use an array to assign random numbers to the digits from 0 to 9. Output the random digits to the screen, input the response from the user, and output whether or not the user’s response correctly matches the PIN number.

15. Programming Project 4.12 asked you to create a `PizzaOrder` class that stores an order consisting of up to three pizzas. Modify the class to store the pizzas using an array. This will allow the class to include an arbitrary number of pizzas in the order instead of a maximum of three. The `setNumPizzas` method can be used to create an array of the appropriate size. The array structure allows you to eliminate the methods `setPizza1`, `setPizza2`, and `setPizza3` and replace them with a single method, `setPizza(int index, Pizza newPizza)`. Include appropriate tests to determine if the new `PizzaOrder` class is working correctly.
16. Programming Project 3.15 asked you to explore Benford’s Law. An easier way to write the program is to use an array to store the digit counts. That is, `count[0]` might store the number of times 0 is the first digit (if that is possible in your data set), `count[1]` might store the number of times 1 is the first digit, and so forth. Redo Programming Project 3.15 using arrays.



Write a program that tests Benford's Law. Collect a list of at least 100 numbers from some real-life data source and enter them into a text file. Your program should use an array to store the digit counts. That is, `count[0]` might store the number of times 0 is the first digit (if that is possible in your data set), `count[1]` might store the number of times 1 is the first digit, and so forth. For each digit, output the percentage it appears as the first digit.

17. Programming Project 4.14 asked you to read in a CSV file of product ratings. The file was limited to exactly five products. Redo Programming Project 4.14, except calculate the name of each product and how many products are in the file based on the header line. Then read the CSV file and translate the data into a 2D array that stores all of the ratings. Finally, output the average rating for each product.
18. Programming Project 4.13 asked you to create a `BoxOfProduce` class representing a box of produce to deliver from a CSA farm. The box contained exactly three items. Modify the class so it uses an array of type `String` to represent the items in the box. You can still start with three random items to place in the box, but your menu should be modified to allow the user to add additional items and still substitute one item for another. You will likely need to modify the constructor of the `BoxOfProduce` class and also add new methods.
19. Some word games require the player to find words that can be formed using the letters of another word. For example, given the word *SWIMMING*, other words that can be formed using the letters include *SWIM*, *WIN*, *WING*, *SING*, *MIMING*, etc. Write a program that lets the user enter a word and then output all the words contained in the file `words.txt` that can be formed from the letters of the entered word. One algorithm to do this is to compare the letter histograms for each word. Create an array that counts up the number of each letter in the entered word (e.g., one S, one W, two I, two M, etc.) and then creates a similar array for the current word read from the file. The two arrays can be compared to see if the word from the file could be created out of the letters from the entered word.
20. Write a program that manages a list of up to 10 players and their high scores in the computer's memory (not on disk for this Programming Project). Use two arrays to manage the list. One array should store the players' names, and the other array should store the players' high scores. Use the index of the arrays to correlate the names with the scores. Your program should support the following features:
  - a. Add a new player and score (up to 10 players).
  - b. Print all the players' names and their scores to the screen.
  - c. Allow the user to enter a player's name and output that player's score or a message if that player's name has not been entered.
  - d. Allow the user to enter a player's name and remove the player from the list.Create a menu system that allows the user to select which option to invoke.
21. Redo Programming Project 6.20 but this time create a class named `Player` that stores a player's name and the player's high score. The class should have suitable constructors, accessors, and mutators. Next create a single array of type `Player` that stores the players' names and scores. Implement the same features as in Programming Project 6.20 using the single array rather than multiple arrays.



This page intentionally left blank



# Inheritance **7**

## **7.1 INHERITANCE BASICS** 460

Derived Classes 461

Overriding a Method Definition 471

Changing the Return Type of an Overridden Method 471

Changing the Access Permission of an Overridden Method 472

The `super` Constructor 474

The `this` Constructor 476

Example: An Enhanced `StringTokenizer` Class ★ 481

## **7.2 ENCAPSULATION AND INHERITANCE** 484

Protected and Package Access 487

## **7.3 PROGRAMMING WITH INHERITANCE** 493

Access to a Redefined Base Method 493

The Class Object 496

The Right Way to Define `equals` 497

*Like mother, like daughter.*

ANONYMOUS

## Introduction

Object-oriented programming (OOP) is a popular and powerful programming philosophy. One of the main techniques of OOP is known as *inheritance*. **Inheritance** means that a very general form of a class can be defined and compiled. Later, more specialized versions of that class may be defined by starting with the already defined class and adding more specialized instance variables and methods. The specialized classes are said to *inherit* the methods and instance variables of the previously defined general class. In this chapter, we cover inheritance in general and more specifically how it is realized in Java.

## Prerequisites

This chapter does not use any material on arrays from Chapter 6. It does require Chapters 1 through 5 with the exception that most of the chapter does not require Section 5.4 on packages and `javadoc`. In this chapter, the subsection “Protected and Package Access” is the only section that requires any knowledge from Section 5.4, and it requires only the material on packages, not the material on `javadoc`. If you omit the subsection “Protected and Package Access,” you will not suffer any loss of continuity in reading this chapter.

## 7.1 Inheritance Basics

*If there is anything that we wish to change in the child, we should first examine it and see whether it is not something that could better be changed in ourselves.*

CARL GUSTAV JUNG, *The Integration of the Personality*,  
FARRAR & RINEHART, Incorporated, 1939, 1939.

Inheritance is the process by which a new class—known as a *derived class*—is created from another class, called the *base class*. A derived class automatically has all the instance variables and all the methods that the base class has, and can have additional methods and/or additional instance variables.

## Derived Classes

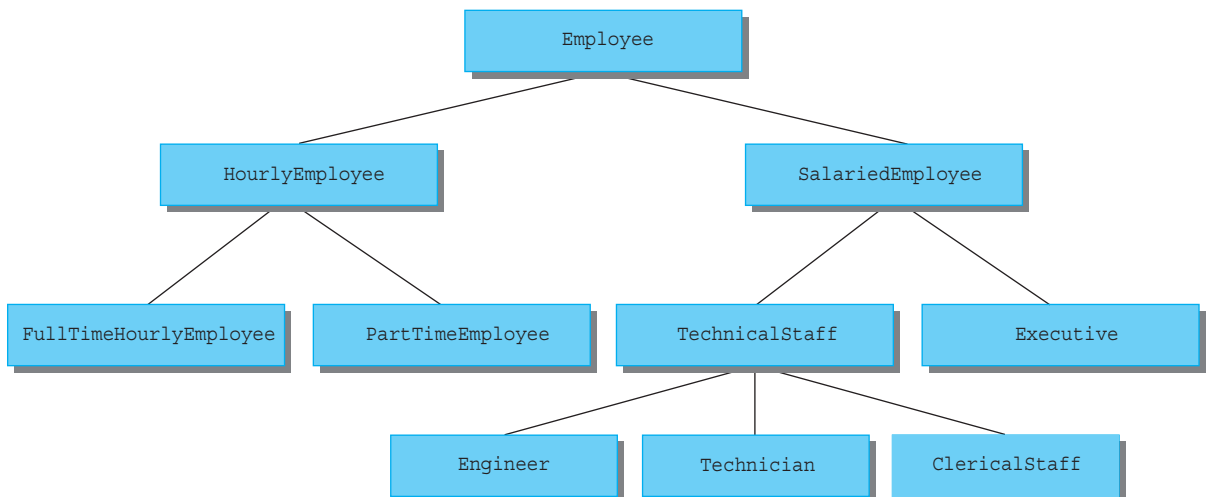
Suppose we are designing a record-keeping program that has records for salaried employees and hourly employees. There is a natural hierarchy for grouping these classes. These are all classes of people who share the property of being employees.

Employees who are paid an hourly wage are one subset of employees. Another subset consists of salaried employees who are paid a fixed wage each month. Although the program may not need any type corresponding to the set of all employees, thinking in terms of the more general concept of employees can be useful. For example, all employees have a name and a hire date (when they started working for the company), and the methods for setting and changing names and hire dates will be the same for salaried and hourly employees. The classes for hourly employees and salaried employees may be further subdivided as diagrammed in Display 7.1.

Within Java, you can define a class called `Employee` that includes all employees (salaried or hourly) and then use this class to define classes for hourly employees and salaried employees. You can then, in turn, use classes such as `HourlyEmployee` to define classes such as `PartTimeHourlyEmployee`, and so forth.

Display 7.2 shows our definition for the class `Employee`. This class is a pretty ordinary class. What is interesting about this class is how we use it to create a class for hourly employees and a class for salaried employees. It is legal to create an object of the class `Employee`, but our reason for defining the class `Employee` is so that we can define derived classes for different kinds of employees.

Display 7.1 A Class Hierarchy



## Display 7.2 The Base Class Employee (part 1 of 2)

```
1 /**
2 Class Invariant: All objects have a name string and hire date.
3 A name string of "No name" indicates no real name specified yet.
4 A hire date of January 1, 1000 indicates no real hire date specified yet.
5 */
6 public class Employee
7 {
8 private String name;
9 private Date hireDate;
10 public Employee()
11 {
12 name = "No name";
13 hireDate = new Date("January", 1, 1000); //Just a placeholder.
14 }
15
16 /**
17 Precondition: Neither theName nor theDate is null.
18 */
19 public Employee(String theName, Date theDate)
20 {
21 if (theName == null || theDate == null)
22 {
23 System.out.println("Fatal Error creating employee.");
24 System.exit(0);
25 }
26 name = theName;
27 hireDate = new Date(theDate);
28
29 public Employee(Employee originalObject)
30 {
31 name = originalObject.name;
32 hireDate = new Date(originalObject.hireDate);
33
34 public String getName()
35 {
36 return name;
37
38 public Date getHireDate()
39 {
40 return new Date(hireDate);
```

Display 7.2 The Base Class Employee (part 2 of 2)

---

```
41 /**
42 * Precondition newName is not null.
43 */
44 public void setName(String newName)
45 {
46 if (newName == null)
47 {
48 System.out.println("Fatal Error setting employee name.");
49 System.exit(0);
50 }
51 else
52 name = newName;
53 }

54 /**
55 * Precondition newDate is not null.
56 */
57 public void setHireDate(Date newDate)
58 {
59 if (newDate == null)
60 {
61 System.out.println("Fatal Error setting employee hire " +
62 "date.");
63 System.exit(0);
64 }
65 else
66 hireDate = new Date(newDate);

67 public String toString()
68 {
69 return (name + " " + hireDate.toString());
70 }

71 public boolean equals(Employee otherEmployee)
72 {
73 return (name.equals(otherEmployee.name)
74 && hireDate.equals(otherEmployee.hireDate));
75 }
76 }
```

---

**derived class****base class****extends****subclass and superclass****inheritance**

Display 7.3 contains the definition of a class for hourly employees. An hourly employee is an employee, so we define the class `HourlyEmployee` to be a *derived* class of the class `Employee`. A **derived class** is a class defined by adding instance variables and methods to an existing class. The existing class that the derived class is built upon is called the **base class**. In our example, `Employee` is the base class and `HourlyEmployee` is the derived class. As you can see in Display 7.3, the way we indicate that `HourlyEmployee` is a derived class of `Employee` is by including the phrase `extends Employee` on the first line of the class definition, like so:

```
public class HourlyEmployee extends Employee
```

A derived class is also called a **subclass**, in which case the base class is usually called a **superclass**. However, we prefer to use the terms *derived class* and *base class*.

When you define a derived class, you give only the added instance variables and the added methods. For example, the class `HourlyEmployee` has all the instance variables and all the methods of the class `Employee`, but you do not mention them in the definition of `HourlyEmployee`. Every object of the class `HourlyEmployee` has instance variables called `name` and `hireDate`, but you do not specify the instance variable `name` or the instance variable `hireDate` in the definition of the class `HourlyEmployee`. The class `HourlyEmployee` (or any other derived class) is said to **inherit** the instance variables and methods of the base class that it extends. For this reason, the topic of derived classes is called **inheritance**.

Just as it inherits the instance variables of the class `Employee`, the class `HourlyEmployee` inherits all the methods from the class `Employee`. So, the class `HourlyEmployee` inherits the methods `getName`, `getHireDate`, `setName`, and `setHireDate`, from the class `Employee`.

For example, suppose you create a new object of the class `HourlyEmployee` as follows:

```
HourlyEmployee joe = new HourlyEmployee();
```

Then, the name of the object `joe` can be changed using the method `setName`, which the class `HourlyEmployee` inherited from the class `Employee`. The inherited method `setName` is used just like any other method; for example,

```
joe.setName("Josephine");
```

A small demonstration of this is given in Display 7.4.

Display 7.5 contains the definition of the class `SalariedEmployee`, which is also derived from the class `Employee`. The class `SalariedEmployee` inherits all the instance variables and methods of the class `Employee`. Inheritance allows you to reuse code, such as the code in the class `Employee`, without needing to literally copy the code into the definitions of the derived classes, such as `HourlyEmployee` and `SalariedEmployee`.



VideoNote  
Inheritance  
Walkthrough

Display 7.3 The Derived Class `HourlyEmployee` (part 1 of 3)

```

1 /**
2 Class Invariant: All objects have a name string, hire date, nonnegative
3 wage rate, and nonnegative number of hours worked. A name string of
4 "No name" indicates no real name specified yet. A hire date of
5 January 1, 1000 indicates no real hire date specified yet.
6 */
7 public class HourlyEmployee extends Employee
8 {
9 private double wageRate;
10 private double hours; //for the month

11 public HourlyEmployee()
12 {
13 super(); ← If this line is omitted, Java will still invoke
14 wageRate = 0; the no-argument constructor for the
15 hours = 0; base class.
16 }

17 /**
18 Precondition: Neither theName nor theDate is null;
19 theWageRate and theHours are nonnegative.
20 */
21 public HourlyEmployee(String theName, Date theDate,
22 double theWageRate, double theHours)
23 {
24 super(theName, theDate);
25 if ((theWageRate >= 0) && (theHours >= 0))
26 {
27 wageRate = theWageRate;
28 hours = theHours;
29 }
30 else
31 {
32 System.out.println(
33 "Fatal Error: creating an illegal hourly employee.");
34 System.exit(0);
35 }
36 }

37 public HourlyEmployee(HourlyEmployee originalObject)
38 {
39 super(originalObject); ← An object of the class
 HourlyEmployee is also an
 instance of the class Employee.

```

(continued)



Display 7.3 The Derived Class `HourlyEmployee` (part 2 of 3)

```
40 wageRate = originalObject.wageRate;
41 hours = originalObject.hours;
42 }
43 public double getRate()
44 {
45 return wageRate;
46 }
47 public double getHours()
48 {
49 return hours;
50 }

51 /**
52 * Returns the pay for the month.
53 */
54 public double getPay()
55 {
56 return wageRate*hours;
57 }

58 /**
59 * Precondition: hoursWorked is nonnegative.
60 */
61 public void setHours(double hoursWorked)
62 {
63 if (hoursWorked >= 0)
64 hours = hoursWorked;
65 else
66 {
67 System.out.println("Fatal Error: Negative hours worked.");
68 System.exit(0);
69 }
70 }

71 /**
72 * Precondition: newWageRate is nonnegative.
73 */
74 public void setRate(double newWageRate)
75 {
76 if (newWageRate >= 0)
77 wageRate = newWageRate;
78 else
79 {
80 System.out.println("Fatal Error: Negative wage rate.");
81 System.exit(0);
```

Display 7.3 The Derived Class `HourlyEmployee` (part 3 of 3)

```

82 }
83 }

The method toString is overridden so it is different in the
derived class HourlyEmployee than it is in the base class
Employee.

84 public String toString()
85 {
86 return (getName() + " " + getHireDate().toString()
87 + "\n$" + wageRate + " per hour for " + hours + " hours");
88 }

89 public boolean equals(HourlyEmployee other)
90 {
91 return (getName().equals(other.getName())
92 && getHireDate().equals(other.getHireDate())
93 && wageRate == other.wageRate
94 && hours == other.hours);
95 }
96 }

We will show you a better way to
define equals in the subsection
"The Right Way to Define equals."

```

## Derived Class (Subclass)

Define a **derived class** by starting with another already defined class and adding (or changing) methods, instance variables, and static variables. The class you start with is called the **base class**. The derived class inherits all the public methods, all the public and private instance variables, and all the public and private static variables from the base class, and it can add more instance variables, more static variables, and more methods. So, of the things we have seen thus far, the only members not inherited are private methods. (As discussed in the subsection "Overriding a Method Definition," the derived class definition can also change the definition of an inherited method.) A derived class is also called a **subclass**, in which case the base class is usually called a **superclass**.

### SYNTAX

```

public class Derived_Class_Name extends Base_Class_Name
{
 Declarations_of_Added_Static_Variables
 Declarations_of_Added_Instance_Variables
 Definitions_of_Added__And_Overridden_Methods
}

```

### EXAMPLE

See Displays 7.3 and 7.5.

## Display 7.4 Inheritance Demonstration

```
1 public class InheritanceDemo The methods getName and setName are
2 { inherited from the base class Employee.
3 public static void main(String[] args)
4 {
5 HourlyEmployee joe = new HourlyEmployee("Joe Worker",
6 new Date("January", 1, 2015),
 50.50, 160);

7 System.out.println("joe's longer name is " +
 joe.getName());

8 System.out.println("Changing joe's name to Josephine.");
9 joe.setName("Josephine");

10 System.out.println("joe's record is as follows:");
11 System.out.println(joe);
12 }
13 }
```

## Sample Dialogue

```
joe's longer name is Joe Worker
Changing joe's name to Josephine.
joe's record is as follows:
Josephine January 1, 2015
$50.5 per hour for 160 hours
```

### Inherited Members

A derived class automatically has all the instance variables, all the static variables, and all the public methods of the base class. These members from the base class are said to be **inherited**. These inherited methods and inherited instance and static variables are, with one exception, not mentioned in the definition of the derived class, but they are automatically members of the derived class. The one exception is as follows: As explained in the subsection “Overriding a Method Definition,” you can give a definition for an inherited method in the definition of the derived class; this will redefine the meaning of the method for the derived class.

Display 7.5 The Derived Class `SalariedEmployee` (part 1 of 2)

```

1 /**
2 Class Invariant: All objects have a name string, hire date,
3 and nonnegative salary. A name string of "No name" indicates
4 no real name specified yet. A hire date of January 1, 1000 indicates
5 no real hire date specified yet.
6 */
7 public class SalariedEmployee extends Employee
8 {
9 private double salary; //annual
10
11 public SalariedEmployee()
12 {
13 super();
14 salary = 0;
15 }
16
17 /**
18 Precondition: Neither theName nor theDate is null;
19 theSalary is nonnegative.
20 */
21 public SalariedEmployee(String theName, Date theDate, double theSalary)
22 {
23 super(theName, theDate);
24 if (theSalary >= 0)
25 salary = theSalary;
26 else
27 {
28 System.out.println("Fatal Error: Negative salary.");
29 System.exit(0);
30 }
31 }
32
33 public SalariedEmployee(SalariedEmployee originalObject)
34 {
35 super(originalObject);
36 salary = originalObject.salary;
37 }
38
39 public double getSalary()
40 {
41 return salary;
42 }

```

*It will take the rest of Section 7.1 to fully explain this class definition.*

*If this line is omitted, Java will still invoke the no-argument constructor for the base class.*

*An object of the class SalariedEmployee is also an object of the class Employee.*

(continued)

Display 7.5 The Derived Class `SalariedEmployee` (part 2 of 2)

```

39 /**
40 * Returns the pay for the month.
41 */
42 public double getPay()
43 {
44 return salary/12;
45 }

46 /**
47 * Precondition: newSalary is nonnegative.
48 */
49 public void setSalary(double newSalary)
50 {
51 if (newSalary >= 0)
52 salary = newSalary;
53 else
54 {
55 System.out.println("Fatal Error: Negative salary.");
56 System.exit(0);
57 }
58 }

59 public String toString()
60 {
61 return (getName() + " " + getHireDate().toString()
62 + "\n$" + salary + " per year");
63 }

64 public boolean equals(SalariedEmployee other)
65 {
66 return (getName().equals(other.getName())
67 && getHireDate().equals(other.getHireDate())
68 && salary == other.salary);
69 }
70 }

```

*We will show you a better way to  
define equals in the subsection  
“The Right Way to Define equals.”*

parent class

child class

ancestor class

descendent  
class

## Parent and Child Classes

A base class is often called the **parent class**. A derived class is then called a **child class**. This analogy is often carried one step further. A class that is a parent of a parent of a parent of another class (or some other number of “parent of” iterations) is often called an **ancestor class**. If class A is an ancestor of class B, then class B is often called a **descendent** of class A.

## Overriding a Method Definition

### overriding

The definition of an inherited method can be changed in the definition of a derived class so that it has a meaning in the derived class that is different from what it is in the base class. This is called **overriding** the definition of the inherited method. For example, the methods `toString` and `equals` are overridden (redefined) in the definition of the derived class `HourlyEmployee`. They are also overridden in the class `SalariedEmployee`. To override a method definition, simply give the new definition of the method in the class definition, just as you would with a method that is added in the derived class.

### Overriding a Method Definition

A derived class inherits methods that belong to the base class. However, if a derived class requires a different definition for an inherited method, the method may be redefined in the derived class. This is called **overriding** the method definition.

### The `final` Modifier

If you add the modifier `final` to the definition of a method, it indicates that the method may not be redefined in a derived class. If you add the modifier `final` to the definition of a class, it indicates that the class may not be used as a base class to derive other classes. We will say more about the `final` modifier in Chapter 8.

## Changing the Return Type of an Overridden Method

### covariant return type

In a derived class, you can override (change) the definition of a method from the base class. As a general rule, when overriding a method definition, you may *not* change the type returned by the method, and you may not change a `void` method to a method that returns a value, nor a method that returns a value to a `void` method. The one exception to this rule is if the returned type is a class type, then you may change the returned type to that of any descendent class of the returned type. For example, if a function returns the type `Employee` (Display 7.2), when you override the function definition in a derived class, you may change the returned type to `HourlyEmployee` (Display 7.3), `SalariedEmployee` (Display 7.5), or any other descendent class of the class `Employee`. This sort of changed return type is known as a **covariant return type** and is new in Java version 5.0; it was not allowed in earlier versions of Java. Earlier versions of Java allowed absolutely no changes to the returned type. We will give complete examples of changing the returned type of an overridden method in Chapter 8. Here we will just outline an example.

For example, suppose one class definition includes the following details:

```
public class BaseClass
{
 ...
 public Employee getSomeone(int someKey)
 ...
}
```

In this case, the following details would be allowed in a derived class:

```
public class DerivedClass extends BaseClass
{
 ...
 public HourlyEmployee getSomeone(int someKey)
 ...
}
```

When the method definition for `getSomeone` is overridden in `DerivedClass`, the returned type is changed from `Employee` to `HourlyEmployee`.

It is worth noting that when you change the returned type of an overridden method in this way, such as from `Employee` to `HourlyEmployee`, you are not really changing the returned type so much as placing additional restrictions on it. Every `HourlyEmployee` is an `Employee` with some additional properties that, while they are properties of every `HourlyEmployee`, are not properties of every `Employee`. Any code that was written for a method of the base class and that assumed the value returned by the method is `Employee` will be legal for an overridden version of the method that returns an `HourlyEmployee`. This is true because every `HourlyEmployee` is an `Employee`.

## Changing the Access Permission of an Overridden Method

You can change the access permission of an overridden method from `private` in the base class to `public` in the derived class (or in any other way that makes access permissions more permissive). For example, if the following is a method heading in a base class:

```
private void doSomething()
```

then you can use the following heading when overriding the method definition in a derived class:

```
public void doSomething()
```

Note that you cannot change permissions to make them more restricted in the derived class. You can change `private` to `public`, but you cannot change `public` to `private`.

This makes sense, because you want code written for the base class method to work for the derived class method. You can use a public method anywhere that you can use a private method, but it is not true that you can use a private method anywhere that you can use a public method.



## PITFALL: Overriding versus Overloading

Do not confuse *overriding* (that is, redefining) a method definition in a derived class with *overloading* a method name. When you override a method definition, the new method definition given in the derived class has the exact same number and types of parameters. On the other hand, if the method in the derived class were to have a different number of parameters or a parameter of a different type from the method in the base class, then the derived class would have both methods. That would be overloading. For example, suppose we add the following method to the definition of the class `HourlyEmployee` (Display 7.3):

```
public void setName(String firstName, String lastName)
{
 if ((firstName == null) || (lastName == null))
 {
 System.out.println("Fatal Error setting employee name.");
 System.exit(0);
 }
 else
 name = firstName + " " + lastName;
}
```

The class `HourlyEmployee` would then have this two-argument method `setName`, and it would also inherit the following one-argument method `setName` from the base class `Employee`:

```
public void setName(String newName)
{
 if (newName == null)
 {
 System.out.println("Fatal Error setting employee name.");
 System.exit(0);
 }
 else
 name = newName;
}
```

The class `HourlyEmployee` would have two methods named `setName`. This would be *overloading* the method name `setName`.

(continued)





## PITFALL: (continued)

On the other hand, both the class `Employee` and the class `HourlyEmployee` define a method with the following method heading:

```
public String toString()
```

In this case, the class `HourlyEmployee` has only one method named `toString()`, but the definition of the method `toString()` in the class `HourlyEmployee` is different from the definition of `toString()` in the class `Employee`; the method `toString()` has been *overridden* (that is, redefined).

If you get overriding and overloading confused, you do have one consolation. They are both legal. ■

## The super Constructor

You can invoke a constructor of the base class within the definition of a derived class constructor. A constructor for a derived class uses a constructor from the base class in a special way. A constructor for the base class normally initializes all the data inherited from the base class. So a constructor for a derived class begins with an invocation of a constructor for the base class. The details are described next.

There is a special syntax for invoking the base class constructor that is illustrated by the constructor definitions for the class `HourlyEmployee` given in Display 7.3. In what follows, we have reproduced the beginning of one of the constructor definitions for the class `HourlyEmployee` taken from that display:

```
public HourlyEmployee(String theName, Date theDate,
 double theWageRate, double theHours)
{
 super (theName, theDate);
 if ((theWageRate >= 0) && (theHours >= 0))
 {
 wageRate = theWageRate;
 hours = theHours;
 }
 else
 ...
}
```

The line

```
super (theName, theDate);
```

is a call to a constructor for the base class, which in this case is a call to a constructor for the class `Employee`.

## Self-Test Exercises

1. Suppose the class named `DiscountSale` is a derived class of a class called `Sale`. Suppose the class `Sale` has instance variables named `price` and `numberOfItems`. Will an object of the class `DiscountSale` also have instance variables named `price` and `numberOfItems`?
2. Suppose the class named `DiscountSale` is a derived class of a class called `Sale`, and suppose the class `Sale` has public methods named `getTotal` and `getTax`. Will an object of the class `DiscountSale` have methods named `getTotal` and `getTax`? If so, do these methods have to perform the exact same actions in the class `DiscountSale` as in the class `Sale`?
3. Suppose the class named `DiscountSale` is a derived class of a class called `Sale`, and suppose the class `Sale` has a method with the following heading and no other methods named `getTax`, as follows:

```
public double getTax()
```

And suppose the definition of the class `DiscountSale` has a method definition with the following heading and no other method definitions for methods named `getTax`, as follows:

```
public double getTax(double rate)
```

How many methods named `getTax` will the class `DiscountSale` have and what are their headings?

4. The class `HourlyEmployee` (Display 7.3) has methods named `getName` and `getRate` (among others). Why does the definition of the class `HourlyEmployee` contain a definition of the method `getRate` but no definition of the method `getName`?

There are some restrictions on how you can use the base class constructor call `super`. You cannot use an instance variable as an argument to `super`. Also, the call to the base class constructor (`super`) must always be the first action taken in a constructor definition. You cannot use it later in the definition of a constructor.

Notice that you use the keyword `super` to call the constructor of the base class. You do not use the name of the constructor; you do *not* use

```
Employee(theName, theDate); //ILLEGAL
```

If a constructor definition for a derived class does not include an invocation of a constructor for the base class, then the no-argument constructor of the base class is invoked automatically as the first action of the derived class constructor. So, the

following definition of the no-argument constructor for the class `HourlyEmployee` (with `super` omitted) is equivalent to the version we gave in Display 7.3:

```
public HourlyEmployee()
{
 wageRate = 0;
 hours = 0;
}
```

A derived class object has all the instance variables of the base class. These inherited instance variables should be initialized, and the base class constructor is the most convenient place to initialize these inherited instance variables. That is why you should always include a call to one of the base class constructors when you define a constructor for a derived class. As already noted, if you do not include a call to a base class constructor (using `super`), then the no-argument constructor of the base class is called automatically. (If there is no no-argument constructor for the base class, that is an error condition.)

### Call to a Base Class Constructor

Within the definition of a constructor for a class, you can use `super` as a name for a constructor of the base class. Any invocation of `super` must be the first action taken by the constructor.

#### EXAMPLE

```
public SalariedEmployee(SalariedEmployee originalObject)
{
 super(originalObject); //Invocation of base class
 //constructor.
 salary = originalObject.salary;
}
```

### The `this` Constructor

`this`

When defining a constructor, it is sometimes convenient to be able to call one of the other constructors in the same class. You can use the keyword `this` as a method name to invoke a constructor in the same class. This use of `this` is similar to the use of `super`, but with `this`, the call is to a constructor of the same class, not to a constructor for the base class. For example, consider the following alternate, and equivalent, definition of the no-argument constructor for the class `HourlyEmployee` (from Display 7.3):

```
public HourlyEmployee()
{
 this("No name", new Date("January", 1, 1000), 0, 0);
}
```

The line with `this` is an invocation of the constructor with the following heading:

```
public HourlyEmployee(String theName, Date theDate,
 double theWageRate, double theHours)
```

The restrictions on how you can use the base class constructor call `super` also apply to the `this` constructor. You cannot use an instance variable as an argument to `this`. Also, any call to the constructor `this` must always be the first action taken in a constructor definition. Thus, a constructor definition cannot contain both an invocation of `super` and an invocation of `this`. If you want to include both a call to `super` and a call to `this`, use a call with `this`, and have the constructor that is called with `this` have `super` as its first action.

### Call to Another Constructor in the Same Class

Within the definition of a constructor for a class, you can use `this` as a name for another constructor in the same class. Any invocation of `this` must be the first action taken by the constructor.

#### EXAMPLE

```
public HourlyEmployee()
{
 this("No name", new Date("January", 1, 1000), 0,0);
}
```



### TIP: An Object of a Derived Class Has More than One Type

An object of a derived class has the type of the derived class. It also has the type of the base class, and more generally, it has the type of every one of its ancestor classes. For example, consider the following copy constructor definition from the class `HourlyEmployee` (Display 7.3):

```
public HourlyEmployee(HourlyEmployee originalObject)
{
 super(originalObject);
 wageRate = originalObject.wageRate;
 hours = originalObject.hours;
}
```

(continued)

**TIP: (continued)**

The line

```
super(originalObject);
```

is an invocation of a constructor for the base class `Employee`. The class `Employee` has no constructor with a parameter of type `HourlyEmployee`, but `originalObject` is of type `HourlyEmployee`. Fortunately, every object of type `HourlyEmployee` is also of type `Employee`. So, this invocation of `super` is an invocation of the copy constructor for the class `Employee`.

The fact that every object is not only of its own type but is also of the type of its ancestor classes simply reflects what happens in the everyday world. An hourly employee is an employee as well as an hourly employee. This sometimes is referred to as the **“is a” relationship**: For example, an `HourlyEmployee` is an `Employee`.

Display 7.6 contains a program demonstrating that an `HourlyEmployee` and a `SalariedEmployee` are also `Employee` objects. The method `showEmployee` requires an argument of type `Employee`. The objects `joe` and `sam` are of type `Employee` because they are instances of classes derived from the class `Employee` and so are suitable arguments for `showEmployee`. ■

“is a”  
relationship

**An Object of a Derived Class Has More than One Type**

An object of a derived class has the type of the derived class, and it also has the type of the base class. More generally, a derived class has the type of every one of its ancestor classes. So, you can assign an object of a derived class to a variable of any ancestor type (but not the other way around). You can plug in a derived class object for a parameter of any of its ancestor types. More generally, you can use a derived class object anywhere you can use an object of any of its ancestor types.

## Display 7.6 An Object Belongs to Multiple Classes

```

1 public class IsADemo
2 {
3 public static void main(String[] args)
4 {
5 SalariedEmployee joe = new SalariedEmployee("Josephine",
6 new Date("January", 1, 2015), 100000);
7 HourlyEmployee sam = new HourlyEmployee("Sam",
8 new Date("February", 1, 2015), 50.50, 40);

9 System.out.println("joe's longer name is " + joe.getName());
10 System.out.println("showEmployee(joe) invoked:");
11 showEmployee(joe);
12 System.out.println("showEmployee(sam) invoked:");
13 showEmployee(sam);
14 }

15 public static void showEmployee(Employee employeeObject)
16 {
17 System.out.println(employeeObject.getName());
18 System.out.println(employeeObject.getHireDate());
19 }
20 }

```

*A SalariedEmployee is an Employee.*

*An HourlyEmployee is an Employee.*

## Sample Dialogue

```

joe's longer name is Josephine
showEmployee(joe) invoked:
Josephine
January 1, 2015
showEmployee(sam) invoked:
Sam
February 1, 2015

```



## PITFALL: The Terms *Subclass* and *Superclass*

### subclass and superclass

Many programmers and authors use the term *subclass* for a derived class and use *superclass* for its base class (or any of its ancestor classes). This is logical. For example, the collection of all hourly employees in the world is a subclass of all employees. Similarly, the collection of all objects of type `HourlyEmployee` is a subcollection of the collection of all objects of the class `Employee`. As you add more instance variables and methods, you restrict the number of objects that can satisfy the class definition. Despite this logic, people often reverse the terms *subclass* and *superclass*. Remember that these terms refer to the collections of objects of the derived class and the base class and not to the number of instance variables or methods. A derived class is a *subclass* (not a *superclass*) of its base class. Another way to remember which is a superclass is to recall that the super constructor invocation is an invocation of the base class, and so the base class is the *superclass*. ■

## Self-Test Exercises

5. Is the following program legal? The relevant classes are defined in Displays 7.2, 7.3, and 7.5.

```
public class EmployeeDemo
{
 public static void main(String[] args)
 {
 HourlyEmployee joe =
 new HourlyEmployee("Joe Young",
 new Date("February", 1, 2015), 10.50, 40);
 SalariedEmployee boss =
 new SalariedEmployee("Mr. Big Shot",
 new Date("January", 1, 2015), 100000);
 printName(joe);
 printName(boss);
 }
 public void printName(Employee object)
 {
 System.out.println(object.getName());
 }
}
```

6. Give a definition for a class `TitledEmployee` that is a derived class of the base class `SalariedEmployee` given in Display 7.5. The class `TitledEmployee` has one additional instance variable of type `String` called `title`. It also has two additional methods: `getTitle`, which takes no arguments and returns a `String`, and `setTitle`, which is a `void` method that takes one argument of type `String`. It also overrides (redefines) the method definition for `getName`, so that the string returned includes the title as well as the name of the employee.

**EXAMPLE: An Enhanced `StringTokenizer` Class ★**

Inheritance allows you to reuse all the code written for a base class in a derived class, and it lets you reuse it without copying it or even seeing the code in the base class. This means that, among other things, if one of the standard Java library classes does not have all the methods you want it to have, you can, in most cases, define a derived class that has the desired additional methods. In this subsection, we give a simple example of this process. This example requires that you have already covered the basics about arrays given in Section 6.1 of Chapter 6. It also requires you to have read the starred section on the `StringTokenizer` class in Chapter 4. If you have not covered this material, you will have to skip this example until you cover it.

The `StringTokenizer` class allows you to generate all the tokens in a string one time, but sometimes you want to cycle through the tokens a second or third time. There are lots of ways to accomplish this. For example, you can use the `StringTokenizer` constructor two (or more) times to create two (or more) `StringTokenizer` objects. However, it would be cleaner and more efficient if you could do it with just one `StringTokenizer` object. Display 7.7 shows a derived class of the `StringTokenizer` class that allows you to cycle through the tokens in a string any number of times. This class is called `EnhancedStringTokenizer`. The class `EnhancedStringTokenizer` behaves exactly the same as the `StringTokenizer` class, except that the class `EnhancedStringTokenizer` has one additional method named `tokensSoFar`. This method has no parameters and returns an array of strings containing all the tokens that have so far been returned by the methods named `nextToken`. After an object of the class `EnhancedStringTokenizer` has gone through all the tokens with the methods `nextToken`, it can invoke the method `tokensSoFar` to produce an array containing all the tokens. This array can be used to cycle through the tokens any number of additional times. A simple example of this is given in the program in Display 7.8.

The class `EnhancedStringTokenizer` has methods, such as `countTokens`, that it inherits unchanged from the class `StringTokenizer`. The class `EnhancedStringTokenizer` also has two methods—namely, the two methods named `nextToken`—whose definitions are overridden. From the outside, the methods named `nextToken` of the class `EnhancedStringTokenizer` behave exactly the same as the methods named `nextToken` in the class `StringTokenizer`. However, each of the two methods named `nextToken` of the class `EnhancedStringTokenizer` also save the tokens in an array instance variable, `a`, so that an array of tokens can be returned by the method `tokensSoFar`. The method `tokensSoFar` is the only completely new method in the derived class `EnhancedStringTokenizer`.

(continued)



**EXAMPLE:** (continued)

Notice that the definitions of the methods named `nextToken` in the class `EnhancedStringTokenizer` each include an invocation of `super.nextToken`, which is the version of the corresponding method `nextToken` in the base class `StringTokenizer`. Each overridden version of the method `nextToken` uses the method `super.nextToken` to produce the token it returns, but before returning the token, it stores the token in the array instance variable `a`. The instance variable `count` contains a count of the number of tokens stored in the array instance variable `a`.<sup>1</sup>

Display 7.7 Enhanced `StringTokenizer` (part 1 of 2)

```

1 import java.util.StringTokenizer;
2
3 public class EnhancedStringTokenizer extends StringTokenizer
4 {
5 private String[] a;
6 private int count;
7
8 public EnhancedStringTokenizer(String theString)
9 {
10 super (theString);
11 a = new String[countTokens()];
12 count = 0;
13 }
14
15 public EnhancedStringTokenizer(String theString, String delimiters)
16 {
17 super (theString, delimiters);
18 a = new String[countTokens()];
19 count = 0;
20 }
21
22 /**
23 Returns the same value as the same method in the StringTokenizer
24 class, but it also stores data for the method tokensSoFar to use.
25 */
26 public String nextToken()
27 {

```

*The method `countTokens` is inherited and is not overridden.*

*This method `nextToken` has its definition overridden.*

<sup>1</sup>The class `StringTokenizer` also has a method named `nextElement` with a return type of `Object`. This method should also be overridden. We have not yet even mentioned this method because we have not yet discussed the class `Object`. For now, you can simply pretend `StringTokenizer` has no such method `nextElement`. We will discuss this point in Self-Test Exercise 23 later in this chapter after we introduce the class `Object`.

Display 7.7 Enhanced `StringTokenizer` (part 2 of 2)

```

25 String token = super.nextToken();
26 a[count] = token;
27 count++;
28 return token;
29 }

30 /**
31 Returns the same value as the same method in the StringTokenizer
32 class, and changes the delimiter set in the same way as does the
33 same method in the StringTokenizer class, but it also stores data
34 for the method tokensSoFar to use.
35 */
36 public String nextToken(String delimiters)
37 {
38 String token = super.nextToken(delimiters);
39 a[count] = token;
40 count++;
41 return token;
42 }
43 /**
44 Returns an array of all tokens produced so far. Array returned
45 has length equal to the number of tokens produced so far.
46 */
47 public String[] tokensSoFar()
48 {
49 String[] arrayToReturn = new String[count];
50 for (int i = 0; i < count; i++)
51 arrayToReturn[i] = a[i];
52 return arrayToReturn;
53 }

```

*super.nextToken is the version of nextToken defined in the base class StringTokenizer. This is explained more fully in Section 7.3.*

*This method nextToken also has its definition overridden.*

*super.nextToken is the version of nextToken defined in the base class StringTokenizer.*

*tokensSoFar is a new method.*

Display 7.8 Use of the `EnhancedStringTokenizer` Class (part 1 of 2)

```

1 import java.util.Scanner;

2 public class EnhancedStringTokenizerDemo
3 {
4 public static void main(String[] args)
5 {
6 Scanner keyboard = new Scanner(System.in);

7 System.out.println("Enter a sentence:");
8 String sentence = keyboard.nextLine();

```

(continued)

Display 7.8 Use of the `EnhancedStringTokenizer` Class (part 2 of 2)

```
9 EnhancedStringTokenizer wordFactory =
10 new EnhancedStringTokenizer(sentence);
11 System.out.println("Your sentence with extra blanks deleted:");
12 while (wordFactory.hasMoreTokens())
13 System.out.print(wordFactory.nextToken() + " ");
14 System.out.println();
15 //All tokens have been dispensed.

16 System.out.println("Sentence with each word on a separate line:");
17 String[] token = wordFactory.tokensSoFar();
18 for (int i = 0; i < token.length; i++)
19 System.out.println(token[i]);
20 }
21 }
```

## Sample Dialogue

```
Enter a sentence:
 I love you, madly.
Your sentence with extra blanks deleted:
I love you, madly.
Sentence with each word on a separate line:
I
love
you,
madly.
```

## 7.2 Encapsulation and Inheritance

*Ignorance is bliss.*

ANONYMOUS

This section is a continuation of Section 7.1 and uses the same example classes we used there. In this section, we consider how the information-hiding facilities of Java, primarily the `private` modifier, interact with inheritance.



## PITFALL: Use of Private Instance Variables from the Base Class

An object of the class `HourlyEmployee` (Display 7.3) inherits, among other things, an instance variable called `name` from the class `Employee` (Display 7.2). For example, the following would set the value of the instance variable `name` of the `HourlyEmployee` object `joe` to `"Josephine"`:

```
joe.setName("Josephine");
```

But you must be a bit careful about how you manipulate inherited instance variables such as `name`. The instance variable `name` of the class `HourlyEmployee` was inherited from the class `Employee`, but the instance variable `name` is a private instance variable in the definition of the class `Employee`. That means that `name` can only be accessed by `name` within the definition of a method in the class `Employee`. An instance variable (or method) that is private in a base class is not accessible *by name* in the definition of a method in *any other class, not even in a method definition of a derived class*.

For example, notice the following method definition taken from the definition of the class `HourlyEmployee` in Display 7.3:

```
public String toString()
{
 return (getName() + " " + getHireDate().toString()
 + "\n$" + wageRate + " per hour for " + hours + " hours");
}
```

You might wonder why we needed to use the methods `getName` and `getHireDate`. You might be tempted to rewrite the method definition as follows:

```
public String toString() //Illegal version
{
 return (name + " " + hireDate.toString()
 + "\n$" + wageRate + " per hour for " + hours + " hours");
}
```

As the comment indicates, this will not work. The instance variables `name` and `hireDate` are private instance variables in the class `Employee`, and although a derived class such as `HourlyEmployee` inherits these instance variables, it cannot access them directly. You must instead use some public methods to access the instance variable `name` or `hireDate`, as we did in Display 7.3.

In the definition of a derived class, you cannot mention a private inherited instance variable by name. You must instead use public accessor and mutator methods (such as `getName` and `setName`) that were defined in the base class.

The fact that a private instance variable of a base class cannot be accessed in the definition of a method of a derived class often seems wrong to people. After all, if you are an hourly employee and you want to change your name, nobody says, "Sorry, `name` is a private instance variable of the class `Employee`." If you are an hourly employee, you

(continued)

**PITFALL: (continued)**

are also an employee. In Java, this is also true; an object of the class `HourlyEmployee` is also an object of the class `Employee`. However, the laws on the use of private instance variables and methods must be as we described, or else they would be compromised. If private instance variables of a class were accessible in method definitions of a derived class, then anytime you want to access a private instance variable, you could simply create a derived class and access it in a method of that class, which would mean that all private instance variables would be accessible to anybody who wants to put in a little extra effort. This scenario illustrates the problem, but the big problem is unintentional errors, not intentional subversion. If private instance variables of a class were accessible in method definitions of a derived class, then the instance variables might be changed by mistake or in inappropriate ways. (Remember, accessor and mutator methods can guard against inappropriate changes to instance variables.)

We will discuss one possible way to get around this restriction on private instance variables of the base class in the upcoming subsection entitled “Protected and Package Access.” ■

**Self-Test Exercises**

7. Would the following be legal for the definition of a method to add to the class `Employee` (Display 7.2)? (Remember, the question is whether it is legal, not whether it is sensible.)

```
public void crazyMethod()
{
 Employee object = new Employee("Joe",
 new Date("January", 1, 2005));
 System.out.println("Hello " + object.name);
}
```

Would it be legal to add this `crazyMethod` to the class `HourlyEmployee`?

8. Suppose you change the modifier before the instance variable name from `private` to `public` in the class `Employee`. Would it then be legal to add the method `crazyMethod` (from Self-Test Exercise 7) to the class `HourlyEmployee`?

**PITFALL: Private Methods Are Effectively Not Inherited**

As we noted in the Pitfall section, “Use of Private Instance Variables from the Base Class,” an instance variable (or method) that is private in a base class is not directly accessible outside of the definition of the base class, *not even in a method definition for a derived class*. The private methods of the base class are just like private variables in terms of not being directly available. But in the case of methods, the restriction



## PITFALL: (continued)

is more dramatic. A private variable can be accessed indirectly via an accessor or mutator method. A private method is simply not available. It is just as if the private method were not inherited. (In one sense, private methods in the base class may be indirectly available in the derived class. If a private method is used in the definition of a public method of the base class, then that public method can be invoked in the derived class, or any other class, so the private method can be indirectly invoked.)

This should not be a problem. Private methods should just be used as helping methods, so their use should be limited to the class in which they are defined. If you want a method to be used as a helping method in a number of inherited classes, then it is not *just* a helping method, and you should make the method public. ■

## Protected and Package Access

As you have seen, you cannot access (by name) a private instance variable or private method of the base class within the definition of a derived class. There are two classifications of instance variables and methods that allow them to be accessed by name in a derived class. The two classifications are *protected access*, which always gives access, and *package access*, which gives access if the derived class is in the same package as the base class.

If a method or instance variable is modified by `protected` (rather than `public` or `private`), then it can be accessed by name inside its own class definition, it can be accessed by name inside any class derived from it, and it can also be accessed by name in the definition of any class in the same package (even if the class in the same package is not derived from it). However, the `protected` method or instance variable cannot be accessed by name in any other classes. Thus, if an instance variable is marked `protected` in the class `Parent` and the class `Child` is derived from the class `Parent`, then the instance variable can be accessed by name inside any method definition in the class `Child`. However, in a class that is not in the same package as `Parent` and is not derived from `Parent`, it is as if the `protected` instance variable were `private`.

For example, consider the class `HourlyEmployee` that was derived from the base class `Employee`. We were required to use accessor and mutator methods to manipulate the inherited instance variables in the definition of `HourlyEmployee`. Consider the definition of the `toString` method of the class `HourlyEmployee`, which we repeat here:

```
public String toString()
{
 return (getName() + " " + getHireDate().toString()
 + "\n$" + wageRate + " per hour for " + hours + " hours");
}
```

If the private instance variables `name` and `hireDate` had been marked `protected` in the class `Employee`, the definition of `toString` in the derived class `HourlyEmployee` could be simplified to the following:

```
public String toString() //Legal if instance variables in
 // Employee are marked protected
{
 return (name + " " + hireDate.toString()
 + "\n$" + wageRate + " per hour for " + hours + " hours");
}
```

### The protected Modifier

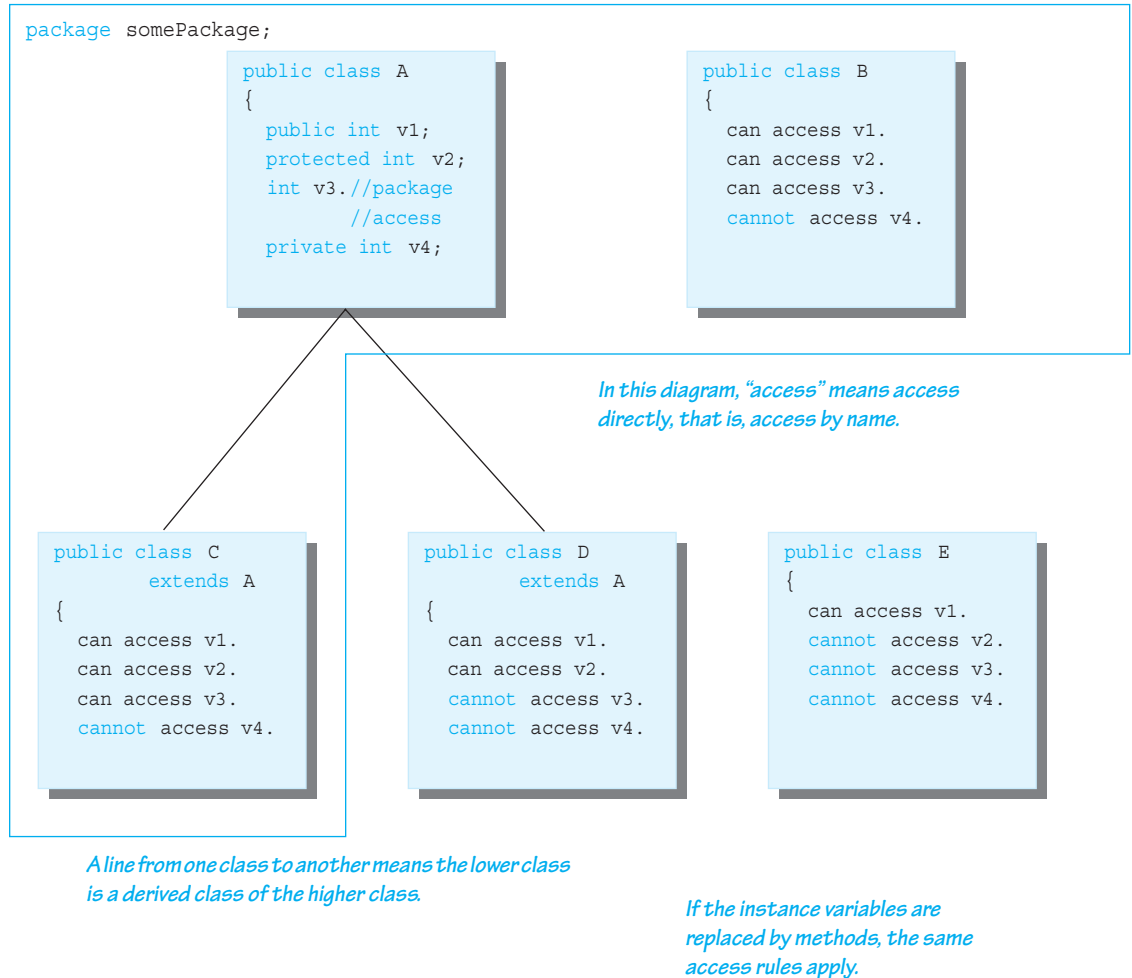
If a method or instance variable is modified by `protected` (rather than `public` or `private`), then it can be accessed by name inside its own class definition, by name inside any class derived from it, and by name in the definition of any class in the same package.

The `protected` modifier provides very weak protection compared to the `private` modifier, because it allows direct access to any programmer who is willing to go through the bother of defining a suitable derived class. Many programming authorities discourage the use of the `protected` modifier. Instance variables should normally not be marked `protected`. On rare occasions, you may want to have a method marked `protected`. If you want an access intermediate between `public` and `private`, then the access described in the next paragraph is often a preferable alternative to `protected`.

You may have noticed that if you forget to place one of the modifiers `public`, `private`, or `protected` before an instance variable or method definition, then your class definition will still compile. If you do not place any of these modifiers before an instance variable or method definition, then the instance variable or method can be accessed by name inside the definition of any class in the same package but not outside of the package. This is called **package access**, **default access**, or **friendly access**. Use package access in situations where you have a package of cooperating classes that act as a single encapsulated unit. Note that package access is more restricted than `protected`, and that package access gives more control to the programmer defining the classes. If you control the package directory (folder), then you control who is allowed package access.

The diagram in Display 7.9 may help you to understand who has access to members with `public`, `private`, `protected`, and `package` access. The diagram tells who can directly access, by name, variables that have `public`, `private`, `protected`, and `package` access. The same access rules apply to methods that have `public`, `private`, `protected`, and `package` access.

Display 7.9 Access Modifiers



## Package Access

If you do not place any of the modifiers `public`, `private`, or `protected` before an instance variable or method definition, then the instance variable or method is said to have **package access**. Package access is also known as **default access** and as **friendly access**. If an instance variable or method has package access, it can be accessed by name inside the definition of any class in the same package, but not outside of the package.





### PITFALL: Forgetting about the Default Package

When considering package access, do not forget the default package. Recall that all the classes in your current directory (that do not belong to some other package) belong to an unnamed package called the *default package*. So, if a class in your current directory is not in any other package, then it is in the default package. If an instance variable or method has package access, then that instance variable or method can be accessed by name in the definition of any other class in the default package. ■



### PITFALL: A Restriction on Protected Access ★

The situation described in this pitfall does not occur often, but when it does, it can be very puzzling if you do not understand what is going on.

Suppose class `D` is derived from class `B`, the instance variable `n` has protected access in class `B`, and the classes `D` and `B` are in different packages, so the class definitions begin as follows:

```
package one;

public class B
{
 protected int n;
 ...
}

package two;

import one.B;

public class D extends B
{
 ...
}
```

Then the following is a legitimate method that can appear in the definition of class `D`:

```
public void demo()
{
 n = 42; //n is inherited from B.
}
```

**PITFALL: (continued)**

The following is also a legitimate method definition for the derived class `D`:

```
public void demo2()
{
 D object = new D();
 object.n = 42; //n is inherited from B.
}
```

However, the following is not allowed as a method of `D`:

```
public void demo3()
{
 B object = new B();
 object.n = 42; //Error
}
```

The compiler will give an error message saying that `n` is protected in `B`.

Similar remarks apply to protected methods.

A class can access its own classes' inherited variables and methods that are marked protected in the base class, but cannot directly access any such instance variables or methods of an object of the base class (or of any other derived class of the base class). In the above example, `n` is an instance variable of the base class `B` and an instance variable of the derived class `D`. `D` can access `n` whenever `n` is used as an instance variable of `D`, but `D` cannot access `n` when `n` is used as an instance variable of `B`.

If the classes `B` and `D` are in the same package, you will not get the error message because, in Java, protected access implies package access. In particular, if the classes `B` and `D` are both in the default package, you will not get the error message. ■

**Self-Test Exercises**

9. Suppose you change the modifier before the instance variable `name` from `private` to `protected` in the class `Employee` (Display 7.2). Would it then be legal to add the method `crazyMethod` (from Self-Test Exercise 7) to the class `HourlyEmployee` (Display 7.3)?
10. Which is more restricted, protected access or package access?

(continued)

**Self-Test Exercises** (continued)

11. Suppose class `D` is derived from class `B`, the method `doStuff()` has protected access in class `B`, and the classes `D` and `B` are in different packages, so the class definitions begin as follows:

```
package one;
public class B
{
 protected void doStuff()
 {
 ...
 }
}

package two;
import one.B;
public class D extends B
{
 ...
}
```

Is the following a legitimate method that can appear in the definition of the class `D`?

```
public void demo()
{
 doStuff(); //doStuff is inherited from B.
}
```

12. Suppose `B` and `D` are as described in Self-Test Exercise 11. Is the following a legitimate method that can appear in the definition of the class `D`?

```
public void demo2()
{
 D object = new D();
 object.doStuff(); //doStuff is inherited from B.
}
```

13. Suppose `B` and `D` are as described in Self-Test Exercise 11. Is the following a legitimate method that can appear in the definition of the class `D`?

```
public void demo3()
{
 B object = new B();
 object.doStuff();
}
```

## 7.3 Programming with Inheritance

*The devil is in the details.*

ANONYMOUS

In the previous section, we described the basic idea and details of derived classes. In this section, we continue that discussion and go on to cover some more subtle points about derived classes. In the process, we also discuss the class `Object`, which is an ancestor class of all Java classes, and we describe a better way to define an `equals` method.



### TIP: Static Variables Are Inherited

Static variables in a base class are inherited by any derived classes. The modifiers `public`, `private`, and `protected`, and package access have the same meaning for static variables as they do for instance variables. ■



### TIP: “is a” versus “has a”

Early in this chapter, we defined a derived class called `HourlyEmployee` using the class `Employee` as the base class. In such a case, an object of the derived class `HourlyEmployee` is also an instance of the class `Employee`, or, stated more simply, an `HourlyEmployee` *is an* `Employee`. This is an example of the **“is a” relationship** between classes. It is one way to make a more complex class out of a simpler class.

Another way to make a more complex class out of a simpler class is known as the **“has a” relationship**. For example, the class `Employee` defined earlier has an instance variable of the class type `Date`. We express this relationship by saying an `Employee` “has a” `Date`. Using the “has a” relationship to build a class (such as building the class `Employee` by using `Date` as an instance variable) is often called **composition**.

Because the class `HourlyEmployee` inherits the instance variable of type `Date` from the class `Employee`, it is also correct to say an `HourlyEmployee` “has a” `Date`. Thus, an `HourlyEmployee` *is an* `Employee` and *has a* `Date`. ■

### Access to a Redefined Base Method

Suppose you redefine a method so that it has a different definition in the derived class from what it has in the base class. The definition that was given in the base class is not completely lost to the derived class objects. However, if you want to invoke the version of the method given in the base class with an object in the derived class, you need some way to say, “use the definition of this method as given in the base class (even though I am an object of the derived class).” The way you say this is to use the keyword `super` as if it were a calling object.

“is a”  
relationship

“has a”  
relationship

composition

super  
relationship

For example, the method `toString()` of the class `HourlyEmployee` (Display 7.3) is defined as follows:

```
public String toString() //in the derived class HourlyEmployee
{
 return (getName() + " " + getHireDate().toString()
 + "\n$" + wageRate + " per hour for " + hours + " hours");
}
```

This overrides the following definition of `toString()` that was given in the definition of the base class `Employee`:

```
public String toString() //in the base class Employee
{
 return (name + " " + hireDate.toString());
}
```

We can use the version of the method `toString()` defined in the base class `Employee` to simplify the definition of the method `toString()` in the derived class `HourlyEmployee`. The following is an equivalent way to define `toString()` in the derived class `HourlyEmployee`:

```
public String toString() //in the derived class HourlyEmployee
{
 return (super.toString()
 + "\n$" + wageRate + " per hour for " + hours + " hours");
}
```

The expression `super.toString()` is an invocation of the method `toString()` using the definition of `toString()` given in the base class `Employee`.

You can only use `super` in this way within the definition of a method in a derived class. Outside of the definition of the derived class, you cannot invoke an overridden method of the base class using an object of the derived class.

### Invoking the Old Version of an Overridden Method?

Within the definition of a method of a derived class, you can invoke the base class version of an overridden method of the base class by prefacing the method name with `super` and a dot. Outside of the derived class definition, there is no way to invoke the base class version of an overridden method using an object of the derived class.

#### EXAMPLE

```
public String toString()
{
 return (super.toString()
 + "\n$" + wageRate + " per hour for " + hours + " hours");
}
```



## PITFALL: You Cannot Use Multiple `super`s

As we already noted, within the definition of a method of a derived class, you can call an overridden method of the base class by prefacing the method name with `super` and a dot. However, you cannot repeat the use of `super` to invoke a method from some ancestor class other than a direct parent. For example, suppose that the class `Employee` were derived from the class `Person`, and the class `HourlyEmployee` is derived from the class `Employee`. You might think that you can invoke a method of the class `Person` within the definition of the class `HourlyEmployee` by using `super.super`, as in

```
super.super.toString() //ILLEGAL!
```

However, as the comment indicates, it is illegal to have such multiple `super`s in Java. ■

## Self-Test Exercises

14. Redefine the `toString` method of the class `SalariedEmployee` (Display 7.5) so that it uses `super.toString()`. This new definition of `toString` will be equivalent to the one given in Display 7.5.
15. Redefine the `equals` method for the class `HourlyEmployee` (Display 7.3) using `super.equals` to invoke the `equals` method of the base class `Employee`.
16. Is the following program legal? The relevant classes are defined in Displays 7.2 and 7.3.

```
public class EmployeeDemo
{
 public static void main(String[] args)
 {
 HourlyEmployee joe =
 new HourlyEmployee("Joe Young",
 new Date("Feb", 1, 2015), 10.50, 40);
 String nameNDate = joe.super.toString();
 System.out.println(nameNDate);
 }
}
```

(continued)

### Self-Test Exercises (continued)

17. Suppose you add the following defined constant to the class `Employee` (Display 7.2):

```
public static final int STANDARD_HOURS = 160; //per month
```

Would it then be legal to add the following method to the class `HourlyEmployee` (Display 7.3)?

```
public void setHoursToStandard()
{
 hours = STANDARD_HOURS;
}
```

### The Class `Object`

**object  
class**

Java has a class that is an ancestor of every class. In Java, every class is a derived class of a derived class of . . . (for some number of iterations of “a derived class of”) of the class `Object`. So, every object of every class is of type `Object`, as well as being of the type of its class (and also of the types of all its ancestor classes). Even classes that you define yourself are descendent classes of the class `Object`. If you do not make your class a derived class of some class, then Java will automatically make it a derived class of the class `Object`.

The class `Object` allows you to write Java code for methods with a parameter of type `Object` that can be replaced by an object of any class whatsoever. You will eventually encounter library methods that accept an argument of type `Object` and hence can be used with an argument that is an object of absolutely any class.

### The Class `Object`

In Java, every class is a descendent of the class `Object`. So, every object of every class is of type `Object`, as well as being of the type of its class.

The class `Object` is in the package `java.lang`, which is always imported automatically. So, you do not need any `import` statement to make the class `Object` available to your code.

The class `Object` does have some methods that every Java class inherits. For example, every object inherits the methods `equals` and `toString` from some ancestor class, which either is the class `Object` or a class that itself inherited the methods ultimately from the class `Object`. However, the inherited methods `equals` and `toString` will not work correctly for (almost) any class you define. You need to override the inherited method definitions with new, more appropriate definitions.

**toString**

**equals**

It is important to include definitions of the methods `toString` and `equals` in the classes you define, because some Java library classes assume your class has such methods. There are no subtleties involved in defining (actually redefining or overriding) the method `toString`. We have seen good examples of the method `toString` in many of our class definitions. The definition of the overridden method `equals` does have some subtleties; we discuss them in the next subsection.

**clone**

Another method inherited from the class `Object` is the method `clone`, which is intended to return a copy of the calling object. We discuss the method `clone` in Chapters 8 and 13.

## The Right Way to Define `equals`

Earlier we said that the class `Object` has an `equals` method, and that when you define a class with an `equals` method, you should override the definition of the method `equals` given in the class `Object`. However, we did not, strictly speaking, follow our own advice. The heading for the method `equals` in our definition of the class `Employee` (Display 7.2) is as follows:

```
public boolean equals(Employee otherEmployee)
```

On the other hand, the heading for the method `equals` in the class `Object` is as follows:

```
public boolean equals(Object otherObject)
```

The two `equals` methods have different parameter types, so we have not overridden the definition of `equals`. We have merely overloaded the method `equals`. The class `Employee` has both of these methods named `equals`.

In most situations, this will not matter. However, there are situations in which it does. Some library methods assume your class's definition of `equals` has the following heading, the same as in the class `Object`:

```
public boolean equals(Object otherObject)
```

We need to change the type of the parameter for the `equals` method in the class `Employee` from type `Employee` to type `Object`. A first try might produce the following:

```
public boolean equals(Object otherObject)
{
 Employee otherEmployee = (Employee)otherObject;
 return (name.equals(otherEmployee.name)
 && hireDate.equals(otherEmployee.hireDate));
}
```

We needed to type cast the parameter `otherObject` from type `Object` to type `Employee`. If we omit the type cast and simply proceed with `otherObject`, the compiler will give an error message when it sees the following:

```
otherObject.name
```

The class `Object` does not have an instance variable named `name`.

This first try at an improved `equals` method does override the definition of `equals` given in the class `Object` and will work well in many cases. However, it still has a shortcoming.

Our definition of `equals` now allows an argument that can be any kind of object whatsoever. What happens if the method `equals` is used with an argument that is not an `Employee`? A run-time error will occur when the type cast to `Employee` is executed.



We need to make our definition work for any kind of object. If the object is not an `Employee`, we simply return `false`. The calling object is an `Employee`, so if the argument is not an `Employee`, they should not be considered equal. But how can we tell whether the parameter is or is not of type `Employee`?

Every object inherits the method `getClass()` from the class `Object`. The method `getClass()` is marked `final` in the class `Object`, so it cannot be overridden. For any object `o`, `o.getClass()` returns a representation of the class used to create `o`. For example, after the following is executed:

```
o = new Employee();
```

`o.getClass()` returns a representation `Employee`.

We will not describe the details of this representation except to say that two such representations should be compared with `==` or `!=` if you want to know if two representations are the same. Thus,

```
if (object1.getClass() == object2.getClass())
 System.out.println("Same class.");
else
 System.out.println("Not the same class.");
```

will output "Same class." if `object1` and `object2` were created with the same class when they were created using `new`, and output "Not the same class." otherwise.

Our final version of the method `equals` is shown in Display 7.10. Note that we have also taken care of one more possible case. The predefined constant `null` can be plugged in for a parameter of type `Object`. The Java documentation says that an `equals` method should return `false` when comparing an object and the value `null`. So that is what we have done.

On the accompanying website, the subdirectory `improvedEquals` (of the directory for this chapter) has a definition of the class `Employee` that includes this definition of `equals`.

extra code  
on website

Display 7.10 A Better `equals` Method for the Class `Employee`

---

```
1 public boolean equals(Object otherObject)
2 {
3 if (otherObject == null)
4 return false;
5 else if (getClass() != otherObject.getClass())
6 return false;
7 else
8 {
9 Employee otherEmployee = (Employee)otherObject;
10 return (name.equals(otherEmployee.name)
11 && hireDate.equals(otherEmployee.hireDate));
12 }
13 }
```

---



### TIP: getClass versus instanceof ★

Many authors suggest that in the definition of equals for a class such as Employee, given in Display 7.10, you should not use

```
else if (getClass() != otherObject.getClass())
 return false;
```

but should instead use

```
else if (!(otherObject instanceof Employee))
 return false;
```

What is the difference, and which should you use? At first glance, it seems like you should use instanceof in the definition of equals. The instanceof operator checks to see if an object is of the type given as its second argument. The syntax is

*Object* instanceof *Class\_Name*

which returns true if *Object* is of type *Class\_Name*; otherwise it returns false. So, the following will return true if otherObject is of type Employee:

```
(otherObject instanceof Employee)
```

Suppose that (contrary to what we really did) we instead used instanceof in our definition of equals for the class Employee and also used instanceof in our definition for the class HourlyEmployee, so that the definition of equals for HourlyEmployee is as follows:

```
public boolean equals(Object otherObject)
//This is NOT the right way to define equals.
{
 if (otherObject == null)
 return false;
 else if (!(otherObject instanceof HourlyEmployee))
 return false;
 else
 {
 HourlyEmployee otherHourlyEmployee =
 (HourlyEmployee) otherObject;
 return (super.equals(otherHourlyEmployee)
 && (wageRate == otherHourlyEmployee.wageRate)
 && (hours == otherHourlyEmployee.hours));
 }
}
```

(continued)

instanceof

**TIP: (continued)**

Assuming that the `equals` method for both `Employee` and `HourlyEmployee` are defined using `instanceof` (as previously mentioned), consider the following situation:

```
Employee e =
 new Employee("Joe Worker", new Date("January", 1, 2004));
HourlyEmployee hourlyE = new HourlyEmployee("Joe Worker",
 new Date("January", 1, 2004), 50.50, 160);
```

Then, with the definition of `equals` that uses `instanceof`, we get that

```
e.equals(hourlyE)
```

returns `true`, because `hourlyE` is an `Employee` with the same name and hire date as `e`. So far, it sounds reasonable.

However, since we are assuming that we also used `instanceof` in the definition of `equals` for the class `HourlyEmployee`, we also get that

```
hourlyE.equals(e)
```

returns `false` because `e instanceof HourlyEmployee` returns `false`. (`e` is an `Employee` but `e` is not an `HourlyEmployee`.)

So, if we define `equals` in both classes using `instanceof`, then `e` equals `hourlyE`, but `hourlyE` does not equal `e`. That is no way for `equals` to behave.

Since `instanceof` does not yield a suitable definition of `equals`, you should instead use `getClass()` as we did in Display 7.10. If we use `getClass()` in a similar way in the definition of `equals` for the class `HourlyEmployee` (see Self-Test Exercise 19), then

```
e.equals(hourlyE)
```

and

```
hourlyE.equals(e)
```

both return `false`. ■

## **instanceof and getClass() ★**

Both the `instanceof` operator and the `getClass()` method can be used to check the class of an object. The `instanceof` operator simply tests an object for type. The `getClass()` method, used in a test with `==` or `!=`, tests if two objects were created with the same class. The details follow.

### **THE instanceof OPERATOR**

The `instanceof` operator checks if an object is of the type given as its second argument. The syntax is

```
Object instanceof Class_Name
```

which returns `true` if *Object* is of type *Class\_Name*; otherwise it returns `false`. So, the following will return `true` if `otherObject` is of type `Employee`:

```
(otherObject instanceof Employee)
```

Note that this means it returns `true` if `otherObject` is of the type of any descendent class of `Employee`, because in that case `otherObject` is also of type `Employee`.

### **THE getClass() METHOD**

Every object inherits the method `getClass()` from the class `Object`. The method `getClass()` is marked `final` in the class `Object`, so it cannot be overridden. For any object of any class,

```
object.getClass()
```

returns a representation of the class that was used with `new` to create `object`. Any two such representations can be compared with `==` or `!=` to determine whether or not they represent the same class. Thus,

```
if (object1.getClass() == object2.getClass())
 System.out.println("Same class.");
else
 System.out.println("Not the same class.");
```

will output `Same class` if `object1` and `object2` were created with the same class when they were created using `new`, and output `Not same class` otherwise.

(continued)

**EXAMPLE**

Suppose that `HourlyEmployee` is a derived class of `Employee` and that `employeeObject` and `hourlyEmployeeObject` are created as follows:

```
Employee employeeObject = new Employee();
HourlyEmployee hourlyEmployeeObject = new HourlyEmployee();
```

Then,

```
employeeObject.getClass() == hourlyEmployeeObject.getClass()
```

returns false.

```
employeeObject instanceof Employee
```

returns true.

```
hourlyEmployeeObject instanceof Employee
```

returns true.

```
employeeObject instanceof HourlyEmployee
```

returns false.

```
hourlyEmployeeObject instanceof HourlyEmployee
```

returns true.

**Self-Test Exercises**

18. Redefine the method `equals` given in Display 7.10 using `instanceof` instead of `getClass()`. Give the complete definition. Remember, we do not want you to define `equals` this way in your class definitions; this is just an exercise.
19. Redefine the `equals` method of the class `HourlyEmployee` (Display 7.3) so that it has a parameter of type `Object` and follows the other guidelines we gave for an `equals` method. Assume the definition of the method `equals` for the class `Employee` has been changed to be as in Display 7.10. (Remember, you should use `getClass()`, not `instanceof`.)

**Self-Test Exercises** (continued)

20. Redefine the `equals` method of the class `SalariedEmployee` (Display 7.5) so that it has a parameter of type `Object` and follows the other guidelines we gave for an `equals` method. Assume the definition of the method `equals` for the class `Employee` has been changed to be as in Display 7.10. (Remember, you should use `getClass()`, not `instanceof`.)
21. Redefine the `equals` method of the class `Date` (Display 4.13) so that it has a parameter of type `Object` and follows the other guidelines we gave for an `equals` method. (Remember, you should use `getClass()`, not `instanceof`.)
22. What is the output produced by the following program? (The classes `Employee` and `HourlyEmployee` were defined in this chapter.)

```
public class Test
{
 public static void main(String[] args)
 {
 Employee object1 = new Employee();
 Employee object2 = new HourlyEmployee();

 if (object1.getClass() == object2.getClass())
 System.out.println("Same class.");
 else
 System.out.println("Not the same class.");
 }
}
```

23. (This exercise requires that you have covered the starred subsection “An Enhanced `StringTokenizer` Class \*,” earlier in this chapter.)

Although we did not discuss it when we covered the class `StringTokenizer`, the class `StringTokenizer` has a method with the following heading:

```
public Object nextElement()
```

The method `nextElement()` returns the same string as the method `nextToken()`, but `nextElement()` returns it as something of type `Object`, as opposed to type `String`. Give a suitable definition of `nextElement` to add to the definition of `EnhancedStringTokenizer`. This definition will override the definition of `nextElement` in the class `StringTokenizer`. (*Hint*: the definition is just like the definition of `nextToken` except for fixing the type of the string returned.)

## Chapter Summary

- *Inheritance* provides a tool for code reuse by deriving one class from another. The *derived class* automatically inherits the features of the old (base) class and may add features as well.
- A derived class object inherits the instance variables, static variables, and public methods of the *base class* and may add additional instance variables, static variables, and methods.
- An object of a derived class has the type of the derived class, and it also has the type of the base class, and more generally, has the type of every one of its *ancestor classes*.
- If an instance variable is marked `private` in a base class, then it cannot be accessed by name in a derived class.
- Private methods are effectively not inherited.
- A method may be redefined in a derived class so that it performs differently from how it performs in the base class. This is called *overriding* the method definition. The definition for an overridden method is given in the class definition of the derived class, in the same way as the definitions of any added methods.
- A constructor of a base class can be used in the definition of a constructor for a derived class. The keyword `super` is used as the name for a constructor of the base class.
- A constructor definition can use the keyword `this`, as if it were a method name, to invoke a constructor of the same class.
- If a constructor does not contain an invocation of either `super` or `this`, then Java automatically inserts an invocation of `super()` as the first action in the body of the constructor definition.
- A `protected` instance variable or method in the base class can be accessed by name in the definition of a method of a derived class and in the definition of any method in the same package.
- If an instance variable or method has none of the modifiers `public`, `private`, or `protected`, then it is said to have *package access*. An instance variable or method with package access can be accessed by name in the definition of any method in the same package.
- The class `Object` is an ancestor class of every class in Java.
- The `equals` method for a class should have `Object` as the type of its one parameter.

## Answers to Self-Test Exercises

1. Yes, it will have the instance variables. A derived class has all the instance variables that the base class has and can add more instance variables besides.
2. Yes, it will have the methods. A derived class has all the public methods that the base class has and can also add more methods. If the derived class does not override (redefine) a method definition, then it performs exactly the same action in the derived class as it does in the base class. However, the base class can contain an overriding definition of (a new definition of) a method, and the new definition will replace the old definition (provided it has the same number and types of parameters).
3. The class `DiscountSale` will have two methods named `getTax` and will have the following two headings. This is an example of overloading.

```
public double getTax()
public double getTax(double rate)
```

4. The method `getName` is inherited from the class `Employee` and so needs no definition. The method `getRate` is a new method added in the class `HourlyEmployee` and so needs to be defined.
5. Yes. You can plug in an object of a derived class for a parameter of the base class type. An `HourlyEmployee` is an `Employee`. A `SalariedEmployee` is an `Employee`.
6. `public class TitledEmployee extends SalariedEmployee`

```
{
 private String title;

 public TitledEmployee()
 {
 super ("no name", new Date("January," 1, 1000), 0);
 title = "No title";
 }

 public TitledEmployee(String theName, String theTitle,
 Date theDate, double theSalary)
 {
 super (theName, theDate, theSalary);
 title = theTitle;
 }

 public String getTitle()
 {
 return title;
 }
}
```



```

 public void setTitle(String theTitle)
 {
 title = theTitle;
 }

 public String getName()
 {
 return (title + super.getName());
 }
}

```

7. It would be legal to add `crazyMethod` to the class `Employee`. It would not be legal to add `crazyMethod` to the class `HourlyEmployee` because, although the class `HourlyEmployee` has an instance variable `name`, `name` is private in the base class `Employee` and so cannot be accessed by name in `HourlyEmployee`.
8. Yes, it would be legal as long as `name` is marked `public` in the base class `Employee`.
9. Yes, it would be legal as long as `name` is marked `protected` in the base class `Employee`.
10. Package access is more restricted. Anything allowed by package access is also allowed by `protected` access, but `protected` access allows even more.
11. Yes, it is legitimate.
12. Yes, it is legitimate.
13. No, it is not legitimate. The compiler will give an error message saying `doStuff()` is `protected` in `B`.

```

14. public String toString()
 {
 return (super.toString()
 + "\n$" + salary + " per year");
 }

```

```

15. public boolean equals(HourlyEmployee other)
 {
 return (super.equals(other)
 && wageRate == other.wageRate
 && hours == other.hours);
 }

```

A better definition of `equals` for the class `HourlyEmployee` is given in Display 7.10.

16. It is not legal. You cannot use `super` in this way. `super.toString()` as used here refers to `toString()` in the class `Employee` and can only be used in definitions of classes derived from `Employee`. Moreover, you cannot have a calling object, such as `joe`, before `super`, so this is even illegal if you add `extends Employee` to the first line of the class definition.

17. Yes, all static variables are inherited. Because a defined constant is a form of static variable, it is inherited. So, the class `HourlyEmployee` inherits the constant `STANDARD_HOURS` from the class `Employee`.

18. `public boolean equals(Object otherObject)`  
*//This is NOT the right way to define equals.*

```
{
 if (otherObject == null)
 return false;
 else if (!(otherObject instanceof Employee))
 return false;
 else
 {
 Employee otherEmployee = (Employee)otherObject;
 return (name.equals(otherEmployee.name)
 && hireDate.equals(otherEmployee.hireDate));
 }
}
```

extra code  
on website

19. A version of the `HourlyEmployee` class with this definition of `equals` is in the subdirectory `improvedEquals` of the `ch07` directory on the accompanying website.

```
public boolean equals(Object otherObject)
{
 if (otherObject == null)
 return false;
 else if (getClass() != otherObject.getClass())
 return false;
 else
 {
 HourlyEmployee otherHourlyEmployee =
 (HourlyEmployee)otherObject;
 return (super.equals(otherHourlyEmployee)
 && (wageRate == otherHourlyEmployee.wageRate)
 && (hours == otherHourlyEmployee.hours));
 }
}
```

extra code  
on website

20. A version of the `SalariedEmployee` class with this definition of `equals` is in the subdirectory `improvedEquals` of the `ch07` directory on the accompanying website.

```
public boolean equals(Object otherObject)
{
 if (otherObject == null)
 return false;
 else if (getClass() != otherObject.getClass())
 return false;
 else
 {
```

```

 SalariedEmployee otherSalariedEmployee =
 (SalariedEmployee) otherObject;
 return (super.equals(otherSalariedEmployee)
 && (salary == otherSalariedEmployee.salary));
 }
}

```

extra code  
on website

21. A version of the `Date` class with this definition of `equals` is in the subdirectory `improvedEquals` of the `ch07` directory on the accompanying website.

```

public boolean equals(Object otherObject)
{
 if (otherObject == null)
 return false;
 else if (getClass() != otherObject.getClass())
 return false;
 else
 {
 Date otherDate = (Date) otherObject;
 return (month.equals(otherDate.month)
 && (day == otherDate.day)
 && (year == otherDate.year));
 }
}

```

22. Not the same class.

extra code  
on website

23. The following is included in the definition of `EnhancedStringTokenizer` on the accompanying website.

```

public Object nextElement()
{
 String token = super.nextToken();
 a[count] = token;
 count++;
 return (Object) token;
}

```

## Programming Projects

1. Define a class named `Person` that contains two instance variables of type `String` that stores the first name and last name of a person and appropriate accessor and mutator methods. Also create a method named `displayDetails` that outputs the details of a person. Next, define a class named `Student` that is derived from `Person`, the constructor for which should receive first name and last name from the class `Student` and also assigns values to student id, course, and teacher name. This class should redefine the `displayDetails` method to person details as well as details of a student. Include appropriate constructor(s). Define a class named `Teacher` that

is derived from `Person`. This class should contain instance variables for the subject name and salary. Include appropriate constructor(s). Finally, redefine the `displayDetails` method to include all teacher information in the printout. Create a main method that creates at least two student objects and two teacher objects with different values and calls `displayDetails` for each.

2. Define a class named `Message` that contains an instance variable of type `String` named `text` that stores any textual content for the `Message`. Create a method named `toString` that returns the text field and also include a method to set this value.

Next, define a class for `SMS` that is derived from `Message` and includes instance variables for the `recipientContactNo`. Implement appropriate accessor and mutator methods. The body of the `SMS` message should be stored in the inherited variable `text`. Redefine the `toString` method to concatenate all text fields.

Similarly, define a class for `Email` that is derived from `Message` and includes an instance variable for the sender, receiver, and subject. The textual contents of the file should be stored in the inherited variable `text`. Redefine the `toString` method to concatenate all text fields.

Create sample objects of type `Email` and `SMS` in your main method. Test your objects bypassing them to the following subroutine that returns true if the object contains the specified keyword in the text property.

```
public static boolean ContainsKeyword(Message messageObject,
 String keyword)
{
 if (messageObject.toString().indexOf(keyword,0) >= 0)
 return true;
 return false;
}
```

Finally, include a method to encode the final message “This is Java” using an encoding scheme, according to which, each character should be replaced by the character that comes after it. For example, if the message contains character B or b, it should be replaced by C or c accordingly, while Z or z should be replaced with an A or a. If the final message is “This is Java”, then the encoded message should be “UijtjtKbwb”.

3. The following is some code designed by J. Hacker for a video game. There is an `Alien` class to represent a monster and an `AlienPack` class that represents a band of aliens and how much damage they can inflict:

```
class Alien
{
 public static final int SNAKE_ALIEN = 0;
 public static final int OGRE_ALIEN = 1;
 public static final int MARSHMALLOW_MAN_ALIEN = 2;

 public int type; // Stores one of the three above types
 public int health; // 0=dead, 100=full strength
 public String name;
```



```
public Alien (int type, int health, String name)
{
 this.type = type;
 this.health = health;
 this.name = name;
}
}
public class AlienPack
{
 private Alien[] aliens;

 public AlienPack (int numAliens)
 {
 aliens = new Alien[numAliens];
 }
 public void addAlien(Alien newAlien, int index)
 {
 aliens[index] = newAlien;
 }
 public Alien[] getAliens()
 {
 return aliens;
 }
}
public int calculateDamage()
{
 int damage = 0;
 for (int i=0; i < aliens.length; i++)
 {
 if (aliens[i].type==Alien.SNAKE_ALIEN)
 {
 damage +=10; // Snake does 10 damage
 }
 else if (aliens[i].type==Alien.OGRE_ALIEN)
 {
 damage +=6; // Ogre does 6 damage
 }
 else if (aliens[i].type==
 Alien.MARSHMALLOW_MAN_ALIEN)
 {
 damage +=1;
 // Marshmallow Man does 1 damage
 }
 }
 return damage;
}
}
```

The code is not very object oriented and does not support information hiding in the `Alien` class. Rewrite the code so that inheritance is used to represent the different types of aliens instead of the “type” parameter. This should result in deletion of the “type” parameter. Also rewrite the `Alien` class to hide the instance variables and create a `getDamage` method for each derived class that returns the amount of damage the alien inflicts. Finally, rewrite the `calculateDamage` method to use `getDamage` and write a `main` method that tests the code.

4. Define a class called `Administrator`, which is a derived class of the class `SalariedEmployee` in Display 7.5. You are to supply the following additional instance variables and methods:
  - An instance variable of type `String` that contains the administrator’s title (such as “Director” or “Vice President”).
  - An instance variable of type `String` that contains the administrator’s area of responsibility (such as “Production”, “Accounting”, or “Personnel”).
  - An instance variable of type `String` that contains the name of this administrator’s immediate supervisor.
  - Suitable constructors, and suitable accessor and mutator methods.
  - A method for reading in an administrator’s data from the keyboard.

Override the definitions for the methods `equals` and `toString` so they are appropriate to the class `Administrator`.

Also, write a suitable test program.

5. Give the definition of a class named `Doctor` whose objects are records for a clinic’s doctors. This class will be a derived class of the class `SalariedEmployee` given in Display 7.5. A `Doctor` record has the doctor’s specialty (such as “Pediatrician”, “Obstetrician”, “General Practitioner”, and so forth; so use the type `String`) and office visit fee (use type `double`). Be sure your class has a reasonable complement of constructors, accessor, and mutator methods, and suitably defined `equals` and `toString` methods. Write a program to test all your methods.
6. Create a class called `Vehicle` that has the manufacturer’s name (type `String`), number of cylinders in the engine (type `int`), and owner (type `Person` given next). Then, create a class called `Truck` that is derived from `Vehicle` and has the following additional properties: the load capacity in tons (type `double` since it may contain a fractional part) and towing capacity in pounds (type `int`). Be sure your class has a reasonable complement of constructors, accessor and mutator methods, and suitably defined `equals` and `toString` methods. Write a program to test all your methods.

The definition of the class `Person` follows. Completing the definitions of the methods is part of this programming project.

```
public class Person
{
 private String name;
```



VideoNote  
Solution to  
Programming  
Project 7.5

```
public Person()
{...}
public Person(String theName)
{...}
public Person(Person theObject)
{...}
public String getName()
{...}
public void setName(String theName)
{...}
public String toString()
{...}
public boolean equals(Object other)
{...}
}
```

7. Give the definition of two classes, `Patient` and `Billing`, whose objects are records for a clinic. `Patient` will be derived from the class `Person` given in Programming Project 7.6. A `Patient` record has the patient's name (inherited from the class `Person`) and primary physician of type `Doctor` defined in Programming Project 7.5. A `Billing` object will contain a `Patient` object, a `Doctor` object, and an amount due of type `double`. Be sure your classes have a reasonable complement of constructors, accessor and mutator methods, and suitably defined `equals` and `toString` methods. First write a driver program to test all your methods, then write a test program that creates at least two patients, at least two doctors, and at least two `Billing` records, and then prints out the total income from the `Billing` records.
8. Programming Project 4.10 required adding an instance variable to the `Pet` class defined in Display 4.15 to indicate if the pet is a dog or cat. A better organization is to define `Pet` as a superclass of the `Dog` and `Cat` classes. This organization eliminates the need for an instance variable to indicate the type of the pet. Do or redo Programming Project 4.10 with inheritance. The `acepromazine()` and `carprofen()` methods should be defined in the `Pet` class to simply return 0. Override both methods in the `Dog` and `Cat` classes to calculate the correct dosage. Write a main method with appropriate tests to exercise the changes.

9. Programming Project 6.18 asked you to use an array of `Strings` to store the fruits and vegetables shipped in a `BoxOfProduce` object for a CSA farm.

Modify your solution further by creating a `Produce` class. This class should have an instance variable of type `String` for the name, appropriate constructors, and a public `toString()` method. Then create a `Fruit` and a `Vegetable` class that are derived from `Produce`. These classes should have constructors that take the name as a `String` and invoke the appropriate constructor from the base class to set the name.

Next, modify the text file of produce so it indicates whether each item is a fruit or a vegetable. Here is one possible organization, although you can use others:

```
Broccoli,Vegetable
Tomato,Fruit
Kiwi,Fruit
Kale,Vegetable
Tomatillo,Fruit
```

Finally, modify the `BoxOfProduce` class so it creates an array of type `Produce` instead of type `String`. The class should read the produce from the text file and create instances of either `Fruit` or `Vegetable`, with the appropriate name, in the array. After a box is finished, loop through the contents of the array and output how many fruit and how many vegetables are in the box. The rest of the program should behave the same as the solution to Programming Project 6.18.



This page intentionally left blank



# Polymorphism and Abstract Classes **8**

## **8.1 POLYMORPHISM** 516

Late Binding 517

The `final` Modifier 519

Example: Sales Records 520

Late Binding with `toString` 527

Downcasting and Upcasting 529

A First Look at the `clone` Method 536

## **8.2 ABSTRACT CLASSES** 541

Abstract Classes 542

# 8

## Polymorphism and Abstract Classes

*Don't make any commitments until you have to.*

HENRY ADAMS

### Introduction

The three main programming mechanisms that constitute object-oriented programming (OOP) are encapsulation, inheritance, and polymorphism. We have already covered the first two. In this chapter, we discuss polymorphism. *Polymorphism* refers to the ability to associate many meanings to one method name by means of a special mechanism known as *late binding* or *dynamic binding*.

This chapter also covers *abstract classes*, which are classes in which some methods are not fully defined. Abstract classes are designed to be used only as base classes for defining new classes. You cannot create instances of (objects of) an abstract class; you can only create instances of its descendent classes.

Both polymorphism and abstract classes deal with code in which a method is used before it is defined. Although this may sound paradoxical, it all works out smoothly in Java.

### Prerequisites

This chapter requires Chapters 1 through 5 and Chapter 7, with the exception that Section 5.4 on packages and `javadoc` is not required. This chapter does not use any material on arrays from Chapter 6.

Sections 8.1 on polymorphism and 8.2 on abstract classes are independent of each other, and you may cover Section 8.2 before Section 8.1 if you wish.

## 8.1 Polymorphism

*All experience is an arch, to build upon.*

HENRY ADAMS

Inheritance allows you to define a base class and to define software for the base class. That software can then be used not only for objects of the base class but also for objects of any class derived from the base class. *Polymorphism* allows you to make changes in the method definition for the derived classes and to have those changes apply to the software written *in the base class*. This all happens automatically in Java, but it is important to understand the process. To understand polymorphism, we need a concrete example. The next subsection begins with such an example.

## Late Binding

Suppose you are designing software for a graphics package that has classes for several kinds of figures, such as rectangles, circles, ovals, and so forth. Each figure might be an object of a different class. For example, the `Rectangle` class might have instance variables for a height, width, and center point, while the `Circle` class might have instance variables for a center point and a radius. In a well-designed programming project, all of these classes would be descendents of a single parent class called, for example, `Figure`. Now, suppose you want a method to draw a figure on the screen. To draw a circle, you need different instructions from those you need to draw a rectangle. So, each class needs to have a different method to draw its kind of figure. However, because the methods belong to the classes, they can all be called `draw`. If `r` is a `Rectangle` object and `c` is a `Circle` object, then `r.draw()` and `c.draw()` can be methods implemented with different code. All this is not new, but next we are going to expand on this.

Now, the parent class `Figure` may have methods that apply to all figures. For example, it might have a method called `center` that moves a figure to the center of the screen by erasing it and then redrawing it in the center of the screen. The method `center` of the class `Figure` might use the method `draw` to redraw the figure in the center of the screen. When you think of using the inherited method `center` with figures of the classes `Rectangle` and `Circle`, you begin to see that there are complications here.

To make the point clear and more dramatic, let's suppose the class `Figure` is already written and in use, and at some later time you add a class for a brand-new kind of figure—say, the class `Triangle`. Now `Triangle` can be a derived class of the class `Figure`, so the method `center` will be inherited from the class `Figure` and thus should apply to (and perform correctly for!) all `Triangles`. But there is a complication. The method `center` uses `draw`, and the method `draw` is different for each type of figure. But, the method `center` is defined in the class `Figure`, which means the method `center` was compiled before we wrote the code for the method `draw` of the class `Triangle`. When we invoke the method `center` with an object of the class `Triangle`, we want the code for the method `center` to use a method that was not even defined when we compiled the method `center`—namely, the method `draw` for the class `Triangle`. Can this be made to happen in Java? Yes, it can, and moreover, it happens automatically. You need not do anything special when you define either the base class `Figure` or the derived class `Triangle`.

The situation we discussed for the method `center` in the derived class `Triangle` works out as we want because Java uses a mechanism known as **late binding** or **dynamic binding**. Let's see how late binding works in this case involving figure classes.

**Binding** refers to the process of associating a method definition with a method invocation. If the method definition is associated with the method invocation when the code is compiled, that is called **early binding**. If the method definition is associated with the method invocation when the method is invoked (at run time), that is called **late binding** or **dynamic binding**. Java uses late binding for all methods except for a few cases discussed later in this chapter. Let's see how late binding works in the case of our method `center`.

late binding

dynamic  
binding

binding

early binding

Recall that the method `center` was defined in the class `Figure` and that the definition of the method `center` included an invocation of the method `draw`. If, contrary to fact, Java used early binding, then when the code for the method `center` compiles, the invocation of the method `draw` would be bound to the currently available definition of `draw`, which is the one given in the definition of `Figure`. If early binding were used, the method `center` would behave exactly the same for all derived classes of the class `Figure` as it does for objects created using the class `Figure`. But, fortunately, Java uses late binding, so when `center` is invoked by an object of the class `Triangle`, the invocation of `draw` (inside the method `center`) is not bound to a definition of `draw` until the invocation actually takes place. At this point in time, the run-time system knows the calling object is an instance of the class `Triangle` and so uses the definition of `draw` given in the definition of the class `Triangle` (even if the invocation of `draw` is inside the definition of the method `center`). So, the method `center` behaves differently for an object of the class `Triangle` than it would for an object that is just a plain old `Figure`. With late binding, as in Java, things automatically work out the way you normally want them to.

Note that in order for late binding to work, each object must somehow know which definition of each method applies to that object. So, when an object is created in a system using late binding, the description of the object must include (either directly or indirectly) a description of where the appropriate definition of each method is located. This additional overhead is the penalty you pay for the convenience of late binding.



VideoNote  
Late Binding  
Example

### Late Binding

With **late binding**, the definition of a method is not bound to an invocation of the method until run time—in fact, not until the time at which the particular invocation takes place. Java uses late binding (for all methods except those discussed in the Pitfall section entitled “No Late Binding for Static Methods”).

## polymorphism

The terms **polymorphism** and *late binding* are essentially just different words for the same concept. The term *polymorphism* refers to the processes of assigning multiple meanings to the same method name using late binding.

### Polymorphism

**Polymorphism** refers to the ability to associate many meanings to one method name by means of the late binding mechanism. Thus, polymorphism and late binding are really the same topic.

## The `final` Modifier

`final`

You can mark a method to indicate that it cannot be overridden with a new definition in a derived class. Do this by adding the `final` modifier to the method heading, as in the following sample heading:

```
public final void someMethod()
{
 .
 .
 .
}
```

An entire class can be declared `final`, in which case you cannot use it as a base class to derive any other class from it. The syntax for declaring a class to be `final` is illustrated in what follows:

```
public final class SomeClass
{
 .
 .
 .
}
```

If a method is marked as `final`, it means the compiler can use early binding with that particular method, which enables the compiler to be more efficient. However, the added efficiency is normally not great, and we suggest not using the `final` modifier solely for reasons of efficiency. (Also, it can sometimes aid security to mark certain methods as `final`.)

You can view the `final` modifier as a way of turning off late binding for a method (or an entire class). Of course, it does more than just turn off late binding—it turns off the ability to redefine the method in any descendent class.

### The `final` Modifier

If you add the modifier `final` to the definition of a method, it indicates that the method may not be redefined in a derived class. If you add the modifier `final` to the definition of a class, it indicates that the class may not be used as a base class to derive other classes.

**EXAMPLE: Sales Records**

Suppose you are designing a record-keeping program for an automobile parts store. You want to make the program versatile, but you are not sure you can account for all possible situations. For example, you want to keep track of sales, but you cannot anticipate all types of sales. At first, there will only be regular sales to retail customers who go to the store to buy one particular part. However, later you may want to add sales with discounts or mail order sales with a shipping charge. All of these sales will be for an item with a basic price and ultimately will produce some bill. For a simple sale, the bill is just the basic price, but if you later add discounts, then some kinds of bills will also depend on the size of the discount. Now your program needs to compute daily gross sales, which intuitively should just be the sum of all the individual sales bills. You may also want to calculate the largest and smallest sales of the day or the average sale for the day. All of these can be calculated from the individual bills, but many of the methods for computing the bills will not be added until later, when you decide what types of sales you will be dealing with. Because Java uses late binding, you can write a program to total all bills, even though you will not determine the code for some of the bills until later. (For simplicity in this first example, we assume that each sale is for just one item, although we could—but will not here—account for sales of multiple items.)

Display 8.1 contains the definition for a class named `Sale`. All types of sales will be derived classes of the class `Sale`. The class `Sale` corresponds to simple sales of a single item with no added discounts and no added charges. Note that the methods `lessThan` and `equalDeals` both include invocations of the method `bill`. We can later define derived classes of the class `Sale` and define their versions of the method `bill`, and the definitions of the methods `lessThan` and `equalDeals` (which we gave with the class `Sale`) will use the version of the method `bill` that corresponds to the object of the derived class.

For example, Display 8.2 shows the derived class `DiscountSale`. Notice that this class requires a different definition for its version of the method `bill`. Now the methods `lessThan` and `equalDeals`, which use the method `bill`, are inherited from the base class `Sale`. But, when the methods `lessThan` and `equalDeals` are used with an object of the class `DiscountSale`, they will use the version of the method definition for `bill` that was given with the class `DiscountSale`. This is indeed a pretty fancy trick for Java to pull off. Consider the method call `d1.lessThan(d2)` for objects `d1` and `d2` of the class `DiscountSale`. The definition of the method `lessThan` (even for an object of the class `DiscountSale`) is given in the definition of the base class `Sale`, which was compiled before we ever even thought of the class `DiscountSale`. Yet, in the method call `d1.lessThan(d2)`, the line that calls the method `bill` knows enough to use the definition of the method `bill` given for the class `DiscountSale`. This all works out because Java uses late binding.

Display 8.3 gives a sample program that illustrates how the late binding of the method `bill` and the methods that use `bill` work in a complete program.

Display 8.1 The Base Class `Sale` (part 1 of 3)

```
1 /**
2 Class for a simple sale of one item with no tax, discount, or other
 adjustments.
3 Class invariant: The price is always nonnegative; the name is a nonempty
 string.
4 */
5 public class Sale
6 {
7 private String name; //A nonempty string
8 private double price; //nonnegative
9
9 public Sale()
10 {
11 name = "No name yet";
12 price = 0;
13 }
14
14 /**
15 Precondition: theName is a nonempty string; thePrice is nonnegative.
16 */
17 public Sale(String theName, double thePrice)
18 {
19 setName(theName);
20 setPrice(thePrice);
21 }
22
22 public Sale(Sale originalObject)
23 {
24 if (originalObject == null)
25 {
26 System.out.println("Error: null Sale object.");
27 System.exit(0);
28 }
29 //else
30 name = originalObject.name;
31 price = originalObject.price;
32 }
33
33 public static void announcement()
34 {
35 System.out.println("This is the Sale class.");
36 }
37
37 public double getPrice()
38 {
39 return price;
40 }
```

(continued)



Display 8.1 The Base Class `Sale` (part 2 of 3)

```
41 /**
42 Precondition: newPrice is nonnegative.
43 */
44 public void setPrice(double newPrice)
45 {
46 if (newPrice >= 0)
47 price = newPrice;
48 else
49 {
50 System.out.println("Error: Negative price.");
51 System.exit(0);
52 }
53 }

54 public String getName()
55 {
56 return name;
57 }

58 /**
59 Precondition: newName is a nonempty string.
60 */
61 public void setName(String newName)
62 {
63 if (newName != null && newName != "")
64 name = newName;
65 else
66 {
67 System.out.println("Error: Improper name value.");
68 System.exit(0);
69 }
70 }

71 public String toString()
72 {
73 return (name + " Price and total cost = $" + price);
74 }

75 public double bill()
76 {
77 return price;
78 }
```

Display 8.1 The Base Class Sale (part 3 of 3)

```

79 /*
80 Returns true if the names are the same and the bill for the calling
81 object is equal to the bill for otherSale; otherwise returns false.
82 Also returns false if otherObject is null.
83 */
84 public boolean equalDeals(Sale otherSale)
85 {
86 if (otherSale == null)
87 return false;
88 else
89 return (name.equals(otherSale.name)
90 && bill() == otherSale.bill());
91 }

92 /*
93 Returns true if the bill for the calling object is less
94 than the bill for otherSale; otherwise returns false.
95 */
96 public boolean lessThan (Sale otherSale)
97 {
98 if (otherSale == null)
99 {
100 System.out.println("Error: null Sale object.");
101 System.exit(0);
102 }
103 //else
104 return (bill() < otherSale.bill());
105 }

106 public boolean equals(Object otherObject)
107 {
108 if (otherObject == null)
109 return false;
110 else if (getClass() != otherObject.getClass())
111 return false;
112 else
113 {
114 Sale otherSale = (Sale)otherObject;
115 return (name.equals(otherSale.name)
116 && (price == otherSale.price));
117 }
118 }
119 }

```

When invoked, these methods will use the definition of the method bill that is appropriate for each of the objects.

Display 8.2 The Derived Class `DiscountSale` (part 1 of 2)

```

1 /**
2 Class for a sale of one item with discount expressed as a percent of
3 the price, but no other adjustments.
4 Class invariant: The price is always nonnegative; the name is a
5 nonempty string; the discount is always nonnegative.
6 */

7 public class DiscountSale extends Sale
8 {
9 private double discount; //A percent of the price. Cannot be
 //negative.

10 public DiscountSale()
11 {
12 super(); ← The meaning would be unchanged if
13 discount = 0; this line were omitted.
14 }

15 /**
16 Precondition: theName is a nonempty string; thePrice is
17 nonnegative; theDiscount is expressed as a percent of the price
18 and is nonnegative.
19 */
20 public DiscountSale(String theName,
21 double thePrice, double theDiscount)
22 {
23 super (theName, thePrice);
24 setDiscount (theDiscount);
25 }

26 public DiscountSale(DiscountSale originalObject)
27 {
28 super (originalObject);
29 discount = originalObject.discount;
30 }

31 public static void announcement()
32 {
33 System.out.println("This is the DiscountSale class.");
34 }

35 public double bill()
36 {
37 double fraction = discount/100;
38 return (1 - fraction)*getPrice();
39 }

```

Display 8.2 The Derived Class `DiscountSale` (part 2 of 2)

```
39 public double getDiscount()
40 {
41 return discount;
42 }

43 /**
44 * Precondition: Discount is nonnegative.
45 */
46 public void setDiscount(double newDiscount)
47 {
48 if (newDiscount >= 0)
49 discount = newDiscount;
50 else
51 {
52 System.out.println("Error: Negative discount.");
53 System.exit(0);
54 }
55 }

56 public String toString()
57 {
58 return (getName() + " Price = $" + getPrice()
59 + " Discount = " + discount + "%\n"
60 + " Total cost = $" + bill());
61 }

62 public boolean equals(Object otherObject)
63 {
64 The rest of the definition of equals is located in Self-Test Exercise 4.
65 }
```

---

## Display 8.3 Late Binding Demonstration

```

1 /**
2 Demonstrates late binding.
3 */
4 public class LateBindingDemo
5 {
6 public static void main(String[] args)
7 {
8 Sale simple = new Sale("floor mat", 10.00);
9 //One item at $10.00.
10 DiscountSale discount = new DiscountSale("floor mat", 11.00,
11 10); //One item at $11.00 with a 10% discount.
12 System.out.println(simple);
13 System.out.println(discount);
14
15 if (discount.lessThan(simple))
16 System.out.println("Discounted item is cheaper.");
17 else
18 System.out.println("Discounted item is not cheaper.");
19
20 Sale regularPrice = new Sale("cup holder", 9.90);
21 //One item at $9.90.
22 DiscountSale specialPrice = new DiscountSale("cup holder",
23 11.00,10);
24 //One item at $11.00 with a 10% discount.
25 System.out.println(regularPrice);
26 System.out.println(specialPrice);
27
28 if (specialPrice.equalDeals(regularPrice))
29 System.out.println("Deals are equal.");
30 else
31 System.out.println("Deals are not equal.");
32 }
33 }

```

*The method `lessThan` uses different definitions for `discount.bill()` and `simple.bill()`.*

*The method `equalDeals` uses different definitions for `specialPrice.bill()` and `regularPrice.bill()`.*

*The `equalDeals` method says that two items are equal provided they have the same name and the same bill (same total cost). It does not matter how the bill (the total cost) is calculated.*

## Sample Dialogue

```

floor mat Price and total cost = $10.0
floor mat Price = $11.0 Discount = 10.0%
 Total cost = $9.9
Discounted item is cheaper.
cup holder Price and total cost = $9.9
cup holder Price = $11.0 Discount = 10.0%
 Total cost = $9.9
Deals are equal

```

## Self-Test Exercises

1. Explain the difference between the terms *late binding* and *polymorphism*.
2. Suppose you modify the definitions of the class `Sale` (Display 8.1) by adding the modifier `final` to the definition of the method `bill`. How would that change the output of the program in Display 8.3?
3. Would it be legal to add the following method definition to the class `DiscountSale`?

```
public static boolean isAGoodBuy(Sale theSale)
{
 return (theSale.getDiscount() > 20);
}
```

4. Complete the definition of the method `equals` for the class `DiscountSale` (Display 8.2).

## Late Binding with `toString`

In the subsection “The Methods `equals` and `toString`” in Chapter 4, we noted that if you include an appropriate `toString` method in the definition of a class, then you can output an object of the class using `System.out.println`. For example, the following works because `Sale` has a suitable `toString` method:

```
Sale aSale = new Sale("tire gauge", 9.95);
System.out.println(aSale);
```

This produces the following screen output:

```
tire gauge Price and total cost = $9.95
```

This happens because Java uses late binding. We explain this here.

The method invocation `System.out.println(aSale)` is an invocation of the method `println` with the calling object `System.out`. One definition of the method `println` has a single argument of type `Object`. The definition is equivalent to the following:

```
public void println(Object theObject)
{
 System.out.println(theObject.toString());
}
```

(The invocation of the method `println` inside the braces is a different, overloaded definition of the method `println`. That invocation inside the braces uses a method `println` that has a parameter of type `String`, not a parameter of type `Object`.)

This definition of `println` was given before the class `Sale` was defined. Yet in the invocation

```
System.out.println(aSale);
```

with an argument `aSale` of type `Sale` (and hence also of type `Object`), it is the definition of `toString` in the class `Sale` that is used, not the definition of `toString` in the class `Object`. Late binding is what makes this work.



### PITFALL: No Late Binding for Static Methods

Java does not use late binding with private methods, methods marked `final`, or static methods. With private methods and `final` methods, this is not an issue because dynamic binding would serve no purpose anyway. However, with static methods it can make a difference when the static method is invoked using a calling object. Such cases arise more often than you might think.

#### static binding

When Java (or any language) does not use late binding, it uses **static binding**. With static binding, the decision of which definition of a method to use with a calling object is made at compile time based on the type of the variable naming the object.

Display 8.4 illustrates the effect of static binding on a static method with a calling object. Note that the static method `announcement()` in the class `Sale` has its definition overridden in the derived class `DiscountSale`. However, when an object of type `DiscountSale` is named by a variable of type `Sale`, it is the definition `announcement()` in the class `Sale` that is used, not the definition of `announcement` in the class `DiscountSale`.

“So, what’s the big deal?” you may ask. A static method is normally called with a class name and not a calling object. It may look that way, but there are cases where a static method has a calling object in an inconspicuous way. If you invoke a static method within the definition of a nonstatic method but without any class name or calling object, then the calling object is an implicit `this`, which is a calling object. For example, suppose you add the following method to the class `Sale`:

```
public void showAdvertisement()
{
 announcement();
 System.out.println(toString());
}
```

Suppose further that the method `showAdvertisement` is not overridden in the class `DiscountSale`. Then the method `showAdvertisement` is inherited unchanged from `Sale`.



## PITFALL: (continued)

Now consider the following code:

```
Sale s = new Sale("floor mat", 10.00);
DiscountSale discount = new DiscountSale("floor mat", 11.00, 10);
s.showAdvertisement();
discount.showAdvertisement();
```

You might expect the following output:

```
This is the Sale class.
floor mat Price and total cost = $10.0
This is the DiscountSale class.
floor mat Price = $11.0 Discount = 10.0%
Total cost = $9.9
```

However, because the definition used for the static method announcement, inside of `showAdvertisement`, is determined at compile time (based on the type of the variable holding the calling object), the output actually is the following, where the change is shown in blue:

```
This is the Sale class.
floor mat Price and total cost = $10.0
This is the Sale class.
floor mat Price = $11.0 Discount = 10.0%
Total cost = $9.9
```

Java uses late binding with the nonstatic method `toString` but static binding with the static method announcement. ■

## Downcasting and Upcasting

The following is perfectly legal (given the class definitions in Displays 8.1 and 8.2):

```
Sale saleVariable;
DiscountSale discountVariable =
 new DiscountSale("paint", 15, 10);
saleVariable = discountVariable;
System.out.println(saleVariable.toString());
```

An object of a derived class (in this case, the derived class `DiscountSale`) also has the type of its base class (in this case, `Sale`) and so can be assigned to a variable of the base class type. Now let's consider the invocation of the method `toString()` on the last line of the preceding code.



## Display 8.4 No Late Binding with Static Methods ★

```

1 /**
2 * Demonstrates that static methods use static binding.
3 */
4 public class StaticMethodsDemo
5 {
6 public static void main(String[] args)
7 {
8 Sale.announcement();
9 DiscountSale.announcement();
10 System.out.println(
11 "That showed that you can override a static method " +
12 "definition.");
13
14 Sale s = new Sale();
15 DiscountSale discount = new DiscountSale();
16 s.announcement();
17 discount.announcement();
18 System.out.println("No surprises so far, but wait.");
19 Sale discount2 = discount;
20 System.out.println(
21 "discount2 is a DiscountSale object in a Sale variable.");
22 System.out.println("Which definition of announcement() will " +
23 "it use?");
24 discount2.announcement();
25 System.out.println(
26 "It used the Sale version of announcement()!");
27 }
28 }

```

*Java uses static binding with static methods so the choice of which definition of a static method to use is determined by the type of the variable, not by the object.*

*discount and discount2 name the same object, but one is a variable of type Sale and one is a variable of type DiscountSale.*

## Sample Dialogue

```

This is the Sale class.
This is the DiscountSale class.
That showed that you can override a static method definition.
This is the Sale class.
This is the DiscountSale class.
No surprises so far, but wait.
discount2 is a DiscountSale object in a Sale variable.
Which definition of announcement() will it use?
This is the Sale class.
It used the Sale version of announcement()!

```

*If Java had used late binding with static methods, then this would have been the other announcement.*

Because Java uses late binding, the invocation

```
saleVariable.toString()
```

uses the definition of the method `toString` given in the class `DiscountSale`. So the output is

```
paint Price = $15.0 Discount = 10.0%
Total cost = $13.5
```

Because of late binding, the meaning of the method `toString` is determined by the object, not by the type of the variable `saleVariable`.

You may well respond, “Who cares? Why would I ever want to assign an object of type `DiscountSale` to a variable of type `Sale`?”<sup>1</sup> You make such assignments more often than you might think, but you tend to not notice them because they happen behind the scenes. Recall that a parameter is really a local variable, so every time you use an argument of type `DiscountSale` for a parameter of type `Sale`, you are assigning an object of type `DiscountSale` to a variable of type `Sale`. For example, consider the following invocation taken from the definition of the copy constructor for `DiscountSale` (Display 8.2):

```
super(originalObject);
```

In this invocation, `originalObject` is of type `DiscountSale`, but `super` is the copy constructor for the base class `Sale`. Therefore, `super` has a parameter of type `Sale`, which is a local variable of type `Sale` that is set equal to the argument `originalObject` of type `DiscountSale`.

Note that the type of the variable naming an object determines which method names can be used in an invocation with that calling object. (Self-Test Exercise 3 may help you to understand this point.) However, the object itself always determines the meaning of a method invocation performed by an object; this is what we mean by *late binding*.

### An Object Knows the Definitions of Its Methods

The type of a class variable determines which method names can be used with the variable, but the object named by the variable determines which definition of the method name is used. A special case of this rule is the following: The type of a class parameter determines which method names can be used with the parameter, but the argument determines which definition of the method name is used.

<sup>1</sup>It is actually the references to the object that are assigned, not the objects themselves, but that subtlety is not relevant to what we are discussing here, and the language is already complicated enough.

**upcasting**

Assigning an object of a derived class to a variable of a base class (or any ancestor class) is often called **upcasting** because it is like a type cast to the type of the base class. In the normal way of writing inheritance diagrams, base classes are drawn above derived classes.<sup>2</sup>

**downcasting**

When you do a type cast from a base case to a derived class (or from any ancestor class to any descendent class), it is called a **downcast**. Upcasting is pretty straightforward; there are no funny cases to worry about, and in Java things always work out the way you want them to. Downcasting is more troublesome. First of all, downcasting does not always make sense. For example, the downcast

```
Sale saleVariable = new Sale("paint", 15);
DiscountSale discountVariable;
discountVariable = (DiscountSale)saleVariable;//Error
```

does not make sense because the object named by `saleVariable` has no instance variable named `discount` and so cannot be an object of type `DiscountSale`. Every `DiscountSale` is a `Sale`, but not every `Sale` is a `DiscountSale`, as indicated by this example. It is your responsibility to use downcasting only in situations where it makes sense.

It is instructive to note that

```
discountVariable = (DiscountSale)saleVariable;
```

produces a run-time error but will compile with no error. However, the following, which is also illegal, produces a compile-time error:

```
discountVariable = saleVariable;
```

Java catches these downcasting errors as soon as it can, which may be at compile time or at run time, depending on the case.

Although downcasting can be dangerous, it is sometimes necessary. For example, we inevitably use downcasting when we define an `equals` method for a class. For example, note the following line from the definition of `equals` in the class `Sale` (Display 8.1):

```
Sale otherSale = (Sale)otherObject;
```

This is a downcast from the type `Object` to the type `Sale`. Without this downcast, the instance variables `name` and `price` in the `return` statement, reproduced as follows, would be illegal, because the class `Object` has no such instance variables:

```
return (name.equals(otherSale.name)
 && (price == otherSale.price));
```

<sup>2</sup> We prefer to think of an object of the derived class as actually having the type of its base class as well as its own type. So this is not, strictly speaking, a type cast, but it does no harm to follow standard usage and call it a type cast in this case.



## PITFALL: Downcasting

It is the responsibility of the programmer to use downcasting only in situations where it makes sense. The compiler makes no checks to see if downcasting is reasonable. However, if you use downcasting in a situation in which it does not make sense, you will usually get a run-time error message. ■



## TIP: Checking to See Whether Downcasting Is Legitimate ★

You can use the `instanceof` operator to test whether or not downcasting is sensible. Downcasting to a specific type is reasonable if the object being cast is an instance of that type, which is exactly what the `instanceof` operator tests for.

`instanceof`

The `instanceof` operator checks whether an object is of the type given as its second argument. The syntax is

```
Object instanceof Class_Name
```

which returns `true` if *Object* is of type *Class\_Name*; otherwise it returns `false`. So, the following will return `true` if `someObject` is of type `DiscountSale`:

```
someObject instanceof DiscountSale
```

Note that because every object of every descendent class of `DiscountSale` is also of type `DiscountSale`, this expression will return `true` if `someObject` is an instance of any descendent class of `DiscountSale`.

So, if you want to type cast to `DiscountSale`, then you can make the casts safer as follows:

```
DiscountSale ds = new DiscountSale();
if (someObject instanceof DiscountSale)
{
 ds = (DiscountSale)someObject;
 System.out.println("ds was changed to " + someObject);
}
else
 System.out.println("ds was not changed.");
```

`someObject` might be, for example, a variable of type `Sale` or of type `Object`. ■

### Self-Test Exercises

5. Consider the following code, which is identical to the code discussed earlier in the opening of the subsection, “Downcasting and Upcasting,” except that we added the type cast shown in color:

```
Sale saleVariable;
DiscountSale discountVariable =
 new DiscountSale("paint", 15, 10);
saleVariable = (Sale)discountVariable;
System.out.println(saleVariable.toString());
```

We saw that without the type cast, the definition of the `toString` method used is the one given in the definition of the class `DiscountSale`. With this added type cast, will the definition of the `toString` method used still be the one given in `DiscountSale` or will it be the one given in the definition of `Sale`?

6. Would it be legal to add the following method definition to the class `DiscountSale`?

What about adding it to the class `Sale`?

```
public static void showDiscount(Sale object)
{
 System.out.println("Discount = "
 + object.getDiscount());
}
```

7. ★ What output is produced by the following code?

```
Sale someObject = new DiscountSale("map", 5, 0);
DiscountSale ds = new DiscountSale();
if (someObject instanceof DiscountSale)
{
 ds = (DiscountSale)someObject;
 System.out.println("ds was changed to " + someObject);
}
else
 System.out.println("ds was not changed.");
```

**Self-Test Exercises** (continued)

8. ★ What output is produced by the following code?

```
Sale someObject = new Sale("map", 5);
DiscountSale ds = new DiscountSale();
if (someObject instanceof DiscountSale)
{
 ds = (DiscountSale)someObject;
 System.out.println("ds was changed to " + someObject);
}
else
 System.out.println("ds was not changed.");
```

9. ★ Suppose we removed the qualifier `static` from the method `announcement()` in both `Sale` (Display 8.1) and `DiscountSale` (Display 8.2). What would be the output produced by the following code (which is similar to the end of Display 8.4)?

```
Sale s = new Sale();
DiscountSale discount = new DiscountSale();
s.announcement();
discount.announcement();
System.out.println("No surprises so far, but wait.");

Sale discount2 = discount;
System.out.println(
 "discount2 is a DiscountSale object in a Sale variable.");
System.out.println(
 "Which definition of announcement() will it use?");
discount2.announcement();
System.out.println(
 "Did it use the Sale version of announcement()?");
```

## A First Look at the `clone` Method

`clone`

Every object inherits a method named `clone` from the class `Object`. The method `clone` has no parameters and is supposed to return a copy of the calling object. However, the inherited version of `clone` was not designed to be used as is. Instead, you are expected to override the definition of `clone` with a version appropriate for the class you are defining. The officially sanctioned way to define the method `clone` turns out to be a bit complicated and requires material we do not cover until Chapter 13, so we will describe how to do so in that chapter. In this section, we will describe a simple way to define `clone` that will work in most situations and will allow us to discuss how polymorphism interacts with the `clone` method. If you are in a hurry to see the officially sanctioned way to define `clone`, you can read Chapter 13 immediately after this section (Section 8.1) with no loss of continuity in your reading.

The method `clone` has no parameters and should return a copy of the calling object. The returned object should have identical data to that of the calling object, but it normally should be a different object (an identical twin or “a clone”). You usually want the `clone` method to return the same kind of copy as what we have been defining for copy constructors, which is what is known as a *deep copy*. (You may want to review the subsection entitled “Copy Constructors” in Chapter 5.)

A `clone` method serves very much the same purpose as a copy constructor, but, as you will see in the Pitfall titled “Limitations of Copy Constructors,” there are situations where a `clone` method works as you want, whereas a copy constructor does not perform as desired.

As with other methods inherited from the class `Object`, the method `clone` needs to be redefined (overridden) before it performs properly. The heading for the method `clone` in the class `Object` is as follows:

```
protected Object clone()
```

If a class has a copy constructor, you can define the `clone` method for that class by using the copy constructor to create the copy returned by the `clone` method. For example, consider the class `Sale` defined in Display 8.1. The following definition of the `clone` method can be added to the definition of `Sale` given in Display 8.1:

```
public Sale clone()
{
 return new Sale(this);
}
```

Using a copy constructor is not the officially sanctioned way to define a `clone` method, and in fact, the Java documentation says you should not define it this way. However, it does work correctly, and some authorities say it is acceptable. In Chapter 13, we will discuss the official way of defining the method `clone` when we introduce the `Cloneable` interface.

Note that, as we defined the method `clone` for the class `Sale`, the method `clone` has `Sale` as its return type and is given `public` rather than `protected` access. Despite these

changes in the method heading, this definition overrides the method `clone` inherited from the class `Object`. As we noted in Chapter 7, a change to a more permissive access, such as from `protected` to `public`, is always allowed when overriding a method definition. Changing the return type from `Object` to `Sale` is allowed because `Sale` (and every other class, for that matter) is a descendent class of the class `Object`. This is an example of a covariant return type, as discussed in the subsection of Chapter 7 entitled “Changing the Return Type of an Overridden Method.”

The `clone` method for the `DiscountSale` class can be defined similarly:

```
public DiscountSale clone()
{
 return new DiscountSale(this);
}
```

extra code  
on website

The definitions of the classes `Sale` and `DiscountSale` on the website that accompanies this book each include the method `clone` defined as we just described.



### PITFALL: Sometimes the `clone` Method Return Type Is `Object`

Prior to version 5.0, Java did not allow covariant return types and so did not allow any changes whatsoever in the return type of an overridden method. In those earlier versions of Java, the `clone` method for all classes had `Object` as its return type. This is because the `clone` method for a class overrides the `clone` method of the class `Object`, and the `clone` method of the class `Object` has a return type of `Object`. If you encounter a `clone` method for a class that was designed and coded before version 5.0 of Java, the `clone` method will have a return type of `Object`. When using such older `clone` methods, you will need to use a type cast on the value returned by `clone`.

For example, suppose the class `OldClass` was defined before Java 5.0. If `original` is an object of the class `OldClass`, then the following will produce a compiler error message:

```
OldClass copy = original.clone();
```

The problem is that `original.clone()` returns a value of type `Object`, while the variable `copy` is of type `OldClass`. To correct the situation, you must add a type cast as follows:

```
OldClass copy = (OldClass)original.clone();
```

(continued)





### PITFALL: (continued)

You may encounter this problem even with classes defined after Java version 5.0. In Java version 5.0 and later, it is perfectly legal to use `Object` as a return type for a `clone` method (even if that is not the preferred return type). When in doubt, it causes no harm to include the type cast. For example, the following is legal for the `clone` method of the class `Sale` defined in the previous section:

```
Sale copySale = originalSale.clone();
```

However, adding the following type cast produces no problems:

```
Sale copySale = (Sale)originalSale.clone();
```

When in doubt about the `clone` method of a class, include the type cast. ■



### PITFALL: Limitations of Copy Constructors ★

Copy constructors work well in most simple cases. However, there are situations where they do not—indeed, cannot—do their job. That is why Java favors using the method `clone` in place of using a copy constructor. Here is a simple example of where the copy constructor does not do its job, but the `clone` method does.

For this discussion, assume that the classes `Sale` and `DiscountSale` each have a `clone` method added. The definitions of these `clone` methods are given in the previous subsection.

Suppose you have a method with the following heading (the methods `Sale` and `DiscountSale` were defined in Displays 8.1 and 8.2):

```
/**
 * Supposedly returns a safe copy of a. That is, if b is the array
 * returned, then b[i] is supposedly an independent copy of a[i].
 */
public static Sale[] badCopy(Sale[] a)
{
 Sale[] b = new Sale[a.length];
 for (int i = 0; i < a.length; i++)
 b[i] = new Sale(a[i]); //Problem here!
 return b;
}
```

Now if your array `a` contains objects from derived classes of `Sale`, such as objects of type `DiscountSale`, then `badCopy(a)` will not return a true copy of `a`. Every element of the array `badCopy(a)` will be a plain old `Sale`, because the `Sale` copy constructor produces only plain old `Sale` objects; no element in `badCopy(a)` will be an instance of the class `DiscountSale`.



## PITFALL: (continued)

If we instead use the method `clone`, things work out as they should; the following is the correct way to define our copy method:

```
public static Sale[] goodCopy(Sale[] a)
{
 Sale[] b = new Sale[a.length];
 for (int i = 0; i < a.length; i++)
 b[i] = a[i].clone();
 return b;
}
```

Because of late binding (polymorphism), `a[i].clone()` always means the correct version of the `clone` method. If `a[i]` is an object created with a constructor of the class `DiscountSale`, `a[i].clone()` will invoke the definition of `clone()` given in the definition of the class `DiscountSale`. If `a[i]` is an object created with a constructor of the class `Sale`, `a[i].clone()` will invoke the definition of `clone()` given in the definition of the class `Sale`. This is illustrated in Display 8.5.

This may seem like a sleight of hand. After all, in the classes `Sale` and `DiscountSale`, we defined the method `clone` in terms of copy constructors. We reproduce the definitions of `clone` from the class `Sale` and `DiscountSale` as follows:

```
//For Sale class
public Sale clone()
{
 return new Sale(this);
}
//For DiscountSale class
public DiscountSale clone()
{
 return new DiscountSale(this);
}
```

So, why is using the method `clone` any different than using a copy constructor? The difference is simply that the method creating the copy of an element `a[i]` has the same name `clone` in all the classes, and polymorphism works with method names. The copy constructors named `Sale` and `DiscountSale` have different names, and polymorphism has nothing to do with methods of different names.

We will have more to say about the `clone` method in Chapter 13 when we discuss the `Cloneable` interface. ■

## Display 8.5 Copy Constructor Versus clone Method (part 1 of 2)

```

1 /**
2 Demonstrates where the clone method works,
3 but copy constructors do not.
4 */
5 public class CopyingDemo
6 {
7 public static void main(String[] args)
8 {
9 Sale[] a = new Sale[2];
10 a[0] = new Sale("atomic coffee mug", 130.00);
11 a[1] = new DiscountSale("invisible paint", 5.00, 10);
12 int i;

13 Sale[] b = badCopy(a);

14 System.out.println("With copy constructors:");
15 for (i = 0; i < a.length; i++)
16 {
17 System.out.println("a[" + i + "] = " + a[i]);
18 System.out.println("b[" + i + "] = " + b[i]);
19 System.out.println();
20 }
21 System.out.println();

22 b = goodCopy(a);

23 System.out.println("With clone method:");
24 for (i = 0; i < a.length; i++)
25 {
26 System.out.println("a[" + i + "] = " + a[i]);
27 System.out.println("b[" + i + "] = " + b[i]);
28 System.out.println();
29 }

30 }

31 /**
32 Supposedly returns a safe copy of a. That is, if b is the
33 array returned, then b[i] is supposedly an independent copy of a[i].
34 */

35 public static Sale[] badCopy(Sale[] a)
36 {
37 Sale[] b = new Sale[a.length];
38 for (int i = 0; i < a.length; i++)
39 b[i] = new Sale(a[i]); // Problem here!
40 return b;
41 }
42 }

```

*This program assumes that a clone method has been added to the class Sale and to the class DiscountSale.*

## Display 8.5 Copy Constructor Versus clone Method (part 2 of 2)

```

43 /**
44 * Returns a safe copy of a. That is, if b is the
45 * array returned, then b[i] is an independent copy of a[i].
46 */
47 public static Sale[] goodCopy(Sale[] a)
48 {
49 Sale[] b = new Sale[a.length];
50 for (int i = 0; i < a.length; i++)
51 b[i] = a[i].clone();
52 return b;
53 }
54 }

```

## Sample Dialogue

With copy constructors:

a[0] = atomic coffee mug Price and total cost = \$130.0

b[0] = atomic coffee mug Price and total cost = \$130.0

a[1] = invisible paint Price = \$5.0 Discount 10.0%

Total cost = \$4.5

b[1] = invisible paint Price and total cost = \$5.0

*The copy constructor  
lost the discount.*

With clone method:

a[0] = atomic coffee mug Price and total cost = \$130.0

b[0] = atomic coffee mug Price and total cost = \$130.0

a[1] = invisible paint Price = \$5.0 Discount 10.0%

Total cost = \$4.5

b[1] = invisible paint Price = \$5.0 Discount 10.0%

Total cost = \$4.5

*The clone method did  
not lose the discount.*

## 8.2 Abstract Classes

*It is for us, the living, rather to be dedicated here to the unfinished work which they who fought here have thus far so nobly advanced.*

ABRAHAM LINCOLN, *Gettysburg Address*, 1864.

An abstract class is a class that has some methods without complete definitions. You cannot create an object using an abstract class constructor, but you can use an abstract class as a base class to define a derived class.

## Abstract Classes

In Chapter 7, we defined a class named `Employee` and two of its derived classes, `HourlyEmployee` and `SalariedEmployee`. Display 8.6 repeats the details of these class definitions, which we will use in this discussion.

Suppose that when we define the class `Employee`, we know that we are going to frequently compare employees to see if they have the same pay. We might add the following method to the class `Employee`:

```
public boolean samePay(Employee other)
{
 return (this.getPay() == other.getPay());
}
```

There is, however, one problem with adding the method `samePay` to the class `Employee`: The method `samePay` includes an invocation of the method `getPay`, and the class `Employee` has no `getPay` method. Moreover, there is no reasonable definition we might give for a `getPay` method so that we could add it to the class `Employee`. The only instance variables in the class `Employee` give an employee's name and hire date, but give no information about pay. To see how we should proceed, let's compare objects of the class `Employee` to employees in the real world.

Every real-world employee does have some pay because every real-world employee is either an hourly employee or a salaried employee, and the two derived classes `HourlyEmployee` and `SalariedEmployee` each have a `getPay` method. The problem is that we do not know how to define the `getPay` method until we know if the employee is an hourly or salaried. We would like to postpone the definition of the `getPay` method and give it only in each derived class of the `Employee` class. We would like to simply add a note to the `Employee` class that says: "There will be a method `getPay` for each `Employee` but we do not yet know how it is defined." Java lets us do exactly what we want. The official Java equivalent of our promissory note about the method `getPay` is to make `getPay` an **abstract method**. An abstract method has a heading just like an ordinary method, but no method body. The syntax rules of Java require the modifier `abstract` and require a semicolon in place of the missing method body, as illustrated by the following:

```
public abstract double getPay();
```

**abstract  
method**

Display 8.6 `Employee` Class and Its Derived Classes (part 1 of 2)

*These show the details needed for the current discussion. You should not need to review the entire class definitions from Chapter 7. Complete definitions of all these classes are given in the subdirectory for this chapter on the website that comes with this text.*

```

1 public class Employee
2 {
3 private String name;
4 private Date hireDate;
5 public Employee()
6 public boolean equals(Object otherObject)
7 }
```

*The class `Date` is defined in Display 4.13, but the details are not important to the current discussion. There is no need to review the definition of the class `Date`.*

*The body of the constructor as given in Display 7.2 should initialize the instance variables, but the details are not needed for this discussion.*

*The body of the method `equals` is the same as in Display 7.10, but the details of the definition are not important to the current discussion.*

*All other constructor and other method definitions are exactly the same as in Display 7.2.*

*The class `Employee` has no method named `getPay`.*

```

1 public class SalariedEmployee extends Employee
2 {
3 private double salary; //annual
4
5 /**
6 * Returns the pay for the month.
7 */
8 public double getPay()
9 {
10 return salary / 12;
11 }
12 public boolean equals(Object otherObject)
```

*The rest of the definition of `equals` is the same as in the answer to Self-Test Exercise 20 of Chapter 7, but the details of the definition are not important to the current discussion.*

*All constructor and other method definitions are exactly the same as in Display 7.5.*

```

12 }
```

(continued)

Display 8.6 `Employee` Class and Its Derived Classes (part 2 of 2)

---

```

1 public class HourlyEmployee extends Employee
2 {
3 private double wageRate;
4 private double hours; //for the month

5 /**
6 * Returns the pay for the month.
7 */
8 public double getPay()
9 {
10 return wageRate * hours;
11 }

12 public boolean equals(Object otherObject)
13 {
14 if (otherObject == null)
15 return false;
16 else if (getClass() != otherObject.getClass())
17 return false;
18 else
19 {
20 HourlyEmployee otherHourlyEmployee =
21 (HourlyEmployee) otherObject;
22 return (super.equals(otherHourlyEmployee)
23 && (wageRate == otherHourlyEmployee.wageRate)
24 && (hours == otherHourlyEmployee.hours));
25 }
26 }

All constructor and other method definitions are exactly the same as in Display 7.3.

27 }

```

---

If we add this abstract method `getPay` to the class `Employee`, then we are free to add the method `samePay` to the class `Employee`.

**abstract  
cannot be  
private**

An abstract method can be thought of as the interface part of a method with the implementation details omitted. Because a private method is normally only a helping method and so not part of the interface for a programmer using the class, it follows that it does not make sense to have a private abstract method. Java enforces this reasoning. In Java, an abstract method cannot be private. Normally an abstract method is public but protected, and package (default) access is allowed.

An abstract method serves a purpose, even though it is not given a full definition. It serves as a placeholder for a method that must be defined in all (nonabstract) derived classes. Note that in Display 8.7, the method `samePay` includes invocations of the method `getPay`. If the abstract method `getPay` were omitted, this invocation of `getPay` would be illegal.

## Abstract Method

An **abstract method** serves as a placeholder for a method that will be fully defined in a descendent class. An abstract method has a complete method heading with the addition of the modifier `abstract`. It has no method body but does end with a semicolon in place of a method body. An abstract method cannot be private.

### EXAMPLES

```
public abstract double getPay();
public abstract void doSomething(int count);
```

### abstract class

A class that has at least one abstract method is called an **abstract class** and, in Java, must have the modifier `abstract` added to the class heading. The redefined, now abstract, class `Employee` is shown in Display 8.7.

An abstract class can have any number of abstract methods. In addition, it can have, and typically does have, other regular (fully defined) methods. If a derived class of an abstract class does not give full definitions to all the abstract methods, or if the derived class adds an abstract method, then the derived class is also an abstract class and must include the modifier `abstract` in its heading.

### concrete class

In contrast with the term *abstract class*, a class with no abstract methods is called a **concrete class**.

Display 8.7 `Employee` Class as an Abstract Class (part 1 of 2)

```
1 /**
2 Class Invariant: All objects have a name string and hire date.
3 A name string of "No name" indicates no real name specified yet.
4 A hire date of January 1, 1000 indicates no real hire date specified
5 yet.
6 */
7 public abstract class Employee
8 {
9 private String name;
10 private Date hireDate;
11
12 public abstract double getPay();
13
14 public Employee()
15 {
16 name = "No name";
17 hireDate = new Date("January", 1, 1000);
18 //Just a placeholder.
19 }
20
21 public boolean samePay(Employee other)
22 {
23 if (other == null)
```

*The class `Date` is defined in Display 4.13, but the details are not relevant to the current discussion of abstract methods and classes. There is no need to review the definition of the class `Date`.*

(continued)



Display 8.7 `Employee` Class as an Abstract Class (part 2 of 2)

---

```

19 {
20 System.out.println("Error: null Employee object.");
21 System.exit(0);
22 }
23 //else
24 return (this.getPay() == other.getPay());
25 }

```

*All other constructor and other method definitions are exactly the same as in Display 7.2. In particular, they are not abstract methods.*

```

26 }

```

---

### Abstract Class

An **abstract class** is a class with one or more abstract methods. An abstract class must have the modifier `abstract` included in the class heading, as illustrated by the example.

#### EXAMPLE

```

public abstract class Employee
{
 private String name;
 private Date hireDate;

 public abstract double getPay();
 ...
}

```



### PITFALL: You Cannot Create Instances of an Abstract Class

You cannot use an abstract class constructor to create an object of the abstract class. You can only create objects of the derived classes of the abstract class. For example, with the class `Employee` defined as in Display 8.7, the following would be illegal:

```

Employee joe = new Employee(); //Illegal because
 //Employee is an abstract class.

```

But, this is no problem. The object `joe` could not correspond to any real-world employee. Any real-world employee is either hourly or a salaried. In the real world, one cannot be just an employee. One must be either an hourly employee or a salaried employee. Still, it is useful to discuss employees in general. In particular, we can compare employees to see if they have the same pay, even though the way of calculating the pay might be different for the two employees. ■



### TIP: An Abstract Class Is a Type

You cannot create an object of an abstract class (unless it is actually an object of some concrete descendent class). Nonetheless, it makes perfectly good sense to have a parameter of an abstract class type such as `Employee` (as defined in Display 8.7). Then, an object of any of the descendent classes of `Employee` can be plugged in for the parameter. It even makes sense to have a variable of an abstract class type such as `Employee`, although it can only name objects of its concrete descendent classes. ■

### An Abstract Class Is a Type

You can have a parameter of an abstract class type such as the abstract class `Employee` defined in Display 8.7. Then, an object of any of the concrete descendent classes of `Employee` can be plugged in for the parameter. You can also have variables of an abstract class type such as `Employee`, although it can only name objects of its concrete descendent classes.

### Self-Test Exercises

10. Can a method definition include an invocation of an abstract method?
11. Can you have a variable whose type is an abstract class?
12. Can you have a parameter whose type is an abstract class?
13. Is it legal to have an abstract class in which all methods are abstract?
14. The abstract class `Employee` (Display 8.7) uses the method definitions from Display 7.2. After we did Display 7.2, we later gave the following improved version of `equals`:

```
public boolean equals(Object otherObject)
{
 if (otherObject == null)
 return false;
 else if (getClass() != otherObject.getClass())
 return false;
 else
 {
 Employee otherEmployee =
 (Employee) otherObject;
 return (name.equals(otherEmployee.name)
 && hireDate.equals(otherEmployee.hireDate));
 }
}
```

Would it be legal to replace the version of `equals` for the abstract class `Employee` with this improved version?

(continued)

**Self-Test Exercises** (continued)

15. The abstract class `Employee` given in Display 8.7 has a constructor (in fact, it has more than one, although only one is shown in Display 8.7). But using a constructor to create an instance of an abstract class, as in the following, is illegal:

```
Employee joe = new Employee(); //Illegal
```

So why bother to have any constructors in an abstract class? Aren't they useless?

**Chapter Summary**

- *Late binding* (also called *dynamic binding*) means that the decision of which version of a method is appropriate is decided at run time. Java uses late binding.
- *Polymorphism* means using the process of late binding to allow different objects to use different method actions for the same method name. *Polymorphism* is essentially another word for *late binding*.
- You can assign an object of a derived class to a variable of its base class (or any ancestor class), but you cannot do the reverse.
- If you add the modifier `final` to the definition of a method, it indicates that the method may not be redefined in a derived class. If you add the modifier `final` to the definition of a class, it indicates that the class may not be used as a base class to derive other classes.
- An *abstract method* serves as a placeholder for a method that will be fully defined in a descendent class.
- An *abstract class* is a class with one or more abstract methods.
- An abstract class is designed to be used as a base class to derive other classes. You cannot create an object of an abstract class type (unless it is an object of some concrete descendent class).
- An abstract class is a type. You can have variables whose type is an abstract class and you can have parameters whose type is an abstract type.

**Answers to Self-Test Exercises**

1. In essence, there is no difference between the two terms. There is only a slight difference in their usage. *Late binding* refers to the mechanism used to decide which method definition to use when a method is invoked, and *polymorphism* refers to the fact that the same method name can have different meanings because of late binding.
2. There would be problems well before you wrote the program in Display 8.3. Since `final` means you cannot change the definition of the method `bill` in a derived

class, the definition of the method `DiscountSale` would not compile. If you omit the definition of the method `bill` from the class `DiscountSale`, the output would change to

```
floor mat Price and total cost = $10.0
floor mat Price = $11.0 Discount = 10.0%
 Total cost = $11.0
Discounted item is not cheaper.
cup holder Price and total cost = $9.9
cup holder Price = $11.0 Discount = 10.0%
 Total cost = $11.0
Items are not equal.
```

Note that all objects use the definition of `bill` given in the definition of `Sale`.

3. It would not be legal to add it to any class definition because the class `Sale` has no method named `getDiscount` and so the invocation

```
theSale.getDiscount()
```

is not allowed. If the type of the parameter were changed from `Sale` to `DiscountSale`, it would then be legal.

4. `public boolean equals(Object otherObject)`

```
{
 if (otherObject == null)
 return false;
 else if (getClass() != otherObject.getClass())
 return false;
 else
 {
 DiscountSale otherDiscountSale =
 (DiscountSale)otherObject;
 return (super.equals(otherDiscountSale)
 && discount == otherDiscountSale.discount);
 }
}
```

5. The definition of `toString` always depends on the object and not on any type cast. So, the definition used is the same as without the added type cast; that is, the definition of `toString` that is used is the one given in `DiscountSale`.
6. It would not be legal to add it to any class definition because the parameter is of type `Sale`, and `Sale` has no method named `getDiscount`. If the parameter type were changed to `DiscountSale`, it would then be legal to add it to any class definition.
7. `ds` was changed to `map Price $ 5.0 discount 0.0%`  
`Total cost $5.0`
8. `ds` was not changed.

9. The output would be the following (the main change from Display 8.4 is shown in blue):

```
This is the Sale class.
This is the DiscountSale class.
No surprises so far, but wait.
discount2 is a DiscountSale object in a Sale variable.
Which definition of announcement() will it use?
This is the DiscountSale class.
Did it use the Sale version of announcement()?
```

10. Yes. See Display 8.7.
11. Yes, you can have a variable whose type is an abstract class.
12. Yes, you can have a parameter whose type is an abstract class.
13. Yes, it is legal to have an abstract class in which all methods are abstract.
14. Yes, it would be legal to replace the version of `equals` for the abstract class `Employee` with this improved version. In fact, the version of `Employee` on the accompanying website does use the improved version of `equals`.
15. No, you can still use constructors to hold code that might be useful in derived classes. The constructors in the derived classes can—in fact, must—include invocations of constructors in the base (abstract) class. (Recall the use of `super` as a name for the base class constructor.)

## Programming Projects



VideoNote  
Solution to  
Programming  
Project 8.1

1. In Programming Project 7.3 from Chapter 7, the `Alien` class was rewritten to use inheritance. The rewritten `Alien` class should be made abstract because there will never be a need to create an instance of it, only its derived classes. Change this to an abstract class and also make the `getDamage` method an abstract method. Test the class from your `main` method to ensure that it still operates as expected.
2. Create a class named `Employee` that can be used to calculate the salaries of different employees. The `Employee` class should keep a track of the employee ID, name, department, salary, and designation with appropriate accessor and mutator methods. Also create an `equals()` method that overrides `Object`'s `equals()` method, where employees can check if their designation is identical. Next, create two additional classes named `Manager` and `Clerk` that are derived from `Employee`. Create an overridden method named `addBonus` that returns the salary of the employee after adding up the bonus. There is a default bonus of \$200/month. Managers have a bonus of \$300/month and clerks have a bonus of \$100/month. Finally create a display method to print the details of the employee. You may assume the initial salary of an employee and other necessary values. Test your classes from a `main` method.
3. Extend the previous problem to calculate the salary deductions based on the number of days an employee is on leave. Consider 20 working days per month. Add a method that calculates the deductions of each employee based on their leave

record. In your main method, create an array of type `Deduction` filled with sample data of all types of `Employees`. Finally calculate the total deduction that iterates through the array and returns the total amount of deductions of all the employees in a month.

4. The goal for this programming project is to create a simple 2D predator–prey simulation. In this simulation, the prey is ants, and the predators are doodlebugs. These critters live in a world composed of a  $20 \times 20$  grid of cells. Only one critter may occupy a cell at a time. The grid is enclosed, so a critter is not allowed to move off the edges of the grid. Time is simulated in time steps. Each critter performs some action every time step.

The ants behave according to the following model:

- Move. Every time step, randomly try to move up, down, left, or right. If the cell in the selected direction is occupied or would move the ant off the grid, then the ant stays in the current cell.
- Breed. If an ant survives for three time steps, then at the end of the third time step (i.e., after moving), the ant will breed. This is simulated by creating a new ant in an adjacent (up, down, left, or right) cell that is empty. If there is no empty cell available, no breeding occurs. Once an offspring is produced, the ant cannot produce an offspring until three more time steps have elapsed.

The doodlebugs behave according to the following model:

- Move. Every time step, if there is an adjacent cell (up, down, left, or right) occupied by an ant, then the doodlebug will move to that cell and eat the ant. Otherwise, the doodlebug moves according to the same rules as the ant. Note that a doodlebug cannot eat other doodlebugs.
- Breed. If a doodlebug survives for eight time steps, then at the end of the time step, it will spawn off a new doodlebug in the same manner as the ant.
- Starve. If a doodlebug has not eaten an ant within the last three time steps, then at the end of the third time step, it will starve and die. The doodlebug should then be removed from the grid of cells.

During one turn, all the doodlebugs should move before the ants.

Write a program to implement this simulation and draw the world using ASCII characters of “o” for an ant and “X” for a doodlebug. Create a class named `Organism` that encapsulates basic data common to both ants and doodlebugs.

This class should have an overridden method named `move` that is defined in the derived classes of `Ant` and `Doodlebug`. You may need additional data structures to keep track of which critters have moved.

Initialize the world with 5 doodlebugs and 100 ants. After each time step, prompt the user to press Enter to move to the next time step. You should see a cyclical pattern between the population of predators and prey, although random perturbations may lead to the elimination of one or both species.

5. Consider a graphics system that has classes for various figures—say, rectangles, boxes, triangles, circles, and so on. For example, a rectangle might have data

members' height, width, and center point, while a box and circle might have only a center point and an edge length or radius, respectively. In a well-designed system, these would be derived from a common class, `Figure`. You are to implement such a system.

The class `Figure` is the base class. You should add only `Rectangle` and `Triangle` classes derived from `Figure`. Each class has stubs for methods `erase` and `draw`. Each of these methods outputs a message telling the name of the class and what method has been called. Because these are just stubs, they do nothing more than output this message. The method `center` calls the `erase` and `draw` methods to erase and redraw the figure at the center. Because you have only stubs for `erase` and `draw`, `center` will not do any “centering” but will call the methods `erase` and `draw`, which will allow you to see which versions of `draw` and `center` it calls. Also, add an output message in the method `center` that announces that `center` is being called. The methods should take no arguments. Also, define a demonstration program for your classes.

For a real example, you would have to replace the definition of each of these methods with code to do the actual drawing. You will be asked to do this in Programming Project 8.6.

6. Flesh out Programming Project 8.5. Give new definitions for the various constructors and methods `center`, `draw`, and `erase` of the class `Figure`; `draw` and `erase` of the class `Triangle`; and `draw` and `erase` of the class `Rectangle`. Use character graphics; that is, the various `draw` methods will place regular keyboard characters on the screen in the desired shape. Use the character '\*' for all the character graphics. That way, the `draw` methods actually draw figures on the screen by placing the character '\*' at suitable locations on the screen. For the `erase` methods, you can simply clear the screen (by outputting blank lines or by doing something more sophisticated). There are a lot of details in this project, and you will have to decide on some of them on your own.
7. Define a class named `MultiItemSale` that represents a sale of multiple items of type `Sale` given in Display 8.1 (or of the types of any of its descendent classes). The class `MultiItemSale` will have an instance variable whose type is `Sale[]`, which will be used as a partially filled array. There will also be another instance variable of type `int` that keeps track of how much of this array is currently used. The exact details on methods and other instance variables, if any, are up to you. Use this class in a program that obtains information for items of type `Sale` and of type `DiscountSale` (Display 8.2) and that computes the total bill for the list of items sold.
8. Programming Project 7.8 required rewriting the solution to Programming Project 4.10 with inheritance. Redo or do Programming Project 7.8, but instead define the `Pet` class as an abstract class. The `acepromazine()` and `carprofen()` methods should be defined as abstract methods.

In your main method, define an array of type `Pet` and add two instances of cats and two instances of dogs to the array. Iterate through the array and output how much `carprofen` and `acepromazine` each pet would require.



VideoNote  
Solution to  
Programming  
Project 8.9

9. The following is a short snippet of code that simulates rolling a 6-sided dice 100 times. There is an equal chance of rolling any digit from 1 to 6.

```
public static void printDiceRolls(Random randGenerator)
{
 for (int i = 0; i < 100; i++)
 {
 System.out.println(randGenerator.nextInt(6) + 1);
 }
}

public static void main(String[] args)
{
 Random randGenerator = new Random();
 printDiceRolls(randGenerator);
}
```

Create your own class, `LoadedDice`, that is derived from `Random`. The constructor for `LoadedDice` needs to only invoke `Random`'s constructor. Override the public `int nextInt(int num)` method so that with a 50% chance, your new method always returns the largest number possible (i.e., `num - 1`), and with a 50% chance, it returns what `Random`'s `nextInt` method would return.

Test your class by replacing the `main` method with the following:

```
LoadedDice myDice = new LoadedDice();
printDiceRolls(myDice);
```

You do not need to change the `printDiceRolls` method even though it takes a parameter of type `Random`. Polymorphism tells Java to invoke `LoadedDice`'s `nextInt()` method instead of `Random`'s `nextInt()` method.



This page intentionally left blank



# Exception Handling 9

## 9.1 EXCEPTION HANDLING BASICS 557

- `try-catch` Mechanism 557
- Exception Handling with the `Scanner` Class 559
- Throwing Exceptions 562
- Example: A Toy Example of Exception Handling 564
- Exception Classes 569
- Exception Classes from Standard Packages 570
- Defining Exception Classes 572
- Multiple `catch` Blocks 583

## 9.2 THROWING EXCEPTIONS IN METHODS 588

- Throwing an Exception in a Method 588
- Declaring Exceptions in a `throws` Clause 590

- Exceptions to the Catch or Declare Rule 593
- `throws` Clause in Derived Classes 594
- When to Use Exceptions 595
- Example: Retrieving a High Score 596
- Event-Driven Programming ★ 599

## 9.3 MORE PROGRAMMING TECHNIQUES FOR EXCEPTION HANDLING 601

- The `finally` Block ★ 601
- Rethrowing an Exception ★ 603
- The `AssertionError` Class ★ 603
- `ArrayIndexOutOfBoundsException` 604

# 9 Exception Handling

*It's the exception that proves the rule.*

ANONYMOUS, 1700s.

## Introduction

One way to divide the task of designing and coding a method is to code two main cases separately: the case where nothing unusual happens and the case where exceptional things happen. Once you have the program working for the case where things always go smoothly, you can then code the second case where notable things can happen. In Java, there is a way to mirror this approach in your code. Write your code more or less as if nothing very unusual happens. After that, use the Java exception handling facilities to add code for those unusual cases.

The most important use of exceptions is to deal with methods that have some special case that is handled differently depending on how the method is used. For example, if there is a division by zero in the method, then it may turn out that for some invocations of the method, the program should end, but for other invocations of the method, something else should happen. Such a method can be defined to throw an exception if the special case occurs; that exception will permit the special case to be handled outside of the method. This allows the special case to be handled differently for different invocations of the method.

In Java, exception handling proceeds as follows: Either some library software or your code provides a mechanism that signals when something unusual happens. This is called **throwing an exception**. At another place in your program, you place the code that deals with the exceptional case. This is called **handling the exception**. This method of programming makes for cleaner code. Of course, we still need to explain the details of how you do this in Java.

throw  
exception

handle  
exception

## Prerequisites

Almost this entire chapter uses only material from Chapters 1 through 5 and Chapter 7. The only exception is the subsection “`ArrayIndexOutOfBoundsException`,” which also uses material from Chapter 6. However, that subsection may be omitted if you have not yet covered Chapter 6. Chapter 8 is not needed for this chapter.

## 9.1 Exception Handling Basics

*Well the program works for most cases. I didn't know it had to work for that case.*

COMPUTER SCIENCE STUDENT, *appealing a grade*

Exception handling is meant to be used sparingly and in some situations that are more involved than what is reasonable to include in an introductory example. So, in some cases, we will teach you the exception handling details of Java by means of simple examples that would not normally use exception handling. This makes a lot of sense for learning about the exception handling details of Java, but do not forget that these examples are toy examples and, in practice, you would not use exception handling for anything this simple.

### try-catch Mechanism

The basic way of handling exceptions in Java consists of the try-throw-catch trio. At this point, we will start with only try and catch. The general setup consists of a try block followed by one or more catch blocks. First let's describe what a try block is. A **try block** has the following syntax:

try block

```
try
{
 Some_Code_That_May_Throw_An_Exception
}
```

This try block contains the code for the basic algorithm that tells what to do when everything goes smoothly. It is called a try block because it “tries” to execute the case where all goes well.

Now, an exception can be “thrown” as a way of indicating that something unusual happened. For example, if our code tries to divide by zero, then an `ArithmeticException` object is thrown. In most of this chapter, our own code will throw the exception, but initially we will have existing Java classes do the throwing.

As the name suggests, when something is “thrown,” something goes from one place to another place. In Java, what goes from one place to another is the flow of control as well as the exception object that is thrown. When an exception is thrown, the code in the surrounding try block stops executing and (normally) another portion of code, known as a **catch block**, begins execution. The catch block has a parameter, and the exception object thrown is plugged in for this catch block parameter. This executing of the catch block is called **catching the exception** or **handling the exception**. When an exception is thrown, it should ultimately be handled by (caught by) some catch block. The appropriate catch block immediately follows the try block; for example,

catch block

handling an exception

```
catch(Exception e)
{
 String message = e.getMessage();
 System.out.println(message);
 System.exit(0);
}
```

This `catch` block looks very much like a method definition that has a parameter of a type `Exception`. By using the type `Exception`, this `catch` block will catch any possible exception that is thrown. We will see at the end of this section that we can also restrict the `catch` block to specific exception classes. The `catch` block is not a method definition, but in some ways, it is like a method. It is a separate piece of code that is executed when your code throws an exception. The `catch` block in the previous example will print out a message about the exception that was thrown.

So, when an exception is thrown, it is similar to a method call, but instead of calling a method, it calls the `catch` block and says to execute the code in the `catch` block. A `catch` block is often referred to as an **exception handler**.

Let's focus on the identifier `e` in the following line from a `catch` block:

```
catch(Exception e)
```

That identifier `e` in the `catch` block heading is called the `catch` **block parameter**. Each `catch` block can have at most one `catch` block parameter. The `catch` block parameter does two things:

- The `catch` block parameter is preceded by an exception class name that specifies what type of thrown exception object the `catch` block can catch. If the class name is `Exception`, then the block can catch any exception.
- The `catch` block parameter gives you a name for the thrown object that is caught, so you can write code in the `catch` block that does things with the thrown object that is caught.

Although the identifier `e` is often used for the `catch` block parameter, this is not required. You may use any nonkeyword identifier for the `catch` block parameter just as you can for a method parameter.

### catch Block Parameter

The `catch` block parameter is an identifier in the heading of a `catch` block that serves as a placeholder for an exception that might be thrown. When a suitable exception is thrown in the preceding `try` block, that exception is plugged in for the `catch` block parameter. The identifier `e` is often used for `catch` block parameters, but this is not required. You can use any legal (nonkeyword) identifier for a `catch` block parameter.

#### SYNTAX

```
catch (Exception_Class_Name Catch_Block_Parameter)
{
 Code to be performed if an exception of the named exception class is thrown in
 the try block.
}
```

You may use any legal identifier for the `Catch_Block_Parameter`.

exception  
handler

catch block  
parameter

**EXAMPLE**

In the following, `e` is the catch block parameter.

```
catch(Exception e)
{
 System.out.println(e.getMessage());
 System.out.println("Aborting program.");
 System.exit(0);
}
```

Let's consider two possible cases of what can happen when a `try` block is executed: (1) no exception is thrown in the `try` block, and (2) an exception is thrown in the `try` block and caught in the `catch` block. (Later in the Tip, "What Happens If an Exception Is Never Caught?," we will describe a third case where the `catch` block does not catch the exception.)

- If no exception is thrown, the code in the `try` block is executed to the end of the `try` block, the `catch` block is skipped, and execution continues with the code placed after the `catch` block.
- If an exception is thrown in the `try` block, the rest of the code in the `try` block is skipped, and (in simple cases) control is transferred to a following `catch` block. The thrown object is plugged in for the `catch` block parameter, and the code in the `catch` block is executed. And then (provided the `catch` block code does not end the program or do something else to end the `catch` block code prematurely), the code that follows that `catch` block is executed.

## Exception Handling with the `Scanner` Class

As a concrete example, consider a program that reads an `int` value from the keyboard using the `nextInt` method of the `Scanner` class. You have probably noticed that the program will end with an error message if the user enters something other than a well-formed `int` value. That is true as far as it goes, but the full detail is that if the user enters something other than a well-formed `int` value, an exception of type `InputMismatchException` will be thrown. If the exception is not caught, your program ends with an error message. However, you can catch the exception, and in the `catch` block, give code for some alternative action, such as asking the user to reenter the input. You are not required to account for an `InputMismatchException` by catching it in a `catch` block or declaring it in a `throws` clause (this is because `InputMismatchException` is a descendent class of `RuntimeException`). However, you are allowed to catch an `InputMismatchException` in a `catch` block, which can sometimes be useful.

`InputMismatchException` is in the standard Java package `java.util`, so if your program mentions `InputMismatchException`, then it needs an import statement, such as the following:

```
import java.util.InputMismatchException;
```

Display 9.1 contains an example of how you might usefully catch an `InputMismatchException`. This program gets an input `int` value from the keyboard and then does nothing with it other than echo the input value. However, you can use code such as this to require robust input for any program that uses keyboard input. The Tip “Exception Controlled Loops” explains the general technique we used for the loop in Display 9.1.



### TIP: Exception Controlled Loops

Sometimes when an exception is thrown, such as an `InputMismatchException` for an ill-formed input, you want your code to simply repeat some code so that the user can get things right on a second or subsequent try. One way to set up your code to repeat a loop every time a particular exception is thrown is as follows:

```
boolean done = false;

while (!done)
{
 try
 {
 Code that may throw an exception in the class Exception_Class.
 done = true; //Will end the loop.
 <Possibly more code.>
 }
 catch (Exception_Class e)
 {
 <Some code.>
 }
}
```

Note that if an exception is thrown in the first piece of code in the `try` block, then the `try` block ends before the line that sets `done` to `true` is executed, so the loop body is repeated. If no exception is thrown, then `done` is set to `true` and the loop body is not repeated.

Display 9.1 contains an example of such a loop. Minor variations on this outline can accommodate a range of different situations for which you want to repeat code on throwing an exception. ■

## Display 9.1 An Exception Controlled Loop

```
1 import java.util.Scanner;
2 import java.util.InputMismatchException;

3 public class InputMismatchExceptionDemo
4 {
5 public static void main(String[] args)
6 {
7 Scanner keyboard = new Scanner(System.in);
8 int number = 0; //to keep compiler happy
9 boolean done = false;

10 while (!done)
11 {
12 try
13 {
14 System.out.println("Enter a whole number:");
15 number = keyboard.nextInt();
16 done = true;
17 }
18 catch(InputMismatchException e)
19 {
20 keyboard.nextLine();
21 System.out.println("Not a correctly written whole
22 number.");
23 System.out.println("Try again.");
24 }
25 }

26 System.out.println("You entered " + number);
27 }
```

*If nextInt throws an exception, the try block ends and the Boolean variable done is not set to true.*

## Sample Dialogue

```
Enter a whole number:
forty two
Not a correctly written whole number.
Try again.
Enter a whole number:
Fortytwo
Not a correctly written whole number.
Try again.
Enter a whole number:
42
You entered 42
```





**throw  
statement**

This try block contains the following three throw statements:

```
throw new Exception("Lesson is canceled. No students.");
throw new Exception("Lesson is canceled. No men.");
throw new Exception("Lesson is canceled. No women.");
```

**throwing an  
exception**

The value thrown is an argument to the throw operator and is always an object of some exception class. The execution of a throw statement is called *throwing an exception*.

**throw Statement****SYNTAX**

```
throw new Exception_Class_Name (Possibly_Some_Arguments);
```

When the throw statement is executed, the execution of the surrounding try block is stopped and (normally) control is transferred to a catch block. The code in the catch block is executed next. See the box entitled "try-throw-catch" later in this chapter for more details.

**EXAMPLE**

```
throw new Exception("Division by zero.");
```

**The getMessage Method**

Every exception has a String instance variable that contains some message, which typically identifies the reason for the exception. For example, if the exception is thrown as follows:

```
throw new Exception(String_Argument);
```

then the string given as an argument to the constructor Exception is used as the value of this String instance variable. If the object is called e, then the method call e.getMessage() returns this string.

**EXAMPLE**

Suppose the following throw statement is executed in a try block:

```
throw new Exception("Input must be positive.");
```

And suppose the following is a catch block immediately following the try block:

```
catch (Exception e)
{
 System.out.println(e.getMessage());
 System.out.println("Program aborted.");
 System.exit(0);
}
```

In this case, the method call e.getMessage() returns the string "Input must be positive."

### EXAMPLE: A Toy Example of Exception Handling

Display 9.2 contains a simple program that might, by some stretch of the imagination, be used at a dance studio. This program does not use exception handling, and you would not normally use exception handling for anything this simple. The setting for use of the program is a dance lesson. The program simply checks to see if there are more men than women or more women than men and then announces how many partners each man or woman will have. The exceptional case is when there are no men or no women or both. In that exceptional case, the dance lesson is canceled.

In Display 9.3, we rewrote the program using exception handling. The nonexceptional cases go inside the `try` block, and the `try` block checks for the exceptional cases. The exceptional cases are not handled in the `try` block, but if detected, they are signaled by throwing an exception. The following three lines taken from inside the multiway `if-else` statement are the code for throwing the exception:

```
throw new Exception("Lesson is canceled. No students.");
throw new Exception("Lesson is canceled. No men.");
throw new Exception("Lesson is canceled. No women.");
```

If the program does not encounter an exceptional case, then none of these statements that throw an exception is executed. In that case, we need not even know what happens when an exception is thrown. If no exception is thrown, then the code in the section labeled “catch block” is skipped and the program proceeds to the last statement, which happens to output “Begin the lesson.” Now, let’s see what happens in an exceptional case.

If the number of men or the number of women is zero (or both), that is an exceptional case in this program and results in an exception being **thrown**. To make things concrete, let’s say that the number of men is zero, but the number of women is not zero. In that case, the following statement is executed, which is how Java throws an exception:

```
throw new Exception("Lesson is canceled. No men.");
```

Let’s analyze this statement. The following is the invocation of a constructor for the class `Exception`, which is the standard Java package `java.lang`:

```
new Exception("Lesson is canceled. No men.");
```

The created `Exception` object is not assigned to a variable, but rather is used as an (anonymous) argument to the `throw` operator. (Anonymous arguments were discussed in Chapter 5.) The keyword `throw` is an operator with syntax similar to the

**EXAMPLE:** (continued)

unary + or unary – operators. To make it look more like an operator, you can write it with parentheses around the argument, as follows:

```
throw (new Exception("Lesson is canceled. No men."));
```

Although it is perfectly legal and sensible to include these extra parentheses, nobody includes them.

To understand this process of throwing, you need to know two things: What is this `Exception` class? And what does the `throw` operator do with the `Exception` object? The class `Exception` is another class from the standard Java package `java.lang`. As you have already seen, the class `Exception` has a constructor that takes a single `String` argument. The `Exception` object created stores this `String` argument (in a private instance variable). As you will see, this `String` argument can later be retrieved from the `Exception` object.

The `throw` operator causes a change in the flow of control and delivers the `Exception` object to a suitable place, as we are about to explain. When the `throw` operator is executed, the `try` block ends immediately and control passes to the following `catch` block. (If it helps, you can draw an analogy between the execution of the `throw` operator in a `try` block and the execution of a `break` statement in a loop or `switch` statement.) When control is transferred to the `catch` block, the `Exception` object that is thrown is plugged in for the `catch` block parameter `e`. So, the expression `e.getMessage()` returns the string "Lesson is canceled. No men." The method `getMessage()` of the class `Exception` is an accessor method that retrieves the `String` in the private instance variable of the `Exception` object—that is, the `String` used as an argument to the `Exception` constructor.

To see if you get the basic idea of how this exception throwing mechanism works, study the Sample Dialogues in Displays 9.2 and 9.3. The next few sections explain this mechanism in more detail.

---

**Display 9.2** Handling a Special Case without Exception Handling (part 1 of 3)

---

```
1 import java.util.Scanner;

2 public class DanceLesson
3 {
4 public static void main(String[] args)
5 {
6 Scanner keyboard = new Scanner(System.in);
7
8 System.out.println("Enter number of male dancers:");
9 int men = keyboard.nextInt();

10 System.out.println("Enter number of female dancers: ");
11 int women = keyboard.nextInt();
```

(continued)

## Display 9.2 Handling a Special Case without Exception Handling (part 2 of 3)

```

12 if (men == 0 && women == 0)
13 {
14 System.out.println("Lesson is canceled. No students.");
15 System.exit(0);
16 }
17 else if (men == 0)
18 {
19 System.out.println("Lesson is canceled. No men.");
20 System.exit(0);
21 }
22 else if (women == 0)
23 {
24 System.out.println("Lesson is canceled. No women.");
25 System.exit(0);
26 }

27 // women >= 0 && men >= 0
28 if (women >= men)
29 System.out.println("Each man must dance with " +
30 women/(double)men + " women.");
31 else
32 System.out.println("Each woman must dance with " +
33 men/(double)women + " men.");
34 System.out.println("Begin the lesson.");
35 }
36 }

```

## Sample Dialogue 1

```

Enter number of male dancers:
4
Enter number of female dancers:
6
Each man must dance with 1.5 women.
Begin the lesson.

```

## Sample Dialogue 2

```

Enter number of male dancers:
0
Enter number of female dancers:
0
Lesson is canceled. No students.

```

---

Display 9.2 Handling a Special Case without Exception Handling (part 3 of 3)

---

## Sample Dialogue 3

```
Enter number of male dancers:
0
Enter number of female dancers:
5
Lesson is canceled. No men.
```

## Sample Dialogue 4

```
Enter number of male dancers:
4
Enter number of female dancers:
0
Lesson is canceled. No women.
```

---

---

Display 9.3 Same Thing Using Exception Handling (part 1 of 3)

---

```
1 import java.util.Scanner;
2
3 public class DanceLesson2
4 {
5 public static void main(String[] args)
6 {
7 Scanner keyboard = new Scanner(System.in);
8
9 System.out.println("Enter number of male dancers:");
10 int men = keyboard.nextInt();
11
12 System.out.println("Enter number of female dancers:");
13 int women = keyboard.nextInt();
```

*This is just a toy example to learn Java syntax. Do not take it  
as an example of good typical use of exception handling.*

(continued)

Display 9.3 Same Thing Using Exception Handling (part 2 of 3)

```

11 try
12 {
13 if (men == 0 && women == 0)
14 throw new Exception("Lesson is canceled. No students.");
15 else if (men == 0)
16 throw new Exception("Lesson is canceled. No men.");
17 else if (women == 0)
18 throw new Exception("Lesson is canceled. No women.");
19
20 // women >= 0 && men >= 0
21 if (women >= men)
22 System.out.println("Each man must dance with " +
23 women/(double)men + " women.");
24 else
25 System.out.println("Each woman must dance with " +
26 men/(double)women + " men.");
27 }
28 catch(Exception e)
29 {
30 String message = e.getMessage();
31 System.out.println(message);
32 System.exit(0);
33
34 System.out.println("Begin the lesson.");
35 }

```

Annotations in the original image:

- try** (line 11)
- try block** (line 18)
- catch block** (line 31)

## Sample Dialogue 1

```

Enter number of male dancers:
4
Enter number of female dancers:
6
Each man must dance with 1.5 women.
Begin the lesson.

```

## Sample Dialogue 2

```

Enter number of male dancers:
0
Enter number of female dancers:
0
Lesson is canceled. No students.

```

*Note that this dialogue and the dialogues that follow do not say "Begin the lesson".*

---

Display 9.3 Same Thing Using Exception Handling (part 3 of 3)

---

## Sample Dialogue 3

```
Enter number of male dancers:
0
Enter number of female dancers:
5
Lesson is canceled. No men.
```

## Sample Dialogue 4

```
Enter number of male dancers:
4
Enter number of female dancers:
0
Lesson is canceled. No women.
```

---

## Exception Classes

There are more exception classes than just the single class `Exception`. There are more exception classes in the standard Java libraries and you can define your own. All the exception classes in the Java libraries have—and the exception classes you define should have—the following properties:

- There is a constructor that takes a single argument of type `String`.
- The class has an accessor method `getMessage()` that can recover the string given as an argument to the constructor when the exception object was created.

### **try-throw-catch**

When used together, the `try`, `throw`, and `catch` statements are the basic mechanism for throwing and catching exceptions. The `throw` statement throws the exception. The `catch` block catches the exception. The `throw` statement is normally included in a `try` block. When the exception is thrown, the `try` block ends and then the code in the `catch` block is executed. After the `catch` block is completed, the code after the `catch` block(s) is executed (provided the `catch` block has not ended the program or performed some other special action).

If no exception is thrown in the `try` block, then after the `try` block is completed, program execution continues with the code after the `catch` block(s). (In other words, if no exception is thrown, the `catch` block(s) are ignored.)

(continued)



**SYNTAX**

```
try
{
 Some_Statements
 <Either a throw statement or
 a method invocation that might throw an exception
 or other statement that might throw an exception.>
 Some_More_Statements
}
catch (Exception_Class_Name Catch_Block_Parameter)
{
 <Code to be performed if an exception of the named exception
 class is thrown in the try block.>
}
```

You may use any legal identifier for the *Catch\_Block\_Parameter*, a common choice is *e*. The code in the *catch* block may refer to the *Catch\_Block\_Parameter*. If there is an explicit *throw* statement, it is usually embedded in an *if* statement or an *if-else* statement. There may be any number of *throw* statements and/or any number of method invocations that may throw exceptions. Each *catch* block can list only one exception, but there can be more than one *catch* block.

**EXAMPLE**

See Display 9.3.

## Exception Classes from Standard Packages

Numerous predefined exception classes are included in the standard packages that come with Java. The names of predefined exceptions are designed to be self-explanatory. Some sample predefined exceptions are

```
IOException
NoSuchMethodException
FileNotFoundException
```

**Exception**

The predefined exception class `Exception` is the root class for all exceptions. Every exception class is a descendent of the class `Exception` (that is, it is derived directly from the class `Exception` or from a class that is derived from the class `Exception`, or it arises from some longer chain of derivations ultimately starting with the class `Exception`). You can use the class `Exception` itself, just as we did in Display 9.3, but you are even more likely to use it to define a derived class of the class `Exception`. The class `Exception` is in the `java.lang` package and so requires no import statement.

## The Class `Exception`

Every exception class is a descendent class of the class `Exception`. You can use the class `Exception` itself in a class or program, but you are even more likely to use it to define a derived class of the class `Exception`. The class `Exception` is in the `java.lang` package and so requires no import statement.

## Self-Test Exercises

3. What output is produced by the following code?

```
int waitTime = 46;

try
{
 System.out.println("Try block entered.");
 if (waitTime > 30)
 throw new Exception("Over 30.");
 else if (waitTime < 30)
 throw new Exception("Under 30.");
 else
 System.out.println("No exception.");
 System.out.println("Leaving try block.");
}
catch(Exception thrownObject)
{
 System.out.println(thrownObject.getMessage());
}
System.out.println("After catch block");
```

4. Suppose that in Self-Test Exercise 3, the line

```
int waitTime = 46;
```

is changed to

```
int waitTime = 12;
```

How would this affect the output?

5. In the code given in Self-Test Exercise 3, what are the `throw` statements?
6. What happens when a `throw` statement is executed? This is a general question. Explain what happens in general, not simply what happens in the code in Self-Test Exercise 1 or some other sample code.
7. In the code given in Self-Test Exercise 3, what is the `try` block?

(continued)

### Self-Test Exercises (continued)

8. In the code given in Self-Test Exercise 3, what is the catch block?
9. In the code given in Self-Test Exercise 3, what is the catch block parameter?
10. Is the following legal?

```
Exception exceptionObject =
 new Exception("Oops!");
```

11. Is the following legal?

```
Exception exceptionObject =
 new Exception("Oops!");
throw exceptionObject;
```

## Defining Exception Classes

A `throw` statement can throw an exception object of any exception class. A common thing to do is to define an exception class whose objects can carry the precise kinds of information you want thrown to the catch block. An even more important reason for defining a specialized exception class is so that you can have a different type to identify each possible kind of exceptional situation.

Every exception class you define must be a derived class of some already defined exception class. An exception class can be a derived class of any exception class in the standard Java libraries or of any exception class that you have already successfully defined. Our examples will be derived classes of the class `Exception`.

### constructors

When defining an exception class, the constructors are the most important members. Often there are no other members, other than those inherited from the base class. For example, in Display 9.4, we have defined an exception class called `DivisionByZeroException` whose only members are a no-argument constructor and a constructor with one `String` parameter. In most cases, these two constructors are all the exception class definition contains. However, the class does inherit all the methods of the class `Exception`.<sup>1</sup> In particular, the class `DivisionByZeroException` inherits the method `getMessage`, which returns a string message. In the no-argument constructor, this string message is set with the following, which is the first line in the no-argument constructor definition:

```
super("Division by Zero!");
```

This is a call to a constructor of the base class `Exception`. As we have already noted, when you pass a string to the constructor for the class `Exception`, it sets the value

---

<sup>1</sup>Some programmers would prefer to derive the `DivisionByZeroException` class from the pre-defined class `ArithmeticException`, but that would make it a kind of exception that you are not required to catch in your code, so you would lose the help of the compiler in keeping track of uncaught exceptions. For more details, see the subsection “Exceptions to the Catch or Declare Rule” later in this chapter. If this footnote does not make sense to you, you can safely ignore it.

of a `String` instance variable that can later be recovered with a call to `getMessage`. The method `getMessage` is an ordinary accessor method of the class `Exception`. The class `DivisionByZeroException` inherits this `String` instance variable as well as the accessor method `getMessage`.

For example, in Display 9.5, we give a sample program that uses this exception class. The exception is thrown using the no-argument constructor, as follows:

```
throw new DivisionByZeroException();
```

---

#### Display 9.4 A Programmer-Defined Exception Class

---

```

1 public class DivisionByZeroException extends Exception
2 {
3 public DivisionByZeroException() You can do more in an exception
4 { constructor, but this form is
5 super("Division by Zero!"); common.
6 }

7 public DivisionByZeroException(String message)
8 {
9 super(message); super is an invocation of the constructor
10 } for the base class Exception.
11 }
```

---



---

#### Display 9.5 Using a Programmer-Defined Exception Class (part 1 of 3)

---

```

1 import java.util.Scanner;
2 public class DivisionDemoFirstVersion We will present an improved version of this
3 { program later in this chapter in Display 9.10.

4 public static void main(String[] args)
5 {
6 try
7 {
8 Scanner keyboard = new Scanner(System.in);

9 System.out.println("Enter numerator:");
10 int numerator = keyboard.nextInt();
11 System.out.println("Enter denominator:");
12 int denominator = keyboard.nextInt();
```

(continued)

## Display 9.5 Using a Programmer-Defined Exception Class (part 2 of 3)

```

13 if (denominator == 0)
14 throw new DivisionByZeroException();

15 double quotient = numerator/(double)denominator;
16 System.out.println(numerator + "/"
17 + denominator
18 + " = " + quotient);
19 }
20 catch (DivisionByZeroException e)
21 {
22 System.out.println(e.getMessage());
23 secondChance();
24 }

25 System.out.println("End of program.");
26 }

27 public static void secondChance()
28 {
29 Scanner keyboard = new Scanner(System.in);
30 System.out.println("Try again.");
31 System.out.println("Enter numerator:");
32 int numerator = keyboard.nextInt();
33 System.out.println("Enter denominator:");
34 System.out.println("Be sure the denominator is not zero.");
35 int denominator = keyboard.nextInt();

36
37 if (denominator == 0)
38 {
39 System.out.println("I cannot do division by zero.");
40 System.out.println("Aborting program.");
41 System.exit(0);
42 }

43 double quotient = ((double)numerator)/denominator;
44 System.out.println(numerator + "/"
45 + denominator
46 + " = " + quotient);
47 }
48 }

```

*Sometimes it is better to handle an exceptional case without throwing an exception.*



---

Display 9.5 Using a Programmer-Defined Exception Class (part 3 of 3)

---

## Sample Dialogue 1

```
Enter numerator:
11
Enter denominator:
5
11/5 = 2.2
End of program.
```

## Sample Dialogue 2

```
Enter numerator:
11
Enter denominator:
0
Division by Zero!
Try again.
Enter numerator:
11
Enter denominator:
Be sure the denominator is not zero.
5
11/5 = 2.2
End of program.
```

## Sample Dialogue 3

```
Enter numerator:
11
Enter denominator:
0
Division by Zero!
Try again.
Enter numerator:
11
Enter denominator:
Be sure the denominator is not zero.
0
I cannot do division by zero.
Aborting program.
```

---

This exception is caught in the `catch` block shown in Display 9.5. Consider the following line from that `catch` block:

```
System.out.println(e.getMessage());
```

This line produces the following output to the screen in Sample Dialogues 2 and 3 (in Display 9.5):

```
Division by Zero!
```

The definition of the class `DivisionByZeroException` in Display 9.4 has a second constructor with one parameter of type `String`. This constructor allows you to choose any message you like when you throw an exception. If the `throw` statement in Display 9.5 had instead used the string argument

```
throw new DivisionByZeroException(
 "Oops. Shouldn't divide by zero.");
```

then in Sample Dialogues 2 and 3, the statement

```
System.out.println(e.getMessage());
```

would have produced the following output to the screen:

```
Oops. Shouldn't divide by zero.
```

Notice that in Display 9.5, the `try` block is the normal part of the program. If all goes routinely, that is the only code that will be executed, and the dialogue will be like the one shown in Sample Dialogue 1. In the exceptional case, when the user enters a zero for a denominator, the exception is thrown and then is caught in the `catch` block. The `catch` block outputs the message of the exception and then calls the method `secondChance`. The method `secondChance` gives the user a second chance to enter the input correctly and then carries out the calculation. If the user tries a second time to divide by zero, the method ends the program. The method `secondChance` is there only for this exceptional case. So, we have separated the code for the exceptional case of a division by zero into a separate method, where it will not clutter the code for the normal case.



### TIP: Preserve `getMessage`

For all predefined exception classes, `getMessage` will return the string that is passed as an argument to the constructor (or will return a default string if no argument is used with the constructor). For example, if the exception is thrown as follows:

```
throw new Exception("Wow, this is exceptional!");
```

then “Wow, this is exceptional!” is used as the value of the `String` instance variable of the object created. If the object is called `e`, the method invocation `e.getMessage()` returns “Wow, this is exceptional!” You want to preserve this behavior in the exception classes you define.

**TIP: (continued)**

For example, suppose you are defining an exception class named `NegativeNumberException`. Be sure to include a constructor with a string parameter that begins with a call to `super`, as illustrated by the following constructor:

```
public NegativeNumberException(String message)
{
 super (message);
}
```

The call to `super` is a call to a constructor of the base class. If the base class constructor handles the message correctly, then so will a class defined in this way.

You should also include a no-argument constructor in each exception class. This no-argument constructor should set a default value to be retrieved by `getMessage`. The constructor should begin with a call to `super`, as illustrated by the following constructor:

```
public NegativeNumberException()
{
 super ("Negative Number Exception!");
}
```

If `getMessage` works as we described for the base class, then this sort of no-argument constructor will work correctly for the new exception class being defined. A full definition of the class `NegativeNumberException` is given in Display 9.9. ■

**Exception Object Characteristics**

The two most important things about an exception object are its type (the exception class) and a message that it carries in an instance variable of type `String`. This string can be recovered with the accessor method `getMessage`. This string allows your code to send a message along with an exception object, so that the `catch` block can use the message.

**Programmer-Defined Exception Classes**

You may define your own exception classes, but every such class must be a derived class of an already existing exception class (either from one of the standard Java libraries or programmer defined).

**GUIDELINES**

- If you have no compelling reason to use any other class as the base class, use the class `Exception` as the base class.
- You should define two (or more) constructors, as described later in this list.

(continued)



- Your exception class inherits the method `getMessage`. Normally, you do not need to add any other methods, but it is legal to do so.
- You should start each constructor definition with a call to the constructor of the base class, such as the following:

```
super("Sample Exception thrown!");
```

- You should include a no-argument constructor, in which case the call to `super` should have a string argument that indicates what kind of exception it is. This string can then be recovered by using the `getMessage` method.
- You should also include a constructor that takes a single string argument. In this case, the string should be an argument in a call to `super`. That way, the string can be recovered with a call to `getMessage`.

#### EXAMPLE

```
public class SampleException extends Exception
{
 public SampleException()
 {
 super("Sample Exception thrown!");
 }

 public SampleException(String message)
 {
 super(message);
 }
}
```

The class `SampleException` is on the website that comes with this text.

extra code  
on website



#### TIP: An Exception Class Can Carry a Message of Any Type

It is possible to define your exception classes so they have constructors that take arguments of other types that are stored in instance variables. In such cases, you would define accessor methods for the value stored in the instance variable. For example, if that is desired, you can have an exception class that carries an `int` as a message. In that case, you would need a new accessor method name, perhaps `getBadNumber()`. An example of one such exception class is given in Display 9.6. Display 9.7 is a demonstration of how to use the accessor method `getBadNumber()`. This is just a toy program, but it does illustrate the details of how an exception object can carry a numeric message. ■

Display 9.6 An Exception Class with an `int` Message (part 1 of 2)

```
1 public class BadNumberException extends Exception
2 {
3 private int badNumber;
4 public BadNumberException(int number)
5 {
```

Display 9.6 An Exception Class with an `int` Message (part 2 of 2)

```
6 super ("BadNumberException");
7 badNumber = number;
8 }

9 public BadNumberException()
10 {
11 super ("BadNumberException");
12 }

13 public BadNumberException(String message)
14 {
15 super (message);
16 }

17 public int getBadNumber()
18 {
19 return badNumber;
20 }
21 }
```

Display 9.7 Demonstration of How to Use `BadNumberException` (part 1 of 2)

```
1 import java.util.Scanner;
2 public class BadNumberExceptionDemo
3 {
4 public static void main(String[] args)
5 {
6 try
7 {
8 Scanner keyboard = new Scanner(System.in);
9
10 System.out.println("Enter one of the numbers 42 and 24:");
11 int inputNumber = keyboard.nextInt();
12 if ((inputNumber != 42) && (inputNumber != 24))
13 throw new BadNumberException(inputNumber);
14
15 System.out.println("Thank you for entering " + inputNumber);
16 }
17 catch(BadNumberException e)
18 {
19 System.out.println(e.getBadNumber() +
20 " is not what I asked for.");
21 }
22 System.out.println("End of program.");
23 }
24 }
```

(continued)

Display 9.7 Demonstration of How to Use `BadNumberException` (part 2 of 2)

## Sample Dialogue 1

```
Enter one of the numbers 42 and 24:
42
Thank you for entering 42
End of program.
```

## Sample Dialogue 2

```
Enter one of the numbers 42 and 24:
44
44 is not what I asked for.
End of program.
```

## Self-Test Exercises

12. Define an exception class called `PowerFailureException`. The class should have a constructor with no parameters. If an exception is thrown with this zero-argument constructor, `getMessage` should return "Power Failure!" The class should also have a constructor with a single parameter of type `String`. If an exception is thrown with this constructor, then `getMessage` returns the value that was used as an argument to the constructor.
13. Define an exception class called `TooMuchStuffException`. The class should have a constructor with no parameters. If an exception is thrown with this zero-argument constructor, `getMessage` should return "Too much stuff!" The class should also have a constructor with a single parameter of type `String`. If an exception is thrown with this constructor, then `getMessage` returns the value that was used as an argument to the constructor.
14. Suppose the exception class `ExerciseException` is defined as follows:

```
public class ExerciseException extends Exception
{
 public ExerciseException()
 {
 super("Exercise Exception thrown!");
 System.out.println("Exception thrown.");
 }

 public ExerciseException(String message)
 {
 super(message);
 System.out.println(
 "ExerciseException invoked with an argument.");
 }
}
```

## Self-Test Exercises (continued)

What output would be produced by the following code (which is just an exercise and not likely to occur in a program)?

```
ExerciseException e =
 new ExerciseException("Do Be Do");
System.out.println(e.getMessage());
```

extra code  
on website

The class `ExerciseException` is on the website that comes with this text.

15. Suppose the exception class `TestException` is defined as follows:

```
public class TestException extends Exception
{
 public TestException()
 {
 super("Test Exception thrown!");
 System.out.println(
 "Test exception thrown!!");
 }

 public TestException(String message)
 {
 super(message);
 System.out.println(
 "Test exception thrown with an argument!");
 }

 public void testMethod()
 {
 System.out.println("Message is " + getMessage());
 }
}
```

What output would be produced by the following code (which is just an exercise and not likely to occur in a program)?

```
TestException exceptionObject = new TestException();
System.out.println(exceptionObject.getMessage());
exceptionObject.testMethod();
```

extra code  
on website

The class `TestException` is on the website that comes with this text.

16. Suppose the exception class `MyException` is defined as follows:

```
public class MyException extends Exception
{
 public MyException()
 {
 super("My Exception thrown!");
 }
}
```

(continued)

**Self-Test Exercises** (continued)

```
public MyException(String message)
{
 super("MyException: " + message);
}
```

What output would be produced by the following code (which is just an exercise and not likely to occur in a program)?

```
int number;
try
{
 System.out.println("try block entered:");
 number = 42;
 if (number > 0)
 throw new MyException("Hi Mom!");
 System.out.println("Leaving try block.");
}

catch(MyException exceptionObject)
{
 System.out.println(exceptionObject.getMessage());
}
System.out.println("End of example.");
```

extra code  
on website

The class `MyException` is on the website that comes with this text.

17. Suppose that in Self-Test Exercise 16, the `catch` block were changed to the following. (The type `MyException` is replaced with `Exception`.) How would this affect the output?

```
catch(Exception exceptionObject)
{
 System.out.println(exceptionObject.getMessage());
}
```

18. Suppose that in Self-Test Exercise 16, the line

```
number = 42;
```

were changed to

```
number = -58;
```

How would this affect the output?

**Self-Test Exercises** (continued)

19. Although an exception class normally carries only a string message, you can define exception classes to carry a message of any type. For example, objects of the following type can also carry a double “message” (as well as a string message):

```
public class DoubleException extends Exception
{
 private double doubleMessage;

 public DoubleException()
 {
 super("DoubleException thrown!");
 }

 public DoubleException(String message)
 {
 super(message);
 }

 public DoubleException(double number)
 {
 super("DoubleException thrown!");
 doubleMessage = number;
 }

 public double getNumber()
 {
 return doubleMessage;
 }
}
```

What output would be produced by the following code (which is just an exercise and not likely to occur in a program)?

```
DoubleException e =
 new DoubleException(41.9);

System.out.println(e.getNumber());
System.out.println(e.getMessage());
```

The class `DoubleException` is on the website that comes with this text.

20. There is an exception class named `IOException` that is defined in the standard Java libraries. Can you define an exception class as a derived class of the predefined class `IOException`, or must a defined exception class be derived from the class `Exception`?

**Multiple catch Blocks**

A `try` block can potentially throw any number of exception values, and they can be of differing types. In any one execution of the `try` block, at most one exception will be thrown (since a `throw` statement ends the execution of

extra code  
on website

the `try` block), but different types of exception values can be thrown on different occasions when the `try` block is executed. Each `catch` block can only catch values of the exception class type given in the `catch` block heading. However, you can catch exception values of differing types by placing more than one `catch` block after a `try` block. For example, the program in Display 9.8 has two `catch` blocks after its `try` block. The class `NegativeNumberException`, which is used in that program, is given in Display 9.9.

Display 9.8 Catching Multiple Exceptions (part 1 of 2)

---

```

1 import java.util.Scanner;

2 public class MoreCatchBlocksDemo
3 {
4 public static void main(String[] args)
5 {
6 Scanner keyboard = new Scanner(System.in);
7
8 try
9 {
10 System.out.println("How many pencils do you have?");
11 int pencils = keyboard.nextInt();

12 if (pencils < 0)
13 throw new NegativeNumberException("pencils");

14 System.out.println("How many erasers do you have?");
15 int erasers = keyboard.nextInt();
16 double pencilsPerEraser;

17 if (erasers < 0)
18 throw new NegativeNumberException("erasers");
19 else if (erasers != 0)
20 pencilsPerEraser = pencils/(double)erasers;
21 else
22 throw new DivisionByZeroException();

23 System.out.println("Each eraser must last through "
24 + pencilsPerEraser + " pencils.");
25 }
26 catch(NegativeNumberException e)
27 {
28 System.out.println("Cannot have a negative number of "
29 + e.getMessage());
30 }
31 catch(DivisionByZeroException e)
32 {
33 System.out.println("Do not make any mistakes.");

```

---

Display 9.8 Catching Multiple Exceptions (part 2 of 2)

---

```
34 }

35 System.out.println("End of program.");
36 }
37 }
```

## Sample Dialogue 1

```
How many pencils do you have?
5
How many erasers do you have?
2
Each eraser must last through 2.5 pencils
End of program.
```

## Sample Dialogue 2

```
How many pencils do you have?
-2
Cannot have a negative number of pencils
End of program.
```

## Sample Dialogue 3

```
How many pencils do you have?
5
How many erasers do you have?
0
Do not make any mistakes.
End of program.
```

**PITFALL: Catch the More Specific Exception First**

When catching multiple exceptions, the order of the `catch` blocks can be important. When an exception is thrown in a `try` block, the `catch` blocks are examined in order, and the first one that matches the type of the exception thrown is the one that is executed. Thus, the following ordering of `catch` blocks would not be good:

```
catch (Exception e)
{
 .
 .
 .
}
```

(continued)



**PITFALL: (continued)**

```
catch(NegativeNumberException e)
{
 .
 .
 .
}
```

*The second catch block can never be reached.*

With this ordering, the `catch` block for `NegativeNumberException` would never be used, because all exceptions are caught by the first `catch` block. Fortunately, the compiler will warn you about this. The correct ordering is to reverse the `catch` blocks so that the more specific exception comes before its parent exception class, as shown in the following:

```
catch(NegativeNumberException e)
{
 .
 .
 .
}

catch(Exception e)
{
 .
 .
 .
}
```

**Display 9.9 The Class `NegativeNumberException`**

```
1 public class NegativeNumberException extends Exception
2 {
3 public NegativeNumberException()
4 {
5 super("Negative Number Exception!");
6 }
7 public NegativeNumberException(String message)
8 {
9 super(message);
10 }
11 }
```

## Self-Test Exercises

21. What output will be produced by the following code? (The definition of the class `NegativeNumberException` is given in Display 9.9.)

```
int n;
try
{
 n = 42;
 if (n > 0)
 throw new Exception();
 else if (n < 0)
 throw new NegativeNumberException();
 else
 System.out.println("Bingo!");
}
catch(NegativeNumberException e)
{
 System.out.println("First catch.");
}
catch(Exception e)
{
 System.out.println("Second catch.");
}
System.out.println("End of exercise.");
```

22. Suppose that in Self-Test Exercise 21, the line

```
n = 42;
```

is changed to

```
n = -42;
```

How would this affect the output?

23. Suppose that in Self-Test Exercise 21, the line

```
n = 42;
```

is changed to

```
n = 0;
```

How would this affect the output?

## 9.2 Throwing Exceptions in Methods

*The buck stops here.*

HARRY S. TRUMAN (sign on Truman's desk while he was president), 1945.

So far, our examples of exception handling have been toy examples. We have not yet shown any examples of a program that makes good and realistic use of exception handling. However, now you know enough about exception handling to discuss more realistic uses of it. This section explains the single most important exception handling technique, namely throwing an exception in a method and catching it outside the method.

### Throwing an Exception in a Method

Sometimes it makes sense to throw an exception in a method but not catch it in the method. For example, you might have a method with code that throws an exception if there is an attempt to divide by zero, but you may not want to catch the exception in that method. Perhaps some programs that use that method should simply end if the exception is thrown, and other programs that use the method should do something else. So, you would not know what to do with the exception if you caught it inside the method. In such cases, it makes sense to not catch the exception in the method definition, but instead to have any program (or other code) that uses the method place the method invocation in a `try` block and catch the exception in a `catch` block that follows that `try` block.

Look at the program in Display 9.10. It has a `try` block, but there is no `throw` statement visible in the `try` block. The statement that does the throwing in that program is

```
if (bottom == 0)
 throw new DivisionByZeroException();
```

This statement is not visible in the `try` block. However, it is in the `try` block in terms of program execution, because it is in the definition of the method `safeDivide`, and there is an invocation of `safeDivide` in the `try` block.

The meaning of `throws DivisionByZero` in the heading of `safeDivide` is discussed in the next subsection.

Display 9.10 Use of a `throws` Clause (part 1 of 2)

```
1 import java.util.Scanner;

2 public class DivisionDemoSecondVersion
3 {
4 public static void main(String[] args)
5 {
6 Scanner keyboard = new Scanner(System.in);

7 try
8 {
9 System.out.println("Enter numerator:");
10 int numerator = keyboard.nextInt();
11 System.out.println("Enter denominator:");
12 int denominator = keyboard.nextInt();
13 double quotient = safeDivide(numerator, denominator);
14 System.out.println(numerator + "/"
15 + denominator
16 + " = " + quotient);
17 }
18 catch (DivisionByZeroException e)
19 {
20 System.out.println(e.getMessage());
21 secondChance();
22 }
23
24 System.out.println("End of program.");
25 }

26

27 public static double safeDivide(int top, int bottom)
28 throws DivisionByZeroException
29 {
30 if (bottom == 0)
31 throw new DivisionByZeroException();

32 return top/(double)bottom;
33 }
```

(continued)

Display 9.10 Use of a `throws` Clause (part 2 of 2)

---

```

34 public static void secondChance()
35 {
36 Scanner keyboard = new Scanner(System.in);
37
38 try
39 {
40 System.out.println("Enter numerator:");
41 int numerator = keyboard.nextInt();
42 System.out.println("Enter denominator:");
43 int denominator = keyboard.nextInt();
44
45 double quotient = safeDivide(numerator, denominator);
46 System.out.println(numerator + "/" +
47 + denominator
48 + " = " + quotient);
49 }
50 catch (DivisionByZeroException e)
51 {
52 System.out.println("I cannot do division by zero.");
53 System.out.println("Aborting program.");
54 System.exit(0);
55 }
56 }

```

*The input/output dialogues are identical to those for the program in Display 9.5.*

---

## Declaring Exceptions in a `throws` Clause

### `throws` clause

#### declaring an exception

If a method does not catch an exception, then (in most cases) it must at least warn programmers that any invocation of the method might possibly throw an exception. This warning is called a *throws clause*, and including an exception class in a `throws` clause is called **declaring the exception**. For example, a method that might possibly throw a `DivisionByZeroException` and that does not catch the exception would have a heading similar to the following:

```
public void sampleMethod() throws DivisionByZeroException
```

### `throws` clause

The part `throws DivisionByZeroException` is a **`throws` clause** stating that an invocation of the method `sampleMethod` might throw a `DivisionByZeroException`.

If there is more than one possible exception that can be thrown in the method definition, then the exception types are separated by commas, as illustrated in what follows:

```
public void sampleMethod()
 throws DivisionByZeroException, SomeOtherException
```

Most “ordinary” exceptions that might be thrown when a method is invoked must be accounted for in one of two ways:

- The possible exception can be caught in a `catch` block within the method definition.
- The possible exception can be declared at the start of the method definition by placing the exception class name in a `throws` clause (and letting whoever uses the method worry about how to handle the exception).

### Catch or Declare Rule

This is often called the **Catch or Declare Rule**. In any one method, you can mix the two alternatives, catching some exceptions and declaring others in a `throws` clause.

You already know about the first technique, handling exceptions in a `catch` block. The second technique is a form of shifting responsibility (“passing the buck”). For example, suppose `yourMethod` has a `throws` clause as follows:

```
public void yourMethod() throws DivisionByZeroException
```

In this case, `yourMethod` is absolved of the responsibility of catching any exceptions of type `DivisionByZeroException` that might occur when `yourMethod` is executed. If, however, there is another method (`myMethod`) that includes an invocation of `yourMethod`, then `myMethod` must handle the exception. When you add a `throws` clause to `yourMethod`, you are saying to `myMethod`, “If you invoke `yourMethod`, you must handle any `DivisionByZeroException` that is thrown.” In effect, `yourMethod` has passed the responsibility for any exceptions of type `DivisionByZeroException` from itself to any method that calls it.

Of course, if `yourMethod` passes responsibility to `myMethod` by including `DivisionByZeroException` in a `throws` clause, then `myMethod` may also pass the responsibility to whoever calls it by including the same `throws` clause in its definition. But in a well-written program, every exception that is thrown should eventually be caught by a `catch` block in some method that does not just declare the exception class in a `throws` clause.

### throws Clause

If you define a method that might throw exceptions of some particular class, then normally either your method definition must include a `catch` block that will catch the exception or you must declare (that is, list) the exception class within a `throws` clause, as described in what follows.

#### SYNTAX (COVERS MOST COMMON CASES)

```
public Type_Or_void Method(Parameter_List) throws List_Of_Exceptions
 Body_Of_Method
```

#### EXAMPLE

```
public void yourMethod(int n) throws MyException, YourException
{
 .
 .
 .
}
```

When an exception is thrown in a method but not caught in that method, that immediately ends the method invocation.

Be sure to note that the `throws` clause for a method is for exceptions that “get outside” the method. If they do not get outside the method, they do not belong in the `throws` clause. If they get outside the method, they belong in the `throws` clause no matter where they originate. If an exception is thrown in a `try` block that is inside a method definition and is caught in a `catch` block inside the method definition, then its exception class need not be listed in the `throws` clause. If a method definition includes an invocation of another method and that other method can throw an exception that is not caught, then the exception class of that exception should be placed in the `throws` clause.

### Throwing an Exception Can End a Method

If a method throws an exception, and the exception is not caught inside the method, then the method invocation ends immediately after the exception is thrown.

In Display 9.10, we have rewritten the program from Display 9.5 so that the exception is thrown in the method `safeDivide`. The method `main` includes a call to the method `safeDivide` and puts the call in a `try` block. Because the method `safeDivide` can throw a `DivisionByZeroException` that is not caught in the method `safeDivide`, we need to declare this in a `throws` clause at the start of the definition of `safeDivide`. If we set up our program in this way, the case in which nothing goes wrong is completely isolated and easy to read. It is not even cluttered by `try` blocks and `catch` blocks.

### Catch or Declare Rule

Most “ordinary” exceptions that might be thrown when a method is invoked must be accounted for in one of two ways:

- The possible exception can be caught in a `catch` block within the method definition.
- The possible exception can be declared at the start of the method definition by placing the exception class name in a `throws` clause (and letting whoever uses the method worry about how to handle the exception).

This is known as the **Catch or Declare Rule**. In any one method, you can mix the two alternatives, catching some exceptions and declaring others in a `throws` clause.

If you use a class that is subject to the Catch or Declare Rule and you do not follow the rule, you will get a compiler error message. The box entitled “Checked and Unchecked Exceptions” explains exactly which exception classes are subject to the Catch or Declare Rule.

The next subsection, entitled “Exceptions to the Catch or Declare Rule,” explains exactly which exception classes are subject to the Catch or Declare Rule. However, the compiler will ensure that you follow the Catch or Declare Rule when it is required. So if you do not know whether a class is subject to the Catch or Declare Rule, you can rely on the compiler to tell you. If you use a class that is subject to the Catch or Declare Rule and you do not follow the rule, you will get a compiler error message.

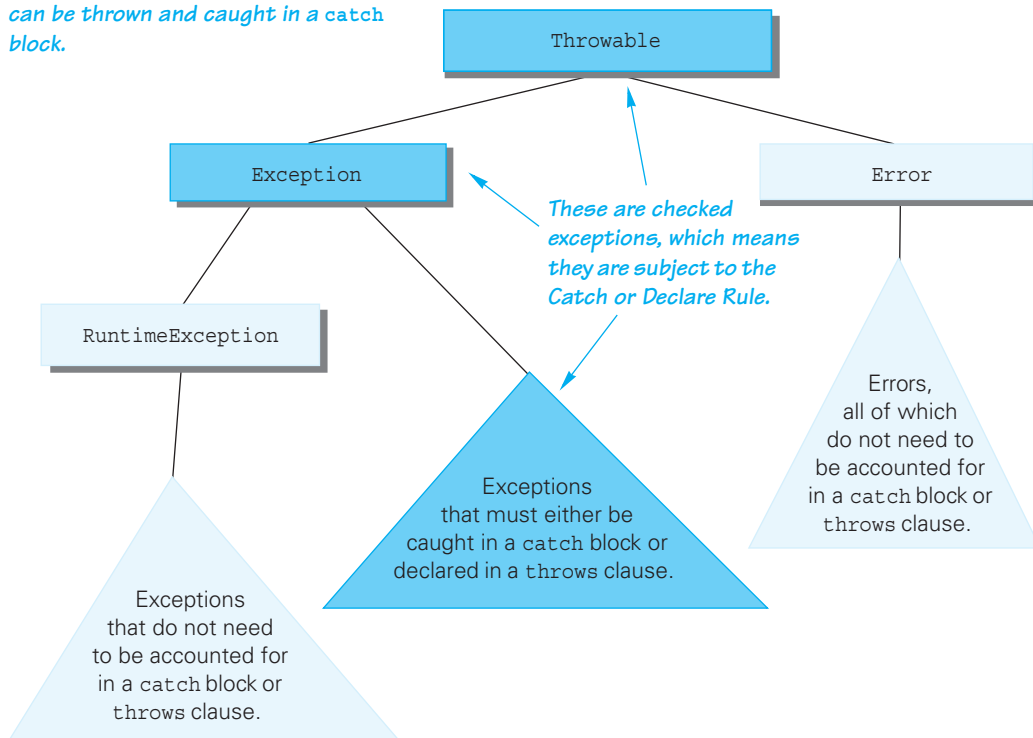
## Exceptions to the Catch or Declare Rule

As we already noted, in most “ordinary” cases, an exception must either be caught in a catch block or declared in a throws clause. This is the Catch or Declare Rule, but there are exceptions to this rule. There are some classes whose exceptions you do not need to account for in this way (although you can catch them in a catch block if you want to). These are typically exceptions that result from errors of some sort. They usually indicate that your code should be fixed, not that you need to add a catch block. They are often thrown by methods in standard library classes, but it would be legal to throw one of these exceptions in the code you write.

Exceptions that are descendents of the class `RuntimeException` do not need to be accounted for in a catch block or throws clause. Another category of classes called `Error` classes behave like exception classes in that they can be thrown and caught in a catch block. However, you are not required to account for `Error` objects in a catch block or throws clause. The situation is diagrammed as a class hierarchy in Display 9.11. All the classes shown in blue follow the Catch or Declare Rule, which says that if their objects are thrown, then they must either be caught in a catch block or declared in a throws clause. All the classes shown in yellow are exempt from the Catch or Declare Rule.

Display 9.11 Hierarchy of Throwable Objects

*All descendents of the class `Throwable` can be thrown and caught in a catch block.*





### What Happens If an Exception Is Never Caught?

If every method up to and including the `main` method simply includes a `throws` clause for a particular class of exceptions, then it may turn out that an exception of that class is thrown but never caught. In such cases, when an exception is thrown but never caught, then for the kinds of programs we have seen so far, the program ends with an error message giving the name of the exception class. (In Chapter 17 we will discuss programs with windowing interfaces that are known as *GUI programs*. For GUI programs, if an exception is thrown but never caught, then nothing happens, but if your code does not somehow account for the thrown exception, then the user may be left in an unexplained situation.)

In a well-written program, every exception that is thrown should eventually be caught by a `catch` block in some method.

#### checked and unchecked exceptions

Exception classes that follow the Catch or Declare Rule are often called **checked exceptions**. Exceptions that are exempt from the Catch or Declare Rule are often called **unchecked exceptions**.

### Checked and Unchecked Exceptions

Exceptions that are subject to the Catch or Declare Rule are called **checked exceptions** because the compiler *checks* to see if they are accounted for with a `catch` block or `throws` clause. Exceptions that are *not* subject to the Catch or Declare Rule are called **unchecked exceptions**. The classes `Throwable`, `Exception`, and all descendents of the class `Exception` are checked exceptions. All other exceptions are unchecked exceptions. The class `Error` and all its descendent classes are called **error classes** and are *not* subject to the Catch or Declare Rule. Although they are technically not exceptions, you can safely consider these error classes to be unchecked exceptions. (Strictly speaking, the class `Throwable` is neither an exception nor an error class, but it is seldom used and can be treated as a checked exception if it is used.)

You need not worry too much about which exceptions you do and do not need to declare in a `throws` clause. If you fail to account for some exception that Java requires you to account for, the compiler will tell you about it, and you can then either catch it or declare it in a `throws` clause.

### `throws` Clause in Derived Classes

When you override a method definition in a derived class, it should have the same exception classes listed in its `throws` clause that it had in the base class, or it should have a `throws` clause whose exceptions are a subset of those in the base class `throws` clause. Put another way, when you override a method definition, you cannot add any exceptions to the `throws` clause (but you can delete some exceptions if you want; you also can replace an exception class by any descendent exception class). This makes sense, because an

object of the derived class might be used anyplace an object of the base class can be used, so an overridden method must fit into any code written for an object of the base class.

## When to Use Exceptions

So far, most of our examples of exception handling have been unrealistically simple. A better guideline for how you should use exceptions is to separate throwing an exception and catching the exception into separate methods. In most cases, you should include any `throw` statement within a method definition, list the exception class in a `throws` clause for that method, and place the `try` and `catch` blocks in *a different method*. In outline form, the technique is as follows:

```
public void yourMethod() throws YourException
{
 ...
 throw new YourException(<Maybe an argument.>);
 ...
}
```

Then, when `yourMethod` is used by some `otherMethod`, the `otherMethod` must account for the exception. For example,

```
public void otherMethod()
{
 ...
 try
 {
 ...
 yourMethod();
 ...
 }
 catch (YourException e)
 {
 <Handle exception.>
 }
 ...
}
```

Even this kind of use of a `throw` statement should be reserved for cases where it is unavoidable. *If you can easily handle a problem in some other way, do not throw an exception. Reserve `throw` statements for situations in which the way the exceptional condition is handled depends on how and where the method is used.* If the way that the exceptional condition is handled depends on how and where the method is invoked, then the best thing to do is to let the programmer who invokes the method handle the exception. In all other situations, it is preferable to avoid throwing exceptions. Let's outline a sample scenario of this kind of situation.

Suppose you are writing a library of methods to deal with patient monitoring systems for hospitals. One method might compute the patient's average daily temperature by

accessing the patient's record in some file and dividing the sum of the temperatures by the number of times the temperature was taken. Now suppose these methods are used for creating different systems to be used in different situations. What should happen if the patient's temperature was never taken and so the averaging would involve a division by zero? In an intensive care unit, this would indicate something is very wrong. So for this system, when this potential division by zero would occur, an emergency message should be sent out. However, for a system that is to be used in a less urgent setting, such as outpatient care or even in some noncritical wards, it might have no significance, and so a simple note in the patient's record would suffice. In this scenario, the method for doing the averaging of the temperatures should throw an exception when this division by zero occurs, list the exception in the `throws` clause, and let each system handle the exception case in the way that is appropriate to that system.

### When to Throw an Exception

Exceptions should be reserved for situations where a method has an exceptional case and individual invocations of the method would handle the exceptional case differently. In this situation, you would throw an exception in the method definition and not catch the exception in the method, but list it in the `throws` clause for the method. This way the programmers who invoke the method can handle the exception differently in different situations.

### EXAMPLE: Retrieving a High Score

Throwing an exception in a method is especially helpful when the exception has no relationship to the return value of the method. For example, consider a method that returns the high score for a game. What should the method return if the high score has never been set? One strategy is to return a special value, such as a negative number. This strategy is employed in the program shown in Display 9.12.

Display 9.12 Method Returning a High Score without an Exception (part 1 of 2)

```
1 public class HighScore
2 {
3 private int score = 0;
4 private boolean scoreSet = false;
5
6 public HighScore()
7 {
8 score = 0;
9 scoreSet = false;
10 }
11
12 public void setScore(int newScore)
```

Display 9.12 Method Returning a High Score without an Exception (part 2 of 2)

```
13 {
14 score = newScore;
15 scoreSet = true;
16 }
17
18 public int getScore()
19 {
20 if (!scoreSet)
21 return -1;
22 else
23 return score;
24 }
25
26 // Short test program
27 public static void main(String[] args)
28 {
29 HighScore highscore = new HighScore();
30 System.out.println(highscore.getScore());
31 highscore.setScore(100);
32 System.out.println(highscore.getScore());
33 }
34 }
```

Sample Dialogue

```
-1
100
```

This program outputs -1 when the high score has not been set. In this case, the return value is treated like a normal high score, but it is really an exceptional condition. We could add a check for a negative number, but what if a negative high score is valid? We have no way to tell if the return value means that the score was never set or if it is an actual high score.

A solution to our quandary is to throw an exception if the high score is never set. The code in the main method can check for the exception and handle it separately from the return value. A modified version of the program that uses a `ScoreNotSetException` is shown in Display 9.13.

Display 9.13 Method Returning a High Score Using an Exception (part 1 of 3)

```
1 public class ScoreNotSetException extends
2 Exception
3 {
4 public ScoreNotSetException()
5 {
6 super("Score not set");
7 }
8 }
```

(continued)

Display 9.13 Method Returning a High Score Using an Exception (part 2 of 3)

```
8 public ScoreNotSetException(String message)
9 {
10 super(message);
11 }
12 }
13
14 public class HighScore
15 {
16 private int score = 0;
17 private boolean scoreSet = false;
18
19 public HighScore()
20 {
21 score = 0;
22 scoreSet = false;
23 }
24
25 public void setScore(int newScore)
26 {
27 score = newScore;
28 scoreSet = true;
29 }
30
31 public int getScore() throws
32 ScoreNotSetException
33 {
34 if (!scoreSet)
35 throw new ScoreNotSetException();
36 else
37 return score;
38 }
39
40 // Short test program
41 public static void main(String[] args)
42 {
43 HighScore highscore = new HighScore();
44 try
45 {
46 System.out.println
47 (highscore.getScore());
48 }
49 catch (ScoreNotSetException e)
50 {
51 System.out.println
52 (e.getMessage());
53 }
54 highscore.setScore(100);
55 try
```

Display 9.13 Method Returning a High Score Using an Exception (part 3 of 3)

```

56 {
57 System.out.println
58 (highscore.getScore());
59 }
60 catch (ScoreNotSetException e)
61 {
62 System.out.println
63 (e.getMessage());
64 }
65 }
66 }

```

Sample Dialogue

```

Score not set
100

```

This program throws a `ScoreNotSetException` when the high score has not been set. This allows the `main` method to differentiate between a high score that is not set and a high score that is `-1` by catching the exception.

## Event-Driven Programming ★

event-driven  
programming  
firing an event

Exception handling is our first example of a programming methodology known as **event-driven programming**. With event-driven programming, objects are defined so that they send **events**, which are themselves objects, to other objects that handle the events. Sending the event is called **firing the event**. In exception handling, the event objects are the exception objects. They are fired (thrown) by an object when the object invokes a method that throws the exception. An exception event is sent to a `catch` block, where it is handled. Of course, a `catch` block is not exactly an object, but the idea is the same. Also, our programs have mixed event-driven programming (exception handling) with more traditional programming techniques. When we study how you construct windowing systems using the Swing libraries (Chapter 17), you will see examples of programming where the dominant technique is event-driven programming.

## Self-Test Exercises

24. What is the output produced by the following program?

```

public class Exercise
{
 public static void main(String[] args)
 {
 try

```

## Self-Test Exercises (continued)

```

 {
 System.out.println("Trying");
 sampleMethod(98.6);
 System.out.println("Trying after call.");
 }
 catch(Exception e)
 {
 System.out.println("Catching.");
 }

 System.out.println("End program.");
 }

 public static void sampleMethod(double test)
 throws Exception
 {
 System.out.println("Starting sampleMethod.");
 if (test < 100)
 throw new Exception();
 }
}

```

extra code  
on website

The class `Exercise` is on the website that comes with this text.

25. Suppose that in Self-Test Exercise 22, the line

```
sampleMethod(98.6);
```

in the `try` block is changed to

```
sampleMethod(212);
```

How would this affect the output?

26. Correct the following method definition by adding a suitable `throws` clause:

```

public static void doStuff(int n)
{
 if (n < 0)
 throw new Exception("Negative number.");
}

```

27. What happens if an exception is thrown inside a method invocation, but the exception is not caught inside the method?
28. Suppose there is an invocation of method A inside of method B, and an invocation of method B inside of method C. When method C is invoked, this leads to an invocation of method B, and that in turn leads to an invocation of method A. Now, suppose that method A throws an exception but does not catch it within A. Where might the exception be caught? In B? In C? Outside of C?

## 9.3 More Programming Techniques for Exception Handling

*Only use this in exceptional circumstances.*

BOB SHAW, *Who Goes Here? Ace Books, 1979.*

In this section, we present a number of the finer points about programming with exception handling in Java.



### PITFALL: Nested try-catch Blocks

You can place a try block and its following catch blocks inside a larger try block or inside a larger catch block. On rare occasions this may be useful, but it is almost always better to place the inner try catch blocks inside a method definition and place an invocation of the method in the outer try or catch block (or maybe just eliminate one or more try blocks completely).

If you place a try block and its following catch blocks inside a larger catch block, you will need to use different names for the catch block parameters in the inner and outer blocks. This has to do with how Java handles nested blocks of any kind. Remember, try blocks and catch blocks are blocks.

If you place a try block and its following catch blocks inside a larger try block, and an exception is thrown in the inner try block but is not caught in the inner catch blocks, then the exception is thrown to the outer try block for processing and might be caught in one of its catch blocks. ■

### The finally Block ★

The finally block contains code to be executed whether or not an exception is thrown in a try block. The finally block, if used, is placed after a try block and its following catch blocks. The general syntax is as follows:

```
try
{
 ...
}
catch(ExceptionClass1 e)
{
 ...
}
.
.
.
```



```
catch(ExceptionClassLast e)
{
 ...
}
finally
{
 < Code to be executed whether or not an exception is thrown or caught.>
}
```

Now, suppose that the `try-catch-finally` blocks are inside a method definition. (After all, every set of `try-catch-finally` blocks is inside of some method, even if it is only the method `main`.) There are three possibilities when the code in the `try-catch-finally` blocks is run:

- The `try` block runs to the end and no exception is thrown. In this case, the `finally` block is executed after the `try` block.
- An exception is thrown in the `try` block and is caught in one of the `catch` blocks positioned after the `try` block. In this case, the `finally` block is executed after the `catch` block is executed.
- An exception is thrown in the `try` block and there is no matching `catch` block in the method to catch the exception. In this case, the method invocation ends and the exception object is thrown to the enclosing method. However, the `finally` block is executed before the method ends. Note that you cannot account for this last case simply by placing code after the `catch` blocks.

## Self-Test Exercises

29. Can you have a `try` block and corresponding `catch` blocks inside another larger `try` block?
30. Can you have a `try` block and corresponding `catch` blocks inside another larger `catch` block?
31. What is the output produced by the following program? What would the output be if the argument to `exerciseMethod` were `-42` instead of `42`? (The class `NegativeNumberException` is defined in Display 9.8, but you need not review that definition to do this exercise.)

```
public class FinallyDemo
{
 public static void main(String[] args)
 {
 try
 {
 exerciseMethod(42);
 }
 }
}
```

**Self-Test Exercises** (continued)

```

 catch(Exception e)
 {
 System.out.println("Caught in main.");
 }
 }
 public static void exerciseMethod(int n) throws Exception
 {
 try
 {
 if (n > 0)
 throw new Exception();
 else if (n < 0)
 throw new NegativeNumberException();
 else
 System.out.println("No Exception.");
 System.out.println("Still in sampleMethod.");
 }
 catch(NegativeNumberException e)
 {
 System.out.println("Caught in sampleMethod.");
 }
 finally
 {
 System.out.println("In finally block.");
 }
 System.out.println("After finally block.");
 }
}

```

extra code  
on website

The class `FinallyDemo` is on the website that comes with this text.

**Rethrowing an Exception ★**

A `catch` block can contain code that throws an exception. In rare cases, you may find it useful to catch an exception and then, depending on the string produced by `getMessage` or depending on something else, decide to throw the same or a different exception for handling further up the chain of exception handling blocks.

**The `AssertionError` Class ★**

When we discussed the `assert` operator and assertion checking in Chapter 3, we said that if your program contains an assertion check and the assertion check fails, your program will end with an error message. This statement is more or less true, but it is incomplete. What happens is that an object of the class `AssertionError` is thrown. If it is not caught in a `catch` block, your program ends with an error message. However, if you wish, you can catch it in a `catch` block, although that is not a very common

thing to do. The `AssertionError` class is in the `java.lang` package and so requires no import statement.

As the name suggests, the class `AssertionError` is derived from the class `Error`, so you are not required to either catch it in a `catch` block or declare it in a `throws` clause.

### `ArrayIndexOutOfBoundsException`

Read Section 6.1 of Chapter 6, which covers array basics, before reading this short subsection. If you have not yet covered some of Chapter 6, omit this section and return to it at a later time.

If your program attempts to use an array index that is out of bounds, an `ArrayIndexOutOfBoundsException` is thrown and your program ends, unless the exception is caught in a `catch` block. `ArrayIndexOutOfBoundsException` is a descendent of the class `RuntimeException` and so need not be caught or accounted for in a `throws` clause. This sort of exception normally indicates that there is something wrong with your code and means that you need to fix your code, not catch an exception. Thus, an `ArrayIndexOutOfBoundsException` normally functions more like a run-time error message than a regular exception.

`ArrayIndexOutOfBoundsException` is in the standard Java package `java.lang` and so requires no import statement should you decide to use it by name.

## Chapter Summary

- Exception handling allows you to design and code the normal case for your program separately from the code that handles exceptional situations.
- An exception can be thrown in a *try block*. Alternatively, an exception can be thrown in a method definition that does not include a *try* block (or does not include a *catch* block to catch that type of exception). In this case, an invocation of the method can be placed in a *try* block.
- An exception is caught in a *catch block*.
- A *try* block must be followed by at least one *catch* block and can be followed by more than one *catch* block. If there are multiple *catch* blocks, always list the *catch* block for a more specific exception class before the *catch* block for a more general exception class.
- The best use of exceptions is to throw an exception in a method (but not catch it in the method)—but to do this only when the way the exception is handled will vary from one invocation of the method to another. There is seldom any other situation that can profitably benefit from throwing an exception.
- If an exception is thrown in a method but not caught in that method, then if the exception is not a descendent of the class `RuntimeException` (and is not a descendent of the class `Error`), the exception type must be listed in the `throws` clause for that method.

## Answers to Self-Test Exercises

1. Assuming the first item input is not a correctly formed `int` value, the program will go into an infinite loop after reading the first item input. The screen will continually output a prompt for an input number. The problem is that unless the new-line symbol `'\n'` is read, the program will continue to try to read on the first input line and so continually reads in the empty string.
2. The following is the method definition embedded in a test program. This program would give the same dialogue as the one in Display 9.1. The program is included on the website that accompanies this book.

extra code  
on website

```
import java.util.Scanner;
import java.util.InputMismatchException;
public class getIntDemo
{
 /**
 * Precondition: keyboard is an object of the class Scanner that
 * has been set up for keyboard input (as we have been doing right
 * along).
 * Returns: An int value entered at the keyboard.
 * If the user enters an incorrectly formed input, she or he
 * is prompted to reenter the value,
 */
 public static int getInt(Scanner keyboard)
 {
 int number = 0; //to keep compiler happy
 boolean done = false;

 while (! done)
 {
 try
 {
 System.out.println("Enter a whole number:");
 number = keyboard.nextInt();
 done = true;
 }
 catch(InputMismatchException e)
 {
 keyboard.nextLine();
 System.out.println(
 "Not a correctly written whole number.");
 System.out.println("Try again.");
 }
 }
 return number;
 }
}
```

```

public static void main(String[] args)
{
 Scanner keyboardArg = new Scanner(System.in);
 int number = getInt(keyboardArg);
 System.out.println("You entered " + number);
}
}

```

3. Try block entered.

Over 30.

After catch block

4. The output would then be

Try block entered.

Under 30.

After catch block

5. There are two throw statements:

```
throw new Exception("Over 30.");
```

```
throw new Exception("Under 30.");
```

6. When a throw statement is executed, it is the end of the enclosing try block. No other statements in the try block are executed, and control passes to the following catch block(s). When we say that control passes to the following catch block, we mean that the exception object that is thrown is plugged in for the catch block parameter and the code in the catch block is executed.

7. try

```

{
 System.out.println("Try block entered.");
 if (waitTime > 30)
 throw new Exception("Over 30.");
 else if (waitTime < 30)
 throw new Exception("Under 30.");
 else
 System.out.println("No exception.");
 System.out.println("Leaving try block.");
}

```

8. catch(Exception thrownObject)

```

{
 System.out.println(thrownObject.getMessage());
}

```

9. thrownObject

10. Yes, it is legal.

11. Yes, it is legal.

12. public class PowerFailureException extends Exception

- ```

{
    public PowerFailureException()
    {
        super("Power Failure!");
    }
    public PowerFailureException(String message)
    {
        super(message);
    }
}

```
13. `public class TooMuchStuffException extends Exception`
- ```

{
 public TooMuchStuffException()
 {
 super("Too much stuff!");
 }
 public TooMuchStuffException(String message)
 {
 super(message);
 }
}

```
14. ExerciseException invoked with an argument.  
Do Be Do
15. Test exception thrown!!  
Test Exception thrown!  
Message is Test Exception thrown!
16. try block entered:  
MyException: Hi Mom!  
End of example.
17. The output would be the same.
18. The output would then be  
try block entered:  
Leaving try block.  
End of example.
19. 41.9  
DoubleException thrown!
20. Yes, you can define an exception class as a derived class of the class  
IOException.
21. Second catch.  
End of exercise.

22. The output would then be

First catch.  
End of exercise.

23. The output would then be

Bingo!  
End of exercise.

24. Trying

Starting sampleMethod.  
Catching.  
End program.

25. The output would then be

Trying  
Starting sampleMethod.  
Trying after call.  
End program.

26. 

```
public static void doStuff(int n) throws Exception
{
 if (n < 0)
 throw new Exception("Negative number.");
}
```

27. If a method throws an exception and the exception is not caught inside the method, then the method invocation ends immediately after the exception is thrown. If the method invocation is inside a try block, then the exception is thrown to a matching catch block, if there is one. If there is no catch block matching the exception, then the method invocation ends as soon as that exception is thrown.

28. It might be caught in method B. If it is not caught in method B, it might be caught in method C. If it is not caught in method C, it might be caught outside of method C.

29. Yes, you can have a try block and corresponding catch blocks inside another larger try block.

30. Yes, you can have a try block and corresponding catch blocks inside another larger catch block.

31. In finally block.

Caught in main.  
If the argument to sampleMethod is -42 instead of 42, the output would be  
Caught in sampleMethod.  
In finally block.  
After finally block.

## Programming Projects



VideoNote  
Solution to  
Programming  
Project 9.1

1. Write a program that calculates the average of  $N$  integers. The program should prompt the user to enter the value for  $N$  and then afterward must enter all  $N$  numbers. If the user enters a nonpositive value for  $N$ , then an exception should be thrown (and caught) with the message “ $N$  must be positive.” If there is any exception as the user is entering the  $N$  numbers, an error message should be displayed, and the user prompted to enter the number again.
2. Define a class to maintain bank accounts of customers. The program should place the code into a `try-catch` block with multiple catches to check for the validity of various attributes based on the following criteria.
  - a. Customer ID must start with a letter and should be followed by three digits.
  - b. Account number must be of five digits.
  - c. Initial balance must be above \$1000.

Print suitable error matches within the catch block. If any of the criteria mentioned above is not fulfilled, the program should loop back and let the user enter new data.

3. Modify the previous exercise to include methods for amount deposited and amount withdrawn. Create your own `exception` class which will check inside the method for the amount deposited so that after the deposit, the maximum balance in the account must not be more than \$5000. Also, check inside the method for amount withdrawn so that the available balance after the withdrawal does not go below \$1000. Invoke the defined methods from your `main` method and catch the exceptions.
4. (This is a version of an exercise from Chapter 5) Programming Project 5.2 from Chapter 5 asked you to create a class named `Fraction`. This class is used to represent a ratio of two integers. It should include mutator functions that allow the user to set the numerator and the denominator along with a method that displays the fraction on the screen as a ratio (e.g., 5/9). Modify the class so that it throws the exception `DenominatorIsZeroException` if the denominator is set to zero. Do not forget to account for the constructors! You will have to create the `DenominatorIsZeroException` class and it should be derived from `Exception`. Write a `main` method that tests the new `Fraction` class, attempts to set the denominator to zero, and catches the `DenominatorIsZeroException` exception.
5. Write a program that converts dates from numerical month/day/year format to normal “month day, year” format (for example, 12/25/2000 corresponds to December 25, 2000). You will define three exception classes, one called `MonthException`, another called `DayException`, and a third called `YearException`. If the user enters



anything other than a legal month number (integers from 1 to 12), your program will throw and catch a `MonthException` and ask the user to reenter the month. Similarly, if the user enters anything other than a valid day number (integers from 1 to either 28, 29, 30, or 31, depending on the month and year), then your program will throw and catch a `DayException` and ask the user to reenter the day. If the user enters a year that is not in the range 1000 to 3000 (inclusive), then your program will throw and catch a `YearException` and ask the user to reenter the year. (There is nothing very special about the numbers 1000 and 3000 other than giving a good range of likely dates.) See Self-Test Exercise 19 in Chapter 4 for details on leap years.

6. Write a program that can serve as a simple calculator. This calculator keeps track of a single number (of type `double`) that is called `result` and that starts out as 0.0. Each cycle allows the user to repeatedly add, subtract, multiply, or divide by a second number. The result of one of these operations becomes the new value of `result`. The calculation ends when the user enters the letter R for “result” (either in upper- or lowercase). The user is allowed to do another calculation from the beginning as often as desired.

The input format is shown in the following sample dialogue. If the user enters any operator symbol other than +, -, \*, or /, then an `UnknownOperatorException` is thrown and the user is asked to reenter that line of input. Defining the class `UnknownOperatorException` is part of this project.

```
Calculator is on.
result = 0.0
+5
result + 5.0 = 5.0
new result = 5.0
* 2.2
result * 2.2 = 11.0
updated result = 11.0
% 10
% is an unknown operation.
Reenter, your last line:
* 0.1
result * 0.1 = 1.1
updated result = 1.1
r
Final result = 1.1
Again? (y/n)
yes
result = 0.0
+10
result + 10.0 = 10.0
new result = 10.0
/2
result / 2.0 = 5.0
```

```

updated result = 5.0
r
Final result = 5.0
Again? (y/n)
N
End of Program

```



VideoNote  
Solution to  
Programming  
Project 9.7

7. A method that returns a special error code is usually better accomplished throwing an exception instead. The following class maintains an account balance:

```

class Account
{
 private double balance;
 public Account()
 {
 balance = 0;
 }
 public Account(double initialDeposit)
 {
 balance = initialDeposit;
 }
 public double getBalance()
 {
 return balance;
 }
 // returns new balance or -1 if error
 public double deposit(double amount)
 {
 if (amount > 0)
 balance += amount;
 else
 return -1; // Code indicating error
 return balance;
 }
 // returns new balance or -1 if invalid amount
 public double withdraw(double amount)
 {
 if ((amount > balance) || (amount < 0))
 return -1;
 else
 balance -= amount;
 return balance;
 }
}

```

Rewrite the class so that it throws appropriate exceptions instead of returning `-1` as an error code. Write test code that attempts to withdraw and deposit invalid amounts and catches the exceptions that are thrown.

8. Study the class `java.util.Arrays` from the Oracle documentation located at <https://docs.oracle.com/javase/8/docs/api/java/util/Arrays.html>. The `sort` method throws an `IllegalArgumentException` and an `OutOfBoundsException`. Write a short Java test program that sorts an array of integers and outputs the array elements in sorted order. Your program should catch both of the exceptions listed above. Modify your program to test that the exceptions are properly caught.



# File I/O 10

## 10.1 INTRODUCTION TO FILE I/O 614

Streams 614

Text Files and Binary Files 615

## 10.2 TEXT FILES 616

Writing to a Text File 616

Appending to a Text File 623

Reading from a Text File 625

Reading a Text File Using `Scanner` 625

Testing for the End of a Text File with  
`Scanner` 628

Reading a Text File Using `BufferedReader` 635

Testing for the End of a Text File with  
`BufferedReader` 639

Path Names 641

Nested Constructor Invocations 642

`System.in`, `System.out`, and `System.err` 643

## 10.3 THE `File` CLASS 645

Programming with the `File` Class 645

## 10.4 BINARY FILES ★ 649

Writing Simple Data to a Binary File 650

UTF and `writeUTF` 654

Reading Simple Data from a Binary File 655

Checking for the End of a Binary File 660

Binary I/O of Objects 662

The `Serializable` Interface 663

Array Objects in Binary Files 666

## 10.5 RANDOM ACCESS TO BINARY FILES ★ 668

Reading and Writing to the Same File 668

*As a leaf is carried by a stream, whether the stream ends in a lake or in the sea, so too is the output of your program carried by a stream, not knowing if the stream goes to the screen or to a file.*

WASHROOM WALL OF A COMPUTER SCIENCE DEPARTMENT, 1995.

## Introduction

In this chapter, we explain how you can write your programs to take input from a file and send output to a file. This chapter covers the most common ways of doing file I/O in Java. However, it is not an exhaustive study of Java I/O classes. The Java I/O class library contains bewilderingly many classes, and an exhaustive treatment of all of them would be a book by itself.

## Prerequisites

You need only some of Chapter 9 on exception handling to read this chapter. You do not need Chapters 6, 7, or 8 on arrays, inheritance, and polymorphism, except in the final subsection, which covers writing and reading of arrays to binary files. If you have not yet covered some basic material on one-dimensional arrays, you can, of course, simply omit this last subsection.

You may postpone all or part of this chapter if you wish. Nothing in the rest of this book requires any of this chapter.

## 10.1 Introduction to File I/O

*Good Heavens! For more than forty years I have been speaking prose without knowing it.*

MOLIÈRE, *Le Bourgeois Gentilhomme*, 1670.

In this section, we go over some basic concepts about file I/O before we go into any Java details.

### Streams

A **stream** is an object that allows for the flow of data between your program and some I/O device or some file. If the flow is into your program, the stream is called an **input stream**. If the flow is out of your program, the stream is called an **output stream**. If the input stream flows from the keyboard, then your program will take input from the keyboard. If the input stream flows from a file, then your program will take its input from that file. Similarly, an output stream can go to the screen or to a file.

stream

input stream

output stream

`System.out``System.in`

Although you may not realize it, you have already been using streams in your programs when you have output something to the screen. `System.out` (used in `System.out.println`) is an output stream connected to the screen. `System.in` is an input stream connected to the keyboard. You used `System.in` in expressions such as the following:

```
Scanner keyboard = new Scanner(System.in);
```

These two streams are automatically available to your program. You can define other streams that come from or go to files. Once you have defined them, you can use them in your program in ways that are similar to how you use `System.out` and `System.in`.

### Streams

A **stream** is a flow of data. If the data flows *into your program*, then the stream is called an **input stream**. If the data flows *out of your program*, the stream is called an **output stream**.

Streams are used for both console I/O, which you have been using already, and file I/O.

## Text Files and Binary Files

`text file``ASCII file``binary file`

Text files are files that appear to contain sequences of characters when viewed in a text editor or read by a program. For example, the files that contain your Java programs are text files. **Text files** are sometimes also called **ASCII files** because they contain data encoded using a scheme known as ASCII coding. Files whose contents must be handled as sequences of binary digits are called **binary files**.

Although it is not technically correct, you can safely think of a text file as containing a sequence of characters, and think of a binary file as containing a sequence of binary digits. Another way to distinguish between binary files and text files is to note that text files are designed to be read by human beings, whereas binary files are designed to be read only by programs.

One advantage of text files is that they are usually the same on all computers, so you can move your text files from one computer to another with few or no problems. The implementation of binary files usually differs from one computer to another, so your binary data files ordinarily must be read only on the same type of computer, and with the same programming language, as the computer that created that file.

The benefit of binary files is that they are more efficient to process than text files. Unlike other programming languages, Java also gives its binary files some of the advantages of text files. In particular, Java binary files are platform independent; that is, with Java, you can move your binary files from one type of computer to another, and your Java programs will still be able to read the binary files. This combines the portability of text files with the efficiency of binary files.

The one big asset of text files is that you can read and write to them using a text editor. With binary files, all the reading and writing must normally be done by a program.

### Text Files versus Binary Files

Files that you write and read using an editor are called **text files**. **Binary files** represent data in a way that is not convenient to read with a text editor, but that can be written to and read from a program very efficiently.

### Self-Test Exercises

1. A stream is a flow of data. From where and to where does the data flow in an input stream? From where and to where does the data flow in an output stream?
2. What is the difference between a binary file and a text file?

## 10.2 Text Files

*Polonius: What do you read, my lord?*

*Hamlet: Words, words, words.*

WILLIAM SHAKESPEARE, *Hamlet*, 1603.

In this section, we describe the most common ways to do text file I/O in Java.

### Writing to a Text File

#### PrintWriter

The class `PrintWriter` is the preferred stream class for writing to a text file. An object of the class `PrintWriter` has the methods `print` and `println`, which are like the methods `System.out.print` and `System.out.println` that you can use for screen output. However, with an object of the class `PrintWriter`, the output goes to a text file. Display 10.1 contains a simple program that uses `PrintWriter` to send output to a text file. Let's look at the details of that program.

#### java.io

All the file I/O–related classes we introduce in this chapter are in the package `java.io`, so all our program files begin with import statements similar to the ones in Display 10.1.

The program in Display 10.1 creates a text file named `stuff.txt` that a person can read using an editor, or that another Java program can read. The program creates an object of the class `PrintWriter` as follows:

```
outputStream =
 new PrintWriter(new FileOutputStream("stuff.txt"));
```

#### opening a file

The variable `outputStream` is of type `PrintWriter` and is declared outside the `try` block. The preceding two lines of code connect the stream named `outputStream` to the file named `stuff.txt`. This is called **opening the file**. When you connect a file to a stream in this way, your program always starts with an empty file. If the file `stuff.txt`

already exists, the old contents of `stuff.txt` will be lost. If the file `stuff.txt` does not exist, then a new, empty file named `stuff.txt` will be created.

**FileOutput  
Stream**

We want to associate the output stream `outputStream` with the file named `stuff.txt`. However, the class `PrintWriter` has no constructor that takes a file name as its argument. So we use the class `FileOutputStream` to create a stream that can be used as an argument to a `PrintWriter` constructor. The expression

```
new FileOutputStream("stuff.txt")
```

takes a file name as an argument and creates an anonymous object of the class `FileOutputStream`, which is then used as an argument to a constructor for the class `PrintWriter` as follows:

```
new PrintWriter(new FileOutputStream("stuff.txt"))
```

**file name**

This produces an object of the class `PrintWriter` that is connected to the file `stuff.txt`. Note that the name of the file, in this case, `stuff.txt`, is given as a `String` value and so is given in quotes.

**reading the  
file name**

If you want to read the file name from the keyboard, you could read the name to a variable of type `String` and use the `String` variable as the argument to the `FileOutputStream` constructor.

**FileNot  
Found  
Exception**

When you open a text file in the way just discussed, a `FileNotFoundException` can be thrown, and any such possible exception should be caught in a `catch` block. (Actually, it is the `FileOutputStream` constructor that might throw the `FileNotFoundException`, but the net effect is the same.)

Notice that the `try` block in Display 10.1 encloses only the opening of the file. That is the only place that an exception might be thrown. Also note that the variable `outputStream` is declared outside of the `try` block—this is so that this variable can be used outside of the `try` block. Remember, anything declared in a block (even in a `try` block) is local to the block.

Display 10.1 Sending Output to a Text File (part 1 of 2)

```
1 import java.io.PrintWriter;
2 import java.io.FileOutputStream;
3 import java.io.FileNotFoundException;

4 public class TextFileOutputDemo
5 {
6 public static void main(String[] args)
7 {
8 PrintWriter outputStream = null;
9 try
10 {
11 outputStream =
12 new PrintWriter(new FileOutputStream("stuff.txt"));
13 }
14 catch (FileNotFoundException e)
```

(continued)



Display 10.1 Sending Output to a Text File (part 2 of 2)

```

15 {
16 System.out.println("Error opening the file stuff.txt.");
17 System.exit(0);
18 }

19 System.out.println("Writing to file.");

20 outputStream.println("The quick brown fox");
21 outputStream.println("jumps over the lazy dog.");

22 outputStream.close();

23 System.out.println("End of program.");
24 }
25 }

```

## Sample Dialogue

```

Writing to file.
End of program.

```

FILE stuff.txt (after the program is run.)

```

The quick brown fox
jumps over the lazy dog.

```

*You can read this file  
using a text editor.*

## Opening a Text File for Writing Output

You create a stream of the class `PrintWriter` and connect it to a text file for writing as follows.

### SYNTAX

```

PrintWriter Output_Stream_Name;
Output_Stream_Name =
 new PrintWriter(new FileOutputStream(File_Name));

```

### EXAMPLE

```

PrintWriter outputStream = null;
outputStream =
 new PrintWriter(new FileOutputStream("stuff.txt"));

```

After this, you can use the methods `println` and `print` to write to the file.

When used in this way, the `FileOutputStream` constructor, and thus the `PrintWriter` constructor invocation, can throw a `FileNotFoundException`, which is a kind of `IOException`.

## File Names

The rules for how you spell file names depend on your operating system, not on Java. When you give a file name to a Java constructor for a stream, you are not giving the constructor a Java identifier. You are giving the constructor a string corresponding to the file name. A suffix, such as `.txt` in `stuff.txt`, has no special meaning to a Java program. We are using the suffix `.txt` to indicate a text file, but that is just a common convention. You can use any file names that are allowed by your operating system.

## A File Has Two Names

Every input file and every output file used by your program has two names: (1) the real file name that is used by the operating system and (2) the name of the stream that is connected to the file.

The stream name serves as a temporary name for the file and is the name that is primarily used within your program. After you connect the file to the stream, your program always refers to the file by using the stream name.

We said that when you open a text file for writing output to the file, the constructor might throw a `FileNotFoundException`. But in this situation you want to create a new file for output, so why would you care that the file was not found? The answer is simply that the exception is poorly named. A `FileNotFoundException` does not mean that the file was not found. In this case, it actually means that the file could not be created. A `FileNotFoundException` is thrown if it is impossible to create the file—for example, because the file name is already used for a directory (folder) name.

## IOException

When dealing with file I/O, there are many situations in which your code might throw an exception of some class, such as `FileNotFoundException`. Many of these various exception classes are descended classes of the class `IOException`. The class `IOException` is the root class for various exception classes having to do with input and output.

A `FileNotFoundException` is a kind of `IOException`, so a catch block for an `IOException` would also work and would look more sensible. However, it is best to catch the most specific exception that you can, because that can give more information.

As illustrated in Display 10.1, the method `println` of the class `PrintWriter` works the same for writing to a text file as the method `System.out.println` works for writing to the screen. The class `PrintWriter` also has the methods `print` and

`println`

`print`

**printf**     `printf`, which behave just like `System.out.print` and `System.out.printf` except that the output goes to a text file. Display 10.2 describes some of the methods in the class `PrintWriter`.

Display 10.2 Some Methods of the Class `PrintWriter` (part 1 of 2)

`PrintWriter` and `FileOutputStream` are in the `java.io` package.

```
public PrintWriter(OutputStream streamObject)
```

This is the only constructor you are likely to need. There is no constructor that accepts a file name as an argument. If you want to create a stream using a file name, use

```
new PrintWriter(new FileOutputStream(File_Name))
```

When the constructor is used in this way, a blank file is created. If there already is a file named *File\_Name*, then the old contents of the file are lost. If you want instead to append new text to the end of the old file contents, use

```
new PrintWriter(new FileOutputStream(File_Name, true))
```

(For an explanation of the argument `true`, read the later subsection “Appending to a Text File.”)

When used in either of these ways, the `FileOutputStream` constructor, and so the `PrintWriter` constructor invocation, can throw a `FileNotFoundException`, which is a kind of `IOException`.

If you want to create a stream using an object of the class `File`, you can use a `File` object in place of the *File\_Name*. (The `File` class will be covered later in Section 10.3. We discuss it here so that you will have a more complete reference in this display, but you can ignore the reference to the class `File` until after you have read that section.)

```
public void println(Argument)
```

The *Argument* can be a string, character, integer, floating-point number, boolean value, or any combination of these, connected with + signs. The *Argument* can also be any object, although it will not work as desired unless the object has a properly defined `toString()` method. The *Argument* is output to the file connected to the stream. After the *Argument* has been output, the line ends, and so the next output is sent to the next line.

```
public void print(Argument)
```

This is the same as `println`, except that this method does not end the line, so the next output will be on the same line.

```
public PrintWriter printf(Argument)
```

This is the same as `System.out.printf`, except that this method sends output to a text file rather than to the screen. It returns the calling object. However, we have always used `printf` as a void method.

---

Display 10.2 Some Methods of the Class `PrintWriter` (part 2 of 2)

---

```
public void close()
```

Closes the stream's connection to a file. The following method calls `flush` before closing the file:

```
public void flush()
```

Flushes the output stream. This forces an actual physical write to the file of any data that has been buffered and not yet physically written to the file. Normally, you should not need to invoke `flush`.

---

When your program is finished writing to a file, it should **close** the stream connected to that file. In Display 10.1, the stream connected to the file `stuff.txt` is closed with the statement

```
outputStream.close();
```

The class `PrintWriter`, and every other class for file output or file input streams, has a method named `close`. When this method is invoked, the system releases any resources used to connect the stream to the file and does any other housekeeping that is needed. If your program does not close a file before the program ends, Java will close it for you when the program ends, but it is safest to close the file with an explicit call to `close`.

buffered

buffer

Output streams connected to files are often **buffered**, which means that, rather than physically writing every instance of output data as soon as possible, the data is saved in a temporary location, known as a **buffer**; when enough data is accumulated in this temporary location, it is physically written to the file. This can add to efficiency, since physical writes to a file can be slow. The method `flush` causes a physical write to the file of any buffered data. The method `close` includes an invocation of the method `flush`.

### Closing a File

When your program is finished writing to a file or reading from a file, it should close the stream connected to that file by invoking the method named `close`.

#### SYNTAX

```
Stream_Object.close();
```

#### EXAMPLE

```
outputStream.close();
inputStream.close();
```

It may seem like there is no reason to use the method `close` to close a file. If your program ends normally but without closing a file, the system will automatically close it for you. So why should you bother to close files with an explicit call to the method `close`? There are at least two reasons. First, if your program ends abnormally, then

Java may not be able to close the file for you. This could damage the file. In particular, if it is an output file, any buffered output will not have been physically written to the file. So, the file will be incomplete. The sooner you close a file, the less likely it is that this will happen. Second, if your program writes to a file and later reads from the same file, it must close the file after it is through writing to the file and then reopen the file for reading. (Java does have a class that allows a file to be opened for both reading and writing, which we will discuss later in Section 10.5.)



### PITFALL: A try Block Is a Block

Notice that in Display 10.1, we declare the variable `outputStream` outside of the `try` block. If you were to move that declaration inside the `try` block, you would get a compiler error message. Let's see why.

Suppose you replace

```
PrintWriter outputStream = null;
try
{
 outputStream =
 new PrintWriter(new FileOutputStream("stuff.txt"));
}
```

in Display 10.1 with the following:

```
try
{
 PrintWriter outputStream =
 new PrintWriter(new FileOutputStream("stuff.txt"));
}
```

This replacement looks innocent enough, but it makes the variable `outputStream` a local variable for the `try` block, which would mean that you could not use `outputStream` outside of the `try` block. If you make this change and try to compile the changed program, you will get an error message saying that `outputStream`, when used outside the `try` block, is an undefined identifier. ■



### PITFALL: Overwriting an Output File

When you connect a stream to a text file for writing to the text file, as illustrated by what follows, you always produce an empty file:

```
outputStream =
 new PrintWriter(new FileOutputStream("stuff.txt"));
```

If there is no file named `stuff.txt`, this will create an empty file named `stuff.txt`. If a file named `stuff.txt` already exists, then this will eliminate that file and create a new, empty file named `stuff.txt`. So if there is a file named `stuff.txt` before this file opening, then all the data in that file will be lost. The later section



## PITFALL: (continued)

“The `File` Class” tells you how to test to see whether a file already exists so that you can avoid accidentally overwriting a file. The following subsection, “Appending to a Text File,” shows you how to add data to a text file without losing the data that is already in the file. ■

### appending

## Appending to a Text File

When you open a text file for writing in the way we did it in Display 10.1 and a file with the given name already exists, the old contents are lost. However, sometimes you instead want to add the program output to the end of the file. This is called **appending to a file**. If you want to append program output to the file `stuff.txt`, connect the file to the stream `outputStream` in the following manner:

```
outputStream =
 new PrintWriter(new FileOutputStream("stuff.txt", true));
```

If the file `stuff.txt` does not already exist, Java will create an empty file of that name and append the output to the end of this empty file. So if there is no file named `stuff.txt`, the effect of opening the file is the same as in Display 10.1. However, if the file `stuff.txt` already exists, then the old contents will remain, and the program’s output will be placed after the old contents of the file.

When appending to a text file in this way, you would still use the same `try` and `catch` blocks as in Display 10.1.

That second argument of `true` deserves a bit of explanation. Why did the designers use `true` to signal appending? Why not something such as the string “append”? The reason is that this version of the constructor for the class `FileOutputStream` was designed to also allow you to use a Boolean variable (or expression) to decide whether you append to an existing file or create a new file. For example, the following might be used:

```
System.out.println(
 "Enter A for append or N for a new file:");
char answer;
<Use your favorite way to read a single character into the variable answer.>
boolean append = (answer == 'A' || answer == 'a');
outputStream = new PrintWriter(
 new FileOutputStream("stuff.txt", append));
```

From this point on, your program writes to the file in exactly the same way that the program in Display 10.1 does. If the user answers with upper- or lowercase A, then any input will be added after the old file contents. If the user answers with upper- or lowercase N (or with anything other than an A), then any old contents of the file are lost.



### TIP: toString Helps with Text File Output

In Chapter 4, we noted that if a class has a suitable `toString()` method and an object is an object of that class, then an object can be used as an argument to `System.out.println`, which will produce sensible output.<sup>1</sup> The same thing applies to the methods `println` and `print` of the class `PrintWriter`. Both `println` and `print` of the class `PrintWriter` can take any object as an argument and will produce reasonable output so long as the object has a sensible `toString()` method. ■

### Opening a Text File for Appending

To create an object of the class `PrintWriter` and connect it to a text file for appending to the end of the text already in the file, proceed as follows.

#### SYNTAX

```
Output_Stream_Name =
 new PrintWriter(
 new FileOutputStream(File_Name, True_Boolean_Expression));
```

#### EXAMPLE

```
PrintWriter outputStream;
outputStream =
 new PrintWriter(new FileOutputStream("stuff.txt", true));
```

After this statement, you can use the methods `println` and `print` to write to the file, and the new text will be written after the old text in the file.

(If you want to create a stream using an object of the class `File`, you can use a `File` object in place of the `File_Name`. The `File` class is discussed later in the section entitled "The File Class.")

When used in this way, the `FileOutputStream` constructor, and so the `PrintWriter` constructor invocation, can throw a `FileNotFoundException`, which is a kind of `IOException`.

### Self-Test Exercises

3. What kind of exception might be thrown by the following, and what would it indicate if this exception is thrown?

```
PrintWriter outputStream =
 new PrintWriter(new FileOutputStream("stuff.txt"));
```

<sup>1</sup>There is a more detailed discussion of this in Chapter 8, but you need not read Chapter 8 to use this fact.

### Self-Test Exercises (continued)

4. Does the class `PrintWriter` have a constructor that accepts a string (for a file name) as an argument, so that the following code would be legal?

```
PrintWriter outputStream =
 new PrintWriter("stuff.txt");
```

5. Write some code that will create a stream named `outStream` that is a member of the class `PrintWriter`, and that connects this stream to a text file named `sam` so that your program can send output to the file. Do this so that the file `sam` always starts out empty. So, if there already is a file named `sam`, the old contents of `sam` are lost.
6. As in Self-Test Exercise 5, write some code that will create a stream named `outStream` that is a member of the class `PrintWriter`, and that connects this stream to a text file named `sam` so that your program can send output to the file. This time, however, do it in such a way that, if the file `sam` already exists, the old contents of `sam` will not be lost, and the program output will be written after the old contents of the file.
7. The class `Person` was defined in Display 5.19 of Chapter 5. Suppose `mary` is an object of the class `Person`, which has a `toString` method defined, and suppose `outputStream` is connected to a text file as in Display 10.1. Will the following send sensible output to the file connected to `outputStream`?

```
outputStream.println(mary);
```

## Reading from a Text File



The two most common stream classes used for reading from a text file are the `Scanner` class and the `BufferedReader` class. We will discuss both of these approaches to reading from a text file. The `Scanner` class offers a richer group of methods for reading from a text file and is our preferred class to use when reading from a text file. However, the `BufferedReader` class is also widely used and is a reasonable choice. You, or your instructor, will need to decide which class you will use.

## Reading a Text File Using `Scanner`

The same `Scanner` class that we used for reading from the keyboard can also be used for reading from a text file. To do so, replace the argument `System.in` (in the `Scanner` constructor) with a suitable stream that is connected to the text file. This is a good illustration of the notion of a *stream*. The class `Scanner` does not care if its stream argument comes from the keyboard or from a text file.

The use of `Scanner` for reading from a text file is illustrated in Display 10.3, which contains a program that reads three numbers and a line of text from a text file named `morestuff.txt` and writes them back to the screen. The file `morestuff.txt` is a text file that a person could have created with a text editor or that a Java program could have created using `PrintWriter`.



**opening a file**

The program opens the `Scanner` stream and connects it to the text file `morestuff.txt` as follows:

```
Scanner inputStream = null;
...
inputStream = new Scanner(new FileInputStream("stuff.txt"));
```

The class `Scanner`, like the class `PrintWriter`, has no constructor that takes a file name as its argument, so we need to use another class—in this case, the class `FileInputStream`—to convert the file name to an object that can be a suitable argument to the `Scanner` constructor.

Note that the methods `nextInt` and `nextLine` read from the text files in exactly the same way as they read from the keyboard. The other `Scanner` methods for reading input (given in Display 2.6 and repeated in Display 10.6) also behave the same when reading from a text file as they do when used to read from the keyboard.

### Opening a Text File for Reading with `Scanner`

Create a stream of the class `Scanner` and connect it to a text file for reading as follows:

#### SYNTAX

```
Scanner Stream_Object =
 new Scanner(new FileInputStream(File_Name));
```

#### EXAMPLE

```
Scanner inputStream =
 new Scanner(new FileInputStream("morestuff.txt"));
```

After this statement, you can use the methods `nextInt`, `nextDouble`, `nextLine`, and so forth to read from the named text files just as you have used these methods to read from the keyboard.

When used in this way, the `FileInputStream` constructor, and hence the `Scanner` constructor invocation, can throw a `FileNotFoundException`, which is a kind of `IOException`.

### **`FileNotFoundException`**

If your program attempts to open a file for reading and there is no such file, then a `FileNotFoundException` is thrown. As you saw earlier in this chapter, a `FileNotFoundException` is also thrown in some other situations. A `FileNotFoundException` is a kind of `IOException`.

## Display 10.3 Reading Input from a Text File Using Scanner (part 1 of 2)

```
1 import java.util.Scanner;
2 import java.io.FileInputStream;
3 import java.io.FileNotFoundException;
4
5 public class TextFileScannerDemo
6 {
7 public static void main(String[] args)
8 {
9 System.out.println("I will read three numbers and a line");
10 System.out.println("of text from the file morestuff.txt.");
11
12 Scanner inputStream = null;
13
14 try
15 {
16 inputStream =
17 new Scanner(new FileInputStream("morestuff.txt"));
18 }
19 catch (FileNotFoundException e)
20 {
21 System.out.println("File morestuff.txt was not found");
22 System.out.println("or could not be opened.");
23 System.exit(0);
24 }
25 int n1 = inputStream.nextInt();
26 int n2 = inputStream.nextInt();
27 int n3 = inputStream.nextInt();
28
29 inputStream.nextLine(); //To go to the next line
30
31 String line = inputStream.nextLine();
32
33 System.out.println("The three numbers read from the file are:");
34 System.out.println(n1 + ", " + n2 + ", and " + n3);
35
36 System.out.println("The line read from the file is:");
37 System.out.println(line);
38
39 inputStream.close();
40 }
41 }
```

FILE morestuff.txt

```
1 2
3 4
Eat my shorts.
```

*This file could have been made with a  
text editor or by another Java program.*

(continued)

Display 10.3 Reading Input from a Text File Using `Scanner` (part 2 of 2)

## Screen Output

```
I will read three numbers and a line
of text from the file morestuff.txt.
The three numbers read from the file are:
1, 2, and 3
The line read from the file is:
Eat my shorts.
```

### Testing for the End of a Text File with `Scanner`

When using the class `Scanner`, if your program tries to read beyond the end of the file with any of the input methods, such as `nextInt` or `nextLine`, then the method throws an exception. If all goes well and there are no problems, such as using `nextInt` when the input is not a correctly formed `int`, then the exception thrown will be `NoSuchElementException`. This throwing of a `NoSuchElementException` can be used to signal the end of input. However, there is a more robust way of testing for the end of input from a text file. Each of the input methods (such as `nextInt` and `nextLine`) has a corresponding method (such as `hasNextInt` and `hasNextLine`) that checks to see if there is any more well-formed input of the appropriate type. The nice thing about these methods is that they report when there is not a suitable next token for any reason; they do not check only for the end of a file. For example, `hasNextInt` returns `false` if there is no more file input of any kind or if the next token is not a well-formed `int` value. It returns `true` if there is a well-formed `int` as the next token.

A sample program that illustrates the use of `hasNextLine` to test for the end of input is given in Display 10.4. A sample program that illustrates the use of `hasNextInt` to test for the end of input is given in Display 10.5. A summary of some of the methods in the `Scanner` class is given in Display 10.6.

#### Checking for the End of a Text File with `Scanner`

You can check for the end of input with methods such as `hasNextInt`, `hasNextLine`, and so forth.

Display 10.4 Checking for the End of a Text File with `hasNextLine` (part 1 of 2)

```
1 import java.util.Scanner;
2 import java.io.FileInputStream;
3 import java.io.FileNotFoundException;
4 import java.io.PrintWriter;
5 import java.io.FileOutputStream;
6
7 public class HasNextLineDemo
8 {
9 public static void main(String[] args)
10 {
11 Scanner inputStream = null;
12 PrintWriter outputStream = null;
13
14 try
15 {
16 inputStream =
17 new Scanner(new FileInputStream("original.txt"));
18 outputStream = new PrintWriter(
19 new FileOutputStream("numbered.txt"));
20 }
21 catch(FileNotFoundException e)
22 {
23 System.out.println("Problem opening files.");
24 System.exit(0);
25 }
26
27 String line = null;
28 int count = 0;
29 while (inputStream.hasNextLine())
30 {
31 line = inputStream.nextLine();
32 count++;
33 outputStream.println(count + " " + line);
34 }
35
36 inputStream.close();
37 outputStream.close();
38 }
39 }
```

File original.txt

```
Little Miss Muffet
sat on a tuffet
eating her curves away.
Along came a spider
who sat down beside her
and said "Will you marry me?"
```

(continued)

Display 10.4 Checking for the End of a Text File with `hasNextLine` (part 2 of 2)

File numbered.txt (after the program is run)

```
1 Little Miss Muffet
2 sat on a tuffet
3 eating her curves away.
4 Along came a spider
5 who sat down beside her
6 and said "Will you marry me?"
```

Display 10.5 Checking for the End of a Text File with `hasNextInt`

```
1 import java.util.Scanner;
2 import java.io.FileInputStream;
3 import java.io.FileNotFoundException;

4 public class HasNextIntDemo
5 {
6 public static void main(String[] args)
7 {
8 Scanner inputStream = null;

9 try
10 {
11 inputStream =
12 new Scanner(new FileInputStream("data.txt"));
13 }
14 catch(FileNotFoundException e)
15 {
16 System.out.println("File data.txt was not found");
17 System.out.println("or could not be opened.");
18 System.exit(0);
19 }

20 int next, sum = 0;
21 while (inputStream.hasNextInt())
22 {
23 next = inputStream.nextInt();
24 sum = sum + next;
25 }

26 inputStream.close();

27 System.out.println("The sum of the numbers is " + sum);
28 }
29 }
```

File data.txt

```
1 2
3 4 hi 5
```

*Reading ends when either the end of the file is reached or a token that is not an int is reached. So, the 5 is never read.*

## Screen Output

The sum of the numbers is 10

Display 10.6 Methods in the Class `Scanner` (part 1 of 4)

`Scanner` is in the `java.util` package.

```
public Scanner(InputStream streamObject)
```

There is no constructor that accepts a file name as an argument. If you want to create a stream using a file name, you can use

```
new Scanner(new FileInputStream(File_Name))
```

When used in this way, the `FileInputStream` constructor, and thus the `Scanner` constructor invocation, can throw a `FileNotFoundException`, which is a kind of `IOException`.

To create a stream connected to the keyboard, use

```
new Scanner(System.in)
```

```
public Scanner(File fileObject)
```

The `File` class will be covered in the section entitled “The `File` Class,” later in this chapter. We discuss it here so that you will have a more complete reference in this display, but you can ignore this entry until after you have read that section.

If you want to create a stream using a file name, you can use

```
new Scanner(new File(File_Name))
```

```
public int nextInt()
```

Returns the next token as an `int`, provided the next token is a well-formed string representation of an `int`.

Throws a `NoSuchElementException` if there are no more tokens.

Throws an `InputMismatchException` if the next token is not a well-formed string representation of an `int`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNextInt()
```

Returns `true` if the next token is a well-formed string representation of an `int`; otherwise returns `false`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public long nextLong()
```

Returns the next token as a `long`, provided the next token is a well-formed string representation of a `long`.

Throws a `NoSuchElementException` if there are no more tokens.

Throws an `InputMismatchException` if the next token is not a well-formed string representation of a `long`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

(continued)

Display 10.6 Methods in the Class `Scanner` (part 2 of 4)

```
public boolean hasNextLong()
```

Returns `true` if the next token is a well-formed string representation of a `long`; otherwise returns `false`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public byte nextByte()
```

Returns the next token as a `byte`, provided the next token is a well-formed string representation of a `byte`.

Throws a `NoSuchElementException` if there are no more tokens.

Throws an `InputMismatchException` if the next token is not a well-formed string representation of a `byte`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNextByte()
```

Returns `true` if the next token is a well-formed string representation of a `byte`; otherwise returns `false`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public short nextShort()
```

Returns the next token as a `short`, provided the next token is a well-formed string representation of a `short`.

Throws a `NoSuchElementException` if there are no more tokens.

Throws an `InputMismatchException` if the next token is not a well-formed string representation of a `short`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNextShort()
```

Returns `true` if the next token is a well-formed string representation of a `short`; otherwise returns `false`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public double nextDouble()
```

Returns the next token as a `double`, provided the next token is a well-formed string representation of a `double`.

Throws a `NoSuchElementException` if there are no more tokens.

Throws an `InputMismatchException` if the next token is not a well-formed string representation of a `double`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

Display 10.6 Methods in the Class `Scanner` (part 3 of 4)

```
public boolean hasNextDouble()
```

Returns `true` if the next token is a well-formed string representation of a `double`; otherwise returns `false`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public float nextFloat()
```

Returns the next token as a `float`, provided the next token is a well-formed string representation of a `float`.

Throws a `NoSuchElementException` if there are no more tokens.

Throws an `InputMismatchException` if the next token is not a well-formed string representation of a `float`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNextFloat()
```

Returns `true` if the next token is a well-formed string representation of a `float`; otherwise returns `false`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public String next()
```

Returns the next token.

Throws a `NoSuchElementException` if there are no more tokens.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNext()
```

Returns `true` if there is another token. May wait for a next token to enter the stream.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public boolean nextBoolean()
```

Returns the next token as a `boolean` value, provided the next token is a well-formed string representation of a `boolean`.

Throws a `NoSuchElementException` if there are no more tokens.

Throws an `InputMismatchException` if the next token is not a well-formed string representation of a `boolean` value.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNextBoolean()
```

Returns `true` if the next token is a well-formed string representation of a `boolean` value; otherwise returns `false`.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

(continued)



Display 10.6 Methods in the Class `Scanner` (part 4 of 4)

```
public String nextLine()
```

Returns the rest of the current input line. Note that the line terminator '`\n`' is read and discarded; it is not included in the string returned.

Throws a `NoSuchElementException` if there are no more lines.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNextLine()
```

Returns `true` if there is a next line. May wait for a next line to enter the stream.

Throws an `IllegalStateException` if the `Scanner` stream is closed.

```
public Scanner useDelimiter(String newDelimiter);
```

Changes the delimiter for input so that `newDelimiter` will be the only delimiter that separates words or numbers. See the subsection “Other Input Delimiters” in Chapter 2 for the details. (You can use this method to set the delimiters to a more complex pattern than just a single string, but we are not covering that.)

Returns the calling object, but we have always used it as a `void` method.

### Unchecked Exceptions

The exception classes `NoSuchElementException`, `InputMismatchException`, and `IllegalStateException` are all unchecked exceptions, which means that an exception of any of these classes is not required to be caught or declared in a `throws` clause.

### Self-Test Exercises

8. Write some code that will create a stream named `fileIn` that is a member of the class `Scanner`. It should connect the stream to a text file named `sally` so that your program can read input from the text file `sally`.
9. Might the method `nextInt` in the class `Scanner` throw an exception? If so, what type of exception?
10. If the method `hasNextInt` returns `false`, does that mean that reading has reached the end of the file?

## Self-Test Exercises (continued)

11. Might the following throw an exception that needs to be caught or declared in a throws clause?

```
Scanner inputStream =
 new Scanner(new FileInputStream("morestuff.txt"));
```

(The stream inputStream would be used to read from the text file morestuff.txt.)

## Reading a Text File Using `BufferedReader`

`BufferedReader`

opening a file

Until the `Scanner` class was introduced with version 5.0 of Java, the class `BufferedReader` was the preferred stream class to use for reading from a text file. It is still a commonly used class for reading from a text file. The use of `BufferedReader` is illustrated in Display 10.7, which contains a program that reads two lines from a text file named `morestuff2.txt` and writes them back to the screen. The file `morestuff2.txt` is a text file that a person could have created with a text editor or that a Java program could have created using `PrintWriter`.

The program opens the text file `morestuff2.txt` as follows:

```
BufferedReader inputStream =
 new BufferedReader(new FileReader("morestuff2.txt"));
```

The class `BufferedReader`, like the classes `PrintWriter` and `Scanner`, has no constructor that takes a file name as its argument, so we need to use another class—in this case, the class `FileReader`—to convert the file name to an object that can be an argument to `BufferedReader`.

`readLine`

An object of the class `BufferedReader` that is connected to a text file, as in Display 10.7, has a method named `readLine` that is like the method `nextLine` of the `Scanner` class. This use of `readLine` to read from a text file is illustrated in Display 10.7.

Display 10.8 describes some of the methods in the class `BufferedReader`. Notice that there are only two methods for reading from a text file, `readLine` and `read`. We have already discussed `readLine`.

Display 10.7 Reading Input from a Text File Using `BufferedReader` (part 1 of 2)

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.FileNotFoundException;
4 import java.io.IOException;

5 public class TextFileInputDemo
6 {
7 public static void main(String[] args)
8 {
```

(continued)

Display 10.7 Reading Input from a Text File Using `BufferedReader` (part 2 of 2)

```
9 try
10 {
11 BufferedReader inputStream =
12 new BufferedReader(new FileReader("morestuff2.txt"));

13 String line = inputStream.readLine();
14 System.out.println(
15 "The first line read from the file is:");
16 System.out.println(line);
17
18 line = inputStream.readLine();
19 System.out.println(
20 "The second line read from the file is:");
21 System.out.println(line);
22 inputStream.close();
23 }
24 catch(FileNotFoundException e)
25 {
26 System.out.println("File morestuff2.txt was not found");
27 System.out.println("or could not be opened.");
28 }
29 catch(IOException e)
30 {
31 System.out.println("Error reading from morestuff2.txt.");
32 }
33 }
34 }
```

FILE morestuff2.txt

```
1 2 3
Jack jump over
the candle stick.
```

*This file could have been made with a  
text editor or by another Java program.*

#### Screen Output

```
The first line read from the file is:
1 2 3
The second line read from the file is:
Jack jump over
```

### Opening a Text File for Reading with `BufferedReader`

Create a stream of the class `BufferedReader` and connect it to a text file for reading as follows:

#### SYNTAX

```
BufferedReader Stream_Object =
 new BufferedReader(new FileReader(File_Name));
```

#### EXAMPLE

```
BufferedReader inputStream =
 new BufferedReader(new FileReader("morestuff2.txt"));
```

After this statement, you can use the methods `readLine` and `read` to read from the file. When used in this way, the `FileReader` constructor, and hence the `BufferedReader` constructor invocation, can throw a `FileNotFoundException`, which is a kind of `IOException`.

## Display 10.8 Some Methods of the Class `BufferedReader` (part 1 of 2)

`BufferedReader` and `FileReader` are in the `java.io` package.

```
public BufferedReader(Reader readerObject)
```

This is the only constructor you are likely to need. There is no constructor that accepts a file name as an argument. If you want to create a stream using a file name, use

```
new BufferedReader(new FileReader(File_Name))
```

When used in this way, the `FileReader` constructor, and thus the `BufferedReader` constructor invocation, can throw a `FileNotFoundException`, which is a kind of `IOException`.

The `File` class will be covered later in the section entitled "The `File` Class." We discuss it here so that you will have a more complete reference in this display, but you can ignore the following reference to the class `File` until after you have read that section.

If you want to create a stream using an object of the class `File`, use

```
new BufferedReader(new FileReader(File_Object))
```

When used in this way, the `FileReader` constructor, and thus the `BufferedReader` constructor invocation, can throw a `FileNotFoundException`, which is a kind of `IOException`.

```
public String readLine() throws IOException
```

Reads a line of input from the input stream and returns that line. If the read goes beyond the end of the file, `null` is returned. (Note that an `EOFException` is not thrown at the end of a file. The end of a file is signaled by returning `null`.)

(continued)

Display 10.8 Some Methods of the Class `BufferedReader` (part 2 of 2)

```
public int read() throws IOException
```

Reads a single character from the input stream and returns that character as an `int` value. If the read goes beyond the end of the file, then `-1` is returned. Note that the value is returned as an `int`. To obtain a `char`, you must perform a type cast on the value returned. The end of a file is signaled by returning `-1`. (All of the “real” characters return a positive integer.)

```
public long skip(long n) throws IOException
```

Skips `n` characters.

```
public void close() throws IOException
```

Closes the stream’s connection to a file.

**read method**

The method `read` reads a single character. But note that `read` returns a value of type `int` that corresponds to the character read; it does not return the character itself. Thus, to get the character, you must use a type cast, as in

```
char next = (char) (inputStream.read());
```

If `inputStream` is in the class `BufferedReader` and is connected to a text file, this will set `next` equal to the first character in the file that has not yet been read.

Notice that the program in Display 10.7 catches two kinds of exceptions, `FileNotFoundException` and `IOException`. An attempt to open the file may throw a `FileNotFoundException`, and any of the invocations of `inputStream.readLine()` may throw an `IOException`. Because `FileNotFoundException` is a kind of `IOException`, you could use only the catch block for `IOException`. However, if you were to do this, then you would get less information if an exception were thrown. If you use only one catch block and an exception is thrown, you will not know if the problem occurred when opening the file or when reading from the file after it was opened.

**Self-Test Exercises**

12. Write some code that will create a stream named `fileIn` that is a member of the class `BufferedReader` and that connects the stream to a text file named `joe` so that your program can read input from the text file `joe`.
13. What is the type of a value returned by the method `readLine` in the class `BufferedReader`? What is the type of a value returned by the method `read` in the class `BufferedReader`?
14. Might the methods `read` and `readLine` in the class `BufferedReader` throw an exception? If so, what type of exception?

### Self-Test Exercises (continued)

15. One difference between the `try` blocks in Display 10.1 and Display 10.7 is that the `try` block in Display 10.1 encloses only the opening of the file, while the `try` block in Display 10.7 encloses most of the action in the program. Why is the `try` block in Display 10.7 larger than the one in Display 10.1?
16. Might the following throw an exception that needs to be caught or declared in a `throws` clause?

```
BufferedReader inputStream =
 new BufferedReader(new FileReader("morestuff2.txt"));
```

(The stream `inputStream` would be used to read from the text file `morestuff2.txt`.)



### TIP: Reading Numbers with `BufferedReader`

Unlike the `Scanner` class, the class `BufferedReader` has no methods to read a number from a text file. You must write your code to read the number as a string and convert the string to a value of a numeric type, such as `int` or `double`. To read a single number on a line by itself, read it using the method `readLine`, and then use `Integer.parseInt`, `Double.parseDouble`, or some similar method to convert the string read to a number. If there are multiple numbers on a single line, read the line using `readLine` and then use the `StringTokenizer` class to decompose the string into tokens. Next, use `Integer.parseInt` or a similar method to convert each token to a number.

`Integer.parseInt`, `Double.parseDouble`, and similar methods that convert strings to numbers are explained in Chapter 5 in the subsection entitled “Wrapper Classes.” The `StringTokenizer` class is discussed in Chapter 4 in the starred subsection entitled “The `StringTokenizer` Class”. ■

### Testing for the End of a Text File with `BufferedReader`

When using the class `BufferedReader`, if your program tries to read beyond the end of the file with either of the methods `readLine` or `read`, then the method returns a special value to signal that the end of the file has been reached. When `readLine` tries to read beyond the end of a file, it returns the value `null`. Thus, your program can test for the end of the file by testing to see if `readLine` returns `null`. This technique is illustrated in Display 10.9. When the method `read` tries to read beyond the end of a file, it returns the value `-1`. Because the `int` value corresponding to each ordinary character is positive, this can be used to test for the end of a file.

### Checking for the End of a Text File with `BufferedReader`

The method `readLine` of the class `BufferedReader` returns `null` when it tries to read beyond the end of a text file. The method `read` of the class `BufferedReader` returns `-1` when it tries to read beyond the end of a text file.

Display 10.9 Checking for the End of a Text File with `BufferedReader` (part 1 of 2)

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.PrintWriter;
4 import java.io.FileOutputStream;
5 import java.io.FileNotFoundException;
6 import java.io.IOException;

7 /**
8 * Makes numbered.txt the same as original.txt, but with each line numbered.
9 */
10 public class TextEOFDemo
11 {
12 public static void main(String[] args)
13 {
14 try
15 {
16 BufferedReader inputStream =
17 new BufferedReader(new FileReader("original.txt"));
18 PrintWriter outputStream =
19 new PrintWriter(new FileOutputStream("numbered.txt"));

20 int count = 0;
21 String line = inputStream.readLine();
22 while (line != null)
23 {
24 count++;
25 outputStream.println(count + " " + line);
26 line = inputStream.readLine();
27 }
28 inputStream.close();
29 outputStream.close();
30 }
31 catch (FileNotFoundException e)
32 {
33 System.out.println("Problem opening files.");
34 }
35 catch (IOException e)
36 {
37 System.out.println("Error reading from original.txt.");
38 }
39 }
40 }
```

---

**Display 10.9** Checking for the End of a Text File with `BufferedReader` (part 2 of 2)

---

FILE `original.txt`

```
Little Miss Muffet
sat on a tuffet
eating her curves away.
Along came a spider
who sat down beside her
and said "Will you marry me?"
```

FILE `numbered.txt` (after the program is run)

```
1 Little Miss Muffet
2 sat on a tuffet
3 eating her curves away.
4 Along came a spider
5 who sat down beside her
6 and said "Will you marry me?"
```

*If your version of `numbered.txt` has numbered blank lines after line 6, that means you had blank lines at the end of `original.txt`.*

---

## Self-Test Exercises

17. Does the class `BufferedReader` have a method to read an `int` value from a text file?
18. What happens when the method `readLine` in the class `BufferedReader` attempts to read beyond the end of a file? How can you use this to test for the end of a file?
19. What is the type of the value returned by the method `read` in the class `BufferedReader`?
20. What happens when the method `read` in the class `BufferedReader` attempts to read beyond the end of a file? How can you use this to test for the end of a file?
21. Does the program in Display 10.9 work correctly if `original.txt` is an empty file?

## Path Names

When giving a file name as an argument to a constructor for opening a file in any of the ways we have discussed, you may use a simple file name, in which case it is assumed that the file is in the same directory (folder) as the one in which the program is run. You can also use a full or relative path name.

### path names

A **path name** not only gives the name of the file, but also tells what directory (folder) the file is in. A **full path name**, as the name suggests, gives a complete path name, starting from the root directory. A **relative path name** gives the path to the



file, starting with the directory that your program is in. The way that you specify path names depends on your particular operating system.

A typical UNIX path name is

```
/user/sallyz/data/data.txt
```

To create a `BufferedReader` input stream connected to this file, use

```
BufferedReader inputStream =
 new BufferedReader(
 new FileReader("/user/sallyz/data/data.txt"));
```

Windows uses `\` instead of `/` in path names. A typical Windows path name is

```
C:\dataFiles\goodData\data.txt
```

To create a `BufferedReader` input stream connected to this file, use

```
BufferedReader inputStream =
 new BufferedReader(
 new FileReader("C:\\dataFiles\\goodData\\data.txt"));
```

using `\`,  
`\\`, or `/`

Note that you need to use `\\` in place of `\`, since otherwise Java will interpret a backslash paired with a character, such as `\d`, as an escape character. Although you must worry about using a backslash (`\`) in a quoted string, this problem does not occur with path names read in from the keyboard.

One way to avoid these escape character problems altogether is to always use UNIX conventions when writing path names. A Java program will accept a path name written in either Windows or UNIX format, even if it is run on a computer with an operating system that does not match the syntax. Thus, an alternate way to create a `BufferedReader` input stream connected to the Windows file

```
C:\dataFiles\goodData\data.txt
```

is the following:

```
BufferedReader inputStream =
 new BufferedReader(
 new FileReader("C:/dataFiles/goodData/data.txt"));
```

## Nested Constructor Invocations

Expressions with two constructors, such as the following, are common when dealing with Java's library of I/O classes:

```
new BufferedReader(new FileReader("original.txt"))
```

This is a manifestation of the general approach to how Java I/O libraries work. Each I/O class serves one or a small number of functions. To obtain full functionality, you normally need to combine two (or more) class constructors. For example, in the previous code, the object `new FileReader("original.txt")` establishes a connection with the file `original.txt` but provides only very primitive methods for input. The constructor for `BufferedReader` takes this file reader object and adds a richer collection of input methods. In these cases, the inner object, such as `new FileReader("original.txt")`, is transformed into an instance variable of the outer object, such as `BufferedReader`.

## Self-Test Exercises

22. Of the classes `PrintWriter`, `Scanner`, `BufferedReader`, `FileReader`, and `FileOutputStream`, which have a constructor that accepts a file name as an argument so that the stream created by the constructor invocation is connected to the named file?
23. Is the following legal?

```
FileReader readerObject =
 new FileReader("myFile.txt");
BufferedReader inputStream =
 new BufferedReader(readerObject);
```

## System.in, System.out, and System.err

The streams `System.in`, `System.out`, and `System.err` are three streams that are automatically available to your Java code. You have already been using `System.in` and `System.out`. `System.err` is just like `System.out`, except that it has a different name. For example, both of the following statements will send the string "Hello" to the screen so the screen receives two lines, each containing "Hello":

```
System.out.println("Hello");
System.err.println("Hello");
```

The output stream `System.out` is intended to be used for normal output from code that is not in trouble. `System.err` is meant to be used for error messages.

redirecting  
output

Having two different standard output streams can be handy when you redirect output. For example, you can redirect the regular output to one file and redirect the error messages to a different file. Java allows you to redirect any of these three standard streams to or from a file (or other I/O device). This is done with the static methods `setIn`, `setOut`, and `setErr` of the class `System`.

For example, suppose your code connects the output stream `errStream` (of a type to be specified later) to a text file. You can then redirect the stream `System.err` to this text file as follows:

```
System.setErr(errStream);
```

If the following appears later in your code,

```
System.out.println("Hello from System.out.");
System.err.println("Hello from System.err.");
```

then "Hello from System.out." will be written to the screen, but "Hello from System.err." will be written to the file connected to the output stream `errStream`. A simple program illustrating this is given in Display 10.10.

Note that the arguments to the redirecting methods must be of the types shown in the following headings, and that these are classes we do not discuss in this book:

```
public static void setIn(InputStream inStream)
public static void setOut(PrintStream outStream)
public static void setErr(PrintStream outStream)
```

None of the input or output streams we constructed in our previous programs are of a type suitable to be an argument to any of these redirection methods. Space constraints keep us from giving any more details on the stream classes that are suitable for producing arguments for these redirection methods. However, you can use Display 10.10 as a model to allow you to redirect either `System.err` or `System.out` to a text file of your choice.

### Self-Test Exercises

24. Suppose you want the program in Display 10.10 to send an error message to the screen and regular (`System.out`) output to the file `errormessages.txt`. (This is the reverse of what the program in Display 10.10 does.) How would you change the program in Display 10.10?
25. Suppose you want the program in Display 10.10 to send all output (both `System.out` and `System.err`) to the file `errormessages.txt`. How would you change the program in Display 10.10?

#### Display 10.10 Redirecting Error Messages (part 1 of 2)

```
1 import java.io.PrintStream;
2 import java.io.FileOutputStream;
3 import java.io.FileNotFoundException;

4 public class RedirectionDemo
5 {
6 public static void main(String[] args)
7 {
8 PrintStream errStream = null;
9 try
10 {
11 errStream =
12 new PrintStream(
13 new FileOutputStream("errormessages.txt"));
14 }
15 catch (FileNotFoundException e)
16 {
17 System.out.println(
18 "Error opening file with FileOutputStream.");
19 System.exit(0);
20 }
21 }
```

*Note the stream classes used.*

## Display 10.10 Redirecting Error Messages (part 2 of 2)

```

21 System.setErr(errStream);
22 System.err.println("Hello from System.err.");
23 System.out.println("Hello from System.out.");
24 System.err.println("Hello again from System.err.");
25 errStream.close(); ← None of System.in, System.out, or
26 } System.err needs to be closed, but the
27 } streams you create should be explicitly
 closed.

```

FILE errormessages.txt

```

Hello from System.err.
Hello again from System.err.

```

## Screen Output

```
Hello from System.out.
```

## 10.3 The File Class

*The scars of others should teach us caution.*

SAINT JEROME, c. 347 – 30.

In this section, we describe the class `File`, which is not really an I/O stream class but is often used in conjunction with file I/O. The class `File` is so important to file I/O programming that it was even placed in the `java.io` package.

### Programming with the File Class

The `File` class contains methods that allow you to check various properties of a file, such as whether there is a file with a specified name, whether the file can be written to, and so forth. Display 10.11 gives a sample program that uses the class `File` with text files. (The class `File` works the same with binary files as it does with text files.)

#### abstract name

Note that the `File` class constructor takes a name, known as the **abstract name**, as an (string) argument. So the `File` class really checks properties of names. For example, the method `exists` tests whether there is a file with the abstract name. Moreover, the abstract name may be a potential directory (folder) name. For example, the method `isDirectory` tests whether the abstract name is the name of a directory (folder). The abstract name may be either a relative path name (which includes the case of a simple file name) or a full path name.

Display 10.12 lists some of the methods in the class `File`.

### The File Class

The `File` class is like a wrapper class for file names. The constructor for the class `File` takes a string as an argument and produces an object that can be thought of as the file with that name. You can use the `File` object and methods of the class `File` to answer questions, such as the following: Does the file exist? Does your program have permission to read the file? Does your program have permission to write to the file? Display 10.12 has a summary of some of the methods for the class `File`.

#### EXAMPLE

```
File fileObject = new File("data.txt");
if (! fileObject.canRead())
 System.out.println("File data.txt is not readable.");
```

Display 10.11 Using the `File` Class (part 1 of 2)

```
1 import java.util.Scanner;
2 import java.io.File;
3 import java.io.PrintWriter;
4 import java.io.FileOutputStream;
5 import java.io.FileNotFoundException;
6 public class FileClassDemo
7 {
8 public static void main(String[] args)
9 {
10 Scanner keyboard = new Scanner(System.in);
11 String line = null;
12 String fileName = null;
13
14 System.out.println("I will store a line of text for you.");
15 System.out.println("Enter the line of text:");
16 line = keyboard.nextLine();
17 System.out.println("Enter a file name to hold the line:");
18 fileName = keyboard.nextLine();
19 File fileObject = new File(fileName);
20
21 while (fileObject.exists())
22 {
23 System.out.println("There already is a file named "
24 + fileName);
25 System.out.println("Enter a different file name:");
26 fileName = keyboard.nextLine();
27 fileObject = new File(fileName);
28 }
29 }
```

Display 10.11 Using the File Class (part 2 of 2)

```

28 PrintWriter outputStream = null;
29 try
30 {
31 outputStream =
32 new PrintWriter(new FileOutputStream(fileName));
33 }
34 catch(FileNotFoundException e)
35 {
36 System.out.println("Error opening the file " + fileName);
37 System.exit(0);
38 }

39 System.out.println("Writing \"" + line + "\"");
40 System.out.println("to the file" + fileName);
41 outputStream.println(line);

42 outputStream.close();
43 System.out.println("Writing completed.");
44 }
45 }

```

*If you wish, you can use `fileObject` instead of `fileName` as the argument to `FileOutputStream`.*



*The dialogue assumes that there already is a file named `myLine.txt` but that there is no file named `mySaying.txt`.*

## Sample Dialogue

```

I will store a line of text for you.
Enter the line of text:
May the hair on your toes grow long and curly.
Enter a file name to hold the line:
myLine.txt
There already is a file named myLine.txt
Enter a different file name:
mySaying.txt
Writing "May the hair on your toes grow long and curly."
to the file mySaying.txt
Writing completed.

```

Display 10.12 Some Methods in the Class File (part 1 of 3)

File is in the `java.io` package.

```
public File(String File_Name)
```

A constructor. *File\_Name* can be either a full or a relative path name (which includes the case of a simple file name). *File\_Name* is referred to as the **abstract path name**.

```
public boolean exists()
```

Tests whether there is a file with the abstract path name.

(continued)